

STS8300

编程手册

(Rev 2.1.15)

注：本手册为技术保密资料，仅供客户了解使用设备，不得传阅！

目录

目录.....	1
1 系统函数.....	17
1.1 测试程序框架函数.....	17
1.1.1 HardWareCfg()	18
1.1.2 InitBeforeTestFlow()	18
1.1.3 InitAfterTestFlow()	18
1.1.4 UserInit()	18
1.1.5 UserLoad()	18
1.1.6 UserExit()	19
1.1.7 OnSot()	19
1.1.8 SetupFailSite()	19
1.1.9 BinOutDut()	19
1.1.10 OnNewLot()	20
1.1.11 OnWaferEnd()	20
1.2 全局函数.....	20
1.2.1 BEGIN_SINGLE_SITE()	20
1.2.2 END_SINGLE_SITE()	21
1.2.3 delay_ms()	21
1.2.4 delay_us()	22
1.2.5 STSEnableCfgCheck()	22
1.2.6 STSSetMultiSiteBind()	23
1.2.7 STSCBITeMultiSiteBind()	27
1.2.8 STSSetMultiSiteBindEx()	29
1.2.9 STSCBITeMultiSiteBindEx()	32
1.2.10 STSSetSiteStatus()	34
1.2.11 STSGetSiteStatus()	35
1.2.12 STSGetSiteRunStatus()	36
1.2.13 STSSetSiteStatusEx()	37
1.2.14 STSGetSiteStatusEx()	38
1.2.15 STSGetSingleSiteStatus()	39
1.2.16 STSGetOperatorID()	39
1.2.17 STSGetWaferID()	40
1.2.18 STSGetLotID()	41
1.2.19 STSGetCurrentDutSwBin()	42
1.2.20 STSGetCurrentDutHwBin()	43
1.2.21 StsGetParam()	43
1.2.22 StsGetBinDutTotalCount()	44
1.2.23 StsAddUserSummaryStr()	45
1.2.24 StsMessageBox()	45

1.2.25	StsGetSiteCount()	46
1.2.26	STSGetPgsName()	46
1.2.27	STSGetPgsPath()	47
1.2.28	STSSetTimeCheck()	47
1.2.29	STSGetTimeElapsed()	48
1.2.30	STSSetSaveInterval()	49
1.2.31	STSStopProgram()	49
1.2.32	STSStartNewLot()	49
1.2.33	STSGetUserPriority()	50
1.2.34	STSSetEnableCalCheck()	50
1.2.35	STSGetPartIDText()	51
1.2.36	STSSetEnableAlarm()	52
1.2.37	STSSetWriteAlarmFile()	52
1.2.38	STSTArming()	53
1.3	测试参数相关函数	53
1.3.1	SetTestResult()	53
1.3.2	GetTestResult()	54
1.3.3	SetResultRemark()	55
1.3.4	GetResultRemark()	56
1.3.5	GetMinLimit()	57
1.3.6	GetMaxLimit()	58
1.3.7	GetSubUnitsCount()	58
1.3.8	GetDispFormat()	59
1.3.9	GetDescription()	59
1.3.10	GetConditionsCnt()	60
1.3.11	GetConditionDispName()	60
1.3.12	GetConditionSymbol()	61
1.3.13	GetConditionShowKind()	63
1.3.14	GetConditionInputKind()	64
1.3.15	GetConditionDispValue()	64
1.3.16	GetConditionDispUnit()	66
1.3.17	GetTestConditionValue()	67
1.3.18	GetConditionCurSelDouble()	68
1.3.19	GetConditionValueSelOrder()	69
1.3.20	GetConditionSelectUnit()	70
1.3.21	GetConditionUnitSelOrder()	71
1.3.22	GetParamStationNo()	72
1.3.23	GetConditionDispTip()	72
1.4	软件示波器显示用户自定义波形函数	73
1.4.1	STSSetSoftViewerData()	73
1.4.2	STSSetSoftViewerData2()	75
1.4.3	STSClearSoftViewerData()	76

1.4.4	STSClearAllSoftViewerData ()	77
1.5	特殊应用函数	77
1.5.1	StsGetSingleDieCorXY()	77
1.5.2	StsSetDieCorXY()	78
1.5.3	STSGetDieCorXY()	79
1.5.4	GetNewLotInfoCnt()	80
1.5.5	GetNewLotItemInfo()	80
1.5.6	STSSetInitSelectDialog()	81
1.5.7	StsSetSaveFileNameStr()	82
1.5.8	STSEnableDebugTool()	83
1.5.9	STSSetAlwaysRunFunctionItem()	83
1.5.10	STSSetAlwaysRunFunction()	84
1.5.11	STSGetAlwaysRun()	85
1.5.12	STSSetFailStop()	89
1.5.13	STSGetExecutedItemsCnt()	89
1.5.14	STSSetPartID()	90
1.5.15	STSGetTestItemsCnt()	90
1.5.16	STSGetQATestItemsCnt()	91
1.5.17	STSGetSiteTestStatus()	91
1.5.18	STSTNewLot()	92
1.5.19	STSGetDataLogFileName()	93
1.5.20	STSGetLotInfo()	93
1.5.21	STSGetTesterID()	94
1.5.22	StsGetParamBySymbol()	95
1.5.23	STSGetParamCount()	95
1.5.24	STSSetTestTimeFunction()	96
1.5.25	STSGetFunTestTimebyIndex()	97
1.5.26	STSSetAllParamSave()	98
1.5.27	STSSetAllParamShow()	98
1.5.28	STSSetQASampleMode()	98
1.5.29	STSTUpdateParamLimit()	99
1.5.30	STSGetSystemVersion()	100
1.5.31	STSGetSystemName()	102
1.5.32	STSGetTHBInfo()	103
1.5.33	STSGetQAStartFunctionIndex()	104
1.5.34	STSGetISQAFile()	105
1.5.35	STSGetEnableQA()	105
1.5.36	STSGetQASettings()	106
1.5.37	STSGetIsQAFunction()	106
1.5.38	STSSetRelayActTimes()	107
1.6	STDF 存储信息设置函数	109
1.6.1	STDF 数据类型	109

1.6.2	STSCBSetSTDInfo()	111
1.6.3	STSCBSetAndUpdateSTDInfo()	112
1.6.4	STSCBSetGDRInfo()	113
1.6.5	STSCBGetSTDInfo()	114
1.7	良率卡控功能 (SYL) 函数	115
1.7.1	STSCBSetSYLDiffData()	115
1.7.2	STSCBGetSYLDiffData()	115
1.7.3	STSCBSetContinueFailStopSetting()	116
1.7.4	STSCBGetContinueFailStopSetting()	117
1.7.5	STSCBSetContinuePassStopSetting()	118
1.7.6	STSCBGetContinuePassStopSetting()	118
1.7.7	STSCBGetRecentSiteSYL()	119
1.7.8	STSCBGetRecentSYLInfo()	120
1.7.9	STSCBSetRecentSYLInfo()	120
1.7.10	STSCBGetSiteDUTCount()	121
1.8	SBL 分档卡控功能应用函数	122
1.8.1	STSCBGetSBLSetting()	122
1.8.2	STSCBSetSBLSetting()	123
1.8.3	STSCBSBLEnable()	123
1.8.4	STSCBSBLSampleCount()	124
1.8.5	STSCBSBLClearCountFlag()	125
1.8.6	STSCBSetSBLData()	125
1.8.7	STSCBGetSBLCount()	126
1.8.8	STSCBGetSBLInfo()	126
1.8.9	STSCBGetHBLCount()	127
1.8.10	STSCBGetHBLInfo()	128
1.8.11	STSCBSetSBLDiffData()	129
1.8.12	STSCBGetSBLDiffCount()	129
1.8.13	STSCBGetSBLDiffinfo()	130
1.8.14	STSCBGetHBLDiffCount()	131
1.8.15	STSCBGetHBLDiffinfo()	131
1.8.16	STSCBGetSBLYieldInfo()	132
1.8.17	STSCBGetHBLYieldInfo()	133
1.8.18	STSCBClearSBLContent()	134
1.9	参数卡控功能 (SPL) 编程接口函数	135
1.9.1	STSCBSPLEnable()	135
1.9.2	STSCBSetSPLClearSetting()	135
1.9.3	STSCBSetSPLSetting()	136
1.9.4	STSCBGetSPLSetting()	137
1.9.5	STSCBSetSPLApplySampleMode()	137
1.9.6	STSCBSetSPLYieldSampleCount()	138
1.9.7	STSCBSetSPLYieldData()	139

1.9.8	STSCBGetSPLYieldInfo()	139
1.9.9	STSCBSetSPLYieldDiffSampleCount()	140
1.9.10	STSCBSetSPLYieldDiffData()	141
1.9.11	STSCBGetSPLYieldDiffInfo()	142
1.9.12	STSCBSetSPLAverageDiffSampleCount()	142
1.9.13	STSCBSetSPLAverageDiffData()	143
1.9.14	STSCBGetSPLAverageDiffInfo()	144
1.9.15	STSCBSetSPLAverageRangeSampleCount()	145
1.9.16	STSCBSetSPLAverageRangeData()	145
1.9.17	STSCBGetSPLAverageRangeInfo()	146
1.9.18	STSCBSetSPLFluctuationRangeSampleCount()	148
1.9.19	STSCBSetSPLFluctuationRangeData()	148
1.9.20	STSCBGetSPLFluctuationRangeInfo()	149
1.9.21	STSCBSetSPLStackingDetectionSampleCount()	150
1.9.22	STSCBSetSPLStackingDetectionData()	151
1.9.23	STSCBGetSPLStackingDetectionInfo()	152
1.9.24	STSCBGetSPLStaticsInfo()	152
1.9.25	STSCBGetSPLPassStaticsInfo()	153
1.9.26	STSCBClearSPLContent()	155
1.9.27	STSCBClearSPLSet()	156
1.10	重测功能函数	156
1.10.1	STSCBSetFailRetestMode()	157
1.10.2	STSCBGetFailRetestMode()	157
1.10.3	STSCBAddRetestSBinID()	158
1.10.4	STSCBAddRetestHBinID()	158
1.10.5	STSCBGetRetestTimes()	159
1.10.6	STSCBGetRetestStatus()	159
1.10.7	STSCBGetPartID()	160
1.11	TestUI 测试界面信息获取函数	161
1.11.1	STSCBGetTesterID()	161
1.11.2	STSCBGetDataLogFileName()	161
1.11.3	STSCBGetSaveFile()	162
1.11.4	STSCBGetSaveFilePath()	163
1.11.5	STSCBGetSaveFileTemporaryPath()	164
1.11.6	STSCBGetSummaryFileName()	165
1.11.7	STSCBGetSummaryFilePath()	165
1.11.8	STSCBGetConvertSTD()	166
1.11.9	STSCBGetConvertExcel()	167
1.11.10	CBIT128STSCBGetConvertCSV()	167
1.11.11	STSCBGetConvertRowModeTXT()	168
1.11.12	STSCBGetConvertTXT()	168
1.11.13	STSCBGetConvertUserDefine()	169

1. 11. 14	STSCBGetHandlerProberName ()	169
1. 11. 15	STSCBSetRelayLifeCheck ()	170
1. 11. 16	STSCBGetRelayLifeCheck ()	171
1. 12	PAT 功能应用函数	171
1. 12. 1	系统 PAT 函数	171
1. 12. 2	自定义 PAT 函数	174
1. 13	进度条功能应用函数	179
1. 13. 1	STSCBSetProgressInfo()	179
1. 13. 2	STSCBSetProgressStep()	180
1. 14	自定义输入界面应用函数	181
1. 14. 1	STSPopupStringDialog ()	181
1. 14. 2	自定义界面编辑功能	182
1. 15	TestNum 功能	196
1. 15. 1	STSGetTestNum ()	196
1. 15. 2	CSTSTestNumber 相关函数	197
1. 16	进阶功能应用函数	205
1. 16. 1	STSTUpdateFunTest ()	205
1. 16. 2	STSShowLimitDlg ()	206
1. 16. 3	STSSetUseLimitbyID ()	207
1. 16. 4	STSSetUseLimitbyName ()	208
1. 16. 5	STSGetLimitCount ()	208
1. 16. 6	STSGetLimitName ()	209
1. 16. 7	STSGetUseLimitID ()	209
1. 16. 8	STSGetUseLimitName ()	210
1. 16. 9	STSGetMustResult ()	210
1. 16. 10	STSGetFunction ()	211
1. 16. 11	STSShowBinDlg()	211
1. 16. 12	STSSetUseBinbyID ()	212
1. 16. 13	STSSetUseBinbyName ()	213
1. 16. 14	STSGetBinCount ()	214
1. 16. 15	STSGetBinName ()	214
1. 16. 16	STSGetUseBinID ()	214
1. 16. 17	STSGetUseBinName ()	215
1. 16. 18	STSSetHardwareCheck ()	215
1. 16. 19	STSSetEnableLog ()	216
1. 16. 20	CFunctionTest 相关函数	217
1. 16. 21	CSTSMustResult 相关函数	223
1. 17	应用举例	229
1. 17. 1	程序中定义 QA 起始函数	229
1. 17. 2	程序中设置 DIE 坐标	231
1. 17. 3	程序中获取 Lot 信息	234
1. 17. 4	程序中控制 FailStop	235

1. 17. 5	程序中控制 Single Test Data Not Save	236
1. 17. 6	程序中设置对话框	237
1. 17. 7	程序中设置储存文件名	238
2	CBITE	239
2. 1	CBITe()	239
2. 2	Init()	240
2. 3	SetOn()	240
2. 4	SetCbiteOn()	242
2. 5	SetCbiteOff()	243
2. 6	GetCbiteStatus()	244
2. 7	SetDutPower()	246
2. 8	GetDutPowerStatus()	247
2. 9	获取板卡固有信息函数	249
2. 9. 1	GetBoardSN()	249
2. 9. 2	GetBoardHDRev()	250
3	QTMUE	252
3. 1	QTMUe()	252
3. 2	Connect()	253
3. 3	Disconnect()	254
3. 4	SetInSource()	255
3. 5	SetMatrix()	257
3. 6	Start()	261
3. 7	Stop()	264
3. 8	SetArm()	267
3. 9	StopArm()	269
3. 10	Measure()	270
3. 11	GetMeasureResult()	272
3. 12	SetAlarmMask()	274
3. 13	GetAlarmMask()	275
3. 14	SiteBindModify()	275
3. 15	获取板卡固有信息函数	278
3. 15. 1	GetBoardSN()	278
3. 15. 2	GetBoardHDRev()	279
3. 16	获取校准信息函数	280
3. 16. 1	GetCalibrationTime()	280
3. 16. 2	GetCalibrationTemperature()	281
3. 16. 3	GetCalibrationHumidity()	281
3. 16. 4	GetCalibrationResult()	282
3. 16. 5	GetCalibrationLogicRev()	283
3. 16. 6	GetCalibrationMeter()	284
3. 16. 7	GetCalibrationCalBoardInfo()	285

3.16.8	GetCalibrationSoftRev()	286
3.16.9	GetCalibrationSlotID()	287
3.17	编程范例	288
3.17.1	测试时间	288
3.17.2	测试频率	290
3.17.3	占空比	291
3.17.4	事件计数	291
3.17.5	外部触发测试时间	292
3.17.6	复杂矩阵应用实例	293
4	DCM	298
4.1	DCM 相关函数	298
4.1.1	DCM	300
4.1.2	LoadVectorFile()	301
4.1.3	SetValidPin()	303
4.1.4	SetPinGroup()	304
4.1.5	InitMCU()	305
4.1.6	SetPeriod()	306
4.1.7	SetEdge()	307
4.1.8	Connect()	309
4.1.9	Disconnect()	310
4.1.10	SetPinLevel()	311
4.1.11	SetChannelStatus()	312
4.1.12	SetWaveDataParam()	313
4.1.13	SetSiteWaveData()	315
4.1.14	WriteWaveData(void)	317
4.1.15	WriteWaveData()	318
4.1.16	SetVT()	320
4.1.17	SetDynamicLoad()	321
4.1.18	SetPrereadVector()	323
4.1.19	SetFailSavingType()	324
4.1.20	RunVectorWithGroup()	326
4.1.21	RunVectorEn()	327
4.1.22	StopVector()	329
4.1.23	SaveFailMap()	330
4.1.24	GetHardwareCaptureData()	331
4.1.25	SetCapture()	333
4.1.26	GetCaptureData()	335
4.1.27	GetCaptureData()	337
4.1.28	GetMCUResult()	340
4.1.29	GetMCUPinResult()	341
4.1.30	GetMCUPinRunStatus()	342

4.1.31	GetFailCount()	343
4.1.32	GetFirstFailLineNo()	345
4.1.33	GetFailLineNo()	346
4.1.34	GetStopLineNo()	348
4.1.35	GetStopLabel()	349
4.1.36	GetRunLineCount()	350
4.1.37	SetPPMU()	351
4.1.38	PPMUMeasure()	353
4.1.39	GetPPMUMeasResult()	355
4.1.40	SetTMUMatrix()	356
4.1.41	SetTMUParam()	357
4.1.42	TMUMeasure()	359
4.1.43	GetTMUMeasureResult()	361
4.1.44	SetAlarmMask()	363
4.1.45	GetAlarmMask()	364
4.1.46	获取板卡固有信息函数	365
4.1.47	取校准信息函数	367
4.1.48	编程范例	377
4.2	I2C 相关函数	384
4.2.1	I2CSet()	384
4.2.2	I2CSetStopStatus()	386
4.2.3	I2CSetSCLEdge()	386
4.2.4	I2CSetSDAEdge()	387
4.2.5	I2CConnect()	389
4.2.6	I2CDisconnect()	390
4.2.7	I2CSetPinLevel()	390
4.2.8	I2CSetPinLevel()	392
4.2.9	I2CSetDynamicLoad()	393
4.2.10	I2CWriteData()	395
4.2.11	I2CWriteData()	396
4.2.12	I2CWriteMultiData()	398
4.2.13	I2CReadData()	399
4.2.14	I2CGetReadData()	400
4.2.15	I2CGetBitData()	401
4.2.16	I2CGetNACKIndex()	402
4.2.17	I2C 编程范例	403
5	HSD	405
5.1	HSD()	405
5.2	Init()	406
5.3	SetChannel()	407
5.4	SetEdgeTrig()	409

5.5	SetNEdgeTrig()	410
5.6	SetPulseTrig()	412
5.7	SetSlopeTrig()	414
5.8	SetTrigPropety()	417
5.9	SetInterLeave()	418
5.10	StartCapture()	419
5.11	WaitFinish()	421
5.12	BlockRead()	422
5.13	GetMeasure()	423
5.14	编程范例	425
6	VI 源表	428
6.1	AWG	428
6.1.1	STSAWGCreateSineData()	428
6.1.2	STSAWGCreateTriangleData()	430
6.1.3	STSAWGCreateSquareData()	432
6.1.4	STSAWGCreateRampData()	434
6.1.5	STSEnableAWG()	435
6.1.6	STSEnableMeas()	437
6.1.7	STSAWGRun()	439
6.1.8	STSPTAWGAndDCMRun()	441
6.1.9	STSAWGRunTriggerStop()	442
6.2	ACM	447
6.2.1	ACM()	447
6.2.2	Set()	448
6.2.3	SetSyn()	450
6.2.4	SetClamp()	453
6.2.5	MeasureVI()	455
6.2.6	GetMeasResult()	457
6.2.7	BlockRead()	459
6.2.8	Pulse()	460
6.2.9	AwgLoader()	463
6.2.10	AwgSelect()	465
6.2.11	AwgRun()	467
6.2.12	AwgStop()	470
6.2.13	AwgClear()	471
6.2.14	SetMeasVTrig()	471
6.2.15	SetMeasITrig()	474
6.2.16	SetAlarmMask()	476
6.2.17	GetAlarmMask()	477
6.2.18	获取板卡固有信息函数	478
6.2.19	获取校准信息函数	480

6.2.20	编程范例	488
6.3	ACM200	493
6.3.1	ACM200()	494
6.3.2	Set()	495
6.3.3	SetSyn()	497
6.3.4	SetClamp()	500
6.3.5	MeasureVI()	502
6.3.6	GetMeasResult()	504
6.3.7	BlockRead()	507
6.3.8	Pulse()	508
6.3.9	AwgLoader()	511
6.3.10	AwgSelect()	513
6.3.11	AwgRun()	516
6.3.12	AwgStop()	518
6.3.13	AwgClear()	520
6.3.14	SetMeasVTrig()	520
6.3.15	SetMeasITrig()	524
6.3.16	ContactCheck()	528
6.3.17	GetContactCheckResult()	529
6.3.18	SetFLOff()	531
6.3.19	GangRelay()	531
6.3.20	GangSet()	534
6.3.21	GangSetSyn()	535
6.3.22	GangMeasure()	537
6.3.23	GangGetMeasResult()	538
6.3.24	GangBlockRead()	540
6.3.25	GangReset()	542
6.3.26	TSet()	543
6.3.27	SetAlarmMask()	546
6.3.28	GetAlarmMask()	546
6.3.29	GetBoardSN()	547
6.3.30	获取校准信息函数	548
6.3.31	VI 源编程范例	557
6.4	FPVie	565
6.4.1	FPVie()	565
6.4.2	Set()	566
6.4.3	SetSyn()	568
6.4.4	SetClamp()	572
6.4.5	MeasureVI()	573
6.4.6	GetMeasResult()	576
6.4.7	BlockRead()	578
6.4.8	Pulse()	579

6.4.9	AwgLoader()	581
6.4.10	AwgSelect()	584
6.4.11	AwgRun()	586
6.4.12	AwgStop()	588
6.4.13	AwgClear()	590
6.4.14	SetMeasVTrig()	590
6.4.15	SetMeasITrig()	592
6.4.16	ContactCheck()	595
6.4.17	GetContactCheckResult()	596
6.4.18	SetAlarmMask()	597
6.4.19	GetAlarmMask()	598
6.4.20	SiteBindModify()	598
6.4.21	获取板卡固有信息函数	601
6.4.22	获取校准信息函数	603
6.4.23	编程范例	611
6.5	FOVie	618
6.5.1	FOVie()	618
6.5.2	Set()	619
6.5.3	SetSyn()	621
6.5.4	SetClamp()	624
6.5.5	MeasureVI()	626
6.5.6	GetMeasResult()	628
6.5.7	BlockRead()	630
6.5.8	Pulse()	632
6.5.9	AwgLoader()	634
6.5.10	AwgSelect()	637
6.5.11	AwgRun()	639
6.5.12	AwgStop()	641
6.5.13	AwgClear()	642
6.5.14	SetMeasVTrig()	642
6.5.15	SetMeasITrig()	645
6.5.16	ContactCheck()	647
6.5.17	GetContactCheckResult()	648
6.5.18	SetAlarmMask()	649
6.5.19	GetAlarmMask()	650
6.5.20	获取板卡固有信息函数	650
6.5.21	获取校准信息函数	652
6.5.22	编程范例	661
6.6	HVie	669
6.6.1	HVie()	669
6.6.2	Set()	670
6.6.3	SetSyn()	672

6.6.4	SetClamp()	676
6.6.5	MeasureVI()	677
6.6.6	GetMeasResult()	679
6.6.7	BlockRead()	681
6.6.8	Pulse()	683
6.6.9	AwgLoader()	685
6.6.10	AwgSelect()	688
6.6.11	AwgRun()	690
6.6.12	AwgStop()	693
6.6.13	AwgClear()	694
6.6.14	SetMeasVTrig()	694
6.6.15	SetMeasITrig()	697
6.6.16	SetAlarmMask()	699
6.6.17	GetAlarmMask()	700
6.6.18	获取板卡固有信息函数	701
6.6.19	获取校准信息函数	703
6.6.20	编程范例	711
6.7	HPVie	714
6.7.1	HPVie()	714
6.7.2	Set()	715
6.7.3	SetSyn()	718
6.7.4	SetClamp()	721
6.7.5	MeasureVI()	723
6.7.6	GetMeasResult()	725
6.7.7	BlockRead()	727
6.7.8	Pulse()	729
6.7.9	AwgLoader()	731
6.7.10	AwgSelect()	734
6.7.11	AwgRun()	736
6.7.12	AwgStop()	738
6.7.13	AwgClear()	738
6.7.14	SetMeasVTrig()	739
6.7.15	SetMeasITrig()	741
6.7.16	KelvinCheck()	744
6.7.17	GetKelvinCheckResult()	745
6.7.18	SetAlarmMask()	747
6.7.19	GetAlarmMask()	747
6.7.20	SiteBindModify()	748
6.7.21	获取板卡固有信息函数	750
6.7.22	获取校准信息函数	752
6.7.23	编程范例	761
6.8	QVMe	768

6.8.1	QVMe()	768
6.8.2	Init()	769
6.8.3	Connect()	769
6.8.4	Disconnect()	770
6.8.5	SetMeasTrig()	770
6.8.6	MeasureLADC()	771
6.8.7	MeasureHADC()	773
6.8.8	GetMeasResult()	775
6.8.9	StartFFT()	776
6.8.10	GetFFTResult()	777
6.8.11	BlockRead()	780
6.8.12	SetAlarmMask()	781
6.8.13	GetAlarmMask()	782
6.8.14	SiteBindModify()	783
6.8.15	获取板卡固有信息函数	785
6.8.16	获取校准信息函数	787
6.8.17	编程范例	795
6.9	FXVie	801
6.9.1	FXVie()	801
6.9.2	Set()	802
6.9.3	SetSyn()	804
6.9.4	SetClamp()	807
6.9.5	MeasureVI()	809
6.9.6	GetMeasResult()	811
6.9.7	BlockRead()	813
6.9.8	Pulse()	814
6.9.9	AwgLoader()	817
6.9.10	AwgSelect()	819
6.9.11	AwgRun()	821
6.9.12	AwgStop()	823
6.9.13	AwgClear()	824
6.9.14	SetMeasVTrig()	825
6.9.15	SetMeasITrig()	827
6.9.16	ContactCheck()	830
6.9.17	GetContactCheckResult()	831
6.9.18	SetFLOff()	832
6.9.19	TMUConnect()	832
6.9.20	TMUDisconnect()	833
6.9.21	TMUSetInSource()	833
6.9.22	TMUSetMatrix()	834
6.9.23	TMUStart()	836
6.9.24	TMUStop()	839

6.9.25	TMUMeasure()	841
6.9.26	TMUGetMeasResult()	843
6.9.27	FXVieDiffMeasure()	845
6.9.28	FXVieDiffGetMeasResult()	846
6.9.29	FXVieDiffBlockRead()	847
6.9.30	GangRelay()	848
6.9.31	GangSet()	851
6.9.32	GangSetSyn()	852
6.9.33	GangMeasure()	854
6.9.34	GangGetMeasResult()	855
6.9.35	GangBlockRead()	856
6.9.36	GangReset()	858
6.9.37	TMUChannel()	858
6.9.38	TMUChannelReset()	860
6.9.39	TSet()	860
6.9.40	SetAlarmMask()	863
6.9.41	GetAlarmMask()	864
6.9.42	GetBoardSN()	865
6.9.43	获取校准信息函数	865
6.9.44	VI 源编程范例	874
6.9.45	TMU 编程范例	881
7	数据分析算法库	886
7.1	STSEInitArray()	886
7.2	查找触发值	886
7.2.1	STSEFindTrigger()	887
7.2.2	Find()	887
7.2.3	UpperTriggerCount()	888
7.2.4	UpperTriggerData()	889
7.2.5	LowerTriggerCount()	890
7.2.6	LowerTriggerData()	890
7.3	数组四则运算	891
7.3.1	STSEArrayAdd()	891
7.3.2	STSEArraySub()	894
7.3.3	STSEArrayMULT()	895
7.3.4	STSEArrayDiv()	896
7.4	STSEAverage()	898
7.5	STSERootMeanSquare()	899
7.6	STSEMovAvg()	900
7.7	STSEVariance()	902
7.8	STSEStandardDeviation()	903
7.9	STSEFindMaxValueAndPos()	904

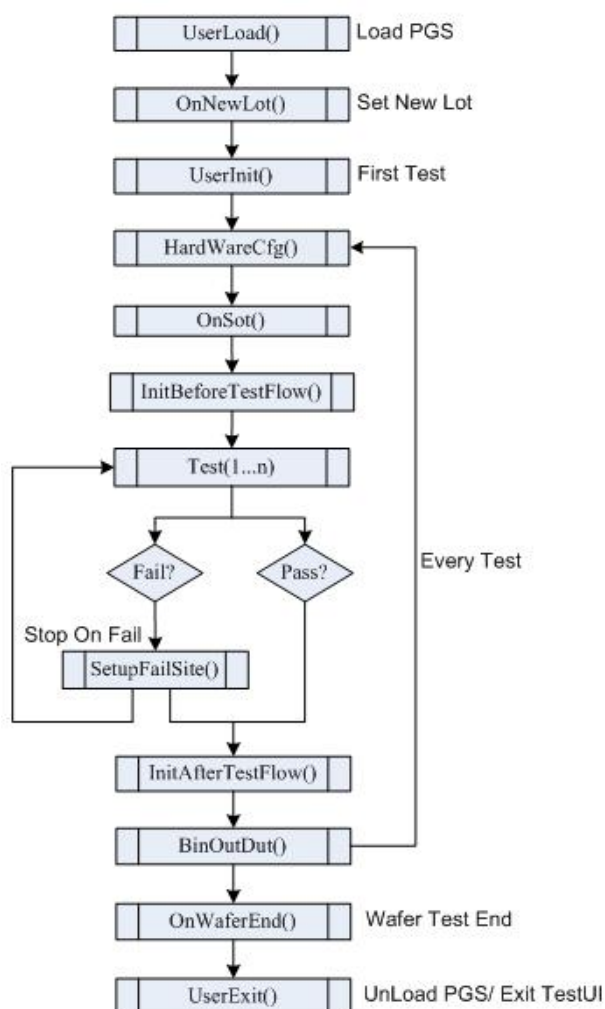
7.10	STSFindMinValueAndPos()	906
7.11	STSGetFFTData()	907
7.12	STSLeastSquareLinearFitting()	908
7.13	STSSort()	909

1 系统函数

1.1 测试程序框架函数

测试程序框架函数共包括 11 个。在任何一个已生成的测试程序中，都自动包含了这些函数。但函数中没有任何内容，由用户根据需要自行添加。

其中，HardWareCfg()、InitBeforeTestFlow()、InitAfterTestFlow()、SetupFailSite()这 4 个函数在 test.cpp 文件中。其余的 7 个函数在 xxxx(测试程序名).cpp 中。系统函数流程图如下图所示：



1.1.1 HardWareCfg()

Remarks

硬件工位配置函数。系统最多可配置成 32 工位。具体硬件配置方式请参考 1.2.6 节。

1.1.2 InitBeforeTestFlow()

Remarks

在所有测试函数之前执行一次此函数，每次测试都会执行。

1.1.3 InitAfterTestFlow()

Remarks

在所有测试函数之后执行一次此函数，每次测试都会执行。

1.1.4 UserInit()

Remarks

Load 程序后第一次测试会调用一次此函数。

1.1.5 UserLoad()

Remarks

在用户 Load 程序时调用一次此函数。

1.1.6 UserExit()

Remarks

用户卸载程序时调用一次此函数。

1.1.7 OnSot()

Remarks

程序在自动运行时，每次测试接收到 sot 后调用此函数。

1.1.8 SetupFailSite()

Remarks

在用户选择了 FailStop 后，每次测试函数中出现 Fail 后调用此函数。

注意：

针对失效工位，可在此函数中设置资源到安全状态。

1.1.9 BinOutDut()

Remarks

被测芯片测试完成确定分档后调用此函数。

用户可以获得此芯片的分档信息；也可以在这个函数中用代码实现器件的分档。

1.1.10 OnNewLot()

Remarks

用户在界面上点击 NewLot 后调用此函数，或者在 wafer 测试时，读取新的 LotID 号后调用此函数。

1.1.11 OnWaferEnd()

Remarks

程序在每次获取 wafer end 信号后调用一次此函数。

1.2 全局函数

该部分函数的主要作用是获取 Site、LOTID、XY 坐标、模块是否存在、Bin 等信息，还可以设置工位状态，实现对某工位的独立控制。

1.2.1 BEGIN_SINGLE_SITE()

```
BEGIN_SINGLE_SITE(SiteID)
```

Parameters

串行操作工位号，建议使用 BYTE 类型。

Remarks

在硬件绑定为多工位时，利用宏 BEGIN_SINGLE_SITE() 和 END_SINGLE_SITE()对某一个工位单独动作，其他工位的硬件不进行动作。

注意：

宏 BEGIN_SINGLE_SITE()和 END_SINGLE_SITE()一定要成对使用，否则会出现不可预料的错误。

Example

```
double result[4] = {1,2,3,4};

fovie0.Set(FV,5,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
//set voltage range isFOVle_10V, current range is FOVle_100MA

for(BYTE i = 0; i < 4; i++)
{
    BEGIN_SINGLE_SITE(i)
        fovie0.Set(FV,result[i],FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
        delay_ms(1);
    END_SINGLE_SITE()
}
```

1.2.2 END_SINGLE_SITE()

与宏 BEGIN_SINGLE_SITE()配合使用，使用详见 1.2.1 说明。

1.2.3 delay_ms()

```
void USERRES_API delay_ms (DWORD dw_ms);
```

Parameters

dw_ms

延时时间，单位毫秒。

Remarks

延时函数。

1.2.4 delay_us()

```
void USERRES_API delay_us (DWORD dw_us);
```

Parameters

dw_us

延时时间，单位微秒。

Remarks

延时函数。

1.2.5 STSEnableCfgCheck()

```
int STSEnableCfgCheck();
```

Remarks

设置对系统做硬件配置检查。检查的硬件模块以工位配置函数 STSSetMultiSiteBind() 的设置为准。

Example

```
STSEnableCfgCheck();  
STSSetMultiSiteBind(MD_FOVle, SITE_1, "S13_0-7");//set Slot13 ch0-7 to SITE1
```

```
STSSetMultiSiteBind(MD_FOVie,SITE_2,"S14_0-7");//set Slot14 ch0-7 to SITE2  
STSSetMultiSiteBind(MD_FOVie,SITE_3,"S19_0-7");//set Slot19 ch0-7 to SITE3  
STSSetMultiSiteBind(MD_FOVie,SITE_4,"S20_0-7");//set Slot20 ch0-7 to SITE4
```

在 `HardWareCfg` 函数中加入 `STSEnableCfgCheck` 函数，则会对 FOVie 做硬件配置检查。该样例中设置了四块 FOVie，如果系统硬件资源少于 4 块则会报错，提示系统硬件配置错误。

1.2.6 STSSetMultiSiteBind()

```
int USERRES_API STSSetMultiSiteBind(  
MODULE_TYPE mdtype,  
AT_SITE_NO sitenum,  
char* ChannelList);
```

Parameters

mdtype

模块列表，可选的值如下：

MD_FPVie
MD_FOVie
MD_HVie
MD_ACM
MD_QVMe
MD_QTMUe
MD_HPVIe
MD_FXVie

sitenum

工位设置，可选的工位从 SITE_1 ~ SITE_32 和 NO_SITE，从如下列表选择：

SITE_1
SITE_2

...

SITE_32

NO_SITE

注意：

分配至 NO_SITE 的通道将不会受到工位屏蔽信号的限制，只要任意一个工位有效，皆可对该通道进行控制。

ChannelList

填写对应模块的“Slot 号+板内通道序号”字符列表。

注意：

ChannelList 对字符要求如下，需严格遵守，否则系统会报错。

1) 双引号内的字符只能出现：“S”、“_”、“,”、“-”和正整数。其中：

- a) “S”用于表示“Slot”，其后紧跟的数字即为槽位号；
- b) “_”用于分隔“槽位号”和“板内通道序号”；
- c) “,”用于分隔不同的通道
- d) “-”用于设置连续的通道
- e) 正整数代表对应模块的“槽位号”或“板内通道序号”。

2) 双引号内的“板内通道序号”不能超出对应模块的单板最大通道数量，“板内通道序号”从 0 开始，每个模块的单板最大通道号如下：

Resources	Maximum Channel Index
MD_FPVIe	1
MD_FOVle	7
MD_HVle	0
MD_ACM	23
MD_QVMe	3
MD_QTMUe	3
MD_HPVIe	0

MD_FXVle	11
----------	----

以下是几种正确配置和错误配置应用举例。

例 1：允许 S13_0-7，不允许 S13_0-3-7。

```
proper usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-7");
Wrong usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-3-7");
```

例 2：逗号后面必须有正整数，否则系统会报错。

```
proper usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0, S13_2, S13_5");
Wrong usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0, S13_2, S13_5,");
```

例 3：模块的同一槽位的同一通道不能配置两次，否则系统会报错。

```
proper usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-1");
STSSetMultiSiteBind(MD_FOVle,SITE_2,"S13_2-3");
Wrong usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-1");
STSSetMultiSiteBind(MD_FOVle,SITE_2,"S13_1-3");
```

例 4：模块的同一工位不能配置两次，否则系统会报错。

```
proper usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-1");
STSSetMultiSiteBind(MD_FOVle,SITE_2,"S13_2-3");
Wrong usage: STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_0-1");
STSSetMultiSiteBind(MD_FOVle,SITE_1,"S13_2-3");
```

Return Values

返回设置到 sitenum 的模块通道总数。

Remarks

设置某个模块的并行工作模式，程序的工位数将由所有模块中最大的工位数目来决定。

注意：

CBITe、DCM 模块不使用此方式做工位绑定。

Example

例 1：假设系统配置为 4 块 FPVle，4 块 FOVle，1 块 ACM，4 块 HVle。

四工位并行工作。

```
FPVle fpvie0("S4_0,S12_0,S21_0,S29_0");
FPVle fpvie1("S4_1,S12_1,S21_1,S29_1");

FOVle fovie0("S5_0,S11_0,S22_0,S28_0");
FOVle fovie1("S5_1,S11_1,S22_1,S28_1");
FOVle fovie2("S5_2,S11_2,S22_2,S28_2");
FOVle fovie3("S5_3,S11_3,S22_3,S28_3");
FOVle fovie4("S5_4,S11_4,S22_4,S28_4");
FOVle fovie5("S5_5,S11_5,S22_5,S28_5");
FOVle fovie6("S5_6,S11_6,S22_6,S28_6");
FOVle fovie7("S5_7,S11_7,S22_7,S28_7");

ACM acm0("S6_0,S6_6,S6_12,S6_18");
ACM acm1("S6_1,S6_7,S6_13,S6_19");
ACM acm2("S6_2,S6_8,S6_14,S6_20");
ACM acm3("S6_3,S6_9,S6_15,S6_21");
ACM acm4("S6_4,S6_10,S6_16,S6_22");
ACM acm5("S6_5,S6_11,S6_17,S6_23");

HVIle hvie0("S3_0,S13_0,S20_0,S30_0");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_FPVle, SITE_1, "S4_0,S4_1"); // set Slot4 ch0-1 to SITE1
STSSetMultiSiteBind(MD_FPVle, SITE_2, "S12_0,S12_1"); // set Slot12 ch0-1 to SITE2
STSSetMultiSiteBind(MD_FPVle, SITE_3, "S21_0,S21_1"); // set Slot21 ch0-1 to SITE3
STSSetMultiSiteBind(MD_FPVle, SITE_4, "S29_0,S29_1"); // set Slot29 ch0-1 to SITE4

STSSetMultiSiteBind(MD_FOVle, SITE_1, "S5_0-7"); // set Slot5 ch0-7 to SITE1
STSSetMultiSiteBind(MD_FOVle, SITE_2, "S11_0-7"); // set Slot11 ch0-7 to SITE2
STSSetMultiSiteBind(MD_FOVle, SITE_3, "S22_0-7"); // set Slot22 ch0-7 to SITE3
STSSetMultiSiteBind(MD_FOVle, SITE_4, "S28_0-7"); // set Slot28 ch0-7 to SITE4

STSSetMultiSiteBind(MD_ACM, SITE_1, "S6_0-5"); // set Slot6 ch0-5 to SITE1
STSSetMultiSiteBind(MD_ACM, SITE_2, "S6_6-11"); // set Slot6 ch6-11 to SITE2
STSSetMultiSiteBind(MD_ACM, SITE_3, "S6_12-17"); // set Slot6 ch12-17 to SITE3
STSSetMultiSiteBind(MD_ACM, SITE_4, "S6_18-23"); // set Slot6 ch18-23 to SITE4

STSSetMultiSiteBind(MD_HVIle, SITE_1, "S3_0"); // set Slot3 ch0 to SITE1
STSSetMultiSiteBind(MD_HVIle, SITE_2, "S13_0"); // set Slot13 ch0 to SITE2
STSSetMultiSiteBind(MD_HVIle, SITE_3, "S20_0"); // set Slot20 ch0 to SITE3
STSSetMultiSiteBind(MD_HVIle, SITE_4, "S30_0"); // set Slot30 ch0 to SITE4
```

例 2: 假设系统配置为 1 块 CBITe、1 块 DCM、1 块 QTMUe 和 1 块 QVMe。

四工位并行工作。

```
CBITecbite;

DCM dcm;

QTMUe qtmue0("S23_0-3");

QVMe qvme0("S2_0-3");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_QVMe, SITE_1, "S2_0"); // set Slot2 ch0 to SITE1
STSSetMultiSiteBind(MD_QVMe, SITE_2, "S2_1"); // set Slot2 ch1 to SITE2
STSSetMultiSiteBind(MD_QVMe, SITE_3, "S2_2"); // set Slot2 ch2 to SITE3
STSSetMultiSiteBind(MD_QVMe, SITE_4, "S2_3"); // set Slot2 ch3 to SITE4

STSSetMultiSiteBind(MD_QTMUe, SITE_1, "S23_0"); // set Slot23 ch0 to SITE1
STSSetMultiSiteBind(MD_QTMUe, SITE_2, "S23_1"); // set Slot23 ch1 to SITE2
STSSetMultiSiteBind(MD_QTMUe, SITE_3, "S23_2"); // set Slot23 ch2 to SITE3
STSSetMultiSiteBind(MD_QTMUe, SITE_4, "S23_3"); // set Slot23 ch3 to SITE4
```

1.2.7 STSCBITeMultiSiteBind()

```
int USERRES_API STSCBITeMultiSiteBind(
AT_SITE_NO siteNo,
char*ChannelList);
```

Parameters

siteNo

工位设置，可选的工位从 SITE_1 ~ SITE_32 和 NO_SITE，从如下列表选择：

SITE_1
SITE_2
...
SITE_32
NO_SITE

ChannelList

填写 CBITe 的通道字符列表。

注意：

ChannelList 对字符要求如下，需严格遵守，否则系统会报错。

- 1) 双引号内的字符只能出现：“,”、“-”和非负数。其中：
 - a) “,” 用于分隔通道
 - b) “-” 用于设置连续的通道

c) 非负数代表 CBITe 的通道号

2) 双引号内的非负数不能超出 CBITe 最大通道号，通道号从 0 开始。

CBITe 最大通道号为 255。

Return Values

返回设置到 siteNo 的通道总数。

Remarks

设置 CBITe 不同工位下可操作的通道。

该函数在 HardWareCfg()函数中调用，进行 CBITe 工位绑定。

以 SetOn 函数为例，其操作规则如下：

- 1) 只有有效工位和 NO_SITE 绑定的通道继电器闭合；
- 2) 无效工位下的通道继电器都断开；
- 3) 未进行工位绑定的通道继电器保持之前的状态；
- 4) 参数中所有未指定的继电器都断开。

具体函数操作规则见 CBITe 函数说明。

注意：

- 1) NO_SITE 不受工位有效性限制，任一个工位有效，NO_SITE 对应的通道都动作。
- 2) 如果 CBITe 不进行工位绑定，则按照原来通道设置规则进行。
- 3) 调试工具不受该绑定函数限制。

Example

例 1：对 1 块 CBITe 的通道进行工位绑定。

绑定如下：

```
STSCBITeMultiSiteBind(SITE_1, "0,1,2,3");  
STSCBITeMultiSiteBind(SITE_2, "4,5,6,7");  
STSCBITeMultiSiteBind(NO_SITE, "10,11 ");
```

CBITe 设置如下：

```
CBITe cbite;
```

```
cbite.SetOn(0, 4, 8, 10, -1);
```

若当前系统 SITE_1 有效，SITE_2 无效，则

- 1) 通道 0 和 10 闭合；
- 2) 通道 4 断开（SITE_2 无效）；
- 3) 通道 8 不动作，保持原来状态（未进行工位绑定）
- 4) 其他所有未设置通道（除 0、4、8、10 外）断开。

1.2.8 STSSetMultiSiteBindEx()

```
int USERRES_API STSSetMultiSiteBindEx (  
MODULE_TYPE mdtype,  
int sitenum,  
char* ChannelList);
```

Parameters

mdtype

模块列表，可选的值如下：

MD_FPVIe

MD_FOVIe

MD_HVIe

MD_ACM

MD_QVMe

MD_QTMUe

MD_HPVIe

MD_FXVIe

sitenum

工位设置，可选的工位为非负数和 NO_SITE。

非负数：表示绑定的工位号，如 0 表示工位 1，...，250 表示工位 251；

-1：表示绑定到 NO_SITE，此时 sitenum 可填-1 或 NO_SITE。

注意：

1 目前 8300 系统最大支持 251 工位；

2 分配至 NO_SITE 的通道将不会受到工位屏蔽信号的限制，只要任意一个工位有效，皆可对该通道进行控制。

ChannelList

填写对应模块的“Slot 号+板内通道序号”字符列表。

注意：

ChannelList 对字符要求如下，需严格遵守，否则系统会报错。

1) 双引号内的字符只能出现：“S”、“_”、“,”、“-”和正整数。其中：

a) “S”用于表示“Slot”，其后紧跟的数字即为槽位号；

b) “_”用于分隔“槽位号”和“板内通道序号”；

c) “,”用于分隔不同的通道

d) “-”用于设置连续的通道

e) 正整数代表对应模块的“槽位号”或“板内通道序号”。

2) 双引号内的“板内通道序号”不能超出对应模块的单板最大通道数量，“板内通道序号”从 0 开始，每个模块的单板最大通道号如下：

Resources	Maximum Channel Index
MD_FPVIe	1
MD_FOVle	7
MD_HVle	0
MD_ACM	23
MD_QVMe	3
MD_QTMUe	3
MD_HPVIe	0
MD_FXVIe	11

以下是几种正确配置和错误配置应用举例。

例 1：允许 S13_0-7，不允许 S13_0-3-7。

```
proper usage: STSSetMultiSiteBindEx(MD_FOVle, 0, "S13_0-7");  
Wrong usage: STSSetMultiSiteBindEx(MD_FOVle, 0, "S13_0-3-7");
```

例 2：逗号后面必须有正整数，否则系统会报错。

```
proper usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0, S13_2, S13_5");  
Wrong usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0, S13_2, S13_5,");
```

例 3：模块的同一槽位的同一通道不能配置两次，否则系统会报错。

```
proper usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0-1");  
STSSetMultiSiteBindEx (MD_FOVle,1,"S13_2-3");  
  
Wrong usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0-1");  
STSSetMultiSiteBindEx (MD_FOVle,1,"S13_1-3");
```

例 4：模块的同一工位不能配置两次，否则系统会报错。

```
proper usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0-1");  
STSSetMultiSiteBindEx (MD_FOVle,1,"S13_2-3");  
  
Wrong usage: STSSetMultiSiteBindEx (MD_FOVle,0,"S13_0-1");  
STSSetMultiSiteBindEx (MD_FOVle,0,"S13_2-3");
```

Return Values

返回设置到 `sitenum` 的模块通道总数。

Remarks

工位数扩展，最大可支持 251 工位。

设置某个模块的并行工作模式，程序的工位数将由所有模块中最大的工位数目来决定。

注意：

CBITe、DCM 模块不使用此方式做工位绑定。

Example

例 1：假设系统配置为 5 块 FOVle，40 工位并行工作。

```
FOVle fovie0("S5_0-7, S6_0-7, S7_0-7, S11_0-7, S12_0-7", "fovie0");
```

对应的工位绑定如下：

```

STSSSetMultiSiteBindEx(MD_FOVle, 0, "S5_0");           // set Slot5 ch0 to SITE1
STSSSetMultiSiteBindEx(MD_FOVle, 1, "S5_1");           // set Slot5 ch1 to SITE2
STSSSetMultiSiteBindEx(MD_FOVle, 2, "S5_2");
STSSSetMultiSiteBindEx(MD_FOVle, 3, "S5_3");
STSSSetMultiSiteBindEx(MD_FOVle, 4, "S5_4");
STSSSetMultiSiteBindEx(MD_FOVle, 5, "S5_5");
STSSSetMultiSiteBindEx(MD_FOVle, 6, "S5_6");
STSSSetMultiSiteBindEx(MD_FOVle, 7, "S5_7");
STSSSetMultiSiteBindEx(MD_FOVle, 8, "S6_0");
STSSSetMultiSiteBindEx(MD_FOVle, 9, "S6_1");
STSSSetMultiSiteBindEx(MD_FOVle, 10, "S6_2");
STSSSetMultiSiteBindEx(MD_FOVle, 11, "S6_3");
STSSSetMultiSiteBindEx(MD_FOVle, 12, "S6_4");
STSSSetMultiSiteBindEx(MD_FOVle, 13, "S6_5");
STSSSetMultiSiteBindEx(MD_FOVle, 14, "S6_6");
STSSSetMultiSiteBindEx(MD_FOVle, 15, "S6_7");
STSSSetMultiSiteBindEx(MD_FOVle, 16, "S7_0");
STSSSetMultiSiteBindEx(MD_FOVle, 17, "S7_1");
STSSSetMultiSiteBindEx(MD_FOVle, 18, "S7_2");
STSSSetMultiSiteBindEx(MD_FOVle, 19, "S7_3");
STSSSetMultiSiteBindEx(MD_FOVle, 20, "S7_4");
STSSSetMultiSiteBindEx(MD_FOVle, 21, "S7_5");
STSSSetMultiSiteBindEx(MD_FOVle, 22, "S7_6");
STSSSetMultiSiteBindEx(MD_FOVle, 23, "S7_7");
STSSSetMultiSiteBindEx(MD_FOVle, 24, "S11_0");
STSSSetMultiSiteBindEx(MD_FOVle, 25, "S11_1");
STSSSetMultiSiteBindEx(MD_FOVle, 26, "S11_2");
STSSSetMultiSiteBindEx(MD_FOVle, 27, "S11_3");
STSSSetMultiSiteBindEx(MD_FOVle, 28, "S11_4");
STSSSetMultiSiteBindEx(MD_FOVle, 29, "S11_5");
STSSSetMultiSiteBindEx(MD_FOVle, 30, "S11_6");
STSSSetMultiSiteBindEx(MD_FOVle, 31, "S11_7");
STSSSetMultiSiteBindEx(MD_FOVle, 32, "S12_0");
STSSSetMultiSiteBindEx(MD_FOVle, 33, "S12_1");
STSSSetMultiSiteBindEx(MD_FOVle, 34, "S12_2");
STSSSetMultiSiteBindEx(MD_FOVle, 35, "S12_3");
STSSSetMultiSiteBindEx(MD_FOVle, 36, "S12_4");
STSSSetMultiSiteBindEx(MD_FOVle, 37, "S12_5");
STSSSetMultiSiteBindEx(MD_FOVle, 38, "S12_6");
STSSSetMultiSiteBindEx(MD_FOVle, 39, "S12_7");

```

1.2.9 STSCBITeMultiSiteBindEx()

```

int USERRES_API STSCBITeMultiSiteBindEx (
int siteNo,
char* Channellist);

```

Parameters

siteNo

工位设置，可选的工位为非负数和 NO_SITE。

非负数：表示绑定的工位号，如 0 表示工位 1，...，250 表示工位 251；

-1: 表示绑定到 NO_SITE, 此时 sitenum 可填-1 或 NO_SITE。

ChannelList

填写 CBITe 的通道字符列表。

注意：

ChannelList 对字符要求如下, 需严格遵守, 否则系统会报错。

1) 双引号内的字符只能出现: “,”、“-” 和非负数。其中:

- a) “,” 用于分隔通道
- b) “-” 用于设置连续的通道
- c) 非负数代表 CBITe 的通道号

2) 双引号内的非负数不能超出 CBITe 最大通道号, 通道号从 0 开始。

CBITe 最大通道号为 255。

Return Values

返回设置到 siteNo 的通道总数。

Remarks

工位数扩展, 最大可支持 251 工位。设置 CBITe 不同工位下可操作的通道。

该函数在 HardWareCfg()函数中调用, 进行 CBITe 工位绑定。

以 SetOn 函数为例, 其操作规则如下:

- 1) 只有有效工位和 NO_SITE 绑定的通道继电器闭合;
- 2) 无效工位下的通道继电器都断开;
- 3) 未进行工位绑定的通道继电器保持之前的状态;
- 4) 参数中所有未指定的继电器都断开。

具体函数操作规则见 CBITe 函数说明。

注意：

1) NO_SITE 不受工位有效性限制, 任一个工位有效, NO_SITE 对应的通道都动作。

- 2) 如果 CBITe 不进行工位绑定，则按照原来通道设置规则进行。
- 3) 调试工具不受该绑定函数限制。

Example

例 1：对 1 块 CBITe 的通道进行工位绑定。

绑定如下：

```
STSCBITeMultiSiteBindEx(0, "0,1,2,3");  
STSCBITeMultiSiteBindEx(1, "4,5,6,7");  
STSCBITeMultiSiteBind(NO_SITE, "10,11 ");
```

CBITe 设置如下：

```
CBITe cbite;  
  
cbite.SetOn(0, 4, 8, 10, -1);
```

若当前系统工位 1 有效，工位 2 无效，则

- 1) 通道 0 和 10 闭合；
- 2) 通道 4 断开（SITE_2 无效）；
- 3) 通道 8 不动作，保持原来状态（未进行工位绑定）

其他所有未设置通道（除 0、4、8、10 外）断开。

1.2.10 STSSetSiteStatus()

```
void USERRES_API STSSetSiteStatus(DWORD siteID);
```

Parameters

siteID

工位有效信息，转换为 2 进制后，每一位代表一个工位，1 表示有效，0 表示无效。

如 0xFFFFFFFF 表示 32 个工位均有效，0x00000000 表示 32 个工位均无效。

Remarks

设置工位有效信息。

注意：

- 1) 使用完成后注意及时恢复之前的工位状态。一般需配合 STSGetSiteStatus 函数使用;
- 2) 最多可设置 32 个工位的状态。

Example

```
STSSetSiteStatus(0x0001); //0000 0000 0000 0001
// Set Site1 is valid, other sites are invalid.

STSSetSiteStatus(0x000D); //0000 0000 0000 1101
// Set Site1, Site3, Site4 is valid, other sites are invalid.

STSSetSiteStatus(0x000F); // 0000 0000 0000 1111
// Set site1, site2, site3, site4 is valid, other sites are invalid.

STSSetSiteStatus(0xFFFF); //1111 1111 1111 1111
// Set site1,... Site16 is valid.
```

1.2.11 STSGetSiteStatus()

```
int STSGetSiteStatus(
    BYTE *bysitestatus,
    int sitecnt);
```

Parameters

bysitestatus

保存获取的工位信息的数组指针。

如果数组成员值为 1，则表示此工位为有效工位；

如果为 0，则表示此工位被屏蔽。

sitecnt

要获取的工位信息的工位数目。

注意：

确保 bysitestatus 的参数指针分配了这么多个工位的存储空间。

Return Values

返回 0: 正常;

返回-1: bysitestatus 为 NULL 或 size 小于 0

Remarks

获取工位的使能状态。

注意：

最多获取的工位数不超过程序中设置的工位数。

Example

```
BYTE sitesta[4];  
STSGetSiteStatus(sitesta, 4); // Obtain effective information of each station
```

1.2.12 STSGetSiteRunStatus()

```
int STSGetSiteRunStatus (  
    BYTE *bysitestatus,  
    int sitecnt);
```

Parameters

bysitestatus

保存获取的工位信息的数组指针。

如果数组成员值为 1，则表示此工位本次执行测试；

如果为 0，则表示此工位本次不执行测试。

sitecnt

要获取的工位信息的工位数目。

注意：

确保 bysitestatus 的参数指针分配了这么多个工位的存储空间。

Return Values

返回 0: 正常;

返回-1: bysitestatus 为 NULL 或 size 小于 0

Remarks

UserInit-->BinOutDut 之间获取 SOT 接收工位状态和 TestUI 界面用户选择工位状态的交集。

UserLoad 中此函数无效。

其它函数中获取 TestUI 界面用户选择的工位状态。

注意：

超过程中设置工位数的工位状态值无效。

Example

```
BYTE sitesta[4];  
STSGetSiteRunStatus (sitesta, 4); // Get the status of whether the site is tested
```

1.2.13 STSSetSiteStatusEx()

```
int USERRES_API STSSetSiteStatusEx (BYTE *siteStatus, int size);
```

Parameters

siteStatus

保存设置的工位信息的数组指针。

如果数组成员值为 1，则表示此工位为有效；

如果为 0，则表示此工位无效。

size

siteStatus 数组大小。

Remarks

工位扩展，设置工位有效信息。

注意：

- 1) 使用完成后注意及时恢复之前的工位状态。一般需配合 STSGetSiteStatusEx 函数使用；
- 2) 最多可设置 251 个工位的状态。

Example

```
BYTE siteStatus[64] = { 0 };  
siteStatus[0] = 1;  
siteStatus[63] = 1;  
STSSetSiteStatusEx(siteStatus, 64); // Set Site1 and site64 is valid, other sites are invalid.
```

1.2.14 STSGetSiteStatusEx()

```
int USERRES_API STSGetSiteStatusEx(BYTE *siteStatus, int size);
```

Parameters

siteStatus

保存获取的工位信息的数组指针。

如果数组成员值为 1，则表示此工位为有效工位；

如果为 0，则表示此工位被屏蔽。

size

要获取的工位信息的工位数目。

注意：

确保 siteStatus 的参数指针分配了这么多个工位的存储空间。

Return Values

返回 0：正常；

返回-1：siteStatus 为 NULL 或 size 小于 0

Remarks

工位扩展，获取工位是否进行测试的信息。

注意

最多可获取 251 个工位的状态。

Example

```
BYTE sitesta[64] = {0};  
STSGetSiteStatusEx(sitesta, 64); // Obtain effective information of each station
```

1.2.15 STSGetSingleSiteStatus ()

```
int USERRES_API STSGetSingleSiteStatus (UINT siteNo);
```

Parameters

siteNo

工位号，获取该工位的工位状态。该值必须小于程序设置的工位数。

0 表示工位 1，...

Return Values

返回 0：此工位被屏蔽；

返回 1：此工位有效；

返回-1：siteNo ≥ pgs 设置的工位数

Remarks

获取指定工位的工位状态。

最大可支持获取第 251 个工位的状态，即 siteNo=250。

Example

```
int siteStatus = STSGetSingleSiteStatus(0); //Get status of site 1
```

1.2.16 STSGetOperatorID()

```
int STSGetOperatorID(  
char* operatorid,
```

```
int charCnt);
```

Parameters

operatorid

操作用户的 ID。

charCnt

用户需要获取的 OperatorID 的长度，必须不大于 operatorid 数组长度。

Return Values

如果未设置（字符串长度为空）则返回-1，正常则返回 0。

Remarks

获取界面操作者的用户登陆 ID。

Example

```
char operatorid[10];  
STSGetOperatorID(operatorid, 5);
```

1.2.17 STSGetWaferID()

```
int STSGetWaferID(  
char*waferid,  
int charCnt);
```

Parameters

waferid

测试的 Wafer 的 ID。

charCnt

用户需要获取的 WaferID 的长度，必须不大于 waferid 数组长度。

Return Values

如果未设置（字符串长度为空）则返回-1，正常则返回 0。

Remarks

获取圆片的 ID。

Example

```
char waferid[10];  
STSGetWaferID(waferid, 5);
```

1.2.18 STSGetLotID()

```
int STSGetLotID(  
char* lotid,  
int charCnt);
```

Parameters

lotid

测试的 Lot 的 ID。

charCnt

用户需要获取的 LotID 的长度，必须不大于 lotid 数组长度。

Return Values

如果未设置（字符串长度为空）则返回-1，正常则返回 0。

Remarks

获取 Lot 的 ID。

Example

```
char lotid[10];  
STSGetLotID(lotid, 5);
```

1.2.19 STSGetCurrentDutSwBin()

```
short STSGetCurrentDutSwBin(int sitecnt);
```

Parameters

sitecnt

需要获取分档信息的工位号。

Return Values

如果程序设置的工位数小于 sitecnt，则返回-1，如果需要获取的分 Bin 号未进行定义，则返回 0。

Remarks

获取实时每个 Site 的器件 SoftBin 号。

Example

例 1：获取工位 1 的 SoftBin 号。

```
short SwBin=0;  
SwBin= STSGetCurrentDutSwBin(0);  
delay_ms(1);
```

1.2.20 STSGetCurrentDutHwBin()

```
short STSGetCurrentDutHwBin(int sitecnt);
```

Parameters

sitecnt

需要获取分档信息的工位号。

Return Values

如果程序设置的工位数小于 sitecnt，则返回-1，如果需要获取的分 Bin 号未进行定义，则返回 0。

Remarks

获取实时每个 Site 的器件 HardBin 号。

Example

例 1：获取工位 1 的 HardBin 号。

```
short HwBin=0;  
HwBin= STSGetCurrentDutHwBin(0);  
delay_ms(1);
```

1.2.21 StsGetParam()

```
CParam *StsGetParam(  
short funcIndex,  
LPCTSTR lpParamName);
```

Parameters

funcIndex

lpParamName

参数名称

Return Values

返回获取的函数参数指针，若参数被删除，则返回 NULL。

Remarks

获取函数参数指针,若返回 NULL，则不可使用该指针来操作参数赋值及获取条件值等操作。

1.2.22 StsGetBinDutTotalCount()

```
int StsGetBinDutTotalCount(  
int iBinNumber,  
int sitecnt);
```

Parameters*iBinNumber*

bin 号

sitecnt

工位号

Return Values

若工位号及 Bin 号满足程序及 PGS 中定义的大小，则返回当前 Bin 的器件在输入工位号的数量，否则返回-1。

Remarks

获取对应工位的分 bin 器件数量。

1.2.23 StsAddUserSummaryStr()

```
int StsAddUserSummaryStr(LPSTR strsum);
```

Parameters

strsum

用户需要自定义添加的 Summary 信息

Return Values

若增加的信息与上次一致，则返回-1，不成功，若自定义信息长度为 0，则返回-2，不成功。其他情况返回 0，执行成功。

Remarks

添加用户定制的 Summary 信息。

1.2.24 StsMessageBox()

```
int StsMessageBox(  
LPCSTR lpText,  
LPCSTR lpCaption);
```

Parameters

lpText

窗体显示的提示信息

lpCaption

窗体显示的标题

Remarks

弹出提示信息窗体

1.2.25 StsGetSiteCount()

```
int StsGetSiteCount();
```

Return Values

返回 PGS 中定义的工位数量

Remarks

获取 PGS 设置的工位个数

1.2.26 STSGetPgsName()

```
int STSGetPgsName(  
LPTSTR pgsName,  
int chNum);
```

Parameters

pgsName

返回的 pgs 文件名参数

chNum

pgsName 参数的缓存大小

Return Values

chNum 参数小于 1 时返回-1，无法获取 PGS 的名称。正常返回 0。

Remarks

获取当前加载的 pgs 文件名

1.2.27 STSGetPgsPath()

```
int STSGetPgsPath(  
LPTSTR pgsPath,  
int chNum);
```

Parameters

pgsPath

返回的 pgs 文件路径参数

chNum

pgsPath 参数的缓存大小

Return Values

chNum 参数小于 1 时返回-1，无法获取 PGS 的路径。正常返回 0。

Remarks

获取当前加载的 pgs 文件的路径

1.2.28 STSSetTimeCheck()

```
int STSSetTimeCheck(int tag);
```

Parameters

tag

需要记录时间的标记号，调用这个函数的时刻记录为起始时间。

Return Values

返回 0

Remarks

设置时间检查标记，此函数与 STSGetTimeElapsed 配对使用，若多次给入的标记，则以最后一次设定的标记的时间为起始时间。

1.2.29 STSGetTimeElapsed()

```
double STSGetTimeElapsed(int tag);
```

Parameters

tag

用来计算时间差的标记

Return Values

返回 STSSetTimeCheck 函数到此函数之间经过的时间。

Remarks

获取时间标记经过的时间，单位毫秒，此函数与 STSSetTimeCheck 配对使用。

1.2.30 STSSetSaveInterval()

```
int STSSetSaveInterval(int interval);
```

Parameters

interval

存储间隔

Return Values

Interval 小于 0，返回-2，其他执行正确返回 0。

Remarks

设置保存数据的采样间隔（此模式为 Fail 数据都保存模式）。

1.2.31 STSStopProgram()

```
int STSStopProgram();
```

Remarks

停止当前测试程序，执行完调用此语句的测试函数，后面的所有测试函数不再执行，跳转到 InitAfterTestFlow 函数。

1.2.32 STSStartNewLot()

```
int STSStartNewLot();
```

Remarks

启动下一个测试新批次，调用此函数后，若之前在自动测试模式，则系统会停止当前测试并弹出 NewLot 对话框。

1.2.33 STSGetUserPriority()

```
int STSGetUserPriority();
```

Return Values

返回值及含义如下：

#define OPERATOR1

#define ENGINEER2

#define ADMIN3

Remarks

获取当前登录用户的权限级别。

1.2.34 STSSetEnableCalCheck()

```
int STSSetEnableCalCheck(  
    BOOL bEnable= TRUE,  
    int iCalMonthInterval= 6);
```

Parameters

bEnable

是否使能硬件板卡的校准检查功能，默认值为 TRUE。

iCalMonthInterval

校准间隔，单位：月（注意，系统按照 30 天/月进行处理）。范围：1~12。

默认值为 6 个月。

Return Values

执行正确返回 0，执行异常返回-1。

Remarks

设置是否使能校准检查功能。

注意：

- 1) 目前版本只能检查 FPVIe、FOVIe、ACM、HVIe、QTMUe、QVMe、DCM、HPVIe 和 FXVIe 的校准是否通过及时间。
- 2) 此检查是根据多工位配置（HardwareCfg 函数中配置的资源）分别针对每个工位进行检查，此函数需要在 UserLoad 函数中调用。
- 3) 此函数同时设置两次校准之间的时间间隔（仅当 bEnable 参数为 TRUE 时有效）。

1.2.35 STSGetPartIDText()

```
const char* STSGetPartIDText(short siteNum);
```

Parameters

siteNum

工位号

Return Values

返回当前工位的 PartID。

Remarks

在测试程序中可以利用此函数获取用户定义的或者是自动生成的 PartID。

1.2.36 STSSetEnableAlarm()

```
int STSSetEnableAlarm(BOOL bEnable= TRUE);
```

Parameters

bEnable

是否使能支持系统的参数 Alarm 功能，默认值为 TRUE。

Return Values

设置成功返回 0，异常返回-1。

Remarks

设置系统开启参数 Alarm 功能，系统界面也可以设置，但是以此函数设置优先，开启 Alarm 后，系统将会将硬件产生的 Alarm（目前有箝位等），以及函数中设置超过范围的参数记录为 Alarm，如果设置了 Alarm Bin 的话，器件将会被分到 Alarm Bin。

1.2.37 STSSetWriteAlarmFile()

```
void STSSetWriteAlarmFile(BOOLalarmfile_flag);
```

Parameters

alarmfile_flag

是否存储参数 Alarm 信息，TRUE 为存储，FALSE 为不存储。

Remarks

系统开启参数 Alarm 功能后，默认存储产生的所有参数 Alarm 信息，若 Alarm 信息较多，可能会占用测试时间。所以，在测试程序执行前利用该函数进行设置，将不会对产生的参数 Alarm 信息进行存储。

1.2.38 STSTArming()

```
intSTSTArming();
```

Remarks

等待 Turbo 模式下运行的内容执行完成。

Example

例：FXVle 模块的 fxvie0 通道进行 TSet 操作。

```
fxvie0.TSet(FV, 4, FXVle_5V, FXVle_100MA, FXVle_RELAY_ON); // Turbo Set  
STSTArming(); // Turbo Arming
```

1.3 测试参数相关函数

测试参数相关函数，主要用于获取在 PGS 文件中测试参数的相关信息，如某参数的上限值、下限值、条件个数等信息，同时也可以对某参数的一些信息进行设置。其中最常用的就是 SetTestResult()函数，用于设置参数的测试结果。

1.3.1 SetTestResult()

```
int SetTestResult(  
short sSiteID,  
short sSubUnitID,  
double dResult);
```

Parameters

sSiteID

设置值对应的工位数，0 代表 Site1，1 代表 Site2.....

sSubUnitID

设置值对应的子单元数，0 代表单元 1，1 代表单元 2.....

dResult

设置的结果值。

Return Values

函数调用成功返回 0，否则返回非 0。

Remarks

设置参数测试结果。

Example

例 1: fovie0 的测流结果赋值给参数 Icc，并将 Icc 的显示结果转化为 mA。

```
fovie0.MeasureVI(20, 10);
for (i=0; i<SITENUM; i++)
{
    adresult[i]= fovie0.GetMeasResult(i, MIRET);
    Icc->SetTestResult(i, 0, adresult[i]*1000);
}
```

1.3.2 GetTestResult()

```
double GetTestResult(
short sSiteID,
short sSubUnitID);
```

Parameters

sSiteID

获取对应的工位数，0 代表 Site1，1 代表 Site2.....

sSubUnitID

获取对应的子单元数，0 代表单元 1，1 代表单元 2.....

Return Values

返回测试结果。

Remarks

获取参数的测试结果。

Example

```
double result=param->GetTestResult(0,0);
```

1.3.3 SetResultRemark()

```
int SetResultRemark(  
short sSiteID,  
short sSubUnitID,  
LPCTSTR lpstrremark);
```

Parameters

sSiteID

注释信息对应的工位数，0 代表 Site1，1 代表 Site2.....

sSubUnitID

注释信息对应的子单元数，0 代表单元 1，1 代表单元 2.....

lpszremark

指向注释信息的字符串指针。

Return Values

函数调用成功返回 0，否则返回非 0。

Remarks

设置参数测试结果的注释信息。

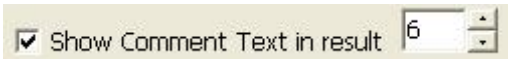
Example

```
fovie0.MeasureVI(20,10);
for (i=0; i<SITENUM; i++)
{
    adresult[i]= fovie0.GetMeasResult(i, MIRET);
    Icc->SetTestResult(i, 0, adresult[i]*1000);
    Icc->SetResultRemark(i, 0, "Icc_5V");
}
```

Parameter	Unit	Min	Max	Time{...	Site1
					PART_ID: 4
				38.18	SW Bin: 1
Icc	mA		3	38.09	1.230 (Icc_5V)

注意：

1) 显示注释信息，需选中测试主界面 Options→View→Show Comment Text in result 选项，并合理设置 Text 的长度。



2) 注释信息不保存在测试数据中。

1.3.4 GetResultRemark()

```
short GetResultRemark(  
short sSiteID,  
short sSubUnitID,  
LPTSTR lpzResultRemark,  
DWORD cbBuf);
```

Parameters

sSiteID

注释信息对应的工位数，0 代表 Site1，1 代表 Site2.....

sSubUnitID

注释信息对应的子单元数，0 代表单元 1，1 代表单元 2.....

lpzResultRemark

保存注释信息的字符串首地址。

cbBuf

保存注释信息的字符串的长度。

Return Values

返回测试结果的注释信息。

Remarks

获取函数测试结果的注释信息。

Example

```
char sRemark[50];  
memset(sRemark,0,50);  
param->GetResultRemark(0,0,sRemark,50);
```

1.3.5 GetMinLimit()

```
double GetMinLimit();
```

Return Values

返回参数下限值。

Remarks

获取参数下限值。

1.3.6 GetMaxLimit()

```
double GetMaxLimit();
```

Return Values

返回参数上限值。

Remarks

获取参数上限值。

1.3.7 GetSubUnitsCount()

```
short GetSubUnitsCount();
```

Return Values

返回子单元数。

Remarks

获取参数的子单元数。

Example

```
short subUnitCnt=param->GetSubUnitsCount ();
```

1.3.8 GetDispFormat()

```
short GetDispFormat(  
LPTSTR lpszDispFormat,  
DWORD cbBuf);
```

Return Values

返回测试结果的显示格式，类似“0.00”的字符串。

Remarks

获取测试结果的显示格式。

1.3.9 GetDescription()

```
short GetDescription(  
LPTSTR lpszDescription,  
DWORD cbBuf);
```

Parameters

lpszDescription

用于保存参数描述信息的字符串首地址。

cbBuf

用于保存参数描述信息字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 cbBuf 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取参数的描述信息。

Example

```
char sDescription[50];  
memset(sDescription,0,50);  
param->GetDescription(sDescription,50);
```

1.3.10 GetConditionsCnt()

```
short GetConditionsCnt();
```

Return Values

返回参数拥有的测试条件个数。

Remarks

获取测试条件个数。

Example

```
short ConditionCnt=param->GetConditionsCnt();
```

1.3.11 GetConditionDispName()

```
short GetConditionDispName(  
const int index,  
LPTSTR lpszCondName,  
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

用于保存条件显示名的字符串首地址。

cbBuf

用于保存条件显示名字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 *cbBuf* 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件显示名。

Example

```
charsCondName[50];  
memset(sCondName,0,50);  
param->GetConditionDispName(0,sCondName,50);
```

1.3.12 GetConditionSymbol()

```
short GetConditionSymbol(  
int index,  
LPTSTR lpszCondSymbol,
```

```
DWORD cbBuf);
```

```
short GetConditionSymbol(  
LPCTSTR lpszCondName,  
LPTSTR lpszCondSymbol,  
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

lpszCondSymbol

用于保存条件别名的字符串首地址。

cbBuf

用于保存条件别名字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 *cbBuf* 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件别名。

Example

```
char sCondSymbol[50];  
memset(sCondSymbol,0,50);  
param->GetConditionSymbol (0,sCondSymbol,50);  
param->GetConditionSymbol ("Voltage",sCondSymbol,50);
```

1.3.13 GetConditionShowKind()

```
BYTE GetConditionShowKind(int index);
```

```
BYTE GetConditionShowKind(LPCTSTR lpszCondName);
```

Parameters

index

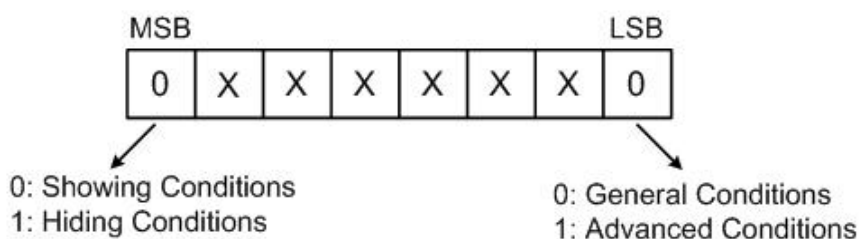
测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

Return Values

返回一个 BYTE 型数据。



Remarks

返回测试条件的显示类型。

Example

```
BYTE ShowKind=param->GetConditionShowKind ("Voltage");  
BYTE ShowKind1=param->GetConditionShowKind (0);
```

1.3.14 GetConditionInputKind()

```
BYTE GetConditionInputKind(int index);
```

```
BYTE GetConditionInputKind(LPCTSTR lpszCondName);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

Return Values

返回测试条件的输入格式类型。

Remarks

获取测试条件的输入格式，0 填空，1 下拉选择，2 多选下拉框。

Example

```
BYTE InputKind=param->GetConditionInputKind("Voltage");  
BYTE InputKind1=param->GetConditionInputKind(0);
```

1.3.15 GetConditionDispValue()

```
short GetConditionDispValue(  
int index,  
LPTSTR lpszCondDispValue,  
DWORD cbBuf);
```

```
short GetConditionDispValue(  
LPCTSTR lpszCondName,  
LPTSTR lpszCondDispValue,  
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

lpszCondDispValue

用于保存条件所有可选值的字符串首地址。

cbBuf

用于保存条件所有可选值字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 *cbBuf* 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件所有可选值，多个值之间以 “;” 分隔。

Example

```
char sCondAllValue[50];  
memset(sCondAllValue,0,50);  
param->GetConditionDispValue(0,sCondAllValue,50);  
param->GetConditionDispValue("Voltage",sCondAllValue,50);
```

1.3.16 GetConditionDispUnit()

```
short GetConditionDispUnit(  
    int index,  
    LPTSTR lpszDispUnit,  
    DWORD cbBuf);
```

```
short GetConditionDispUnit(  
    LPCTSTR lpszCondName,  
    LPTSTR lpszDispUnit,  
    DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

lpszDispUnit

用于保存条件所有可选单位的字符串首地址。

cbBuf

用于保存条件所有可选单位字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 *cbBuf* 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件所有可选单位，多个值之间以“;”分隔。

Example

```
char sCondAllUnit[50];
memset(sCondAllUnit,0,50);
param->GetConditionDispUnit (0,sCondAllUnit,50);
param->GetConditionDispUnit ("Voltage",sCondAllUnit,50);
```

1.3.17 GetTestConditionValue()

```
short GetTestConditionValue(
int index,
LPTSTR lpszCondValue,
DWORD cbBuf);
```

```
short GetTestConditionValue(
LPCTSTR lpszCondName,
LPTSTR lpszCondValue,
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

lpszCondValue

用于保存条件当前值的字符串首地址。

cbBuf

用于保存条件当前值字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 cbBuf 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件当前值位。

Example

```
char sCondValue[50];  
memset(sCondValue,0,50);  
param->GetTestConditionValue (0,sCondValue,50);  
param->GetTestConditionValue ("Voltage",sCondValue,50);
```

1.3.18 GetConditionCurSelDouble()

```
double GetConditionCurSelDouble (short index);
```

```
double GetConditionCurSelDouble (LPCTSTR lpszCondName);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

Return Values

返回 double 型的测试条件当前选择值。

Remarks

获取条件当前值的 double 型数据。

Example

```
double value=param->GetConditionCurSelDouble(0);  
double value=param->GetConditionCurSelDouble("Voltage");
```

1.3.19 GetConditionValueSelOrder()

```
int GetConditionValueSelOrder(short index);
```

```
int GetConditionValueSelOrder(LPCTSTR lpszCondName);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

Return Values

对于条件的输入格式为下拉框时，返回当前选择值在所有可选值中的次序。

Remarks

当前选择值在所有可选值中的次序。

Example

```
// All the optional values are "50; 40; 20; 10; 5; 2; 1", when the current selection value is 20, the  
return value is 2.  
double valueOrder=param->GetConditionValueSelOrder(0);  
double valueOrder =param->GetConditionValueSelOrder("Voltage");
```

1.3.20 GetConditionSelectUnit()

```
short GetConditionSelectUnit(  
int index,  
LPTSTR lpszCondSelUnit,  
DWORD cbBuf);
```

```
short GetConditionSelectUnit(  
LPCTSTR lpszCondName,  
LPTSTR lpszCondSelUnit,  
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

lpszCondSelUnit

用于保存条件当前值的字符串首地址。

cbBuf

用于保存条件当前值字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 cbBuf 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件当前选择单位。

Example

```
char sCondUnit[50];  
memset(sCondUnit,0,50);  
param->GetConditionSelectUnit(0,sCondUnit,50);  
param->GetConditionSelectUnit("Voltage",sCondUnit,50);
```

1.3.21 GetConditionUnitSelOrder()

```
int GetConditionUnitSelOrder(short index);
```

```
int GetConditionUnitSelOrder(LPCTSTR lpszCondName)
```

Parameters

index

测试条件的序列号，从 0 开始。

lpszCondName

测试条件名。

Return Values

对于条件的输入格式为下拉框时，返回当前选择单位在所有可选单位中的次序。

Remarks

获取当前选择单位在所有可选单位中的次序。

Example

```
// All the optional values are "V; mV; uV", when the current selection value is "V", the return value is 0.  
double valueOrder=param->GetConditionUnitSelOrder(0);  
double valueOrder =param->GetConditionUnitSelOrder("Voltage");
```

1.3.22 GetParamStationNo()

```
short GetParamStationNo();
```

Parameters

Return Values

参数所在的 Station 。

Remarks

多站模式，获取参数所在的测试站（不包括空站）。

Example

```
short nStationNum = Icc-> GetParamStationNo ();
```

1.3.23 GetConditionDispTip()

```
short GetConditionDispTip(  
int index,  
LPTSTR lpszCondDispTip,  
DWORD cbBuf);
```

```
short GetConditionDispTip(  
LPCTSTR lpszCondName,  
LPTSTR lpszCondDispTip,  
DWORD cbBuf);
```

Parameters

index

测试条件的序列号，从 0 开始。

lpzCondName

测试条件名。

lpzCondDispTip

用于保存当前值的字符串首地址。

cbBuf

用于保存当前值字符串空间的大小。

Return Values

函数调用成功返回 0；返回正整数表示 cbBuf 值比需要的空间值小，返回值为实际需要的空间大小；返回负数表示操作失败。

Remarks

获取条件当前提示信息。

Example

```
char sCondUnit[50];
memset(sCondUnit,0,50);
param->GetConditionDispTip (0,sCondUnit,50);
param->GetConditionDispTip ("Voltage",sCondUnit,50);
```

1.4 软件示波器显示用户自定义波形函数

1.4.1 STSSetSoftViewerData ()

```
int STSSetSoftViewerData (
AT_SITE_NO siteNo,
char*      label,
```

```
int      dataSerialNum,  
int      dataCount,  
double*  value,  
char*    valueUnit,  
double*  interval,  
char*    intervalUnit);
```

Parameters

siteNo

工位号。

label

软件示波器中显示的波形名称。 字符长度限定为 **1~100**。

dataSerialNum

第几组数 该变量是波形的唯一标识，内部处理为通道号。范围为 **0~99**

dataCount

每组数据的个数。范围为 **1~65536**

value

数据

valueUnit

数据单位 为一个字符串

interval

采样间隔 个数和数据一样多

intervalUnit

采样间隔单位 为一个字符串

Return Values

成功返回 0，失败返回非 0。

Remarks

用于写入要在软件示波器中显示的数据。允许不同的数据间有不同的采样间隔。采样间隔单位要一致。

Example

显示含有 10 个点的一个波形，数据为 value，单位为 mA，采样间隔为 interval，单位为 us，在软件示波器显示的波形 label 为 LABEL_MI_1，组号为 0，工位为 SITE_1。

```
double value[10] = {1, 1.2, 2, 3, 0.5, 0.6, 0.8, 0, 4, 3};
```

```
double interval[10] = {8, 9, 10, 10, 10, 10, 10, 10, 10, 10};
```

```
STSSetSoftViewerData(SITE_1, "LABEL_MI_1", 0, 10, value, "mA", interval, "us");
```

1.4.2 STSSetSoftViewerData2 ()

```
int STSSetSoftViewerData2(  
AT_SITE_NO siteNo,  
char*      label,  
int        dataSerialNum,  
int        dataCount,  
double*    value,  
char*      valueUnit,  
double     interval,  
char*      intervalUnit);
```

Parameters

siteNo

工位号。

label

软件示波器中显示的波形名称。 字符长度限定为 1~100。

dataSerialNum

第几组数 该变量是波形的唯一标识，内部处理为通道号。范围为 0~99

dataCount

每组数据的个数。范围为 1~65536

value

数据

valueUnit

数据单位 为一个字符串

interval

采样间隔 一个数值

intervalUnit

采样间隔单位 为一个字符串

Return Values

成功返回 0，失败返回非 0。

Remarks

用于写入要在软件示波器中显示的数据。不同的数据间采样间隔一致。

Example

显示含有 10 个点的一个波形，数据为 value，单位为 mA，采样间隔为 interval，单位为 us，在软件示波器显示的波形 label 为 LABEL_MI_1，组号为 0，工位为 SITE_1。
double value[10] = {1, 1.2, 2, 3, 0.5, 0.6, 0.8, 0, 4, 3};
double interval = 10;
STSSetSoftViewerData2(SITE_1, "LABEL_MI_1", 0, 10, value, "mA", interval, "us");

1.4.3 STSClearSoftViewerData ()

```
int STSClearSoftViewerData(int dataSerialNum);
```

Parameters

dataSerialNum

组号。该变量是波形的唯一标识，内部处理为通道号。范围为 0~99

Return Values

成功返回 0，失败返回非 0。

Remarks

清除指定组号的数据。数据清除后，在软件示波器中不再显示数据及波形。

Example

清除组号为 0 的波形数据。
STSClearSoftViewerData(0);

1.4.4 STSClearAllSoftViewerData ()

```
int STSClearAllSoftViewerData();
```

Parameters

Return Values

成功返回 0，失败返回非 0。

Remarks

清除所有的波形数据。

Example

```
清除所有的波形数据。  
STSClearAllSoftViewerData();
```

1.5 特殊应用函数

1.5.1 StsGetSingleDieCorXY()

```
int StsGetSingleDieCorXY(  
int sitecnt,  
int&CorX,  
int&CorY);
```

Parameters

sitecnt

获取对应的工位数，0 代表 Site1，1 代表 Site2.....

注意：

一般情况下 prober 传来的第一个 DIE 的坐标即为 Site1 的坐标，因此此参数可设置为 0 来获取 Site1 的坐标信息。

CorX

用于保存 X 坐标信息的变量。

CorY

用于保存 Y 坐标信息的变量。

Return Values

如果获取成功则返回 0，失败则返回-1。

Remarks

用于获取某一个 DIE 的坐标信息。

注意：

获取到第一个 DIE 坐标后，可以根据针卡的排列方式来确定其他的工位的坐标。具体样例可参考 1.5.2 节。

1.5.2 StsSetDieCorXY()

```
int StsSetDieCorXY(  
int sitecnt,  
int CorX,  
int CorY);
```

Parameters

sitecnt

设置 DIE 坐标对应的工位数，0 代表 Site1，1 代表 Site2.....

注意：

一般要修改的是除了 Site1 以外的其他工位的坐标；

CorX

设置 X 坐标的值。

CorY

设置 Y 坐标的值。

注意：

一般先通过 StsGetSingleDieCorXY()函数获取第一个 DIE 的坐标信息，再通过 StsSetDieCorXY()函数设置其他 DIE 的坐标信息。具体样例可参考 1.5.2 节。

Remarks

用于设置各工位 DIE 的坐标值。

1.5.3 STSGetDieCorXY()

```
int STSGetDieCorXY(  
int*CorX,  
int*CorY,  
int sitecnt);
```

Parameters

CorX, *CorY*

保存获取的 Die 的 X、Y 坐标的数组名。

sitecnt

需要获取的工位信息的工位数目，注意确保下一个参数指针分配了这么多工位的存储空间。

Return Values

如果程序设置的工位数小于 sitecnt，则返回-1，否则返回 0。

Remarks

获取正在测试的圆片的 Die 的 X、Y 坐标，需要与工位是否测试信息进行配合，有效的工位坐标才有意义。

Example

```
int CorX[4]={0};  
int CorY[4]={0};  
STSGetDieCorXY(CorX, CorY, 4);
```

1.5.4 GetNewLotInfoCnt()

```
int GetNewLotInfoCnt();
```

Return Values

该函数的返回值即为 Lot Item 的项目个数。

Remarks

获取 Lot Item 界面设置的项目个数。

1.5.5 GetNewLotItemInfo()

```
int GetNewLotItemInfo(  
short index,  
LPSTR chlotinfoname,  
LPSTR chlotinfo,  
short buffsize);
```

Parameters

index

对应 Lot Item 界面中 item 的序号，项目序号从上至下为 0, 1, 2...

chlotinfoname

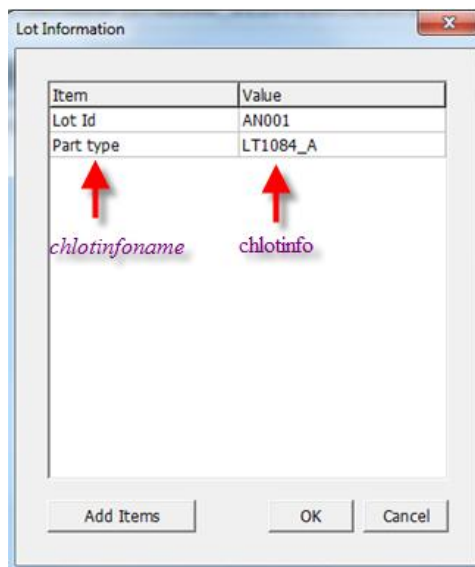
用于保存所选 item 的名称。

chlotininfo

用于保存所选 item 的具体信息。

buffsize

设置存储 item 信息的字符串大小。



Remarks

获取 Lot Item 界面中各 item 的具体信息。

注意：

具体样例可参考 1.5.3 节。

1.5.6 STSSetInitSelectDialog()

```
int STSSetInitSelectDialog(
LPSTR lpTitle,
```

```
BYTE byCount,  
BYTE byDefault,  
LPSTR lpUserSelectSTR);
```

Parameters

lpTitle

设置对话框界面的标题。

byCount

设置参数 lpUserSelectSTR 中可选择 item 的个数。

byDefault

设置参数 lpUserSelectSTR 中的默认 item 的序号，从 0 开始。

lpUserSelectSTR

设置显示选择的字符内容，多个选择内容需要用逗号“,”分隔。字符内容可以由任意数字字符组成。

Return Values

该函数的返回值即为用户选择的 item 的序号，从 0 开始。

Remarks

设置对话框界面。

注意：

具体样例可参考 1.5.6 节。

1.5.7 StsSetSaveFileNameStr()

```
int StsSetSaveFileNameStr(LPSTR strFileName);
```

Parameters

strFileName

设置程序的文件名。

Remarks

设置程序的文件名。

注意：

具体样例可参考 1.5.7 节。

1.5.8 STSEnableDebugTool()

```
void STSEnableDebugTool ();
```

Remarks

AutoRun 连续测试过程中，使能在 Debug 调试模式下启动 DebugTool 工具。

注意：

该函数仅在 “{ }” 作用域内起作用。

Example

```
void test_fovie()
{
    int n = 0;
    ++n;
    STSEnableDebugTool(); // Enable the DebugTool, DebugTool can be used ++n
}
```

1.5.9 STSSetAlwaysRunFunctionItem ()


```
int USERTYPE STSSetAlwaysRunFunctionItem(unsigned int nItem);
```

Parameters

nItem

函数序号。函数序号按照 PGS 中定义的函数顺序排列，不区分是否测试。

Return Values

正常返回 0，失败返回-1。

Remarks

通过函数序号设置必须执行函数。此测试函数在选择测试的前提下，无论是否触发 failstop 都将执行。

如果设置的函数为 QA 函数，QA 执行，则函数必须执行。QA 不执行则函数不执行。

该接口函数实时生效。

Example

```
unsigned int nItem = 256;  
STSSetAlwaysRunFunctionItem(nItem);
```

1.5.10 STSSetAlwaysRunFunction ()

```
int USERTYPE STSSetAlwaysRunFunction(const char*pcData);
```

Parameters

pcData

函数名称。

Return Values

正常返回 0，失败返回-1。

Remarks

通过函数名称设置必须执行函数。所有同名函数都将被设置为必执行函数。

此测试函数在选择测试的前提下，无论是否触发 failstop 都将执行。

如果设置的函数为 QA 函数，QA 执行，则函数必须执行。QA 不执行则函数不执行。

该接口函数实时生效。

Example

```
char pcData[] = "AlwaysRunFunction";  
STSSetAlwaysRunFunction(pcData);
```

1.5.11 STSGetAlwaysRun ()

```
CAlwaysRun* STSGetAlwaysRun();
```

Parameters

Return Values

正常返回 CAlwaysRun*，失败返回 NULL。

Remarks

获取必执行函数的控制类。

数据结构定义：

```
class CAlwaysRun
```

```
{
```

```
public:
```

```
//清空所有通过函数名称设置的必执行函数
```

```
virtual int ClearFunctionName()=0;
```

```
//通过函数名称添加必执行函数
```

```
virtual int AddFunctionbyName(const char *)=0;
    //获取通过函数名称设置的必执行函数名称的个数
virtual unsigned int GetFunctionNameCnt()=0;
    //按照序号查看设置的必执行函数的名称
virtual const char *GetFunctionName(unsigned int nID)=0;
    //通过序号删除设置的必执行函数的名称
virtual int DeleteFunctionName(unsigned int nID)=0;

//清空通过函数序号设置的必执行函数
virtual int ClearFunctionIndex()=0;
    //通过函数序号添加必执行函数
virtual int AddFunctionbyIndex(unsigned int nIndex)=0;
    //获取通过函数序号设置的必执行函数的个数
virtual unsigned int GetFunctionIndexCnt()=0;
    //根据序号获取设置的必执行函数的序号
virtual unsigned int GetFunctionIndex(unsigned int nID)=0;
    //通过序号删除指定的必执行函数序号
virtual int DeleteFunctionIndex(unsigned int nID)=0;

//获取总的必执行函数的个数
virtual unsigned int GetRunFunctionCnt()=0;
    //根据序号获取设置的必执行函数序号
virtual unsigned int GetRunFunctionIndex(unsigned int nID)=0;
};
```

Example

```
CAlwaysRun* pAlways =STSGetAlwaysRun();
if (pAlways != NULL)
{
    //同名函数 Test_Fun5 必须执行
    pAlways->AddFunctionbyName("Test_Fun5");
    pAlways->AddFunctionbyIndex(9);
}

//查询设置的执行函数
if (pAlways != NULL)
{
```

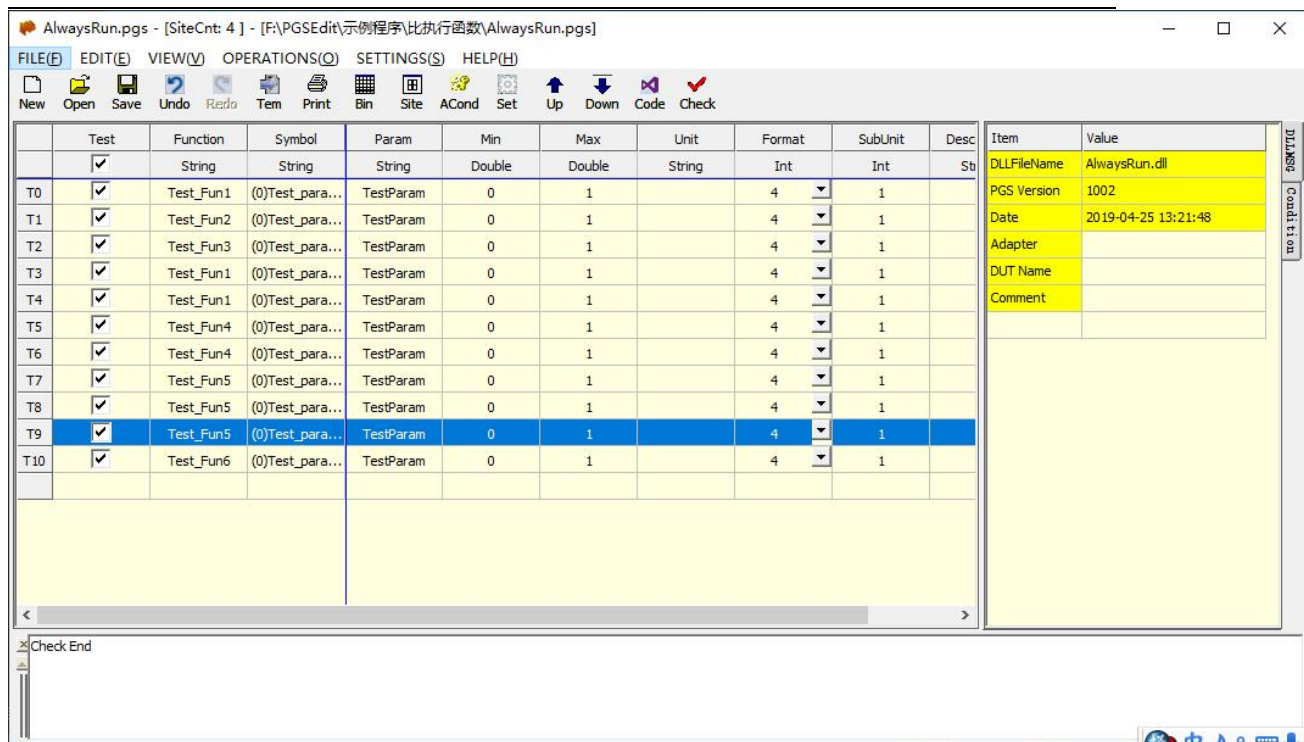
```
        unsigned int nCnt = pAlways->GetFunctionNameCnt();
        for (unsigned int i = 0; i < nCnt;i++)
        {
            const char * pcFunName = pAlways->GetFunctionName(i);
        }
        nCnt = pAlways->GetFunctionIndexCnt();
        for (unsigned int i = 0; i < nCnt;i++)
        {
            unsigned int nFunIndex = pAlways->GetFunctionIndex(i);
        }
    }

//设置代码
//第 6 个函数 Test_Fun4 必须执行
STSSetAlwaysRunFunctionItem(6);
//所有同名函数 Test_Fun1 必须执行
STSSetAlwaysRunFunction("Test_Fun1");

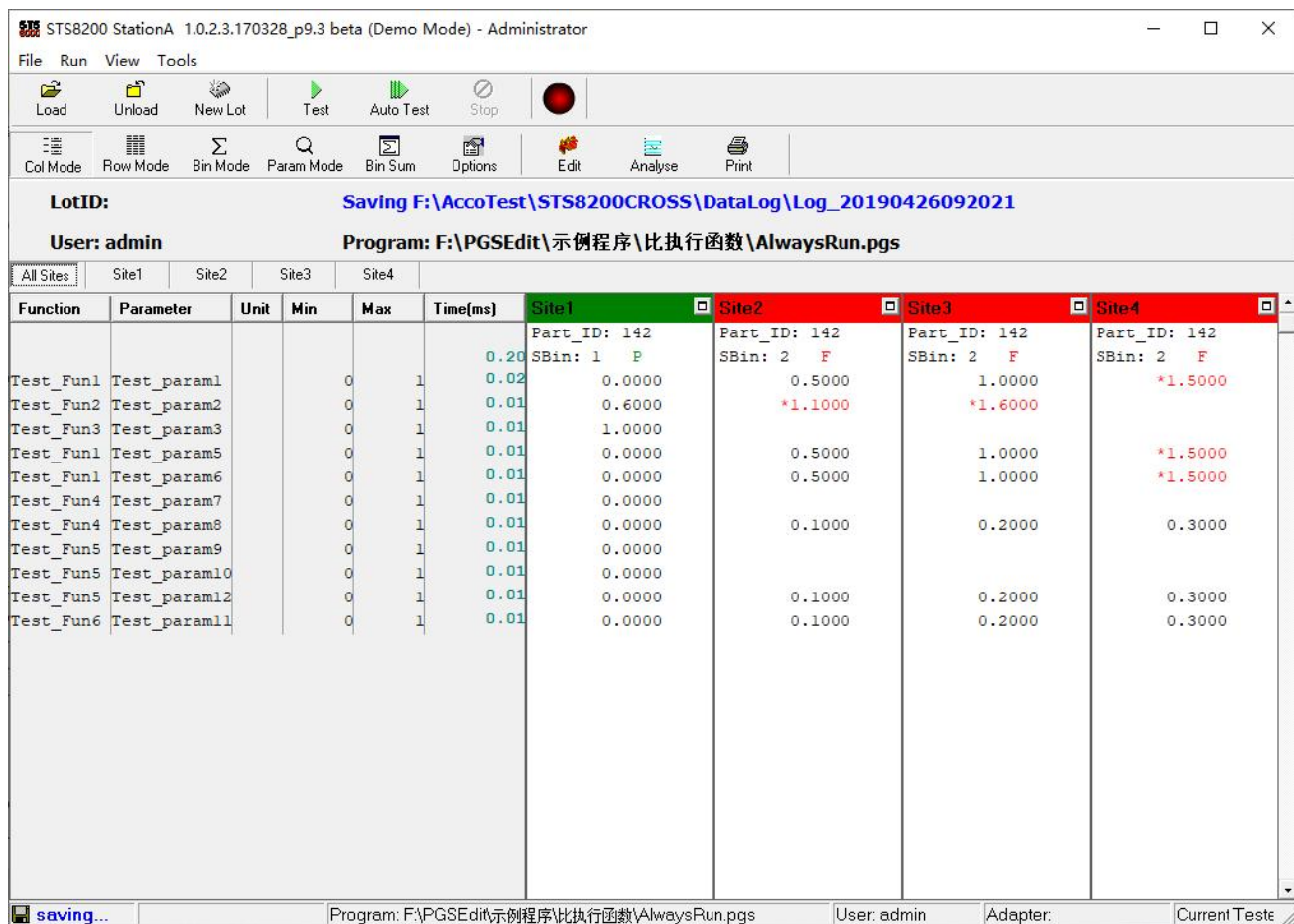
CAlwaysRun* pAlways =STSGetAlwaysRun();
if (pAlways != NULL)
{
    //同名函数 Test_Fun6 必须执行
    pAlways->AddFunctionbyName("Test_Fun6");
    pAlways->AddFunctionbyIndex(9);
}

//查询设置的执行函数
if (pAlways != NULL)
{
    unsigned int nCnt = pAlways->GetFunctionNameCnt();
    for (unsigned int i = 0; i < nCnt;i++)
    {
        const char * pcFunName = pAlways->GetFunctionName(i);
    }
    nCnt = pAlways->GetFunctionIndexCnt();
    for (unsigned int i = 0; i < nCnt;i++)
    {
        unsigned int nFunIndex = pAlways->GetFunctionIndex(i);
    }
}
```

PGS 截图:



测试效果截图:



1.5.12 STSSetFailStop ()

```
int USERTYPE STSSetFailStop(bool bStop);
```

Parameters

bStop

是否 FailStop。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 FailStop 功能。调用此函数后，界面上的 Failstop 设置无效。

Example

```
bool bStop = true;  
STSSetFailStop(bStop);
```

1.5.13 STSGetExecutedItemsCnt ()

```
int USERTYPE STSGetExecutedItemsCnt(int nSite);
```

Parameters

nSite

工位号，从 0 开始。

Return Values

已执行的测试项。

Remarks

获取该工位，当前已执行的测试参数个数。

此函数不支持多站测试程序的调用。

Example

```
int nSite=1;  
int nCnt = STSGetExecutedItemsCnt(nSite);
```

1.5.14 STSSetPartID ()

```
int USERTYPE STSSetPartID(  
int nSite,  
char *pcPartID);
```

Parameters

nSite

工位号。

pcPartID

器件 PartID，长度[1,254]。

Return Values

正常返回 0，失败返回-1。

Remarks

设置器件 PartID。

此函数不支持多站测试程序的调用。

Example

```
int nSite = 1;  
char pcPartID[]="Part1234";  
STSSetPartID(nSite, pcPartID);
```

1.5.15 STSGetTestItemsCnt ()

```
int USERTYPE STSGetTestItemsCnt();
```

Parameters

无

Return Values

需测试项个数。

Remarks

获取程序中需测试 QA 参数（不含系统生成的 DPAT 参数）的个数。

Example

```
int nCount=STSGeTestItemsCnt ();
```

1.5.16 STSGeQATestItemsCnt ()

```
int USERTYPE STSGeQATestItemsCnt ();
```

Parameters

无

Return Values

需测试项个数。

Remarks

获取程序中需测试 QA 参数的个数。

此函数不支持多站测试程序的调用。

Example

```
int nCount= STSGeQATestItemsCnt ();
```

1.5.17 STSGeSiteTestStatus ()

```
int USERTYPE STSGeSiteTestStatus (int nSite);
```

Parameters

nSite

工位号。

Return Values

返回 1，当前工位测试 pass。

返回 0 当前工位测试 fail。

返回-1 当前工位触发 Alarm。

返回-2 当前工位没有测试项执行。

Remarks

获取工位当前测试状态。实时获取，当前的测试状态。

此函数不支持多站测试程序的调用。

Example

```
int nSite = 1;  
Int nRet=STSGetSiteTestStatus (nSite);
```

1.5.18 STSNewLot ()

```
int STSNewLot (  
char *pcLotID,  
char *pcFileName);
```

Parameters

pcLotID

新批次 LotID。

pcFileName

存储文件文件名。

Return Values

0 正常

-1 文件命名不符合 Windows 标准

-2、-3 系统版本异常。

Remarks

新建批次操作，不显示批次信息输入框。此函数只能在 InitBeforeTestFlow 中使用。

Example

```
char  pplot[] = "lotid1";  
char  pcfile[] = "testdata_0100";  
STSTNewLot (pplot, pcfile);
```

1.5.19 STSGetDataLogFileName ()

```
int STSGetDataLogFileName (  
char *pcFileName,  
bool bext = false);
```

Parameters

pcFileName

数据文件名称。

bext

名称是否包含后缀，默认不包含。

Return Values

0 正常

-1 系统版本异常。

Remarks

获取存储文件名称。

Example

```
char  pcData[MAX_PATH] = {0};  
STSGetDataLogFileName (pcData,false);
```

1.5.20 STSGetLotInfo ()

```
int STSGetLotInfo(  
char *pcName,  
char *pcData,  
int nLen);
```

Parameters

pcName

Lot 项名称。

pcData

Lot 项值。

nLen

pcData 数据长度。

Return Values

0 正常

-1 未找到 lot 信息项。

Remarks

获取输入的 Lot 信息。

Example

```
char pcData[256] = {0};  
STSGetLotInfo("LotID", pcData, 256);
```

1.5.21 STSGetTesterID ()

```
int STSGetTesterID(  
char *pcData,  
int nLen);
```

Parameters

pcData

TesterID 项值。

nLen

pcData 数据长度。

Return Values

0 正常

-1 系统版本异常。

Remarks

获取 TesterID。

Example

```
char pcData[256] = {0};  
STSGetTesterID (,pcData,256);
```

1.5.22 StsGetParamBySymbol ()

```
CParam *StsGetParamBySymbol (  
LPCTSTR lpParamSymbol)
```

Parameters

lpParamSymbol

参数 Symbol。

Return Values

参数指针。

Remarks

通过参数 Symbol 获取参数指针。

Example

```
StsGetParamBySymbol("ICC");
```

1.5.23 STSGetParamCount ()

```
int STSGetParamCount(  
short funcIndex)
```

Parameters

funcIndex

函数序号。

Return Values

函数中参数的个数。

Remarks

获取函数中参数的个数。

Example

```
STSGetParamCount(0);
```

1.5.24 STSSetTestTimeFunction ()

```
int STSSetTestTimeFunction(  
bool bEnable,  
const char *pcFile)
```

Parameters

bEnable

是否启用。

pcFile

文件路径和名称。如不包含路径，文件存放在程序相对目录下。

Return Values

0 成功。

Remarks

设置是否启用测试时间记录功能。如果文件名称不为空，单次测试结束会

生成时间记录文件。自动测试不生成文件。

时间记录功能，在此函数调用之后的下一个测试函数生效。

文件中包含函数的执行时间与程序中调用的延时时间。

Example

```
STSSetTestTimeFunction (true,NULL);
```

1.5.25 STSGetFunTestTimebyIndex ()

```
int STSGetFunTestTimebyIndex(  
int nFunIndex,  
double &fTestTime,  
double &fDelayTime)
```

Parameters

nFunIndex

函数序号。

fTestTime

测试用时，单位（ms）

fDelayTime

测试过程中调用系统延时函数所产生的延时时间

Return Values

0 成功。

<0 失败

Remarks

获取单个函数的测试用时与函数中所用到的所有系统延时。在 STSSetTestTimeFunction 函数设置时间记录功能开启的状态有效。

Example

```
double fTestTime(0), fDelayTime(0);  
STSGetFunTestTimebyIndex (0, fTestTime, fDelayTime);
```

1.5.26 STSSetAllParamSave ()

```
int STSSetAllParamSave ()
```

Parameters

Return Values

0 成功。

<0 失败

Remarks

设置存储所有参数测试数据。

Example

```
STSSetAllParamSave ();
```

1.5.27 STSSetAllParamShow ()

```
int STSSetAllParamShow ()
```

Parameters

Return Values

0 成功。

<0 失败

Remarks

设置显示所有参数的测试数据。

Example

```
STSSetAllParamShow ();
```

1.5.28 STSSetQASampleMode ()

```
int STSSetQASampleMode (  
QASAMPLEMODE byMode)
```

Parameters

byMode

QA 测试模式。

```
enum QASAMPLEMODE  
{  
    PASSSAMPLE = 0,  
    TOUCHDOWNSAMPLE = 1,  
    TOUCHDOWNEACHSITESAMPLE = 2  
};
```

Return Values

0 成功。

<0 失败

Remarks

设置 QA 测试模式。

Example

```
STSSetQASampleMode(PASSSAMPLE);
```

1.5.29 STSUpdateParamLimit ()

```
int STSUpdateParamLimit(const char*pcsymbol,double fMin,double  
fMax) ;
```

Parameters

pcSymbol

需要更新判据的参数标识

fMin

待更新的判据下限，最大值为（1e15-1）最小值为(-1e15+1)。

fMax

待更新的判据上限，最大值为（1e15-1）最小值为(-1e15+1)。

Return Values

- 0 设置成功:
- 1, -4 系统错误
- 2 symbol 内容为空
- 3 min 或 max 不符合要求
- 5 没有找到参数

Remarks

修改参数的判据，并同步更新分档判据。

1. 接口只在 userload 起作用。
2. 此接口如果删除判据（判据值不在（-1e15+1,1e15-1）范围内）可能导致部分分档失效
3. 不支持有组合分档的程序调用

Example

```
STUpdateParamLimit("ParamSymbol",1,2,2.4);
```

1.5.30 STSGetSystemVersion ()

```
int STSGetSystemVersion(char*pcVer,int nLen);
```

Parameters

pcVer

软件版本号

nLen

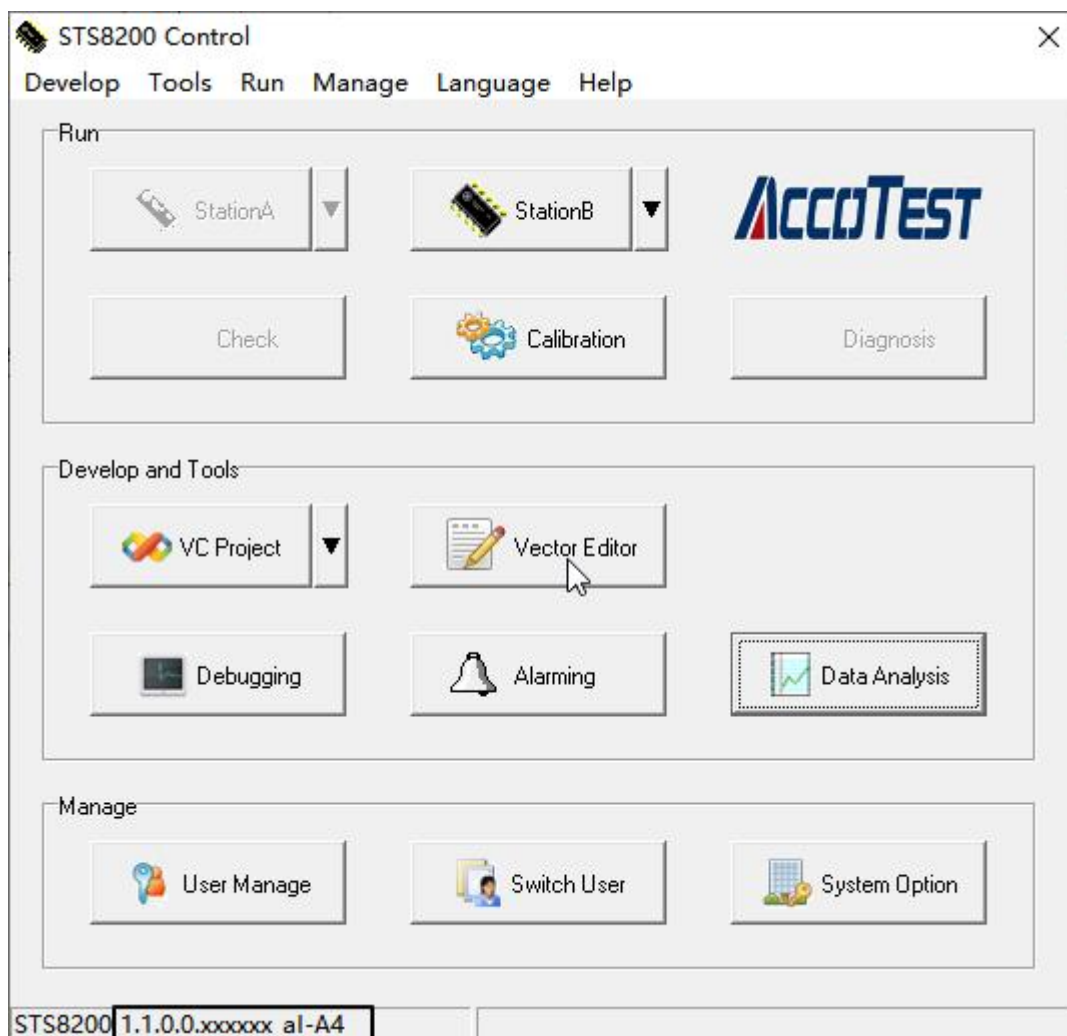
数据长度，如长度小于读取的数据则返回失败。

Return Values

读取数据长度

Remarks

读取软件版本信息，图中标注内容。



Example

```
char pcVer[256]={0};  
STSGetSystemVersion (pcVer ,256);
```

1.5.31 STSGetSystemName ()

```
int STSGetSystemName(char*pcName, int nLen);
```

Parameters

pcName

软件名称

nLen

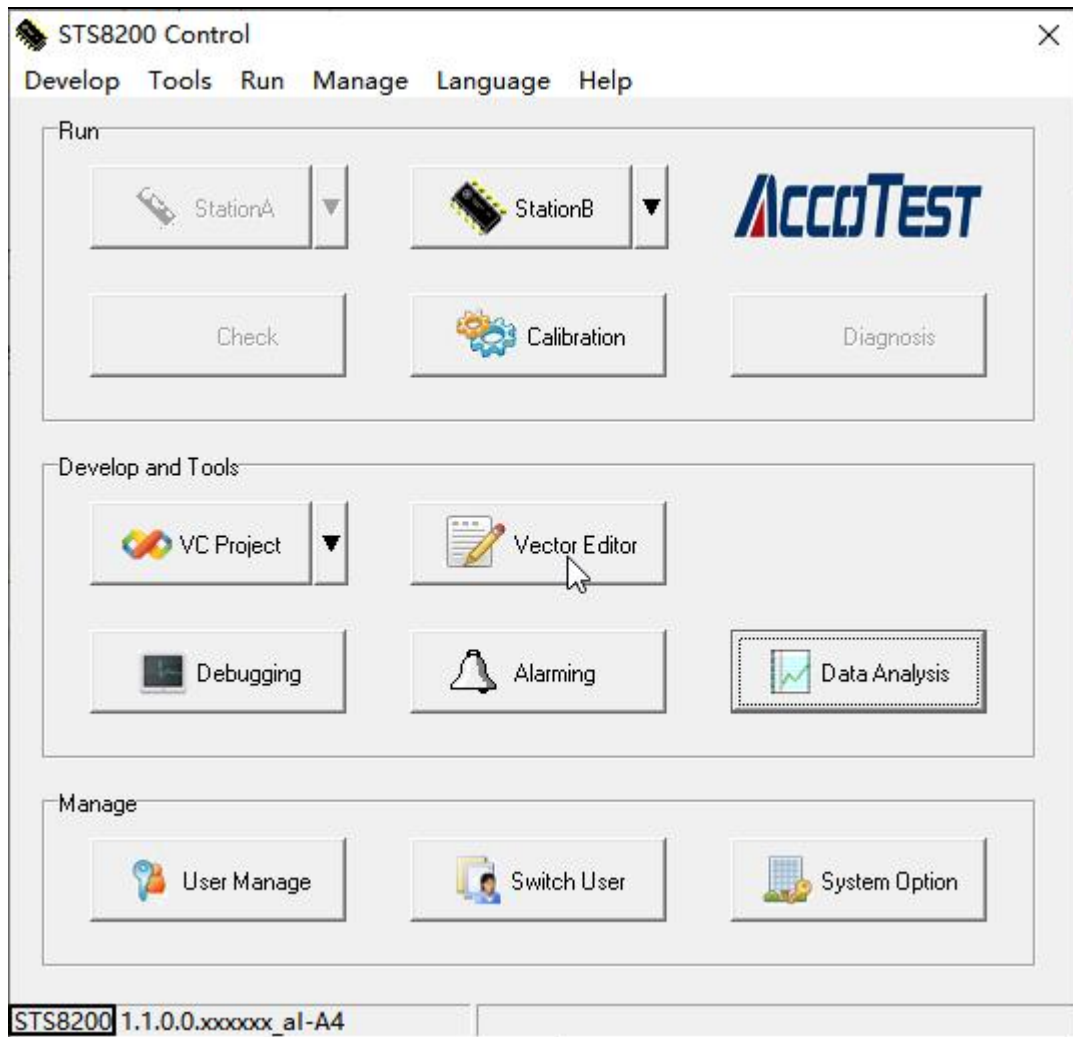
数据长度，如长度小于读取的数据则返回失败。

Return Values

0 读取数据长度

Remarks

读取软件版本名称，图中标注内容。



Example

```
char pcName[256]={0};
STSGetSystemName (pcName,256);
```

1.5.32 STSGetTHBInfo()

```
int STSGetTHBInfo(
THBInfo_Type type,
char* buf,
int bufsize);
```

Parameters

type

THB 参数类型

THBInfo_Productname	产品名称
THBInfo_Version	硬件版本
THBInfo_SoftVersion	软件版本
THBInfo_Engineer	工程师信息
THBInfo_Date	参数写入日期
THBInfo_Custominfo	自定义信息

buf

char 型变量，存储 THB 信息。由用户定义，如：char buf[256]={0}。

bufsize

数组 buf 的大小。

Return Values

返回 0：读取成功；

返回-1：读取失败。

Remarks

获取 THB 中存储的信息。

Example

例 1：获取 THB 中存储的软件版本信息。

```
char buf[256] = {0};  
STSGetTHBInfo(THBInfo_SoftVersion, buf, 256);
```

1.5.33 STSGetQAStartFunctionIndex()

```
int STSGetQAStartFunctionIndex ();
```

Parameters

Return Values

>0: QA 测试的开始函数序号 (PGS 中定义的函数顺序, 从 0 开始)

<=0: 非 QA 程序。

Remarks

获取 inline QA 程序中, QA 测试的开始函数。

Example

```
int nQAIndex = STSGetQAStartFunctionIndex();
```

1.5.34 STSGetISQAFile ()

```
int STSGetISQAFile();
```

Return Values

1: Inline QA 程序

0: 非 InLine QA 程序。

Remarks

获取是否是 InlineQA 程序。

Example

```
int nQAFile = STSGetQAStartFunctionIndex();  
if(nQAFile==1)  
{
```

1.5.35 STSGetEnableQA()

```
int STSGetEnableQA();
```

Return Values

1: 启用 QA 测试

0: 未启用 QA 测试。

Remarks

获取是否启用 InlineQA 测试。

Example

```
int nQAEEnable = STSGetEnableQA ();  
if(nQAEEnable==1)  
{
```

1.5.36 STSGetQASettings ()

```
int STSGetQASettings (QASAMPLEMODE &nMode, int &nTotal, int &nFreq);
```

Parameters

nMode

QA 测试的模式，参考 QASAMPLEMODE

nTotal

QA 采样总数。

nFreq

QA 采样频率。

Return Values

0: 未启用 QA 测试。

1: 启用 QA 测试

Remarks

获取 InlineQA 测试抽样模式、总数和频率。

Example

```
QASAMPLEMODE nMode= PASSSAMPLE;  
int nTotal=0;  
int nFreq=0;  
STSGetQASettings (nMode, nTotal, nFreq);
```

1.5.37 STSGetIsQAFunction()

```
int STSGetIsQAFunction(int nFuntindex);
```

Parameters

nFuntindex

函数序号，从 0 开始

Return Values

0: 非 QA 函数。

1: QA 函数

Remarks

获取是否是 QA 函数。

Example

```
int nQA= STSGetIsQAFunction(10);
if(nQA==1)
{
}
```

1.5.38 STSSetRelayActTimes()

```
int STSSetRelayActTimes (
THB_MODULE_TYPE type,
int relay1Times,
int relay2Times = -1,
int relay3Times = -1,
int relay4Times = -1);
```

Parameters

type

THB 模块类型

RELAY_LIFE_STS8300_THB 产 8300 系统的 THB

RELAY_LIFE_PIM_AC_DIB PIM 系统的 AC-DIB

RELAY_LIFE_PIM_DC_DIB PIM 系统的 DC-DIB

RELAY_LIFE_PIM_THB PIM 系统的 THB

relay1Times

int 型变量，存储对应模块第一个继电器单次测试的动作次数，取值范围 0 到 65535。

relay2Times

int 型变量，存储对应模块第二个继电器单次测试的动作次数，取值范围 0 到 65535，默认值为-1，-1 表示不存在该继电器信息。

relay3Times

int 型变量，存储对应模块第三个继电器单次测试的动作次数，取值范围 0 到 65535，默认值为-1，-1 表示不存在该继电器信息。

relay4Times

int 型变量，存储对应模块第四个继电器单次测试的动作次数，取值范围 0 到 65535，默认值为-1，-1 表示不存在该继电器信息。

Return Values

返回 0：设置成功；

返回-1：所选模块未成功回读 eeprom；

返回-2：参数设置错误，继电器数量与记录情况不匹配；

返回-3：参数设置错误，继电器单次测试动作次数超过有效范围；

返回-4：参数调用位置有误，应在 UserLoad 函数中调用；

Remarks

设置每次测试继电器的动作次数，优先级高于 eeprom 记录的值，只能在测试程序的 UserLoad 函数里调用。

Example

例 1：设置 PIM 系统 DC-DIB 上记录的继电器动作次数。

```
STSSetRelayActTimes (RELAY_LIFE_PIM_DC_DIB, 1000, 850);
```

1.6 STDF 存储信息设置函数

1.6.1 STDF 数据类型

```
enum STD_TYPE
{
    MIR_MODE_COD = 1000,
    MIR_RTST_COD,
    MIR_PROT_COD,
    MIR_BURN_TIM,
    MIR_CMOD_COD,
    MIR_LOT_ID,
    MIR_PART_TYP,
    MIR_NODE_NAM,
    MIR_TSTR_TYP,
    MIR_JOB_NAM,
    MIR_JOB_REV,
    MIR_SBLLOT_ID,
    MIR_OPER_NAM,
    MIR_EXEC_TYP,
    MIR_EXEC_VER,
    MIR_TEST_COD,
    MIR_TST_TEMP,
    MIR_USER_TXT,
    MIR_AUX_FILE,
    MIR_PKG_TYP,
    MIR_FAMLY_ID,
    MIR_DATE_COD,
```

MIR_FACIL_ID,
MIR_FLOOR_ID,
MIR_PROC_ID,
MIR_OPER_FRQ,
MIR_SPEC_NAM,
MIR_SPEC_VER,
MIR_FLOW_ID,
MIR_SETUP_ID,
MIR_DSGN_REV,
MIR_ENG_ID,
MIR_ROM_COD,
MIR_SERL_NUM,
MIR_SUPR_NAM,
WRR_WAFER_ID = 2000,
WRR_FABWF_ID,
WRR_FRAME_ID,
WRR_MASK_ID,
WRR_USR_DESC,
WRR_EXC_DESC,
WCR_WAFR_SIZ = 3000,
WCR_DIE_HT,
WCR_DIE_WID,
WCR_WF_UNITS,
WCR_WF_FLAT,
WCR_CENTER_X,
WCR_CENTER_Y,
WCR_POS_X,
WCR_POS_Y,

```
SDR_HAND_TYP = 4000,  
SDR_HAND_ID,  
SDR_CARD_TYP,  
SDR_CARD_ID,  
SDR_LOAD_TYP,  
SDR_LOAD_ID,  
SDR_DIB_TYP,  
SDR_DIB_ID,  
SDR_CABL_TYP,  
SDR_CABL_ID,  
SDR_CONT_TYP,  
SDR_CONT_ID,  
SDR_LASR_TYP,  
SDR_LASR_ID,  
SDR_EXTR_TYP,  
SDR_EXTR_ID,  
};
```

1.6.2 STSCBSetSTDInfo()

```
int USERTYPE STSCBSetSTDInfo (  
STD_TYPE std_type,  
char *pcData,  
char *pcLable = NULL);
```

Parameters

std_type

STDF 中定义的字段。字段名称由 STD 中结构和字段名称两部分组成，中

间有下划线间隔，所有字母大写。例如 MIR 结构中的 JOB_NAME 字段，输入为 MIR_JOB_NAME。

pcData

设置内容，最大长度 256。

pcLable

自定义名称。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 STDF 存储信息。如果设置的 STDF 信息在上层 LOT 信息项中已存在，则忽略。

Example

```
char pcData[] = "JOB_NAM";  
STSCBSetSTDInfo (MIR_JOB_NAM, pcData);
```

1.6.3 STSCBSetAndUpdateSTDInfo()

```
Int USERTYPE STSCBSetAndUpdateSTDInfo(  
STD_TYPE std_type,  
char *pcData,  
char *pcLable = NULL);
```

Parameters

std_type

STDF 中定义的字段。字段名称由 STD 中结构和字段名称两部分组成，中间有下划线间隔，所有字母大写。例如 MIR 结构中的 JOB_NAME 字段，输入为 MIR_JOB_NAME。

pcData

设置内容，最大长度 256。

pcLable

自定义名称。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 STDF 存储信息，如果设置的 STDF 信息在上层 LOT 信息项中已存在，则更新显示内容。

Example

```
char pcData[] = "JOB_NAM";  
STSCBSetAndUpdateSTDInfo (MIR_JOB_NAM, pcData);
```

1.6.4 STSCBSetGDRInfo()

```
int STSCBSetGDRInfo (unsigned char FieldType, char *FieldName,  
char *FieldValue);
```

Parameters

FieldType

类型。

FieldName

名称。

FieldValue

值。

Return Values

0 成功，其他失败

Remarks

设置 STDF 中 GDR 信息。内容将保存到 STDF 文件中的 GDR 格式中。

Example

```
STSCBSetGDRInfo(0, "TestGDR", "GDRValeu1");
```

1.6.5 STSCBGetSTDInfo()

```
int STSCBGetSTDInfo(STD_TYPE std_type, char pcData[256], char pcLable[256]);
```

Parameters

std_type

数据类型，详见 STD_TYPE。

pcData[256]

数据值。

pcLable[256]

测试系统中此项的标识。

Return Values

0 成功，其他失败

Remarks

获取系统中存储的 STDF，各个字段的内容。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

只能获取批次信息界面上添加的内容。

Example

```
char pcData[256]={0};  
char pcLable[256]={0};  
STSCBGetSTDInfo(MIR_JOB_NAM, pcData, pcLable);
```

1.7 良率卡控功能（SYL）函数

SYL 功能，程序中设置的优先级最高。

1.7.1 STSCBSetSYLDiffData()

```
int STSCBSetSYLDiffData(int nEnableFlag, int nSampleCount, double  
fcurDiffLimit, int nClearCount);
```

Parameters

nEnableFlag

是否开启工位良率差卡控。 0：不开启 1：开启

nSampleCount

各工位最小采样数。

fcurDiffLimit

良率差卡控值。超过此值卡控停机。

nClearCount

卡控停机后是否清除已有统计值。0：不清除 1：清除

Return Values

0：正常 -1：设置失败 -2：设置值不正确，或者良率大于 100，或者采样数小于 1。

Remarks

设置工位良率差卡控内容。

Example

```
STSCBSetSYLDiffData (1,100,5.8, 1);
```

1.7.2 STSCBGetSYLDiffData()

```
int STSCBGetSYLDiffData(int &nEnableFlag, int &nSampleCount, double
```



```
&fcurlDiffLimit, int &nClearCount);
```

Parameters

nEnableFlag

是否开启工位良率差卡控。 0：不开启 1：开启

nSampleCount

各工位最小采样数。

fcurlDiffLimit

良率差卡控值。超过此值卡控停机。

nClearCount

卡控停机后是否清除已有统计值。0：不清除 1：清除

Return Values

正常返回 0，失败返回-1。

Remarks

获取工位良率差卡控设置内容。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nEnableFlag=0;
int nSampleCount=0;
double fcurlDiffLimit=0;
int nClearCount=0;
STSCBGetSYLDiffData (nEnableFlag, nSampleCount, fcurlDiffLimit, nClearCount);
```

1.7.3 STSCBSetContinueFailStopSetting()

```
int STSCBSetContinueFailStopSetting(int nEnableFlag, int
nContinueFailStopCount);
```

Parameters

nEnableFlag

是否开启连续失效停机卡控，此功能针对各分工位。 0：不开启 1：开

启

nContinueFailStopCount

各工位连续失效数。

Return Values

正常返回 0，失败返回-1。

Remarks

设置连续失效停机卡控。

Example

```
STSCBSetContinueFailStopSetting (1, 200);
```

1.7.4 STSCBGetContinueFailStopSetting()

```
int STSCBGetContinueFailStopSetting(int &nEnableFlag, int  
&nContinueFailStopCount);
```

Parameters

nEnableFlag

是否开启连续失效停机卡控，此功能针对各分工位。 0：不开启 1：开

启

nContinueFailStopCount

各工位连续失效数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取连续失效停机卡控设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nEnableFlag=0;  
int nContinueFailStopCount=0;  
STSCBGetContinueFailStopSetting (nEnableFlag, nContinueFailStopCount);
```

1.7.5 STSCBSetContinuePassStopSetting()

```
int STSCBSetContinuePassStopSetting(int nEnableFlag, int  
nContinuePassStopCount);
```

Parameters

nEnableFlag

是否开启连续合格停机卡控，此功能针对各分工位。 0：不开启 1：开启

nContinuePassStopCount

各工位连续合格数。

Return Values

正常返回 0，失败返回-1。

Remarks

设置连续合格停机卡控。

Example

```
STSCBSetContinuePassStopSetting (1, 200);
```

1.7.6 STSCBGetContinuePassStopSetting()

```
int STSCBGetContinuePassStopSetting(int &nEnableFlag, int  
&nContinuePassStopCount);
```

Parameters

nEnableFlag

是否开启连续合格停机卡控，此功能针对各分工位。 0：不开启 1：开启

nContinuePassStopCount

各工位连续合格数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取连续合格停机卡控设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nEnableFlag=0;
int nContinuePassStopCount=0;
STSCBGetContinuePassStopSetting (nEnableFlag, nContinuePassStopCount);
```

1.7.7 STSCBGetRecentSiteSYL()

```
int STSCBGetRecentSiteSYL(int nSite, double &fYield);
```

Parameters

nSite

工位号（从 0 开始）

fYield

当前工位最近良率。

Return Values

正常返回 0，失败返回-1。

Remarks

获取当前工位最近良率，超过采样数才有返回值。

Example

```
int nSite=1;
double fYield=0;
STSCBGetSiteSYL(nSite, fYield);
```

1.7.8 STSCBGetRecentSYLinfo()

```
int STSCBGetRecentSYLinfo(int &nEnableFlag, int &nSample, double  
&fYield);
```

Parameters

nEnableFlag

是否开启最近良率停机卡控。 0: 不开启 1: 开启

nSample

各工位最小采样数。

fYield

卡控良率。

Return Values

正常返回 0，失败返回-1。

Remarks

获取最近良率停机卡控设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nEnableFlag=0;  
int nSample=0;  
double fYield=0;  
STSCBGetSYLinfo(nEnableFlag, nSample, fYield);
```

1.7.9 STSCBSetRecentSYLinfo()

```
int STSCBSetRecentSYLinfo(int nEnableFlag, int nSample, double  
fYield);
```

Parameters

nEnableFlag

是否开启最近良率停机卡控。 0：不开启 1：开启

nSample

各工位最小采样数。

fYield

卡控良率。

Return Values

0：正常 -1：设置失败 -2：设置值不正确，或者良率大于 100，或者采样数小于 1。

Remarks

设置最近良率停机卡控。

Example

```
STSCBSetSYInfo (1, 200, 0.5);
```

1.7.10 STSCBGetSiteDUTCount()

```
int STSCBGetSiteDUTCount(int nSite, unsigned int &nTotal, unsigned  
int &nPass, unsigned int &nFail);
```

Parameters

nSite

工位号（从 0 开始）

nTotal

当前工位测试器件总数。

nPass

合格数。

nFail

失效数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取当前工位测试统计信息：测试总数、合格数、失效数

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSite=1;
unsigned int nTotal=0;
unsigned int nPass=0;
unsigned int nFail=0;
STSCBGetSiteDUTCount(nSite, nTotal, nPass, nFail);
```

1.8 SBL 分档卡控功能应用函数

SBL、HBL 功能，程序中设置的优先级最高。

1.8.1 STSCBGetSBLSetting()

```
int STSCBGetSBLSetting(int &nEnableFlag, int &nSampleCount, int
&nClearCount);
```

Parameters

nEnableFlag

是否开启分档卡控。 0：不开启 1：开启

nSampleCount

最小采样数。

nClearCount

卡控停机后是否清除已有统计值。0：不清除 1：清除

Return Values

正常返回 0，失败返回-1。

Remarks

获取分档卡控设置

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nEnableFlag = 0;
int nSampleCount = 0;
int nClearCount = 0;
STSCBGetSBLSetting (nEnableFlag, nSampleCount, nClearCount);
```

1.8.2 STSCBSetSBLSetting()

```
int STSCBSetSBLSetting(int nEnableFlag, int nSampleCount, int
nClearCount);
```

Parameters

nEnableFlag

是否开启分档卡控。 0：不开启 1：开启

nSampleCount

最小采样数。

nClearCount

卡控停机后是否清除已有统计值。0：不清除 1：清除

Return Values

正常返回 0，失败返回-1。

Remarks

设置分档卡控。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSBLSetting (1, 200, 1);
```

1.8.3 STSCBSBLEnable()


```
int STSCBSBLENable(int nEnableFlag);
```

Parameters

nEnableFlag

是否启用 SBL 功能，0 不使用 1 使用。

Return Values

正常返回 0，失败返回-1。

Remarks

开启 SBL 功能设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSBLENable(1);
```

1.8.4 STSCBSBLSampleCount()

```
int STSCBSBLSampleCount(int nSampleCount);
```

Parameters

nSampleCount

Sample 颗器件之后，进行良率卡控。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SBL 功能采样数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSBLSampleCount (100);
```

1.8.5 STSCBSBLClearCountFlag()

```
int STSCBSBLClearCountFlag(int nClearCountFlag);
```

Parameters

nClearCountFlag

发生良率报警之后，是否清除之前的统计 0 不清除，1 清除。

Return Values

正常返回 0，失败返回-1。

Remarks

清除 SBL 功能统计。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSBLClearCountFlag (1);
```

1.8.6 STSCBSBSetSBLData()

```
int STSCBSBSetSBLData(int nIndex, int nsbID, double fcurLimit);
```

Parameters

nIndex

分档卡控类型。 0：软分档 1：硬分档

nsbID

分档号

fcurLimit

良率卡控值。超过此值卡控停机。

Return Values

正常返回 0，失败返回-1。

Remarks

设置分档良率卡控内容

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSBLData (0, 1, 10);  
STSCBSetSBLData (1, 1, 10);
```

1.8.7 STSCBGetSBLCount()

```
int STSCBGetSBLCount (int &nCount);
```

Parameters

nCount

项数

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 卡控内容软分档项数

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nCount = 0;  
STSCBGetSBLCount (nCount);
```

1.8.8 STSCBGetSBLInfo()

```
int STSCBGetSBLInfo (int nSite, int nSBID, double &fcurLimit,
```

```
double &factYield);
```

Parameters

nSite

工位号（从 0 开始）

nSBID

软分档号。

fcurLimit

良率卡控值。超过此值卡控停机。

factYield

当前工位当前软分档的统计良率。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 卡控软分档统计信息

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSite = 0;
int nSBID = 1;
double fcurLimit = 0;
double factYield = 0;
STSCBGetSBLInfo(nSite, nSBID, fcurLimit, factYield);
```

1.8.9 STSCBGetHBLCount()

```
int STSCBGetHBLCount(int &nCount);
```

Parameters

nCount

项数

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 卡控内容硬分档项数

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nCount = 0;  
STSCBGetHBLCount (nCount);
```

1.8.10 STSCBGetHBLInfo()

```
int STSCBGetHBLInfo(int nSite, int nHBID, double &fcurLimit,  
double &factYield);
```

Parameters

nSite

工位号（从 0 开始）

nHBID

硬分档号。

fcurLimit

良率卡控值。超过此值卡控停机。

factYield

当前工位当前硬分档的统计良率。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 卡控硬分档统计信息

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSite = 0;
int nHBID = 1;
double fcurLimit = 0;
double factYield = 0;
STSCBGetHBLInfo(nSite, nHBID, fcurLimit, factYield);
```

1.8.11 STSCBSetSBLDiffData()

```
int STSCBSetSBLDiffData(int nIndex, int nsbID, double fcurDiffLimit);
```

Parameters

nIndex

0 表示 SBIN，1 表示 HBIN。

nsbID

要设置的软 Bin 或硬 Bin 号。

fcurDiffLimit

SBL 或 HBL 良率差设置值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SBL 或 HBL 良率差卡控值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSBLDiffData(0, 3, 0.5);
STSCBSetSBLDiffData(1, 4, 0.5);
```

1.8.12 STSCBGetSBLDiffCount()

```
int STSCBGetSBLDiffCount(int &nCount);
```

Parameters

nCount

SBL 软分 BIN 良率差设置个数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 软分 BIN 良率差设置个数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nCount = 0;  
STSCBGetSBLDiffCount (nCount);
```

1.8.13 STSCBGetSBLDiffinfo()

```
int STSCBGetSBLDiffinfo(int nSBID, double &fcurDiffLimit);
```

Parameters

nSBID

软 Bin 号。

fcurDiffLimit

SBL 软 Bin 良率差卡控设置值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 软 Bin 良率差卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSBID = 1;  
double fcurDiffLimit = 0;  
STSCBGetSBLDiffInfo (nSBID, fcurDiffLimit);
```

1.8.14 STSCBGetHBLDiffCount()

```
int STSCBGetHBLDiffCount (int &nCount);
```

Parameters

nCount

SBL 硬分 BIN 良率差设置个数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 硬分 BIN 良率差设置个数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nCount = 0;  
STSCBGetHBLDiffCount (nCount);
```

1.8.15 STSCBGetHBLDiffInfo()


```
int STSCBGetHBLDiffinfo(int nHBID, double &fcurlDiffLimit);
```

Parameters

nHBID

设置的硬 Bin 号。

fcurlDiffLimit

SBL 硬 Bin 良率差卡控设置值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 硬 Bin 良率差卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nHBID = 1;  
double fcurlDiffLimit = 0;  
STSCBGetHBLDiffinfo (nHBID, fcurlDiffLimit);
```

1.8.16 STSCBGetSBLYieldInfo()

```
int STSCBGetSBLYieldInfo(int nSite, int nSBID, int &AllCount, int  
&nBinCount);
```

Parameters

nSite

工位号（从 0 开始）

nSBID

软分 Bin 号。

AllCount

采样器件统计总数。

PassCount

对应分 Bin 统计数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 软分 Bin 统计信息。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSite = 0;
int nSBID = 1;
int AllCount = 0;
int nBinCount = 0;
STSCBGetSBLYieldInfo(nSite, nSBID, AllCount, nBinCount);
```

1.8.17 STSCBGetHBLYieldInfo()

```
int STSCBGetHBLYieldInfo (int nSite, int nHBID, int &AllCount, int
&nBinCount);
```

Parameters

nSite

工位号（从 0 开始）

nSBID

硬分 Bin 号。

AllCount

采样器件统计总数。

nBinCount

对应分 Bin 统计数。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SBL 硬分 Bin 统计信息。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int nSite = 0;
int nHBID = 1;
int AllCount = 0;
int nBinCount = 0;
STSCBGetHBLInfo(nSite, nHBID, AllCount, nBinCount);
```

1.8.18 STSCBClearSBLContent()

```
int STSCBClearSBLContent (int nIndex, int nsbID, int ClearSetFlag);
```

Parameters

nIndex

0 表示 SBIN，1 表示 HBIN。

nsbID

要设置的软 Bin 或硬 Bin 号。

ClearSetFlag

SBL 设置清除标志。0：分 Bin 良率卡控 1：分 Bin 良率差卡控

Return Values

正常返回 0，失败返回-1。

Remarks

清除 SBL 或 HBL 卡控设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBClearSBLContent (0, 1, 0);
STSCBClearSBLContent (1, 1, 0);
STSCBClearSBLContent (0, 1, 1);
STSCBClearSBLContent (1, 1, 1);
```

1.9 参数卡控功能（SPL）编程接口函数

- 1、SPL 功能，程序中设置的优先级最高。
- 2、下面所有参数卡控接口函数中的参数名是指 TestUI 窗口左测的参数名，此值对应 PGESEEDIT 中的 symbol 名。

All Sites	Site1	Site2	Site3	Site4	
Parameter	Unit	Min	Max	Time(ms)	Site1 ☐
				0.00	
RMS	V	1.3	1.6	0.00	
THD	dB		-70	0.00	
THD1	dB		-70	0.00	
SNR1	dB	65		0.00	
SINAD1	dB	65		0.00	
THD_Blackman	dB		-70	0.00	

1.9.1 STSCBSPLEnable()

```
int STSCBSPLEnable(int nEnableFlag);
```

Parameters

nEnableFlag

是否启用 SPL 功能，0 不使用 1 使用。

Return Values

正常返回 0，失败返回-1。

Remarks

开启 SPL 功能。

Example

```
STSCBSPLEnable(1);
```

1.9.2 STSCBSetSPLClearSetting()

```
int STSCBSetSPLClearSetting(bool nClearCount);
```

Parameters

nClearCountFlag

发生良率报警之后，是否清除之前的统计。 0：不清除，1：清除。

Return Values

正常返回 0，失败返回-1。

Remarks

清除 SPL 功能统计。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLClearSetting(1);
```

1.9.3 STSCBSetSPLSetting()

```
int STSCBSetSPLSetting(int nEnableFlag, int nClearCount);
```

Parameters

nEnableFlag

是否启用 SBL 功能，0 不使用 1 使用。

nClear

发生良率报警之后，是否清除之前的统计 0 不清除，1 清除。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 开启功能。

Example

```
STSCBSetSPLSetting (1,1);
```

1.9.4 STSCBGetSPLSetting()

```
int STSCBGetSPLSetting(int &nEnableFlag, int &nClearCount);
```

Parameters

nEnableFlag

是否启用 SPL 功能，0 不使用 1 使用。

nClear

发生 SPL 报警之后，是否清除之前的统计 0 不清除，1 清除。

Return Values

正常返回 0，失败返回-1。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Remarks

获取 SPL 功能设置。

```
Example  
int nEnableFlag;  
int nClearCount;  
STSCBGetSPLSetting (nEnableFlag, nClearCount);
```

1.9.5 STSCBSetSPLApplySampleMode()

```
int STSCBSetSPLApplySampleMode(int curSampleMode);
```

Parameters

curSampleMode

在某个工位 SPL 测试数达到最小采样数时，SPL 卡控的工位应用范围

0: 卡控所有分工位，

1: 仅卡控 SPL 测试数达到最小采样数的分工位。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 卡控的工位应用范围。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLApplySampleMode (1);
```

1.9.6 STSCBSetSPLYieldSampleCount()

```
int STSCBSetSPLYieldSampleCount(int curSampleCount, int  
OnlyTotalSite);
```

Parameters

curSampleCount

SPL 良率卡控最小采样数。

OnlyTotalSite

SPL 良率卡控最小采样数是否按总工位统计。0:单工位 1: 总工位

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 良率卡控最小采样数的工位应用范围。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLYieldSampleCount (100, 1);
```

1.9.7 STSCBSetSPLYieldData()

```
int STSCBSetSPLYieldData(char* nParamID,int SubUnit, double  
fcurYieldLimit);
```

Parameters

nParamID

SPL 良率卡控参数名称。

SubUnit

SPL 良率卡控子单元号

fcurYieldLimit

SPL 良率卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 良率卡控值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLYieldData (nParamID , 0, 0.9);
```

1.9.8 STSCBGetSPLYieldInfo()

```
int STSCBGetSPLYieldInfo(char* nParamID,int SubUnit, double  
&fcurYieldLimit);
```

Parameters

nParamID

SPL 良率卡控参数名称。

SubUnit

SPL 良率卡控子单元号

fcurYieldLimit

SPL 良率卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SPL 良率卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double nYieldLimit;  
STSCBGetSPLYieldInfo (nParamID ,0, nYieldLimit);
```

下面是参数良率差异卡控

1.9.9 STSCBSetSPLYieldDiffSampleCount()

```
int STSCBSetSPLYieldDiffSampleCount(int curSampleCount);
```

Parameters

curSampleCount

SPL 良率差卡控最小采样数。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 良率差卡控最小采样数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLYieldDiffSampleCount (100);
```

1.9.10 STSCBSetSPLYieldDiffData()

```
int STSCBSetSPLYieldDiffData(char* nParamID, int SubUnit, double  
fcurYieldDIFFLimit);
```

Parameters

nParamID

SPL 良率差卡控参数名称。

SubUnit

SPL 良率差卡控子单元号

fcurYieldDIFFLimit

SPL 良率差卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 良率差卡控值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLYieldDiffSampleCount (nParamID , 0, 1.2);
```

1.9.11 STSCBGetSPLYieldDiffInfo()

```
int STSCBGetSPLYieldDiffInfo(char* nParamID, int SubUnit, double  
&fcurYieldDIFFLimit);
```

Parameters

nParamID

SPL 良率差卡控参数名称。

SubUnit

SPL 良率差卡控子单元号

fcurYieldDIFFLimit

SPL 良率差卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SPL 良率差卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double curYieldDIFFLimit;  
STSCBGetSPLYieldDiffInfo (nParamID , 0, curYieldDIFFLimit);
```

下面是参数平均值差异卡控

1.9.12 STSCBSetSPLAverageDiffSampleCount()

```
int STSCBSetSPLAverageDiffSampleCount(int curSampleCount);
```

Parameters

curSampleCount

SPL 平均值差异卡控最小采样数。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 合格器件平均值差异卡控最小采样数。

Example

```
STSCBSetSPLAverageDiffSampleCount (120);
```

1.9.13 STSCBSetSPLAverageDiffData()

```
int STSCBSetSPLAverageDiffData(char* nParamID, int SubUnit, double  
fcurAverageDiffLimit);
```

Parameters

nParamID

SPL 平均值差异卡控参数名称。

SubUnit

SPL 平均值差异卡控子单元号

fcurAverageDiffLimit

SPL 平均值差异卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 合格器件平均值差异卡控值。参数各工位平均值差超限停机功能：计算此参数所有合格器件的测试值的平均值，然后比较各工位之间的平均值的差，如果平均值的差值超过设置卡控值，报警停机。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLAverageDiffData (nParamID, 0, 1.2);
```

1.9.14 STSCBGetSPLAverageDiffInfo()

```
int STSCBGetSPLAverageDiffInfo(char* nParamID, int SubUnit, double  
&fcurAverageDiffLimit);
```

Parameters

nParamID

SPL 平均值差异卡控参数名称。

SubUnit

SPL 平均值差异卡控子单元号

fcurAverageDiffLimit

SPL 平均值差异卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SPL 合格器件平均值差异卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double fcurAverageDiffLimit=0;  
STSCBGetSPLAverageDiffInfo (nParamID , 0, fcurAverageDiffLimit);
```

下面是参数平均值范围卡控

1.9.15 STSCBSetSPLAverageRangeSampleCount()

```
int STSCBSetSPLAverageRangeSampleCount(int curSampleCount, int  
OnlyTotalSite);
```

Parameters

curSampleCount

SPL 平均值范围卡控最小采样数。

OnlyTotalSite

SPL 平均值范围卡控最小采样数是否按总工位统计。0:单工位 1: 总工位

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 合格器件平均值范围卡控最小采样数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLAverageRangeSampleCount (150,1);
```

1.9.16 STSCBSetSPLAverageRangeData()

```
int STSCBSetSPLAverageRangeData(char* nParamID, int SubUnit, double  
fcurAverageMinLimit, double fcurAverageMaxLimit, int LimitFlag);
```

Parameters

nParamID

SPL 平均值范围卡控参数名称。

SubUnit

SPL 平均值范围卡控子单元号

fcurAverageMinLimit

SPL 平均值范围卡控最小值。

fcurAverageMaxLimit

SPL 平均值范围卡控最大值。

LimitFlag

此参数平均值对应的最小值、最大值是否有效的标志

Bit 0 为 1, 最小 Limit 有效

Bit 1 为 1, 最大 Limit 有效

Return Values

正常返回 0, 失败返回-1。

Remarks

设置 SPL 合格器件平均值范围卡控值。参数平均值超范围停机功能：计算此参数所有合格器件的测试值的平均值，如果平均值超过设置卡控范围值，报警停机。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLAverageRangeData (nParamID , 0, 0.5, 0.8, 3);
```

1.9.17 STSCBGetSPLAverageRangeInfo()

```
int STSCBGetSPLAverageRangeInfo(char* nParamID, int SubUnit, double  
&fcurAverageRangeMinLimit, double &fcurAverageRangeMaxLimit, int  
&LimitFlag);
```

Parameters

nParamID

SPL 平均值范围卡控参数名称。

SubUnit

SPL 平均值范围卡控子单元号

fcurAverageMinLimit

SPL 平均值范围卡控最小值。

fcurAverageMaxLimit

SPL 平均值范围卡控最大值。

LimitFlag

此参数平均值对应的最小值、最大值是否有效的标志

Bit 0 为 1, 最小 Limit 有效

Bit 1 为 1, 最大 Limit 有效

Return Values

正常返回 0, 失败返回-1。

Remarks

获取 SPL 合格器件平均值范围卡控设置值。

Use permission

此函数必须在载入测试程序之后, 即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double fcurAverageRangeMinLimit=0;  
double fcurAverageRangeMaxLimit=0;  
int LimitFlag=0;  
STSCBGetSPLAverageRangeInfo(nParamID, 0, fcurAverageRangeMinLimit,  
fcurAverageRangeMaxLimit, LimitFlag);
```

下面是参数波动卡控

1.9.18 STSCBSetSPLFluctuationRangeSampleCount()

```
int STSCBSetSPLFluctuationRangeSampleCount(int curSampleCount, int  
OnlyTotalSite);
```

Parameters

curSampleCount

SPL 参数测试值波动卡控最小采样数。

OnlyTotalSite

SPL 参数测试值波动卡控最小采样数是否按总工位统计。0:单工位 1:
总工位

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 合格器件参数测试值波动卡控最小采样数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLFluctuationRangeSampleCount (100,1);
```

1.9.19 STSCBSetSPLFluctuationRangeData()

```
int STSCBSetSPLFluctuationRangeData(char* nParamID, int SubUnit,  
double fcurFluctuationRangeLimit);
```

Parameters

nParamID

SPL 参数测试值波动卡控参数名称。

SubUnit

SPL 参数测试值波动卡控子单元号

fcurFluctuationRangeLimit

SPL 参数测试值波动卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 合格器件波动卡控值。参数测试波动超限停机功：计算此参数所有合格器件的测试值的平均值，然后比较所有测试值与平均值的最大偏差，如果最大偏差值超过设置卡控值，报警停机。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLFluctuationRangeData (nParamID , 0, 1.8);
```

1.9.20 STSCBGetSPLFluctuationRangeInfo()

```
int STSCBGetSPLFluctuationRangeInfo(char* nParamID, int SubUnit,  
double &fcurFluctuationRangeLimit);
```

Parameters

nParamID

SPL 参数测试值波动卡控参数名称。

SubUnit

SPL 参数测试值波动卡控子单元号

fcurFluctuationRangeLimit

SPL 参数测试值波动卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SPL 合格器件参数测试值波动卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double fcurFluctuationRangeLimit=0;  
STSCBGetSPLFluctuationRangeInfo (nParamID , 0, fcurFluctuationRangeLimit);
```

下面是参数叠料卡控

1.9.21 STSCBSetSPLStackingDetectionSampleCount()

```
int STSCBSetSPLStackingDetectionSampleCount(int curSampleCount);
```

Parameters

curSampleCount

SPL 叠料卡控最小采样数。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 叠料卡控最小采样数。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
STSCBSetSPLStackingDetectionSampleCount (160);
```

1.9.22 STSCBSetSPLStackingDetectionData()

```
int STSCBSetSPLStackingDetectionData(char* nParamID, int SubUnit,  
double fcurStackingDetectionLimit);
```

Parameters

nParamID

SPL 叠料卡控参数名称。

SubUnit

SPL 叠料卡控子单元号

fcurStackingDetectionLimit

SPL 叠料卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

设置 SPL 叠料卡控值。

参数叠料停机功能：记录此参数最近的叠料采样数的器件的测试值，根据记录的测试值，实时统计出测试值的最小值与最大值，然后计算此参数最大值与最小值之间的差值，如果差值小于设置的卡控值，报警停机。如果有多个参数用于叠料卡控，则多个参数的差值都小于对应设置的卡控值，才报警停机。只要有一个参数的差值大于卡控值，则不报叠料停机。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBSetSPLStackingDetectionData (nParamID , 0, 1.5);
```

1.9.23 STSCBGetSPLStackingDetectionInfo()

```
int STSCBGetSPLStackingDetectionInfo(char* nParamID, int SubUnit,  
double &fcurStackingDetectionLimit);
```

Parameters

nParamID

SPL 叠料卡控参数名称。

SubUnit

SPL 叠料卡控子单元号

fcurStackingDetectionLimit

SPL 叠料卡控值。

Return Values

正常返回 0，失败返回-1。

Remarks

获取 SPL 叠料卡控设置值。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
double fcurStackingDetectionLimit=0;  
STSCBGetSPLStackingDetectionInfo (nParamID , 0, fcurStackingDetectionLimit);
```

1.9.24 STSCBGetSPLStaticsInfo()

```
int STSCBGetSPLStaticsInfo(int nSite, char* nParamID, int SubUnit,  
int &ActDutCount, int &ActPassCount, double &ActMinValue, double  
&ActMaxValue);
```

Parameters

nSite:

工位号（从 0 开始）

nParamID

SPL 卡控参数名称。

SubUnit

SPL 卡控子单元号

ActDutCount:

测试器件总数

ActPassCount:

测试合格数

ActMinValue:

测试值的最小值

ActMaxValue;;

测试值的最大值

Return Values

正常返回 0，失败返回-1。

Remarks

根据参数名称与子单元号，获取某工位的测试统计信息。此函数可用于获取参数良率卡控、参数良率差卡控、参数叠料卡控的统计信息。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
int ActDutCount;  
int ActPassCount=0;  
double ActMinValue=0;  
double ActMaxValue=0;  
STSCBGetSPLStaticsInfo (nParamID, 0, ActDutCount, ActPassCount, ActMinValue,  
ActMaxValue);
```

1.9.25 STSCBGetSPLPassStaticsInfo()

```
int STSCBGetSPLPassStaticsInfo(int nSite, char* nParamID, int SubUnit,  
int &ActDutCount, double &ActMinValue, double &ActMaxValue, double  
&ActSum);
```

Parameters

nSite:

工位号（从 0 开始）

nParamID

SPL 卡控参数名称。

SubUnit

SPL 卡控参数子单元号

ActDutCount:

测试值总数

ActMinValue:

测试值的最小值

ActMaxValue;

测试值的最大值

ActSum:

测试值的和

Return Values

正常返回 0，失败返回-1。

Remarks

根据参数名称与子单元号，获取测试合格器件的某工位的测试统计信息。
此函数可用于获取参数平均值卡控、参数平均值差卡控、参数测试值波动的统计信息

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
int actDutCnt=0;  
double actminvalue=0;
```

```
double actmaxvalue =0;
double actsumvalue =0;
char* nParamID = "THD";
STSCBGetSPLPassStaticsInfo(i, nParamID, 0, actDutCnt, actminvalue, actmaxvalue,
actsumvalue);
```

1.9.26 STSCBClearSPLContent()

```
int STSCBClearSPLContent(char* nParamID, int SubUnit,int
ClearSetFlag);
```

Parameters

nParamID

SPL 卡控参数名称。

SubUnit

SPL 卡控子单元号

ClearSetFlag

SPL 卡控内容设置清除标志。

Bit 0=1，清除此参数良率超限停机功能

Bit 1=1，清除此参数良率差超限停机功能

Bit 2=1，清除此参数各工位平均值差超限停机功能

Bit 3=1，清除此参数平均值超范围停机功能

Bit 4=1，清除此参数各工位平均值波动超限停机功能

Bit 5=1，清除此参数各工位叠料停机功能

Return Values

正常返回 0，失败返回-1。

Remarks

清除 SPL 卡控内容设置。

Use permission

此函数必须在载入测试程序之后，即不能在 userload 里调用。

Example

```
char* nParamID = "THD";  
STSCBClearSPLContent (nParamID , 0, 1);
```

1.9.27 STSCBClearSPLSet()

```
int STSCBClearSPLSet (int ClearSetFlag);
```

Parameters

ClearSetFlag

SPL 卡控内容设置清除标志。

Bit 0=1, 清除良率超限停机功能

Bit 1=1, 清除良率差超限停机功能

Bit 2=1, 清除各工位平均值差超限停机功能

Bit 3=1, 清除平均值超范围停机功能

Bit 4=1, 清除各工位平均值波动超限停机功能

Bit 5=1, 清除各工位叠料停机功能

Return Values

正常返回 0, 失败返回-1。

Remarks

清除 SPL 卡控内容设置。

Use permission

此函数必须在载入测试程序之后, 即不能在 userload 里调用。

Example

```
STSCBClearSPLSet (1);
```

1.10 重测功能函数

1.10.1 STSCBSetFailRetestMode()

```
int STSCBSetFailRetestMode(int nEnableFailRetest);
```

Parameters

nEnableFailRetest

重测模式，1 重测。

Return Values

正常返回 0，其他失败。

Remarks

设置重测模式。

Example

```
STSCBSetFailRetestMode(1);
```

1.10.2 STSCBGetFailRetestMode()

```
int STSCBGetFailRetestMode (int &nEnableFailRetest);
```

Parameters

nEnableFailRetest

重测模式，1 重测。

Return Values

正常返回 0，其他失败。

Remarks

读取重测模式。

Example

```
int nEnableFailRetest;  
STSCBGetFailRetestMode (nEnableFailRetest);
```

1.10.3 STSCBAddRetestSBinID()

```
int STSCBAddRetestSBinID(int nSBID);
```

Parameters

nSBID

需要重测的软件分档。

Return Values

正常返回 0，其他失败。

Remarks

设置重测的软 BIN 号。

Example

```
STSCBAddRetestSBinID(22);
```

1.10.4 STSCBAddRetestHBinID()

```
int STSCBAddRetestHBinID(int nHBID);
```

Parameters

nHBID

需要重测的硬件分档。

Return Values

正常返回 0，其他失败。

Remarks

设置重测的硬 BIN 号。

Example

```
STSCBAddRetestHBinID(2);
```

1.10.5 STSCBGetRetestTimes ()

```
int STSCBGetRetestTimes(int &nTimes);
```

Parameters

nTimes

设置的重测次数

Return Values

正常返回 0，其他失败。

Remarks

获取界面设置的重测次数。

Example

```
int nTimes =0;  
STSCBGetRetestTimes (nTimes);
```

1.10.6 STSCBGetRetestStatus ()

```
int STSCBGetRetestStatus(int nIndex, int &nStatus);
```

Parameters

nIndex

多工位模式为工位数，从 0 开始。

多站模式为站编号，从 0 开始，不包括空站。

nStatus

此工位/站的重测状态

0: 非重测

1: 重测

Return Values

正常返回 0，其他失败。

Remarks

获取是否为重测状态。

Example

```
int nStatus =0;
STSCBGetRetestStatus (0,nStatus);
```

1.10.7 STSCBGetPartID ()

```
int STSCBGetPartID (int nIndex, char*pcPartID, int nLen);
```

Parameters

nIndex

多工位模式为工位数，从 0 开始。

多站模式为站编号，从 0 开始，不包括空站。

pcPartID

此工位/站上器件的 PartID

nLen

pcPartID 的数据长度

Return Values

正常返回 0，其他失败。

Remarks

获取工位/站上器件的 PartID。

Example

```
char pcPartID[256]={0};
STSCBGetPartID (0, pcPartID,256);
```

1.11 TestUI 测试界面信息获取函数

1.11.1 STSCBGetTesterID()

```
int STSCBGetTesterID (char *pcData, int nLen);
```

Parameters

pcData

TesterID 项值。

nLen

pcData 数据长度。

Return Values

0 正常

-1 系统版本异常。

Remarks

获取 TesterID。

Example

```
char pcData[256] = {0};  
STSCBGetTesterID (pcData,256);
```

1.11.2 STSCBGetDataLogFileName()

```
int STSCBGetDataLogFileName (  
char *pcFileName,  
bool bext = false);
```

Parameters

pcFileName

数据文件名称。

bext

名称是否包含后缀，默认不包含。

Return Values

0 正常

-1 系统版本异常。

Remarks

获取存储文件名称。

Use permission

在生成存储文件后调用。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetDataLogFileName (pcData,false);
```

1.11.3 STSCBGetSaveFile ()

```
int STSCBGetSaveFile (  
char *pcFileName,  
int nLen);
```

Parameters

pcFileName

数据文件名称。

nLen

pcFileName 的长度。

Return Values

>0 正常

-1002 没有此功能

<-nLen（输入值得相反数） 数据实际长度的相反数

其他<0 失败。

Remarks

获取存储文件名称，包含存储路径。此函数只能获取系统当前状态下存储的文件名，测试过程或者结束之后可能修改文件名与路径。

Use permission

在生成存储文件后调用。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetSaveFile (pcData, MAX_PATH);
```

1.11.4 STSCBGetSaveFilePath ()

```
int STSCBGetSaveFilePath (  
char *pcFile,  
int nLen);
```

Parameters

pcFile

数据文件存储路径。

nLen

pcFile 的长度。

Return Values

≥ 0 正常

-1002 没有此功能

$< -nLen$ （输入值得相反数） 数据实际长度的相反数

其他 < 0 失败。

Remarks

获取存储文件路径。此函数只能获取系统当前状态下存储的路径，测试过程或者结束之后可能修改路径。

Use permission

在生成存储文件后调用。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetSaveFilePath (pcData, MAX_PATH);
```

1.11.5 STSCBGetSaveFileTemporaryPath ()

```
int STSCBGetSaveFileTemporaryPath(  
char *pcFile,  
int nLen);
```

Parameters

pcFile

数据文件临时路径。

nLen

pcFile 的长度。

Return Values

≥ 0 正常

-1002 没有此功能

$< -nLen$ (输入值得相反数) 数据实际长度的相反数

其他 < 0 失败。

Remarks

获取存储文件的临时路径。

Use permission

在生成存储文件后调用。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetSaveFileTemporaryPath (pcData, MAX_PATH);
```

1.11.6 STSCBGetSummaryFileName ()

```
int STSCBGetSummaryFileName (  
char *pcFile,  
int nLen);
```

Parameters

pcFile

Summary 文件名。

nLen

pcFile 的长度。

Return Values

≥ 0 正常

-1002 没有此功能

$< -nLen$ (输入值得相反数) 数据实际长度的相反数

其他 < 0 失败。

Remarks

获取 Summary 文件名。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetSummaryFileName (pcData, MAX_PATH);
```

1.11.7 STSCBGetSummaryFilePath ()

```
int STSCBGetSummaryFilePath (  
char *pcFile,  
int nLen);
```

Parameters

pcFile

Summary 文件名。

nLen

pcFile 的长度。

Return Values

>=0 正常

-1002 没有此功能

<-nLen（输入值得相反数） 数据实际长度的相反数

其他<0 失败。

Remarks

获取 Summary 文件的路径。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
char pcData[MAX_PATH] = {0};  
STSCBGetSummaryFilePath (pcData, MAX_PATH);
```

1.11.8 STSCBGetConvertSTD ()

```
int STSCBGetConvertSTD ();
```

Parameters

Return Values

0 不生成

1 转换生成

2 实时存储

<0 失败

Remarks

获取是否输出 STD 格式文件。此函数只能获取系统当前状态，测试过程

或者结束之后可能被修改。

Example

```
int nRet=STSCBGetConvertSTD ();
```

1.11.9 STSCBGetConvertExcel ()

```
int STSCBGetConvertExcel ();
```

Parameters

Return Values

0 不生成

1 生成

<0 失败

Remarks

获取是否输出 Excel 格式文件。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
int nRet= STSCBGetConvertExcel ();
```

1.11.10 CBIT128STSCBGetConvertCSV ()

```
int STSCBGetConvertCSV ();
```

Parameters

Return Values

0 不生成

1 生成

<0 失败

Remarks

获取是否输出 CSV 格式文件。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
int nRet= STSCBGetConvertCSV ();
```

1.11.11 STSCBGetConvertRowModeTXT ()

```
int STSCBGetConvertRowModeTXT ();
```

Parameters

Return Values

0 不生成

1 生成

<0 失败

Remarks

获取是否输出行模式 TXT 格式文件。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
int nRet= STSCBGetConvertRowModeTXT ();
```

1.11.12 STSCBGetConvertTXT ()

```
int STSCBGetConvertTXT ();
```

Parameters

Return Values

0 不生成

1 生成

<0 失败

Remarks

获取是否输出 TXT 格式文件。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
int nRet= STSCBGetConvertTXT ();
```

1.11.13 STSCBGetConvertUserDefine ()

```
int STSCBGetConvertUserDefine ();
```

Parameters

Return Values

0 不生成

1 生成

<0 失败

Remarks

获取是否输出用户自定义格式文件。此函数只能获取系统当前状态，测试过程或者结束之后可能被修改。

Example

```
int nRet= STSCBGetConvertUserDefine ();
```

1.11.14 STSCBGetHandlerProberName ()

```
int STSCBGetHandlerProberName(char*pcData, int nLen);
```

Parameters

pcData

Prober/Handler 名称

nLen

pcData 的长度。

Return Values

>=0 正常

-1002 没有此功能

<-nLen（输入值得相反数） 数据实际长度的相反数

其他<0 失败。

Remarks

获取选择的 Prober/Handler 名称

Example

```
char pcData[256] = {0};  
STSCBGetHandlerProberName (pcData, 256);
```

1.11.15 STSCBSetRelayLifeCheck ()

```
int STSCBSetRelayLifeCheck(int nData);
```

Parameters

nData

0 不启用

1 启用

Return Values

=0 成功

-1002 没有此功能

其他<0 失败。

Remarks

设置是否启用继电器检测功能，接口设置优先级高于界面设置。

Example

```
STSCBSetRelayLifeCheck (1);
```

1.11.16 STSCBGetRelayLifeCheck ()

```
int STSCBGetRelayLifeCheck(int &nData);
```

Parameters

nData

0 不启用

1 启用

Return Values

=0 成功

-1002 没有此功能

其他<0 失败。

Remarks

获取是否启用继电器检测功能

Example

```
int nData=0;
STSCBGetRelayLifeCheck (nData);
if(nData == 0)
{
}
else if(nData==1)
{
}
```

1.12 PAT 功能应用函数

1.12.1 系统 PAT 函数

1.12.1.1 STSInitDPAT ()

```
int STSInitDPAT()
```

Parameters

Return Values

Remarks

删除 DPAT 生成的判据，初始化 DAPT 状态，重新采样进行 DPAT 流程。

Example

```
STSEInitDPAT ();
```

1.12.1.2 STSSetDPATSampleBin ()

```
int STSSetDPATSampleBin(int nSwBin,int nHWBin,const char*  
pcName,int nType=0)
```

Parameters

nSwBin

SWBin ID

nHWBin

HWBin ID

pcName

SWBin 描述

nType

Bin 类型 0-PassBin 其他 -FailBin 默认值 0

Return Values

-12 UserLoad 函数以外使用

-1 BinID 小于 0

-10 DPAT 模块异常

-11 未启用 DPAT

-3 SWBin 类型与已有分 BIN 冲突

-4 HWBin 类型与已有分 BIN 冲突

0 成功

Remarks

设置 DPAT 样本器件的分档，只能在 UserLoad 函数中调用。

可以选择 Pass/Fail 分档，设置的分档 ID (HWBin/SWBin) 在原有程序中是否定义都可以，如果已经定义那么不能与已有分档类型上存在冲突，分档说明将覆盖原有设置。

如果设置器件分档为 FailBin，则样本阶段良率为 0，需要关闭良率卡控等功能。

Example

```
STSSetDPATSampleBin (15,15,"DPAT Sample",0);
```

1.12.1.3 STSGetDPATTestStatus ()

```
int STSGetDPATTestStatus (BOOL bQA,int nStation=0)
```

Parameters

bQA

DPAT 参数是否为 QA 参数。FALSE 非 QA 参数，TRUE QA 参数

nStation

多站测试中 DPAT 参数所在的站，从 1 开始。FT 不使用此参数，参数默认值 0。

Return Values

-1 没有找到对应的 DPAT 测试数据（没有 DPAT 参数或者 DPAT 第一颗样本还没有生成）

0 样本采样阶段

>0 周期采样第几个周期

Remarks

获取 DPAT 测试所处各阶段。

Example

```
STSGetDPATTestStatus (FALSE, 0);
```

1.12.1.4 STSSetDPATSampleQA ()

```
int STSSetDPATSampleQA(int nType)
```

Parameters

nType

0 初始采样阶段禁用 QA 采样

其他 初始采样阶段同样做 QA 采样

Return Values

0

Remarks

设置 DPAT 初始采样阶段是否执行 QA 采样。默认不执行 QA 采样。

Example

```
STSSetDPATSampleQA(0);
```

1.12.2 自定义 PAT 函数

1.12.2.1 InitPat ()

```
void CParam::InitPat()
```

Parameters

Return Values

Remarks

初始化 PAT 计算的状态。参数状态为未计算 PAT 状态，PAT 判据为初始判据。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");  
ICC->InitPat();
```

1.12.2.2 SetPATMultiple ()

```
void CParam::SetPATMultiple(double fMulti)
```

Parameters

fMulti

PAT Limits = Mean $\pm$ n*sigma。此参数对应 n。

Return Values**Remarks**

设置 PAT 计算中 sigma 的倍数。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");  
ICC->SetPATMultiple(2);
```

1.12.2.3 AddPATData ()

```
void CParam::AddPATData (  
int nSite,  
int nSub)
```

Parameters

nSite

工位号，从 0 开始。

nSub

数据对应的子单元个数。

Return Values

Remarks

将数据添加到 PAT 计算的样本数据。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");
int nSite = StsGetSiteCount();
int nSub = ICC->GetSubUnitsCount();
for (int i = 0; i < nSite;i++)
{
    for (int j = 0; j < nSub;j++)
    {
        ICC->SetTestResult(i,j,10*sin(nTestCnt*40));
        ICC->AddPATData(i,j);
    }
}
```

1.12.2.4 GetPATDataCount ()

```
unsigned int CParam::GetPATDataCount();
```

Parameters

Return Values

样本数据个数。

Remarks

获取样本数据个数。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");
int nCnt = ICC->GetPATDataCount();
```

1.12.2.5 CalculateLimit ()

```
int CParam::CalculateLimit(bool bDelete = true);
```

Parameters

bDelete

是否清空之前存储的样本数据，默认为 **true**，清空之前存储的样本数据以节省内存。如需将之前的样本数据作为下次计算的样本数据，该标志需置为 **false**。

Return Values

- 1 样本数据小于 2，不计算 PAT。
- 0 计算成功。

Remarks

计算 PAT 判据，更新 PAT 状态与判据。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");  
ICC->CalculateLimit();
```

1.12.2.6 GetPATLimit ()

```
int CParam::GetPATLimit(  
double &fLow,  
double &fHigh);
```

Parameters

fLow

下限。

fHigh

上限。

Return Values

- 正常返回 0。通过样本数据计算的 PAT 判据将赋值到相应参数中。
- 失败返回 -1。原始判据将赋值到相应的参数中。

Remarks

获取 PAT 测试 计算的判据。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");
CParam *ICCMAX = StsGetParam(funcindex,"ICCMAX");
CParam *ICCPAT = StsGetParam(funcindex,"ICCPAT");
CParam *ICCMIN = StsGetParam(funcindex,"ICCMIN");
double fMin(0),fMax(0);
ICC->GetPATLimit(fMin,fMax);
ICCMAX->SetTestResult(i,j,fMax);
ICCMIN->SetTestResult(i,j,fMin);
```

1.12.2.7 GetPATStatus ()

```
int CParam::GetPATStatus(int nSite);
```

Parameters

nSite

工位号，从 0 开始。

Return Values

返回 0，参数测试结果 PASS。

返回-1，参数测试结果 lowfail。

返回 1，参数测试结果 Highfail。

返回 -2，没有计算 PAT 判据且测试结果为 FAIL。

Remarks

获取 PAT 测试的结果。

Example

```
CParam *ICC = StsGetParam(funcindex,"ICC");
CParam *ICCMAX = StsGetParam(funcindex,"ICCMAX");
CParam *ICCPAT = StsGetParam(funcindex,"ICCPAT");
CParam *ICCMIN = StsGetParam(funcindex,"ICCMIN");
static int nTestCnt = 0;
static bool bFirst = true;
static bool bPAT = false;
if (bFirst)
{
    ICC->IntiPat();
}
```

```
ICC->SetPATMultiple(1);
    bFrist = false;
}
int nSite = StsGetSiteCount();
int nSub = ICC->GetSubUnitsCount();
for (int i = 0; i < nSite;i++)
{
    for (int j = 0; j < nSub;j++)
    {
        ICC->SetTestResult(i,j,10*sin(nTestCnt*40));
        nTestCnt++;
        if (bPAT)
        {
            double fMin(0),fMax(0);
            ICC->GetPATLimit(fMin,fMax);
            ICCMAX->SetTestResult(i,j,fMax);
            ICCMIN->SetTestResult(i,j,fMin);
            if (ICC->GetPATStatus(i)==0)
            {
                //pass
                ICCPAT->SetTestResult(i,j,0);
            }
            else
            {
                //fail
                ICCPAT->SetTestResult(i,j,-10);
            }
        }
        else
        {
            ICC->AddPATData(i,j);
            ICCMAX->SetTestResult(i,j,ICC->GetMaxLimit());
            ICCMIN->SetTestResult(i,j,ICC->GetMinLimit());
            ICCPAT->SetTestResult(i,j,ICC->GetPassFail(i)?0:-20);
        }
    }
}
if (ICC->GetPATDataCount() > 20)
{
    ICC->CalculateLimit();
    bPAT = true;
}
return 0;
```

1.13 进度条功能应用函数

测试过程中，进度条函数无效。

1.13.1 STSCBSetProgressInfo()


```
int USERTYPE STSCBSetProgressInfo (const char * pcname, unsigned int  
nlength);
```

Parameters

pcname

进度条名称。

nlength

进度条长度。

Return Values

正常返回 0，失败返回-1。

Remarks

设置进度条显示的名称与长度。

Example

```
STSCBSetProgressInfo ("Name", 200);
```

1.13.2 STSCBSetProgressStep()

```
int USERTYPE STSCBSetProgressStep (unsigned int nstep);
```

Parameters

nstep

进度条进度。

Return Values

正常返回 0，失败返回-1。

Remarks

设置进度条显示的进度。

Example

```
DUT_API void UserLoad()
{
    int nProgressLength = 1000;
    STSSetProgressInfo("UserLoad...", nProgressLength);
    for (int step = 0; step < nProgressLength; step++)
    {
        delay_ms(10);
        STSCBSetProgressStep (step);
    }
}
```

1.14 自定义输入界面应用函数

1.14.1 STSPopupStringDialog ()

```
int STSPopupStringDialog(
char *pctitle,
char **ppcDescription,
char **ppcStrings,
int nsize,
char *pcmessage,
bool bScan);
```

Parameters

pctitle

窗口标题名称。

ppcDescription

输入字段名称。

ppcStrings

输入字符串（每个字符串的大小必须为 256）。

nsize

输入字段个数。

pcmessage

界面底部显示内容。

bScan

true 只能扫描输入，false 支持手动输入和复制粘贴。

Return Values

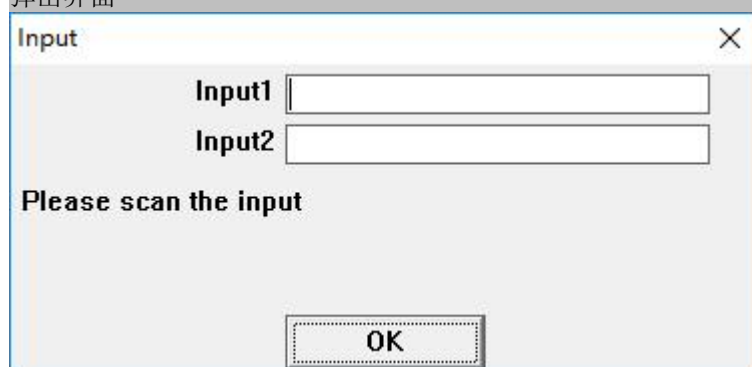
- 1 用户点击 OK。
- 2 用户点击关闭。

Remarks

打开输入界面。界面将以阻塞方式弹出。输入字符串长度为 256。

Example

```
char pDes1[] = "Input1";
char pData1[256] = { 0 };
char pDes2[] = "Input2";
char pData2[256] = { 0 };
char*ppDes[] = { pDes1, pDes2 };
char *ppData[] = { pData1, pData2 };
STSPopupStringDialog("Input", ppDes, ppData, 2, "Please scan the input", false);
弹出界面
```



1.14.2 自定义界面编辑功能

1.14.2.1 CInputData 相关函数

1.14.2.1.1 控件类型

类型	说明
----	----

CONTROL_NULL	不显示输入框，只显示控件名称
CONTROL_EDIT	编辑框
CONTROL_STATIC_TEXT	只读状态编辑框
CONTROL_COMBOX	下拉选择框
CONTROL_CHECK	复选按钮
CONTROL_RADIO	单选按钮

1.14.2.1.2 *SetName()*

```
void SetName(const char *pcData);
```

Parameters

pcData

输入字段的名称。

Return Values

Remarks

设置输入字段的名称。

1.14.2.1.3 *SetData ()*

```
void SetData(
char * pcData,
int nLen);
```

Parameters

pcData

输入内容的存储地址。

nLen

内存长度。

Return Values

Remarks

设置输入内容存储地址。

1.14.2.1.4 SetType()

```
void SetType(CONTROL_TYPE nType);
```

Parameters

nType

输入框类型。

Return Values

Remarks

设置输入框类型，具体类型详见控件类型章节。

1.14.2.1.5 SetEnableInput ()

```
void SetEnableInput(bool bInput);
```

Parameters

bInput

是否支持键盘输入。true 支持键盘输入与粘贴；false 不支持粘贴并限制输入时长，超时清空输入内容。

Return Values

Remarks

设置是否支持键盘输入，此属性只对编辑框有效。

1.14.2.1.6 InsertSelect ()

```
void InsertSelect(  
const char *pcData,  
int nIndex = -1);
```

Parameters

pcData

可选项。

nIndex

该选项所对应的位置，默认-1 添加到选择项的末尾。

Return Values

Remarks

添加选择项，此设置只对选择框或者单选按钮等需要选择的控件。

1.14.2.1.7 GetSelect ()

```
const char * GetSelect(int nIndex);
```

Parameters

nIndex

该选项所对应的位置。

Return Values

设置的选择项。

Remarks

获取设置的可选项。

1.14.2.1.8 GetSelectCnt ()

```
unsigned int GetSelectCnt();
```

Parameters

Return Values

设置的选择项个数。

Remarks

获取设置的可选项个数。

1.14.2.1.9 DeleteSelect ()

```
void DeleteSelect();
```

Parameters

Return Values

Remarks

删除所有可选项。

1.14.2.1.10 DeleteItem ()

```
void DeleteItem(unsigned int);
```

Parameters

nIndex

选择项所对应的位置。

Return Values

Remarks

删除指定位置的可选项。

1.14.2.1.11 SetDefaultValue ()

```
void SetDefaultValue(const char*);
```

Parameters

*const char**;

默认输入值。

Return Values

Remarks

设置输入或者单选按钮的默认输入值，此设置只针对编辑框，选择框和单选按钮。

1.14.2.1.12 SetCheck ()

```
void SetCheck(BOOL bCheck);
```

Parameters

BOOL;

是否选中多选框。

Return Values

Remarks

设置多选框的状态，此设置只针对多选框。

1.14.2.1.13 isCheck ()

```
BOOL isCheck();
```

Parameters

Return Values

BOOL;

复选框是否被选中。

Remarks

获取多选框的状态，只针对多选框。

1.14.2.1.14 *GetInputStr ()*

```
const char*GetInputStr ();
```

Parameters

Return Values

const char;*

返回输入的字符串指针。

Remarks

返回输入的字符串指针，此接口只针对编辑框，选择框，单选按钮。

1.14.2.1.15 *GetInputDouble ()*

```
double    GetInputDouble ();
```

Parameters

Return Values

double;

返回输入的内容转换生成的双精度数。

Remarks

返回输入的内容转换生成的双精度数，此接口只针对编辑框，选择框，单选按钮。

1.14.2.1.16 *GetInputInt ()*

```
int    GetInputInt ();
```

Parameters

Return Values

int;

返回输入的内容转换生成的整数。

Remarks

返回输入的内容转换生成的整数，此接口只针对编辑框，选择框，单选按钮。

1.14.2.1.17 *GetInputBOOL ()*

```
BOOL GetInputBOOL ();
```

Parameters

Return Values

BOOL;

返回输入的内容转换生成的 BOOL 值。

Remarks

返回输入的内容转换生成的 BOOL 值。

1.14.2.1.18 *GetType ()*

```
CONTROL_TYPE GetType ();
```

Parameters

Return Values

CONTROL_TYPE;

返回控件类型。

Remarks

返回控件类型。

1.14.2.1.19 *GetName ()*

```
const char*GetName();
```

Parameters

Return Values

*const char**;

返回控件名称。

Remarks

返回控件名称。

1.14.2.2 CInputDialog 相关函数

1.14.2.2.1 *InsertItem ()*

```
CInputData* InsertItem(const char *pcName,  
char *pcData,  
int nDatalen,  
CInputData::CONTROL_TYPE nType,  
bool bInput,  
int nIndex = -1);
```

Parameters

pcName

输入项名称。

pcData

输入内容。

nDatalen

输入内容的最大长度。

nType

输入控件的类型。

bInput

是否支持键盘输入。**true** 支持键盘输入与粘贴；**false** 不支持粘贴于限制输入时长，超时自动清空。

nIndex

该项所对应的位置，默认-1 添加到末尾。

Return Values

输入项的指针。

Remarks

添加输入项。

1.14.2.2.2 InsertConrollItem ()

```
CInputData* InsertConrollItem (const char *pcName,  
CInputData::CONTROL_TYPE nType,  
int nIndex = -1)
```

Parameters

pcName

输入项名称。

nType

输入控件的类型。

nIndex

该项所对应的位置，默认-1 添加到末尾。

Return Values

输入项的指针。

Remarks

添加输入项。

1.14.2.2.3 *GetItemCount ()*

```
unsigned int GetItemCount();
```

Parameters

Return Values

输入项的个数。

Remarks

获取输入项个数。

1.14.2.2.4 *DeleteItem ()*

```
int DeleteItem(unsigned int nIndex);
```

Parameters

nIndex

该项所对应的位置。

Return Values

0

Remarks

删除指定输入项。

1.14.2.2.5 *ClearItem ()*

```
int ClearItem();
```

Parameters

Return Values

0

Remarks

清空输入项。

1.14.2.2.6 SetWindowsName ()

```
int SetWindowsName(const char *pcData)
```

Parameters

pcData

窗体标题。

Return Values

0

Remarks

设置窗体标题。

1.14.2.2.7 SetDes ()

```
int SetDes(char *pcData)
```

Parameters

pcData

窗体说明。

Return Values

0

Remarks

设置窗体说明。

1.14.2.2.8 ShowDlg ()

```
int ShowDlg()
```

Parameters

Return Values

1: 用户点击 OK。

其他: 点击取消或直接关闭。

Remarks

显示输入界面。

1.14.2.3 STSGetDialog ()

```
CInputDialog* STSGetDialog(char *pctitle, char *pcmessage);
```

Parameters

pctitle

窗口标题名称。

pcmessage

界面底部显示内容。

Return Values

CInputDialog 输入框指针。

Remarks

获取输入框界面指针。

1.14.2.4 示例

```
CInputDialog* pDlg = STSGetDialog("inputdlg2", "Input All Item");
if (pDlg!= NULL)
{
    CInputData* pInput4 = pDlg->InsertItem("Value4", CInputData::CONTROL_EDIT);
    if (pInput4!= NULL)
    {
        pInput4->SetDefaultValue("Def4");
    }
    CInputData* pInput5 = pDlg->InsertItem("Value5",
CInputData::CONTROL_STATIC_TEXT);
    if (pInput5 != NULL)
    {
        char pcmesage[256] = { 0 };
        sprintf_s(pcmesage, "Show message");
    }
}
```

```

        pInput5->SetData(pcmessag, 256);
    }
    CInputData*          pInput6          =          pDlg->InsertItem("Value6",
CInputData::CONTROL_COMBOX);
    if (pInput6 != NULL)
    {
        pInput6->InsertSelect("Item1");
        pInput6->InsertSelect("Item2");
        pInput6->InsertSelect("Item3");
        pInput6->InsertSelect("Item4");
        pInput6->SetDefaultValue("Item3");
    }
    CInputData*          pInput7          =          pDlg->InsertItem("Value7",
CInputData::CONTROL_CHECK);
    if (pInput7 != NULL)
    {
        pInput7->SetCheck(TRUE);
    }
    CInputData*          pInput8          =          pDlg->InsertItem("Value8",
CInputData::CONTROL_CHECK);
    if (pInput8 != NULL)
    {
        pInput8->SetCheck(FALSE);
    }
    CInputData*          pInput9          =          pDlg->InsertItem("Value9",
CInputData::CONTROL_RADIO);
    if (pInput9 != NULL)
    {
        pInput9->InsertSelect("Radio1");
        pInput9->InsertSelect("Radio2");
        pInput9->SetDefaultValue("Radio2");
    }
    CInputData*          pInput10=          pDlg->InsertItem("Value10",
CInputData::CONTROL_RADIO);
    if (pInput10 != NULL)
    {
        pInput10->InsertSelect("Radio3");
        pInput10->InsertSelect("Radio4");
        pInput10->InsertSelect("Radio5");
        pInput10->SetDefaultValue("Radio4");
    }
    pDlg->ShowDlg();
    if (pInput4 != NULL)
    {
        printf("Value4=%s\r\n", pInput4->GetInputStr());
    }
    if (pInput6 != NULL)
    {
        printf("Value6=%s\r\n", pInput6->GetInputStr());
    }
    if (pInput7 != NULL)
    {
        printf("Value7=%d\r\n", pInput7->isCheck());
    }
    if (pInput8 != NULL)

```



```
{
    printf("Value8=%d\r\n", pInput8->isCheck());
}
if (pInput9 != NULL)
{
    printf("Value9=%s\r\n", pInput9->GetInputStr());
}
if (pInput10 != NULL)
{
    printf("Value10=%s\r\n", pInput10->GetInputStr());
}
```

弹出界面

1.15 TestNum 功能

1.15.1 STSGetTestNum ()

```
CSTSTestNumber* STSGetTestNum ()
```

Parameters

Return Values

Remarks

获取 TestNum 功能操作类，类使用说明详见 CSTSTestNumber 章节。

Example

```
CSTSTestNumber *pTestNum = STSGetTestNum();
```

1.15.2 CSTSTestNumber 相关函数

1.15.2.1 GetEnable ()

```
BOOL GetEnable()
```

Parameters

Return Values

TRUE TestNum 功能启用

FALSE 不使用 TestNum 功能

Remarks

获取是否启用 TestNum 功能。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();
if (pTestNum!= NULL)
{
    BOOL bData = pTestNum ->GetEnable();
    if (bData)
    {
        //启用 TestNum 功能
    }
}
```

1.15.2.2 SetEnable ()

```
void SetEnable(BOOL bData)
```

Parameters

bData

是否启用 TestNum 功能。

Return Values

Remarks

设置是否启用 TestNum 功能。此功能只在 UserLoad 过程中生效。启用 TestNum 功能后，每个参数都需要设置相应的 TestNum，且不能重复。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    pTestNum -> SetEnable (TRUE);  
}
```

1.15.2.3 GetTestNum ()

```
unsigned int GetTestNum(const char*pcSymbol, int nSub)
```

Parameters

pcSymbol

参数 Symbol。

nSub

子单元数。

Return Values

参数的 TestNum

Remarks

获取参数 TestNum。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    unsigned int nTestNum = pTestNum -> GetTestNum ("ICC",0);  
}
```

```
unsigned int GetTestNum (int nFun, int nParam, int nSub)
```

Parameters

nFun

函数编号。

nParam

函数内部参数编号。

nSub

子单元数。

Return Values

参数的 TestNum

Remarks

获取参数 TestNum。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    unsigned int nTestNum = pTestNum -> GetTestNum (0,0,0);  
}
```

1.15.2.4 SetTestNum ()

```
void SetTestNum (  
const char*pcSymbol,
```

```
unsigned int nData ,  
unsigned int nSubMid)
```

Parameters

pcSymbol

参数 Symbol。

nData

TestNum（大于 0 的整数）。

nSubMid

子单元数大于 1 时，相邻子单元 TestNum 的差值。默认值是 1。

Return Values

Remarks

设置参数 TestNum。此功能只在 UserLoad 过程中生效。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    pTestNum->SetTestNum (ICC,10010,1);  
}
```

```
void SetTestNum (int nFun,  
int nParam ,  
unsigned int nData ,  
unsigned int nSubMid)
```

Parameters

nFun

函数编号。

nParam

函数内部参数编号。

nData

TestNum（大于 0 的整数）。

nSubMid

子单元数大于 1 时，相邻子单元 TestNum 的差值。默认值是 1。

Return Values

Remarks

设置参数 TestNum。此功能只在 UserLoad 过程中生效。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    pTestNum->SetTestNum (0,0,10010,1);  
}
```

1.15.2.5 GetFunctionCount ()

```
unsigned int GetFunctionCount ()
```

Parameters

Return Values

测试程序中函数的个数。

Remarks

获取程序中函数的个数。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    unsigned int nCount = pTestNum-> GetFunctionCount ();  
}
```

1.15.2.6 GetParamCount ()

```
unsigned int GetParamCount(unsigned int nFunIndex)
```

Parameters

nFunIndex

函数编号，从 0 开始。

Return Values

指定函数中参数的个数。

Remarks

获取某个函数中参数的个数。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    unsigned int nCount = pTestNum-> GetParamCount (0);  
}
```

1.15.2.7 GetFunctionTest ()

```
BOOL GetFunctionTest(unsigned int nFunIndex)
```

Parameters

nFunIndex

函数编号，从 0 开始。

Return Values

TRUE 测试

FALSE 不测试

Remarks

获取函数是否测试。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();  
if (pTestNum!= NULL)  
{  
    BOOL bTest= pTestNum-> GetFunctionTest (0);  
}
```

1.15.2.8 SetAllTestNum ()

```
int SetAllTestNum(unsigned int nFunBegin,  
unsigned int nFunDiff,  
unsigned int nParamBegin,  
unsigned int nParamDiff,  
unsigned int nSubDiff=1);
```

Parameters

nFunBegin

函数起始值（大于 0 整数）。

nFunDiff

函数增加量（大于 0 整数）。

nParamBegin

函数中参数起始值（大于 0 整数）。

nParamDiff

参数的增量（大于 0 整数）。

nSubDiff

子单元的增量（大于 0 整数）。

Return Values

设定结果 0 成功，非 0 失败。

Remarks

按照指定的规则设置 TestNum。此功能只在 UserLoad 过程中生效。

$$\text{TestNum} = \text{nFunBegin} + (\text{FunIndex} * \text{nFunDiff}) \\ + \text{nParamBegin} + (\text{ParamIndex} * \text{nParamDiff}) \\ + \text{SubIndex} * \text{nSubDiff}.$$

FunIndex 程序中函数的顺序号，从 0 开始。

ParamIndex 函数中参数的顺序号，从 0 开始。

SubIndex 参数中子单元的顺序号，从 0 开始。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();
if (pTestNum!= NULL)
{
    pTestNum->SetAllTestNum(30000,10000, 4000,300, 10);
}
```

1.15.2.9 SetAllTestNumParam ()

```
int SetAllTestNumParam(unsigned int nParamBegin,
    unsigned int nParamDiff,
    unsigned int nSubDiff=1)
```

Parameters

nParamBegin

函数中参数起始值（大于 0 整数）。

nParamDiff

参数的增量（大于 0 整数）。

nSubDiff

子单元的增量（大于 0 整数）。

Return Values

设定结果 0 成功，非 0 失败。

Remarks

按照指定的规则设置 TestNum。此功能只在 UserLoad 过程中生效。

$$\text{TestNum} = \text{nParamBegin} + (\text{ParamIndex} * \text{nParamDiff}) + \text{SubIndex} * \text{nSubDiff}。$$

ParamIndex 程序中参数的顺序号，从 0 开始。

SubIndex 参数中子单元的顺序号，从 0 开始。

Example

```
CSTSTestNumber*pTestNum = STSGetTestNum ();
if (pTestNum!= NULL)
{
    pTestNum->SetAllTestNumParam(30000, 400, 10);
}
```

1.16 进阶功能应用函数

1.16.1 STSUpdateFunTest ()

```
int STSUpdateFunTest(int nFun, BOOL bTest);
```

Parameters

nFun

函数编号。

bTest

是否执行测试。

Return Values

0 设置成功

-1 在 userload 函数之外调用

-2 函数不存在

Remarks

设置函数是否执行测试。

此功能只能在 userload 函数中使用。

Example

```
int nRet = STSUpdateFunTest (0,TRUE);  
if(nRet == 0)  
{  
    //设置成功  
}  
else  
{  
    //设置失败  
}
```

1.16.2 STSShowLimitDlg ()

```
int STSShowLimitDlg ();
```

Parameters

无

Return Values

- 0 设置判据成功
- 1 界面没有点击 OK 按钮
- 2 在 userload 函数之外调用
- 1 要设置的判据不存在

Remarks

显示选择判据界面。

此功能只能在 userload 函数中使用。

Example

```
int nRet = STSShowLimitDlg();  
if(nRet == 0)
```

```
{  
    //设置成功  
}  
else  
{  
    //设置失败  
}
```

1.16.3 STSSetUseLimitbyID ()

```
int STSSetUseLimitbyID (  
    unsigned int nID);
```

Parameters

nID

判据组编号。

Return Values

- 0 成功
- 2 在 userload 函数之外调用
- 1 失败。

Remarks

通过判据编号设置测试用判据。此功能只能在 userload 函数中调用。

Example

```
int nRet = STSSetUseLimitbyID (0);  
if(nRet == 0)  
{  
    //设置成功  
}  
else  
{  
    //设置失败  
}
```

1.16.4 STSSetUseLimitbyName ()

```
int STSSetUseLimitbyName (  
const char*pcName);
```

Parameters

pcName

判据名称。

Return Values

0 成功

2 在 userload 函数之外调用

-1 失败。

Remarks

通过判据名称设置测试用判据。此功能只能在 userload 函数中调用。

Example

```
int nRet = STSSetUseLimitbyName ("FT");  
if(nRet == 0)  
{  
    //设置成功  
}  
else  
{  
    //设置失败  
}
```

1.16.5 STSGetLimitCount ()

```
unsigned int STSGetLimitCount ();
```

Parameters

Return Values

系统中判据个数。

Remarks

获取系统中设置的判据的个数。

Example

```
unsigned int nRet = STSGetLimitCount();
```

1.16.6 STSGetLimitName ()

```
const char*STSGetLimitName(  
unsigned int nID)
```

Parameters

nID

判据编号。

Return Values

判据名称。

Remarks

通过判据编号获取判据名称。

Example

```
const char*pcData= STSGetLimitName(nID);
```

1.16.7 STSGetUseLimitID ()

```
unsigned int STSGetUseLimitID()
```

Parameters

Return Values

正在使用的判据编号。

Remarks

获取正在使用的判据编号。

Example

```
unsigned int nUserLimit= STSGetUseLimitID();
```

1.16.8 STSGetUseLimitName ()

```
const char* STSGetUseLimitName ()
```

Parameters

Return Values

正在使用的判据名称。

Remarks

获取正在使用的判据的名称。

Example

```
const char* pcName = STSGetUseLimitName();
```

1.16.9 STSGetMustResult ()

```
CSTSMustResult *STSGetMustResult ()
```

Parameters

Return Values

必须赋值功能操作类

Remarks

获取必须赋值功能操作类，类使用说明详见 CSTSMustResult 章节。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();
```

1.16.10 STSGetFunction ()

```
CFunctionTest *STSGetFunction(unsigned int nID)
```

Parameters

nID

函数编号。

Return Values

函数指针

Remarks

获取函数指针，类使用说明详见 CFunctionTest 章节。

Example

```
CFunctionTest *pFun = STSGetFunction(0);
```

1.16.11 STSShowBinDlg()

```
int STSShowBinDlg()
```

Parameters

Return Values

- 0 设置判据成功
- 1 界面没有点击 OK 按钮
- 2 在 userload 函数之外调用
- 3 没有可选项
- 1 要设置的分档组合不存在

Remarks

显示选择分档组合界面。

此功能只能在 userload 函数中使用。

Example

```
int nRet = STSShowBinDlg ();
if(nRet == 0)
{
    //设置成功
}
else
{
    //设置失败
}
```

1.16.12 STSSetUseBinbyID ()

```
int STSSetUseBinbyID (
unsigned int nID);
```

Parameters

nID

分档组编号。

Return Values

- 0 成功
- 2 在 userload 函数之外调用
- 1 失败。

Remarks

通过编号设置测试用分档。此功能只能在 userload 函数中调用。

Example

```
int nRet = STSSetUseBinbyID (0);
if(nRet == 0)
{
```

```
//设置成功  
}  
else  
{  
//设置失败  
}
```

1.16.13 STSSetUseBinbyName ()

```
int STSSetUseBinbyName (  
const char*pcName);
```

Parameters

pcName

组合名称。

Return Values

0 成功

2 在 userload 函数之外调用

-1 失败。

Remarks

通过分档组合名称设置测试用分档。此功能只能在 userload 函数中调用。

Example

```
int nRet = STSSetUseBinbyName ("FT");  
if(nRet == 0)  
{  
//设置成功  
}  
else  
{  
//设置失败  
}
```

1.16.14 STSGetBinCount ()

```
unsigned int STSGetBinCount ();
```

Parameters

Return Values

参数分档组合个数。

Remarks

获取系统中设置的参数分档组合的个数。

Example

```
unsigned int nRet = STSGetBinCount ();
```

1.16.15 STSGetBinName ()

```
const char*STSGetBinName (  
unsigned int nID)
```

Parameters

nID

分档编号。

Return Values

分档组合名称。

Remarks

通过编号获取分档组合名称。

Example

```
const char*pcData= STSGetBinName (nID);
```

1.16.16 STSGetUseBinID ()

```
unsigned int STSGetUseBinID ()
```

Parameters

Return Values

正在使用的分档组合编号。

Remarks

获取正在使用的分档组合编号。

Example

```
unsigned int nUserLimit= STSGetUseBinID ();
```

1.16.17 STSGetUseBinName ()

```
const char* STSGetUseBinName ()
```

Parameters

Return Values

正在使用的分档组合名称。

Remarks

获取正在使用的分档组合的名称。

Example

```
const char* pcName = STSGetUseBinName ();
```

1.16.18 STSSetHardwareCheck ()

```
int STSSetHardwareCheck(BOOL bEnable)
```

Parameters

bEnable

TRUE 启用

FALSE 不启用

Return Values

0

Remarks

是否使用硬件检查功能，函数优先级高于界面设置。此设置需要在 UserLaod 函数中使用，否则可能导致界面不能同步。PGS 文件版本号大于等于 1004，则默认强制开启硬件检查功能。如需禁用只能调用此接口设置，界面设置将禁用。

Example

```
STSSetHardwareCheck(TRUE);
```

1.16.19 STSSetEnableLog ()

```
int STSSetEnableLog (BOOL bEnable)
```

Parameters

bEnable

TRUE 启用

FALSE 不启用

Return Values

0

Remarks

是否使用 EnableLog 功能，函数优先级高于界面设置。此设置需要在 UserLaod 函数中使用，否则可能导致界面不能同步。PGS 文件版本号大于等于 1004，则默认强制开启 EnableLog 功能。如需禁用只能调用此接口设置，界面设置将禁用

Example

```
STSSetEnableLog (TRUE);
```

1.16.20 CFunctionTest 相关函数

1.16.20.1 STSGetParamCount ()

```
unsigned int STSGetParamCount ()
```

Parameters

Return Values

函数中参数的个数

Remarks

获取函数中参数的个数。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    unsigned int nCount= pFun -> STSGetParamCount ();  
}
```

1.16.20.2 STSGetParam ()

```
CParam*STSGetParam(unsigned int nID);
```

Parameters

nID

函数中参数编号。

Return Values

参数指针

Remarks

通过编号获取函数中的参数指针

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    CParam*pParam= pFun -> STSGetParam (0);  
}
```

1.16.20.3 STSGetParamByName ()

```
CParam*STSGetParamByName (const char* pcName)
```

Parameters

pcName

参数名称。

Return Values

参数指针

Remarks

通过名称获取函数中的参数指针

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    CParam*pParam= pFun -> STSGetParamByName ("ICC");  
}
```

1.16.20.4 STSGetParambySymbol ()

```
CParam* STSGetParambySymbol(const char*pcSymbol)
```

Parameters

pcSymbol

参数 Symbol。

Return Values

参数指针

Remarks

通过参数标识获取函数中的参数指针

Example

```
CFunctionTest *pFun = STSGetFunction(0);  
if (pFun!= NULL)  
{  
    CParam*pParam= pFun -> STSGetParambySymbol ("ICC");  
}
```

1.16.20.5 STSISQAFunction ()

```
BOOL STSISQAFunction()
```

Parameters

Return Values

TREU InLine QA 函数

FALSE 非 InLine QA 函数

Remarks

获取函数是否是 inline QA 函数。

Example

```
CFunctionTest *pFun = STSGetFunction(0);  
if (pFun!= NULL)  
{
```



```
    BOOL bQA= pFun -> STSISQAFunction ();  
}
```

1.16.20.6 STSISDPATFunction ()

```
BOOL STSISDPATFunction();
```

Parameters

Return Values

TRUE DPAT 测试函数

FALSE 其他函数

Remarks

是否为 DPAT 测试函数。

Example

```
CFunctionTest *pFun = STSGetFunction(0);  
if (pFun!= NULL)  
{  
    BOOL bDPAT= pFun -> STSISDPATFunction ();  
}
```

1.16.20.7 STSSetTest ()

```
void STSSetTest(BOOL bData);
```

Parameters

bData

是否测试。

Return Values

Remarks

设置函数是否执行测试。此功能只在 uesrload 函数中生效。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    pFun -> STSSetTest (TRUE);  
}
```

1.16.20.8 STSGetTest ()

```
BOOL STSGetTest ()
```

Parameters

Return Values

TRUE 函数执行测试

FALSE 函数不执行测试

Remarks

获取函数是否测试。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    BOOL bTest= pFun -> STSGetTest ();  
}
```

1.16.20.9 STSGetStation ()

```
int STSGetStation()
```

Parameters

Return Values

多站测试中函数所在的 Station 编号

Remarks

获取函数所在的站号。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    int nStation= pFun -> STSGetStation ();  
}
```

1.16.20.10 STSIsAlwaysRun ()

```
BOOL STSIsAlwaysRun ()
```

Parameters

Return Values

TRUE 如果函数勾选测试那么函数必须执行

FALSE 设置 FailStop 功能后，此函数在 Fail 之后将不被执行

Remarks

获取函数是否为比执行函数。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    BOOL bAlways = pFun-> STSIsAlwaysRun();  
}
```

1.16.20.11 STSSetAlwaysRun ()

```
int STSSetAlwaysRun (BOOL bData)
```

Parameters

bData

是否必须执行。

Return Values

Remarks

设置函数是否比执行。

Example

```
CFunctionTest *pFun = STSGetFunction(0) ;  
if (pFun!= NULL)  
{  
    pFun-> STSSetAlwaysRun (TRUE);  
}
```

1.16.21 CSTSMustResult 相关函数

1.16.21.1 GetEnable ()

```
BOOL GetEnable()
```

Parameters

Return Values

TRUE 必须赋值功能启用

FALSE 不使用必须赋值功能

Remarks

获取是否启用必须赋值功能。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{  
    BOOL bData = pMustResult->GetEnable();  
    if (bData)
```

```
{  
    //启用必须赋值功能  
}  
}
```

1.16.21.2 SetEnable ()

```
void SetEnable (BOOL bData)
```

Parameters

bData

是否启用必须赋值功能。

Return Values

Remarks

设置是否启用必须赋值功能。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{  
    pMustResult-> SetEnable (TRUE);  
}
```

1.16.21.3 GetMustResult ()

```
BOOL GetMustResult (const char*pcSymbol)
```

Parameters

pcSymbol

参数 Symbol。

Return Values

TRUE 参数必须赋值。

FALSE 参数可以不赋值

Remarks

获取参数是否必须赋值。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();
if (pMustResult!= NULL)
{
    BOOL bData = pMustResult-> GetMustResult ("ICC");
    if (bData)
    {
        //参数必须赋值
    }
}
```

```
BOOL GetMustResult(int nFun, int nParam)
```

Parameters

nFun

函数编号。

nParam

函数内部参数编号。

Return Values

TRUE 参数必须赋值。

FALSE 参数可以不赋值

Remarks

获取参数是否必须赋值。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();
if (pMustResult!= NULL)
{
    BOOL bData = pMustResult-> GetMustResult (0,0);
    if (bData)
```

```
{  
    //参数必须赋值  
}  
}
```

1.16.21.4 SetMustResult ()

```
void SetMustResult (  
const char*pcSymbol,  
BOOL bData)
```

Parameters

pcSymbol

参数 Symbol。

bData

是否必须赋值。

Return Values

Remarks

设置参数是否必须赋值。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{  
    pMustResult-> SetMustResult (ICC,TRUE);  
}
```

```
void SetMustResult (int nFun,  
int nParam ,  
BOOL bData)
```

Parameters

nFun

函数编号。

nParam

函数内部参数编号。

bData

是否必须赋值。

Return Values

Remarks

设置参数是否必须赋值。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{  
    BOOL bData = pMustResult-> SetMustResult (0,0,TRUE);  
}
```

1.16.21.5 GetMustResultParamCount ()

```
int GetMustResultParamCount ()
```

Parameters

Return Values

必须赋值参数的个数

Remarks

获取必须赋值参数的个数。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{
```



```
int nCount =pMustResult-> GetMustResultParamCount ();  
}
```

1.16.21.6 GetMustResultParam ()

```
CParam *GetMustResultParam(unsigned int nItem)
```

Parameters

nItem

参数编号。

Return Values

参数指针

Remarks

获取第 *nItem* 个必须赋值的参数。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();  
if (pMustResult!= NULL)  
{  
    CParam *pParam =pMustResult-> GetMustResultParam (0);  
}
```

1.16.21.7 SetAllParamMustValue ()

```
void SetAllParamMustValue(BOOL bData)
```

Parameters

bData

是否启用必须赋值。

Return Values

Remarks

设置所有参数是否必须赋值。

Example

```
CSTSMustResult *pMustResult = STSGetMustResult();
if (pMustResult!= NULL)
{
    pMustResult-> SetAllParamMustValue (TRUE);
}
```

1.17 应用举例

1.17.1 程序中定义 QA 起始函数

Inline QA 执行流程：InitBeforeTestFlow→FT→QA→InitAfterTestFlow

用户除了可以在 PGSEdit 界面设置 QA 的起始函数外，也可以在程序中设置 QA 的起始函数，即在程序中添加 SetQAStartFuncNum()函数。

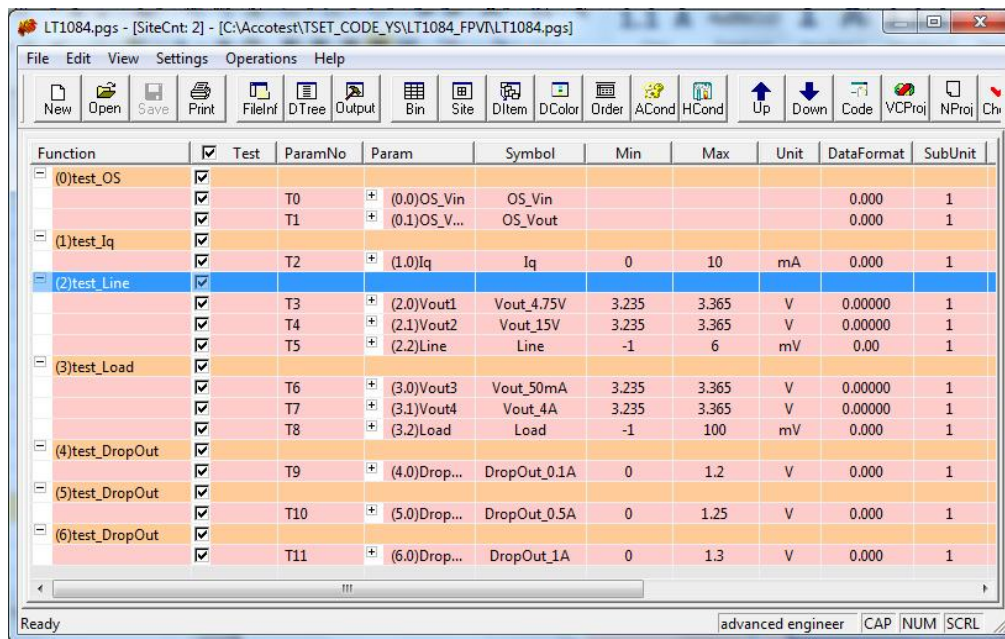
```
const int g_iQAStartFuncSet = X;
DUT_API int SetQAStartFuncNum()
{
    return g_iQAStartFuncSet;
}
```

其中，g_iQAStartFuncSet 为 SetQAStartFuncNum()函数的返回值。该返回值代表了设置 QA 起始函数为当前被测试函数列表的第几个函数，函数列表的函数从 0 开始计数。

g_iQAStartFuncSet 应介于 1 和 N 之间，N=(被测试函数个数-1)。

Example:

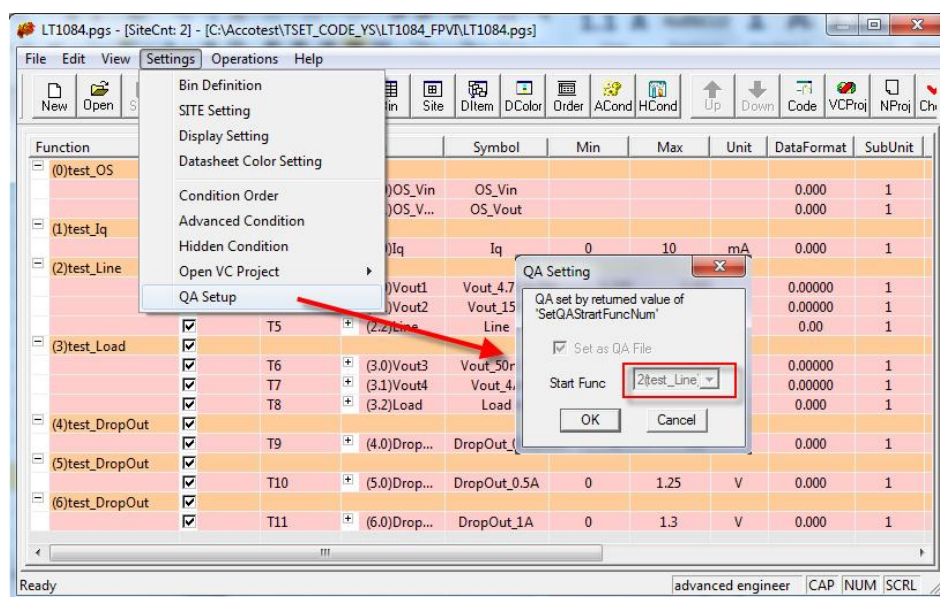
如下图所示的 PGSEdit 界面，设置函数 test_Line 为 QA 起始函数。



测试代码如下：

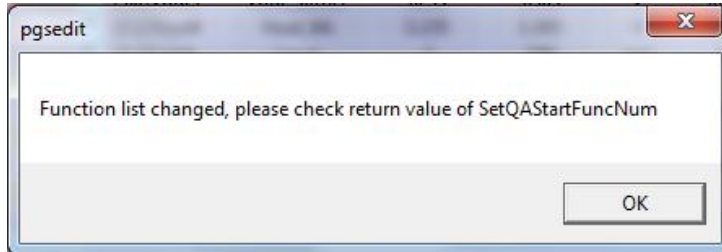
```
const int g_iQAStrtFuncSet = 2;
DUT_API int SetQAStrtFuncNum()
{
    return g_iQAStrtFuncSet;
}
```

程序设置好之后，在 PGSEdit 界面打开 QA Setting 对话框，此时 QA 的起始函数已经设置为 test_Line，并且在此界面无法更改 QA 起始函数，仅能在程序中更改。

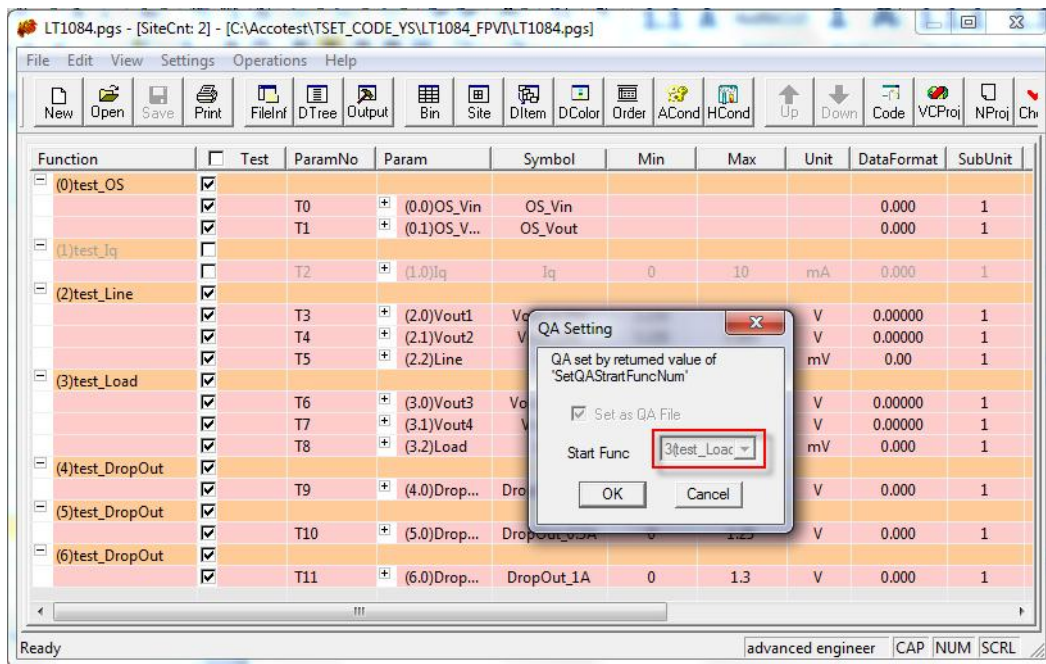


注意：

若函数列表有变动，比如增加了或者减少了一个测试函数项，此时 QA 的起始函数也会发生改变，出于慎重考虑，在保存 PGSEdit 界面时会弹出如下对话框：



例如，如下图所示，若不勾选测试函数项 test_Iq，此时 QA 的起始函数变为 test_Load。



1.17.2 程序中设置 DIE 坐标

用户除了可以在 TestUI 的 Option 界面设置 DIE 坐标之外，也可以在程序中设置 DIE 坐标，即在 InitBeforeTestFlow()或 InitAfterTestFlow()中，通过调用函数 StsGetSingleDieCorXY()和函数 StsSetDieCorXY()来实现 DIE 坐标设

置。

```
DUT_API void InitBeforeTestFlow()
{
}
```

```
DUT_API void InitAfterTestFlow()
{
}
```

- **StsGetSingleDieCorXY()**: 用于获取 prober 传来的第一个 DIE 的坐标信息（一般指 Site1 的 DIE 坐标信息）；
- **StsSetDieCorXY()**: 用于设置其他工位的坐标信息；
函数具体定义可参考 1.4.1 和 1.4.2 节内容。

Example 1:

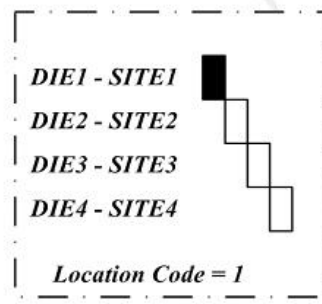
四工位测试，设置按照下图方向排列针卡的坐标（默认 X 轴增长方向是向右，Y 轴增长方向是向下）。



```
#define sitedut 4
DUT_API void InitAfterTestFlow()
{
    int x = 0,y= 0;
    StsGetSingleDieCorXY(0,x,y);//Get the coordinate of the first DIE from the prober
    for(int site = 1; site <sitedut; site++)
    {
        StsSetDieCorXY(site,x+site,y);
        //Set the coordinates for the other site according to the coordinate of the first DIE
    }
}
```

Example 2:

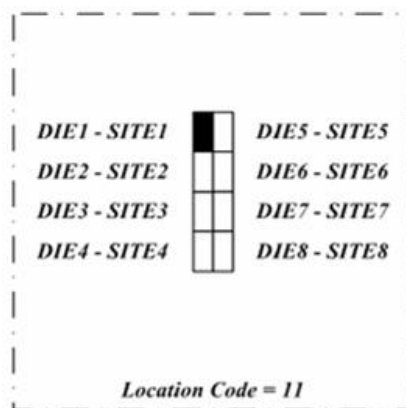
四工位测试，设置按照下图方向排列针卡的坐标（默认 X 轴增长方向是向右，Y 轴增长方向是向下）。



```
#define sitedut 4
DUT_API void InitAfterTestFlow()
{
    int x = 0,y= 0;
    StsGetSingleDieCorXY(0,x,y);// Get the coordinate of the first DIE from the prober
    for(int site = 1; site <sitedut; site++)
    {
        StsSetDieCorXY(site,x+site,y+site);
        //Set the coordinates for the other site according to the coordinate of the first DIE
    }
}
```

Example 3:

八工位测试，设置按照下图方向排列针卡的坐标（默认 X 轴增长方向是向右，Y 轴增长方向是向下）。



```
#define sitedut 8
DUT_API void InitAfterTestFlow()
{
    int x = 0,y= 0;
    StsGetSingleDieCorXY(0,x,y);// Get the coordinate of the first DIE from the prober
    for(int site = 1; site <( sitedut/2); site++)
    {
        StsSetDieCorXY(site,x,y+site);
        //Set the coordinates for the other site according to the coordinate of the first DIE
    }
    for(int site = ( sitedut/2); site <sitedut; site++)
    {
        StsSetDieCorXY(site,x+1,y+site);
        //Set the coordinates for the other site according to the coordinate of the first DIE
    }
}
```

1.17.3 程序中获取 Lot 信息

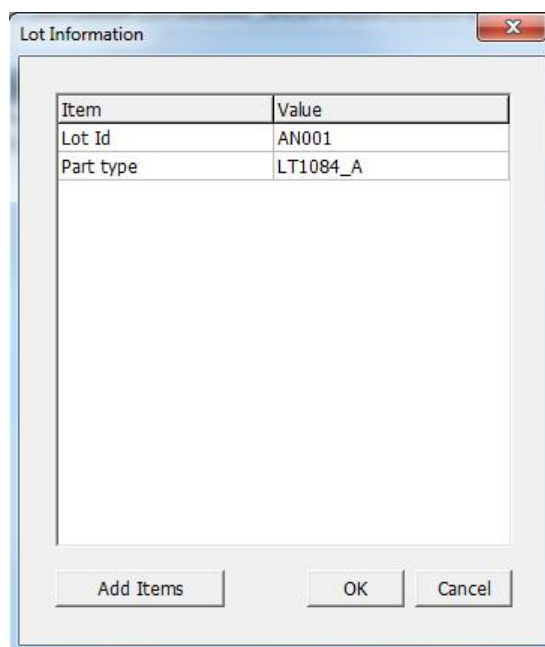
在 OnNewLot() 函数中调用函数 GetNewLotInfoCnt() 和函数 GetNewLotItemInfo()来获取 Lot 信息。

```
DUT_API void OnNewLot(const char *Lotid)
{
}
```

- GetNewLotInfoCnt(): 用于获取 Lot Item 的项目个数;
 - GetNewLotItemInfo(): 用于获取 Lot Items 的具体信息。
- 函数具体定义可参考 1.4.3 和 1.4.4 节内容。

Example:

读回下图所示 Lot 界面所设置的 Lot 信息。



```
#define SIZE 20
char LotID_name[SIZE];
char LotID_value[SIZE];
char Part_name[SIZE];
char Part_value[SIZE];
int LotCount = 0;
```



```
int i_lot = 0;
DUT_API void OnNewLot(const char*Lotid)
{
    LotCount = GetNewLotInfoCnt();//Get the counts of Lot Item
    for (i_lot = 0; i_lot<LotCount; i_lot++)
    {
        GetNewLotItemInfo(i_lot, LotID_name, LotID_value, SIZE);
        //Get the information for lot items.
    }
}
```

1.17.4 程序中控制 FailStop

在 TestUI 界面的 Option 选项中，用户可以设置下图所示的选项：

☐ Stop on Fail

- 勾选：出现失效参数时，该参数所在的测试函数之后的参数停止测试
- 不勾选：出现失效参数时，该参数所在的测试函数之后的参数继续测试

以上设置也可通过程序进行控制，即在程序中添加 EnableFailStop()函数。

```
const int g_iFailStopSet = X;
DUT_API int EnableFailStop()
{
    return g_iFailStopSet;
}
```

其中，g_iFailStopSet 为 EnableFailStop()函数的返回值。该返回值可设置为：

- g_iFailStopSet=1 时：

测试主界面不论是哪种用户权限（admin, op, engineer）登录，下图的选项都处于灰色未勾选状态（后面带有 Program Set），用户在 Option 界面没有修改权限。

☐ Stop on Fail[Program Set]

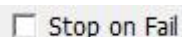
- g_iFailStopSet=2:

测试主界面不论是哪种用户权限（admin, op, engineer）登录，下图的选项都处于灰色已勾选状态（后面带有 Program Set），用户在 Option 界面没有修改权限。



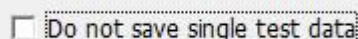
- g_iFailStopSet=X, X 为非 1, 非 2 的其他值:

下图的选项与未设置 EnableFailStop()函数的效果一样，用户可在 Option 界面设置该选项。



1.17.5 程序中控制 Single Test Data Not Save

在 TestUI 界面的 Option 选项中，用户设置下图所示的选项：



- 勾选：量产时临时点击 Test 按钮，不保存本次测试（SingleTest）的数据；
- 不勾选：量产时临时点击 Test 按钮，保存本次测试（SingleTest）的数据；

以上设置也可通过程序进行控制，即在程序中添加 EnableSingleTestSave () 函数。

```
const int g_iSingleTestSaveSet = X;
DUT_API int EnableSingleTestSave()
{
    return g_iSingleTestSaveSet;
}
```

其中，g_iSingleTestSaveSet 为 EnableSingleTestSave()函数的返回值。该返回值可设置为：

- g_iSingleTestSaveSet=1 时：

测试主界面不论是哪种用户权限（admin, op, engineer）登录，下图的选项都处于灰色未勾选状态（后面带有 Program Set），用户在 Option 界面没有修改权限。

☐ Do not save single test data[Program Set]

● g_iSingleTestSaveSet=2:

测试主界面不论是哪种用户权限（admin, op, engineer）登录，下图的选项都处于灰色已勾选状态（后面带有 Program Set），用户在 Option 界面没有修改权限。

☒ Do not save single test data[Program Set]

● g_iSingleTestSaveSet=X, X 为非 1, 非 2 的其他值:

下图的选项与未设置 EnableSingleTestSave()函数的效果一样，用户可在 Option 界面设置该选项。

☐ Do not save single test data

1.17.6 程序中设置对话框

建议在 UserLoad()函数中调用函数 STSSetInitSelectDialog()来设置对话框。

```
DUT_API void UserLoad()
{
}
```

- STSSetInitSelectDialog(): 用于设置对话框的内容。该函数的返回值即为可选择 item 的序号。
函数具体定义可参考 1.4.5 节内容。

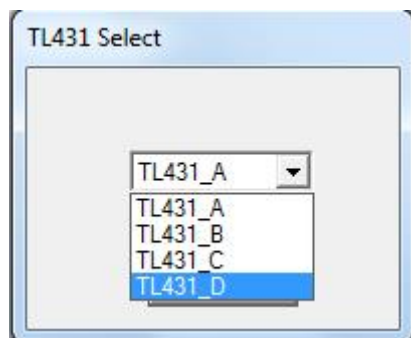
Example:

在程序中设置对话框，对话框的抬头是“TL431 Select”，对话框的下拉菜单可选择“TL431_A,TL431_B,TL431_C,TL431_D”。

```
char selectstr[] = "TL431_A,TL431_B,TL431_C,TL431_D";
```

```
int item_num = 0;
DUT_API void UserLoad()
{
    item_num = STSSetInitSelectDialog("TL431 Select",4,0,selectstr);
}
```

当用户 Load 程序时会弹出以下对话框，在对话框的下拉菜单选择器件的种类。当选择 TL431_D 时，STSSetInitSelectDialog()的返回值 item_num=3。



1.17.7 程序中设置储存文件名

用户除了可以在 TestUI 界面的 Option 选项中设置存储文件名之外，也可以在程序中设置存储文件名，即在 UserLoad() 函数中调用函数 StsSetSaveFileNameStr()来实现程序中设置存储文件名。

```
DUT_API void UserLoad()
{
}
```

- StsSetSaveFileNameStr(): 用于设置存储文件名。
函数具体定义可参考 1.4.6 节内容。

Example:

在程序中设置存储文件名为“UC3842”。

```
DUT_API void UserLoad()
{
    StsSetSaveFileNameStr("UC3842");
}
```

2 CBITe

2.1 CBITe()

```
CBITe ();
```

Remark

定义 CBITe 资源。

注意：

CBITe 模块不使用“STSSetMultiSiteBind”函数进行工位绑定。

CBITe 使用“STSCBITeMultiSiteBind”进行工位绑定。

Example

例 1：四工位测试，每个工位使用 2 个继电器 K1、K2。对应如下：

SITE1: K1=CBite0, K2= CBite1;

SITE2: K1=CBite8, K2= CBite9;

SITE3: K1=CBite16, K2= CBite17;

SITE4: K1=CBite23, K2= CBite24;

方法 1:

```
CBITecbite;  
  
BYTE K1_S1=0;  
BYTE K2_S1=1;  
  
BYTE K1_S2=8;  
BYTE K2_S2=9;  
  
BYTE K1_S3=16;  
BYTE K2_S3=17;  
  
BYTE K1_S4=23;  
BYTE K2_S4=24;  
  
cbite.SetOn(K1_S1, K1_S2, K1_S3, K1_S4, K2_S1, K2_S2, K2_S3, K2_S4,-1);
```

```
// set the K1_S1, K1_S2, K1_S3, K1_S4, K2_S1, K2_S2, K2_S3, K2_S4 on
```

方法 2:

```
#define K1 0,8,16,23  
#define K2 1,9,17,24  
  
cbite.SetOn(K1, K2, -1); // set the K1, K2 on of all 4 sites
```

2.2 Init()

```
void Init();
```

Remarks

初始化 CBITe 控制位。执行该语句之后 CBITe 控制位和电源状态如下:

- 1)所有 CBITe 控制位输出状态为高电平;
- 2)电源 (+5V, +12V, $\pm 15V$) 状态为关闭。

Example

```
cbite.Init();  
// set all cbite off status and power off all userpower
```

2.3 SetOn()

```
BYTE SetOn(int k1, ...);
```

Parameters

k1

对应的 CBITe 控制位。

Return Values

-1: 设置失败

0: 设置成功

Remarks

(1) 未进行工位绑定:

当函数执行时, k1、...指向的 CBITe 控制位输出低电平, 对应的继电器吸合; 参数中没有指定的 CBITe 控制位输出高电平, 对应的继电器断开。

(2) 进行工位绑定:

- 1) 参数中指定的 CBITe 控制位只有在有效工位或 NO_SITE 时输出低电平, 对应的继电器吸合;
- 2) 参数中指定的 CBITe 控制位在无效工位时, 输出高电平, 对应的继电器断开;
- 3) 若参数中指定的 CBITe 控制位未进行工位绑定, 对应的继电器保持之前的状态;
- 4) 参数中所有未指定的 CBITe 控制位, 对应继电器都断开。

注意:

参数个数为任意多, 但最后一个参数必须为-1, 作为结束参数标志。

Example

例 1: 假设用户 DUT 板上有 4 个继电器 K1、K2、K3、K4, 对应的 CBITe 控制位分别为 0、1、2、3, 未进行工位绑定。

```
CBITecbite;  
  
BYTE K1 = 0;  
BYTE K2 = 1;  
BYTE K3 = 2;  
BYTE K4 = 3;  
  
cbite.SetOn(K1, K2, -1); // only set K1 and K2 on, K3 and K4 off  
cbite.SetOn(K1, K2, K3, K4, -1);  
cbite.SetOn(-1);
```

例 2: 控制位 0~7、10、11 进行工位绑定, SITE_1 有效, SITE_2 无效。

绑定如下:

```
STSCBITeMultiSiteBind(SITE_1, "0,1,2,3");  
STSCBITeMultiSiteBind(SITE_2, "4,5,6,7");  
STSCBITeMultiSiteBind(NO_SITE, "10,11 ");
```

CBITe 设置如下:

```
CBITecbite;  
cbite.SetOn(0, 4, 8, 10, -1); //set 0 and 10 on, 4 off, 8 keep, others off;
```

2.4 SetCbiteOn()

```
BYTE SetCbiteOn (BYTE nIndex);
```

Parameters

nIndex

要设置的 CBITe 控制位。

Return Values

-2: 板卡无效

0: 设置成功

Remarks

(1) 未进行工位绑定:

设置 *nIndex* 指向的 CBITe 控制位输出低电平。调用该函数一次, 仅能设置一个 CBITe 控制位的状态, 其他控制位的状态保持不变。

(2) 进行工位绑定:

若 *nIndex* 指向的 CBITe 控制位所在的工位有效或为 NO_SITE, 则对应继电器吸合; 若所在工位无效或未进行工位绑定, 则对应继电器保持上一个状态。

Example

例 1: 假设用户 DUT 板上有 4 个继电器 K1、K2、K3、K4, 对应的 CBITe 控制位分别为 0、1、2、3, 未进行工位绑定。

```
CBITecbite;
```

```

BYTE K1 = 0;
BYTE K2 = 1;
BYTE K3 = 2;
BYTE K4 = 3;

cbite.SetCbiteOn (K1); //set K1 on
cbite.SetCbiteOn (K2); //set K2 on, K1 is still on
cbite.SetCbiteOn (K3); // set K3 on, K1and K2 is still on
cbite.SetCbiteOn (K4); // set K2 on, K1, K2 and K3 is still on
cbite.SetCbiteOff (K1); // set K1 off, K2, K3 and K4 is still on
cbite.SetCbiteOff (K4); // set K4 off, K2, and K3 is still on

```

例 2：控制位 0~7、10、11 进行工位绑定，SITE_1 有效，SITE_2 无效。

绑定如下：

```

STSCBITEMultiSiteBind(SITE_1, "0,1,2,3");
STSCBITEMultiSiteBind(SITE_2, "4,5,6,7");
STSCBITEMultiSiteBind(NO_SITE, "10,11 ");

```

CBITE 设置如下：

```

CBITecbite;

cbite.SetOn(0, 4, 8, 10, -1); // set 0and 10on, 4 off, 8 keep, others off
cbite.SetCBITOn(2);           // set 2 on
cbite.SetCBITOn(11);          //set 11 on
cbite.SetCBITOn(4);           //set 4keep off

```

2.5 SetCbiteOff()

```

BYTE SetCbiteOff (BYTE nIndex) ;

```

Parameters

nIndex

要设置的 CBITe 控制位。

Return Values

-2: 板卡无效

0: 设置成功

Remarks

(1) 未进行工位绑定：

设置 nIndex 指向的 CBITe 控制位输出高电平。调用该函数一次，仅能设置一个 CBITe 控制位的状态，其他控制位的状态保持不变。

(2) 进行工位绑定：

若 nIndex 指向的 CBITe 控制位所在的工位有效或为 NO_SITE，则对应继电器断开；若所在工位无效或未进行工位绑定，则对应继电器保持上一个状态。

Example

例 1：假设用户 DUT 板上有 4 个继电器 K1、K2、K3、K4，对应的 CBITe 控制位分别为 0、1、2、3，未进行工位绑定。

```
CBITecbite;

BYTE K1 = 0;
BYTE K2 = 1;
BYTE K3 = 2;
BYTE K4 = 3;

cbite.SetCbiteOn (K1); //set K1 on
cbite.SetCbiteOn (K2); //set K2 on, K1 is still on
cbite.SetCbiteOn (K3); // set K3 on, K1and K2 is still on
cbite.SetCbiteOn (K4); // set K2 on, K1, K2 and K3 is still on
cbite.SetCbiteOff (K1); // set K1 off, K2, K3 and K4 is still on
cbite.SetCbiteOff (K4); // set K4 off, K2, and K3 is still on
```

例 2：控制位 0~7、10、11 进行工位绑定，SITE_1 和 SITE2 均有效。

绑定如下：

```
STSCBITeMultiSiteBind(SITE_1, "0,1,2,3");
STSCBITeMultiSiteBind(SITE_2, "4,5,6,7");
STSCBITeMultiSiteBind(NO_SITE, "10,11 ");
```

CBITe 设置如下：

```
CBITecbite;

cbite.SetOn(0, 4, 8, 10, -1); //set 0, 4 and 10 on, 8 keep, others off
cbite.SetCBITOn(11); //set 11 on
cbite.SetCBITOff(11); //set 11 off
cbite.SetCBITOff(0); //set 0 off

BEGIN_SINGLE_SITE(0) //only SITE_1 is valid
cbite.SetCBITOff(4); //set 4 keep on
END_SINGLE_SITE();
```

2.6 GetCbiteStatus()

```
WORD GetCbiteStatus (CBITE_BYTE_NOnByte) ;
```

Parameters

nByte

系统最大支持两块 CBITe，共 256 个 CBITe 控制位，将 256 个 CBITe 控制位分为 16 组，分别被 CBITE_CHIPS_1，CBITE_CHIPS_2，...，CBITE_CHIPS_16 控制，每个 CBITE_CHIPS 控制 16 个 CBITe 位。

CBITE_CHIPS_1,
CBITE_CHIPS_2,
CBITE_CHIPS_3,
CBITE_CHIPS_4,
CBITE_CHIPS_5,
CBITE_CHIPS_6,
CBITE_CHIPS_7,
CBITE_CHIPS_8,
CBITE_CHIPS_9,
CBITE_CHIPS_10,
CBITE_CHIPS_11,
CBITE_CHIPS_12,
CBITE_CHIPS_13,
CBITE_CHIPS_14,
CBITE_CHIPS_15,
CBITE_CHIPS_16

Return Values

返回对应 *nByte* 的 16 个控制位状态。

注意：

返回值为十进制数，转化为二进制数后，每一位表示一个控制位的状态，1 表示继电器吸合，0 表示继电器断开。

Remarks

获取 CBITE 指定的 16 个控制位的状态。

Example

例 1: 获取 CBITE_CHIPS_1 对应控制位的状态, 即控制位 0 到 15 的状态。

```
BYTE result=0;
cbite.SetOn(2,3,4,5,6,7, -1);
delay_ms(1);
result = cbite.GetCbiteStatus(CBITE_CHIPS_1);
// The result is 252( 0000000011111100 in binary),the binary number of highest is channel 15, the lowest is channel 0.
```

例 2: 获取 CBITE_CHIPS_8 对应控制位的状态, 即控制位 112 到 127 的状态。

```
BYTE result=0;
cbite.SetOn(112,114,116,118,120,123,124,125,126,127, -1);
delay_ms(1);
result = cbite.GetCbiteStatus(CBITE_CHIPS_8);
// The result is 63829(1111100101010101),the binary number of highest is channel 127, the lowest is channel 112.
```

2.7 SetDutPower()

```
int SetDutPower(
CBITE_POWERpower_sel,
CBITE_POWER_STATUSstatus,
CBITE_BOARD_SELboardno);
```

Parameters

power_sel

电源名称:

CBITE_15V_POSITIVE:	+15V 电源
CBITE_15V_NEGATIVE:	-15V 电源
CBITE_12V_POSITIVE:	+12V 电源
CBITE_5V_POSITIVE:	+5V 电源

CBITE_ALL_USERPOWER: +15V、-15V、+12V、+5V 四种电源

status

电源状态:

CBITE_POWERON: 电源开启

CBITE_POWEROFF: 电源关闭

boardno

板卡序号

CBITE_BOARD1: 第一块板卡

CBITE_BOARD2: 第二块板卡

Return Values

0: 选定的板卡有效

-2: 选定的板卡无效

Remarks

控制选定的 CBITe 板卡（第一块或第二块）的四种用户电源的状态。

Example

例 1: 打开第二块板卡的全部用户电源。

```
cbite.SetDutPower(CBITE_ALL_USERPOWER,CBITE_POWERON,CBITE_BOARD2);  
// set all userpower of CBITE_BOARD2 on
```

2.8 GetDutPowerStatus()

```
int DutPowerStatus(  
CBITe_POWER power_sel,  
CBITe_BOARD_SEL boardno);
```

Parameters

power_sel

电源名称:

CBITE_15V_POSITIVE: +15V 电源

CBITE_15V_NEGATIVE: -15V 电源

CBITE_12V_POSITIVE: +12V 电源

CBITE_5V_POSITIVE: +5V 电源

CBITE_ALL_USERPOWER: +15V、-15V、+12V、+5V 四种电源

boardno

板卡序号

CBITE_BOARD1: 第一块板卡

CBITE_BOARD2: 第二块板卡

Return Values

返回 1: 所查询的电源处于打开状态, 且运行正常;

返回 0: 所查询的电源处于关闭状态;

返回-1: 所查询的电源应处于打开状态, 但运行异常 (例如, 应输出 5V 但实际只输出 3V);

返回-2: 所查询的板卡无效;

返回-3: 所查询的板卡支持程控, 但不支持电源状态查询;

返回-4: 所查询的板卡不支持电源程控;

注意:

当查询类型为 **CBITE_ALL_USERPOWER** 时, 返回值为一个 8bit 的值, 其中前 4 个 bit 对应四种电源的开关状态, 后 4 个 bit 对应电源的运行状态。

Remarks

获取板卡电源状态。

Example

例 1：获取第一块板卡的+15V 电源状态。

```
int result=0;  
result = cbite.GetDutPowerStatus (CBITE_15V_POSITIVE,CBITE_BOARD1);
```

2.9 获取板卡固有信息函数

2.9.1 GetBoardSN()

```
int GetBoardSN(  
CBITe_BOARD_SEL boardno,  
char* boardSN,  
UINT snSize);
```

Parameters

boardno

板卡序号

CBITE_BOARD1: 第一块板卡

CBITE_BOARD2: 第二块板卡

boardSN

char 型变量，存储板卡的 SN 号。未存储返回 N/A。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 boardSN 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

获取板卡 SN 号。

Example

```
char boardSN[255] = {0};  
cbite0.GetBoardSN(CBITE_BOARD1, boardSN, 255);    // Get the board serial number
```

2.9.2 GetBoardHdRev()

```
int GetBoardHdRev(  
    CBITE_BOARD_SEL siteNo,  
    char* hardRev,  
    UINT revSize);
```

Parameters

boardno

板卡序号

CBITE_BOARD1: 第一块板卡

CBITE_BOARD2: 第二块板卡

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 hardRev 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

获取板卡硬件版本号。

Example

```
char hardRev [255] = { 0 };  
cbite0.GetBoardHDRev(CBITE_BOARD1, hardRev, 255); // Get the board hardware revision
```


3 QTMUe

3.1 QTMUe()

```
QTMUe(char* channelList, char* chName = NULL);
```

Parameters

channelList

QTMUe 绑定通道字符串。形如“S23_0, S23_1”，其中“S23”表示槽位号 23（S 即 Slot），“0、1”表示板内序号。

注意：

QTMUe 一块板卡最多 4 个通道，因此板内序号最大值为 3。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 QTMUe_xx。

Remarks

定义一路 QTMUe 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 1 块 QTMUe（位于 Slot23），四工位并行工作，定义可使用的 QTMUe。

```
QTMUe qtmue0("S23_0-3");
```

对应的工位绑定如下：

```
STSSSetMultiSiteBind(MD_QTMUe, SITE_1, "S23_0"); // set Slot23 ch0 to SITE1
STSSSetMultiSiteBind(MD_QTMUe, SITE_2, "S23_1"); // set Slot23 ch1 to SITE2
STSSSetMultiSiteBind(MD_QTMUe, SITE_3, "S23_2"); // set Slot23 ch2 to SITE3
STSSSetMultiSiteBind(MD_QTMUe, SITE_4, "S23_3"); // set Slot23 ch3 to SITE4
```

3.2 Connect()

```
int Connect(  
QTMUe_RELAY_CHANNEL RelayChabSel,  
UINT InoutDelay = 500);
```

Parameters

RelayChabSel

设置接通继电器通道。

QTMUe_RELAY_CHA	只接通 CHA 继电器
QTMUe_RELAY_CHB	只接通 CHB 继电器
QTMUe_RELAY_CHAB	同时接通 CHA 和 CHB 继电器

InoutDelay

设置接通继电器后的延时时间，单位 uS，取值范围为 0~10000，默认值为 500。

注意：

- 1) 默认值 500uS 为继电器接通所必需的时间；
- 2) 延时时间可以共享以提高测试效率，例如：

```
qtmue0.Connect(QTMUe_RELAY_CHA,0);  
qtmue1.Connect(QTMUe_RELAY_CHA,500);
```

- 3) 在函数 Connect()之后测量函数之前，如果有其它资源动作或者其它延时，可以适当减少 Connect()中的延时时间以提高测试效率，例如：

```
qtmue0.Connect(QTMUe_RELAY_CHA,300);  
.....
```

Remarks

接通 QTMUe 当前逻辑通道的输入继电器。

Example

```
qtmue0.Connect(QTMUe_RELAY_CHA, 500);// connect CHA of qtmue0, delay500uS
```

3.3 Disconnect()

```
int Disconnect(  
QTMUE_RELAY_CHANNEL RelayChabSel,  
UINT InoutDelay = 100);
```

Parameters

RelayChabSel

设置断开继电器通道。

QTMUe_RELAY_CHA	只断开 CHA 继电器
QTMUe_RELAY_CHB	只断开 CHB 继电器
QTMUe_RELAY_CHAB	同时断开 CHA 和 CHB 继电器

InoutDelay

设置断开继电器后的延时时间，单位 uS，取值范围为 0~10000，默认值为 100。

注意：

- 1) 默认值 100uS 为继电器断开所必需的时间；
- 2) 延时时间可以共享以提高测试效率，例如：

```
qtmue0.Disconnect(QTMUe_RELAY_CHA,0);  
qtmue1.Disconnect(QTMUe_RELAY_CHA,100);
```

- 3) 在函数 Disconnect()之后，如果有其它资源动作或者其它延时，可以适当减少 Disconnect ()中的延时时间以提高测试效率，例如：

```
qtmue0.Disconnect(QTMUe_RELAY_CHA,0);  
.....
```

Remarks

断开当前逻辑通道的输入继电器。

Example

```
qtmue0.Disconnect(QTMUe_RELAY_CHB,100); // disconnect CHB of qtmue0, delay 100uS.
```

3.4 SetInSource()

```
int SetInSource(  
QTMUe_SOURCE_AB sourceSel = QTMUe_SINGLE_SOURCE_A)
```

Parameters

sourceSel

测量信号源选择。

QTMUe_SINGLE_SOURCE_A: 单信号源测试，Start 和 Stop 信号源都是 SOURCE_A

QTMUe_SINGLE_SOURCE_B: 单信号源测试，Start 和 Stop 信号源都是 SOURCE_B

QTMUe_DUAL_SOURCE_START_A: 双信号源测试，Start 和 Stop 信号源分别是 SOURCE_A 和 SOURCE_B

QTMUe_DUAL_SOURCE_START_B: 双信号源测试，Start 和 Stop 信号源分别是 SOURCE_B 和 SOURCE_A

默认值为 QTMUe_SINGLE_SOURCE_A。

Remarks

(1) 设置当前测量信号源是单信号源还是双信号源，Start 和 Stop 信号分别是 SOURCE_A 还是 SOURCE_B。

(2) 设置用户端通道单元和板卡内部测量单元的矩阵关系，此时，用户端通道单元和板卡内部测量单元为一一对应关系。矩阵关系请参考下面 Example。

注意：

1) 若使用该函数设置测量信号源，则默认使用的时间测量单元（TMU）为 MU1；

2) 该函数必须在 *Start* 和 *Stop* 函数之前设置, 保证实际设置到硬件的 TMU 和 *Start* (*Stop*) 函数中参数相关的 TMU 保持一致;

3) 由 2) 延伸, 如果 *SetInSource* 在 *Start* (*Stop*) 函数之后设置, 可能会导致两者的 TMU 不匹配, 在 *Measure* 函数中会对两者进行比较, 如果不一致, 则不进行测量, 返回异常值;

4) 系统每次测试结束后 (*InitAfterTestFlow* 函数) 后调用 *SetInSource* (*QTMUe_SINGLE_SOURCE_A*), 恢复默认单信号状态。

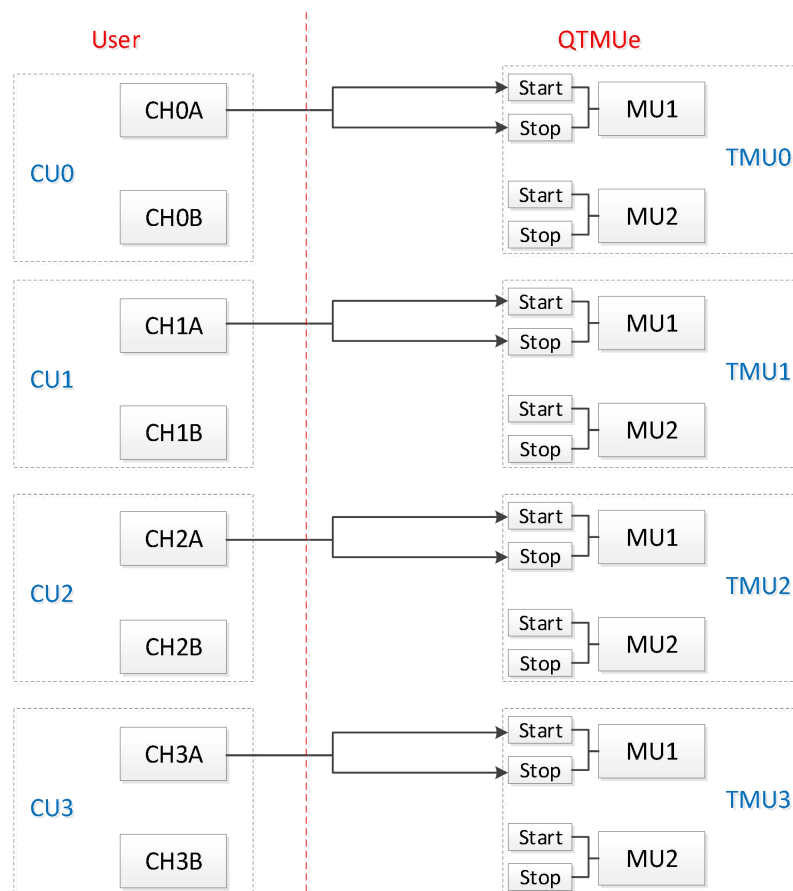
5) 如果需要设置复杂的矩阵关系, 请参考函数 *SetMatrix*。

Example

例 1: *qtmue0* 的测量信号源为单信号, 且 *Start* 和 *Stop* 信号源都是 *SOURCE_A*。

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A); // Singlesource
```

对应矩阵关系如下:

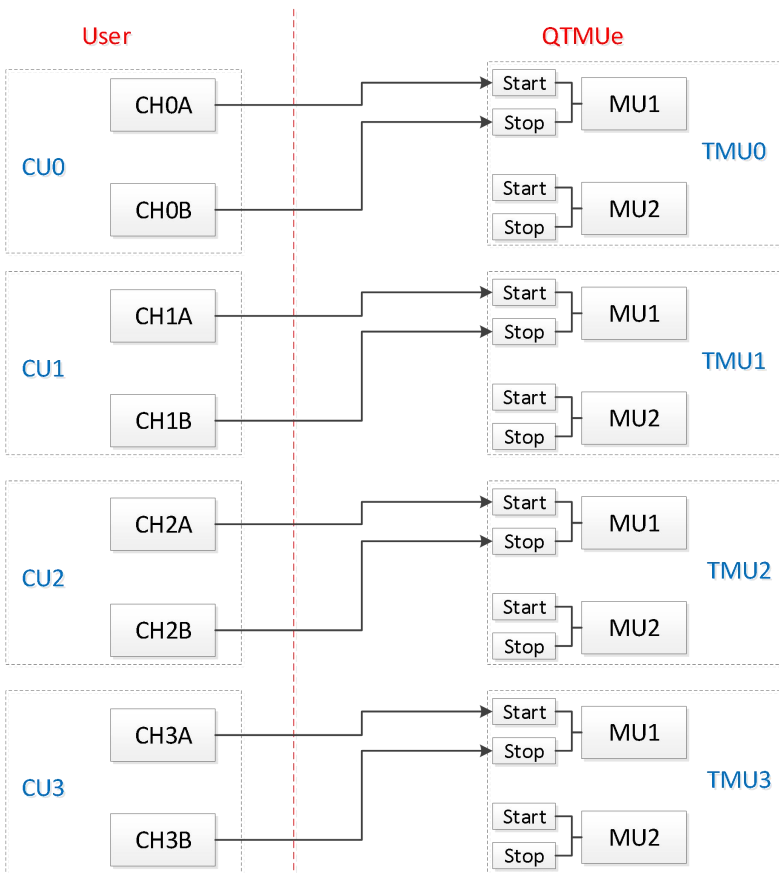


例 2: *qtmue0* 的测量信号源为双信号, 且 *Start* 和 *Stop* 信号源分别是

SOURCE_A 和 SOURCE_B。

```
qtmue0.SetInSource(QTMUe_DUAL_SOURCE_START_A); //Dualsource
```

对应矩阵关系如下：



3.5 SetMatrix()

```
int SetMatrix(  
QTMUe_TMU TmuSel,  
QTMUe_CHANNEL StartChSel,  
QTMUe_CHANNEL StopChSel,  
Int siteNo = QTMUe_ALLSITE);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

StartChSel

设置 Start 信号源通道。QTMUe 每个物理通道分别有 CHA 和 CHB。

QTMUe_CHANNEL_A	逻辑通道包含的所有物理通道的 CHA
QTMUe_CHANNEL_B	逻辑通道包含的所有物理通道的 CHB
QTMUe_CHANNEL_0A	物理通道 0 的 CHA
QTMUe_CHANNEL_1A	物理通道 1 的 CHA
QTMUe_CHANNEL_2A	物理通道 2 的 CHA
QTMUe_CHANNEL_3A	物理通道 3 的 CHA
QTMUe_CHANNEL_0B	物理通道 0 的 CHB
QTMUe_CHANNEL_1B	物理通道 1 的 CHB
QTMUe_CHANNEL_2B	物理通道 2 的 CHB
QTMUe_CHANNEL_3B	物理通道 3 的 CHB

StopChSel

设置 Stop 信号源通道。QTMUe 每个物理通道分别有 CHA 和 CHB。

QTMUe_CHANNEL_A	逻辑通道包含的所有物理通道的 CHA
QTMUe_CHANNEL_B	逻辑通道包含的所有物理通道的 CHB
QTMUe_CHANNEL_0A	物理通道 0 的 CHA
QTMUe_CHANNEL_1A	物理通道 1 的 CHA
QTMUe_CHANNEL_2A	物理通道 2 的 CHA
QTMUe_CHANNEL_3A	物理通道 3 的 CHA
QTMUe_CHANNEL_0B	物理通道 0 的 CHB
QTMUe_CHANNEL_1B	物理通道 1 的 CHB
QTMUe_CHANNEL_2B	物理通道 2 的 CHB
QTMUe_CHANNEL_3B	物理通道 3 的 CHB

siteNo

工位选择，范围[-1, 250]，若超出范围，没有进行矩阵设置，函数返回异常值-1。

QTMUe_ALLSITE 即-1，所有工位

非负值 N 某一个工位

默认值为 QTMUe_ALLSITE (-1)。

注意：

- 1) 当 *StartChSel* 或 *StopChSel* 为 QTMUe_CHANNEL_A 或 QTMUe_CHANNEL_B 时，*siteNo* 必须为 QTMUe_ALLSITE，可省缺；
- 2) 当 *StartChSel* 为 QTMUe_CHANNEL_A 或 QTMUe_CHANNEL_B 时，*StopChSel* 也必须为 QTMUe_CHANNEL_A 或 QTMUe_CHANNEL_B；
- 3) 当 *StartChSel* 为 QTMUe_CHANNEL_0A~QTMUe_CHANNEL_3B 时，*stopChSel* 也必须是 QTMUe_CHANNEL_0A~QTMUe_CHANNEL_3B，且 *siteNo* 必须为具体工位号（非负数），不能为 QTMUe_ALLSITE (-1)。

Remarks

设置用户端通道单元和板卡内部测量单元的矩阵关系，矩阵关系请参考下面 Example。

注意：

- 1) 该函数和 *SetInSource* 都是设置矩阵，实际硬件以最后一次设置为准；
- 2) 该函数设置矩阵后，一直有效，除非设置 *SetInSource*；
- 3) 该函数必须在 *Start* 和 *Stop* 函数之前设置，保证实际设置到硬件的 TMU 和 *Start* (*Stop*) 函数中参数相关的 TMU 保持一致；
- 4) 由 3) 延伸，如果 *SetMatrix* 在 *Start* (*Stop*) 函数之后设置，可能会导致两者的 TMU 不匹配，在 *Measure* 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 5) 如果 *SetMatrix* 对不同工位设置不同的 MU，那么 *Start*、*Stop*、*Measure* 函数必须使用串行工位宏对每个工位的 MU 单独设置，否则在 *Measure* 函数中进行的判断，会不满足条件，不进行测量，返回异常值；
- 6) 仅当 *siteNo* 为 QTMUe_ALLSITE 时，该函数支持 Group 设置。

Example

双工位配置，SITE_1 包含通道 0 和 1，SITE_2 包含通道 2 和 3，qtmue0 包含通道 0 和 2，qtmue1 包含通道 1 和 3。

```
QTMUe qtmue0("S23_0, S23_2");
QTMUe qtmue1("S23_1, S23_3");
```

例 1：qtmue0 所有工位包含的物理通道都选择测量单元为 MU1，且 Start 和 Stop 信号源都是 QTMUe_CHANNEL_A。

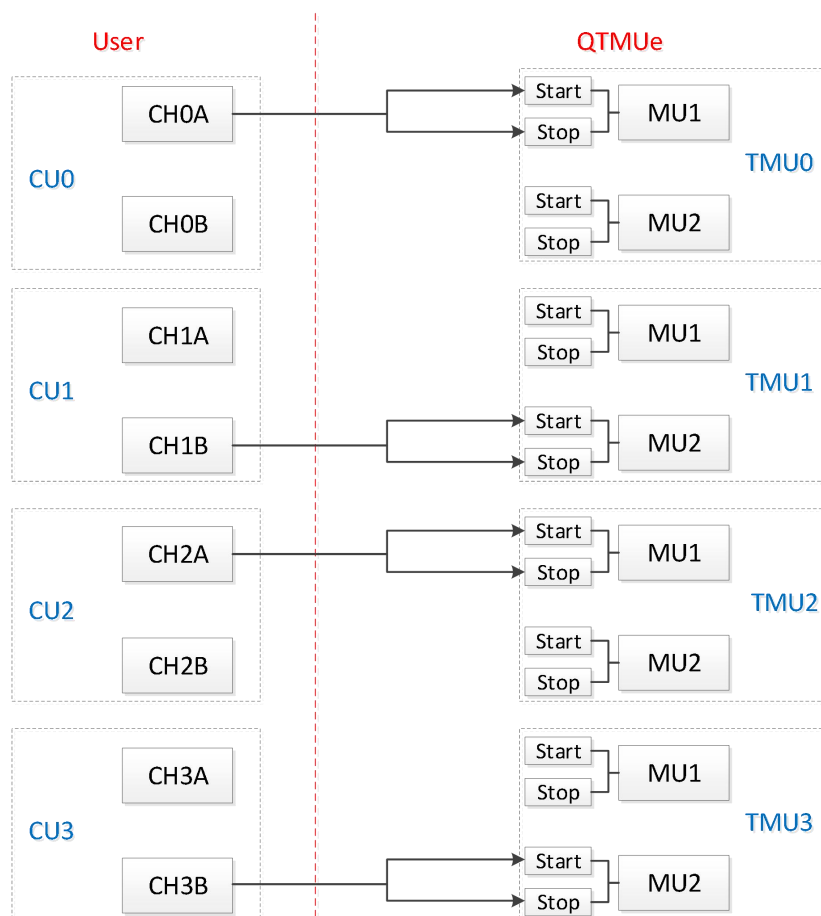
qtmue1 所有工位包含的物理通道都选择测量单元为 MU2，且 Start 和 Stop 信号源都是 QTMUe_CHANNEL_B。

```
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_A, QTMUe_CHANNEL_A,
QTMUe_ALLSITE);
qtmue1.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B,
QTMUe_ALLSITE);
```

上述 qtmue0 设置的矩阵和下面 SetInSource 设置的矩阵相同

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A); // Singlesource
```

对应矩阵关系如下：



例 2：qtmue0SITE_1 对应的 MU1，Start 和 Stop 测量信号分别为

QTMUe_CHANNEL_0A 和 QTMUe_CHANNEL_1A;

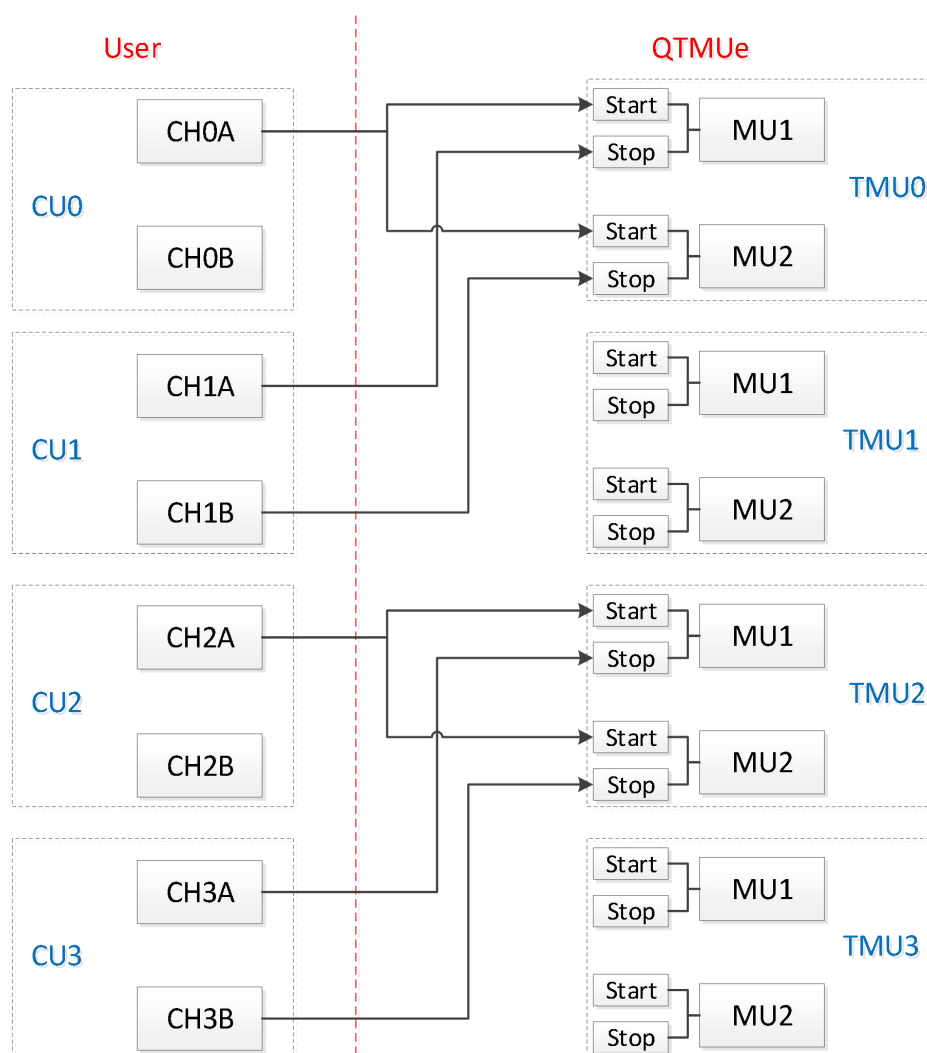
SITE_1 对应的 MU2, Start 和 Stop 测量信号分别为 QTMUe_CHANNEL_0A 和 QTMUe_CHANNEL_1B;

SITE_2 对应的 MU1, Start 和 Stop 测量信号分别为 QTMUe_CHANNEL_2A 和 QTMUe_CHANNEL_3A;

SITE_2 对应的 MU2, Start 和 Stop 测量信号分别为 QTMUe_CHANNEL_2A 和 QTMUe_CHANNEL_3B;

```
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3A, 1);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3B, 1);
```

对应矩阵关系如下:



3.6 Start()

```
void Start(  
QTMUe_TMU TmuSel=QTMUe_MU1,  
QTMUe_VRANGE Vrange= QTMUe_50V,  
QTMUe_SLOPE Slope = QTMUe_POS,  
double TriggerLevel = 0.0,  
QTMUe_FILTER Filter = QTMUe_FILTER_PASS,  
UINT TriggerWidth = 200,  
UINT HoldoffTime = 0,  
UINT HoldoffNum = 0);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

VRange

设置信号电压量程。

QTMUe_50V

QTMUe_25V

QTMUe_10V

QTMUe_5V

QTMUe_N2P6V

默认值为 QTMUe_50V。

Slope

设置比较器的触发极性。

QTMUe_POS 上升沿触发

QTMUe_NEG 下降沿触发

默认值为 QTMUe_POS。

TriggerLevel

设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

Filter

设置滤波器，此参数在 QTMUe_N2P6V 量程下无效。

QTMUe_FILTER_PASS	直通通路，不使用滤波器
QTMUe_FILTER_10MHz	10MHz 低通滤波器
QTMUe_FILTER_1MHz	1MHz 低通滤波器
QTMUe_FILTER_100KHz	100KHz 低通滤波器

默认值为 QTMUe_FILTER_PASS。

注意：

- 1)如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；
- 2)被测信号为方波信号时不建议使用输入滤波器。

TriggerWidth

设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。默认值为 200。

如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。例如：输入值为 18，内部处理为 20。

注意：

被测信号到达触发电平 TriggerLevel 之后，若在 TriggerWidth 时间之内一直高于（上升沿）或低于（下降沿）TriggerLevel，则满足触发条件，否则，继续等待下次触发电平。

HoldoffTime

设置延迟触发时间，取值范围 0~1e9，步进为 5，单位 nS。默认值为 0

如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。
例如：输入值为 18，内部处理为 20。

HoldoffNum

忽略的 START 事件个数，取值范围 0~65535，默认值为 0。

Remarks

设置 START 信号。

注意：

当进行单信号测量时，Start 和 Stop 函数的参数 VRange、Filter 也必须设置一致。

Example

例 1：设置 qtmue0 的 START 信号电压量程为 10V，上升沿 3.5V 触发，使用 1MHz 滤波器，采用 MU1 测量。

```
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 3.5, QTMUe_FILTER_1MHz);
```

例 2：设置 qtmue0 的 START 信号电压量程为 5V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10nS，延迟触发时间为 0，忽略 1 个 START 事件个数，采用 MU2 测量。

```
qtmue0.Start(QTMUe_MU2, QTMUe_5V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS, 10, 0, 1);
```

3.7 Stop()

```
void Stop(  
QTMUe_TMU TmuSel=QTMUe_MU1,  
QTMUe_VRANGE Vrange= QTMUe_50V,  
QTMUe_SLOPE Slope = QTMUe_POS,  
double TriggerLevel = 0.0,  
QTMUe_FILTER Filter = QTMUe_FILTER_PASS,  
UINT TriggerWidth = 200,  
UINT HoldoffTime = 0,  
UINT HoldoffNum = 0);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

VRange

设置信号电压量程。

QTMUe_50V

QTMUe_25V

QTMUe_10V

QTMUe_5V

QTMUe_N2P6V

默认值为 QTMUe_50V。

Slope

设置比较器的触发极性。

QTMUe_POS 上升沿触发

QTMUe_NEG 下降沿触发

默认值为 QTMUe_POS。

TriggerLevel

设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

Filter

设置滤波器，此参数在 QTMUe_N2P6V 量程下无效。

QTMUe_FILTER_PASS 直通通路，不使用滤波器

QTMUe_FILTER_1MHz 1MHz 低通滤波器

QTMUe_FILTER_10MHz 10MHz 低通滤波器

QTMUe_FILTER_100KHz 100KHz 低通滤波器

默认值为 QTMUe_FILTER_PASS。

注意：

- 1) 如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；
- 2) 被测信号为方波信号时不建议使用输入滤波器。

TriggerWidth

设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。默认值为 200。

如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。例如：输入值为 18，内部处理为 20。

注意：

被测信号到达触发电平 TriggerLevel 之后，若在 TriggerWidth 时间之内一直高于（上升沿）或低于（下降沿）TriggerLevel，则满足触发条件，否则，继续等待下次触发电平。

HoldoffTime

设置延迟触发时间，取值范围 0~1e9，步进为 5，单位 nS，默认值为 0。

如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍，例如：输入值为 18，内部处理为 20。

HoldoffNum

忽略的 START 事件个数，取值范围 0~65535，默认值为 0。

Remarks

设置 STOP 信号。

注意：

- 1) 当进行单信号测量时，Start 和 Stop 函数的参数 VRange、Filter 也必须设置一致。
- 2) 测量信号频率时，此函数可以不用调用。

Example

例 1：设置 qtmue0 的 STOP 信号电压量程为 10V，下降沿 3.5V 触发，使用 1MHz 滤波器，采用 MU1 测量。

```
qtmue0.Stop(QTMUe_MU1,QTMUe_10V,QTMUe_NEG,3.5,QTMUe_FILTER_1MHz);
```

例 2：设置 qtmue0 的 STOP 信号电压量程为 5V，上升沿 1.5V 触发，不使用滤波器，忽略 2 个 STOP 事件个数，延迟触发时间为 0，触发脉宽为 10nS，采用 MU2 测量。

```
qtmue0.Stop(QTMUe_MU2,QTMUe_5V,QTMUe_POS,1.5,QTMUe_FILTER_PASS,10,0,2);
```

3.8 SetArm()

```
int SetArm(  
QTMUe_TMU TmuSel=QTMUe_MU1,  
QTMUe_TRIGGER_SOURCE Source = QTMUe_ARM_SOURCE,  
QTMUe_ARM_MODE TriggerMode = QTMUe_EDGE,  
QTMUe_SLOPE Slope = QTMUe_POS,  
UINT Holdoff = 0,  
UINT ArmNum = 1);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

Source

设置外部触发源，每个物理通道有一个触发输入。

QTMUe_ARM_SOURCE 逻辑通道包含的所有物理通道的触发输入

QTMUe_ARM_SOURCE_0 物理通道 0 的触发输入

QTMUe_ARM_SOURCE_1 物理通道 1 的触发输入

QTMUe_ARM_SOURCE_2 物理通道 2 的触发输入

QTMUe_ARM_SOURCE_3 物理通道 3 的触发输入

默认值为 QTMUe_ARM_SOURCE。

TriggerMode

设置触发模式。

QTMUe_EDGE 边沿触发

QTMUe_LEVEL 电平触发

默认值为 QTMUe_EDGE。

Slope

设置比较器的触发极性/电平。

QTMUe_POS 上升沿（高电平）触发

QTMUe_NEG 下降沿（低电平）触发

默认值 QTMUe_POS。

Holdoff

设置边沿计数个数/电平计数时间。默认为 0。

当为边沿触发时，取值范围为 0~65535。

当为电平触发时，取值范围 0~327675，步进为 5nS，如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。例如：输入值为 18，将当作 20 处理。

ArmNum

设置触发次数，范围为 1，当前仅支持 1 次外部触发。

默认值为 1，参数可不填写。

Remarks

设置外部触发条件。

注意：

- 1) 外部触发源的设置范围为：2V≤高电平≤5.5V，0≤低电平≤0.8V。
- 2) 如果使用外部触发，则 *SetArm* 函数的参数 *TmuSel* 必须和 *Start*、*Stop* 设置一样。
- 3) 不调用该函数时，默认为自动触发。

Example

例 1：设置 qtmue0 触发源为外部触发，边沿触发，上升沿触发，采用 MU1 测量。

```
qtmue0.SetArm(QTMUe_MU1, QTMUe_ARM_SOURCE, QTMUe_EDGE, QTMUe_POS);
// external trigger, edge trigger, rising edge
```

3.9 StopArm()

```
int StopArm(
QTMUe_TM U TmuSel=QTMUe_MU1);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

Remarks

恢复自动触发模式，且 *SetArm* 函数中的参数 *TriggerMode*、*Slope*、*Holdoff*、*ArmNum* 恢复默认值。

3.10 Measure()

```
void Measure(
    QTMUe_TMU TmuSel=QTMUe_MU1,
    QTMUe_MEAS_TYPE MeasType = QTMUe_MEAS_TIME,
    UINT SampleNum = 1,
    Double msTimeout = 1,
    QTMUe_TRANGE Trange = QTMUe_TRANGE_MS);
```

Parameters

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

MeasType

设置测量类型。

QTMUe_MEAS_TIME 测量时间

QTMUe_MEAS_FREQ 测量频率

QTMUe_MEAS_HIGH_DUTY 测量高电平在一个周期之内所占的时间
比率

QTMUe_MEAS_LOW_DUTY 测量低电平在一个周期之内所占的时间
比率

QTMUe_MEAS_EVENT 测量事件

默认值为 QTMUe_MEAS_TIME。

SampleNum

设置采样个数，取值范围为 1~65535，默认值为 1。

当 MeasType 为 QTMUe_MEAS_TIME 类型时，SampleNum 为采样次数；

当 MeasType 为 QTMUe_MEAS_FREQ 类型时，SampleNum 为周期数；

当 MeasType 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时, SampleNum 为跳变沿个数;

当 MeasType 为 QTMUe_MEAS_EVENT 类型时, SampleNum 为事件个数 (包含 start 和 stop)。

注意：

当 MeasType 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时, 为保证正确测试, 要求 SampleNum 不小于 3 使得采样至少一个周期。并且, Start()和 Stop()的 slope 设置必须一个为上升沿, 另一个为下降沿, 函数 start()和 stop()中的参数 HoldoffNum 必须设置为 0。

msTimeout

设置测量等待时间, 等待 QTMUe 完成设定次数采样, 取值范围 0.001~2e5, 单位 mS, 默认值为 1mS。

注意：

如果超范围, 默认设置为 1。

Trange

设置测量时间量程。

QTMUe_TRANGE_MS	mS 级量程档 (5nS~40S)
QTMUe_TRANGE_US	uS 级量程档 (800nS~4mS)
QTMUe_TRANGE_NS	nS 级量程档 (5nS~950nS)

默认值为 QTMUe_TRANGE_MS。

注意：

MeasType 为 QTMUe_MEAS_HIGH_DUTY 、 QTMUe_MEAS_LOW_DUTY 或 QTMUe_MEAS_EVENT 时, Trange 必须为 QTMUe_TRANGE_MS。

MeasType 为 QTMUe_MEAS_FREQ 时, SampleNum 个周期的总时间要在对应的 Trange 测量范围内。

Remarks

启动测量。

注意：

- 1) 如果 Measure 函数设置的 MU 没有在 SetMatrix（或 SetInSource）函数中设置，则不进行测量，返回异常值；
- 2) Measure 函数设置的 MU 在 SetMatrix（或 SetInSource）函数中设置，但该 MU 没有在 Start 或 Stop 函数中设置，则不进行测量，返回异常值。

Example

例 1：设置 qtmue0 测量时间，使用 MU1 进行测量，采样点数为 1，超时时间为 2mS。

```
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME,1,2);
```

3.11 GetMeasureResult()

```
double GetMeasureResult(  
UINT siteNo,  
int retType = AVERAGE_RESULT,  
QTMUe_TMU TmuSel = QTMUe_MU1,  
UINT ArmNo = 0);
```

Parameters

siteNo

工位号，选择要读取测量结果的工位。0 代表 Site1，1 代表 Site2.....

retType

返回结果类型。

AVERAGE_RESULT

非负值 N， $0 \leq N \leq \text{SampleNum}-1$

默认值为 AVERAGE_RESULT。

当 Measure() 函数中的 *MeasType* 为 QTMUe_MEAS_TIME 且 *retType* 为

AVERAGE_RESULT 时，返回时间平均值；*retType* 为非负值 N 时，返回第 N+1 次采样的测量值。单位为 uS。

当 Measure() 函数中的 *MeasType* 为 QTMUe_MEAS_FREQ 类型时，*retType* 参数无效，返回频率值，单位 KHz。

当 Measure() 函数中的 *MeasType* 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时，*retType* 参数无效，返回高低占空比，单位%。

当 Measure() 函数中的 *MeasType* 为 QTMUe_MEAS_EVENT 且 *retType* 为 AVERAGE_RESULT 时，返回满足触发条件的事件个数；*retType* 为非负值 N 时，返回第 N+1 次事件的时刻，单位为 uS。

TmuSel

设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

注意：

必须和测量时 Start() 和 Stop() 函数中设置的 TmuSel 一致。

ArmNo

获取第 N+1 次外部触发的测量结果，取值范围为 0。当前仅支持获取第 1 次外部触发的结果。

默认值为 0，参数可不填写。

Remarks

根据 Measure() 函数中的 *MeasType* 参数类型，获取相应的测量结果。

Example

例 1：qtmue0 获取 SITE_1 的时间测量结果。

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_50V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS);
//signal input to CHA, select 50V range, trigger=1.5V, rising edge
qtmue0.Stop(QTMUe_MU1, QTMUe_50V, QTMUe_POS, 13.5, QTMUe_FILTER_PASS);
//50V range, trigger=13.5V, rising edge
qtmue0.Connect(QTMUe_RELAY_CHA,500); //connect CHA relay
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 10, 20, QTMUe_TRANGE_MS);
//sampleNum=10, Timeout=20mS

qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1); // Get average time of site1
qtmue0.GetMeasureResult(0, 5, QTMUe_MU1); // Get the of sixth point time of site1
```

例 2：qtmue0 获取 SITE_1 的高占空比测量结果。

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.0, QTMUe_FILTER_PASS);
qtmue0.Connect(QTMUe_RELAY_CHA, 500); //connect CHA relay
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_HIGH_DUTY, 20, 10,
QTMUe_TRANGE_MS);
//measure high duty cycle, timeout=10mS, sample 20 edges

qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1); // Get high duty of site1.
```

3.12 SetAlarmMask()

```
void SetAlarmMask(
UINT siteNo,
bool maskStatus = true);
```

Parameters

siteNo

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

maskStatus

bool 型变量，可设置为：

true: 屏蔽该工位的 Alarm 信息。

false: 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽指定工位的 Alarm 信息。

Example

```
qtmue0.SetAlarmMask(0,true);           // mask the Alarm information of SITE_1
// The alarm information about overranging voltage scale is not output
qtmue0.Start(QTMUe_MUI,QTMUe_10V,QTMUe_POS,15,QTMUe_FILTER_1MHz);
```

3.13 GetAlarmMask()

```
bool GetAlarmMask(UINT siteNo);
```

Parameters

siteNo

UINT 型变量, 选择读取 Alarm 状态的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息;

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取指定工位的 Alarm 状态。

Example

```
bool status = qtmue0.GetAlarmMask(0); // Get Alarm status of SITE_1
```

3.14 SiteBindModify ()

```
int SiteBindModify(char* modifySiteList);
```


Parameters

modifySiteList

工位重配置的字符串，格式为：“原工位号 1:目标工位号 1,原工位号 2:目标工位号 2, ...”，原工位和目标工位之间用冒号分隔，多组修改工位间用逗号分隔。

原工位号和目标工位号最小为 0（SITE_1），最大为 250（SITE_251）。

例如，将 SITE_1 的通道修改到 SITE_3，SITE_2 的通道修改到 SITE_4，对应的字符串格式为“0:2,1:3”。

Return Values

返回 0：工位重配置成功；

返回非 0：工位重配置失败。

注意：

以下情况会导致工位重配置失败：

- (1) 多组重配置工位之间必须以逗号分隔，第一个字符和最后一个字符不能为逗号，两个逗号间必须有工位修改信息；
- (2) 原工位号和目标工位号之间必须以冒号分隔，只能有一个冒号，且冒号前后必须都是数字，不能包含其它字符；
- (3) 原工位号和目标工位号必须大于等于 0 且小于系统最大工位数 251；
- (4) 不可以对定义为 NO_SITE 的通道进行工位重配置；
- (5) 同一次修改时（调用一次函数），同一个工位不能被修改两次，如“0:1,0:2”；
- (6) 同一次修改时（调用一次函数），不同工位不能被修改到同一个工位上，如“0:2,1:2”；
- (7) 目标工位如果已经进行过通道配置，此次也必须被修改。示例如下：

```
QTMUe qtmue0("S4_0,S4_1");
```

初始的工位配置如下：

```
STSSetMultiSiteBind(MD_QTMUe, SITE_1, "S4_0,S4_2");
```

```
STSSSetMultiSiteBind(MD_QTMUe, SITE_2, "S4_1,S4_3");
```

工位重配置的正确做法（将 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4）：

```
qtmue0.SiteBindModify("0:2,1:3")
```

工位重配置的错误做法（仅将 SITE_1 重配置为 SITE_2）：

```
qtmue0.SiteBindModify("0:1")
```

Remarks

对通道的工位配置信息进行重配置，未进行工位重配置的通道工位信息保持不变。

注意：

该函数的重配置过程不受工位有效性信息的影响，例如将 SITE_1 重配置成 SITE_2，无论当前 SITE_1 是否无效，都会将 SITE_1 对应的物理通道配置到 SITE_2。

Example

通道定义和初始工位配置如下：

```
QTMUe qtmue0("S4_0,S4_1");
QTMUe qtmue1("S4_2,S4_3");
QTMUe gpqtmue0("S4_0,S4_1,S4_2,S4_3");
```

```
STSSSetMultiSiteBind(MD_QTMUe, SITE_1, "S4_0,S4_2");
STSSSetMultiSiteBind(MD_QTMUe, SITE_2, "S4_1,S4_3");
```

例 1：设置 qtmue0 的 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4。

```
qtmue0.SiteBindModify("0:2,1:3");
```

注：由于 qtmue0 工位信息重新进行了配置，gpqtmue0 由原来的 Group Pin 变为了 Multisite Pin。

例 2：设置 qtmue0 的 SITE_1 重配置为 SITE_2，SITE_2 重配置为 SITE_3。

```
qtmue0.SiteBindModify("0:1,1:2");
```

例 3：设置 qtmue0 的 SITE_1 重配置为 SITE_3，SITE_2 保持不变。

```
qtmue0.SiteBindModify("0:2");
```

例 4: 设置 gpqtmue0 的 SITE_1 重配置为 SITE_3, SITE_2 重配置 SITE_4。

```
gpqtmue0.SiteBindModify("0:2,1:3");
```

注: 由于 gpqtmue0 由 qtmue0 和 qtmue1 组成, 因此 qtmue0 和 qtmue1 对应的工位信息都发生了变化。

3.15 获取板卡固有信息函数

3.15.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteNo,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteNo

UINT 型变量, 选择读取板卡 SN 号的工位。0 代表 Site1, 1 代表 Site2.....

boardSN

char 型变量, 存储板卡的 SN 号。未存储返回 N/A。

由用户定义, 如: char boardSN[255]={0}。

snSize

UINT 型变量, 数组 *boardSN* 的大小, 正整数。

Return Values

返回 0: 读取成功;

返回-1: 工位无效, 读取失败。

Remarks

获取板卡 SN 号。

Example

```
char boardSN[255] = {0};  
qtvue0.GetBoardSN(0, boardSN, 255);    // Get the board serial number
```

3.15.2 GetBoardHDRev()

```
int GetBoardHDRev(  
UINT siteNo,  
char*hardRev,  
UINT revSize);
```

Parameters

siteNo

UINT 型变量，选择读取板卡硬件版本的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

获取板卡硬件版本号。

Example

```
char hardRev [255] = { 0 };  
qtmue0.GetBoardHdRev(0, hardRev, 255); // Get the board hardware revision
```

3.16 获取校准信息函数

3.16.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量, 选择要获取校准日期的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

返回该通道校准发生的日期。未存储或工位无效时返回 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取指定通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = qtmue0.GetCalibrationTime(0); // Get calibration date of SITE_1
```

3.16.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = qtmue0.GetCalibrationTemperature(0); //Get calibration temperature of SITE_1
```

3.16.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表

Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = qtmue0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

3.16.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择要获取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果，为枚举值。

CAL_PASS：校准 PASS

CAL_FAIL：校准 FAIL

STS_NO_RESULT_RECORD：未存储或该工位无效。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取指定通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = qtmue0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

3.16.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;
```


3.16.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteNo,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteNo

UINT 型变量，选择要获取校准表信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表的 SN 号。未存储返回 N/A。

由用户定义，如：char meterSN[255]={0}。

默认值为 NULL，不获取校准表的 SN 号。

snSize

UINT 型变量，数组 meterSN 的大小，正整数。

默认值为 0，不获取校准表的 SN 号。

Return Values

返回该通道校准表的类型，为枚举值。

STS_KEITHLEY2000: Keithley2000

STS_AGILENT34401: Agilent34401

STS_AGILENT3458A: Agilent3458A

STS_NO_METER_RECORD: 未存储或该工位无效。

Remarks

读取该通道校准表的类型和校准表的 SN 号。

Example

例 1：分别获取 qtmue0 下 SITE_1 的校准表的类型和 SN 号。

```
char meterSN[255] = {0};  
CAL_METER calMeter;  
calMeter = qtmue0.GetCalibrationMeter(0, meterSN, 255);
```

3.16.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteNo,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* hardRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteNo

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

hardRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回“N/A”。

由用户定义，如：char hardRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 boardHdRev 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的硬件版本信息，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = qtmue0.GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

3.16.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(
UINT siteNo,
char *softRev,
UINT revSize);
```

Parameters

siteNo

UINT 型变量，选择要获取校准使用的软件版本的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。未存储返回 N/A。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 softRev 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

读取该通道校准时使用的软件版本。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Example

```
char softRev[20] = { 0 };  
qtmue0.GetCalibrationSoftRev(0, softRev, 20); // Get calibration software revision of SITE_1
```

3.16.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

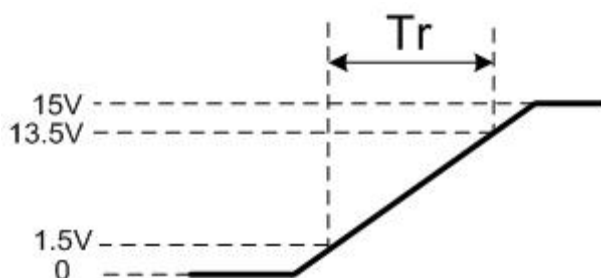
Example

```
intslotID = qtmue0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

3.17 编程范例

3.17.1 测试时间

例 1：测试下图所示信号的上升时间 T_r 。



测试程序：

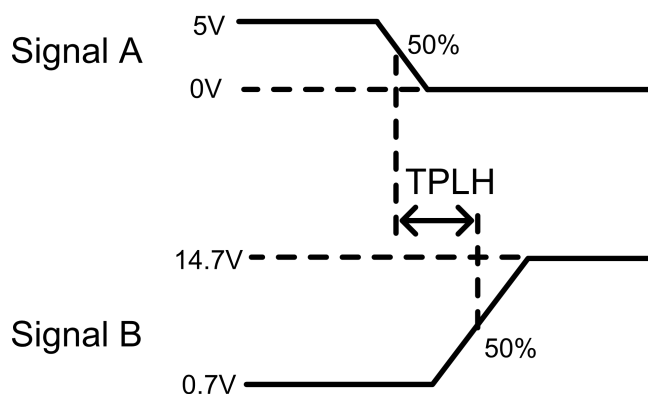
```
double val[4] = { 0.0 };  
int i = 0;
```

```
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 13.5, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);

for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Tr->SetTestResult(i, 0, val[i]); //uS
}

qtmue0.Disconnect(QTMUe_RELAY_CHA,100); //disconnect CHA relay
```

例 2：测试下图所示信号的时间间隔 TPLH。



测试程序:

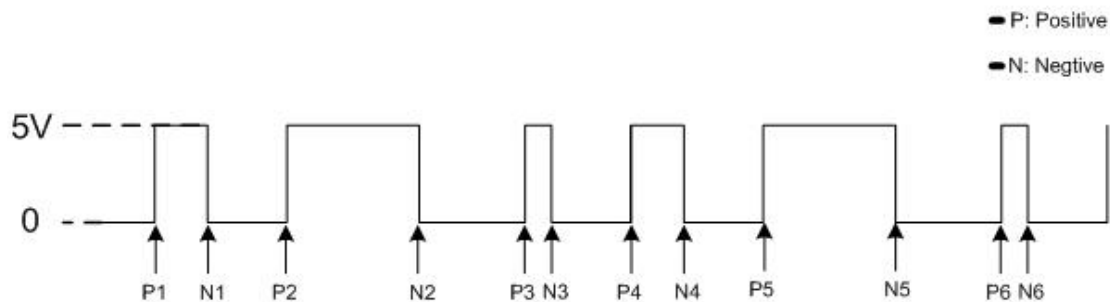
```
double val[4] = {0.0};
int i=0;

qtmue0.Connect(QTMUe_RELAY_CHAB,500);
qtmue0.SetInSource(QTMUe_DUAL_SOURCE_START_A);
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_NEG, 2.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 7.7, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 10, 1, QTMUe_TRANGE_US);

for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    TPLH ->SetTestResult(i, 0, val[i]); //uS
}

qtmue0.Disconnect(QTMUe_RELAY_CHAB,100); //disconnect CHA and CHB relay
```

例 3：测试下图所示信号 P2 到 N3 的时间间隔。



测试程序:

```
double val[4] = { 0.0 };
int i = 0;

qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.5, QTMUe_FILTER_PASS, 10, 0, 1); // holdoff=1
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.5, QTMUe_FILTER_PASS, 10, 0, 2); // holdoff=2
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 10, 1, QTMUe_TRANGE_US);

for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    TP2N3->SetTestResult(i, 0, val[i]); //uS
}

qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

3.17.2 测试频率

例 1：测试下图所示信号的频率。



测试程序:

```
double val[4] = { 0.0 };
int i = 0;

qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
```

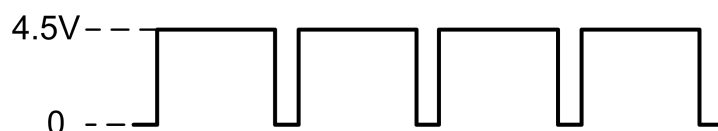
```
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_FREQ, 20,10, QTMUe_TRANGE_US);
//measure frequency, sample 20 cycles,timeout=10mS

for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Freq->SetTestResult(i, 0, val[i]);//KHz
}

qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

3.17.3 占空比

例 1：测试下图所示信号的高占空比。



测试程序：

```
double val[4] = { 0.0 };
int i = 0;

qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_HIGH_DUTY, 20,10,
QTMUe_TRANGE_MS); //measure high duty cycle,timeout=10mS,sample 20 edges

//Apply excitation to generate signal

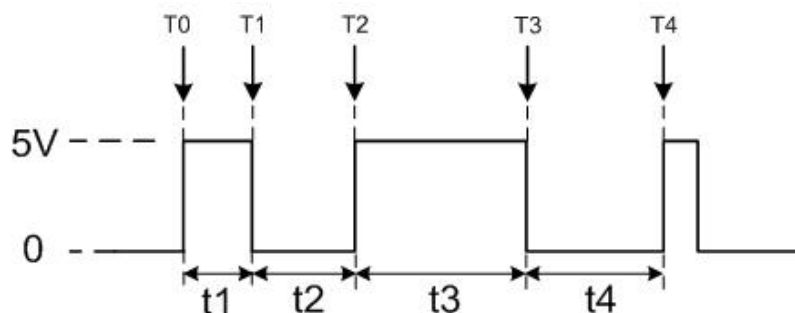
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Duty->SetTestResult(i, 0, val[i]);//%
}

qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

3.17.4 事件计数

例 1：测试下图所示信号，从第一个有效的上升沿开始捕捉，记录并获取

五个跳变时刻（T0、T1、T2、T3、T4）。并计算 t1、t2、t3、t4。



测试程序:

```
double Tevent[5][SITENUM] = { 0.0 };

qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1,QTMUe_MEAS_EVENT, 5,10, QTMUe_TRANGE_MS);
//measure event, timeout=10mS, sample 5 events

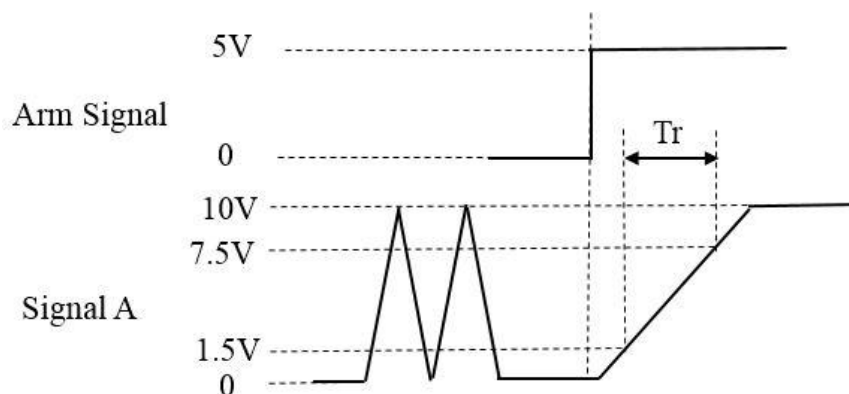
//Apply excitation to generate signal

for (int SITEID = 0; SITEID < SITENUM; SITEID++)
{
    for (int i = 0; i < 5; i++)
    {
        Tevent[i][SITEID] = qtmue0.GetMeasureResult(SITEID, i, QTMUe_MU1);
    }
    t1->SetTestResult(SITEID, 0, (Tevent[1][SITEID] - Tevent[0][SITEID]));
    t2->SetTestResult(SITEID, 0, (Tevent[2][SITEID] - Tevent[1][SITEID]));
    t3->SetTestResult(SITEID, 0, (Tevent[3][SITEID] - Tevent[2][SITEID]));
    t4->SetTestResult(SITEID, 0, (Tevent[4][SITEID] - Tevent[3][SITEID]));
}

qtmue0.Disconnect(QTMUe_RELAY_CHA);
```

3.17.5 外部触发测试时间

例 1：施加 0~5V 的外部 Arm 触发信号，接收到 Arm 触发信号后，测试下图所示信号的上升时间 T_r 。



测试程序:

```
double val[4] = { 0.0 };
int i = 0;

qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 7.5, QTMUe_FILTER_PASS, 10);
qtmue0.SetArm(QTMUe_MU1, QTMUe_ARM_SOURCE, QTMUe_EDGE, QTMUe_POS);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 100,QTMUe_TRANGE_US);
//measure time,timeout=100mS

//Apply excitation to generate external Arm signal

for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Tr->SetTestResult(i, 0, val[i]); //uS
}

qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

3.17.6 复杂矩阵应用实例

例 1：一块板卡单工位设置，用 4 个通道的 8 个 TMU，测试 8 个脉冲信号脉宽。

```
QTMUe qtmue0("S23_0");
QTMUe qtmue1("S23_1");
QTMUe qtmue2("S23_2");
QTMUe qtmue3("S23_3");
```

测试程序:

```
double Tevent[5][SITENUM] = { 0.0 };
qtmue0.Connect(QTMUe_RELAY_CHAB);
```

```

qtmue1.Connect(QTMUe_RELAY_CHAB);
qtmue2.Connect(QTMUe_RELAY_CHAB);
qtmue3.Connect(QTMUe_RELAY_CHAB);

qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue1.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue2.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue3.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.SetMatrix(QTMUe_MU2,
QTMUe_CHANNEL_B,QTMUe_CHANNEL_B,QTMUe_ALLSITE);
qtmue1.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B,
QTMUe_ALLSITE);
qtmue2.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B,
QTMUe_ALLSITE);
qtmue3.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B,
QTMUe_ALLSITE);

qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue1.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue1.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue2.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue2.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue3.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue3.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);

qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue1.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue1.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue2.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue2.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
qtmue3.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue3.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);

qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue1.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue1.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue2.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue2.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue3.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue3.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);

double time1_mu1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double time1_mu2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
double time2_mu1 = qtmue1.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double time2_mu2 = qtmue1.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
double time3_mu1 = qtmue2.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double time3_mu2 = qtmue2.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
double time4_mu1 = qtmue3.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double time4_mu2 = qtmue3.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);

qtmue0.Disconnect(QTMUe_RELAY_CHAB);
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
qtmue2.Disconnect(QTMUe_RELAY_CHAB);
qtmue3.Disconnect(QTMUe_RELAY_CHAB);

```

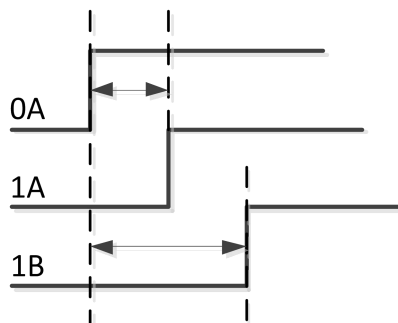
例 2：一块板卡双工位设置，SITE_1 包含通道 0 和 1，SITE_2 包含通道 2

和 3，qtmue0 包含通道 0 和 2，qtmue1 包含通道 1 和 3。

对于 SITE_1，测量 0A 到 1A 的传输延时，同时测量 0A 到 1B 的传输延时。

对于 SITE_2，测量 2A 到 3A 的传输延时，同时测量 2A 到 3B 的传输延时。

(TMU1 和 TMU2 都是上升沿到上升沿)



```
QTMUe qtmue0("S23_0, S23_2");
QTMUe qtmue1("S23_1, S23_3");
```

测试程序:

```
qtmue0.Connect(QTMUe_RELAY_CHA);
qtmue1.Connect(QTMUe_RELAY_CHAB);

qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3A, 1);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3B, 1);

qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);

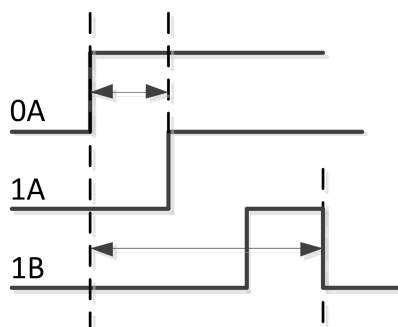
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);

double delayTime1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double delayTime2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
double delayTime3 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU1);
double delayTime4 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU2);

qtmue0.Disconnect(QTMUe_RELAY_CHAB);
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
```

例 3：一块板卡单工位设置，测量 0A 到 1A 的传输延时，同时测量 0A 到 1B 的传输延时。

(TMU1: 上升沿到上升沿; TMU2: 上升沿到下降沿)。



```
QTMUe qtmue0("S23_0");
QTMUe qtmue1("S23_1");
QTMUe qtmue2("S23_2");
QTMUe qtmue3("S23_3");
```

测试程序:

```
qtmue0.Connect(QTMUe_RELAY_CHA);
qtmue1.Connect(QTMUe_RELAY_CHAB);

qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);

qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);

qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 2, QTMUe_FILTER_PASS, 200);

qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);

double delayTime1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double delayTime2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);

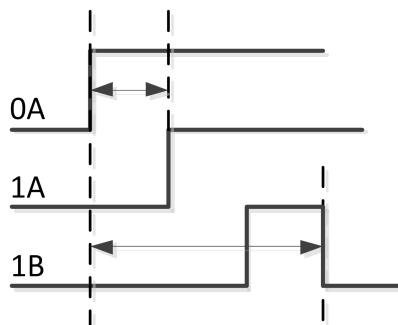
qtmue0.Disconnect(QTMUe_RELAY_CHAB);
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
```

例 4: 一块板卡双工位设置, SITE_1 包含通道 0 和 1, SITE_2 包含通道 2 和 3, qtmue0 包含通道 0 和 2, qtmue1 包含通道 1 和 3。

对于 SITE_1, 测量 0A 到 1A 的传输延时, 同时测量 0A 到 1B 的传输延时。

对于 SITE_2, 测量 2A 到 3A 的传输延时, 同时测量 2A 到 3B 的传输延时。

(TMU1: 上升沿到上升沿; TMU2: 上升沿到下降沿)。



```
QTMUe qtmue0("S23_0, S23_2");
QTMUe qtmue1("S23_1, S23_3");
```

测试程序:

```
qtmue0.Connect(QTMUe_RELAY_CHA);
qtmue1.Connect(QTMUe_RELAY_CHAB);

qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3A, 1);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3B, 1);

qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);

qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 2, QTMUe_FILTER_PASS, 200);

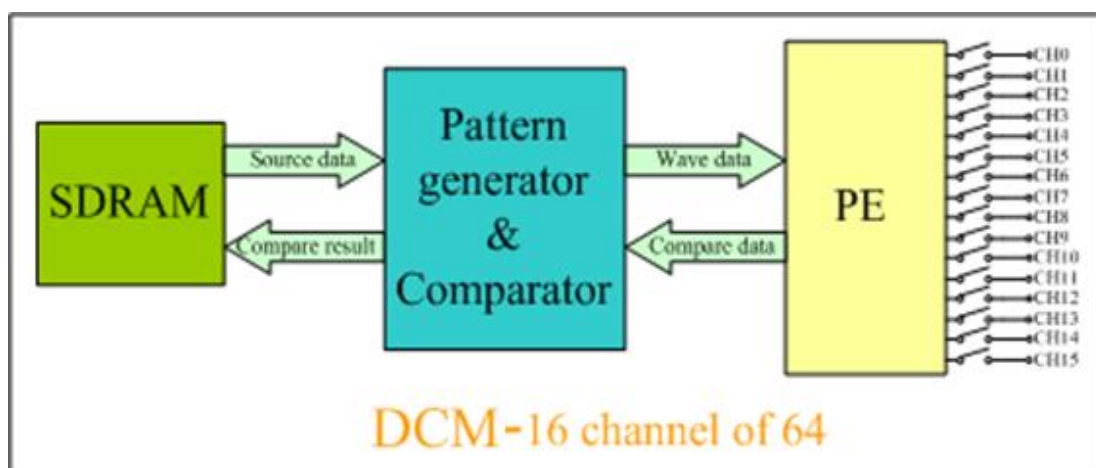
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);

double delayTime1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);
double delayTime2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
double delayTime3 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU1);
double delayTime4 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU2);

qtmue0.Disconnect(QTMUe_RELAY_CHAB);
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
```

4 DCM

DCM 波形发生以控制器为单位，一个控制器控制 16 个通道，一块 DCM 有四个控制器，分别控制 CH0~CH15、CH16~CH31、CH32~CH47、CH48~CH63，如下图所示（其中一个控制器）：



当一个控制器中的多个通道有产生波形的向量（如 CH0、CH1、CH2、CH3、CH4），如果使用 PinGroup（多个管脚的集合）让其中部分通道产生波形（如 CH0、CH1、CH2），向量运行时，同控制器其他 MCU 通道(如 CH3、CH4)会被修改成高阻态，不输出波形。

向量运行时，无效工位对应的 MCU 通道会被置成高阻态。

MCU 功能和 PMU 功能是二选一的，无法同时使用两个功能。

I2C 相关函数是对 MCU 功能进行的封装应用，二者存在相互影响。当 I2C 与其他功能共用通道时，则需要考虑 I2C 功能与其他功能的互相影响，例如边沿设置、电平设置及继电器操作等，共用通道的实际状态以最近一次执行的 MCU 函数或 I2C 函数为准。

在多站的测试程序中，每个站需要使用单独定义 DCM 实例或者变量，不允许多个站使用同一个实例。

4.1 DCM 相关函数

DCM 除 I2C 相关函数外，函数运行错误时的返回值：

返回值	返回值意义	备注
-1	向量文件未加载成功	检查向量文件是否正确，重新加载
-2	管脚组未定义	在向量编辑器中定义管脚组
-3	分离名称不存在	分离集名称只能是向量文件中的分离集名称
-4	主时钟速率超范围	
-5	前沿/后沿超范围	
-6	起始标签不存在	在向量表中定义起始标签
-7	结束标签不存在	在向量表中定义结束标签
-8	指令错误或者不支持	
-9	管脚名称未定义	在向量编辑器中定义管脚名
-10	工位号不存在	工位号必须小于工位总数
-11	读取结果的总行数小于 1	
-12	不存在错误行	最近一次运行向量无 FAIL 行
-13	失效文件写入失败	
-14	电平设置超范围	查阅函数的电平有效范围
-15	偏移量超过下一个标签行	
-16	板卡未找到	检查 DCM 板卡是否插入
-17	PMU 测量出错	
-18	电流值设置超范围	
-19	结束标签在起始标签之前	
-20	地址超范围	
-21	预载入向量段超出限制	
-22	管脚组名称冲突	
-23	管脚组格式错误	
-24	操作数错误	
-25	数组内存不足	

-26	向量行数超限	
-27	输入参数为 NULL	
-28	申请内存出错	
-29	函数使用顺序错误	
-30	没有触发信号	
-32	工位无效	
-33	没有运行向量	
-34	向量正在运行	
-35	通道不存在	
-36	钳位值超限	
-37	非前一次运行向量	获取的 Fail 行不是前一次运行
-38	多个通道连接一个 TMU 模块	
-39	TMU 模块号超限	
-40	通道未连接 TMU	操作的通道没有连接 TMU
-41	失效行超存储容量	
-42	向量中的硬件 Capture 行超限	删除超范围的 Capture
-43	管脚不属于当前实例或者变量	检查是否错误使用管脚
-44	获取校准信息失败	
-45	设置 TMU 参数的通道超过 1	分别设置通道的 TMU 参数
-46	不支持软件 Capture 功能	保存指定行的失效信息时，不支持 GetCaptureData

4.1.1 DCM

DCM dcm;

Remark

定义 DCM。

注意：

对于多站程序，每个 DCM 实例或者变量只能在一个站中使用，即每站只能使用各自的 DCM 实例或者变量。

Example

例 1：定义 DCM，并设置管脚组。

```
DCM dcm; //define DCM
dcm.SetPinGroup("G_I2C","SDA,SCL"); //<Define the pin group
```

例 2：定义两个 DCM，分别设置管脚组。

```
DCM dcm; //define DCM
DCM dcm1; //define another DCM for muti task program
dcm.SetPinGroup("G_I2C", " SCL_DP,SDA_DP "); //<Define the pin group for dcm
dcm1.SetPinGroup("G_I2C ", " SCL_DP,SDA_DP "); //< Define the pin group for dcm1
```

4.1.2 LoadVectorFile()

```
int LoadVectorFile(
const char* lpszFileName,
BOOL bLoadVectorFile = TRUE);
```

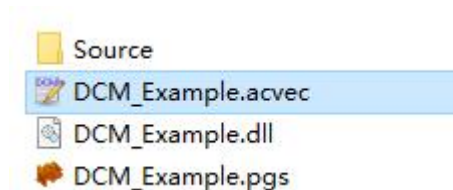
Parameters

lpszFileName

向量文件名。

注意：

该向量文件必须与当前测试程序的 pgs 文件存放在同一目录下。如下图所示。



bLoadVectorFile

是否重复加载向量文件，可选择：

TRUE：每次 load 程序都加载向量文件。

FALSE：同时满足以下条件时，此次 load 程序不再加载向量文件。

1. 前一次 load 程序与此次 load 程序的向量文件名相同；
2. 两次 load 程序之间未修改向量文件；
3. 两次 load 程序之间没有关机或者自检。

默认值为 TRUE。

注意：

推荐在量产时，此参数设置为 TRUE。

Return Values

0：向量文件加载成功

-1：向量文件未加载成功

Remarks

载入向量文件数据到板载存储器内。

注意：

- 1) 除 I2C 函数外，其他函数必须使用此函数加载向量；
- 2) 此函数必须在 UserLoad 函数中使用；
- 3) 多个 DCM 实例或者变量可以使用一个向量文件，只需要使用其中一个实例或者变量加载向量即可；
- 4) 没有单独加载向量的实例或者变量，将共享最先被加载的向量文件；
- 5) 使用不同向量文件时，需要注意不同向量文件的通道不可以交叉，否则相同通道的向量会被覆盖。

Example

例 1：载入向量文件 DCM_Example.acvec 到板载存储器内。

```
//Load the vector file DCM_Example.acvec into the onboard memory  
dcm.LoadVectorFile("DCM_Example.acvec",TRUE);
```

例 2：多个 DCM 实例共享向量文件,dcm 和 dcm2 使用相同向量文件，dcm1 使用另一个向量文件。

```
DCM dcm;  
DCM dcm1;  
DCM dcm2;  
//Load the vector file DCM_Example.acvec into the onboard memory  
dcm.LoadVectorFile("DCM_Example.acvec",TRUE);  
dcm1.LoadVectorFile("DCM_Example_2.acvec",TRUE);
```

4.1.3 SetValidPin()

```
int SetValidPin(const char* lpszPinNameList);
```

Parameters

lpszPinNameList

设置有效的管脚名称表，管脚名称间以“,”分割。

Return Values

- 0: 设置成功
- 9: 管脚名未定义
- 16: 板卡未找到

Remarks

设置一个当前实例或者变量的有效管脚，此函数是为了使用相同向量文件的不同实例间的管脚互不干扰，多站必须使用此函数设置不同站的管脚。

注意：

- 1) 管脚必须在向量文件中定义；
- 2) 此函数必须在 UserLoad 函数中使用；
- 3) 此函数需要在 LoadVectorFile 之后和 SetPinGroup 之前使用；

- 4) 使用相同向量文件的实例，必须使用此函数设置有效的管脚，否则在使用不指定管脚或者管脚组的函数时，每个实例都会检查所有管脚的运行结果。

Example

例 1：设置两个共用向量文件的实例对应的有效管脚。

```
DCM dcm;
DCM dcm1;
//Load the vector file DCM_Example.acvec into the onboard memory
dcm.LoadVectorFile("DCM_Example.acvec",TRUE);
dcm.SetValidPin("SCL_DP,SDA_DP");
dcm1.SetValidPin("SCL1_DP,SDA1_DP");
dcm.SetPinGroup("G_I2C","SCL_DP,SDA_DP");//<The pin group G_I2C can only be used for dcm
dcm1.SetPinGroup("G_SERIAL ","SCL1_DP,SDA1_DP");//<The pin group G_SERIAL can only be used
for dcm1
```

4.1.4 SetPinGroup()

```
int SetPinGroup(
const char*    lpszPinGroupName,
const char*    lpszPinNameList);
```

Parameters

lpszPinGroupName

要设置的管脚组名称。

lpszPinNameList

管脚组包含的管脚名称表，管脚名称间以“,”分割。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 9: 管脚名未定义
- 16: 板卡未找到
- 22: 管脚组名称已经存在

-43: 部分管脚不属于当前 DCM 实例

Remarks

设置一个管脚组。

注意：

- 1) 在使用管脚组之前，必须使用此函数设置管脚组；
- 2) 此函数必须在 UserLoad 函数中使用。
- 3) 管脚组只对当前 DCM 实例或者变量有效。

Example

例 1：设置管脚组 “G_I2C”，管脚组中包含管脚：SCL_DP 和 SDA_DP。

```
dcm.SetPinGroup("G_I2C","SCL_DP,SDA_DP");
```

例 2：两个 DCM 实例分别设置管脚组 “G_I2C”，“G_SERIAL”，管脚组中包含管脚：SCL_DP 和 SDA_DP。

```
dcm.SetPinGroup("G_I2C","SCL_DP,SDA_DP");//<The pin group G_I2C can only be used for dcm  
dcm1.SetPinGroup("G_SERIAL","SCL_DP,SDA_DP");//<The pin group G_SERIAL can only be used for  
dcm1
```

4.1.5 InitMCU()

```
int InitMCU(const char* lpszPinGroup);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

Return Values

0: 初始化成功

- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 16: 板卡不存在

Remarks

初始化指定管脚组中的通道，输入管脚中有效工位的通道会被切换成 MCU 模式。

Example

```
dcm.InitMCU("G_ALLPIN"); //Initialize the controller including the G_ALLPIN pin group
```

4.1.6 SetPeriod()

```
int SetPeriod(  
const char*    lpszTimeSetName,  
double         dPeriod);
```

Parameters

lpszTimeSetName

分离集名称，向量表中定义的分离集名称。

dPeriod

主时钟速率值，取值范围 12 ~ 4000000(4ms)，单位为 nS。分辨率为 1nS。

注意：

仅适用于逻辑版本不低于 0x30 的 DCM 板卡。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 3: 分离集名称未定义

-4: 主时钟速率超范围

-16: 板卡未找到

Remarks

设置分离集的主时钟速率。

注意：

- 1) 向量文件加载时，分离集时钟频率会被重置成向量文件中定义的时钟频率；
- 2) 分离集的时钟频率以最近一次设置为准。

Example

例 1: 设置 DCM 的分离集 Timeset1 的主时钟速率为 100nS。

```
dcm.SetPeriod("Timeset1", 100);
```

4.1.7 SetEdge()

```
int SetEdge(  
const char*    lpszPinGroup,  
const char*    lpszTimeSetName,  
DataFormat    dataFormat,  
IOFormat       ioFormat,  
double         dT1R,  
double         dT1F,  
double         dIOR,  
double         dIOF,  
double         dSTBR);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszTimeSetName

分离集名称，向量表中定义的分离集名称。

dataFormat

驱动数据波形格式代码，可选择：

DCM_DTFT_NRZ: 不归零格式

DCM_DTFT_RZ: 归零格式

DCM_DTFT_RO: 归一格式

DCM_DTFT_SBL: 低电平包围格式

DCM_DTFT_SBH: 高电平包围格式

DCM_DTFT_SBC: 反码环绕格式

ioFormat

比较格式代码，可选择：

DCM_IO_NRZ: 不归零格式

DCM_IO_RO: 归一格式

dTIR

驱动数据前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dTIF

驱动数据后沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dIOR

驱动 I/O 方向前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dIOF

驱动 I/O 方向后沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dSTBR

比较前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 3: 分离集超范围
- 5: 前沿/后沿超范围
- 16: 板卡未找到

Remarks

设置指定分离集的指定通道的时间参数和波形格式。

注意：

- 1) 向量文件加载时，分离集的边沿会被重置为向量文件中定义的边沿；
- 2) 分离集的边沿值以最近一次设置为准；
- 3) 各个边沿的实际含义，请参考硬件手册中对于 DCM 的描述；
- 4) 除 NRZ 波形格式外， $T1F \geq T1R + 4ns$ ；
- 5) 除 NRZ、RZ、RO 的波形格式外， $T1R \geq IOR + 4ns$ ；
- 6) RO 的 IO 格式下， $IOF \geq IOR + 4ns$ 。

Example

例 1：设置管脚组 G_ALLPIN 对应通道的驱动数据波形格式 NRZ，I/O 的格式为 NRZ，驱动数据前沿时刻值 5nS，驱动数据后沿时刻值 10nS，驱动 I/O 方向前沿时刻值 5nS，驱动 I/O 方后沿时刻值为 10ns，比较前沿时刻值 30nS，分离集名称为 Timeset1。

```
dcm.SetEdge("G_ALLPIN", "Timeset1", DCM_DTFT_NRZ, DCM_IO_NRZ, 5, 10, 5, 10, 30);
```

4.1.8Connect()

```
int Connect(const char* lpszPinGroup);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 16: 板卡未插入

Remarks

闭合指定通道的输出继电器。

Example

例 1: 闭合 G_ALLPIN 管脚组对应通道的输出继电器。

```
dcm.Connect("G_ALLPIN");
```

4.1.9 Disconnect()

```
int Disconnect(const char* lpszPinGroup);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

Return Values

- 0: 设置成功

- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 16: 板卡未插入

Remarks

断开指定通道的输出继电器。

Example

例 1: 断开 G_ALLPIN 管脚组对应通道的输出继电器。

```
dcm.Disconnect("G_ALLPIN");
```

4.1.10 SetPinLevel()

```
int SetPinLevel(  
const char*    lpszPinGroup,  
double        dVIH,  
double        dVIL,  
double        dVOH,  
double        dVOL);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

dVIH

驱动高电平，取值范围-1.5~6，单位为 V。

dVIL

驱动低电平，取值范围-1.5~6，单位为 V。

dVOH

比较高电平，取值范围-1.5~6，单位为 V。

dVOL

比较低电平，取值范围-1.5~6，单位为 V。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 14: 电平设置超范围
- 16: 板卡未插入

Remarks

设置指定通道的驱动和比较高、低电平的电压值。

注意：

- 1) 管脚组中所有有效工位的通道会自动切换至 MCU 模式；
- 2) 每个通道只有一个电压值，以最近一次设置为准；
- 3) 此函数会关掉通道的动态负载。

Example

例 1：设置管脚组 G_ALLPIN 对应的通道，驱动高电平 3.3V，驱动低电平 0V，比较高电平 2.4V，比较低电平 0.5V。

```
dcm.SetPinLevel("G_ALLPIN", 3.3, 0, 2.4, 0.5);
```

4.1.11 SetChannelStatus()

```
int SetChannelChannel(  
const char* lpszPinGroup,
```

```
CHANNEL_STATUS ChannelStatus);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

ChanenlStatus

通道管脚状态，可选

DCM_HIGH: 高电平状态

DCM_LOW: 低电平状态

DCM_HIGH_IMPEDANCE: 高阻态

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 14: 电平设置超范围
- 16: 板卡未插入

Remarks

- 1) 设置通道的状态;
- 2) 仅支持逻辑版本不低于 0x30 的 DCM;
- 3) 管脚中有效工位对应的通道将自动切换至 MCU 模式。

Example

例 1: 设置管脚 SDA_DP 所有有效工位的管脚状态为高:

```
dcm.SetChannelStatus("SDA_DP", DCM_HIGH);
```

4.1.12 SetWaveDataParam()

```
int SetWaveDataParam(  
const char*    lpszPinGroup,  
const char*    lpszStartLabel,  
ULONG         ulOffset,  
int           nWriteVectorLineCount);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszStartLabel

指定向量行标签。

ulOffset

设置偏移量，此偏移量的含义是：第一个要修改的向量行，相对于标签行的行数。

例如：向量标签行对应的行号为 1，第一个要修改的向量行行号为 2，那么偏移量 $ulOffset = 2 - 1 = 1$ 。

nWriteVectorLineCount

设置需修改的向量总行数。

注意：

起始标签行+偏移量+修改的总行数 ≤ 向量表中向量总行数

Return Values

- 0: 设置成功;
- 1: 向量文件未加载成功;
- 2: 管脚组未定义;
- 6: 起始标签未找到;
- 16: 板卡未插入
- 26: 修改向量超过向量总行数;

-28: 申请内存出错。

-35: 通道不存在

Remarks

设置要修改向量的参数。

注意：

此函数需要配合 SetSiteWaveData 和 WriteWaveData(void)使用，后两个函数使用前，必须至少使用一次此函数。

Example

例 1: G_ALLPIN 管脚组对应通道，标签 A1 后第 10 行开始，连续修改 20 行向量，四工位修改的数据分别为 0x05AA55，0x0AA55A，0x0A5A5A，0x05A5A5。

```
#define SITE_NUM 4
dcm.SetWaveDataParam("G_ALLPIN", "A1", 10, 20);
BYTE pbySiteWaveData[SITE_NUM][3] = { {0x5A, 0xA5, 0x05}, {0xAA, 0x55, 0x0A}, {0xA5,
0xA5, 0x0A}, {0x5A, 0xA5, 0x05} };
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    dcm.SetSiteWaveData(usSiteNo, pbySiteWaveData[usSiteNo]);
}
dcm.WriteWaveData();
```

4.1.13 SetSiteWaveData()

```
int SetSiteWaveData(
USHORT      usSiteNo,
BYTE*       pbyWaveData);
```

Parameters

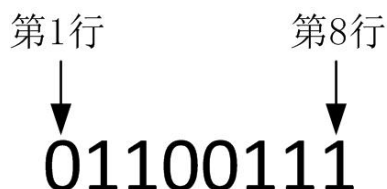
usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

pbyWaveData

设置波形数据。

例如：修改 8 行向量，参数 ulWaveData 设置为 0x67，如下图所示：



注意：

- 1) 此参数对应的数据有效位数，不得小于 SetWaveDataParam 中设置的修改向量总行数 nWriteVectorLineCount，否则会造成严重的内存越界问题；
- 2) 修改的行数不是 8 的整数倍时，需要注意传入数据的最后一个字节的也是高位在前，但是不在整字节的最高位，例如修改 10 行 0x016A(0101101010b)，传入的数据应该是 0x5A 和 0x02。

Return Values

- 0: 设置成功；
- 1: 向量文件未加载成功
- 6: 起始标签未找到
- 10: 工位错误
- 27: 字符串参数为空指针
- 29: 未设置修改向量的参数，即未使用 SetWaveDataParam
- 32: 工位无效

Remarks

设置每工位要修改的波形数据。

注意：

- 1) 此函数必须配合 SetWaveDataParam 和 WriteWaveData(void)使用；
- 2) 此函数之前必须先使用 SetWaveDataParam，设置修改波形的参数。

Example

例 1: G_ALLPIN 管脚组对应通道, 标签 A1 后第 10 行开始, 连续修改 20 行向量, 四工位修改的数据分别为 0x05AA55, 0x0AA55A, 0x0A5A5A, 0x05A5A5。

```
#define SITE_NUM 4
dcm.SetWaveDataParam("G_ALLPIN", "A1", 10, 20);
BYTE pbySiteWaveData[SITE_NUM][3] = { {0x5A, 0xA5, 0x05}, {0xAA, 0x55, 0x0A}, {0xA5, 0xA5, 0x0A}, {0x5A, 0xA5, 0x05} };
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    dcm.SetSiteWaveData(usSiteNo, pbySiteWaveData[usSiteNo]);
}
dcm.WriteWaveData();
```

4.1.14 WriteWaveData(void)

```
int WriteWaveData();
```

Return Values

- 0: 设置成功;
- 1: 向量文件未加载成功
- 10: 工位超限
- 16: 板卡未插入
- 29: 函数使用错误, 未设置修改向量的参数或者修改的向量行数为 0, 即调用 SetWaveDataParam, 或者没有设置工位的波形数据 SetSiteWaveData。
- 32: 工位无效

Remarks

将各工位要修改的波形数据, 写入到 DCM 板卡中。

注意:

- 1) 此函数需要和 SetWaveDataParam、SetSiteWaveData 配合使用;
- 2) 使用此函数前, 必须先调用 SetWaveDataParam 和 SetSiteWaveData。

Example

例 1: G_ALLPIN 管脚组对应通道, 标签 A1 后第 10 行开始, 连续修改 20 行向量, 四工位修改的数据分别为 0x05AA55, 0x0AA55A, 0x0A5A5A, 0x05A5A5。

```
#define SITE_NUM 4
dcm.SetWaveDataParam("G_ALLPIN", "A1", 10, 20);
BYTE pbySiteWaveData[SITE_NUM][3]={ {0x5A, 0xA5,0x05}, {0xAA, 0x55,0x0A},{0xA5,
0xA5,0x0A},{0x5A, 0xA5,0x05}};
for(int nSiteIndex = 0; nSiteIndex< SITE_NUM; ++nSiteIndex)
{
    dcm.SetSiteWaveData(nSiteIndex, pbySiteWaveData [nSiteIndex]);
}
dcm.WriteWaveData();
```

4.1.15 WriteWaveData()

```
int WriteWaveData(
const char*    lpszPinGroup,
const char*    lpszStartLabel,
USHORT        usSiteNo,
ULONG         ulOffset,
int           nNumber,
ULONG         ulWaveData);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszStartLabel

指定向量行标签。

usSiteNo

工位号。0 代表 Site1, 1 代表 Site2.....

DCM_ALLSITE: 所有工位

ulOffset

设置偏移量，此偏移量的含义是：指定修改操作数的向量行相对于标签行向后的行数。例如：向量行标签对应的行号为 1，修改向量的第一行行号为 2，那么偏移量 $ulOffset = 2 - 1 = 1$ 。

nNumber

设置需修改的向量总行数，取值范围 1~32。

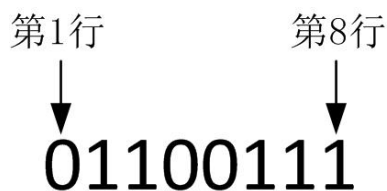
注意：

起始标签行+偏移量+修改行数 ≤ 向量表中向量总行数

ulWaveData

设置波形数据。

例如：修改 8 行向量，参数 *ulWaveData* 设置为 0x67，如下图所示：



Return Values

- 0: 设置成功;
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 6: 起始标签未找到
- 10: 工位超限
- 15: 偏移量超限
- 16: 板卡不存在
- 26: 向量行数超范围
- 28: 申请内存出错

-32: 工位无效

-35: 通道不存在

Remarks

修改指定行的波形数据。

Example

例 1: G_ALLPIN 管脚组对应通道, 标签 A1 后第 10 行开始, 连续修改 20 行向量, 数据修改为 0x55AA55。

```
dcm.WriteWaveData("G_ALLPIN", "A1", 0, 10, 20, 0x55AA55);
```

4.1.16 SetVT()

```
int SetVT(  
const char*    lpszPinGroup,  
double         dVTVoltValue = 3.5,  
VTMode         Mode = DCM_VT_CLOSE);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

dVTVoltValue

VT 电压值, 取值范围-1.5~6, 单位: V, 默认值 3.5。

Mode

VT 工作模式, 有三种模式:

DCM_VT_FORCE: 强制输出模式

DCM_VT_REALTIME: 实时输出模式

DCM_VT_CLOSE: 关闭 VT 模式

强制输出模式：管脚立即输出 VT 电压；

实时输出模式：运行向量时，当管脚为高阻时，输出 VT 电压。

关闭 VT 模式：关闭 VT，向量正常运行。

默认值为 DCM_VT_CLOSE。

Return Values

0：设置成功

-1：向量文件未加载成功

-2：管脚组未定义

-8：VT 模式错误

-14：VT 电压超范围

-16：板卡未插入

Remarks

设置 VT 电压及工作模式。

注意：

此函数会将输入管脚的有效工位对应的通道切换到 MCU 模式。

Example

例 1：设置管脚组 “G_ALLPIN”，VT 电压 3.5V，工作模式为实时模式。

```
dcm.SetVT("G_ALLPIN",3.5,DCM_VT_REALTIME);
```

4.1.17 SetDynamicLoad()

```
int SetDynamicLoad(  
const char*      lpszPinGroup,  
BOOL             bEnable,  
double           dIOH,  
double           dIOL,
```

```
double      dVTVoltValue);
```

Parameters

lpszPinGroup

管脚或者管脚组名称。

bEnable

是否开启动态负载：

TRUE: 开动态负载

FALSE: 关闭动态负载

dIOH

非驱动模式时，管脚电压高于 VT 时，DCM 通道的电流值，取值范围-0.006~0.018，单位：A，默认值 0。正方向为 DCM 输入电流。

dIOL

非驱动模式时，管脚电压低于 VT 时，DCM 通道的电流值，取值范围-0.006~0.018，单位：A，默认值 0。正方向为 DCM 输出电流。

dVTVoltValue

VT 电压值，取值范围-1.25~5.75，单位：V，默认值 3.5。

Return Values

0: 设置成功

-2: 管脚组未定义

-14: VT 电压超范围

-16: 板卡未插入

-18: 电流超范围

-35: 通道不存在

Remarks

设置动态负载的电流及工作方式。

注意：

- 1) IOH 和 IOL 的电流由 VT 提供，因此 IOH/IOL 提供的最大电压为 VT；
- 2) 此函数会将输入管脚的有效工位对应的通道切换到 MCU 模式；
- 3) 设置通道电平会关掉动态负载。

Example

例 1：设置管脚 “G_ALLPIN”，VT 电压 3.5V，开启动态负载，IOH 为 0.010A，IOL 为 0.010A。

```
dcm.SetDynamicLoad("G_ALLPIN", TRUE, 0.010, 0.010, 3.5);
```

4.1.18 SetPrereadVector()

```
int SetPrereadVector(  
const char*    lpszStartLabel,  
const char*    lpszStopLabel);
```

Parameters

lpszStartLabel

起始标签。

lpszStopLabel

结束标签。

注意：

起始标签到结束标签，最多 256 行向量，否则只预载入前 256 行向量。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 6: 起始标签未定义
- 7: 结束标签未定义
- 16: 板卡未插入
- 19: 结束标签在起始标签之前
- 21: 预载入的向量超限

Remarks

预载入预定段的向量，提高修改预载入向量的效率。

注意：

- 1) 一个程序中，此函数只能预载入 10 段向量；
- 2) 每段向量不超过 256 行；
- 3) 建议在 UserLoad 函数中使用此函数；
- 4) 经常修改的向量段，建议使用此函数进行预载入。

Example

例 1：预载入标签“Start”到标签“Stop”的向量。

```
dcm.SetPrereadVector("Start", "Stop");
```

4.1.19 SetFailSavingType ()

```
int SetFailSavingType (  
const char*          lpszPinGroup,  
FAIL_SAVING_TYPE     FailSavingType);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

FailSavingType

失效保存类型，有两种模式：

DCM_SAVING_ALL_FAIL: 保存所有失效行

DCM_SAVING_SELECT_LINE_FAIL: 保存选定行的失效信息

Return Values

0: 获取成功

-1: 向量文件未加载成功

-2: 管脚名称未定义

-16: 板卡不存在

Remarks

设置失效保存的类型。当选择保存选定行的失效时，只有选定行失效时才会保存，未选定的行，即使失效也不会保存。

注意：

- 1) 选择保存选定行失效信息时，基于失效行信息的功能会受影响，例如 GetFirstFailLineNo/GetFailLineNo;
- 2) 选择保存选定行失效信息时，可以获取到的失效行只有选定行，但是失效行总数不受影响；
- 3) 使用保存选定行失效信息和 GetCaptureData 不能共存，即选择保存选定行失效信息时，GetCaptureData 将不能使用。

Example

例 1：设置保存选定行的失效信息。

```
dcm.SetFailSavingType("G_ALLPIN", DCM_SAVING_SELECT_LINE_FAIL);
```

4.1.20 RunVectorWithGroup()

```
int RunVectorWithGroup(  
const char*    lpszPinGroup,  
const char*    lpszStartLabel,  
const char*    lpszStopLabel,  
bool           bWaitFinish = TRUE);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszStartLabel

起始标签。

注意：

此参数为""，则从第 0 行开始。

lpszStopLabel

结束标签。

注意：

- 1) 此参数为""，则运行到停止指令（HALT）结束；
- 2) 结束标签必须在起始标签之后。

bWaitFinish

可设置为：

TRUE：等待向量运行完成或者运行 160s 后退出函数

FALSE：向量运行开始后直接退出函数

默认值为 TRUE。

注意：

- 1) 向量运行过程中，如果需要进行其他操作时，可以选择 FALSE；

- 2) 无限循环的向量，必须选择 FALSE。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 6: 起始标签未找到
- 7: 结束标签未找到
- 16: 板卡不存在
- 19: 结束标签在起始标签之前

Remarks

启动运行指定通道的向量。

注意：

- 1) 此函数会将输入管脚的有效工位的通道打开，并切换到 MCU 模式；
- 2) 无效工位的管脚会自动关闭，不输出波形；

Example

例 1: G_ALLPIN 管脚组对应通道从第 0 行运行到停止指令（HALT）结束，向量启动运行后，函数立即退出。

```
dcm.RunVectorWithGroup("G_ALLPIN", "", "", FALSE);
```

例 2: G_ALLPIN 管脚组对应通道从标签 A1 运行到标签 A2 结束，向量运行结束后函数退出。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);
```

4.1.21 RunVectorEn()

```
int RunVectorEn(  
const char*    lpszPinGroup,  
const char*    lpszStartLabel,  
const char*    lpszStopLabel);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszStartLabel

起始标签。

注意：

此参数为""，则从第 0 行开始。

lpszStopLabel

结束标签。

注意：

- 1) 此参数为""时，则运行到停止指令（HALT）结束；
- 2) 结束标签必须在起始标签之后；

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 6: 起始标签未找到
- 7: 结束标签未找到
- 16: 板卡不存在
- 19: 结束标签在起始标签之前

Remarks

设置同步运行向量，本函数与 STSAWGRun 函数一起使用，可以实现 DCM 与其它 VI 源或表的同步功能。

注意：

- 1) 此函数会将输入管脚的有效工位的通道打开，并切换到 MCU 模式；
- 2) 无效工位的管脚会自动关闭，不输出波形；

Example

例 1: G_ALLPIN 管脚组对应通道与 FPVleAWG 同步, DCM 运行 VEC_S 到 VEC_E 之间的向量。

```
double adPattern[100] = { 0 };
for (int nPatternIndex = 0; nPatternIndex < 100; nPatternIndex++)
{
    if (5 > nPatternIndex % 10)
    {
        adPattern[nPatternIndex] = 3;
    }
    else
    {
        adPattern[nPatternIndex] = 0;
    }
}
fpvie0.AwgLoader("SquareWave", FV, FPVle_5V, FPVle_1MA, adPattern, 100);
fpvie0.AwgSelect("SquareWave", 0, 99, 99, 10);
dcm.RunVectorEn("G_ALLPIN", "VEC_S", "VEC_E");
STSEnableAWG(&fpvie0);
STSAWGRun();
```

4.1.22 StopVector()

```
int StopVector(const char* lpszPinGroup);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

Return Values

- 0: 设置成功
- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 16: 板卡不存在

Remarks

停止指定通道所在控制器的向量运行。

注意：

此函数会停止有效工位的通道所在控制器的所有通道运行的向量。

Example

例 1：停止 G_ALLPIN 管脚组对应通道的向量运行。

```
dcm.StopVector("G_ALLPIN");
```

4.1.23 SaveFailMap()

```
int SaveFailMap(unsigned long ulaxErrLine);
```

Parameters

ulMaxErrLine

unsigned long 类型，设置可记录的最大失效行数。

注意：

- 1) 此参数设置为 0 或者-1 时，表示记录最近一次运行的向量段内的所有失效行；
- 2) 对于循环指令(Repeat/Loop)，只要向量行有一次 Fail，则标记为 Fail。

Return Values

0: 未开始运行, 或正在运行向量

-1: 向量文件未加载成功

-28: 申请内存出错

Remarks

保存失效行数, 可以通过向量编辑器反标显示观察失效行所在位置。

注意：

每次调用此函数会覆盖前一次的失效图, 即只保存最近一次的失效图。

Example

例 1: 保存标签 A1 到标签 A2 的不超过 100 个失效行。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);  
dcm.SaveFailMap(100);
```

例 2: 保存标签 A1 到标签 A2 的所有失效行。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);  
dcm.SaveFailMap(0);
```

4.1.24 GetHardwareCaptureData()

```
int GetHardwareCaptureData(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
BYTE*         pbyData,  
int&          nBuffSize);
```

Parameters

lpszPinName

管脚名称

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

pbyData

存储读取的测试结果。

0 表示 Pass

1 表示 Fail

每字节对应的值的含义如下，第一行表示此次运行的第一个 capture 的值。

第1行 第8行
↓ ↓
01100111

注意：

获取测试结果失败时，测试结果数组 (*pbyData*) 的每个元素都会被设置成 0xFFFFFFFFF。

nBuffSize

存储测量结果的数组大小。

Return Values

>=0: 前一次向量运行的 Capture 总行数

-1: 向量文件未加载成功

-9: 管脚名称未定义

-10: 工位号不存在

-16: 板卡不存在

-32: 工位无效

-33: 向量未运行

-34: 向量正在运行

-35: 通道不存在

-42: 向量中的 Capture 行数超范围，无法获取

Remarks

获取指定工位，指定通道和对应的硬件 Capture 值。

注意：

- 1) 此功能适用于在向量文件中使用了 Capture 功能的向量；
- 2) 建议使用此函数代替 GetCaptureData 函数；
- 3) 仅支持逻辑版本不低于 0x30 的 DCM；
- 4) 一次运行的向量，最多可以设置 1023 个 Capture；
- 5) 此函数支持循环指令，循环体每次的比较结果都可以获取到。

Example

例 1：获取 SDA_DP 通道的工位 1 对应的硬件 Capture 值。

```
BYTE abyCapture[10] = { 0 };  
dcm.GetHardwareCaptureData("SDA_DP", 0, abyCapture, sizeof(abyCapture));
```

4.1.25 SetCapture()

```
int SetCapture(  
const char*    lpszPinGroup,  
const char*    lpszStartLabel,  
USHORT        usSiteNo,  
ULONG         ulOffset,  
int            nCount);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

lpszStartLabel

设置向量行的标签。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

DCM_ALLSITE: 所有工位

ulOffset

设置偏移量，此偏移量的含义是：指定开始获取比较结果的向量行相对于标签行向后的行数。

nCount

读取结果的总行数，取值范围 1~32。

注意：

起始标签行+偏移量+读取的总行数≤向量表中向量总行数

Return Values

- 0: 获取成功
- 1: 向量文件未加载成功
- 2: 管脚名称未定义
- 6: 起始标签未找到
- 10: 工位号不存在
- 15: 偏移量超范围
- 16: 板卡不存在
- 33: 向量未运行
- 34: 向量正在运行
- 37: 前一次运行向量非向量文件中向量

Remarks

获取指定工位，指定通道和指定向量行的测试比较结果。

注意：

此函数需要和 GetCaptureData 配合使用。

Example

例 1：获取管脚组“G_SDA”，所有工位的对应标签行 A0 向后偏移 20 行开始的一共 16 行的测试比较结果。（从第 20 行开始，16 行的预期结果为“HHHHHHHHLLLLLLLL”，实际结果为 LHHHHHHHHLLLLLLHLL，此时调用此函数返回的 data[31:0] = 0x00008004，代表第 1 行和第 14 行测试比较结果为 Fail。）

```
#define SITE_NUM 4
ULONG ulData[SITE_NUM] = { 0 };
dcm.SetPinGroup("G_SDA", "SDA");
dcm.SetCapture("G_SDA", "A0", DCM_ALLSITE, 20, 16);
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    dcm.GetCaptureData("SDA_DP", usSiteNo, ulData[usSiteNo]);
}
```

4.1.26 GetCaptureData()

```
int GetCaptureData(
const char*    lpszPinName,
USHORT        usSiteNo,
ULONG&        ulCaptureData);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulCaptureData

存储读取的测试结果。

0 表示 Pass

1 表示 Fail

第1行 第8行
↓ ↓
01100111

Return Values

- 0: 成功
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 10: 工位号不存在
- 16: 板卡未插入
- 29: 未设置获取的向量参数，即未使用 SetCapture
- 32: 工位无效
- 33: 未运行向量
- 34: 向量正在运行
- 35: 通道不存在
- 41: 最近一次向量运行的 Fail 行超存储容量，无法获取比较结果
- 43: 管脚不属于当前 DCM 实例
- 46: 该通道使用了保存指令行失效信息，不支持 Capture 功能

Remarks

获取指定工位，指定通道和指定向量行的测试比较结果。

注意：

- 1) 此函数必须与 SetCapture 配合使用；
- 2) 使用此函数前，必须先调用 SetCapture，设置获取比较结果的位置；
- 3) 获取比较结果出错时，测试结果(*ulCapture*)将被设置成 0xFFFFFFFF。

- 4) 此函数基于失效存储器实现;
- 5) 每个控制器的失效存储器只存储前 1023 个失效行;
- 6) 一旦控制器中的一个通道失效, 就会占用 1 行失效存储器;
- 7) 基于以上原因, 当通道的第一个失效行在控制器的前 1023 个失效行之后, 那么这个通道的失效行将不会被存储;
- 8) 此函数不支持循环指令, 循环体只要有一个失效, 就认为这一行是失效的;
- 9) 如果使用了只保存失效行的功能, 则此功能不再支持, 详见 4.1.19SetFailSavingType。

Example

例 1: 获取管脚组 G_SDA, 所有工位的对应标签行 A0 向后偏移 20 行开始的一共 16 行的测试比较结果。(从第 20 行开始, 16 行的预期结果为“HHHHHHHHLLLLLLLL”, 实际结果为 LHHHHHHHHLLLLHLL, 此时调用此函数返回的 data[31:0]=0x00008004, 代表第 1 行和第 14 行测试比较结果为 Fail。)

```
#define SITE_NUM 4
ULONG ulData[SITE_NUM] = { 0 };
dcm.SetPinGroup("G_SDA", "SDA");
dcm.SetCapture("G_SDA", "A0", DCM_ALLSITE, 20, 16);
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    dcm.GetCaptureData("SDA_DP", usSiteNo, ulData[usSiteNo]);
}
```

4.1.27 GetCaptureData()

```
int GetCaptureData(
const char*    lpszPinName,
const char*    lpszStartLabel,
USHORT        usSiteNo,
ULONG         ulOffset,
int           nCount,
ULONG&        ulData);
```

Parameters

lpszPinName

管脚名称。

lpszStartLabel

设置向量行的标签。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulOffset

设置偏移量，此偏移量的含义是：指定开始获取比较结果的向量行相对于标签行向后的行数。

注意：

标签行加上此参数不得超过下一个标签行。

nCount

读取结果的总行数，取值范围 1~32。

注意：

此参数加上偏移量不得超过下一个标签行，否则下一个标签行及超出部分无效。

ulData

存储读取的测试结果。

0 表示 Pass

1 表示 Fail

第1行 第8行
↓ ↓
01100111

注意：

获取比较结果失败时，比较结果(*ulData*)将返回 0xFFFFFFFF。

Return Values

- 0: 成功
- 1: 向量文件未加载成功
- 6: 起始标签未找到
- 9: 管脚名称未定义
- 10: 工位号不存在
- 11: 读取结果的总行数小于 1
- 15: 偏移量超范围
- 16: 板卡不存在
- 32: 工位无效
- 33: 向量未运行
- 34: 向量正在运行
- 35: 通道不存在
- 37: 前一次运行向量不是从此标签行运行
- 41: 最近一次向量运行的 Fail 行超存储容量，无法获取比较结果
- 43: 管脚不属于当前 DCM 实例
- 46: 该通道使用了保存指令行失效信息，不支持 Capture 功能

Remarks

获取指定工位，指定通道和指定向量行的测试比较结果。

注意：

- 1) 此函数基于失效存储器实现；
- 2) 每个控制器的失效存储器只存储前 1023 个失效行；
- 3) 一旦控制器中的一个通道失效，就会占用 1 行失效存储器；
- 4) 基于以上原因，当通道的第一个失效行在控制器的前 1023 个失效行之后，那么这个通道的失效行将不会被存储；
- 5) 此函数不支持循环指令，循环体只要有一个失效，就认为这一行是失效的；
- 6) 如果使用了只保存失效行的功能，则此功能不再支持，详见

4.1.19SetFailSavingType。

Example

例 1: 获取管脚 SDA_DP 对应标签行 A0 向后偏移 20 行开始的一共 16 行的测试比较结果。（从第 20 行开始，16 行的预期结果为“HHHHHHHHLLLLLLLL”，实际结果为 LHHHHHHHLLLLLLHLL，此时调用此函数返回的 ulData[31:0]=0x00008004，代表第 1 行和第 14 行测试比较结果为 Fail。）

```
ULONG ulData = 0;
dcm.GetCaptureData("SDA_DP", "A0", 0, 20, 16, ulData);
```

4.1.28 GetMCUResult()

```
int GetMCUResult(USHORT usSiteNo);
```

Parameters

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

Return Values

0: PASS

1: FAIL

-1: 向量文件未加载成功

-10: 工位号不存在

-16: 板卡不存在

-32: 工位无效

-33: 向量未运行

-34: 向量正在运行

Remarks

获取最近一次向量运行，指定工位管脚的总测试结果。总测试结果的含义：指定

管脚的比较行的比较结果全部 PASS，则总测试结果为 PASS，否则为 FAIL。

注意：

最近一次 RunVectorWithGroup 的结果，未使用的管脚不统计；

Example

例 1：运行从标签行 A1 到标签行 A2 的向量，并获取工位 1 所有管脚的总测试结果。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);  
int nResult= dcm.GetMCUResult(0);
```

4.1.29 GetMCUPinResult()

```
int GetMCUPinResult(  
const char*   lpszPinName,  
USHORT       usSiteNo);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

Return Values

0: PASS

1: FAIL

-1: 向量文件未加载成功

-9: 管脚名称未定义

-10: 工位号不存在

- 16: 板卡不存在
- 32: 工位无效
- 33: 向量未运行
- 34: 向量正在运行
- 35: 通道不存在
- 43: 管脚不属于当前 DCM 实例

Remarks

获取指定管脚最近一次运行向量的测试结果。

Example

例 1: 运行从标签行 A1 到标签行 A2 的向量, 并获取工位 1, SDA_DP 管脚的测试结果。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);  
int nResult=dcm.GetMCUPinResult("SDA_DP", 0);
```

4.1.30 GetMCUPinRunStatus()

```
int GetMCUPinRunStatus(  
const char*    lpszPinName,  
USHORT        usSiteNo);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1, 1 代表 Site2.....

Return Values

- 0: 正在运行向量
- 1: 向量运行结束
- 2: 向量未运行
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 16: 板卡未找到
- 32: 工位无效
- 35: 通道不存在
- 43: 管脚不属于当前 DCM 实例

Remarks

获取管脚运行状态。

Example

例 1: 获取管脚 SDA_DP 的工位 1 对应通道的运行状态。

```
int nStatus = dcm.GetMCUPinRunStatus ("SDA_DP", 0);
```

4.1.31 GetFailCount()

```
int GetFailCount(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
ULONG&        ulFailCount);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulFailCount

存储前一次向量运行失效的总行数。

注意：

获取失效行数失败时，比较失效的总行数(*ulFailCount*)返回 0xFFFFFFFF;

Return Values

- 0: 获取失效行数成功;
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 16: 板卡未找到
- 32: 工位无效
- 33: 向量未运行
- 34: 向量正在运行
- 35: 通道不存在
- 43: 管脚不属于当前 DCM 实例

Remarks

获取指定工位、指定管脚的最近一次运行向量失效的总行数。

Example

例 1: 运行从标签行 A1 到标签行 A2 的向量，并获取工位 1，SDA_DP 管脚的最近一次向量运行的总失效行数。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);
ULONG ulFailCount = 0;
dcm.GetFailCount("SDA_DP", 0, ulFailCount);
```

4.1.32 GetFirstFailLineNo()

```
int GetFirstFailLineNo(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
ULONG&        ulFirstFailLineNo);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulFirstFailLineNo

存储第一个失效行相对于前一次运行向量的起始标签的行号。例如，最近一次运行向量的起始标签 StartLabel 所在的行号为 2，而第一个失效行对应的行号是 12，那么 *ulFirstFailLineNo*=12-2=10。

注意：

- 1) 以向量编辑器中的行数为准，即 Repeat 行以 1 行计数、Loop 循环指令的循环体以一次循环的行数计算；
- 2) 获取失效行数失败时，第一个失效行行号(*ulFirstFailLineNo*)返回 0xFFFFFFFF；

Return Values

- 0: 获取第一个失效行号成功；
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 12: 不存在失效行。
- 16: 板卡未找到
- 32: 工位无效

- 33: 向量未运行
- 34: 向量正在运行
- 35: 通道不存在
- 41: 失效行存储器满，通道失效行没有存储
- 43: 管脚不属于当前 DCM 实例

Remarks

获取指定管脚的最近一次向量运行的第一个失效行相对于起始标签的行号。

注意：

- 1) 每个控制器的失效存储器只存储前 1023 个失效行；
- 2) 一旦控制器中的一个通道失效，就会占用 1 行失效存储器；
- 3) 基于以上原因，当通道的第一个失效行在控制器的前 1023 个失效行之后，那么这个通道的失效行将不会被存储。

Example

例 1：运行从标签行 A1 到标签行 A2 的向量，并获取工位 1，SDA_DP 管脚的最近一次向量运行的第一个失效行相对于标签 A1 的行号。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);
ULONG ulFirstFailLine = 0;
int nRetVal = dcm.GetFirstFailLineNo("SDA_DP", 0, ulFirstFailLine);
```

4.1.33 GetFailLineNo()

```
int GetFailLineNo(
const char*      lpszPinName,
USHORT          usSiteNo,
int              nCount,
ULONG            ulFailLineNo[]);
```

Parameters

lpsPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

nCount

设置获取失效行的最大行数。

ulFailLineNo

失效行相对最近一次向量运行的起始标签的行号。例如，最近一次运行向量的起始标签所在的行号为 2，而第一个失效行对应的行号是 12，那么 $ulFailLineNo[0]=12-2=10$ 。

注意：

- 1) *ulFailLineNo* 数组长度需不小于 *nCount* 的值，否则可能会发生不可预期的内存越界错误；
- 2) 如果失效行数组的元素总数大于实际失效行时，大于实际行数的元素会被设置成 0xFFFFFFFF；
- 3) 除失效存储器满以外的失效时，失效行数组(*ulFailLineNo*)的所有元素都会设置为 0xFFFFFFFF。

Return Values

- 0: 获取失效行数成功
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 12: 不存在失效行
- 16: 板卡未找到
- 32: 工位无效
- 33: 向量未运行

- 34: 向量正在运行
- 41: 失效存储器满，部分失效行没有存储
- 43: 管脚不属于当前 DCM 实例

Remarks

获取指定管脚的最近一次运行向量的比较失效的向量行相对于起始标签的行号。

注意：

- 1) 每个控制器的失效存储器只存储前 1023 个失效行；
- 2) 一旦控制器中的一个通道失效，就会占用 1 行失效存储器；
- 3) 基于以上原因，当通道的第一个失效行在控制器的前 1023 个失效行之后，那么这个通道的失效行将不会被存储。

Example

例 1：运行从标签行 A1 到标签行 A2 的向量，并获取工位 1，SDA_DP 管脚的最近一次向量运行的前 10 个失效行的相对于标签行 A1 的行号。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);
ULONG aulFailLineNo[10] = { 0 };
int nRetVal = dcm.GetFailLineNo("SDA_DP", 0, 10, aulFailLineNo);
```

4.1.34 GetStopLineNo()

```
int GetStopLineNo(
USHORT      usSiteNo,
ULONG&      ulStopLineNo);
```

Parameters

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulStopLineNo

存储向量运行结束后停止行在向量表中对应的行号。

注意：

函数返回值不为 0 时，停止行号返回 0xFFFFFFFF。

Return Values

- 0: 获取向量表行号成功
- 1: 获取行号失败
- 10: 工位超范围
- 16: 板卡不存在
- 32: 工位无效
- 33: 向量未运行
- 34: 向量正在运行
- 35: 通道不存在

Remarks

获取指定工位向量运行结束后，停止行在向量表中对应的行号。

Example

例 1：运行从标签行 A1 到标签行 A2 的向量，并获取工位 1，向量运行结束后停止行在向量表中对应的行号。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);
ULONG ulStopLineNo = 0;
dcm.GetStopLineNo(0, ulStopLineNo);
```

4.1.35 GetStopLabel()

```
char* GetStopLabel (USHORT  usSiteNo);
```

Parameters

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

Return Values

返回停止行对应的标签。

注意：

当工位号错误、无效或者未找到标签时返回空字符串。

Remarks

获取指定工位运行结束后，停止行对应的标签。

Example

例 1：运行从标签行 A1 到标签行 A2 的向量，并获取工位 1，向量运行结束后停止行的标签。

```
dcm.RunVectorWithGroup("G_ALLPIN", "A1", "A2", TRUE);  
char* lpszStopLabel = nullptr;  
lpszStopLabel = dcm.GetStopLabel(0);
```

4.1.36 GetRunLineCount()

```
int GetRunLineCount(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
ULONG&        ullLineCount);
```

Parameters

lpszPinName

管脚名称

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

ulLineCount

正在运行向量已经运行的行数，或者前一次运行向量的总行数。

注意：

参数错误或工位无效时，行数将被设置成 0xFFFFFFFF。

Return Values

- 0: 成功获取运行行数
- 1: 向量文件未加载成功
- 9: 管脚名称未定义
- 10: 工位号不存在
- 16: 板卡不存在
- 32: 工位无效
- 35: 通道不存在
- 43: 管脚不属于当前 DCM 实例

Remarks

正在运行向量已经运行的行数，或者前一次运行向量的总行数。这里的总行数为向量实际运行行数，包括指令实际执行的行数。

Example

例 1：获取 SDA_DP 通道的工位 1 对应通道运行的向量总行数。

```
dcm.RunVectorWithGroup("G_ALLPIN", "LABEL1", "LABEL2");  
dcm.GetRunLineCount("SDA_DP", 0, ulLineCount);
```

4.1.37 SetPPMU()

```
int SetPPMU(  
const char*    lpszPinGroup,  
PPMUMode      Mode,
```

```
double          dSetValue,  
PPMUIRange     IRange = DCM_PPMUIRANGE_2UA);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

Mode

PMU 工作模式，有四种模式：

DCM_PPMU_FVMI：恒压测流模式

DCM_PPMU_FIMV：恒流测压模式

DCM_PPMU_FIMI：恒流测流模式

DCM_PPMU_FVMV：恒压测压模式

dSetValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，取值范围-1.5~6，单位为 V。

选择恒流模式时，该参数为恒流值，取值范围-0.032~0.032，单位为 A。

IRange

电流量程。

DCM_PPMUIRANGE_2UA

DCM_PPMUIRANGE_20UA

DCM_PPMUIRANGE_200UA

DCM_PPMUIRANGE_2MA

DCM_PPMUIRANGE_32MA

默认值为 DCM_PPMUIRANGE_2UA。

Return Values

0：设置成功

- 1: 向量文件未加载成功
- 2: 管脚组未定义
- 16: 板卡不存在

Remarks

设置 PMU 的状态，包括工作模式、恒压或恒流值、电流量程。

注意：

PMU 和 MCU 只能二选一，即使用此函数后，对应的有效工位通道的会切换到 PMU 模式，之前处于 MCU 模式下的状态不再保持。

Example

例 1：管脚组 G_ALLPIN，设置 PMU 工作模式恒压测流，恒压 3.3V。

```
dcm.SetPPMU("G_ALLPIN", DCM_PPMU_FVMI, 3.3, DCM_PPMUIRANGE_32MA);
```

例 2：管脚组 G_ALLPIN，设置 PMU 工作模式恒流测流，恒流 1uA。

```
dcm.SetPPMU("G_ALLPIN", DCM_PPMU_FIMI, 1e-6, DCM_PPMUIRANGE_2UA);
```

4.1.38 PPMUMeasure()

```
int PPMUMeasure(  
const char*          lpszPinGroup,  
UINT                uSampleTimes,  
PPMU_SAMPLE_PERIOD SamplePeriod);
```

Parameters

lpszPinGroup

管脚或管脚组名称。

uSampleTimes

采样次数。取值范围：1~512。大于 512 取 512

SamplePeriod

采样间隔。可选：

DCM_PPMU_5US

DCM_PPMU_10US

DCM_PPMU_20US

DCM_PPMU_40US

DCM_PPMU_80US

DCM_PPMU_160US

DCM_PPMU_320US

Return Values

0：设置成功

-1：向量文件未加载成功

-2：管脚组未定义

-16：板卡不存在

-17：PMU 测量出错

Remarks

启动指定通道的 PMU 测量。

注意：

DCM 相邻的两个通道为一组（如 CH0 和 CH1，CH2 和 CH3 分别为一组 PMU 通道），同组内的两个通道无法同时测量，如果输入的管脚中有同一组的通道，则会分别测量组内的两个通道。

Example

例 1：启动 G_ALLPIN 管脚组对应通道的 PMU 测量，采样 10 次，采样间隔为 10uS。

```
dcm.PPMUMeasure("G_ALLPIN", 10, DCM_PPMU_10US);
```

4.1.39 GetPPUMeasResult()

```
double GetPPUMeasResult(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
int            nSampleNumber = AVERAGE_RESULT);
```

Parameters

lpszPinName

管脚名称。

注意：

此管脚必须在 PMU 测量的管脚组内。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

nSampleNumber

AVERAGE_RESULT 平均值

非负值 N： 返回的是第 N+1 个采样点的测量值。假设 PPMUMeasure 的采样次数为 *uSampleTimes*，则需满足 $0 \leq N \leq uSampleTimes-1$ 。

Return Values

返回指定管脚 PMU 的测量结果。

注意：

参数错误或工位无效时，则会返回极大值 $1e15-1$

Remarks

获取 PMU 的测量结果。

注意：

- 1) 如果 SetPPMU 函数选择测压模式，则测量结果为电压值，单位为 V；
- 2) 如果 SetPPMU 函数选择测流模式，则测量结果为电流值，单位为 A。

Example

例 1：获取工位 1，SDA_DP 管脚测流结果的平均值。SDA_DP 是管脚组 G_ALLPIN 的其中一个管脚。

```
dcm.SetPPMU("G_ALLPIN", DCM_PPMU_FVMI, 3.3, DCM_PPMUIRANGE_32MA);  
dcm.PPMUMeasure("G_ALLPIN", 10, 10);  
double dMeasureValue= dcm.GetPPMUMeasResult("SDA_DP", 0, AVERAGE_RESULT);
```

4.1.40 SetTMUMatrix()

```
int SetTMUMatrix(  
const char*    lpszPinGroup,  
USHORT        usSiteNo,  
DCM_TMU       TMUUnit);
```

Parameters

lpszPinGroup

管脚组或者管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

DCM_ALLSITE: 所有工位

TMUUnit

TMU 单元数，可选

DCM_TMU1: TMU 单元 1

DCM_TMU2: TMU 单元 2

Return Values

- 0: 成功设置 TMU 单元连接的通道
- 1: 没有加载向量文件
- 2: 管脚或者管脚名称不存在
- 10: 工位超范围
- 16: 板卡不存在
- 32: 工位无效
- 35: 通道不存在
- 38: 每个控制器的 TMU 单元只能连接一个通道
- 39: TMU 单元错误

Remarks

设置 TMU 单元连接的通道。

注意：

每个控制器有两个 TMU 单元，每个单元可以连接同控制器的任意一个通道。

Example

例 1: TMU 单元 1 连接管脚 SDA_DP 工位 1 的通道。

```
dcm.SetTMUMatrix("SDA_DP", 0, DCM_TMU1);
```

4.1.41 SetTMUParam()

```
int SetTMUParam(  
const char*    lpszPinGroup,  
DCM_SLOPE     Slope,  
UINT           uHoldOffTime,  
UINT           uHoldOffNum,  
DCM_TMU        TMUUnit=DCM_ALL_TMU);
```

Parameters

lpszPinGroup

管脚组或者管脚名称。

Slope

设置比较器的触发极性，可选

DCM_POS: 上升沿触发

DCM_NEG: 下降沿触发

uHoldoffTime

设置延迟触发时间，取值范围 4~508，分辨率为 8，单位 ns，默认值为 0（即无触发延迟）。对于非整数倍的数据，向下取整。大于 508 取 508。

HoldoffNum

忽略的 START 事件个数，取值范围 0~2046，默认值为 0。大于 2046 取 2046。

TMUUnit

设置参数对应的 TMU 单元，默认 DCM_ALL_TMU,可选。

DCM_TMU1: TMU 单元 1

DCM_TMU2: TMU 单元 2

DCM_ALL_TMU: TMU 的所有单元

注意：

当不选择 DCM_ALL_TMU 时，每个控制器只允许设置一个通道的 TMU 参数。

Return Values

0: 成功设置 TMU 测量参数

-1: 没有加载向量文件

-2: 管脚或者管脚名称不存在

- 16: 板卡不存在
- 35: 通道不存在
- 40: 通道未连接 TMU 单元
- 43: 管脚不属于当前实例
- 45: 指定 TMU 单元时(不是 DCM_ALL_TMU), 管脚名或者管脚组在每个控制器中只允许有一个通道

Remarks

设置 TMU 的测量参数。

注意：

- 1) 每个控制器有两个 TMU 单元，每个单元只能连接同控制器的任意一个通道；
- 2) 触发电平通过 SetPinLevel 的 VOH 和 VOL 设置；
- 3) 频率、占空比及传输延时测量模式下，采用的触发电平为 VOH；
- 4) 测量边沿时间时，TMU1 触发电平为 VOH，TMU2 触发电平为 VOL。

Example

例 1: 设置 SDA_DP 的所有工位对应的 TMU 测量参数，上升沿触发。

```
dcm.SetTMUParam("SDA_DP", DCM_POS, 0, 0);
```

4.1.42 TMUMeasure()

```
int TMUMeasure(  
const char*          lpszPinGroup,  
DCM_TMU_MEAS_MODE   TMUMeasMode,  
UINT                 uSampleNum,  
double                dTimeout);
```

Parameters

lpszPinGroup

管脚组或者管脚名称。

TMUMeasMode

设置 TMU 的测量模式，可选

DCM_MEAS_FREQ_DUTY: 频率和占空比测量

DCM_MEAS_EDGE: 边沿测量

DCM_MEAS_DELAY: 延时测量

注意：

测量 **DCM_MEAS_EDGE** 和 **DCM_MEAS_DELAY** 时，两单元的 HoldoffNum 需要保持一致。

uSampleNum

频率及占空比测量时的采样次数，取值范围 1~2046。大于 2046 取 2046

dTimeout

测量超时时间，取值范围 4e-6~1073.74182，单位 ms，分辨率 4ns。不满足分辨率时，向下取整。大于最大测量超时，取最大超时时间

Return Values

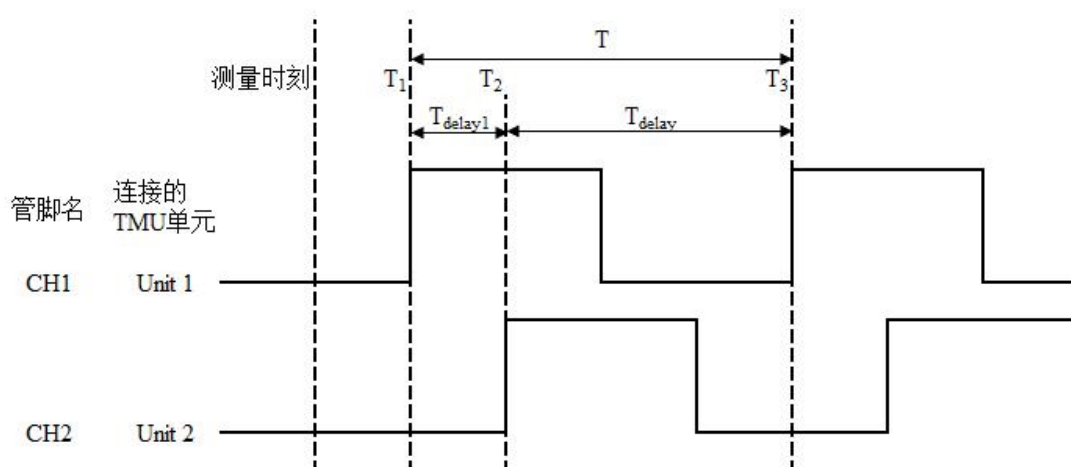
- 0: 成功启动测量
- 1: 没有加载向量文件
- 2: 管脚或者管脚名称不存在
- 10: 工位超范围
- 16: 板卡不存在
- 32: 工位无效
- 35: 通道不存在
- 40: 通道未连接 TMU 单元

Remarks

启动 TMU 测量。

注意：

- 1) 边沿测量或者传输延时测量模式下，必须使用同一个控制器的两个单元；
- 2) 传输延时测量模式下，不可以设置先触发的 TMU 单元，测量值为两个 TMU 单元触发时刻的差值，以 GetTMUMeasureResult 函数中通道对应单元的触发时刻值为被减数。具体如下图



序号	源码样例	测量值
1	dcm.SetTMUMatrix("CH1", DCM_ALLSITE, DCM_TMU1);	——
2	dcm.SetTMUMatrix("CH2", DCM_ALLSITE, DCM_TMU2);	——
3	dcm.SetTMUParam("G_CH1CH2", DCM_ALLSITE, DCM_POS, 0, 0);	——
4	dcm.TMUMeasure("G_CH1CH2", DCM_MEAS_DELAY, 1, 5);	——
5	double dMeasureResult = dcm.GetTMUMeasureResult("CH1", 0, DCM_DELAY);	$dMeasureResult = T_1 - T_2$
6	dMeasureResult = dcm.GetTMUMeasureResult("CH2", 0, DCM_DELAY);	$dMeasureResult = T_2 - T_1$

DCM的TMU无法指定触发顺序。当测量信号为周期波形时，测量时刻是不确定的，测量时刻可能出现在 T_1 之前或者在 T_1 与 T_2 之间，这时测量值就会出现两种（正数和负数）。对于这种情况，需要判断测量值得正负，当测量值小于0时，需要使用波形周期加上测量值。

Example

例 1：启动 SDA_DP 的 TMU 测量，测量模式为频率和占空比测量，采样次数为 15，测量超时为 10ms。

```
dcm.TMUMeasure("SDA_DP", DCM_MEAS_FREQ_DUTY, 15, 10);
```

4.1.43 GetTMUMeasureResult()

```
double GetTMUMeasureResult(  
const char*          lpszPinName,  
USHORT              usSiteNo,  
DCM_TMU_MEAS_TYPE   TMUMeasType);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

工位号。0 代表 Site1，1 代表 Site2.....

TMUMeasType

TMU 的测量值类型，可选

DCM_FREQ: 频率

DCM_HIGH_DUTY: 高占空比

DCM_LOW_DUTY: 低占空比

DCM_EDGE: 边沿时间

DCM_DELAY: 传输延时时间

注意：

频率的单位：KHz

占空比的单位：%

边沿时间和传输延时的单位：us

Return Values

!= 1e15-1: 成功启动测量测量值

1e15-1: 输入参数、测量错误或者工位无效，具体信息请参照 Alarm 信息

Remarks

获取 TMU 测量结果。

注意：

- 1) 传输延时的测量模式下，不可以设置先触发的 TMU 单元，测量值为两个 TMU 单元触发时刻的差值，以此函数使用通道对应单元的时刻值为被减数，详情请参照 TMUMeasure 的注意事项。

Example

例 1: 获取 SDA_DP 的工位 1 连接的 TMU 单元的测量结果，测量结果类型为频率。

```
double dMeasureResult = dcm.GetTMUMeasureResult("SDA_DP", 0, DCM_FREQ);
```

4.1.44 SetAlarmMask()

```
int SetAlarmMask(  
const char*    lpszPinName,  
UINT          usSiteNo,  
bool          bMaskStatus = true);
```

Parameters

lpszPinName

管脚名称。

usSiteNo

选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

bMaskStatus

bool 型变量，可设置为：

true: 屏蔽该管脚及工位对应通道的 Alarm 信息。

false: 输出该管脚及工位对应通道的 Alarm 信息。

默认值为 true。

Return Values

0: ALRAM 屏蔽状态设置成功

-1: 向量未加载

-9: 管脚名称未定义

-10: 工位号错误

-16: 板卡未插入

-32: 工位无效

Remarks

设置屏蔽指定工位和管脚对应通道的 Alarm 信息。

Example

例 1: 设置屏蔽管脚 SDA_DP 工位 1 对应通道的 ALARM 信息。

```
dcm.SetAlarmMask("SDA_DP",0, true); // mask the Alarm information of SITE_1
```

4.1.45 GetAlarmMask()

```
bool GetAlarmMask(  
const char*   lpszPinName,  
UINT         usSiteNo);
```

Parameters

lpszPinName

管脚名称

usSiteNo

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

true: 该通道屏蔽 Alarm 信息;

false: 该通道输出 Alarm 信息。

Remarks

读取指定通道的是否屏蔽 Alarm 信息。

Example

例 1：获取管脚 SDA_DP 的工位 1 对应通道的 ALARM 屏蔽状态。

```
bool bAlarmStatus = dcm.GetAlarmMask("SDA_DP",0);// Get Alarm shield status of SITE_1
```

4.1.46 获取板卡固有信息函数

4.1.46.1 GetBoardSN()

```
int GetBoardSN(  
const char*      lpszPinName,  
USHORT          usSiteNo,  
char*           lpszBoardSN,  
UINT            uSNSize);
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择读取板卡硬件版本的工位。0 代表 Site1，1 代表 Site2.....

lpszBoardSN

char 型变量，存储板卡的 SN 号。未存储返回“N/A”。

由用户定义，如：char lpszBoardSN[256]={0}。

uSNSize

UINT 型变量，数组 *lpszBoardSN* 的大小，无符号整型。

Return Values

- 0: 读取成功;
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到。

Remarks

获取指定工位和指定通道对应板卡不同模块的 SN 号。

Example

例 1: 获取工位 1，管脚 SDA_DP 对应通道所在板卡的主模块的 SN 号。

```
char lpszBoardSN[256] = { 0 };  
///  
//<Read the serial number of the board  
dcm.GetBoardSN("SDA_DP", 0, lpszBoardSN, sizeof(lpszBoardSN));
```

4.1.46.2 GetBoardHDRev()

```
int GetBoardHDRev(  
const char*      lpszPinName,  
USHORT          usSiteNo,  
char*           lpszHardRev,  
UINT            uRevSize);
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择读取板卡硬件版本的工位。0 代表 Site1，1 代表 Site2.....

lpszHardRev

char 型变量，存储板卡的硬件版本号。未存储返回 “N/A”。

由用户定义，如：char lpszHardRev[256]={0}。

uRevSize

UINT 型变量，数组 *lpszHardRev* 的大小，正整数。

Return Values

- 0: 读取成功;
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到。

Remarks

获取指定工位和指定通道对应板卡不同模块的的硬件版本号。

Example

例 1：获取工位 1，管脚 SDA_DP 对应通道所在的大板的硬件版本号。

```
char lpszHardRev[256] = { 0 };  
// Read the hardware version  
dcm.GetBoardHdRev("SDA_DP", 0, lpszHardRev, sizeof(lpszHardRev));
```

4.1.47 取校准信息函数

4.1.47.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime(  
const char* lpszPinName,
```

```
USHORT      usSiteNo) const;
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择要获取校准日期的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回指定通道校准发生的日期。未存储、工位无效、管脚名称未定义或板卡未找到时返回 1970-1-1 8:0:0。

Remarks

获取指定通道校准发生的日期。

Example

例 1：获取工位 1，管脚 SDA_DP 对应通道的校准日期。

```
SYSTEMTIME CalDatetime = dcm.GetCalibrationTime("SDA_DP", 0);
```

4.1.47.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(  
const char*      lpszPinName,  
USHORT          usSiteNo) const;
```

Parameters

lpszPinName

管脚名称。

usSiteNo

查询通道对应的工位。0 代表 Site1，1 代表 Site2.....

Return Values

- >0: 通道校准时的温度
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到;
- 43: 管脚不属于当前 DCM 实例。

Remarks

获取该通道校准时的温度。

Example

例 1: 获取工位 1，管脚 SDA_DP 对应通道校准时的温度。

```
double dTemper = dcm.GetCalibrationTemperature("SDA_DP",0);// Get the temperature of calibration
```

4.1.47.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(  
const char*    lpszPinName,  
USHORT        usSiteNo) const;
```

Parameters

lpszPinName

管脚名称

usSiteNo

查询通道对应的工位。0 代表 Site1，1 代表 Site2.....

Return Values

- >0: 通道校准时的湿度
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到。

Remarks

获取该通道校准时的湿度。

Example

例 1: 获取工位 1，管脚 SDA 对应通道校准时的湿度。

```
double dHumidity = dcm.GetCalibrationHumidity("SDA", 0); // Get the humidity when calibration
```

4.1.47.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(  
const char*   lpszPinName,  
USHORT       usSiteNo) const;
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择要获取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果，为枚举值。

CAL_PASS: 校准 PASS

CAL_FAIL: 校准 FAIL

STS_NO_RESULT_RECORD: 未存储、该工位失效、管脚名未定义或板卡未找到

Remarks

获取指定通道的校准结果。

Example

例 1：获取工位 1，管脚 SDA_DP 对应通道校准时的校准结果。

```
CAL_RESULT CalResult = dcm.GetCalibrationResult("SDA_DP", 0);
```

4.1.47.5 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
const char*    lpszPinName,  
USHORT        usSiteNo,  
char*         lpszMeterSN = NULL,  
UINT          uSNSize = 0);
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择要获取校准表信息的工位。0 代表 Site1，1 代表 Site2.....

lpszMeterSN

char 型变量，存储校准表的 SN。未存储返回 “N/A”。

由用户定义，如：char *lpszMeterSN*[256]={0}。

默认值为 `nullptr`，不获取校准表的 SN。

uSNSize

UINT 型变量，数组 *lpszMeterSN* 的大小，正整数。

默认值为 0，不获取校准表的 SN。

Return Values

返回该通道校准表的类型，为枚举值。

STS_KEITHLEY2000:	Keithley2000
STS_AGILENT34401:	Agilent34401
STS_AGILENT3458A:	Agilent3458A
STS_NO_METER_RECORD:	未存储、工位无效、管脚名未定义或板卡未找到。

Remarks

获取指定通道校准表的类型和校准表的 SN。

Example

例 1：获取工位 1，管脚 SDA_DP 对应通道校准时使用校准表的类型和校准表的 SN。

```
char lpszMeterSN[256] = { 0 };  
CAL_METER CalMeter = dcm.GetCalibrationMeter("SDA_DP", 0, lpszMeterSN, sizeof(lpszMeterSN));
```

4.1.47.6 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(  
const char*    lpszPinName,  
USHORT        usSiteNo) const;
```

Parameters

lpszPinName

管脚名称。

usSiteNo

选择要获取校准使用的逻辑版本的工位。0 代表 Site1，1 代表 Site2.....

Return Values

- >=0: 正常返回逻辑版本号;
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到;
- 43: 管脚不属于当前 DCM 实例。

Remarks

获取指定通道校准时的逻辑版本号。

Example

例 1: 获取工位 1，管脚 SDA_DP 对应通道校准时的逻辑版本号。

```
int nCalLogicRev = dcm.GetCalibrationLogicRev("SDA_DP", 0); // Get calibration logic Version of SITE_1
```

4.1.47.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
const char*   lpszPinName,  
char*         lpszBoardSN,  
UINT          uSNSize = 0,  
char*         lpszHardRev = NULL,  
UINT          uRevSize = 0);
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

lpszBoardSN

存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char *lpszBoardSN*[255]={0}。

uSNSize

数组 *lpszBoardSN* 的大小。

lpszHardRev

存储校准时使用的校准板的硬件版本信息。异常返回 “N/A”。

由用户定义，如：char *lpszHardRev*[256]={0}。

uRevSize

数组 *lpszBoardHardRev* 的大小。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：

- 1) 未加载向量；
- 2) 管脚名称未定义；
- 3) 工位超限；
- 4) 通道未校准。

Remarks

获取指定通道校准时使用的校准板相关信息。

Example

例 1：获取工位 1，管脚 SDA_DP 对应通道校准时使用的校准板信息。

```
char lpszBoardSN[256] = { 0 };          ///< The buffer for saving Serial NO. of calibration board
char lpszBoardHardRev[256] = { 0 };     ///< The buffer for saving hardware version of calibration board
int nLogicRev = dcm.GetCalibrationCalBoardInfo("SDA_DP", 0, lpszBoardSN, sizeof(lpszBoardSN),
lpszBoardHardRev, sizeof(lpszBoardHardRev));
```

4.1.47.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(
const char*    lpszPinName,
USHORT        usSiteNo,
char*          lpszSoftRev,
UINT          uRevSize);
```

Parameters

lpszPinName

管脚名称

usSiteNo

选择要获取校准使用的软件版本的工位。0 代表 Site1，1 代表 Site2.....

lpszSoftRev

存储校准使用的软件版本。未存储返回“N/A”。

由用户定义，如：char lpszSoftRev[256]={0}。

uRevSize

数组 lpszSoftRev 的大小，正整数。

Return Values

- 0: 读取成功;
- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到;
- 43: 管脚不属于当前 DCM 实例。

Remarks

获取指定通道校准时使用的软件版本。

Example

例 1: 获取工位 1, 管脚 SDA_DP 对应通道校准时使用的软件版本。

```
char lpszSoftRev[255] = { 0 };  
dcm.GetCalibrationSoftRev("SDA_DP", 0, lpszSoftRev, sizeof(lpszSoftRev)); // Get calibration software  
revision of SITE_1
```

4.1.47.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(  
const char*    lpszPinName,  
USHORT        usSiteNo) const;
```

Parameters

lpszPinName

管脚名称。

usSiteNo

选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

>0: 板卡校准时所处的槽位信息

- 9: 管脚名称未定义;
- 10: 工位号不存在;
- 16: 板卡未找到;
- 43: 管脚不属于当前 DCM 实例。

Remarks

获取指定通道校准时所处的槽位信息。

Example

例 1: 获取工位 1, 管脚 SDA_DP 对应通道校准时使用的槽位。

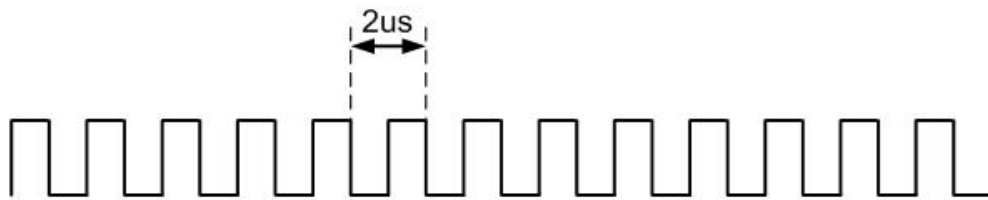
```
intnSlotID = dcm.GetCalibrationSlotID("SDA_DP",0);// Get the slot ID of calibration
```

4.1.48 编程范例

4.1.48.1 编辑向量文件

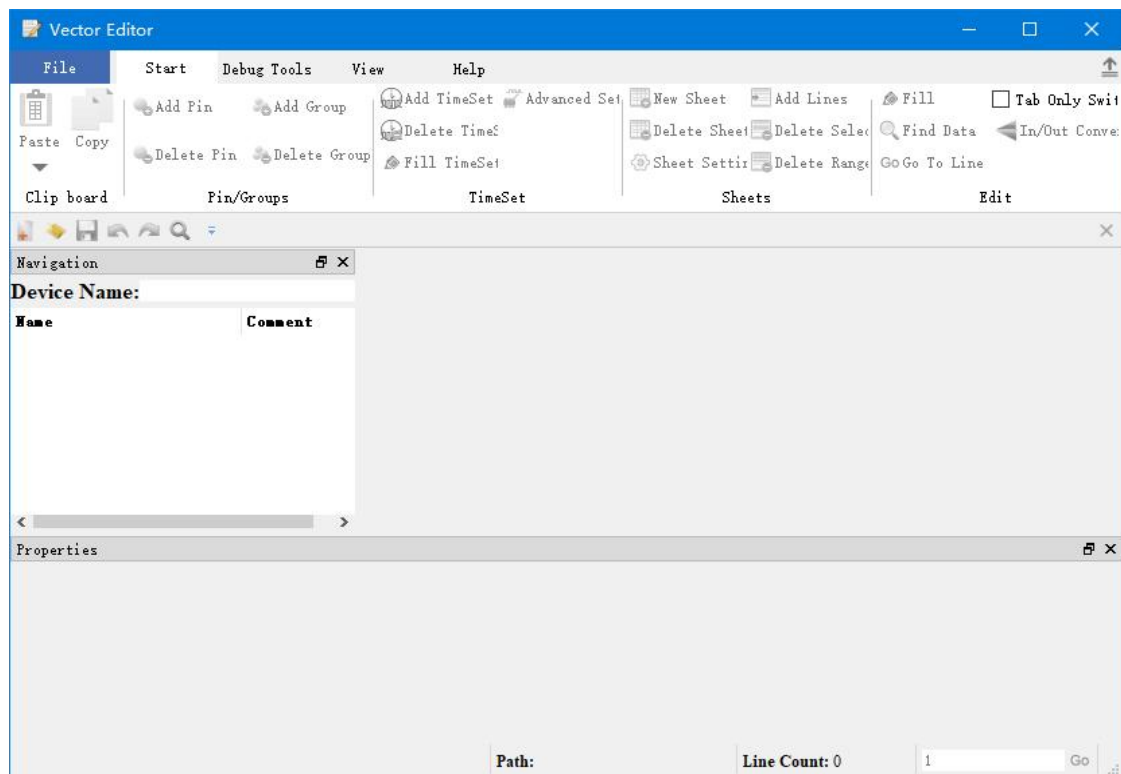
测试条件:

四工位测试, 控制 DCM 的通道 0/2/4/6 同步输出 NRZ 格式的逻辑信号为 10101010101010101010101010 的波形, 如下图所示波形, 频率为 500KHz, 低电平为 0V, 高电平为 5V, 驱动前沿为 500nS;

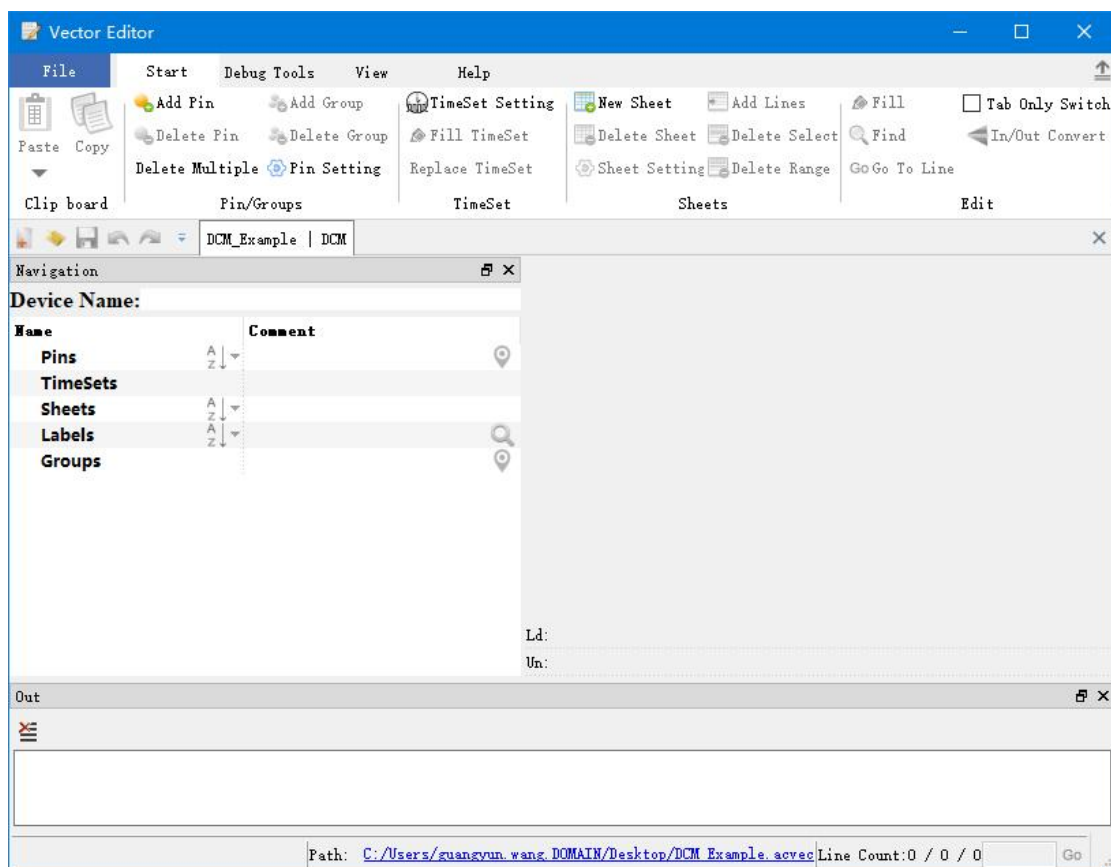


同时, 控制 DCM 的通道 1/ 3/5/7 同步对 4 个 Site 上的被测器件的输出波形进行比较, 比较高电平阈值电压为 2.5V, 比较低电平阈值电压为 0.4V, 边沿比较方式, 边沿比较时刻 1000nS:

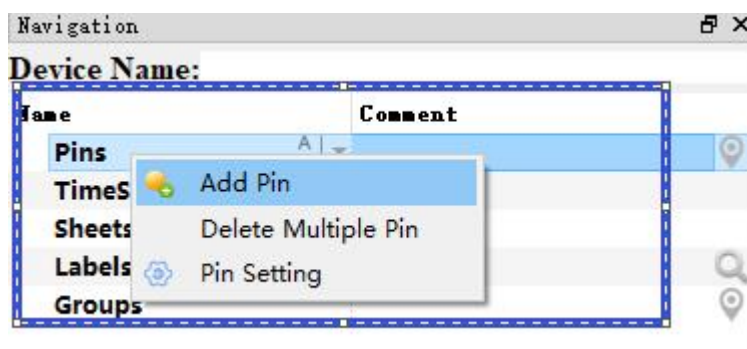
Step1: 点击 AccoTEST System Control 界面中的 Vector Editor, 弹出 Vector Editor 界面, 如下图



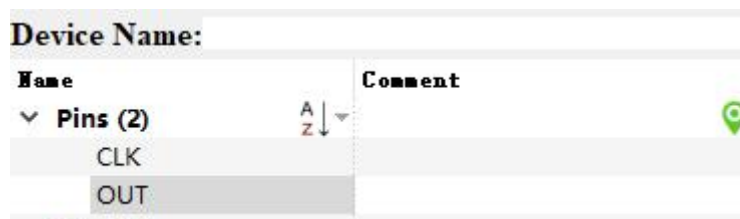
点击“File”，然后点击“New”新建一个向量文件，在弹出界面中选择文件路径和文件名称 DCM_Example，之后弹出下图界面：



Setp1:在上图中 Device Name 处填写 Device Name，右击“Pins”选择“Add Pin”，如下图所示

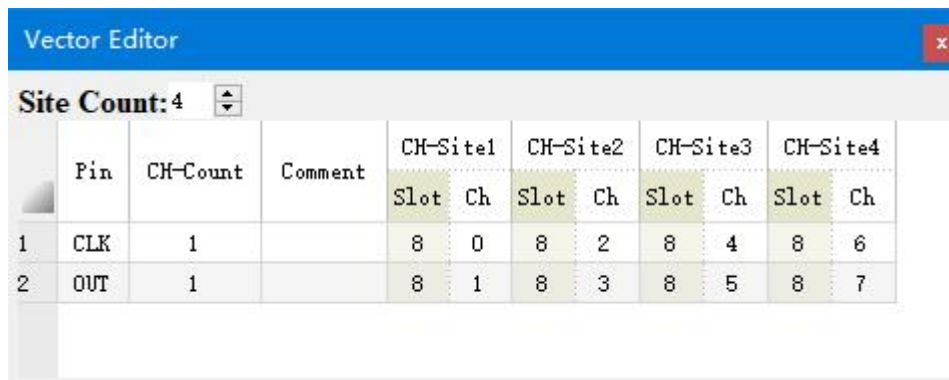


依次添加管脚 CLK、OUT，如下图所示



之后右击选择“Pin Setting”进入管脚通道设置界面，填入每个 pin 的通道，如下

图：

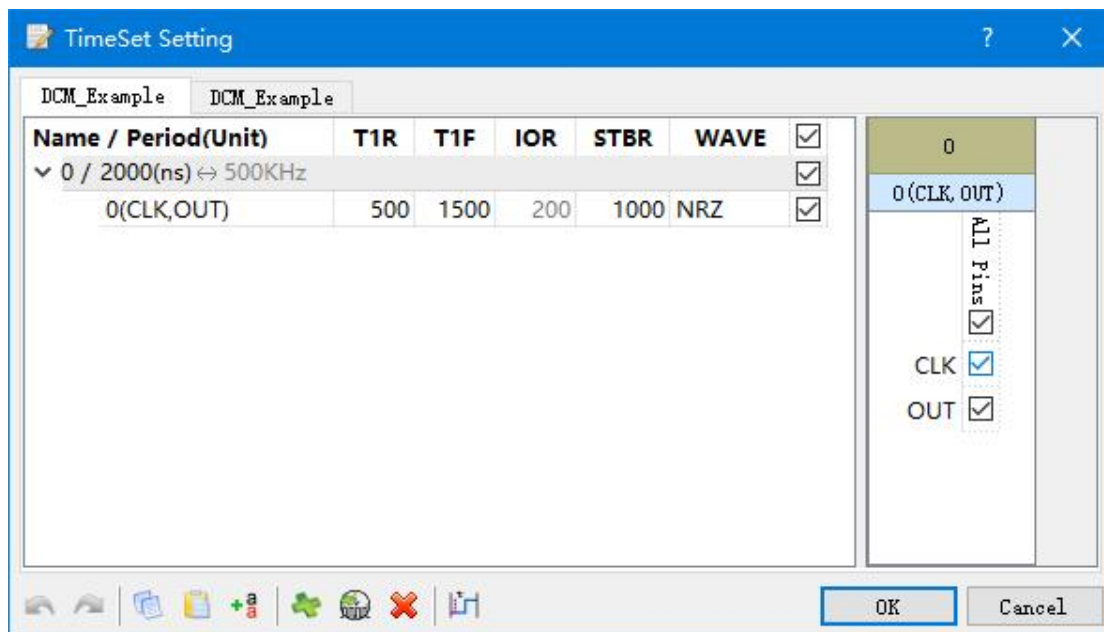


	Pin	CH-Count	Comment	CH-Site1		CH-Site2		CH-Site3		CH-Site4	
				Slot	Ch	Slot	Ch	Slot	Ch	Slot	Ch
1	CLK	1		8	0	8	2	8	4	8	6
2	OUT	1		8	1	8	3	8	5	8	7

Step2 : 右击 “ TimeSets ” ， 点击 “ TimeSet Setting ”



进入 TimeSet 设置界面；



Period: 波形速率，波形周期 2000nS，对应波形频率为 500KHz

T1R: 驱动前沿时刻，500nS

T1F: 驱动后沿时刻，1500nS（驱动波形格式为 NRZ 时，T1F 的设置无效）

IOR: IO 波形前沿时刻，200nS，从 200nS 时使能 IO

STBR: 比较前沿，1000nS，如果 Stb Type 选择边沿比较模式，则从 1000nS 开始比较

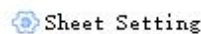
Wave: 驱动波形格式，NRZ

Step3: 右击 Sheets, 选择 “New Sheet”



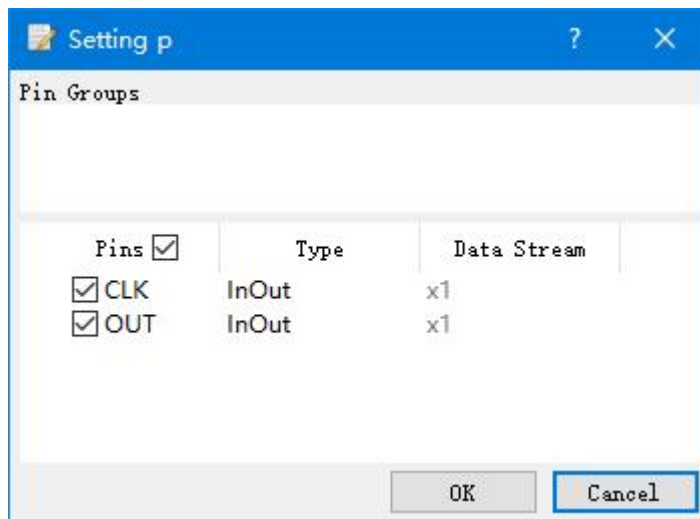
添加一

张表。点击上方工具栏中的 “Sheet Setting”

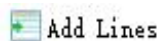


, 在弹出的窗口中选择

所有的 Pin, 并点击 OK, 如下图:

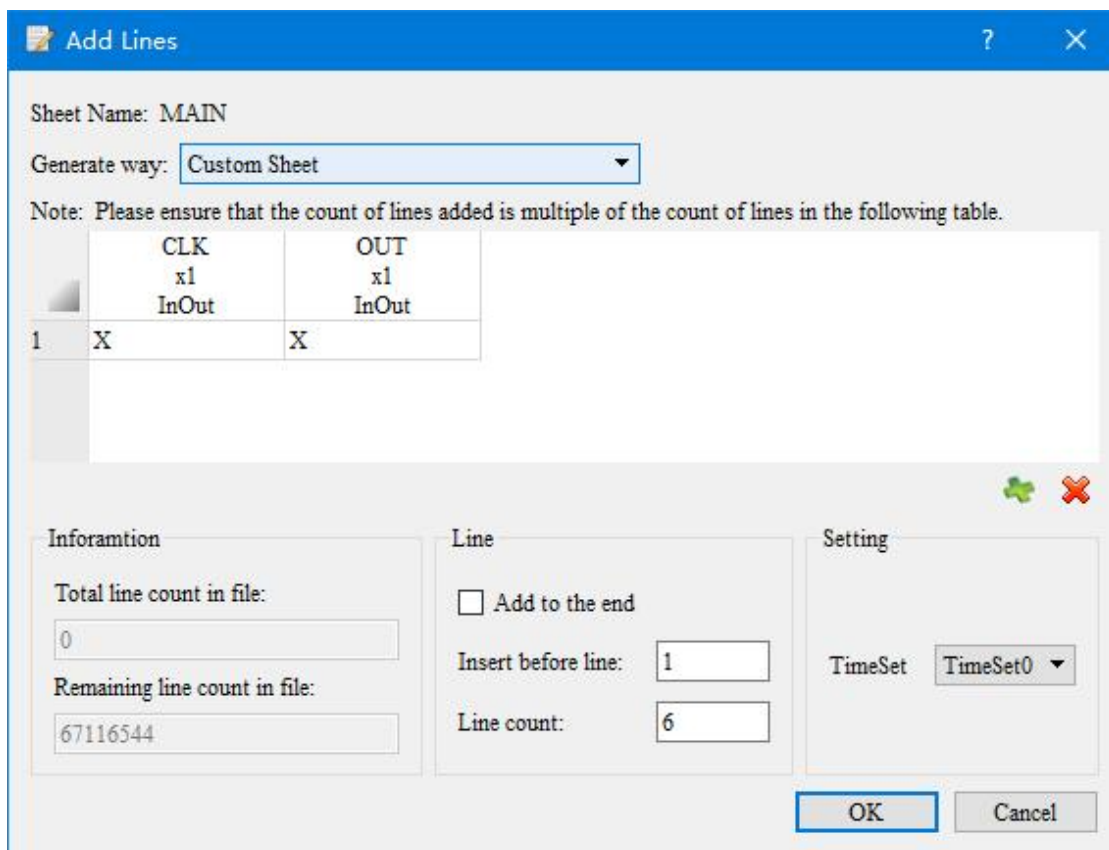


点击上方工具栏中的 “Add Lines”



, 在弹出的窗口中设置增加的向

量行数 6, 然后点击 “OK”, 如下图



添加如下图所示向量

	Label	CLK x1 InOut	OUT x1 InOut	Instruction	TimeSet	PTMU	Comment
1	START	X	X	INC	TimeSet0		
2		1	H	SET_LOOPA:12	TimeSet0		
3		0	L	INC	TimeSet0		
4	LOOP_LABEL	1	H	INC	TimeSet0		
5		0	L	END_LOOPA:LOOP_LABEL	TimeSet0		
6	STOP	X	X	INC	TimeSet0		

START: 向量起始行标号

STOP: 向量结束行标号

SET_LOOPA 指令, 后跟 Operand 设置循环的次数

INC: 加 1 指令, 执行完此行会继续执行下一行向量

END_LOOPA 指令, 后跟 Operand 设置循环的起始点

LOOP_LABEL: 辅助 END_LOOPA 指令完成循环起始点的设置

时间分离集的选择, 选择分离集 “TimeSet0”, 即选择了如图 step2 中对波形速率 (500KHz), 格式(NRZ)等的设置

现详述各行向量的运行结果:

LineNo2 执行时, CLK 驱动波形为高电平, OUT 预期波形为低电平, 同时设置循环次数为 12;

LineNo3 执行时, CLK 端波形为低电平, OUT 预期波形为高电平;

LineNo4 执行时, CLK 端波形为高电平, OUT 预期波形为低电平;

LineNo5 执行时, CLK 端波形为低电平, OUT 预期波形为高电平, 且执行完此行向量, 指令会跳转到 LineNo4 并执行 LineNo4 行向量, 然后再执行 LineNo5 行向量, 然后再跳转到 LineNo4, 依次类推, 直到将 LineNo4 和 LineNo5 两行向量循环执行 13 次;

由此, LineNo2 和 LineNo3 产生一个脉冲周期, LineNo4 和 LineNo5 产生一个脉冲周期, 然后循环 12 次, 又产生 12 个脉冲周期, 即循环体 LineNo4 和 LineNo5 总共执行 13 次, 因此加起来刚好 14 个脉冲周期, 同时, 在每个脉冲周期, OUT 端的波形也会同步和预期波形进行比较。

文件保存后, 会在选择的路径中生成 DCM_Example.acvec 的文件, 即向量文件。

4.1.48.2 Load Pattern

DCM 是通过在测试开始前（加载 *.pgs 文件时），将向量文件（*.acvec 文件）载入，然后再通过在测试程序中调用相关的函数来运行向量文件，从而达到控制 DCM 各 I/O 通道的目的。

一般会将 DCM 的 LoadVectorFile()函数写在 UserLoad()函数中，这样在加载 *.pgs 文件时，就会同时载入整个向量文件，这种做法的好处是可以节约时间，通过在测试开始前一次性载入整个向量文件，然后在整个测试过程中只需调用相应的函数运行向量文件中的某一段向量即可。另外，需要设置之后使用的管脚组，即调用 SetPinGroup 设置管脚组。

具体的调用方式如下：

```
DUT_API void UserLoad()
{
    dcm.LoadVectorFile("DCM_Example.acvec");
    dcm.SetPinGroup("G_CLK","CLK");
    dcm.SetPinGroup("G_OUT","OUT");
    dcm.SetPinGroup("G_ALLPIN","CLK,OUT");
}
```

4.1.48.3 Run Vector

测试程序中需通过调用一些用户函数来控制向量的运行，调用方法如下：

测试程序：

```
#define SITE_NUM 4
dcm.Connect("G_ALLPIN");//connect channel relay
dcm.InitMCU("G_ALLPIN");//Initialize the MCU
dcm.SetPinLevel("G_CLK", 5, 0, 0, 0);//VIH=5V,VIL=0V,Freq=500KHz,NRZ
dcm.SetPinLevel("G_OUT", 0, 0, 2.8, 0.5);
dcm.Connect("G_ALLPIN");
dcm.RunVectorWithGroup("G_ALLPIN", "START", "STOP",TRUE);
int nPinResult = 0;
for (USHORT usSiteNo = 0; usSiteNo< SITE_NUM; ++usSiteNo)
{
    nPinResult = dcm.GetMCUPinResult("OUT", usSiteNo);
    // CompareResult->SetTestResult(usSiteNo, 0, nPinResult);
}
dcm.Disconnect("G_ALLPIN");// disconnect channel relay
```

4.2 I2C 相关函数

4.2.1 I2CSet()

```
int I2CSet(  
float          fTime,  
int            nSiteCount,  
I2C_REG_ADD_MODE RegAddrMode,  
const char*    lpszSCLChannel,  
const char*    lpszSDAChannel);
```

Parameters

fTime

I2C 的周期, 范围: 12~4000000, 单位 nS。低于 12nS 自动选择 12nS; 高于 4000000(即 4ms), 选择 4000000。

nSiteCount

工位数, 取值范围 1~32。

RegAddrMode

寄存器地址模式, 有四种模式:

DCM_REG8: 寄存器地址为 8 位

DCM_REG16: 寄存器地址为 16 位

DCM_REG24: 寄存器地址为 24 位

DCM_REG32: 寄存器地址为 32 位

lpszSCLChannel

各工位的 SCL 对应的通道。每工位通道的格式为 “S%d_ %d”, 例如槽 8 的通道 10 表示为 “S8_10”。工位间以 “,” 分隔。

lpszSDAChannel

各工位的 SDA 对应的通道。每工位通道的格式为“S%d_%d”，例如槽 8 的通道 10 表示为“S8_10”。工位间以“,”分隔。

Return Values

- 0: 设置成功
- 1: 工位超限
- 2: 寄存器模式错误
- 3: 参数 *lpszSCLChannel* 或者 *lpszSDAChannel* 为空
- 4: 板卡未插入
- 5: 通道超限
- 6: 参数 *lpszSCLChannel* 或者 *lpszSDAChannel* 的格式错误
- 7: 参数 *lpszSCLChannel* 或者 *lpszSDAChannel* 的通道数和工位不匹配
- 8: SCL 和 SDA 通道冲突

Remarks

如果程序中使用了向量文件，则此函数不能在函数 LoadVectorFile()之前调用。设置 I2C 工作周期、模式、工位，指定使用为 I2C 协议的物理管脚。建议工位数使用宏定义，减少错误概率。

注意：

- 1) 使用 I2C 操作函数前，必须先设置通道，即必须先调用 I2CSet;
- 2) 此函数调用后，会重置 SCL 和 SDA 的边沿设置，SCL 默认(10,10+fTime/2, 10, 10+fTime/2, 10 + fTime/2, fTime *3 / 4), SDA 默认(fTime/4, fTime*3/4, fTime/4, fTime*3/4, 10 + fTime/2,fTime * 3 / 4)。

Example

例 1: I2C 工作频率 100KHz（周期 10000nS），正常模式，寄存器地址 8 位，4

工位，各工位的 SCL 为对应为 8 槽的 1/3/5/7，SDA 对应的为 8 槽的 2/4/6/8。

```
#define SITE_NUM 4  
dcm.I2CSet(10000, SITE_NUM, DCM_REG8, "S8_1,S8_3,S8_5,S8_7", "S8_2,S8_4,S8_6,S8_8");
```

4.2.2 I2CSetStopStatus()

```
void I2CSetStopStatus(BOOL bHighImpedance);
```

Parameters

bHighImpedance

设置 I2C 结束时，时钟和数据通道是否保持高阻态。

TRUE: 保持高阻

FALSE: 保持高电平

Remarks

修改 I2C 运行结束时，时钟和数据通道的默认状态。

注意：

- 1) 不调用此函数设置时，I2C 停止时的默认状态为驱动高；
- 2) 建议在 I2CSet 后进行设置，I2C 已运行的向量越多，此函数耗时越多；
- 3) 此函数支持串行工位宏，需要注意工位有效性。

Example

例 1：设置 I2C 运行停止时，时钟和数据通道的状态为高阻态。

```
dcm.I2CSetStopStatus(TRUE);
```

4.2.3 I2CSetSCLEdge()

```
int I2CSetSCLEdge(  
double dT1R,
```

```
double dT1F);
```

Parameters

dT1R

驱动数据前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dT1F

驱动数据后沿时刻值，取值范围(*dT1R* + 4)~速率值 *dRate*，单位 nS。

Return Values

- 0: 设置成功
- 1: 没有设置工位通道
- 2: 前沿/后沿超范围

Remarks

修改 I2C 时钟通道 SCL 波形的时间参数。

注意：

- 1) 系统会默认设置 I2C 各通道的时间边沿参数，此函数用于调整 SCL 波形边沿参数；
- 2) SCL 的波形格式为 RO；
- 3) I2C 边沿以最近一次设置为准；

Example

例 1：设置 I2C 的 SCL 的时间参数，其中驱动数据前沿时刻值 5nS，驱动数据后沿时刻值 10nS。

```
dcm.I2CSetSCLEdge(5, 10);
```

4.2.4 I2CSetSDAEdge()


```
int I2CSetSDAEdge(  
double    dT1R,  
double    dT1F,  
double    dIOR,  
double    dSTBR);
```

Parameters

dT1R

驱动数据前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dT1F

驱动数据后沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dIOR

驱动 I/O 方向前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

dSTBR

比较前沿时刻值，取值范围 0~速率值 *dRate*，单位 nS。

Return Values

- 0: 设置成功
- 1: 没有设置工位通道
- 2: 前沿/后沿超范围

Remarks

修改 I2C 数据通道 SDA 波形的时间参数。

注意：

- 1) 系统会默认设置 I2C 各通道的时间参数，此函数用于调整 SDA 的波形边沿参数；
- 2) SDA 的波形格式为 NRZ；

- 3) SDA 的 T1R 和 IOR 需要比 SCL 的 T1R 大 2ns，以防由于通道边沿对齐误差，导致时钟高电平期间，数据管脚电平出现高低变化，导致 I2C 通信失败；
- 4) 建议将 T1R 与 IOR 设置成相同值，保证 I/O 切换与数据同步。

Example

例 1：设置 I2C 的 SDA 的时间参数，其中驱动数据前沿时刻值 5nS，驱动数据后沿时刻值 10nS，驱动 I/O 方向前沿时刻 5nS，比较前沿时刻 30nS。

```
dcm.I2CSetSDAEdge(5, 10, 5, 30);
```

4.2.5 I2CConnect()

```
int I2CConnect();
```

Return Values

- 0：成功
- 1：没有设置工位通道

Remarks

闭合 I2C 各工位的继电器。

注意：

- 1) 此函数需要在设置各工位通道后使用，即使用 I2CSet 之后；
- 2) 此函数与继电器操作函数(Connect/Disconnect)的功能相似，且会相互影响。当 I2C 与其他功能共用通道，则需要考虑这函的相互影响，通道继电器的实际状态以最近一次设置为准。

Example

例 1：闭合所有有效工位的 I2C 通道的继电器。

```
dcm.I2CConnect();
```

4.2.6 I2CDisconnect()

```
int I2CDisconnect();
```

Return Values

- 0: 成功
- 1: 没有设置工位通道

Remarks

断开 I2C 各工位的继电器。

注意：

- 1) 此函数需要在设置各工位通道后使用，即使用 I2CSet 之后；
- 2) 此函数与继电器操作函数(Connect/Disconnect)的功能相似，且会相互影响。当 I2C 与其他功能共用通道，则需要考虑这函的相互影响，通道继电器的实际状态以最近一次设置为准。

Example

例 1：断开所有有效工位的 I2C 通道继电器。

```
dcm.I2CDisconnect();
```

4.2.7 I2CSetPinLevel()

```
int I2CSetPinLevel(  
double    dVIH,  
double    dVIL,  
double    dVOH,  
double    dVOL);
```

Parameters

dVIH

驱动高电平电压值，取值范围-1.5~6，单位 V。

dVIL

驱动低电平电压值，取值范围-1.5~*dVIH*，单位 V。

dVOH

比较高阈值电压，取值范围-1.5~6，单位 V。

dVOL

比较低阈值电压值，取值范围-1.5~*dVOH*，单位 V。

Return Values

- 0: 成功
- 1: 电压值设置错误
- 2: 未设置通道信息

Remarks

设置 I2C 波形的高低电压，比较电压阈值。

注意：

- 1) 与 SetPinLevel 作用相同，管脚的全局电平值，如果 I2C 与其他功能共用管脚，则需要注意这两个函数会相互影响，管脚的实际电平以最近设置的电平为准；
- 2) 此函数会将 I2C 有效工位的通道切换成 MCU 模式；
- 3) 此函数会关掉通道的动态负载。

Example

例 1：设置 I2CSCL 和 SDA 的电平，驱动高 3.3V，驱动低 0V，比较高电平 2.4V，比较低电平 0.8V。

```
dcm.I2CSetPinLevel( 3.3, 0, 2.4, 0.8);
```

4.2.8 I2CSetPinLevel()

```
int I2CSetPinLevel(  
double          dVIH,  
double          dVIL,  
double          dVOH,  
double          dVOL,  
DCM_I2C_CHANNEL I2CChannelType);
```

Parameters

dVIH

驱动高电平电压值，取值范围-1.5~6，单位 V。

dVIL

驱动低电平电压值，取值范围-1.5~*dVIH*，单位 V。

dVOH

比较高阈值电压，取值范围-1.5~6，单位 V。

dVOL

比较低阈值电压值，取值范围-1.5~*dVOH*，单位 V。

I2CChannelType

设置电平的 I2C 通道类型，可选：

DCM_I2C_SCL: I2C 的 SCL 通道

DCM_I2C_SDA: I2C 的 SDA 通道

DCM_I2C_BOTH: I2C 的 SCL 和 SDA 通道

Return Values

0: 成功

-1: 电压值设置错误

-2: 未设置通道信息

Remarks

按照通道类型设置 I2C 波形的驱动高低电压，比较电压阈值。

注意：

- 1) 与 SetPinLevel 作用相同，管脚的全局电平值，如果 I2C 与其他功能共用管脚，则需要注意这两个函数会相互影响，管脚的实际电平以最近设置的电平为准；
- 2) 此函数最后一个参数可以不填写，不填写的情况下调用不带最后一个参数的 I2CSetPinLevel 函数；
- 3) 此函数会将 I2C 有效工位的通道切换成 MCU 模式；
- 4) 此函数会关掉通道的动态负载。

Example

例 1：设置 I2CSCL 的电平，驱动高 3.3V，驱动低 0V，比较高电平 2.4V，比较低电平 0.8V。

```
dcm.I2CSetPinLevel( 3.3, 0, 2.4, 0.8, DCM_I2C_SCL);
```

例 2：设置 I2CSDA 的电平，驱动高 3.0V，驱动低 0V，比较高电平 1.5V，比较低电平 0.8V。

```
dcm.I2CSetPinLevel( 3.3, 0, 1.5, 0.8, DCM_I2C_SDA);
```

例 3：设置 I2CSCL 和 SDA 的电平，驱动高 3.3V，驱动低 0.5V，比较高电平 2.0V，比较低电平 1V。

```
dcm.I2CSetPinLevel( 3.3, 0.5, 2.0, 1.0, DCM_I2C_BOTH);
```

4.2.9 I2CSetDynamicLoad()

```
int I2CSetDynamicLoad(  
DCM_I2C_CHANNEL    I2CChannelType,  
BOOL                bEnable,
```

```
double      dIOH=0.005,  
double      dIOL=0.005,  
double      dVTVoltValue=3.5);
```

Parameters

I2CChannelType

设置电平的 I2C 通道类型，可选：

DCM_I2C_SCL: I2C 的 SCL 通道

DCM_I2C_SDA: I2C 的 SDA 通道

DCM_I2C_BOTH: I2C 的 SCL 和 SDA 通道

bEnable

是否开启动态负载：

TRUE: 开动态负载

FALSE: 关闭动态负载

dIOH

非驱动模式时，管脚电压高于 VT 时，DCM 通道的电流值，取值范围-0.006~0.018，单位：A，默认值 0。正方向为 DCM 输入电流。

dIOL

非驱动模式时，管脚电压低于 VT 时，DCM 通道的电流值，取值范围-0.006~0.018，单位：A，默认值 0。正方向为 DCM 输出电流。

dVTVoltValue

VT 电压值，取值范围-1.25~5.75，单位：V，默认值 3.5。

Return Values

0: 设置成功

-1: 没有设置 I2C 通道

-2: 电流超范围

-3: VT 电压超范围

Remarks

设置动态负载的电流及工作方式。

注意：

- 1) IOH 和 IOL 的电流由 VT 提供，因此 IOH/IOL 提供的最大电压为 VT；
- 2) 此函数会将输入管脚的有效工位对应的通道切换成 MCU 模式；
- 3) 修改通道电平会关掉动态负载；

Example

例 1：设置 I2C 所有通道，VT 电压 3.5V，开启动态负载，IOH 为 0.010A，IOL 为 0.010A。

```
dcm.I2CSetDynamicLoad(DCM_I2C_BOTH, TRUE, 0.010, 0.010, 3.5);
```

4.2.10 I2CWriteData()

```
int I2CWriteData(  
BYTE    bySAddress,  
ULONG   ulRegAddress,  
int      nDataLength,  
DWORD   dwDataArray[]);
```

Parameters

bySAddress

I2C 从机地址。

ulRegAddress

I2C 寄存器地址。

nDataLength

要写数据的字节数，取值范围 1~4。

dwdataArray

I2C 写数据数组，依次存放各工位要写的的数据。

Return Values

- 0: 成功
- 1: 数据字节数小于 1
- 2: 未设置通道信息

Remarks

I2C 写数据，各工位数据可以不同。

注意：

- 1) 此函数会将关闭 I2C 无效工位的通道；
- 2) 此函数会将 I2C 有效工位的通道打开，并切换成 MCU 模式。

Example

```
//Write device 0,address 5,four sites,data width 16bit,site data is in turn 0x55AA, 0xAA55,0xA5A5,
0x5A5A
#define SITE_NUM 4

ULONG uldataArray[SITE_NUM] = {0x55AA, 0xAA55, 0xA5A5, 0x5A5A};
dcm.I2CWriteData(0, 5, 2, uldataArray);
```

4.2.11 I2CWriteData()

```
int I2CWriteData(
BYTE    bySAddress,
ULONG   ulRegAddress,
int     nDataLength,
```

```
BYTE* pbyDataWritten);
```

Parameters

bySAddress

I2C 从机地址。

ulRegAddress

I2C 寄存器地址。

nDataLength

要写数据的字节数，取值范围 1~4。

dwDataWritten

所有工位需要写入的数据。

Return Values

0: 成功

-1: 数据字节数小于 1

-2: 未设置通道信息

Remarks

I2C 各工位写入相同的数据。

注意：

- 1) 此函数会将关闭 I2C 无效工位的通道；
- 2) 此函数会将 I2C 有效工位的通道打开，并切换成 MCU 模式。

Example

```
//Write device 0,address 5,four sites,data width 16bits:0xAA, 0x55,0xA5, 0x5A  
BYTEabyDataWritten[4]={0xAA,0x55,0xA5,0x5A};  
dcm.I2CWriteData(0, 5, 4, abyDataWritten);
```

4.2.12 I2CWriteMultiData()

```
int I2CWriteMultiData(  
BYTE    bySAddress,  
ULONG   ulRegAddress,  
int      nDataLength,  
BYTE*    pbyDataAddArray[]);
```

Parameters

bySAddress

I2C 从机地址。

ulRegAddress

I2C 寄存器地址。

nDataLength

要写数据的字节数。参数的取值范围为 1~1000。

pbyDataArray

指针数组，存放各工位数据数组的地址。

注意：

指针数组的元素个数不能小于 I2C 的工位，且指向的数组的字节数不能小于要写入的字节数，否则会导致内存越界的严重错误。

Return Values

- 0: 成功
- 1: 数据字节数小于 1
- 2: 未设置 I2C 的工位及通道
- 3: 数据指针为空
- 4: 向量行数不足

Remarks

I2C 写数据，各工位的数据可以不同；*nDataLength* 的大小必须与数据数组中数据的总长度相对应。

注意：

- 1) 此函数会关闭 I2C 无效工位的通道；
- 2) 此函数会将 I2C 有效工位的通道会被打开，并切换成 MCU 模式。

Example

```
#define SITE_NUM 4
BYTEbySite1Data[6] = {0xAA,0x55,0xAA,0x55, 0xAA,0xBB};
BYTEbySite2Data[6] = { 0x55,0xAA,0x55,0xAA, 0x55,0xAA };
BYTE bySite3Data[6] = { 0xAA,0xBB,0xAA,0xBB, 0xCC,0xDD };
BYTE bySite4Data[6] = { 0xAA,0xBB,0xCC,0xDD, 0xA5,0xA5 };
BYTE *byMultiDataArray[SITE_NUM] = { bySite1Data, bySite2Data, bySite3Data, bySite4Data };
dcm.I2CWriteMultiData(0, 5, 6, byMultiDataArray);
```

4.2.13 I2CReadData()

```
int I2CReadData (
BYTE    bySAddress,
ULONG   ulRegAddress,
int      nDataCount = 1);
```

Parameters

bySAddress

I2C 从机地址。

ulRegAddress

I2C 寄存器地址。

nDataCount

I2C 读取字节数。参数的取值范围为 1~100，缺省为 1。

Return Values

- 0: 成功
- 1: 数据字节数小于 1
- 2: 空间不足
- 3: 没有设置通道信息

Remarks

I2C 回读为同步存储，回读指令发送完成后，回读数据也存入了控制器内。

注意：

- 1) 此函数会关闭 I2C 无效工位的通道；
- 2) 此函数会将 I2C 有效工位的通道打开，并切换到 MCU 模式。

Example

```
//read device 0,address 5, readback 2 bytes  
dcm.I2CReadData(0, 5, 2);
```

4.2.14 I2CGetReadData()

```
int I2CGetReadData(  
USHORT    usSiteNo,  
int        nDataIndex);
```

Parameters

usSiteNo

工位号，取值范围 0 ~ I2CSet 有效工位数 - 1。

nDataIndex

回读数据的字节序号，取值范围 0 ~ *nDataCount*- 1

注意：

nDataCount 是函数 `I2CReadData()` 的参数。

Return Values

回读数据。

- 1: 工位号错误
- 2: 当前工位失效
- 3: 回读数据的字节序号错误
- 4: 没有设置通道信息
- 5: 没有读取 I2C

Remarks

获取 I2C 读操作的数据。实际上读操作已经完成，数据已经存入到对应的存储器中，本函数就是从指定的存储器读出指定的字节数据。

Example

```
//Get the first data nReadData of site1
dcm.I2CReadData(0, 5, 2);
int nReadData = dcm.I2CGetReadData(0,0);
```

4.2.15 I2CGetBitData()

```
ULONG I2CGetBitData(
USHORT    usSiteNo,
int        nBitPos,
int        nBitCount);
```

Parameters

usSiteNo

工位号，取值范围 0 ~ I2CSet 有效工位数 - 1。

nBitPos

获取数据位的起始位，取值范围 0 ~ $nDataCount * 8 - 1$ 。

注意：

nDataCount 最近一次 I2C 读取的字节数。

nBitCount

回读数据的位数，取值范围 1 ~ 32

Return Values

获取 I2C 读取到的数据。

注意：

未设置工位通道、工位无效、参数出错都会返回 0xFFFFFFFF

Remarks

获取 I2C 读操作的数据。实际上读操作已经完成，数据已经存入到对应的存储器中，本函数就是从指定的存储器读出指定的字节数据。

Example

```
ULONGulReadData = dcm.I2CGetBitData(0,10,2);
```

4.2.16 I2CGetNACKIndex()

```
int I2CGetNACKIndex(USHORT usSiteNo);
```

Parameters

usSiteNo

工位号，取值范围 0~I2C 工位数 -1。

Return Values

1, 2, 3,表示第一个 NACK 的位置

0: 没有 NACK

-1: 工位号错误

-2: 当前工位失效

-3: 没有设置通道信息

-4: 没有 I2C 读写操作

Remarks

获取最近一次 I2C 读写操作中，待测芯片发送的 NACK 信号的相对位置。

Example

```
//Get site1 the relative position of the NACK signal in the previous I2C read
dcm.I2CReadData(0, 5, 2);
dcm.I2CGetNACKIndex(0);
```

4.2.17 I2C 编程范例

四工位程序，每工位分别使用 I2C 读取 2 字节数据，然后写入 8 字节数据。

```
#define SITE_NUM 4

dcm.I2CSet(1000, SITE_NUM, DCM_REG8, "S9_0,S9_16,S9_32,S9_48", "S9_1,S9_17,S9_33,S9_49");
// Set the I2C pin of all sites, the length of register address is 8 bits.
// SITE   SCL       SCL
// 1      S9_0      S9_1
// 2      S9_16     S9_17
// 3      S9_32     S9_33
// 4      S9_48     S9_49
dcm.I2CConnect();//Connect I2C relay
// Set the Pin level of I2C.
// VIH    VIL      VOH    VOL
```



```
// 3.5    0.5    2.7    1.3
dcm.I2CSetPinLevel(3.5, 0.5, 2.7, 1.3);
// Send 2 bytes data to chips
// SITE_1 0xAABB
// SITE_2 0xAACC
// SITE_3 0xAA55
// SITE_4 0xA5A5
ULONG aulWriteData[SITE_NUM] = { 0xAABB, 0xAACC, 0xAA55, 0xA5A5 };
dcm.I2CWriteData(0xA0, 0xAA, 2, aulWriteData);
int anNACKIndex[SITE_NUM] = { 0 };

DWORD dwSiteStatus = STSGetsSiteStatus();
auto IsSiteValid = [&](USHORT usSiteNo)
{
    return dwSiteStatus >> usSiteNo & 0x01;
};
// Get the NACK index of each site.
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    if (!IsSiteValid(usSiteNo))
    {
        continue;
    }
    anNACKIndex[usSiteNo] = dcm.I2CGetNACKIndex(usSiteNo);
}
// Read 2 bytes data from the slave address 0xA0 and register address 0xAA
dcm.I2CReadData(0xA0, 0xAA, 2);
ULONG anReadData[SITE_NUM] = { 0 };
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    // Get the data read of each site.
    if (!IsSiteValid(usSiteNo))
    {
        continue;
    }
    anReadData[usSiteNo] = (dcm.I2CGetReadData(usSiteNo, 0) << 8) + dcm.I2CGetReadData(usSiteNo, 1);
    ReadResult->SetTestResult(usSiteNo, 0, anReadData[usSiteNo]);
}
// Get the NACK index of read operation.
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
{
    if (!IsSiteValid(usSiteNo))
    {
        continue;
    }
    anNACKIndex[usSiteNo] = dcm.I2CGetNACKIndex(usSiteNo);
    NACKIndex->SetTestResult(usSiteNo, 0, anNACKIndex[usSiteNo]);
}

// Write 8 bytes data to chips using I2C.
BYTE abySite1Data[8] = { 0xAA, 0xBB, 0xCC, 0xDD, 0xAA, 0x55, 0xAA, 0x55 };
BYTE abySite2Data[8] = { 0xA5, 0xA5, 0xA5, 0xA5, 0x5A, 0x5A, 0x5A, 0x5A };
BYTE abySite3Data[8] = { 0x55, 0xAA, 0xAA, 0x55, 0xAA, 0x55, 0x55, 0xAA };
BYTE abySite4Data[8] = { 0xAA, 0xAA, 0x55, 0x55, 0x55, 0xAA, 0x55, 0xAA };
BYTE* pabyWriteDataArray[4] = { abySite1Data, abySite2Data, abySite3Data, abySite4Data };
dcm.I2CWriteMultiData(0xA0, 0x00, 8, pabyWriteDataArray);
// Get the NACK index of each site.
for (USHORT usSiteNo = 0; usSiteNo < SITE_NUM; ++usSiteNo)
```

```
{
    if (!IsSiteValid(usSiteNo))
    {
        continue;
    }
    anNACKIndex[usSiteNo] = dcm.I2CGetNACKIndex(usSiteNo);
}
dcm.I2CDisconnect();//Disconnect the relay of all valid sites.
```

5 HSD

5.1 HSD()

```
HSD (BYTE logicNo);
```

```
HSD (BYTE logicNo, char *strName);
```

Parameters

logicNo

HSD 逻辑板卡号。范围 0~15。

Remarks

定义一块 HSD 板卡,并指定使用的逻辑板卡号。当按照系统最大配置即 16 块 HSD 板卡时,单工位工作时最大逻辑板卡号为 15,双工位并行工作时最大逻辑板卡号为 7,四工位并行工作时最大逻辑板卡号为 3。

Example

例 1: 假设系统配置为 2 块 HSD 板卡,双工位并行工作,定义可使用的 HSD。

```
HSD hsd(0); //使用逻辑板卡 0
```

对应的工位绑定如下:

```
StsSetModuleToSite(MD_HSD, SITE_1, 0, -1);
```

```
//绑定 HSD 的物理板卡 0 为 SITE1  
StsSetModuleToSite(MD_HSD, SITE_2, 1, -1);  
//绑定 HSD 的物理板卡 1 为 SITE2
```

5.2 Init()

```
int Init(  
    BOOL autoMeasure = FALSE);
```

Parameters

autoMeasure

使能或禁止自动测量功能。

TRUE 使能

FALSE 禁止

默认为 FALSE。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

初始化 HSD 板卡。执行该语句之后：

- 1) 采样频率/间隔 HSD_SAMPLE_RATE_1GHZ_OR_1NS;
- 2) 采样分辨率为 HSD_RESO_8BIT;
- 3) 通道电压范围为 HSD_VOLT_RANGE_10V;
- 4) 内插功能关闭；
- 5) 触发偏移点为 0.5；
- 6) 触发模式为 HSD_TRIG_MODE_AUTO。

Example

例 1：初始化 HSD 板卡。

```
hsd.Init(); //初始化 HSD 板卡
```

5.3 SetChannel()

```
int SetChannel(  
HSD_CHANNEL channel,  
HSD_VOLT_RANGE voltRange,  
HSD_COUPLING coupling = HSD_COUP_DC,  
HSD_BANDLIMIT bandLimit = HSD_BANDLIMIT_OFF,  
double voltOfst = 0.0,  
long phaseOfst = 0);
```

Parameters

channel

通道号，请注意不是板卡号。

HSD_CHANNEL_0	通道 0
HSD_CHANNEL_1	通道 1
HSD_CHANNEL_ALL	所有通道（通道 0 和通道 1）

voltRange

通道电压范围。

HSD_VOLT_RANGE_0P5V	电压范围 0.5V（-0.25~0.25）
HSD_VOLT_RANGE_1V	电压范围 1.0V（-0.5~0.5）
HSD_VOLT_RANGE_2V	电压范围 2.0V（-1.0~1.0）
HSD_VOLT_RANGE_5V	电压范围 5.0V（-2.5~2.5）
HSD_VOLT_RANGE_10V	电压范围 10.0V（-5.0~5.0）
HSD_VOLT_RANGE_20V	电压范围 20.0V（-10.0~10.0）
HSD_VOLT_RANGE_50V	电压范围 50.0V（-25.0~25.0）

coupling

输入耦合方式。

HSD_COUP_DC DC 耦合

HSD_COUP_AC AC 耦合

默认为 HSD_COUP_DC。

bandLimit

带宽抑制。

HSD_BANDLIMIT_OFF 关闭带宽抑制功能

HSD_BANDLIMIT_ON 打开带宽抑制功能

默认为 HSD_BANDLIMIT_OFF。

voltOfst

通道垂直偏移电压，取值范围-50 到 50，单位 V，默认值为 0.0。

phaseOfst

通道水平相位偏移，取值范围-100 到 100，单位 ns，默认值为 0。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

设置 HSD 板卡通道参数，包括电压范围、耦合方式、带宽抑制、垂直电压偏移和水平相位偏移。设置通道参数时，需要指明具体的通道号，也支持使用 HSD_CHANNEL_ALL 参数同时设置所有通道。

Example

例 1：设置所有通道的电压范围为 5V。

```
hsd.SetChannel(HSD_CHANNEL_ALL, HSD_VOLT_RANGE_5V);
```

5.4 SetEdgeTrig()

```
int SetEdgeTrig(  
HSD_TRIG_EDGE trigEdge = HSD_TRIG_EDGE_RISING,  
HSD_TRIG_SRC trigSrc = HSD_TRIG_SRC_CH0,  
double trigLevel = 0.0);
```

Parameters

trigEdge

触发边沿选择，上升沿或下降沿。

HSD_TRIG_EDGE_RISING 上升沿

HSD_TRIG_EDGE_FALLING 下降沿

默认为 HSD_TRIG_EDGE_RISING。

trigSrc

触发源选择。

HSD_TRIG_SRC_CH0 CH0 通道

HSD_TRIG_SRC_CH1 CH1 通道

HSD_TRIG_SRC_EXT0 外部触发 0

HSD_TRIG_SRC_EXT1 外部触发 1

默认为 HSD_TRIG_SRC_CH0。

trigLevel

触发电平，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发电平的取值范围为 -0.5Range 到 +0.5Range，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发电平的取值范围为 -2.5~+2.5。

当触发源为 EXT0 或 EXT1 时，触发电平参数无效。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

选择边沿触发并设置参数，包括边沿类型、触发源和触发电平。本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

注意：

1) HSD 支持的触发类型有边沿触发、N 边沿触发、脉宽触发、斜率触发，分别可以通过函数 SetEdgeTrig、SetNEdgeTrig、SetPulseTrig、SetSlopeTrig 来设置，实际使用时，只能使用其中一种触发类型，不能同时使用多种触发类型。

2) 当 StartCapture 函数的 trigMode 参数为 HSD_TRIG_MODE_AUTO 时，该函数所设置的参数失效。

Example

例 1：选择边沿触发，设置触发源为 CH0，上升沿，触发电平 0.0V。

```
hsd.SetEdgeTrig(HSD_TRIG_EDGE_RISING, HSD_TRIG_SRC_CH0, 0.0);
```

5.5 SetNEdgeTrig()

```
int SetNEdgeTrig(  
    DWORD edgeNum = 1,  
    HSD_TRIG_EDGE trigEdge = HSD_TRIG_EDGE_RISING,  
    HSD_TRIG_SRC trigSrc = HSD_TRIG_SRC_CH0,  
    double trigLevel = 0.0);
```

Parameters

edgeNum

N 边沿个数，表示第 N 个边沿触发，取值范围 1~65535，默认为 1。

trigEdge

触发边沿选择，上升沿或下降沿。

HSD_TRIG_EDGE_RISING 上升沿

HSD_TRIG_EDGE_FALLING 下降沿

默认为 HSD_TRIG_EDGE_RISING。

trigSrc

触发源选择。

HSD_TRIG_SRC_CH0 CH0 通道

HSD_TRIG_SRC_CH1 CH1 通道

HSD_TRIG_SRC_EXT0 外部触发 0

HSD_TRIG_SRC_EXT1 外部触发 1

默认为 HSD_TRIG_SRC_CH0。

trigLevel

触发电平，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发电平的取值范围为 -0.5Range 到 +0.5Range，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发电平的取值范围为-2.5~+2.5。

当触发源为 EXT0 或 EXT1 时，触发电平参数无效。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

选择 N 边沿触发并设置参数，包括 N 边沿个数、边沿类型、触发源、和触发电平。

本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

注意：

- 1) HSD 支持的触发类型有边沿触发、N 边沿触发、脉宽触发、斜率触发，分别可以通过函数 SetEdgeTrig、SetNEdgeTrig、SetPulseTrig、SetSlopeTrig 来设置，实际使用时，只能使用其中一种触发类型，不能同时使用多种触发类型。
- 2) 当 StartCapture 函数的 trigMode 参数为 HSD_TRIG_MODE_AUTO 时，该函数所设置的参数失效。

Example

例 1：选择 N 边沿触发，设置 N 边沿数量为 10，触发源为 CH0，上升沿，触发电平 0.0V。

```
hsd.SetNEdgeTrig(10, HSD_TRIG_EDGE_RISING, HSD_TRIG_SRC_CH0, 0.0);
```

5.6 SetPulseTrig()

```
int SetPulseTrig(  
double trigTime = 16E-9,  
HSD_TRIG_PULSE trigPulse = HSD_TRIG_PULSE_POS_MORE,  
HSD_TRIG_SRC trigSrc = HSD_TRIG_SRC_CH0,  
double trigLevel = 0.0);
```

Parameters*trigTime*

触发时间，取值范围 16E-9 到 10，单位 s，默认值 16E-9。

trigPulse

脉宽触发类型选择。

HSD_TRIG_PULSE_POS_MORE

正脉宽大于，下图右上部分

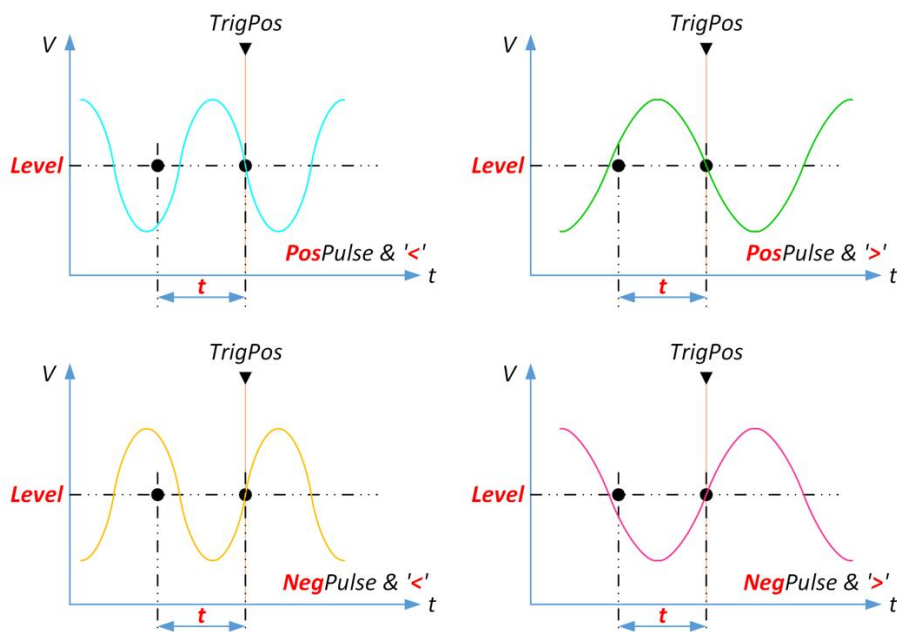
HSD_TRIG_PULSE_POS_LESS

正脉宽小于，下图左上部分

HSD_TRIG_PULSE_NEG_MORE

负脉宽大于，下图右下部分

HSD_TRIG_PULSE_NEG_LESS 负脉宽小于，下图左下部分默认为 HSD_TRIG_PULSE_POS_MORE。



trigSrc

触发源选择。

HSD_TRIG_SRC_CH0	CH0 通道
HSD_TRIG_SRC_CH1	CH1 通道
HSD_TRIG_SRC_EXT0	外部触发 0
HSD_TRIG_SRC_EXT1	外部触发 1

默认为 HSD_TRIG_SRC_CH0。

trigLevel

触发电平，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发电平的取值范围为 -0.5Range 到 +0.5Range，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发电平的取值范围为 -2.5~+2.5。

当触发源为 EXT0 或 EXT1 时，触发电平参数无效。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

选择脉宽触发并设置参数，包括触发时间、脉宽类型、触发源、和触发电平。本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

注意：

1) HSD 支持的触发类型有边沿触发、N 边沿触发、脉宽触发、斜率触发，分别可以通过函数 SetEdgeTrig、SetNEdgeTrig、SetPulseTrig、SetSlopeTrig 来设置，实际使用时，只能使用其中一种触发类型，不能同时使用多种触发类型。

2) 当 StartCapture 函数的 trigMode 参数为 HSD_TRIG_MODE_AUTO 时，该函数所设置的参数失效。

Example

例 1：选择脉宽触发，设置正脉宽大于 2.5us 触发，触发源为 CH0，触发电平 0.0V。

```
hds.SetPulseTrig(2.5E-6, HSD_TRIG_PULSE_POS_MORE, HSD_TRIG_SRC_CH0, 0.0);
```

5.7 SetSlopeTrig()

```
int SetSlopeTrig(  
double trigLevelA = 0.0,  
double trigLevelB = 0.0,  
double trigTime = 16E-9,  
HSD_TRIG_SLOPE trigSlope = HSD_TRIG_SLOPE_POS_MORE,  
HSD_TRIG_SRC trigSrc = HSD_TRIG_SRC_CH0);
```

Parameters

trigLevelA

触发电平 A，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发电平的取值范围为 -0.5Range 到 $+0.5\text{Range}$ ，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发电平的取值范围为 $-2.5\sim+2.5$ 。

当触发源为 EXT0 或 EXT1 时，触发电平参数无效。

trigLevelB

触发电平 B，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发电平的取值范围为 -0.5Range 到 $+0.5\text{Range}$ ，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发电平的取值范围为 $-2.5\sim+2.5$ 。

当触发源为 EXT0 或 EXT1 时，触发电平参数无效。

trigTime

触发时间，取值范围 $16\text{E}-9$ 到 10，单位 s，默认值 $16\text{E}-9$ 。

trigSlope

斜率触发类型选择。

HSD_TRIG_SLOPE_POS_MORE

正斜率大于，左图绿色波形

HSD_TRIG_SLOPE_POS_LESS

正斜率小于，左图蓝色波形

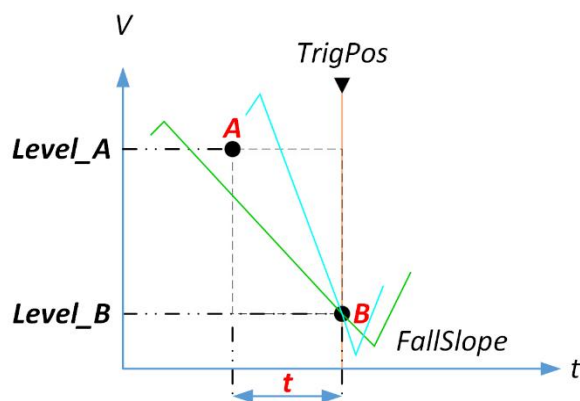
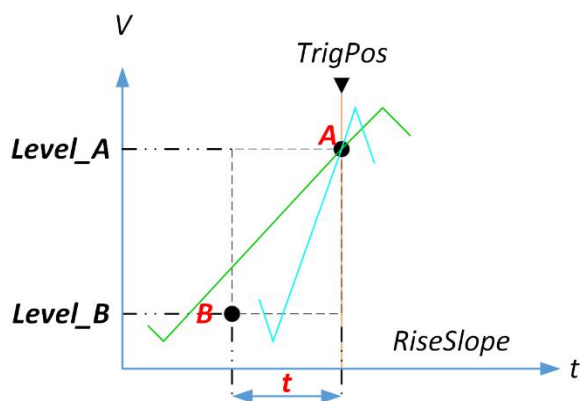
HSD_TRIG_SLOPE_NEG_MORE

负斜率大于，右图绿色部分

HSD_TRIG_SLOPE_NEG_LESS

负斜率小于，右图蓝色部分

默认为 HSD_TRIG_SLOPE_POS_MORE。



trigSrc

触发源选择。

HSD_TRIG_SRC_CH0 CH0 通道

HSD_TRIG_SRC_CH1 CH1 通道

默认为 HSD_TRIG_SRC_CH0。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

选择斜率触发并设置参数，包括触发电平、触发时间、斜率类型和触发源。本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

注意：

- 1) HSD 支持的触发类型有边沿触发、N 边沿触发、脉宽触发、斜率触发，分别可以通过函数 SetEdgeTrig、SetNEdgeTrig、SetPulseTrig、SetSlopeTrig 来设置，实际使用时，只能使用其中一种触发类型，不能同时使用多种触发类型。
- 2) 当 StartCapture 函数的 trigMode 参数为 HSD_TRIG_MODE_AUTO 时，该函数所设置的参数失效。
- 3) 选择斜率触发时，需要调用 SetTrigPropety 设置触发迟滞，且触发迟滞不能为 0。

Example

例 1: 选择斜率触发，设置触发电平 A 为+1V，触发电平 B 为-1V，正斜率大于 2.5us 触发，触发源为 CH0。

```
hsd.SetPulseTrig(1.0, -1.0, 2.5E-6, HSD_TRIG_SLOPE_POS_MORE, HSD_TRIG_SRC_CH0);
```

5.8 SetTrigPropety()

```
int SetTrigPropety(  
double trigOfst = 0.5,  
double trigDelay = 16E-9,  
double hystVal = 0.00);
```

Parameters

trigOfst

触发偏移位置，取值范围为 0.01 到 0.99，默认值为 0.5。

trigDelay

触发延迟，取值范围为 16E-9 到 10，单位 s，默认值为 16E-9。(该参数暂未开放，请使用默认值)。

hystVal

触发迟滞，与触发源有关，单位 V。

当触发源为 CH0 或 CH1 时，且 CH0 或 CH1 的通道电压范围为 Range，则触发迟滞的取值范围为 0 到 Range，比如通道电压范围为 HSD_VOLT_RANGE_5V，则触发迟滞的取值范围为 0~5.0V。

当触发源为 EXT0 或 EXT1 时，触发迟滞参数无效。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

设置触发属性，包括触发偏移点，触发延迟和触发迟滞，一般情况下，本函数可以不调用。本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

Example

例 1：设置触发偏移点为 10%位置处。

```
hsd.SetTrigPropety(0.1);
```

5.9 SetInterLeave()

```
int SetInterLeave(HSD_INTER_LEAVE interLeave);
```

Parameters

interLeave

内插倍数。

HSD_INTER_LEAVE_1X	1 倍内插
HSD_INTER_LEAVE_2X	2 倍内插
HSD_INTER_LEAVE_5X	5 倍内插
HSD_INTER_LEAVE_10X	10 倍内插
HSD_INTER_LEAVE_25X	25 倍内插
HSD_INTER_LEAVE_50X	50 倍内插

Return Values

调用成功返回 0，出错返回非零值。

Remarks

设置采样内插倍数。本函数所设置的参数为板卡级参数，HSD 的双通道共享该参数。

注意：

1) 开启内插功能后(内插倍数不为 1)，调用 BlockRead 函数将读取到内插数据。

2) 调用 BlockRead 读取采样数据时,读取的最大数据量为采样深度乘以内插倍数,比如设置采样深度为 2000(采样深度设置参考 StartCapture 函数),内插倍数为 10 倍时,最大可以读取 20000 个点的数据。

Example

例 1: 设置 10 倍内插。

```
hsd.SetInterLeave(HSD_INTER_LEAVE_10X);
```

5.10 StartCapture()

```
int StartCapture(  
HSD_TRIG_MODE trigMode = HSD_TRIG_MODE_AUTO,  
DWORD sampDepth = 2000,  
HSD_SAMPLE_RATE sampRate = HSD_SAMPLE_RATE_1GHZ_OR_1NS,  
HSD_RESOLUTION resolution = HSD_RESO_8BIT);
```

Parameters

trigMode

触发模式设置。

HSD_TRIG_MODE_AUTO AUTO 模式

HSD_TRIG_MODE_NORMAL NORMAL 模式

默认为 HSD_TRIG_MODE_AUTO。该模式下, SetEdgeTrig、SetNEdgeTrig、SetPulseTrig、SetSlopeTrig 函数设置的参数均无效。

sampDepth

采样深度。默认为 2000。

当使用 8BIT 采样时,取值范围 2000~500000000;

当使用 12BIT 采样时,取值范围 2000~2500000000;

sampRate

采样频率/采样间隔，默认值为 HSD_SAMPLE_RATE_1GHZ_OR_1NS。

HSD_SAMPLE_RATE_1GHZ_OR_1NS

HSD_SAMPLE_RATE_500MHZ_OR_2NS

HSD_SAMPLE_RATE_250MHZ_OR_4NS

HSD_SAMPLE_RATE_125MHZ_OR_8NS

HSD_SAMPLE_RATE_50MHZ_OR_20NS

HSD_SAMPLE_RATE_25MHZ_OR_40NS

HSD_SAMPLE_RATE_10MHZ_OR_100NS

HSD_SAMPLE_RATE_5MHZ_OR_200NS

HSD_SAMPLE_RATE_2MHZ_OR_500NS

HSD_SAMPLE_RATE_1MHZ_OR_1US

HSD_SAMPLE_RATE_500KHZ_OR_2US

HSD_SAMPLE_RATE_200KHZ_OR_5US

HSD_SAMPLE_RATE_100KHZ_OR_10US

HSD_SAMPLE_RATE_50KHZ_OR_20US

HSD_SAMPLE_RATE_20KHZ_OR_50US

HSD_SAMPLE_RATE_10KHZ_OR_100US

HSD_SAMPLE_RATE_5KHZ_OR_200US

HSD_SAMPLE_RATE_2KHZ_OR_500US

HSD_SAMPLE_RATE_1KHZ_OR_1MS

1GHz 的采样频率只能用于 8BIT 模式。

resolution

采样分辨率。

HSD_RESO_8BIT 8BIT

HSD_RESO_12BIT 12BIT

默认为 HSD_RESO_8BIT。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

启动 HSD 采样，设置触发模式、采样深度、采样速度和采样分辨率。本函数设置的参数为板卡级参数，HSD 的双通道共享该参数。

Example

例 1：设置 AUTO 触发模式，以 8 位分辨率和 1GHz 采样 2000 个点。

```
hsd.StartCapture(HSD_TRIG_MODE_AUTO, 2000, HSD_SAMPLE_RATE_1GHZ_OR_1NS);
```

5.11 WaitFinish()

```
int WaitFinish(DWORD maxTime);
```

Parameters

maxTime

超时时间，单位毫秒。当所设置时间为 0 时，驱动自动使用 100 毫秒作为超时时间。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

等待 HSD 采样结束。

Example

例 1：等待 HSD 采样完成，超时时间为 200 毫秒。

```
hsd.WaitFinish(200);
```

5.12 BlockRead()

```
int BlockRead(  
BYTE siteNo,  
HSD_CHANNEL channel,  
float *pBuff,  
DWORD startPos,  
DWORD totalSize);
```

Parameters

siteNo

选择工位号。0 代表 Site1，1 代表 Site2

channel

HSD 通道号。

HSD_CHANNEL_0 通道 0

HSD_CHANNEL_1 通道 1

注意不能使用 HSD_CHANNEL_ALL。

pBuff

float 型缓冲区，用来存储采样到的数据。

startPos

偏移位置，指定从第几个采样点开始读取数据，偏移位置计算从 0 开始。(该参数功能暂未开放，请使用 0)。

totalSize

缓冲区大小，指定需要读取的采样点总数。

Return Values

调用成功返回 0，出错返回非零值。

Remarks

读取 HSD 采样数据。

Example

例 1：获取 HSD 板卡通道 0 的采样数据，从第 0 个点开始，总共获取 2000 个点。

```
float fResult[2000];  
hsd.BlockRead (0, HSD_CHANNEL_0, fResult, 0, 2000);
```

5.13 GetMeasure()

```
int GetMeasure(  
BYTE siteNo,  
HSD_CHANNEL channel,  
HSD_MEASURE_ITEM measureItem);
```

Parameters

siteNo

选择测量数据工位。0 代表 Site1，1 代表 Site2

channel

HSD 通道号。

HSD_CHANNEL_0 通道 0

HSD_CHANNEL_1 通道 1

注意不能使用 HSD_CHANNEL_ALL。

measureItem

测试项。

HSD_MEAS_MAXIMUM 最大值

HSD_MEAS_MINIMUM 最小值

HSD_MEAS_VPP 峰峰值

HSD_MEAS_AVERAGE	平均值
HSD_MEAS_RMS	有效值
HSD_MEAS_TOP	基顶
HSD_MEAS_BASE	基底
HSD_MEAS_AMP	幅度
HSD_MEAS_OVER_SHOOT	过冲
HSD_MEAS_PRE_SHOOT	预冲
HSD_MEAS_RISE_TIME	上升沿时间
HSD_MEAS_FALL_TIME	下降沿时间
HSD_MEAS_PERIOD	周期
HSD_MEAS_FREQUENCY	频率
HSD_MEAS_PULSE_POS	高电平时间
HSD_MEAS_PULSE_NEG	低电平时间
HSD_MEAS_DUTY_POS	高占空比
HSD_MEAS_DUTY_NEG	低占空比
HSD_MEAS_FIRST_RISE	第一个上升沿时刻
HSD_MEAS_FIRST_FALL	第一个下降沿时刻
HSD_MEAS_RISE_CNT	上升沿个数
HSD_MEAS_FALL_CNT	下降沿个数

Return Values

返回测量值。

Remarks

读取 HSD 的测量值，注意，需要在使能自动测量功能后才能使用。

Example

例 1：获取 HSD 板卡通道 0 的最大值和最小值。

```
double dbMax, dbMin;
```

```
dbMax = hsd.GetMeasure (0, HSD_CHANNEL_0, HSD_MEAS_MAXIMUM);  
dbMin = hsd.GetMeasure (0, HSD_CHANNEL_0, HSD_MEAS_MINIMUM);
```

5.14 编程范例

信号源:

CH0: 方波, 1MHz, 5Vpp, 偏移 0Vdc, 占空比 50%, 上升沿和下降沿均为 20ns, 起始相位 0°。

CH1: 方波, 1MHz, 5Vpp, 偏移 0Vdc, 占空比 50%, 上升沿和下降沿均为 20ns, 起始相位 10°。

参数值:

CH0 和 CH1 电压范围 10V, 上升沿触发, 触发源为 CH0, 触发电平为 0.0V, 10 倍内插, NORMAL 模式触发, 采样速度 1GHz, 采样深度 2000。

功能:

测量 CH0 和 CH1 的上升沿时间(-2.0V 到 2.0V)、CH0 和 CH1 的延迟时间。

例程代码如下:

```
HSD hsd(0);  
  
float fResultCh0[20000], fResultCh1[20000];  
  
DUT_API void HardWareCfg()  
{  
    StsSetModuleToSite(MD_HSD, SITE_1, 0, -1);  
    StsSetModuleToSite(MD_HSD, SITE_2, 1, -1);  
}  
  
DUT_API void InitBeforeTestFlow()  
{  
    static int flag = 0;  
  
    if (0 == flag)
```

```
{
    hsd.Init();
    hsd.SetChannel(HSD_CHANNEL_ALL, HSD_VOLT_RANGE_10V);
    hsd.SetEdgeTrig(HSD_TRIG_EDGE_RISING, HSD_TRIG_SRC_CH0, 0.0);
    hsd.SetInterLeave(HSD_INTER_LEAVE_10X);
    flag = 1;
}

DUT_API int EdgeTest(short funcindex, LPCTSTR funclabel)
{
    //{AFX_STS_PARAM_PROTOTYPES
    CParam *RisingEdge0 = StsGetParam(funcindex, "RisingEdge0");
    CParam *RisingEdge1 = StsGetParam(funcindex, "RisingEdge1");
    CParam *DelayTime = StsGetParam(funcindex, "DelayTime");
    //}AFX_STS_PARAM_PROTOTYPES

    int siteNum;

    siteNum = StsGetSiteCount();
    hsd.StartCapture(HSD_TRIG_MODE_NORMAL, 2000, HSD_SAMPLE_RATE_1GHZ_OR_1NS,
HSD_RESO_8BIT);
    hsd.WaitFinish(100);
    for (int siteNo = 0; siteNo < siteNum; siteNo++)
    {
        hsd.BlockRead(siteNo, HSD_CHANNEL_0, fResultCh0, 0, 20000);
        hsd.BlockRead(siteNo, HSD_CHANNEL_1, fResultCh1, 0, 20000);

        RisingEdge0->SetTestResult(siteNo, 0, GetRisingTime(fResultCh0, 20000));
        RisingEdge1->SetTestResult(siteNo, 0, GetRisingTime(fResultCh1, 20000));
        DelayTime->SetTestResult(siteNo, 0, GetDelayTime(fResultCh0, fResultCh1, 20000));
    }

    return 0;
}

static double GetRisingTime(float *pResult, DWORD pointNum)
{
    int nPos0, nPos1, index;

    nPos0 = nPos1 = -1;
    for (index = pointNum / 2 - 2000; index < (int)pointNum; index++)
    {
        if (pResult[index] > -2.0)
        {
            nPos0 = index;
            break;
        }
    }
    for (; index < (int)pointNum; index++)
    {
        if (pResult[index] > 2.0)
        {
            nPos1 = index;
            break;
        }
    }

    if ((nPos0 < 0) || (nPos1 < 0))
```

```
    {
        return 0.0;
    }
    else
    {
        return (nPos1 - nPos0) * 0.1;
    }
}

static double GetDelayTime(float *pResultCh0, float *pResultCh1, DWORD pointNum)
{
    int nPos0, nPos1, index;

    nPos0 = nPos1 = -1;
    for (index = pointNum / 2 - 2000; index < (int)pointNum; index++)
    {
        if (pResultCh0[index] > 0.0)
        {
            nPos0 = index;
            break;
        }
    }
    for (index = pointNum / 2 - 2000; index < (int)pointNum; index++)
    {
        if (pResultCh1[index] > 0.0)
        {
            nPos1 = index;
            break;
        }
    }

    if ((nPos0 < 0) || (nPos1 < 0))
    {
        return 0.0;
    }
    else
    {
        return (nPos1 - nPos0) * 0.1;
    }
}
```


6 VI 源表

6.1 AWG

AWG 相关函数，主要用于源的 AWG 设置以及源的同步设置及启动。

6.1.1 STSAWGCreateSineData()

```
int USERRES_API STSAWGCreateSineData(  
double* awgData,  
UINT dataNumber,  
double waveCycles = 1,  
double Vpp = 0,  
double DCOffset = 0,  
double phase = 0);
```

Parameters

awgData

double 型指针，存储 AWG 波形数据的首地址。

dataNumber

AWG 波形数据的长度。

waveCycles

正弦波的周期数。该参数默认为 1。

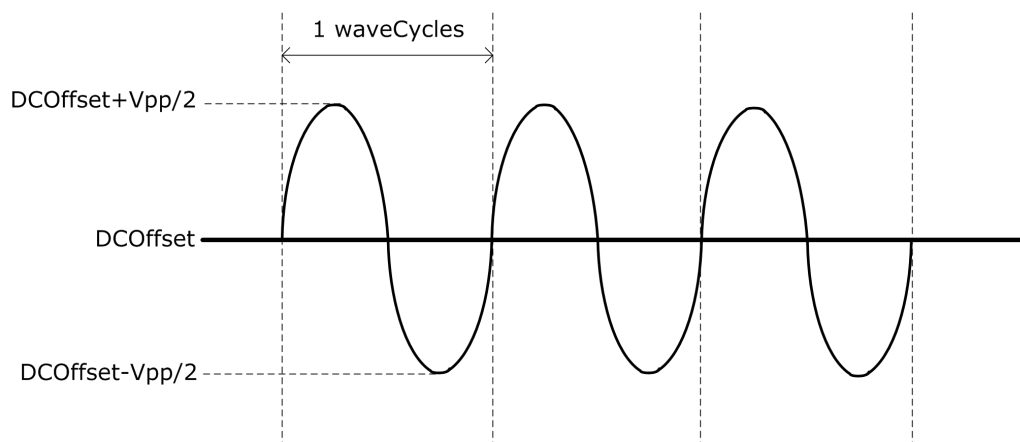
Vpp

正弦波的峰峰值。该参数默认为 0。

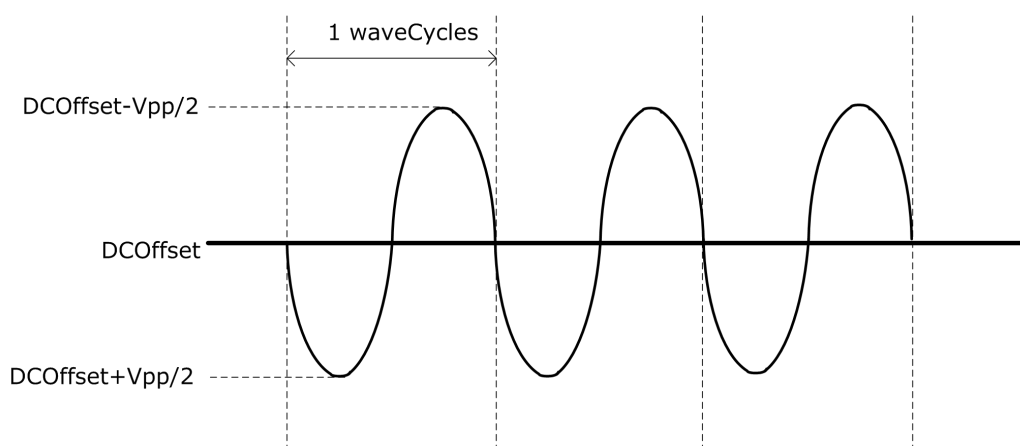
该参数可正可负。下图为三个周期正弦波分别在 V_{pp} 为正值和负值时的波形图。

二者相位相差 180 度。

当 V_{pp} 为正值时，各参数关系及波形如下：



当 V_{pp} 为负值时，各参数关系及波形如下：



DCOffset

正弦波的直流偏置。该参数默认为 0。

phase

正弦波的相位。单位：度。该参数默认为 0。

Remarks

建立一个正弦波数据组。设置该正弦波的数据长度、周期数、峰峰值、直流偏置以及相位等信息。

Example

例 1：设置一个正弦波数据组，首地址从 0 开始，数据长度为 100，周期数为 1，峰峰值为-5V，直流偏置为 0，相位为 180 度。

```
double awg_pattern[100]={0.0};  
STSAWGCreateSineData(&awg_pattern[0],100,1,-5,0,180);
```

6.1.2 STSAWGCreateTriangleData()

```
int USERRES_API STSAWGCreateTriangleData(  
double* awgData,  
UINT dataNumber,  
double waveCycles = 1,  
double Vpp = 0,  
double DCOffset = 0,  
double phase = 0);
```

Parameters

awgData

double 型指针，存储 AWG 波形数据的首地址。

dataNumber

AWG 波形数据的长度。

waveCycles

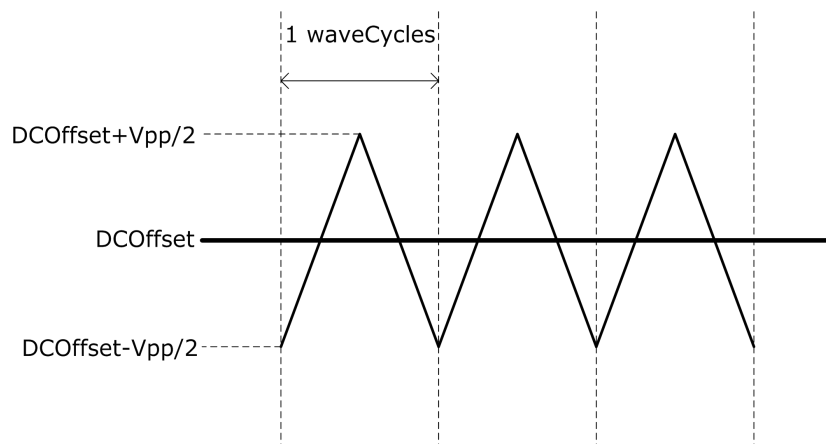
三角波的周期数。该参数默认为 1。

Vpp

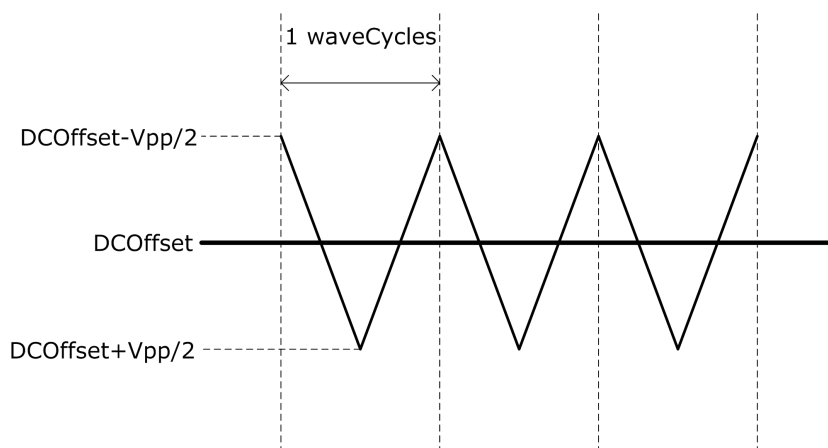
三角波的峰峰值。该参数默认为 0。

该参数可正可负。下图为三个周期三角波分别在 V_{pp} 为正值和负值时的波形图。二者相位相差 180 度。

当 V_{pp} 为正值时，各参数关系及波形如下：



当 V_{pp} 为负值时，各参数关系及波形如下：



DCOffset

三角波的直流偏置。该参数默认为 0。

phase

三角波的相位。单位：度。该参数默认为 0。

Remarks

建立一个三角波数据组。设置该三角波的数据长度、周期数、峰峰值、直流偏置以及相位等信息。

Example

例 1：设置一个三角波数据组，首地址从 0 开始，数据长度为 100，周期数为 4，峰峰值为-5V，直流偏置为 0，相位为 0 度。

```
double awg_pattern[1000]={0.0};  
STSAWGCreateTriangleData(&awg_pattern[0],100,4,-5,0,0);
```

6.1.3 STSAWGCreateSquareData()

```
int USERRES_API STSAWGCreateSquareData(  
double* awgData,  
UINT dataNumber,  
double waveCycles = 1,  
double Vpp = 0,  
double DCOffset = 0,  
double dutyCycle = 50);
```

Parameters

awgData

double 型指针，存储 AWG 波形数据的首地址。

dataNumber

AWG 波形数据的长度。

waveCycles

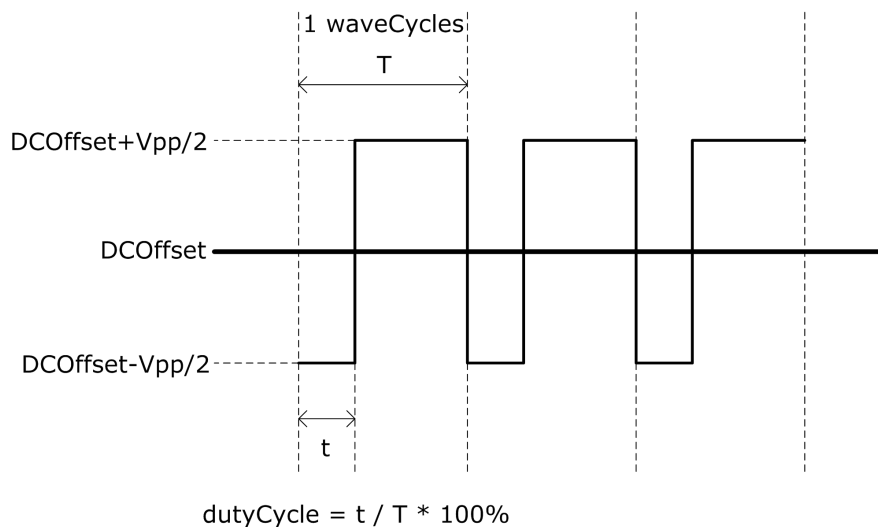
方波的周期数。该参数默认为 1。

Vpp

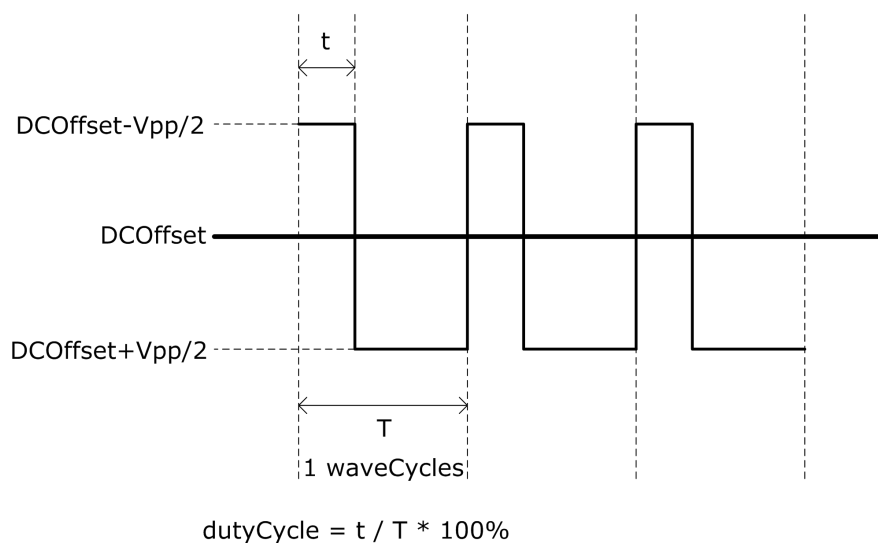
方波的峰峰值。该参数默认为 0。

该参数可正可负。下图为三个周期方波分别在 V_{pp} 为正值和负值时的波形图。二者相位相差 180 度。

当 V_{pp} 为正值时，各参数关系及波形如下：



当 V_{pp} 为负值时，各参数关系及波形如下：



DCOffset

方波的直流偏置。该参数默认为 0。

dutyCycle

方波的占空比。单位：%。该参数默认为 50。

Remarks

建立一个方波数据组。设置该方波的数据长度、周期数、峰峰值、直流偏置以及占空比等信息。

Example

例 1：设置一个方波数据组，首地址从 0 开始，数据长度为 100，周期数为 4，峰峰值为-5V，直流偏置为 0，占空比为 50。

```
double awg_pattern[1000]={0.0};  
STSAWGCreateSquareData(&awg_pattern[0],100,4,-5,0,50);
```

6.1.4 STSAWGCreateRampData()

```
int USERRES_API STSAWGCreateRampData(  
double* awgData,  
UINT dataNumber,  
double waveCycles = 1,  
double startValue = 0,  
double stopValue = 0);
```

Parameters

awgData

double 型指针，存储 AWG 波形数据的首地址。

dataNumber

AWG 波形数据的长度。

waveCycles

斜波的周期数。该参数默认为 1。

startValue

斜波的起始值。该参数默认为 0。

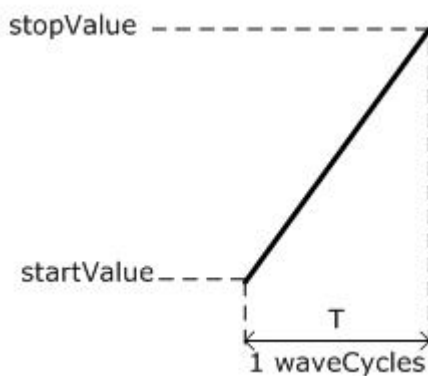
stopValue

斜波的结束值。该参数默认为 0。

Remarks

建立一个斜波数据组。设置该斜波的数据长度、周期数、起始值以及结束值等信息。

下图表示了该函数各个参数的关系，该图 $\text{startValue} < \text{stopValue}$ ，因此为一个上升的斜波。



Example

例 1：设置一个斜波数据组，首地址从 0 开始，数据长度为 100，周期数为 1，起始值为 1V，结束值为 5V。

```
double awg_pattern[1000]={0.0};  
STSAWGCreateRampData(&awg_pattern[0],100,1,1,5);
```

6.1.5 STSEnableAWG()

```
int USERRES_API STSEnableAWG(  
FloatingVI *vi1,  
FloatingVI *vi2 = NULL,  
FloatingVI *vi3 = NULL,  
FloatingVI *vi4 = NULL,
```



```
FloatingVI *vi5 = NULL,  
FloatingVI *vi6 = NULL,  
FloatingVI *vi7 = NULL,  
FloatingVI *vi8 = NULL,  
FloatingVI *vi9 = NULL,  
FloatingVI *vi10 = NULL,  
FloatingVI *vi11 = NULL,  
FloatingVI *vi12 = NULL,  
FloatingVI *vi13 = NULL,  
FloatingVI *vi14 = NULL,  
FloatingVI *vi15 = NULL,  
FloatingVI *vi16 = NULL,  
FloatingVI *vi17 = NULL,  
FloatingVI *vi18 = NULL,  
FloatingVI *vi19 = NULL,  
FloatingVI *vi20 = NULL,  
FloatingVI *vi21 = NULL,  
FloatingVI *vi22 = NULL,  
FloatingVI *vi23 = NULL,  
FloatingVI *vi24 = NULL,  
FloatingVI *vi25 = NULL,  
FloatingVI *vi26 = NULL,  
FloatingVI *vi27 = NULL,  
FloatingVI *vi28 = NULL,  
FloatingVI *vi29 = NULL,  
FloatingVI *vi30 = NULL,  
FloatingVI *vi31 = NULL,  
FloatingVI *vi32 = NULL);
```

Parameters

**vi1, *vi2, *vi3,*

设置需要同步 AWG 波形输出的 VI 源名称，此处 VI 源只能填 FPVle、FOVle、ACM、HVle、HPVle、FXVle 对应源的名称。一共可以填 32 路源。

Remarks

设置需要同步 AWG 波形输出的有效资源列表。

注意：

STSEnableAWG 函数之前，需先启动 AwgLoader 和 AwgSelect 函数。

Example

例 1：定义四路 FOVle，一路 FPVle，设置 VCC 的 AWG 波形输出有效。如下：

```
FOVle VCC("S13_0");
FOVle CTL("S13_1");
FOVle VOUT("S13_2");
FOVle ADJ("S13_3");
FPVle VIN_VOUT("S15_0");

VCC.AwgLoader("AWG_Vst",FV,FOVle_40V,FOVle_100MA,awg_pattern1,sam1);
VCC.Set(FV, 16.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
VCC.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);

STSEnableAWG(&VCC);
```

6.1.6 STSEnableMeas()

```
int USERRES_API STSEnableMeas (
FloatingVI *vi1,
FloatingVI *vi2 = NULL,
FloatingVI *vi3 = NULL,
FloatingVI *vi4 = NULL,
FloatingVI *vi5 = NULL,
FloatingVI *vi6 = NULL,
FloatingVI *vi7 = NULL,
FloatingVI *vi8 = NULL,
FloatingVI *vi9 = NULL,
FloatingVI *vi10 = NULL,
FloatingVI *vi11 = NULL,
FloatingVI *vi12 = NULL,
FloatingVI *vi13 = NULL,
FloatingVI *vi14 = NULL,
FloatingVI *vi15 = NULL,
FloatingVI *vi16 = NULL,
FloatingVI *vi17 = NULL,
FloatingVI *vi18 = NULL,
FloatingVI *vi19 = NULL,
```

```
FloatingVI *vi20 = NULL,  
FloatingVI *vi21 = NULL,  
FloatingVI *vi22 = NULL,  
FloatingVI *vi23 = NULL,  
FloatingVI *vi24 = NULL,  
FloatingVI *vi25 = NULL,  
FloatingVI *vi26 = NULL,  
FloatingVI *vi27 = NULL,  
FloatingVI *vi28 = NULL,  
FloatingVI *vi29 = NULL,  
FloatingVI *vi30 = NULL,  
FloatingVI *vi31 = NULL,  
FloatingVI *vi32 = NULL);
```

Parameters

*vi1, *vi2, *vi3,

填写需要同步测量的 VI 源和表的名称，此处 VI 源只能填 FPVle、FOVle、ACM、HVle、HPVle、FXVle 对应源的名称以及 QVMe 对应表的名称。一共可以填 32 路。

Remarks

使能同步测量。

注意：

STSEnableMeas 函数之前，需先启动 MeasureVI 函数。

Example

例 1：定义四路 FOVle，一路 FPVle，使能 VCC，VOUT 和 VIN_VOUT 同步测量：

```
FOVle VCC("S13_0");  
FOVle CTL("S13_1");  
FOVle VOUT("S13_2");  
FOVle ADJ("S13_3");  
FPVle VIN_VOUT("S15_0");  
  
VCC.MeasureVI(50,20,FOVle_MI_X1,MEAS_AWG);  
VOUT.MeasureVI(50,20,FOVle_MI_X1,MEAS_AWG);  
VIN_VOUT.MeasureVI(50,20,FPVle_MV_X1,FPVle_MI_X1,MEAS_AWG);  
  
STSEnableMeas(&VCC,&VOUT,&VIN_VOUT);
```

6.1.7 STSAWGRUN()

```
int USERRES_API STSAWGRUN(UINT delayTime_ms = 0);
```

Parameters

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。

设置延时时间。AWG 输出 delayTime_ms 之后，系统允许 STSAWGRUN 之后的操作开始运行。delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间或同步测量时间，以二者较长时间为准，只有在 AWG 扫描和测量运行完毕之后，系统才允许 STSAWGRUN 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

AWG 同步启动。运行完这个函数之后，AWG 扫描和测量才开始运行。通常需要配合 STSEnableAWG 和 STSEnableMeas 一起使用。

此函数可以实现 AWG 与 DCM 向量同步运行，具体参照 4.1.21RunVectorEn()。

注意：

STSAWGRUN 语句之后不需要设置延时，该语句的延时由系统内部设定，系统会根据 AWG 的持续时间和测量的持续时间来确定具体的延时。

Example

例 1.

fovie0: 建立一个 AWG 波形数据组，使能 fovie0 的 AWG 波形和同步测量。

fovie1: 待 fovie0 的 AWG 波形输出和测量完全执行完后, fovie1 设置为 FV 为 5V。

```
int sam = 100; // data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

fovie1.Set(FV, 0, FOVle_20V, FOVle_1MA, FOVle_RELAY_ON);
delay_ms(1);

STSAWGCreateTriangleData(&awg_pattern[0],sam,1,16.0,0,270);
//Set a triangle data array, the start address starts from 0, data size is 100.

fovie0.AwgLoader("Triangle_AWG",FV,FOVle_20V,FOVle_100MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Triangle_AWG'. Set to FV mode.

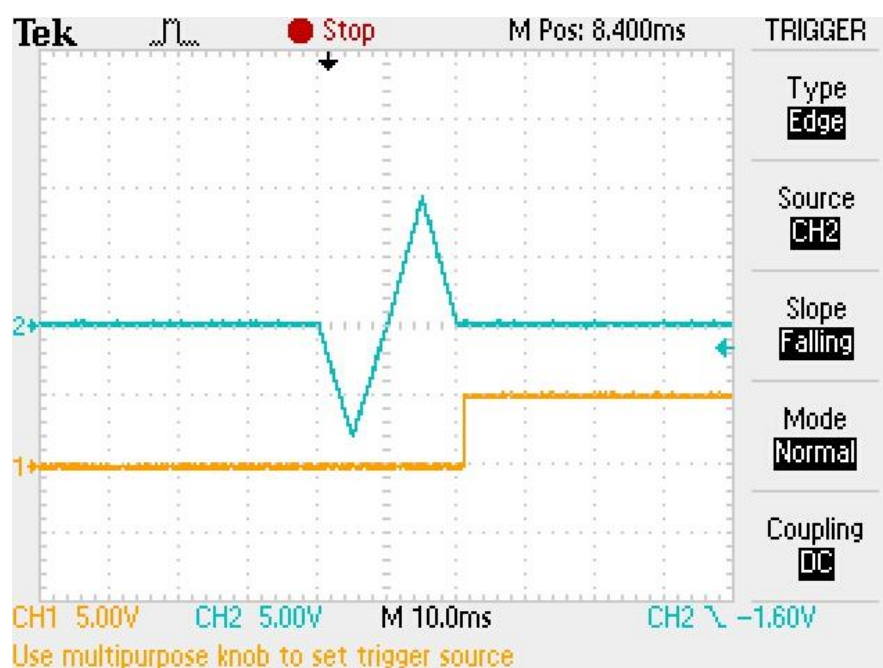
fovie0.Set(FV, 0, FOVle_20V, FOVle_100MA, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Triangle_AWG' of its
'AwgLoader()'

fovie0.AwgSelect("Triangle_AWG ", 0, sam-1, sam-1, interval);
//Select pattern of 'Triangle_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fovie0.MeasureVI(sam,interval, FOVle_MI_X1,MEAS_AWG);
fovie1.MeasureVI(sam,interval, FOVle_MI_X1,MEAS_AWG);
STSEnableAWG(&fovie0);//enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1);//enable measurement for fovie0 and fovie1
STSAWGRun();//Enable channel of fovie0 for AWG output and measurement synchronously, after run
over, then execute the following code

fovie1.Set(FV,5,FOVle_20V,FOVle_100MA,FOVle_RELAY_ON);//set fovie1 in FV=5V
delay_ms(1);
```

执行上述程序，用示波器观察 fovie0 和 fovie1 的波形如下，其中绿色为 fovie0 的波形，黄色为 fovie1 的波形。



例 2.

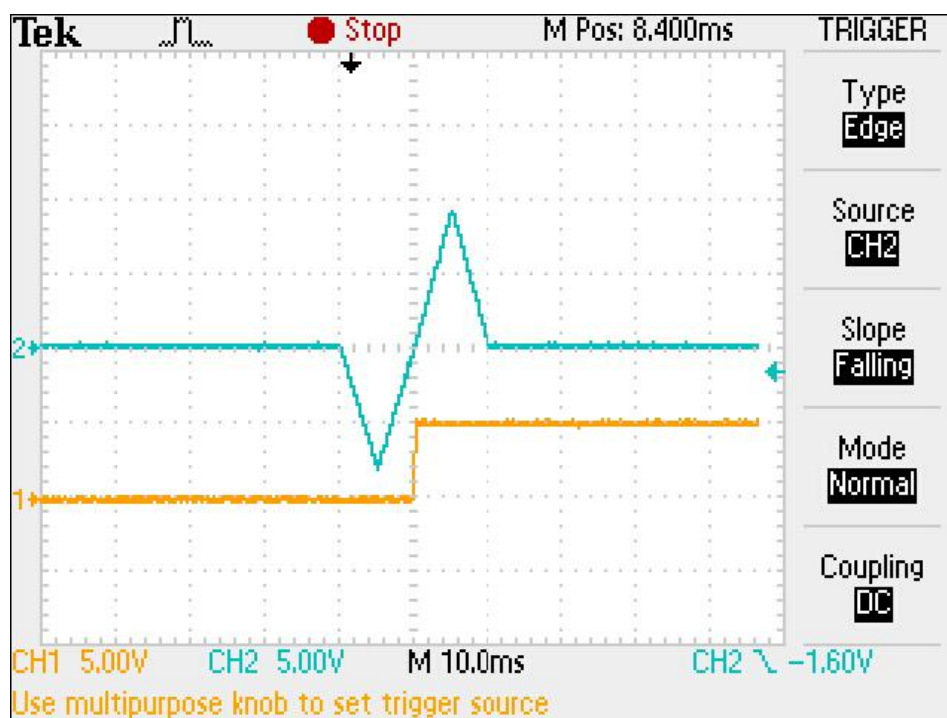
fovie0: 建立一个 AWG 波形数据组，使能 fovie0 的 AWG 波形和同步测量。

fovie1: 待 fovie0 的 AWG 波形和测量执行 10mS 后，fovie1 设置为 FV 为 5V。

```
STSAWGRUN(10); // Enable channel of fovie0 for AWG output and measurement synchronously, after run 10mS, then execute the following code
```

```
fovie1.Set(FV,5,FOVie_20V,FOVie_100MA,FOVie_RELAY_ON); //set fovie1 in FV=5V  
delay_ms(1);
```

执行上述程序，用示波器观察 fovie0 和 fovie1 的波形如下，其中绿色为 fovie0 的波形，黄色为 fovie1 的波形。



6.1.8 STSPTAWGAndDCMRUN()

```
int USERRES_API STSPTAWGAndDCMRUN(DCM& dcm, UINT delayTime_ms = 0);
```

Parameters

dcm

DCM 实例或者变量，指定与 AWG 同步运行的 DCM 实例，主要用于多站并发程序中，指定同步运行的 DCM 实例，具体参照第 4DCM 章节。

注意：

STSAWGRUN 也可以实现 AWG 与 DCM 同步，但是只能与第一个 DCM 变量或者实例实现同步。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。

设置延时时间。AWG 输出 delayTime_ms 之后，系统允许 STSAWGRUN 之后的操作开始运行。delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间或同步测量时间，以二者较长时间为准，只有在 AWG 扫描和测量运行完毕之后，系统才允许 STSAWGRUN 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

此函数的功能与 STSAWGRUN 函数相同，区别只是此函数可以指定 DCM 实例。

Example

例 1.

fovie0: 建立一个 AWG 波形数据组，使能 fovie0 的 AWG 波形和同步测量。

fovie1: 待 fovie0 的 AWG 波形和测量执行 10mS 后，fovie1 设置为 FV 为 5V。

dcm1: 与 fovie0 同步运行

```
DCM dcm;
STSPtAWGAndDCMRUN (dcm,10);// Enable channel of fovie0 for AWG output and measurement
synchronously, after run 10mS, then execute the following code

fovie1.Set(FV,5,FOVIE_20V,FOVIE_100MA,FOVIE_RELAY_ON);//set fovie1 in FV=5V
delay_ms(1);
```

6.1.9 STSAWGRUNTriggerStop()

```
int USERRES_API STSAWGRunTriggerStop(  
FloatingVI *viTrigger,  
FloatingVI *stopVi1,  
FloatingVI *stopVi2 = NULL,  
FloatingVI *stopVi3 = NULL,  
FloatingVI *stopVi4 = NULL,  
FloatingVI *stopVi5 = NULL,  
FloatingVI *stopVi6 = NULL,  
FloatingVI *stopVi7 = NULL,  
FloatingVI *stopVi8 = NULL,  
FloatingVI *stopVi9 = NULL,  
FloatingVI *stopVi10 = NULL,  
FloatingVI *stopVi11 = NULL,  
FloatingVI *stopVi12 = NULL,  
FloatingVI *stopVi13 = NULL,  
FloatingVI *stopVi14 = NULL,  
FloatingVI *stopVi15 = NULL,  
FloatingVI *stopVi16 = NULL);
```

Parameters

**viTrigger*

填写设置触发的 VI 源名称。该 VI 源与 SetMeasVTrig 或者 SetMeasITrig 对应的源一致。

**stopVi1, *stopVi2, ..., *stopVi16*

填写触发生效后同时停止运行的 VI 源名称，最多可填写 16 个 VI 源。

Remarks

触发生效后，AWG 波形停止运行。运行完这个函数之后，AWG 扫描和测量才开始运行，对应的 VI 源触发后，与之同步的所有 VI 源停止运行。通常需要配合 STSEnableAWG 和 STSEnableMeas 一起使用。

注意：

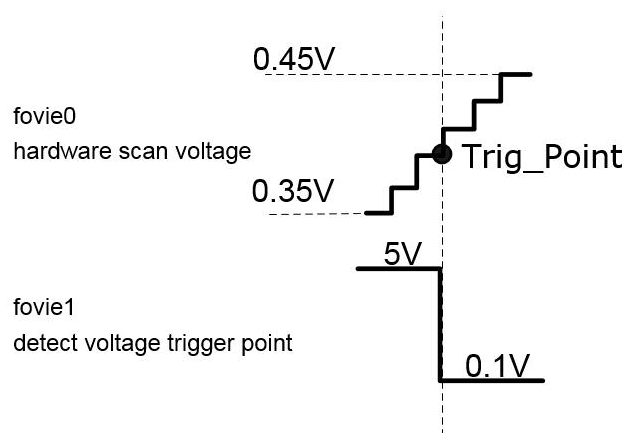
STSAWGRunTriggerStop 语句之后不需要设置延时，该语句的延时由系统内部设

定，系统会根据 AWG 的持续时间和测量的持续时间来确定具体的延时。

Example

例 1：建立一个波形数据组，fovie0 从 0.35V 上升 0.45V，步距 0.001V。测试 fovie1 端的电压小于 2V 时，fovie0 跳变点的电压值。

分别使用 STSAWGRun 函数和 STSAWGRunTriggerStop 函数进行测试，观察波形的区别。



使用 STSAWGRun 函数的测试程序和 SoftView 波形如下：

```
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

fovie1.Set(FI, 0, FOVle_10V, FOVle_1MA, FOVle_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
//Set anramp data array, the start address starts from 0, data size is 50.

fovie0.AwgLoader("Vcspre_AWG", FV, FOVle_1V, FOVle_100MA, awg_pattern, sam);
//import the awg_pattern array into fovie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

fovie0.Set(FV, 0.30, FOVle_1V, FOVle_100MA, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

fovie0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

fovie1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, Falling edge

fovie0.MeasureVI(sam, interval, FOVle_MI_X1, MEAS_AWG);
fovie1.MeasureVI(sam, interval, FOVle_MI_X1, MEAS_AWG);
```

```

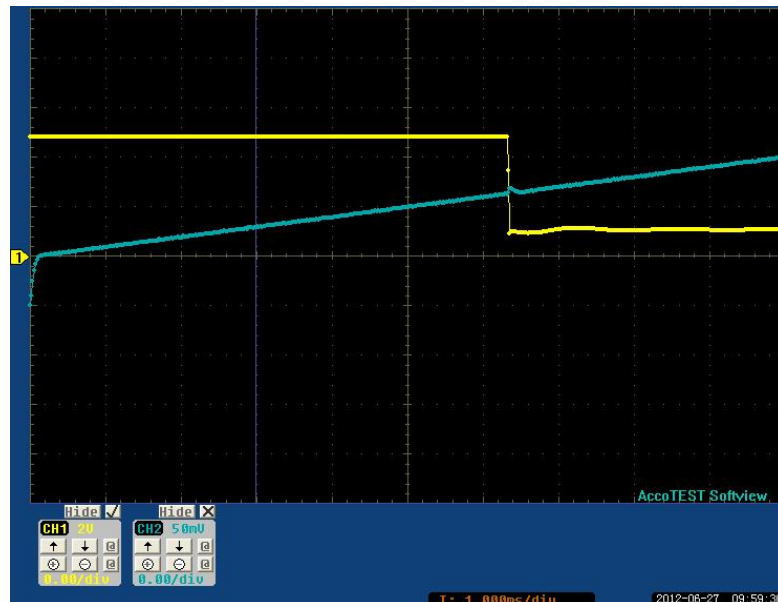
STSEnableAWG(&fovie0);//enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1);//enable measurement for fovie0 and fovie1
STSAWGRun();//Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] =fovie1.GetMeasResult(i,MVRET,TRIG_RESULT);
    //Get the position of trigger point for fovie1

    result[i] = fovie0.GetMeasResult(i,MVRET,Trig_Point[i]);
    //Read the voltage value of fovie0 on trigger position

    Vcspre->SetTestResult(i, 0, result[i]);
}

```



使用 STSAWGRunTriggerStop 函数的测试程序和 SoftView 波形如下：

```

double Trig = 2; //trigger voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

fovie1.Set(FI, 0, FOVie_10V, FOVie_1MA, FOVie_RELAY_ON);//OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0],sam,1,0.35,0.45);
//Set anramp data array, the start address starts from 0, data size is 50.

fovie0.AwgLoader("Vcspre_AWG",FV,FOVie_1V,FOVie_100MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

fovie0.Set(FV, 0.30, FOVie_1V, FOVie_100MA, FOVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its

```

```

'AwgLoader()'

fovie0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

fovie1.SetMeasVTrig(Trig,TRIG_FALLING);// trigger value is 2V, Falling edge

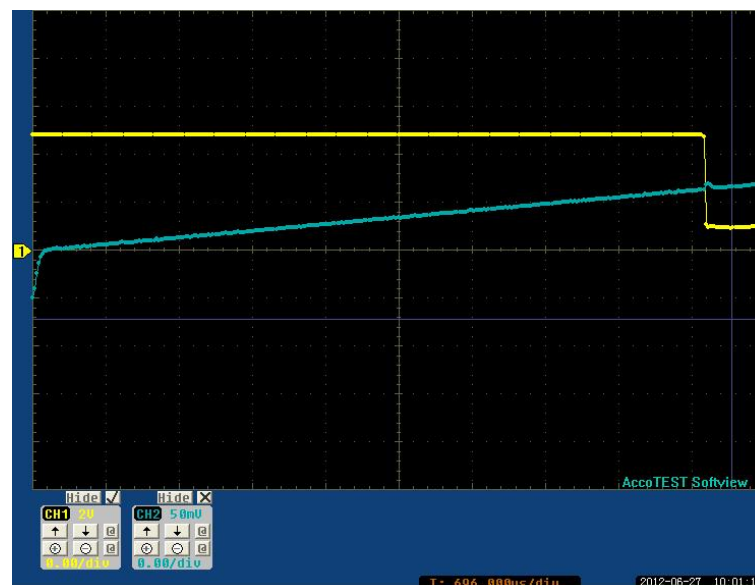
fovie0.MeasureVI(sam,interval, FOVle_MI_X1,MEAS_AWG);
fovie1.MeasureVI(sam,interval, FOVle_MI_X1,MEAS_AWG);
STSEnableAWG(&fovie0);//enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1);//enable measurement for fovie0 and fovie1
STSAWGRunTriggerStop(&fovie1, &fovie1, &fovie0);//When fovie1 start triggered, the channels fovie1
and fovie0 will stop running synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] =fovie1.GetMeasResult(i,MVRET,TRIG_RESULT);
    //Get the position of trigger point for fovie1

    result[i] = fovie0.GetMeasResult(i,MVRET,Trig_Point[i]);
    //Read the voltage value of fovie0 on trigger position

    Vcspre->SetTestResult(i, 0, result[i]);
}

```



6.2 ACM

6.2.1 ACM()

```
ACM(char*channelList, char*chName = NULL);
```

Parameters

channelList

ACM 绑定通道字符串。形如“S6_0, S6_1”，其中“S6”表示槽位号 6（S 即 Slot），“0、1”表示板内序号。

注意：

ACM 一块板卡最多 24 个通道，因此板内序号最大值为 23。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 ACM_xx。

Remarks

定义一路 ACM 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

每块 ACM 有 24 个物理通道，分为 CH0~5，CH6~11，CH12~17，CH18~23 四组。

Example

例 1：假设系统配置为 2 块 ACM（分别位于 Slot5 和 Slot6），四工位并行工作，定义可使用的 ACM。

```
ACM acm0("S5_0,S5_12,S6_0,S6_12");
ACM acm1("S5_1,S5_13,S6_1,S6_13");
ACM acm2("S5_2,S5_14,S6_2,S6_14");
ACM acm3("S5_3,S5_15,S6_3,S6_15");
ACM acm4("S5_4,S5_16,S6_4,S6_16");
ACM acm5("S5_5,S5_17,S6_5,S6_17");
```

```
ACM acm6("S5_6,S5_18,S6_6,S6_18");
ACM acm7("S5_7,S5_19,S6_7,S6_19");
ACM acm8("S5_8,S5_20,S6_8,S6_20");
ACM acm9("S5_9,S5_21,S6_9,S6_21");
ACM acm10("S5_10,S5_22,S6_10,S6_22");
ACM acm11("S5_11,S5_23,S6_11,S6_23");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_ACM, SITE_1, "S5_0-11"); // set Slot5 ch0-11 to SITE1
STSSetMultiSiteBind(MD_ACM, SITE_2, "S5_12-23"); // set Slot5 ch12-23 to SITE2
STSSetMultiSiteBind(MD_ACM, SITE_3, "S6_0-11"); // set Slot6 ch0-11 to SITE3
STSSetMultiSiteBind(MD_ACM, SITE_4, "S6_12-23"); // set Slot6 ch12-23 to SITE4
```

6.2.2Set()

```
int Set(
VIMode viMode,
double setValue,
ACM_VRNG vRange,
ACM_IRNG iRange,
ACM_OUT_RELAY relayStatus = ACM_RELAY_HOLD,
double risingTime = 0.1);
```

Parameters

viMode

ACM 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-2~18。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.5~0.5。

vRange

ACM 电压量程。

ACM_N2P18V

iRange

ACM 电流量程。

ACM_500MA

ACM_200MA

ACM_20MA

ACM_2MA

ACM_200UA

ACM_20UA

ACM_5UA

注意：

- 1) 选择 ACM_500MA 档位时，当电流绝对值小于 215mA 时，板卡工作在直流模式，当电流绝对值大于 215mA 时，板卡工作在脉冲模式，持续时间为 10mS。215mA 是一个近似值；
- 2) 当源工作在上述脉冲模式且脉冲持续时间超出 10ms 后，电流档位会自动切换到 20mA 电流档位，此时源的输出电流变为之前的 1/25，此时的测流值异常，系统会有 alarm 报警；
- 3) 当源工作在上述脉冲模式且脉冲持续时间超出 10ms 后，如果需要再次将源设置成 500mA 电流档位，则需要先将源设置成其他电流档位（除 500mA 档位以外的其他电流档位），再设置成 500mA 电流档位。

relayStatus

设置输出继电器如何动作，分为三种情况：

ACM_RELAY_ON：打开输出继电器；

ACM_RELAY_OFF：关闭输出继电器；

ACM_RELAY_HOLD：输出继电器保持上一个状态。

risingTime

double 型变量，设置积分变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，

则默认为 0.1mS;

注意：

- 1) 该参数设置小于等于 0.1mS 时, risingTime=0.1mS, 为 Normal 模式;
- 2) 该参数设置大于 0.1mS 且小于等于 0.2mS 时, risingTime=0.2mS, 为 Slow 模式;
- 3) 该参数设置大于 0.2mS 时, 自动启动 Capload 功能, risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿, 并且上升或者下降时间可以根据需要来设定, Capload 的最大值为 65mS。

Remarks

设置 ACM 的状态, 包括工作模式、恒压或恒流值、电压量程、电流量程、输出继电器状态、积分时间。

Example

例 1: 设置 acm0 电压输出 5V, risingTime 不填则默认为 0.1mS。

```
acm0.Set(FV,5,ACM_N2P18,ACM_200MA,ACM_RELAY_ON);
```

例 2: 设置 acm0 电压输出 5V, 电流量程选择 ACM_200MA, risingTime 设置为 1mS。

```
acm0.Set(FV,0,ACM_N2P18V,ACM_200MA,ACM_RELAY_ON);  
delay_ms(1);  
acm0.Set(FV,5,ACM_N2P18V,ACM_200MA,ACM_RELAY_ON, 1);
```

例 3: 设置 acm0 电压输出 5V, 电流量程选择 ACM_5UA, risingTime 设置为 1mS。

```
acm0.Set(FV,0,ACM_N2P18V,ACM_5UA,ACM_RELAY_ON);  
delay_ms(1);  
acm0.Set(FV,5,ACM_N2P18V,ACM_5UA,ACM_RELAY_ON, 1);
```

注意：

例 2 和例 3 中, 同样是设置 1mS 的积分时间, 例 2 实际的波形建立时间为 1mS, 例 3 选择了 5uA 档位, 则实际建立时间为 1.1mS。

6.2.3 SetSyn()

```
int SetSyn(  
VIMode viMode,  
const double* setValue,  
UINT siteSize,  
ACM_VRNG vRange,  
ACM_IRNG iRange,  
ACM_OUT_RELAY relayStatus = ACM_RELAY_HOLD,  
double risingTime = 0.1);
```

Parameters

viMode

ACM 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-2~18。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-0.5~0.5。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1) setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误。
- 2) 若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3) 若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

ACM 电压量程。

ACM_N2P18V

iRange

ACM 电流量程。

ACM_500MA

ACM_200MA

ACM_20MA

ACM_2MA

ACM_200UA

ACM_20UA

ACM_5UA

注意：

- 1) 选择 ACM_500MA 档位时，当电流绝对值小于 215mA 时，板卡工作在直流模式，当电流绝对值大于 215mA 时，板卡工作在脉冲模式，持续时间为 10mS。215mA 是一个近似值；
- 2) 当源工作在上述脉冲模式且脉冲持续时间超出 10ms 后，电流档位会自动切换到 20mA 电流档位，此时源的输出电流变为之前的 1/25，此时的测流值异常，系统会有 alarm 报警；
- 3) 当源工作在上述脉冲模式且脉冲持续时间超出 10ms 后，如果需要再次将源设置成 500mA 电流档位，则需要先将源设置成其他电流档位（除 500mA 档位以外的其他电流档位），再设置成 500mA 电流档位。

enableRelay

设置输出继电器如何动作，分为三种情况：

ACM_RELAY_ON: 打开输出继电器；

ACM_RELAY_OFF: 关闭输出继电器；

ACM_RELAY_HOLD: 输出继电器保持上一个状态。

risingTime

double 型变量，设置积分时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默

认为 0.1mS;

注意：

- 1) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 2) 该参数设置大于 0.1mS 且小于等于 0.2mS 时，risingTime=0.2mS，为 Slow 模式；
- 3) 该参数设置大于 0.2mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1：acm0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

```
double setValue[2] = {3, 5};
int siteSize = 2;

acm0.SetSyn(FV, setValue, siteSize, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
```

例 2：acm0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;

acm0.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
delay_ms(1);
acm0.SetSyn(FV, setValue, siteSize, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON, 3);
```

6.2.4 SetClamp()

```
int SetClamp(
double percent_PFS,
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。

范围：箝压 10%~105%

箝流 10%~112%

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。

范围：箝压 10%~105%

箝流 10%~112%

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-1：函数调用失败。

Remarks

设置 ACM 的正负箝位值。

注意：

- 1) SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成满箝位值。未切换 FV 和 FI 状态时，箝位设置会保持不变。
- 2) 如果设置的箝位值超出箝位上下限时，按照上下限执行，且软件报 Alarm。

Example

例 1：设置 acm1 为 FI=0 模式，箝位到(50, 20)，然后切换 acm1 为 FV=1V 模式，箝位到(25, 25)。

```
acm1.Set(FI, 0, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);
```

```
//positive clamp to 50% of full range(ACM_N2P18V), negative clamp to 20% of full range
acm1.SetClamp(50, 20);

//switch from FI to FV mode, then positive/negative clamps recover back to 100% of full range
acm1.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
//reset positive clamp to 25% of full range(ACM_200MA), negative clamp to 25% of full range
acm1.SetClamp(25,25);
//Set FV=1V
acm1.Set(FV,1,ACM_N2P18V,ACM_200MA,ACM_RELAY_ON);
```

例 2：设置 acm1 为 FV=10V 模式，箝位到(50, 25)，然后切换 acm1 为 FV=-2V，测量结束后设置 acm1 为 FV=0，箝位到(100, 100)。

```
acm1.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
//positive clamp to 50% of full range(ACM_200MA), negative clamp to 25% of full range
acm1.SetClamp(50, 25);
acm1.Set(FV, 10, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
delay_ms(1);

//still in FV mode, clamp setting will keep unchanged
acm1.Set(FV, -2, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
acm1.MeasureVI(ACM_MV, 100, 20);

acm1.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
//after FV=0, set clamps back to 100%
acm1.SetClamp(100, 100);
```

6.2.5 MeasureVI()

```
int MeasureVI(
ACM_MEASTYPE measType,
UINT sampleTimes,
double samplePeriod,
FLOATMEASMODE measMode = MEAS_NORMAL,
UINT T1 = 0,
UINT T2 = 0,
UINT T3 = 0,
UINT T4 = 0);
```

Parameters

measType

设置 ACM 测量类型。可选值为：

ACM_MV：测压模式，回读的测量结果为电压值；

ACM_MI: 测流模式，回读的测量结果为电流值。

sampleTimes

设置 AD 采样次数。范围 1~4096。

注意：

当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；

当 $sampleTimes > 4096$ 时， $sampleTimes = 4096$ 。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：5~30000。

注意：

当 $samplePeriod < 5$ 时， $samplePeriod = 5$ ；

当 $samplePeriod > 30000$ 时， $samplePeriod = 30000$ 。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不设置，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 ACM 的测量结果类型、测量次数、测量间隔、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1：设置 acm0 测量电流值，普通模式测量，采样 20 次，间隔 10uS。

```
acm0.MeasureVI(ACM_MI, 20, 10);
```

6.2.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N： 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数默认值为 AVERAGE_RESULT。

triggNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数默认值为 0，表示回读第一次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

注意：

1) ACM 每次测量时，只能单独测量电压或者电流，因此用户调用此函数的回读结果与 MeasureVI 函数中选择的测量结果类型相关。

2) 当参数 *sampleNumber* 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回值为-3。

Example

例 1：返回 acm0 电流测量结果平均值。

```
//acm0MI, sampleTimes=20, samplePeriod=10uS
acm0.MeasureVI(ACM_MI, 20, 10);
for(i=0; i<SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value
    of current
    adresult[i] = acm0.GetMeasResult(i);
}
```

例 2：返回 acm0 电压测量结果第 15 个点的值。

```
//acm0MV, sampleTimes=20, samplePeriod=10uS
acm0.MeasureVI(ACM_MV, 20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i] = acm0.GetMeasResult(i, 14);
}
```

例 3：AWG 同步测量时，返回 acm0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge
acm0.SetMeasVTrig(2, TRIG_FALLING);
```

```
// acm1 MV, sampleTime=50, samplePeriod=20uS, measure mode is MEAS_AWG
acm1.MeasureVI(ACM_MV, 50, 20, MEAS_AWG);
// the acm0's parameters of MeasureVI function are same with acm1
acm0.MeasureVI(ACM_MV, 50, 20, MEAS_AWG);
// acm1 AWG enable
STSEnableAWG(&acm1);
// acm0 and acm1 measure enable
STSEnableMeas(&acm1, &acm0);
STSAWGRun();
for(i=0; i<SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] =acm0.GetMeasResult(0, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
    Trig_Point1[i] =acm0.GetMeasResult(0, TRIG_RESULT, 1);
}
```

6.2.7 BlockRead()

```
int BlockRead(
UINT siteCount,
UINT start,
UINT size,
double* buffer);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

通过该函数回读的测量结果与 MeasureVI 函数中选择的测量结果类型相关，如果在测量函数中选择的是 ACM_MV 模式，则此处回读的为电压的测量结果，反之为电流的测量结果。

Example

```
double result[SITENUM][200]={0.0};
acm0.MeasureVI(ACM_MV, 200, 10);           // Measure Voltage
for(i=0; i<SITENUM; i++)
{
    //start=0; size=200; Get MVRET, saved to result.
    acm0.BlockRead(i, 0, 200, result[i]);
}
```

6.2.8Pulse()

```
int Pulse(
double pulseValue,
UINTpulseTime= 300,
intmeasStartTime= 0,
ACM_MEASTYPE measType= ACM_MV,
UINTsampleTimes= 10,
double samplePeriod = 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-2~18；

选择 FI 模式时，单位 A。最大范围为-0.5~0.5。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。

范围：

300~40000（当源的电流值绝对值小于 215mA 时）；

300~10000（当源的电流值绝对值大于 215mA 时）；

该参数默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

该参数默认为 0。

measType

设置 ACM 测量类型。可选值为：

ACM_MV：测压模式，回读的测量结果为电压值；

ACM_MI：测流模式，回读的测量结果为电流值。

该参数默认为 ACM_MV。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

该参数默认为 10。

samplePeriod

UINT 型变量，设置采样间隔时间，单位 uS，范围：5~30000，分辨率 1uS。

该参数默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

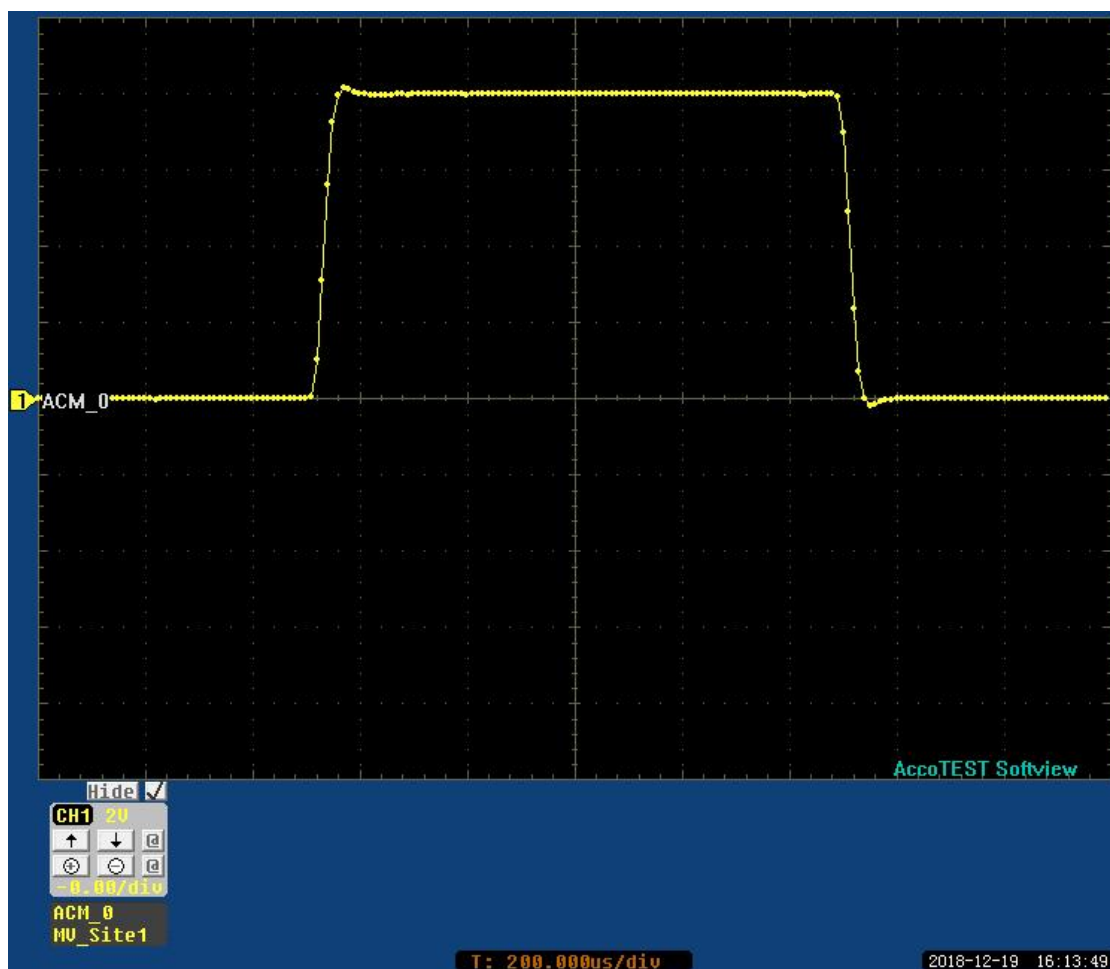
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 acm0 起始状态 FV 为 0。设置 acm0 输出 1mS 的 8V 脉冲；测压，采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
acm0.Set(FV,0,ACM_N2P18V,ACM_20MA,ACM_RELAY_ON);  
acm0.Pulse (8,1000, -500, ACM_MV, 200, 10);
```

使用 SoftView 软件抓取的波形如下：



6.2.9 AwgLoader()

```
int AwgLoader(
char* awgName,
VIMode viMode,
ACM_VRNG vRange,
ACM_IRNG iRange,
double* awgData,
UINT awgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

ACM 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

ACM 电压量程。

ACM_N2P18V

iRange

ACM 电流量程。

ACM_500MA

ACM_200MA

ACM_20MA

ACM_2MA

ACM_200UA

ACM_20UA

ACM_5UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意：

1) 档位信息不涉及档位的切换，仅用于波形数据换算；

- 2) 同一路 ACM 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 ACM 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 ACM 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 ACM 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，acm0 从-1.5V 上升到 3.5V，步距 0.1V。

```
int sam = 50;           //AWG waveform data length
int interval = 200;     //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,-1.5,3.5);
//pattern array saved in 'awg_pattern', start from -1.5 to 3.5
acm0.AwgLoader("Vst",FV,ACM_N2P18V,ACM_20MA,awg_pattern,sam);
//pattern named as 'Vst', FV mode
```

例 2：建立一个波形数据组，acm0 输出频率为 100Hz、峰峰值为 4V、直流偏置为 5V 的正弦波。

```
int sam = 1000;        //AWG waveform data length
int interval = 10;     //AWGdata interval time, unit is uS
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,5,0);
//set a sinewave data array of 100Hz, Vpp=4, DC Offset=5
acm0.AwgLoader("Sine",FV,ACM_N2P18V,ACM_2MA,awg_pattern,sam);
// pattern named as 'SINE', FV mode
```

6.2.10 AwgSelect()

```
int AwgSelect(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值设定为-1 时，AWG 波形保持在运行停止的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点

以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 ACM 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;         //data size for AWG pattern
int sam2 = 500;          // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,-1.5,3.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,-0.5,-1.5);

// pattern named as 'AWG_Vst', FV mode
acm0.AwgLoader("AWG_Vst",FV,ACM_N2P18V,ACM_200MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
acm0.AwgLoader("AWG_Vuvlo",FV,ACM_N2P18V,ACM_200MA,awg_pattern2,sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of
// its 'AwgLoader()'
acm0.Set(FV, -1.5,ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
acm0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
.....
STSAWGRun();    //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.2.11 AwgRun()

```
int AwgRun (
```



```
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval = 10.0,  
BOOL runMode = AWG_SINGLE,  
UINT delayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 *AwgLoader* 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点。

当该值设定为-1 时，AWG 波形保持在运行停止的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 **AWG_SINGLE**。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 *delayTime_ms* 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，*delayTime_ms* 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 ACM 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRUN。

Example

例 1：acm0 连续输出 120Hz 正弦波，峰峰值 0.66V，直流偏置为 3.3V。

```
int sam = 830;           //size of AWG pattern
double interval = 10;     //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);
```

```
// pattern named as 'Sin_AWG', FV mode
acm0.AwgLoader("Sin_AWG",FV, ACM_N2P18V, ACM_500MA,awg_pattern,sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
acm0.Set(FV, 0, ACM_N2P18V, ACM_500MA, ACM_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
acm0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
```

6.2.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 ACM 通道的 AWG 循环运行。

Example

例 1：acm0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150;           //data size for AWG pattern
double interval = 10;    // interval time for AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,2,3,0);
//set a sinewave data array, Vpp=2, DC Offset=3

acm0.AwgLoader("Sin_AWG", FV, ACM_N2P18V, ACM_500MA, awg_pattern, sam);
// pattern named as 'Sin_AWG'

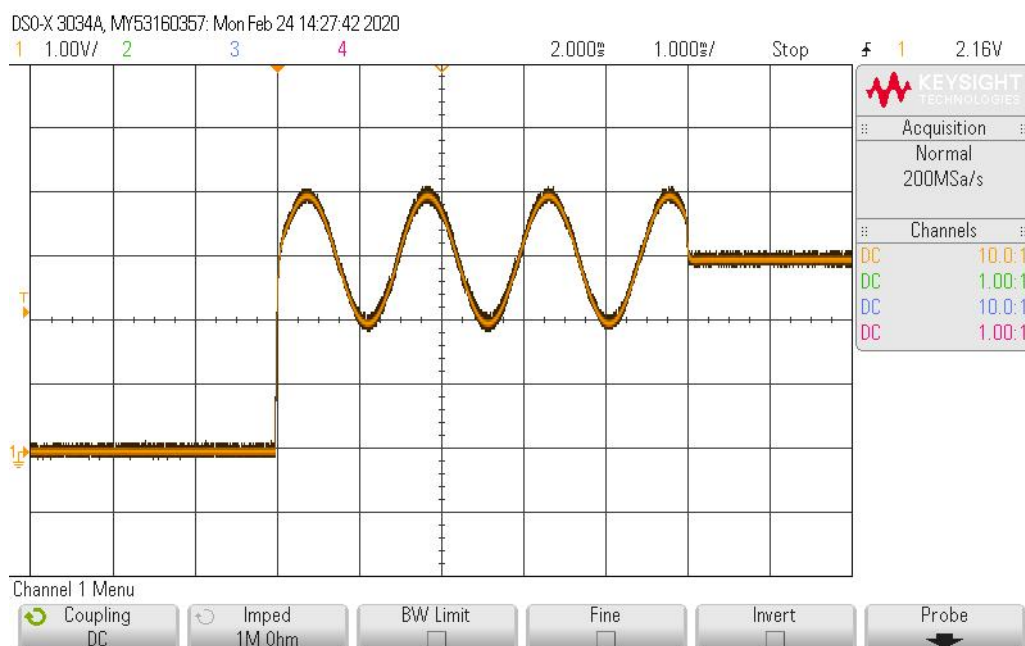
acm0.Set(FV, 0, ACM_N2P18V, ACM_500MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'

acm0.AwgRun("Sin_AWG", 0, (sam-1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode

delay_ms(5);
acm0.AwgStop();          //stop running AWG waveform of acm0
```

该正弦波周期为 1.5mS，延时 5mS 后，停止 awg 波形。

下图为示波器抓取的实际波形。



6.2.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 ACM 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 acm0 通道的 AWG 数据。

```
acm0.AwgClear();           //clear AWG data of acm0
acm0.AwgLoader("P1",FV,ACM_N2P18V,ACM_200MA,awg_pattern1,sam);
```

6.2.14 SetMeasVTrig()

```
int SetMeasVTrig(
double vTrig,
TRIG_MODE trigMode = TRIG_FALLING,
```

```
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数默认为 TRIG_FALLING 下降沿触发。

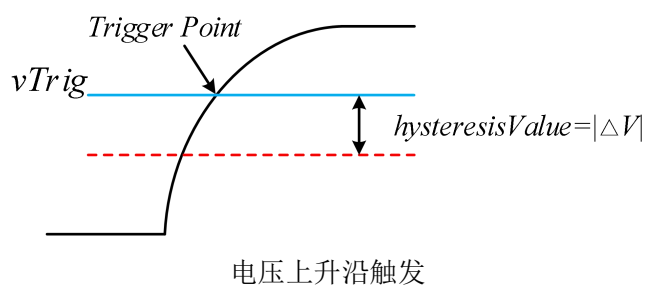
hysteresisValue

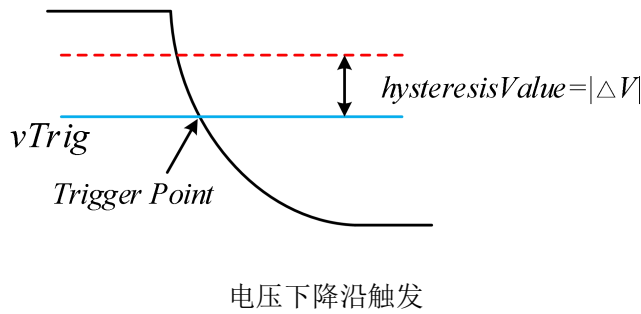
double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($vTrig + hysteresisValue$) 值之后又小于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发。





Remarks

电压测量触发设置。

注意：

SetMeasureVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1：建立一个波形数据组，acm0 从 10.5V 下降到 8V，步距 0.1V。设置 acm1 下降沿触发，触发电平 2V。

```
acm1.Set(FI, 0, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);

double Trig = 2;    //triggering voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 25;
double interval = 200;
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.

acm0.AwgLoader("Uvlo",FV,ACM_N2P18V,ACM_200MA,awg_pattern,sam);
//import the awg_pattern array into acm0 channel, named it as 'Uvlo'. Set to FV mode

acm0.Set(FV, 10.5, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'

acm0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
acm1.SetMeasVTrig(Trig, TRIG_FALLING);
//trigger value is 2V, falling edge

acm0.MeasureVI(ACM_MV, sam,interval, MEAS_AWG);
acm1.MeasureVI(ACM_MV, sam,interval,MEAS_AWG);
```

```
STSEnableAWG(&acm0);           //enable AWG pattern for acm0  
STSEnableMeas(&acm0, &acm1); //enable measurement for acm0 and acm1  
STSAWGRun();                   //Enable AWG and measurement synchronously
```

6.2.15 SetMeasITrig()

```
int SetMeasITrig(  
double iTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数默认为 TRIG_FALLING 下降沿触发。

hysteresisValue

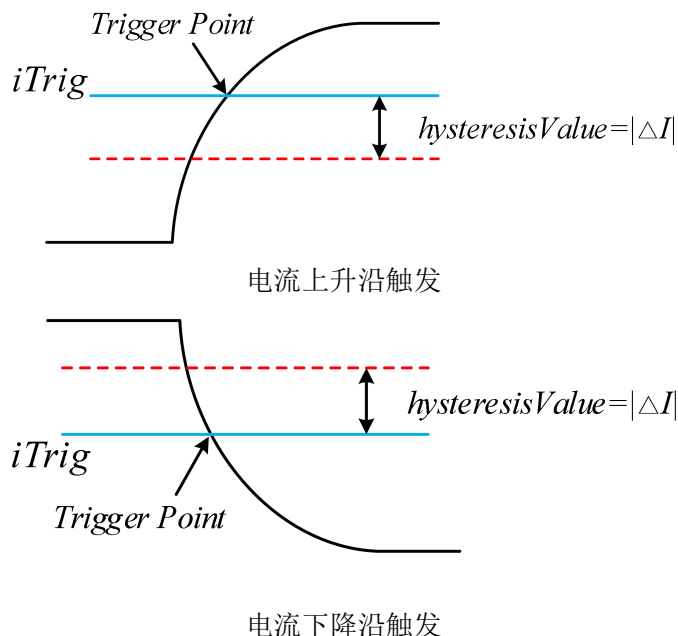
double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (*iTrig-hysteresisValue*) 值之后又大于 *iTrig* 值时，此时在 *iTrig* 处判定为触发；

3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($iTrig + hysteresisValue$)值之后又小于 $iTrig$ 值时, 此时在 $iTrig$ 处判定为触发。



Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时, 对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1: 建立一个波形数据组, acm0 从-1.5V 上升到 3.5V, 步距 0.1V。设置 acm0 上升沿触发, 触发电流值 100uA。

```
double Trig = 100e-6;           // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                   // data size of AWG pattern
int interval = 200;             // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from -1.5 to 3.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, -1.5, 3.5);
```



```
// import the awg_pattern array into acm0 channel, named it as 'Vst'. Set to FV mode.
acm0.AwgLoader("Vst", FV, ACM_N2P18V, ACM_20MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
acm0.Set(FV, -1.5, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
acm0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
acm0.SetMeasITrig(Trig, TRIG_RISING);           // trigger value is 100uA, rising edge

acm0.MeasureVI(ACM_MI, sam, interval, MEAS_AWG);
STSEnableAWG(&acm0);           // enable AWG pattern for acm0
STSEnableMeas(&acm0);           // enable measurement for acm0
STSAWGRun();                     // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
acm0.SetMeasITrig(Trig, TRIG_RISING);           // trigger value is 100uA, rising edge

//insert a Set() call, change its current range from ACM_20MA to ACM_2MA, it will cause a utility error.
acm0.Set(FV, -1.5, ACM_N2P18V, ACM_2MA, ACM_RELAY_ON);

acm0.MeasureVI(ACM_MI, sam, interval, MEAS_AWG);
STSEnableAWG(&acm0);           // enable AWG pattern for acm0
STSEnableMeas(&acm0);           // enable measurement for acm0
STSAWGRun();                     // Enable AWG and measurement synchronously
```

6.2.16 SetAlarmMask()

```
void SetAlarmMask(
    UINT siteCount,
    bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
acm0.SetAlarmMask(0);           // mask the Alarm information of SITE_1
// The alarm information about overranging voltage scale is not output
acm0.Set(FV, 19, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);
```

6.2.17 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息；

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = acm0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.2.18 获取板卡固有信息函数

6.2.18.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
acm0.GetBoardSN(0, boardSN, 255);           // Get the board serial number
```

6.2.18.2 GetBoardHDRev()

```
int GetBoardHDRev (  
UINT siteCount,  
char* hardRev,  
UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡硬件版本号。。

Example

```
char hardRev [255] = {0};  
acm0.GetBoardHDRev(0, hardRev, 255);    // Get the board hardware revision
```

6.2.19 获取校准信息函数

6.2.19.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = acm0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.2.19.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = acm0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.2.19.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = acm0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.2.19.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS;
CAL_FAIL	校准 FAIL;
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = acm0.GetCalibrationResult(0);           // Get calibration result of SITE_1
```

6.2.19.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量, 选择读取校准时逻辑版本信息的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

正常返回逻辑版本号; 异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = acm0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.2.19.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(
```



```
UINT siteCount,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000	Keithley2000
STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0}; // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = acm0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.2.19.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteCount,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* boardHdRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回“N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回“N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 boardHdRev 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = acm0.GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.2.19.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(
UINT siteCount,
char* softRev,
UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回 “N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0}; // the buffer for saving softwareversion
// Get the version of calibration software
acm0.GetCalibrationSoftRev(0, softRev, 255);
```

6.2.19.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = acm0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.2.20 编程范例

6.2.20.1 设置一组波形数据组的拼接

测试条件：

- 第一组：设置一个正弦波数据组，首地址从 0 开始，数据长度为 100；
- 第二组：设置一个三角波数据组，首地址从 100 开始，数据长度为 100；
- 第三组：设置一个方波数据组，首地址从 200 开始，数据长度为 100；
- 第四组：设置一个斜波数据组，首地址从 300 开始，数据长度为 100。

```
int sam=400; //AWG waveform data length
int interval=200; //AWG data interval time, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],100,1,-4,0,180);
// Set a sinewave data array, the start address starts from 0, data size is 100.
```

```
STSAWGCreateTriangleData(&awg_pattern[100],100,4,-4,0,0);
//Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200],100,4,-4,0,50);
//Set a square wave data array, the start address starts from 200, data size is 100.

STSAWGCreateRampData(&awg_pattern[300],100,1,1,5);
//Set an ramp data array, the start address starts from 300, data size is 100.

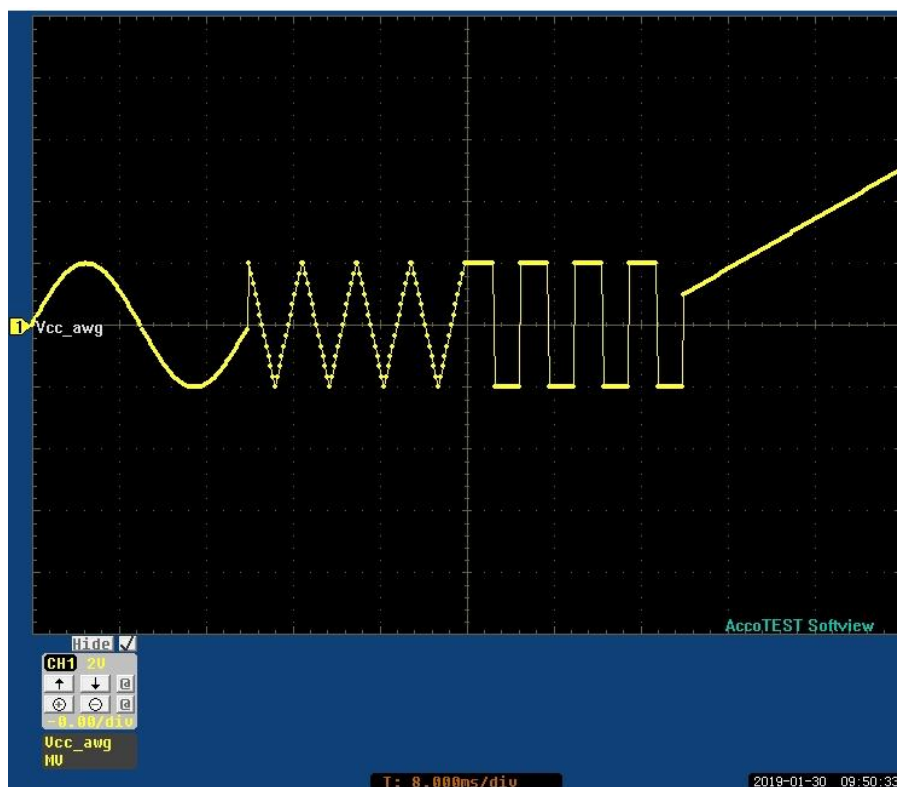
acm0.AwgLoader("Vcc_awg",FV,ACM_N2P18V,ACM_20MA,awg_pattern,sam);
//import the awg_pattern array into acm0 channel, named it as 'Vcc_awg'. Set to FV mode.

acm0.Set(FV, 0, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

acm0.AwgSelect("Vcc_awg", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

acm0.MeasureVI(ACM_MV, sam, interval, MEAS_AWG);
STSEnableAWG(&acm0);//enable AWG pattern for acm0
STSEnableMeas(&acm0);//enable measurement for acm0
STSAWGRun();//Enable AWG and measurement synchronously
```

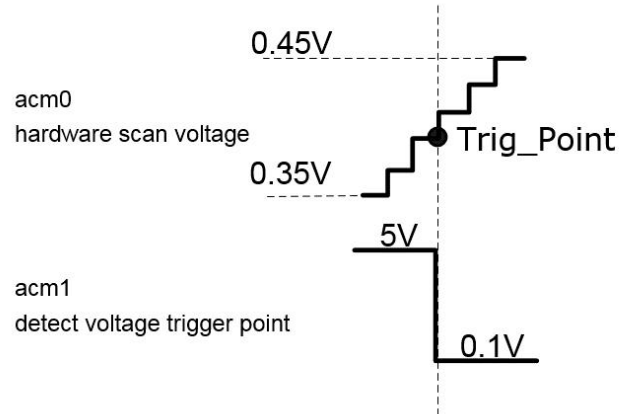
使用 SoftView 软件抓取的波形如下：



6.2.20.2 一路源生成 AWG 另一路源作为触发源

测试条件：

建立一个波形数据组，acm0 从 0.35V 上升 0.45V，步距 0.001V。测试 acm1 端的电压小于 2V 时，acm0 跳变点的电压值。



```
double Trig = 2; //trigger voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

acm1.Set(FI, 0, ACM_N2P18V, ACM_2MA, ACM_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0],sam,1,0.35,0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

acm0.AwgLoader("Vcspre_AWG",FV,ACM_N2P18V,ACM_200MA,awg_pattern,sam);
//import the awg_pattern array into acm0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

acm0.Set(FV, 0.30, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'
delay_ms(1);

acm0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

acm1.SetMeasVTrig(Trig,TRIG_FALLING); // trigger value is 2V, Falling edge

acm0.MeasureVI(ACM_MV,sam,interval,MEAS_AWG);
acm1.MeasureVI(ACM_MV,sam,interval,MEAS_AWG);
STSEnableAWG(&acm0); //enable AWG pattern for acm0
STSEnableMeas(&acm0,&acm1); //enable measurement for acm0 and acm1
STSAWGRun(); //Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] =acm1.GetMeasResult(i,TRIG_RESULT);
    //Get the position of trigger point for acm1

    result[i] = acm0.GetMeasResult(i,Trig_Point[i]);
    //Read the voltage value of acm0 on trigger position
}
```

```
Vcspre->SetTestResult(i, 0, result[i]);  
}
```

6.2.20.3 AWG 生成正弦波测试纹波抑制比

测试条件:

acm0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。测试 acm1 输出纹波抑制比。

```
int Datacount=0;  
double AD_SAMPLE=500;//sampling points  
double sum[4]={0.0};  
double temp[4]={0.0},temp2[4]={0.0};  
int sam = 830; //data size of AWG pattern  
int interval = 10; // interval time of AWG pattern, unit is uS  
double awg_pattern[1000]={0.0};  
double data[5000]={0.0};  
double result[4]={0.0};  
  
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);  
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3  
  
acm0.AwgLoader("Sin_AWG",FV,ACM_N2P18V,ACM_200MA,awg_pattern,sam);  
// pattern named as 'Sin_AWG, FV mode  
  
acm0.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);  
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its  
'AwgLoader()'  
  
acm0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);  
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode  
delay_ms(1);  
  
acm1.Set(FI, -100e-3, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);  
//Vout FI mode, prepares to measure  
delay_ms(2);  
acm1.MeasureVI(ACM_MV, AD_SAMPLE, 20); //acm1 measurement  
  
for (i=0;i<SITENUM;i++)  
{  
    for(Datacount=0;Datacount<AD_SAMPLE;Datacount++)  
    {  
        data[Datacount]= acm1.GetMeasResult(i, Datacount);  
        //acm1acquires every voltage values from 500 sampling points  
        result[i]= acm1.GetMeasResult(i);  
        //acm1acquiresaverage voltage of 500 sampling points  
        sum[i]+=(data[Datacount]-result[i])*(data[Datacount]-result[i]);  
    }  
    temp[i]=(sqrt(sum[i]/AD_SAMPLE))*2.828;  
    //calculate peak to peak value of sine wave received from output  
    temp2[i]=-20*log10((temp[i]+1e-12)/0.66);  
    //calculate PSRR  
    sum[i]=0;
```



```
}  
  
for(i=0; i<SITENUM; i++)  
{  
    PSRR->SetTestResult(i, 0, temp2[i]);  
}  
  
acm1.Set(FI, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_OFF);  
acm0.AwgStop();//stop running AWG waveform of acm0  
acm0.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_OFF);
```

6.3 ACM200

返回值	返回值意义	备注
-1	该函数不支持 Group 功能	
-2	工位无效	
-3	未发生触发	
-4	Group 通道不能进行 AWG 模式下的测量	
-5	AWG 波形不存在	
-6	当前逻辑通道大于系统支持的最大逻辑通道数	检查逻辑通道号
-7	未设置 GangRelay	检查是否设置了 GangRelay
-8	选择的 AWG 波形的起始位置大于(该波形的长度-1)	检查 AWG 起始位置参数
-9	通道无效	
-10	回读失败，测量未启动	检查是否进行了测量
-11	该逻辑通道下的所有工位皆无效	
-12	设置的触发状态与回读的触发状态不一致	检查是否设置了触发
-13	主通道与并联通道的属性不同	检查所有通道属性
-14	主通道与并联通道的工位个数不相同	
-15	相同工位的并联通道不属于同一 Bank	
-16	不同工位中的并联通道属于同一 Bank	
-17	所有并联通道中存在相同的通道	检查并联通道是否有重复
-18	主通道与一个并联通道相同	检查主通道与并联通道
-19	数组大小无效	请检查数组大小
-20	参数设置异常	
-21	数据长度小于 1	
-22	未设置第二次触发阈值	
2.00E+15	工位错误	
4.00E+15	Group 属性通道不能进行回读操作	
9.00E+15	其他错误，回读失败	

6.3.1 ACM200()

```
ACM200(char*channelList, char*chName = NULL);
```

Parameters

channelList

ACM200 绑定通道字符串。形如“S5_0,S5_1”，其中“S5”表示槽位号 5（S 即 Slot），“0、1”表示板内序号。

注意：

ACM200 一块板卡最多 24 个通道，共四个 bank，每个 bank 最多有 6 个通道。因此板内序号最大值为 23。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 ACM200_xx。

Remarks

定义一路 ACM200 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 4 块 ACM200（分别位于 Slot5、Slot11、Slot22 和 Slot28），四工位并行工作，定义可使用的 ACM200。

```
ACM200 ACM200_0("S5_0,S11_0,S22_0,S28_0");
ACM200 ACM200_1("S5_1,S11_1,S22_1,S28_1");
ACM200 ACM200_2("S5_2,S11_2,S22_2,S28_2");
ACM200 ACM200_3("S5_3,S11_3,S22_3,S28_3");
ACM200 ACM200_4("S5_4,S11_4,S22_4,S28_4");
ACM200 ACM200_5("S5_5,S11_5,S22_5,S28_5");
ACM200 ACM200_6("S5_6,S11_6,S22_6,S28_6");
ACM200 ACM200_7("S5_7,S11_7,S22_7,S28_7");
ACM200 ACM200_8("S5_8,S11_8,S22_8,S28_8");
ACM200 ACM200_9("S5_9,S11_9,S22_9,S28_9");
ACM200 ACM200_10("S5_10,S11_10,S22_10,S28_10");
ACM200 ACM200_11("S5_11,S11_11,S22_11,S28_11");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_ACM200, SITE_1, "S5_0-11");           // set Slot5 ch0-11 to SITE1
STSSetMultiSiteBind(MD_ACM200, SITE_2, "S11_0-11");          // set Slot11 ch0-11 to SITE2
STSSetMultiSiteBind(MD_ACM200, SITE_3, "S22_0-11");          // set Slot22 ch0-11 to SITE3
STSSetMultiSiteBind(MD_ACM200, SITE_4, "S28_0-11");          // set Slot28 ch0-11 to SITE4
```

6.3.2Set()

```
int Set(
VIMode viMode,
double setValue,
ACM200_VRNG vRange,
ACM200_IRNG iRange,
ACM200_OUT_RELAY relayStatus = ACM200_RELAY_HOLD,
double risingTime = 0.1);
```

Parameters

viMode

ACM200 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.2~0.2。

vRange

ACM200 电压量程。

ACM200_40V

ACM200_20V

ACM200_10V

ACM200_3p6V

iRange

ACM200 电流量程。

ACM200_200MA

ACM200_100MA

ACM200_10MA

ACM200_1MA

ACM200_100UA

ACM200_10UA

ACM200_1UA

relayStatus

设置输出继电器如何动作，分为四种情况：

ACM200_RELAY_HOLD：输出继电器保持上一个状态；

ACM200_RELAY_ON：打开输出继电器；

ACM200_RELAY_OFF：关闭输出继电器；

ACM200_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 ACM200 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.1mS；

- 1) 该参数设置小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) 该参数设置大于 0.1mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Return Values

返回 0：设置成功；

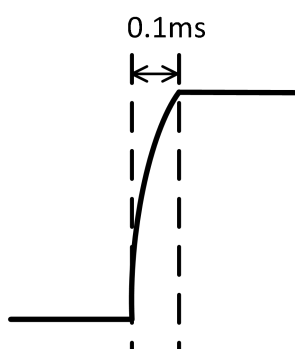
Remarks

设置 ACM200 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

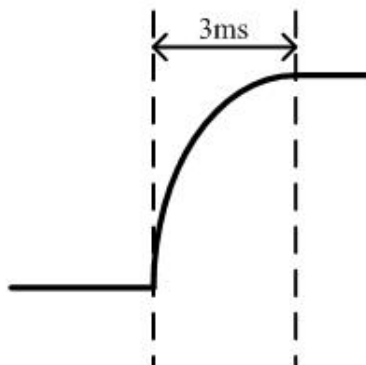
例 1：设置 ACM200_0 电压输出 5V，risingTime 不填则默认为 0.1mS。

```
ACM200_0.Set(FV, 5, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
```



例 2：设置 ACM200_0 电压输出 5V，risingTime 设置为 3mS。

```
ACM200_0.Set(FV, 0, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
delay_ms(1);
ACM200_0.Set(FV, 5, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON, 3);
```



6.3.3 SetSyn()

```
int SetSyn(
    VIMode viMode,
    const double* setValue,
    UINT siteSize,
```

```
ACM200_VRNG vRange,  
ACM200_IRNG iRange,  
ACM200_OUT_RELAY relayStatus = ACM200_RELAY_HOLD,  
double risingTime = 0.1);
```

Parameters

viMode

ACM200 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

etValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.2~0.2。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

ACM200 电压量程。

ACM200_40V

ACM200_20V

ACM200_10V

ACM200_3p6V

iRange

ACM200 电流量程。

ACM200_200MA

ACM200_100MA

ACM200_10MA

ACM200_1MA

ACM200_100UA

ACM200_10UA

ACM200_1UA

relayStatus

设置输出继电器如何动作，分为七种情况：

ACM200_RELAY_HOLD：输出继电器保持上一个状态；

ACM200_RELAY_ON：打开输出继电器；

ACM200_RELAY_OFF：关闭输出继电器；

ACM200_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 ACM200 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.1mS；

- 1) 该参数设置小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) 该参数设置大于 0.1mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Return Values

返回 0：设置成功；

返回-1：该函数不支持 Group 功能。

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1：ACM200_0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

```
double setValue[2] = {3, 5};
int siteSize = 2;
ACM200_0.SetSyn(FV, setValue, siteSize, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
```

例 2：ACM200_0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;

ACM200_0.Set(FV, 0, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
delay_ms(1);
ACM200_0.SetSyn(FV, setValue, siteSize, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON, 3);
```

6.3.4SetClamp()

```
int SetClamp(
double percent_PFS,
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：2~105。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：2~105。

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-9：通道无效。

Remarks

设置 ACM200 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 105%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 ACM200_1 为 FI=0 模式，箝位到(50, 20)，然后切换 ACM200_1 为 FV=1V 模式，箝位到(25, 25)。

```
ACM200_1.Set(FI, 0, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);  
// positive clamp to 50% of full range(ACM200_10V), negative clamp to 20% of full range  
ACM200_1.SetClamp(50, 20);  
  
// switch from FI to FV mode, then positive/negative clamps recover back to 110% of full range  
ACM200_1.Set(FV, 0, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);  
// reset positive clamp to 25% of full range(ACM200_100MA), negative clamp to 25% of full range  
ACM200_1.SetClamp(25, 25);  
// Set FV=1V  
ACM200_1.Set(FV, 1, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
```

例 2：设置 ACM200_1 为 FV=1V 模式，箝位到(50, 25)，然后切换 ACM200_1 为 FV=25V，测量结束后设置 ACM200_1 为 FV=0，箝位到(100, 100)。

```
ACM200_1.Set(FV, 0, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);  
// positive clamp to 50% of full range(ACM200_100MA), negative clamp to 25% of full range
```

```
ACM200_1.SetClamp(50, 25);
ACM200_1.Set(FV, 1, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
delay_ms(1);

// still in FV mode, clamp setting will keep unchanged
ACM200_1.Set(FV, 25, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);
ACM200_1.MeasureVI(100, 20);

ACM200_1.Set(FV, 0, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);
// after FV=0, set clamps back to 100%
ACM200_1.SetClamp(100, 100);
```

6.3.5 MeasureVI()

```
int MeasureVI(
UINT sampleTimes,
double samplePeriod,
FLOATMEASMODE measMode = MEAS_NORMAL,
UINT T1 = 0,
UINT T2 = 0,
UINT T3 = 0,
UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096

注意：

当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；

当 $sampleTimes > 4096$ 时， $sampleTimes = 4096$ 。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 1uS，分辨率 1uS。范围：1~30000。

注意：

当 $samplePeriod < 1$ 时， $samplePeriod = 1$ ；

当 samplePeriod > 30000 时, samplePeriod = 30000。

measMode

测量模式选择, 可设置为:

MEAS_NORMAL: 普通测量模式。调用函数后完成测量。

MEAS_AWG: AWG 测量模式。只做设置, 运行 AWG 时启动。

该参数可以不写, 默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS, 范围: 0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下, 设置测量启动时刻。一次 AWG 中, 可以启动四次测量。四个参数均可以不写, 默认启动一次, 启动时刻为 0。对于这种测量模式, 建议跟同步启动测量配合使用。

注意:

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间, T3 的启动时间为相对 T2 的启动时间, T4 的启动时间为相对 T3 的启动时间。

Return Values

返回 1: 函数运行成功, AWG 模式下测量设置;

返回 0: 设置成功, DC 模式下测量设置;

返回-4: Group 通道不能进行 AWG 模式下的测量。

Remarks

设置 ACM200 的测量次数、测量间隔、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1: 设置 ACM200_0 为普通模式测量, 采样 20 次, 间隔 10uS。

```
ACM200_0.MeasureVI(20,10);
```

6.3.6 GetMeasResult()

```
double GetMeasResult(  
    UINT siteCount,  
    MeasRet retType = MVRET,  
    int sampleNumber = AVERAGE_RESULT,  
    BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retType

选择测量读回的数据类型，可设置为

MVRET：回读电压测量数据

MIRET：回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT：平均值

MAX_RESULT：最大值

MIN_RESULT：最小值

TRIG_RESULT：触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N：返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读 vTrig\iTrig 第一次发生触发的位置，该参数设置为 1，表示回读 vTrig\iTrig 第二次发生触发的位置。该参数设置为 2，表示回读 vTrig2\iTrig2 第一次发生触发的位置，该参数设置为 3，表示回读 vTrig2\iTrig2 第二次发生触发的位置（vTrig\iTrig 含义详见函数 SetMeasVTrig\SetMeasITrig）。

Return Values

当 *sampleNumber* 选择 TRIG_RESULT 时：

正常情况下：

返回触发点位置数据，函数运行正常；

异常情况下，回读失败：

返回-1：Group 属性的通道不能进行回读操作；

返回-2：工位无效；

返回-3：未发生触发；

返回-12：设置的触发状态与回读的触发状态不一致；

返回-20：参数设置异常；

返回-22：未设置第二次触发阈值

当 *sampleNumber* 未选择 TRIG_RESULT 时：

正常情况下：

返回对应的测量结果（平均值、最大值、最小值、单点值），函数运行正常；

异常情况下，回读失败：

返回 2.00e+15：工位错误；

返回 4.00e+15：Group 属性的通道不能进行回读操作；

返回 9.00e+15：其他错误，回读失败；

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

1) ACM200 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。

Example

例 1：返回 ACM200_0 电流测量结果平均值。

```
// ACM200_0 sampleTimes=20, samplePeriod=10uS
ACM200_0.MeasureVI(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is
    // average value of current
    adresult[i] = ACM200_0.GetMeasResult(i, MIRET);
}
```

例 2：返回 ACM200_0 电压测量结果第 15 个点的值。

```
//ACM200_0 sampleTimes=20, samplePeriod=10uS
ACM200_0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=ACM200_0.GetMeasResult(i, MVRET, 14);
}
```

例 3：AWG 同步测量时，返回 ACM200_0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge
ACM200_0.SetMeasVTrig(2, TRIG_FALLING);
// ACM200_1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG
ACM200_1.MeasureVI(50, 20, MEAS_AWG);
// the ACM200_0's parameters of MeasureVI function are same with ACM200_1
ACM200_0.MeasureVI(50, 20, MEAS_AWG);
// ACM200_1 AWG enable
STSEnableAWG(&ACM200_1);
// ACM200_0 and ACM200_1 measure enable
STSEnableMeas(&ACM200_1, &ACM200_0);
STSAWGRun();
for (i = 0; i < SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] = ACM200_0.GetMeasResult(0, MVRET, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
}
```

```
Trig_Point1[i] = ACM200_0.GetMeasResult(0, MVRET, TRIG_RESULT, 1);  
}
```

6.3.7 BlockRead()

```
int BlockRead(  
UINT siteCount,  
UINT start,  
UINT size,  
double* buffer,  
MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET：读回电压测量数据

MIRET：读回电流测量数据

Return Values

返回 0：回读成功；

返回-1：该函数不支持 Group 功能；

返回-2：工位无效；

返回-10：回读失败，测量未启动。

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200] = { 0.0 };
ACM200_0.MeasureVI(200, 10);
for (i = 0; i < SITENUM; i++)
{
    // start=0; size=200; Get MVRET, saved to result.
    ACM200_0.BlockRead(i, 0, 200, result[i], MVRET);
}
```

6.3.8Pulse()

```
int Pulse(
double pulseValue,
UINT pulseTime = 300,
int measStartTime = 0,
UINT sampleTimes = 10,
double sampleInterval = 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-40~40；

选择 FI 模式时，单位 A。最大范围为-0.2~0.2。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。范围：1~30000。最小采样间隔为 1uS，分辨率 1uS。

可以不写，默认为 30uS。

Return Values

返回 0：设置成功；

返回-1：该函数不支持 Group 功能。。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

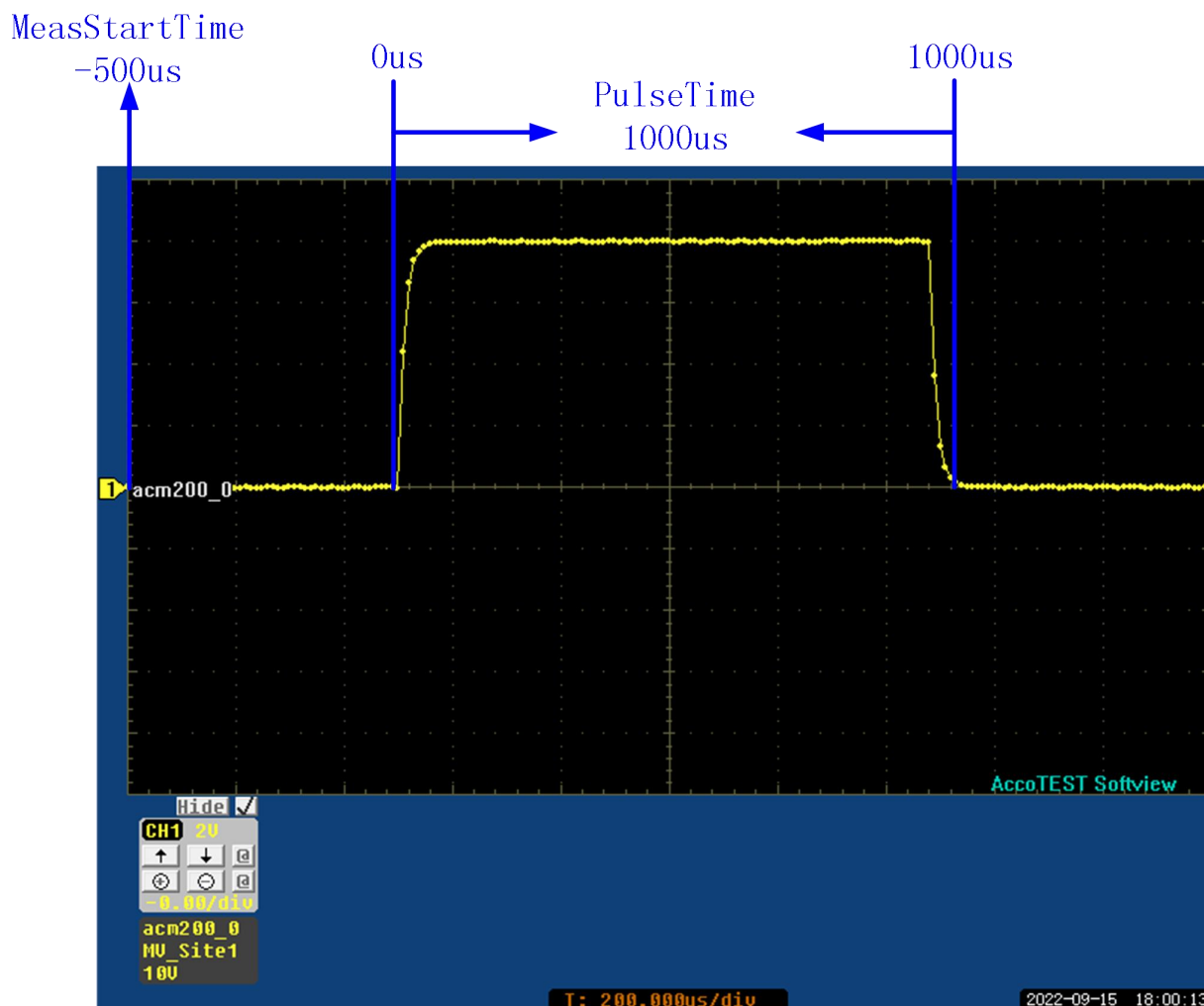
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 ACM200_0 起始状态 FV 为 0。设置 ACM200_0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
ACM200_0.Set(FV, 0, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);
// first set ACM200_0 in FV=0 mode,
ACM200_0.Pulse (8,1000, -500, 200, 10);
//pulse output value=8v, pulse duration=1000uS, start time for measuring is -500uS
//sample=200,interval time=10uS
```

使用 SoftView 软件抓取的波形如下：



6.3.9 AwgLoader()

```
int AwgLoader(  
char* awgName,  
VIMode viMode,  
ACM200_VRNG vRange,  
ACM200_IRNG iRange,  
double* awgData,  
UINT awgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

ACM200 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

ACM200 电压量程。

ACM200_40V

ACM200_20V

ACM200_10V

ACM200_3p6V

iRange

ACM200 电流量程。

ACM200_200MA

ACM200_100MA

ACM200_10MA

ACM200_1MA

ACM200_100UA

ACM200_10UA

ACM200_1UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Return Values

返回 1：数据加载成功，该逻辑通道下的某一物理通道缺失该 AWG 数据，需重新加载；

返回 2：数据加载成功，该逻辑通道中不存在此 AWG 数据。

返回 0：数据加载成功，该逻辑通道下的所有物理通道都存在有该 AWG 数据；

返回-1：该函数不支持 Group 功能。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 ACM200 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右

的时间，写入时间与写入点数成正比；

4) 同一个测试程序中，同一路 ACM200 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；

5) 同一个测试程序中，建议同一路 ACM200 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；

6) 同一个测试程序中，如果同一路 ACM200 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，ACM200_0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50;           //AWG waveform data length
int interval = 200;     //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5); //pattern array saved in 'awg_pattern',
start from 16.5 to 21.5
ACM200_0.AwgLoader("Vst ", FV, ACM200_40V, ACM200_100MA, awg_pattern, sam);
```

例 2：建立一个波形数据组，ACM200_0 输出频率为 100Hz、峰峰值为 4V、直流偏置为 10V 的正弦波。

```
int sam = 1000;        //data size for AWG pattern
int interval = 10;     //interval time for AWG pattern
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0); //set a sinewave data array of 100Hz,
Vpp=4, DC Offset=10
ACM200_0.AwgLoader("Sine", FV, ACM200_40V, ACM200_100MA, awg_pattern, sam); // pattern
named as 'SINE', FV mode
```

6.3.10 AwgSelect()

```
int AwgSelect(
char* awgName,
UINT startAddr,
UINT stopAddr,
int loopBackAddr,
```

```
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Return Values

返回 0：数据选择成功。

返回-1：该函数不支持 Group 功能。

返回-5：AWG 波形不存在；

返回-8：选择的 AWG 波形的起始位置大于(该波形的长度-1)；

返回-11：该逻辑通道下的所有工位皆无效；

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 ACM200 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;          //data size for AWG pattern
int sam2 = 500;           // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0], sam1, 1, 16.5, 21.5);
// set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0], sam2, 1, 16.5, 10.5);

// pattern named as 'AWG_Vst', FV mode
ACM200_0.AwgLoader("AWG_Vst", FV, ACM200_40V, ACM200_100MA, awg_pattern1, sam1);
// pattern named as 'AWG_Vuvlo', FV mode
ACM200_0.AwgLoader("AWG_Vuvlo", FV, ACM200_10V, ACM200_100MA, awg_pattern2, sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of its
'AwgLoader()'
ACM200_0.Set(FV, 16.5, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);
// Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
ACM200_0.AwgSelect("AWG_Vst", 0, sam1 - 1, 0, interval);

STSAWGRun();    //AWG synchronous running started for pattern of 'AWG_Vst'
```


6.3.11 AwgRun()

```
int AwgRun(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0,  
BOOL runMode = AWG_SINGLE,  
UINT delayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE：运行一次 AWG 波形。

AWG_LOOP：循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 AWG_SINGLE。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Return Values

返回 0：数据选择成功。

返回-1：该函数不支持 Group 功能。

返回-5：AWG 波形不存在；

返回-8：选择的 AWG 波形的起始位置大于(该波形的长度-1)；

返回-11：该逻辑通道下的所有工位皆无效；

Remarks

某一路 ACM200 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRun。

Example

例 1：ACM200_0 连续输出 120Hz 正弦波，Vpp 为 0.66V，直流偏置为 3.3V。

```
int sam = 830;          //size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 0.66, 3.3, 0);

// pattern named as 'Sin_AWG', FV mode
ACM200_0.AwgLoader("Sin_AWG", FV, ACM200_10V, ACM200_200MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
ACM200_0.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
ACM200_0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
```

6.3.12 AwgStop()

```
int AwgStop(void);
```

Return Values

- 返回 0： 设置成功；
- 返回-9： 通道无效。

Remarks

停止某一路 ACM200 通道的 AWG 循环运行。

Example

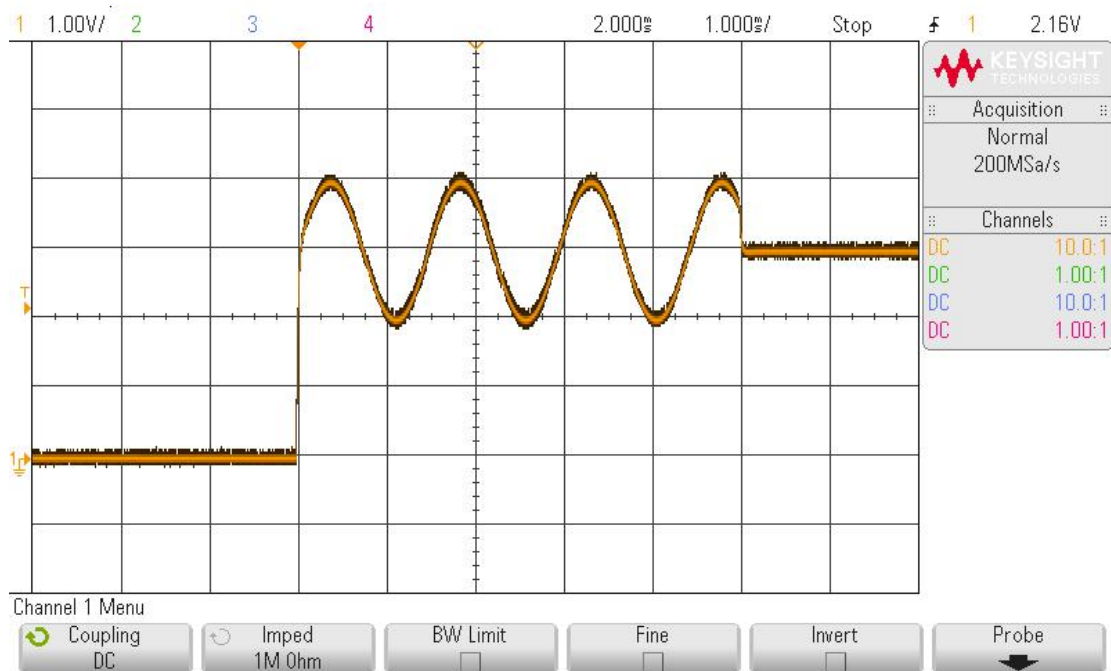
例 1: ACM200_0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150;           //data size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0], sam, 1, 2, 3, 0);
//Set a Sine wave data array, data size is 150, period is 1, Vpp=2V, DC offset = 3V, phase = 0.
ACM200_0.AwgLoader("Sin_AWG", FV, ACM200_10V, ACM200_200MA, awg_pattern, sam);
//Import awg_pattern data array into ACM200_0 channel, pattern named as 'Sin_AWG'
ACM200_0.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
ACM200_0.AwgRun("Sin_AWG", 0, (sam - 1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(6); //ACM200_0 AWG pattern keeps running for 6mS
ACM200_0.AwgStop();//stop loops for ACM200_0 AWG running
```

该正弦波周期为 1.5mS，延时 5mS 后，停止 awg 波形。

下图为示波器抓取的 actual 波形。



6.3.13 AwgClear()

```
int AwgClear(void);
```

Return Values

返回 0: 设置成功;

返回-1: 该函数不支持 Group 功能。

返回-6: 当前逻辑通道大于系统支持的最大逻辑通道数;

Remarks

一般用于 AwgLoader 函数之前, 清除该 ACM200 通道的 AWG 数据。

Example

例 1: Load 波形之前清除 ACM200_0 通道的 AWG 数据。

```
ACM200_0.AwgClear();           //clear AWG data of ACM200_0  
ACM200_0.AwgLoader("P1", FV, ACM200_40V, ACM200_100MA, awg_pattern1, sam);
```

6.3.14 SetMeasVTrig()

```
int SetMeasVTrig(  
double vTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

```
int SetMeasVTrig(  
double vTrig,  
TRIG_MODE trigMode,  
double hysteresisValue,  
double vTrig2,  
TRIG_MODE trigMode2,  
double hysteresisValue2);
```

Parameters

vTrig

double 型变量，设置第一次触发电压。单位 V。

注意：

- 1) 该参数的值与其对应通道所调用的 Set() 函数的电压档位相关；
- 2) 系统在 vTrig 电压处最多可以触发两次。

trigMode

第一次触发模式设置，可设置为：

TRIG_FALLING：下降沿触发

TRIG_RISING：上升沿触发

当使用单次触发功能时，该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue

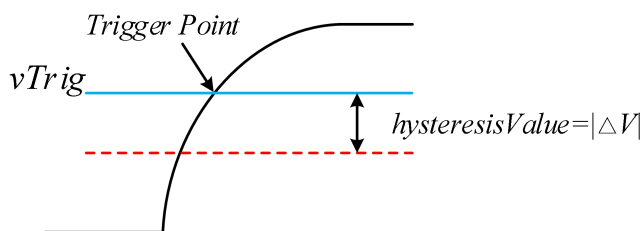
double 型变量，设置第一次触发电压的滞回值。单位 V。

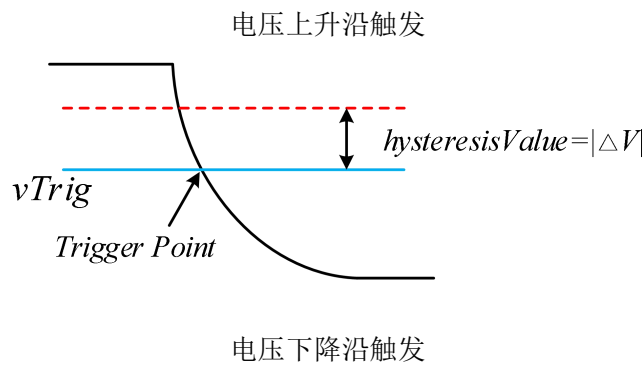
当使用单次触发功能时，该参数可以不写，在默认条件下等于 0。

注意：

当使用单次触发功能时：

- 1) 滞回参数 hysteresisValue 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (vTrig-hysteresisValue) 值之后又大于 vTrig 值时，此时在 vTrig 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 (vTrig+hysteresisValue) 值之后又小于 vTrig 值时，此时在 vTrig 处判定为触发。





vTrig2

double 型变量，设置第二次触发电压。单位 V。

注意：

- 1) 该参数的值与其对应通道所调用的 Set() 函数的电压档位相关；
- 2) 系统在 vTrig2 电压处最多可以触发两次。

trigMode2

第二次触发模式设置，可设置为：

TRIG_FALLING：下降沿触发

TRIG_RISING：上升沿触发

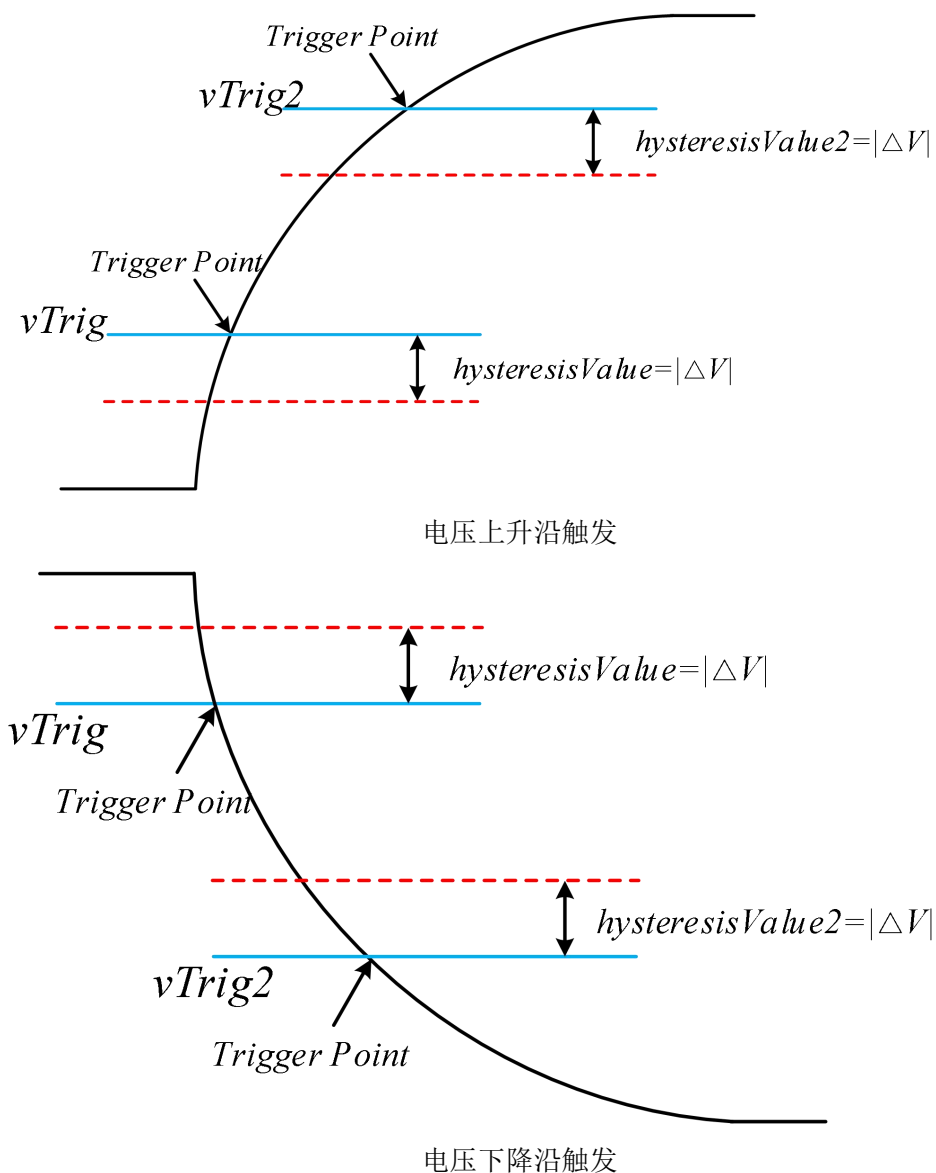
hysteresisValue2

double 型变量，设置第二次触发电压的滞回值。单位 V。

注意：

当使用二次触发功能时：

- 1) 滞回参数 hysteresisValue (hysteresisValue2) 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 $vTrig - hysteresisValue$ ($vTrig2 - hysteresisValue2$) 值之后又大于 $vTrig$ ($vTrig2$) 值时，此时在 $vTrig$ ($vTrig2$) 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 $vTrig + hysteresisValue$ ($vTrig2 + hysteresisValue2$) 值之后又小于 $vTrig$ ($vTrig2$) 值时，此时在 $vTrig$ ($vTrig2$) 处判定为触发。



Return Values

返回 0: 设置成功;

返回-1: 该函数不支持 Group 功能。

Remarks

电压测量触发设置。

注意：

SetMeasVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。

二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1: 建立一个波形数据组, ACM200_0 从 10.5V 下降到 8V, 步距 0.1V。ACM200_1 设置第一次下降沿触发, 触发电平 9V, 第二次为上升沿触发, 触发电平为 8.6V。

```
ACM200_1.Set(FI, 0, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);

double Trig1 = 9; //the first triggering voltage
double Trig1_Point[4] = { 0.0 };
double Trig2 = 8.6; //the second triggering voltage
double Trig2_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 25; //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 10.5, 8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.

ACM200_0.AwgLoader("Uvlo", FV, ACM200_40V, ACM200_100MA, awg_pattern, sam);
//import the awg_pattern array into ACM200_0 channel, named it as 'Uvlo'. Set to FV mode.

ACM200_0.Set(FV, 10.5, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
ACM200_0.AwgSelect("Uvlo", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

设置触发电平的正确做法是:

```
ACM200_1.Set(FI, 0, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);
//set ACM200_1 on FI mode.
ACM200_1.SetMeasVTrig(Trig1, TRIG_FALLING, 0, Trig2, TRIG_RISING, 0);
// the first trigger value is 2V, falling edge, the second trigger value is 5V, rising edge
ACM200_0.MeasureVI(sam, interval, MEAS_AWG);
ACM200_1.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&ACM200_0); //enable AWG pattern for ACM200_0
STSEnableMeas(&ACM200_0, &ACM200_1); //enable measurement for ACM200_0 and ACM200_1
STSAWGRun(); //Enable AWG and measurement synchronously
```

6.3.15 SetMeasITrig()

```
int SetMeasITrig(
double iTrig,
TRIG_MODE trigMode,
double hysteresisValue,
double iTrig2,
```

```
TRIG_MODE trigMode2,  
double hysteresisValue2);
```

Parameters

iTrig

double 型变量，设置第一次触发电流。单位 A。

注意：

- 1) 该参数的值与其对应通道所调用的 Set() 函数的电压档位相关；
- 2) 系统在 iTrig 电压处最多可以触发两次。

trigMode

第一次触发模式设置，可设置为：

TRIG_FALLING：下降沿触发

TRIG_RISING：上升沿触发

当使用单次触发功能时，该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue

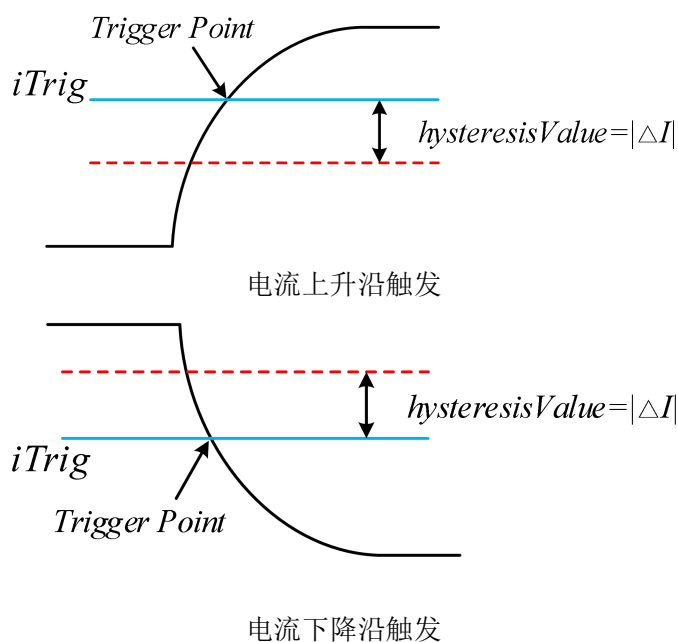
double 型变量，设置第一次触发电流的滞回值。单位 A。

当使用单次触发功能时，该参数可以不写，在默认条件下等于 0。

注意：

当使用单次触发功能时：

- 1) 滞回参数 hysteresisValue 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (iTrig-hysteresisValue) 值之后又大于 iTrig 值时，此时在 iTrig 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 (iTrig+hysteresisValue) 值之后又小于 iTrig 值时，此时在 iTrig 处判定为触发。



iTrig2

double 型变量，设置第二次触发电流。单位 A。

注意：

- 1) 该参数的值与其对应通道所调用的 Set() 函数的电压档位相关；
- 2) 系统在 iTrig2 电压处最多可以触发两次。

trigMode2

第二次触发模式设置，可设置为：

TRIG_FALLING 下降沿触发
TRIG_RISING 上升沿触发

hysteresisValue2

double 型变量，设置第二次触发电流的滞回值。单位 A。

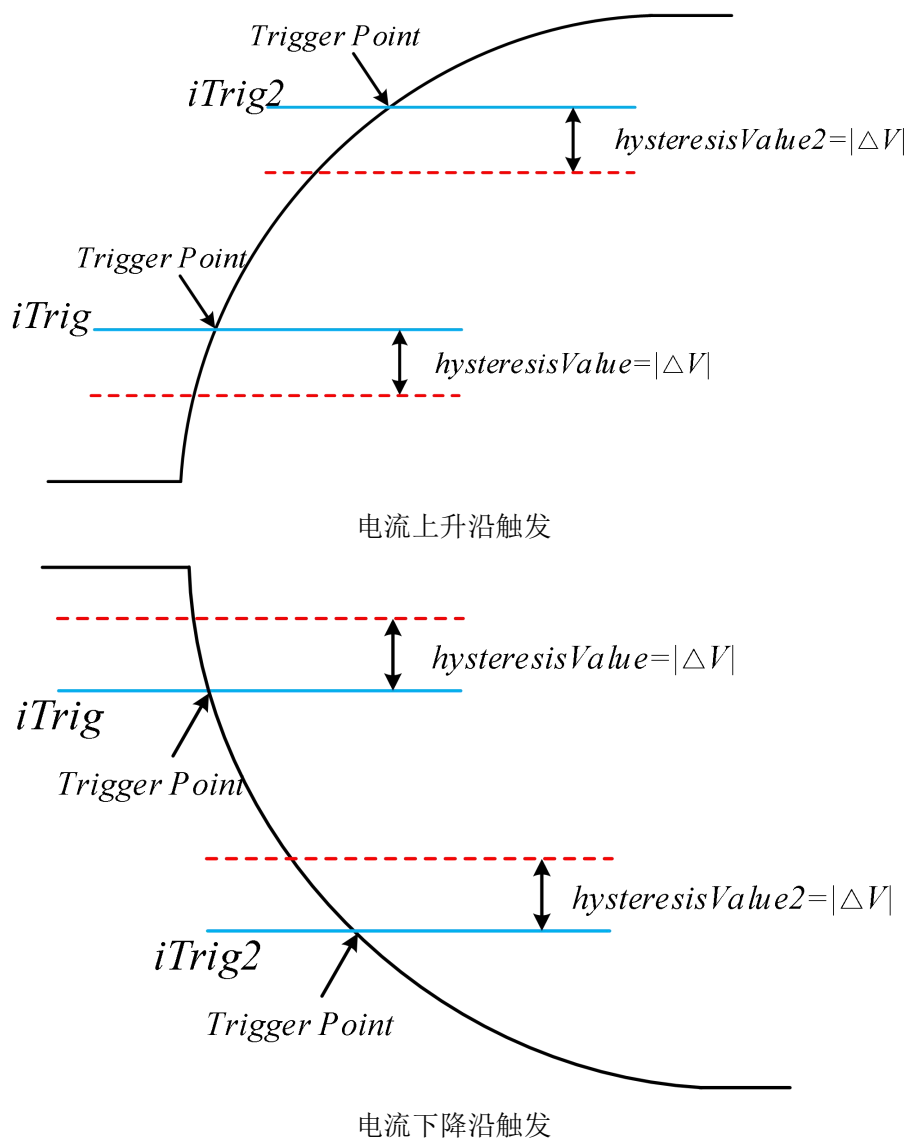
注意：

当使用单次触发功能时：

- 1) 滞回参数 hysteresisValue (hysteresisValue2) 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 iTrig-hysteresisValue (iTrig2-hysteresisValue2) 值之后又大于 iTrig (iTrig2) 值时，

此时在 $iTrig2$ ($iTrig2$) 处判定为触发；

3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 $iTrig+hysteresisValue$ ($iTrig2+hysteresisValue2$) 值之后又小于 $iTrig$ ($iTrig2$) 值时，此时在 $iTrig$ ($iTrig2$) 处判定为触发。



Return Values

返回 0: 设置成功；

返回-1: 该函数不支持 Group 功能。

Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，ACM200_0 从 16.5V 上升到 21.5V，步距 0.1V。ACM200_0 设置第一次上升沿触发，触发电流值 200uA，设置第二次下降沿触发，触发电流为 500uA。

```
double Trig1 = 200e-6;           //the first trigger current level
double Trig1_Point[4] = { 0.0 };
double Trig2 = 500e-6;           //the second trigger current level
double Trig2_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                    // data size of AWG pattern
int interval = 200;              // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into ACM200_0 channel, named it as 'Vst'. Set to FV mode.
ACM200_0.AwgLoader("Vst", FV, ACM200_40V, ACM200_100MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its
// 'AwgLoader()'
ACM200_0.Set(FV, 16.5, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
ACM200_0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);
```

设置触发电平的正确做法是：

```
ACM200_0.MeasureVI(sam, interval, MEAS_AWG);
ACM200_0.SetMeasITrig(Trig1, TRIG_RISING, Trig2, TRIG_FALLING); // the first trigger value is
200uA, rising edge the second trigger value is 500uA, falling edge

STSEnableAWG(&ACM200_0); // enable AWG pattern for ACM200_0
STSEnableMeas(&ACM200_0); // enable measurement for ACM200_0
STSAWGRun(); // Enable AWG and measurement synchronously
```

6.3.16 ContactCheck()

```
int ContactCheck(ACM200_CONTACTMODE checkMode);
```

Parameters

checkMode

接触测试的模式，可设置为：

ACM200_HIGH_SIDE: 检测高端电路的连接

ACM200_LOW_SIDE: 检测低端电路的连接

Return Values

返回 0：函数运行正常；

Remarks

设置高低端接触检测的模式。

注意：

- 1) 通过该函数对高低端电路进行接触检测，检测结束后，VI 源将恢复至检测之前的状态；
- 2) 该函数支持 Group 操作；
- 3) 同一个 bank 的通道不能同时进行 LOW 端接触检查。

Example

```
ACM200_0.ContactCheck(ACM200_HIGH_SIDE); // set the contact check mode to high side
```

6.3.17 GetContactCheckResult()

```
int GetContactCheckResult(  
    UINT siteCount,  
    double &highRes,  
    double &lowRes);
```

Parameters

siteCount

UINT 型变量，选择读取接触检测结果的工位。0 代表 Site1，1 代表 Site2.....

highRes

double 型变量，输出高端检测的电阻值。

若回读失败则返回如下值：

2.00e+15：回读失败，工位无效；

4.00e+15：回读失败，该函数不支持 Group 功能。

lowRes

double 型变量，输出低端检测的电阻值。

若回读失败则返回如下值：

2.00e+15：回读失败，工位无效；

4.00e+15：回读失败，该函数不支持 Group 功能。

Return Values

返回 2：该工位接触检测低端 Fail。

返回 1：该工位接触检测高端 Fail；

返回 0：该工位接触检测 Pass；

返回-1：该函数不支持 Group 功能；

返回-2：工位无效；

Remarks

读取该工位接触检测的结果。

Example

```
double highRes = 0;  
double lowRes = 0;
```

```
int result = ACM200_0.GetContactCheckResult(0, highRes, lowRes); // Get contact check result of
SITE_1
```

6.3.18 SetFLOff()

```
int SetFLOff();
```

Return Values

返回 0：设置成功；

Remarks

断开当前通道所在 bank 的 ForceLow 继电器。

Example

```
ACM200_0.SetFLOff();
```

6.3.19 GangRelay()

```
int GangRelay(
ACM200Parallel_CH0);
```

```
int GangRelay(
ACM200Parallel_CH0,
ACM200Parallel_CH1);
```

```
int GangRelay(
ACM200Parallel_CH0,
ACM200Parallel_CH1,
ACM200Parallel_CH2);
```

```
int GangRelay(
```



```
ACM200Parallel_CH0,  
ACM200Parallel_CH1,  
ACM200Parallel_CH2,  
ACM200Parallel_CH3);
```

```
int GangRelay(  
ACM200Parallel_CH0,  
ACM200Parallel_CH1,  
ACM200Parallel_CH2,  
ACM200Parallel_CH3,  
ACM200Parallel_CH4);
```

Parameters

Parallel_CHx

ACM200 类型变量，并联通道 x，将该通道并联到主通道上

Return Values

返回 0：设置成功；

返回-1：该函数不支持 Group 功能；

返回-13：主通道与并联通道的属性不同；

返回-14：主通道与并联通道的工位个数不相同；

返回-15：相同工位的并联通道不属于同一 Bank；

返回-16：不同工位中的并联通道属于同一 Bank；

返回-17：所有并联通道中存在相同的通道；

返回-18：主通道与一个并联通道相同。

Remarks

设置通道并联，将选定的通道并联到一起。

注意：

- 1) 该函数不支持 group；
- 2) 并联通道属性必须都是多工位或都是 NOSITE；

- 3) 相同工位下，所有并联通道必须在同一 bank;
- 4) 不同工位下，所有通道必须在不同 bank;
- 5) 对于并联通道，使用该函数后，若未进行复位操作，则 Set 函数设置无效;
- 6) 该函数设置后，所有并联通道状态为 FI 为 0，电压档位 20V，电流档位 100UA。

Example

例 1：设置 ACM200_1 和 ACM200_2 通道并联到 ACM200_0 通道上。复位后，方可进行 Set 操作。

正确示例如下：

```
ACM200 ACM200_0("S18_0,S18_6", "ACM200_0");
ACM200 ACM200_1("S18_1,S18_7", "ACM200_1");
ACM200 ACM200_2("S18_2,S18_8", "ACM200_2");
ACM200 ACM200_3("S18_3,S18_9", "ACM200_3");
ACM200 ACM200_4("S18_4,S18_10", "ACM200_4");
ACM200 ACM200_5("S18_5,S18_11", "ACM200_5");

STSSetMultiSiteBind(MD_ACM200,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");
STSSetMultiSiteBind(MD_ACM200,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");

ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangReset();

ACM200_0.Set(FV, 0, ACM200_10V, ACM200_1MA, ACM200_RELAY_OFF);
```

错误示例 1：

对应注意事项 2)，ACM200_0 和 ACM200_1 的通道属性为多工位，ACM200_2 属性为 NOSITE。

```
ACM200 ACM200_0("S18_0", "ACM200_0");
ACM200 ACM200_1("S18_1", "ACM200_1");
ACM200 ACM200_2("S18_5", "ACM200_2");

STSSetMultiSiteBind(MD_ACM200,SITE_1,"S18_0,S18_1");
STSSetMultiSiteBind(MD_ACM200, NO_SITE, "S18_5");
ACM200_0.GangRelay(ACM200_1, ACM200_2);
```

错误示例 2：

对应注意事项 3)，S18_8 和 S18_1 不属于相同的 bank

```
ACM200 ACM200_0("S18_2,S18_8", "ACM200_0");
ACM200 ACM200_1("S18_0,S18_1", "ACM200_1");
```

```
ACM200 ACM200_2("S18_3,S18_9", "ACM200_2");

STSSetMultiSiteBind(MD_ACM200,SITE_1,"S18_0,S18_2,S18_3");
STSSetMultiSiteBind(MD_ACM200,SITE_2,"S18_1,S18_8,S18_9");

ACM200_0.GangRelay(ACM200_1, ACM200_2);
```

错误示例 3:

对应注意事项 4), S18_0 和 S18_3、S18_1 和 S18_4、S18_2 和 S18_5 属于相同的 bank

```
ACM200 ACM200_0("S18_0,S18_3", "ACM200_0");
ACM200 ACM200_1("S18_1,S18_4", "ACM200_1");
ACM200 ACM200_2("S18_2,S18_5", "ACM200_2");

STSSetMultiSiteBind(MD_ACM200,SITE_1,"S18_0,S18_1,S18_2");
STSSetMultiSiteBind(MD_ACM200,SITE_2,"S18_3,S18_4,S18_5");

ACM200_0.GangRelay(ACM200_1, ACM200_2);
```

6.3.20 GangSet()

```
int GangSet(
double setValue,
double risingTime = 0.1);
```

Parameters

setValue

double 型变量，设置恒流值。

单位为 A。最大范围为 $-0.2*N \sim 0.2*N$ 。

其中：N 为并联的通道数

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.1mS；最大范围为 0.05~65。

1) “该参数/并联通道个数”后，其值小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；

2) “该参数/并联通道个数”后，其值小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；

3) “该参数/并联通道个数”后，其值大于 0.1mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Return Values

返回 0: 设置成功;

返回-1: 该函数不支持 Group 功能;

返回-7: 未设置 GangRelay;

Remarks

设置通道电流并联功能，包括电流值及 Capload 时间。

注意：

- 1) 该函数不支持 group;
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用;
- 3) 该函数设置后，电流档位 200mA。

Example

例 1: 设置 ACM200_1 和 ACM200_2 通道并联到 ACM200_0 通道上, 输出电流为 0.6A, risingTime 默认为 0.1mS。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);  
ACM200_0.GangSet(0.6);  
ACM200_0.GangReset();
```

6.3.21 GangSetSyn()

```
int GangSetSyn(  
    const double* setValue,  
    UINT siteSize,  
    double risingTime = 0.1);
```

Parameters

setValue

double 型变量，设置恒流值。

单位为 A。最大范围为 $-0.2*N \sim 0.2*N$ 。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.1mS；最大范围为 0.05~65。

- 1) “该参数/并联通道个数”后，其值小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) “该参数/并联通道个数”后，其值小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) “该参数/并联通道个数”后，其值大于 0.1mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Return Values

- 返回 0：设置成功；
- 返回-1：该函数不支持 Group 功能；
- 返回-7：未设置 GangRelay；
- 返回-19：数组大小无效。

Remarks

设置通道电流并联功能，对不同的工位设定不同的电流值，且该操作对不同工位完全同步。

注意：

- 1) 该函数不支持 group;
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用;
- 3) 该函数设置，电流档位 200mA。

Example

例 1：设置 ACM200_1 和 ACM200_2 通道并联到 ACM200_0 通道上，SITE1 设置其输出电流为 0.6A，SITE2 设置其输出 0.5A，risingTime 默认为 0.1mS。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);  
double setValue[2] = { 0.6, 0.5 };  
int siteSize = 2;  
ACM200_0.GangSetSyn(setValue, siteSize)  
ACM200_0.GangReset();
```

6.3.22 GangMeasure()

```
int GangMeasure(  
    UINT sampleTimes,  
    double samplePeriod);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096

注意：

当 sampleTimes < 1 时，sampleTimes = 1;

当 sampleTimes > 4096 时，sampleTimes = 4096。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 1uS，分辨率 1uS。范围：1~30000。

注意：

当 samplePeriod < 1 时，samplePeriod = 1；

当 samplePeriod > 30000 时，samplePeriod = 30000。

Return Values

返回 0：设置成功；

返回-1：该函数不支持 Group 功能；

返回-7：未设置 GangRelay。

Remarks

设置 ACM200 并联模式下的测量次数、测量间隔。

注意：

1) 该函数不支持 group；

2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

例 1：设置 ACM200_0 采样 20 次，间隔 10uS。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
ACM200_0.GangMeasure(20, 10);
ACM200_0.GangReset();
```

6.3.23 GangGetMeasResult()

```
double GangGetMeasResult(
UINT siteCount,
MeasRet retType = MVRET,
int sampleNumber = AVERAGE_RESULT);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retType

选择测量读回的数据类型，可设置为

MVRET：回读电压测量数据

MIRET：回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT：平均值

MAX_RESULT：最大值

MIN_RESULT：最小值

非负值 N：返回的是第 N+1 个采样点的测量值。假设 GangMeasure 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

Return Values

正常情况下：

返回对应的测量结果（平均值、最大值、最小值、单点值），函数运行正常；

异常情况下，回读失败：

返回 2.00e+15：工位错误；

返回 4.00e+15：Group 属性的通道不能进行回读操作；

返回 9.00e+15：其他错误，回读失败。

Remarks

获取 GangMeasure()函数的测量结果。

1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；

- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

- 1) ACM200 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可；
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

例 1：返回 ACM200_0 电流测量结果平均值。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
delay_ms(1);
// ACM200_0 sampleTimes=20, samplePeriod=10uS
ACM200_0.GangMeasure(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average
    value of current
    adresult[i] = ACM200_0.GangGetMeasResult(i, MIRET);
}
ACM200_0.GangReset();
```

例 2：返回 ACM200_0 电压测量结果第 15 个点的值。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
delay_ms(1);
//ACM200_0sampleTimes=20, samplePeriod=10uS
ACM200_0.GangMeasure(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=ACM200_0.GangGetMeasResult (i, MVRET, 14);
}
ACM200_0.GangReset();
```

6.3.24 GangBlockRead()

```
int GangBlockRead(
UINT siteCount,
UINT start,
UINT size,
```

```
double* buffer,  
MeasRetretType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。

size

UINT 型变量，数据块的大小，正整数。

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0}。

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET：读回电压测量数据

MIRET：读回电流测量数据

Return Values

- 返回 0：回读成功；
- 返回-1：该函数不支持 Group 功能；
- 返回-2：工位无效；
- 返回-7：设置失败，未设置 GangRelay；
- 返回-10：回读失败，测量未启动。

Remarks

此函数读取并联测量时一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

```
double result[SITENUM][200] = { 0.0 };
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
delay_ms(1);
ACM200_0.GangMeasure(200, 10);
for (i = 0; i < SITENUM; i++)
{
    // start=0; size=200; Get MVRET, saved to result.
    ACM200_0.GangBlockRead(i, 0, 200, result[i], MVRET);
}
ACM200_0.GangReset();
```

6.3.25 GangReset()

```
int GangReset();
```

Return Values

返回 0：设置成功；

返回-7：未设置 GangRelay。

Remarks

并联通道复位功能，将并联通道复位，并联模式解除。

恢复至恒压为 0，电压档位为 40V，电流档位为 1mA，继电器 RelayOff。

注意：

在进行其它非并联模式操作之前，必须先使用该函数解除并联模式。

Example

例 1：将 ACM200_0、ACM200_1 和 ACM200_2 的并联模式解除。

```
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
ACM200_0.GangMeasure(20, 10);
ACM200_0.GangReset();
```

6.3.26 TSet()

```
int TSet(
VIMode viMode,
double setValue,
ACM200_VRNG vRange,
ACM200_IRNG iRange,
ACM200_OUT_RELAY relayStatus = ACM200_RELAY_HOLD,
double risingTime = 0.1);
```

Parameters

viMode

ACM200 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.2~0.2。

vRange

ACM200 电压量程。

ACM200_40V

ACM200_20V

ACM200_10V

ACM200_3p6V

iRange

ACM200 电流量程。

ACM200_200MA

ACM200_100MA

ACM200_10MA

ACM200_1MA

ACM200_100UA

ACM200_10UA

ACM200_1UA

relayStatus

设置输出继电器如何动作，分为四种情况：

ACM200_RELAY_HOLD：输出继电器保持上一个状态；

ACM200_RELAY_ON：打开输出继电器；

ACM200_RELAY_OFF：关闭输出继电器；

ACM200_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 ACM200 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.1mS；

- 1) 该参数设置小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) 该参数设置大于 0.1mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Return Values

返回 0：设置成功；

Remarks

以 Turbo 模式设置 ACM200 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间，配合 STSTArming 函数保证状态设置完成。

注意：

1)该函数与 Set 函数的区别是：

a. Set 函数执行结束时，对应通道已达到目标状态，而 TSet 函数只是设置了对应通道的目标状态，需要配合 STSTArming 函数等待状态设置完成；

2)不同通道间设置状态时可依次使用 TSet 函数，增加一条 STSTArming 函数等待所有状态设置完成，可达到节省测试时间的目的；

Example

例 1：Turbo 模式下，设置 ACM200_0 在恒压模式下，使用 10V/100MA 档位，输出 4V 电压，risingTime 使用默认值；设置 ACM200_1 在恒压模式下，使用 10V/10MA 档位，输出 1.5V 电压，risingTime 使用默认值。

```
ACM200_0.TSet(FV, 4, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
ACM200_1.TSet(FV, 1.5, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);
STSTArming();
```

例 2：ACM200_0 在恒压模式下，使用 10V/100MA 档位，输出 4V 电压，risingTime 使用默认值；ACM200_1 在恒压模式下，使用 10V/10MA 档位，输出 1.5V 电压，risingTime 使用默认值；对这两个通道进行 Group 操作（gpACM200），目标状态设置为恒压为 0，10V/1MA 档位，RelayOFF。

```
ACM200_0.Set(FV, 4, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
ACM200_1.Set(FV, 1.5, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);

// Turbomodel
gpACM200.TSet(FV, 0, ACM200_10V, ACM200_1MA, ACM200_RELAY_OFF);
STSTArming();
```

6.3.27 SetAlarmMask()

```
void SetAlarmMask(  
UINT siteCount,  
bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true：屏蔽该工位的 Alarm 信息。

false：输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
ACM200_0.SetAlarmMask(0);           // mask the Alarm information of SITE_1  
// The alarm information about overranging voltage scale is not output  
ACM200_0.Set(FV, 5, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);
```

6.3.28 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息;

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = ACM200_0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.3.29 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：板卡信息回读成功；

返回-1：该函数不支持 Group 功能；

返回-2：工位无效。

返回-21：回读失败，数据长度小于 1。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
ACM200_0.GetBoardSN(0, boardSN, 255);    // Get the board serial number
```

6.3.30 获取校准信息函数

6.3.30.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINTsiteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = ACM200_0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.3.30.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = ACM200_0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.3.30.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = ACM200_0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.3.30.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS：校准 PASS；

CAL_FAIL：校准 FAIL；

STS_NO_RESULT_RECORD：读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = ACM200_0.GetCalibrationResult(0);           // Get calibration result of SITE_1
```

6.3.30.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = ACM200_0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.3.30.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000: Keithley2000

STS_AGILENT34401: Agilent34401

STS_AGILENT3458A: Agilent3458A

STS_NO_METER_RECORD: 读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0}; // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = ACM200_0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.3.30.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteCount,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* boardHdRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回 “N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 *boardHdRev* 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = ACM200_0.GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.3.30.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev (
    UINT siteCount,
    char* softRev,
    UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回“N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0}; // the buffer for saving softwareversion
// Get the version of calibration software
ACM200_0.GetCalibrationSoftRev(0, softRev, 255);
```

6.3.30.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = ACM200_0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.3.31 VI 源编程范例

6.3.31.1 设置一组波形数据组的拼接

测试条件:

- 第一组: 设置一个正弦波数据组, 首地址从 0 开始, 数据长度为 100;
- 第二组: 设置一个三角波数据组, 首地址从 100 开始, 数据长度为 100;
- 第三组: 设置一个方波数据组, 首地址从 200 开始, 数据长度为 100;
- 第四组: 设置一个斜坡数据组, 首地址从 300 开始, 数据长度为 100。

```
int sam = 400;//AWG waveform data length
int interval = 200;//AWGdata interval time, unit is uS
double awg_pattern[1000] = { 0.0 };

STSAWGCreateSineData(&awg_pattern[0], 100, 1, -5, 0, 180);
// Set a sinewave data array, the start address starts from 0, data size is 100.

STSAWGCreateTriangleData(&awg_pattern[100], 100, 4, -5, 0, 0);
//Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200], 100, 4, -5, 0, 50);
//Set a square wave data array, the start address starts from 200, data size is 100.

STSAWGCreateRampData(&awg_pattern[300], 100, 1, 1, 5);
//Set an ramp data array, the start address starts from 300, data size is 100.

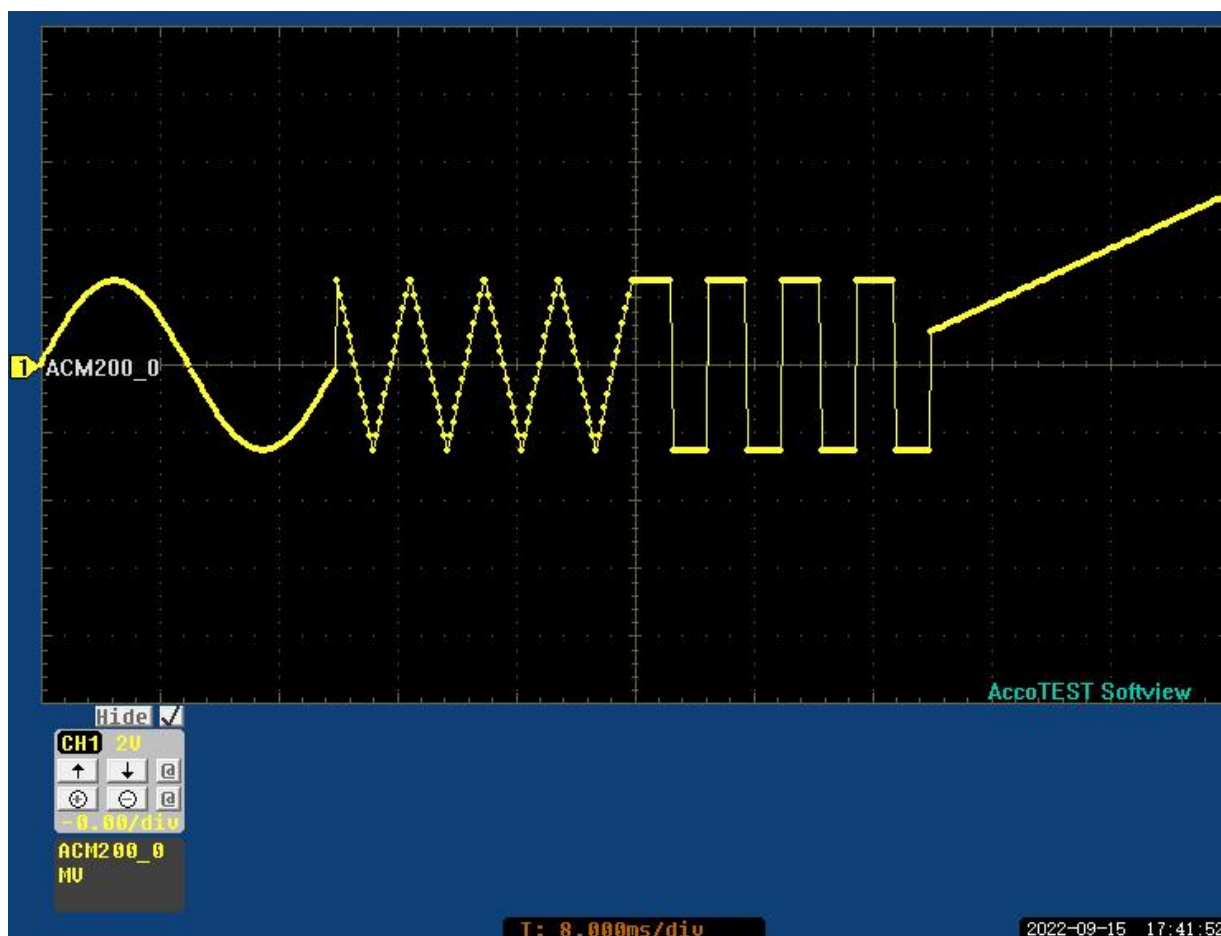
ACM200_0.AwgLoader("Vcc_awg", FV, ACM200_10V, ACM200_10MA, awg_pattern, sam);
//import the awg_pattern array into ACM200_0 channel, named it as 'Vcc_awg'. Set to FV mode.

ACM200_0.Set(FV, 0, ACM200_10V, ACM200_10MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

ACM200_0.AwgSelect("Vcc_awg", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

ACM200_0.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&ACM200_0);//enable AWG pattern for ACM200_0
STSEnableMeas(&ACM200_0);//enable measurement for ACM200_0
STSAWGRun();//Enable AWG and measurement synchronously
```

组合之后的波形如下:



6.3.31.2 生成正弦波测试纹波抑制比

测试条件:

ACM200_0 连续输出 120Hz 正弦波， V_{pp} 为 0.33V，直流偏置为 3.3V。测试 ACM200_1 输出纹波抑制比。

```
int Datacount = 0;
int i = 0;
int AD_SAMPLE = 500; // sampling points
double sum[4] = { 0.0 };
double temp[4] = { 0.0 }, temp2[4] = { 0.0 };
int sam = 830; // data size of AWG pattern
int interval = 10; // interval time of AWG pattern, unit is uS
double awg_pattern[1000] = { 0.0 };
double data[5000] = { 0.0 };
double result[4] = { 0.0 };

STSAWGCreateSineData(&awg_pattern[0], sam, 1, 0.66, 3.3, 0);
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3

ACM200_0.AwgLoader("Sin_AWG", FV, ACM200_10V, ACM200_200MA, awg_pattern, sam);
```

```
// pattern named as 'Sin_AWG, FV mode

ACM200_0.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'

ACM200_0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(1);

ACM200_1.Set(FI, -100e-3, ACM200_10V, ACM200_100MA, ACM200_RELAY_ON);
//Vout FI mode, prepares to measure
delay_ms(2);
ACM200_1.MeasureVI(AD_SAMPLE, 20); //ACM200_1 measurement

for (i = 0; i < SITENUM; i++)
{
    for (Datacount = 0; Datacount < AD_SAMPLE; Datacount++)
    {
        data[Datacount] = ACM200_1.GetMeasResult(i, MVRET, Datacount);
        //ACM200_1 acquires every voltage values from 500 sampling points
        result[i] = ACM200_1.GetMeasResult(i, MVRET);
        //ACM200_1 acquires average voltage of 500 sampling points
        sum[i] += (data[Datacount] - result[i]) * (data[Datacount] - result[i]);
    }
    temp[i] = (sqrt(sum[i] / AD_SAMPLE)) * 2.828;
    //calculate peak to peak value of sine wave received from output
    temp2[i] = -20 * log10((temp[i] + 1e-12) / 0.66);
    //calculate PSRR
    sum[i] = 0;
}

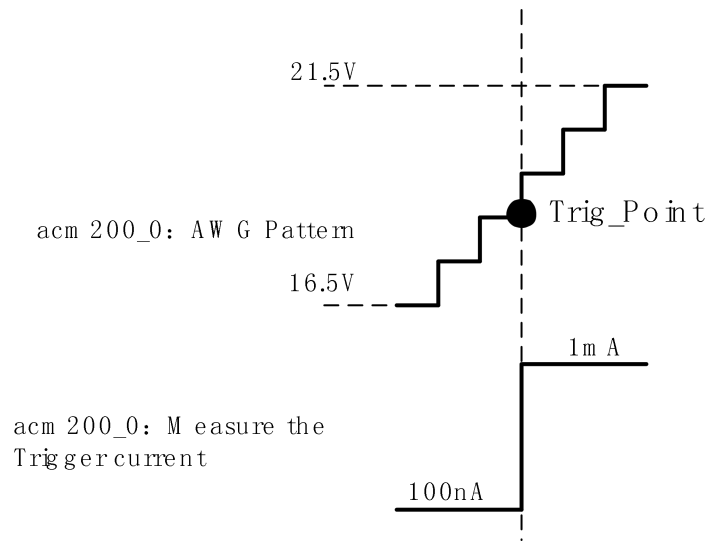
for (i = 0; i < SITENUM; i++)
{
    PSRR->SetTestResult(i, 0, temp2[i]);
}

ACM200_1.Set(FI, 0, ACM200_40V, ACM200_100MA, ACM200_RELAY_OFF);
ACM200_0.AwgStop(); //stop running AWG waveform of ACM200_0
ACM200_0.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_OFF);
```

6.3.31.3 同一路生成 AWG 且作为触发源

测试条件:

建立一个波形数据组，ACM200_0 从 16.5V 上升到 21.5V，步距 0.1V。测试 ACM200_0 电流大于 200uA 时的跳变点的电压值。



```

int i = 0;
double Trig = 200e-6; // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50; // data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
// Set an ramp data array, the start address starts from 0, data size is 50.

ACM200_0.AwgLoader("Vst", FV, ACM200_40V, ACM200_100MA, awg_pattern, sam);
// import the awg_pattern array into ACM200_0 channel, named it as 'Vst'. Set to FV mode.

ACM200_0.Set(FV, 16.5, ACM200_40V, ACM200_100MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its
// 'AwgLoader()'

ACM200_0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);
// Select pattern of 'Vst', start step=0, stop=sam-1; loop=sam-1; interval time=200uS

ACM200_0.MeasureVI(sam, interval, MEAS_AWG);

ACM200_0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge

STSEnableAWG(&ACM200_0); // enable AWG pattern for ACM200_0
STSEnableMeas(&ACM200_0); // enable measurement for ACM200_0
STSAWGRun(); // Enable AWG and measurement synchronously

for (i = 0; i < SITENUM; i++)
{
    Trig_Point[i] = ACM200_0.GetMeasResult(i, MIRET, TRIG_RESULT);
    // Get the position of trigger point

    result[i] = ACM200_0.GetMeasResult(i, MVRET, Trig_Point[i]);
    // Read the voltage value on trigger position

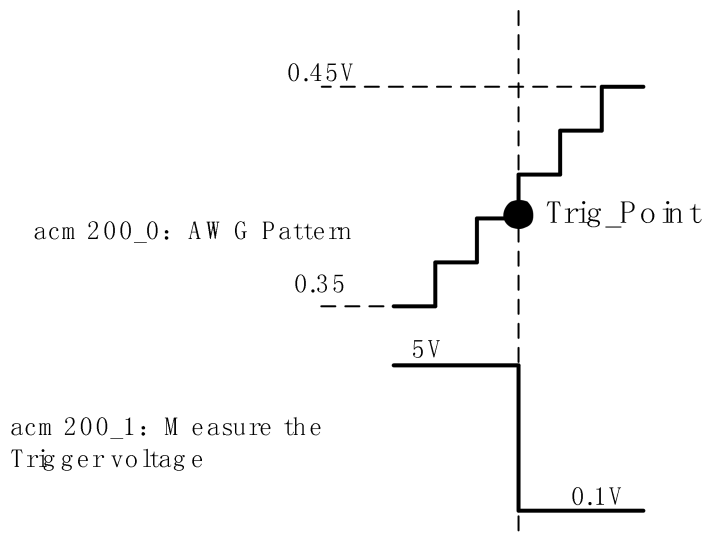
    Vst->SetTestResult(i, 0, result[i]);
}

```

6.3.31.4 一路源生成 AWG 另一路作为触发源

测试条件:

建立一个波形数据组，ACM200_0 从 0.35V 上升 0.45V，步距 0.001V。测试 ACM200_1 端的电压小于 2V 时，ACM200_0 跳变点的电压值。



```
int i = 0;
double Trig = 2; //trigger voltage
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

ACM200_1.Set(FI, 0, ACM200_10V, ACM200_1MA, ACM200_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

ACM200_0.AwgLoader("Vcspre_AWG", FV, ACM200_3p6V, ACM200_100MA, awg_pattern, sam);
//import the awg_pattern array into ACM200_0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

ACM200_0.Set(FV, 0.30, ACM200_3p6V, ACM200_100MA, ACM200_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

ACM200_0.AwgSelect("Vcspre_AWG", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

ACM200_1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, Falling edge

ACM200_0.MeasureVI(sam, interval, MEAS_AWG);
ACM200_1.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&ACM200_0); //enable AWG pattern for ACM200_0
```

```
STSEnableMeas(&ACM200_0, &ACM200_1);//enable measurement for ACM200_0 and ACM200_1
STSAWGRun();//Enable AWG and measurement synchronously

for (i = 0; i < SITENUM; i++)
{
    Trig_Point[i] = ACM200_1.GetMeasResult(i, MVRET, TRIG_RESULT);
    //Get the position of trigger point for ACM200_1
    result[i] = ACM200_0.GetMeasResult(i, MVRET, Trig_Point[i]);
    //Read the voltage value of ACM200_0 on trigger position
    Vcspre->SetTestResult(i, 0, result[i]);
}
```

6.3.31.5 多路并联输出电流

测试条件:

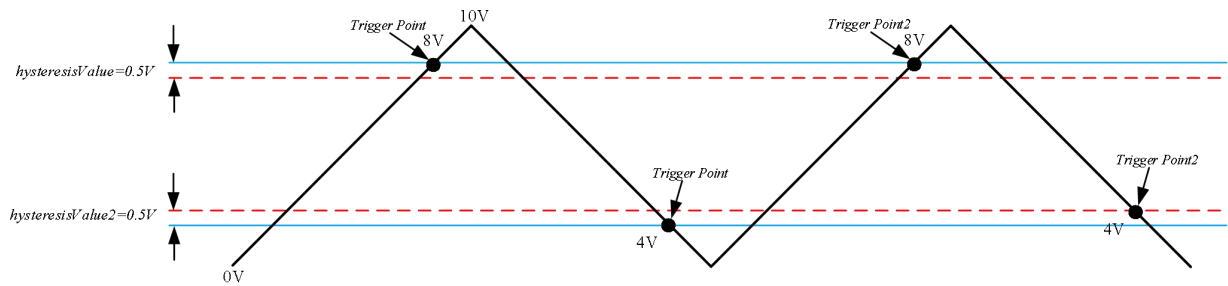
将 ACM200_0 和 ACM200_1、ACM200_2 并联，输出 600mA 电流，ACM200_0 通道外接 10ohm，并进行测压测流

```
int i = 0;
double adresultV[SITENUM]={ 0 };
double adresultI[SITENUM]={ 0 };
ACM200_0.GangRelay(ACM200_1, ACM200_2);
ACM200_0.GangSet(0.6);
delay_ms(1); // ACM200_0 sampleTimes=20, samplePeriod=10uS ACM200_0.GangMeasure(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value
    of current
    adresultV[i] = ACM200_0.GangGetMeasResult(i, MVRET);
    adresultI[i] = ACM200_0.GangGetMeasResult(i, MIRET);
}
ACM200_0.GangSet(0);
delay_ms(1);
ACM200_0.GangReset();
```

6.3.31.6 多点触发

测试条件:

ACM200_0 生成 AWG 波形，ACM200_1 作为触发源，实现多点触发



```
double Trig[] = { 0.0 };
double Trig_Point[4] = { 0.0 };
double result[255] = { 0.0 };
double resultA[255] = { 0.0 };
double resultB[255] = { 0.0 };
int sam = 2000; // AWG 波形长度
int interval = 10; // 采样间隔 uS
double awg_pattern[4000] = { 0.0 };
// 从 0-10, sam 长度, 斜波函数
acm200_0.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_ON);
acm200_1.Set(FI, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_SENSE_ON);
delay_ms(100);
STSAWGCreateRampData(&awg_pattern[0], 500, 1, 0, 10); // 斜波
STSAWGCreateRampData(&awg_pattern[500], 500, 1, 10, 0); // 斜波
STSAWGCreateRampData(&awg_pattern[1000], 500, 1, 0, 10); // 斜波
STSAWGCreateRampData(&awg_pattern[1500], 500, 1, 10, 0); // 斜波
// 数组导入, awg_pattern 数组, sam 波形大小
acm200_0.AwgLoader("Vst", FV, ACM200_10V, ACM200_200MA, awg_pattern, sam);
// acm200_0.Set(FI, 0.2, ACM200_40V, ACM200_200MA, ACM200_RELAY_ON);
delay_ms(1);
acm200_0.AwgSelect("Vst", 0, sam - 1, 0, interval);

acm200_1.SetMeasVTrig(8, TRIG_RISING, 0.5, 4, TRIG_FALLING, 0.5);

acm200_1.MeasureVI(sam, interval, MEAS_AWG); // 测量通道
acm200_0.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&acm200_0, &acm200_1); // AWG 模式启用
STSEnableMeas(&acm200_0, &acm200_1); // 测量
STSAWGRun(); // 启用
// STSAWGRunTriggerStop(&acm200_1, &acm200_1, &acm200_0);
for (BYTE i = 0; i < SITE; i++)
{
    // BEGIN_SINGLE_SITE(i)
    Trig_Point[i] = acm200_1.GetMeasResult(i, MVRET, TRIG_RESULT, 1);
    result[i] = acm200_0.GetMeasResult(i, MVRET, Trig_Point[i]);
    resultA[i] = acm200_0.GetMeasResult(i, MVRET, (Trig_Point[i] - 1));
    resultB[i] = acm200_0.GetMeasResult(i, MVRET, (Trig_Point[i] + 1));
    AWG_ponint1->SetTestResult(i, 0, result[i]);
    AWG_ponint1->SetTestResult(i, 1, resultA[i]);
    AWG_ponint1->SetTestResult(i, 2, resultB[i]);
    Trig_Point[i] = acm200_1.GetMeasResult(i, MVRET, TRIG_RESULT, 3);
    result[i] = acm200_0.GetMeasResult(i, MVRET, Trig_Point[i]);
    resultA[i] = acm200_0.GetMeasResult(i, MVRET, (Trig_Point[i] - 1));
    resultB[i] = acm200_0.GetMeasResult(i, MVRET, (Trig_Point[i] + 1));
    AWG_ponint2->SetTestResult(i, 0, result[i]);
    AWG_ponint2->SetTestResult(i, 1, resultA[i]);
}
```



```
    AWG_ponint2->SetTestResult(i, 2, resultB[i] );  
  
    //END_SINGLE_SITE()  
}  
delay_ms(500);  
acm200_0.Set(FI, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_OFF);  
acm200_1.Set(FV, 0, ACM200_10V, ACM200_200MA, ACM200_RELAY_OFF);  
return 0;
```

6.4 FPVle

6.4.1 FPVle()

```
FPVle(char*channelList, char*chName = NULL);
```

Parameters

channelList

FPVle 绑定通道字符串。形如“S4_0, S4_1”，其中“S4”表示槽位号 4（S 即 Slot），“0、1”表示板内序号。

注意：

FPVle 一块板卡最多 2 个通道，因此板内序号最大值为 1。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 FPVle_xx。

Remarks

定义一路 FPVle 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 4 块 FPVle（分别位于 Slot4、Slot12、Slot21 和 Slot29），四工位并行工作，定义可使用的 FPVle。

```
FPVle fpvie0("S4_0,S12_0,S21_0,S29_0");  
FPVle fpvie1("S4_1,S12_1,S21_1,S29_1");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_FPVle, SITE_1, "S4_0,S4_1"); // set Slot4 ch0 ch1 to SITE1  
STSSetMultiSiteBind(MD_FPVle, SITE_2, "S12_0,S12_1"); // set Slot12 ch0 ch1 to SITE2  
STSSetMultiSiteBind(MD_FPVle, SITE_3, "S21_0,S21_1"); // set Slot21 ch0 ch1 to SITE3  
STSSetMultiSiteBind(MD_FPVle, SITE_4, "S29_0,S29_1"); // set Slot29 ch0 ch1 to SITE4
```

6.4.2Set()

```
int Set(  
VIMode viMode,  
double setValue,  
FPVie_VRNG vRange,  
FPVie_IRNG iRange,  
FPVie_OUT_RELAY relayStatus = FPVie_RELAY_HOLD,  
double risingTime = 0.2);
```

Parameters

viMode

FPVie 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-100~100。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-10~10。

vRange

FPVie 电压量程。

FPVie_100V

FPVie_40V

FPVie_20V

FPVie_10V

FPVie_5V

FPVie_2V

FPVie_1V

FPVle_100MV

注意：

FPVle_100MV 电压档位仅能用于测量。

iRange

FPVle 电流量程。

FPVle_10A

FPVle_2A

FPVle_1A

FPVle_100MA

FPVle_10MA

FPVle_1MA

FPVle_100UA

FPVle_10UA

relayStatus

设置输出继电器如何动作，分为四种情况：

FPVle_RELAY_ON：打开输出继电器；

FPVle_RELAY_OFF：关闭输出继电器；

FPVle_RELAY_HOLD：输出继电器保持上一个状态；

FPVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FPVle 处于独立电压表模式。

risingTime

double 型变量，设置积分时间。单位为 mS，分辨率为 0.1mS(设定值大于 0.2mS 时)。该参数不写，则默认为 0.2mS；

- 1) 该参数设置小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Slow 模式；

4) 该参数设置大于 0.2mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

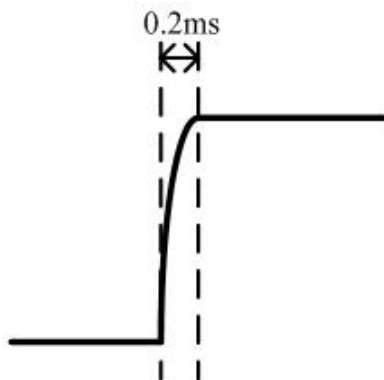
Remarks

设置 FPVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

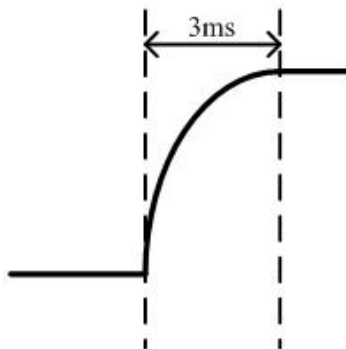
例 1：设置 fpvie0 电压输出 5V，risingTime 不填则默认为 0.2mS。

```
fpvie0.Set(FV,5,FPVle_10V,FPVle_100MA,FPVle_RELAY_ON);
```



例 2：设置 fpvie0 电压输出 5V，risingTime 设置为 3mS。

```
fpvie0.Set(FV,0,FPVle_10V,FPVle_100MA,FPVle_RELAY_ON);  
delay_ms(1);  
fpvie0.Set(FV,5,FPVle_10V,FPVle_100MA,FPVle_RELAY_ON, 3);
```



6.4.3SetSyn()

```
int SetSyn(  
VIMode viMode,  
const double* setValue,  
UINT siteSize,  
FPVie_VRNG vRange,  
FPVie_IRNG iRange,  
FPVie_OUT_RELAY relayStatus = FPVie_RELAY_HOLD,  
double risingTime = 0.2);
```

Parameters

viMode

FPVie 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-100~100。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-10~10。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

FPVie 电压量程。

FPVie_100V

FPVie_40V

FPVie_20V

FPVie_10V

FPVie_5V

FPVie_2V

FPVie_1V

FPVie_100MV

注意：

FPVie_100MV 电压档位仅能用于测量。

iRange

FPVie 电流量程。

FPVie_10A

FPVie_2A

FPVie_1A

FPVie_100MA

FPVie_10MA

FPVie_1MA

FPVie_100UA

FPVie_10UA

enableRelay

设置输出继电器如何动作，分为四种情况：

FPVie_RELAY_ON: 打开输出继电器

FPVie_RELAY_OFF: 关闭输出继电器

FPVie_RELAY_HOLD: 输出继电器保持上一个状态

FPVie_RELAY_SENSE_ON: 启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FPVie 处于独立电压表模式。

risingTime

double 型变量，设置积分时间。单位为 mS，分辨率为 0.1mS(设定值大于 0.2mS 时)。该参数不写，则默认为 0.2mS

- 1) 该参数设置小于等于 0.05mS 时，risingTime=0.05mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 3) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Slow 模式；
- 4) 该参数设置大于 0.2mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

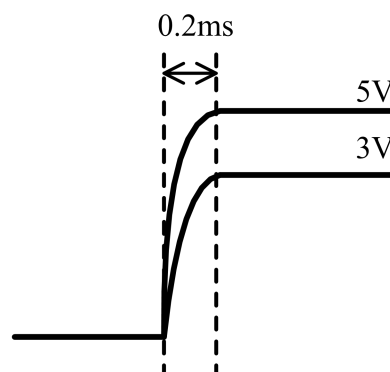
Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1：fpvie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

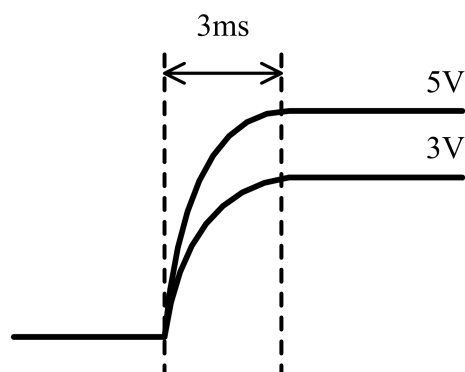
```
double setValue[2] = {3, 5};  
int siteSize = 2;  
fpvie0.SetSyn(FV, setValue, siteSize, FVPVle_10V, FVPVle_100MA, FVPVle_RELAY_ON);
```



例 2：fpvie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。


```
double setValue[2] = {3, 5};
int siteSize = 2;

fpvie0.Set(FV,0,FPVle_10V,FPVle_100MA,FPVle_RELAY_ON);
delay_ms(1);
fpvie0.SetSyn(FV, setValue, siteSize, FPVle_10V, FPVle_100MA, FPVle_RELAY_ON, 3);
```



6.4.4 SetClamp()

```
int SetClamp(
double percent_PFS,
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：10~102。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：10~102。

Return Values

- 返回 3：正负箝位皆发生变化；
- 返回 2：负箝位变化；
- 返回 1：正箝位变化；
- 返回 0：正负箝位皆未发生变化；
- 返回-1：函数调用失败。

Remarks

设置 FPVle 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 102%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 fpvie1 为 FI=0 模式，箝位到(50, 20)，然后切换 fpvie1 为 FV=1V 模式，箝位到(25, 25)。

```
fpvie1.Set(FI, 0, FPVle_10V, FPVle_10MA, FPVle_RELAY_ON);  
//positive clamp to 50% of full range(FPVle_10V), negative clamp to 20% of full range  
fpvie1.SetClamp(50, 20);  
  
//switch from FI to FV mode, then positive/negative clamps recover back to 102% of full range  
fpvie1.Set(FV, 0, FPVle_10V, FPVle_100MA, FPVle_RELAY_ON);  
//reset positive clamp to 25% of full range(FPVle_100MA), negative clamp to 25% of full range  
fpvie1.SetClamp(25, 25);  
//Set FV=1V  
fpvie1.Set(FV, 1, FPVle_10V, FPVle_100MA, FPVle_RELAY_ON);
```

例 2：设置 fpvie1 为 FV=1V 模式，箝位到(50, 25)，然后切换 fpvie1 为 FV=35V，测量结束后设置 fpvie1 为 FV=0，箝位到(100, 100)。

```
fpvie1.Set(FV, 0, FPVle_10V, FPVle_100MA, FPVle_RELAY_ON);  
//positive clamp to 50% of full range(FPVle_100MA), negative clamp to 25% of full range  
fpvie1.SetClamp(50, 25);  
fpvie1.Set(FV, 1, FPVle_10V, FPVle_100MA, FPVle_RELAY_ON);  
delay_ms(1);  
  
//still in FV mode, clamp setting will keep unchanged  
fpvie1.Set(FV, 35, FPVle_40V, FPVle_100MA, FPVle_RELAY_ON);  
fpvie1.MeasureVI(100, 20);  
  
fpvie1.Set(FV, 0, FPVle_40V, FPVle_100MA, FPVle_RELAY_ON);  
//after FV=0, set clamps back to 100%  
fpvie1.SetClamp(100, 100);
```

6.4.5 MeasureVI()

```
int MeasureVI(  
    UINT sampleTimes,  
    double samplePeriod,  
    FPVle_MV_GAIN mvGain = FPVle_MV_X1,  
    FPVle_MI_GAIN miGain = FPVle_MI_X1,  
    FLOATMEASMODE measMode = MEAS_NORMAL,  
    UINT T1 = 0,  
    UINT T2 = 0,  
    UINT T3 = 0,  
    UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096。

注意：

当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；

当 $sampleTimes > 4096$ 时， $sampleTimes = 4096$ 。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 1uS，分辨率 1uS。范围：1~30000。

注意：

当 $samplePeriod < 1$ 时， $samplePeriod = 1$ ；

当 $samplePeriod > 30000$ 时， $samplePeriod = 30000$ 。

mvGain

FPVle 测压增益。

FPVle_MV_X1

FPVle_MV_X2

FPVle_MV_X5

FPVle_MV_X10

该参数可以不写，默认值为 FPVle_MV_X1。

miGain

FPVle 测流增益。

FPVle_MI_X1

FPVle_MI_X2

FPVle_MI_X5

FPVle_MI_X10

该参数可以不写，默认值为 FPVle_MI_X1。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数可以不写，默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 FPVle 的测量次数、测量间隔、测压增益、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1：设置 fpvie0 为普通模式测量，采样 20 次，间隔 5uS。

```
fpvie0.MeasureVI(20,5);
```

6.4.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

retType

选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N： 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

- 1)FPVie 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。
- 2)当参数 *sampleNumber* 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回值为-3，如果测量模式与回读模式不符，则函数返回值为-5。

Example

例 1：返回 fpvie0 电流测量结果平均值。

```
//fpvie0 sampleTimes=20, samplePeriod=5uS
fpvie0.MeasureVI(20, 5);
for(i=0; i<SITENUM; i++)
{
// the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value of
current
adresult[i]=fpvie0.GetMeasResult(i, MIRET);
}
```

例 2：返回 fpvie0 电压测量结果第 15 个点的值。

```
//fpvie0 sampleTimes=20, samplePeriod=5uS
fpvie0.MeasureVI(20, 5);
for(i=0; i<SITENUM; i++)
{
// the result is fifteenth voltage value
```

```
adresult[i]=fpvie0.GetMeasResult(i, MVRET, 14);  
}
```

例 3：AWG 同步测量时，返回 fpvie0 两次跳变触发的位置。

```
// fpvie1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG  
fpvie1.MeasureVI(50, 20, FPVle_MV_X1, FPVle_MI_X1, MEAS_AWG);  
// the fpvie0's parameters of MeasureVI function are same with fpvie1  
fpvie0.MeasureVI(50, 20, FPVle_MV_X1, FPVle_MI_X1, MEAS_AWG);  
// the trigger value is 2V, falling edge  
fpvie0.SetMeasVTrig(2, TRIG_FALLING);  
// fpvie1 AWG enable  
STSEnableAWG(&fpvie1);  
// fpvie0 and fpvie1 measure enable  
STSEnableMeas(&fpvie1, &fpvie0);  
STSAWGRun();  
for(i=0; i<SITENUM; i++)  
{  
    // Get the voltage position of first trigger  
    Trig_Point0[i]=fpvie0.GetMeasResult(0, MVRET, TRIG_RESULT, 0);  
    // Get the voltage position of second trigger  
    Trig_Point1[i]=fpvie0.GetMeasResult(0, MVRET, TRIG_RESULT, 1);  
}
```

6.4.7 BlockRead()

```
int BlockRead(  
    UINT siteCount,  
    UINT start,  
    UINT size,  
    double* buffer,  
    MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200]={0.0};
fpvie0.MeasureVI(200, 10);
for(i=0; i<SITENUM; i++)
{
    //start=0; size=200; Get MVRET, saved to result.
    fpvie0.BlockRead(i, 0, 200, result[i], MVRET);
}
```

6.4.8Pulse()

```
int Pulse(
double pulseValue,
UINT pulseTime = 300,
```



```
Int measStartTime = 0,  
UINT sampleTimes= 10,  
double sampleInterval= 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-100~100；

选择 FI 模式时，单位 A。最大范围为-10~10。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。最小采样间隔为 1uS，分辨率 1uS。

可以不写，默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

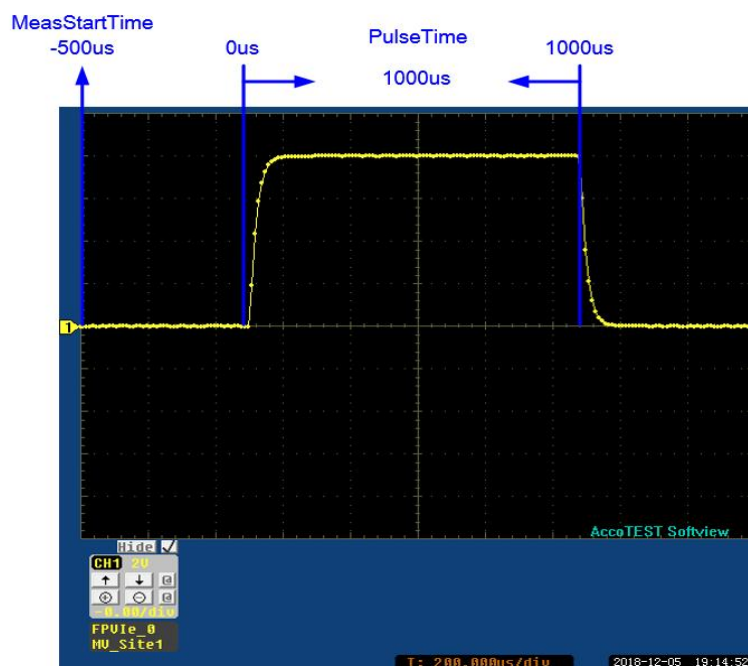
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 fpvie0 起始状态 FV 为 0。设置 fpvie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
// use set() function to set the state of fpvie0
fpvie0 .Set(FV,0,FPVie_10V,FPVie_10MA,FPVie_RELAY_ON);
fpvie0.Pulse (8,1000, -500, 200, 10);
```

使用 SoftView 软件抓取的波形如下：



6.4.9 AwgLoader()

```
int AwgLoader(
char* awgName,
VIMode viMode,
FPVie_VRNG vRange,
FPVie_IRNG iRange,
double* awgData,
```

```
UINT avgSize) ;
```

Parameters

avgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

FPVle 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

FPVle 电压量程。

FPVle_100V

FPVle_40V

FPVle_20V

FPVle_10V

FPVle_5V

FPVle_2V

FPVle_1V

FPVle_100MV

注意：

FPVle_100MV 电压档位仅能用于测量。

iRange

FPVle 电流量程。

FPVle_10A

FPVle_2A

FPVle_1A

FPVle_100MA

FPVie_10MA

FPVie_1MA

FPVie_100UA

FPVie_10UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 FPVie 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 FPVie 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 FPVie 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 FPVie 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，fpvie0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50;           // AWG waveform data length
int interval = 200;     // AWG data interval time, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
// pattern array saved in 'awg_pattern', start from 16.5 to 21.5
fpvie0.AwgLoader("Vst ", FV, FPVle_40V, FPVle_100MA, awg_pattern, sam);
//pattern named as 'Vst', FV mode
```

例 2：建立一个波形数据组，fpvie0 输出频率为 100Hz、幅值为 2V、直流偏置为 10V 的正弦波。

```
int sam = 1000;        //AWG waveform data length
int interval = 10;     //AWG data interval time, unit is uS
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0);
// set a sinewave data array of 100HZ, Vpp=4, DC Offset=10
fpvie0.AwgLoader("Sine",FV,FPVle_10V,FPVle_1MA,awg_pattern, sam);
//pattern named as 'SINE', FV mode
```

6.4.10 AwgSelect()

```
int AwgSelect(
char* awgName,
UINT startAddr,
UINT stopAddr,
int loopBackAddr,
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 FPVie 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst

波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;         // data size for AWG pattern
int sam2 = 500;          // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,16.5,21.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,16.5,10.5);

// pattern named as 'AWG_Vst', FV mode
fpvie0.AwgLoader("AWG_Vst",FV,FPVle_40V,FPVle_100MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
fpvie0.AwgLoader("AWG_Vuvlo",FV,FPVle_20V,FPVle_100MA,awg_pattern2,sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of
its 'AwgLoader()'
fpvie0.Set(FV, 16.5, FPVle_40V, FPVle_100MA, FPVle_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
fpvie0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
.....
STSAWGRun();    //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.4.11 AwgRun()

```
int AwgRun(
char*awgName,
UINTstartAddr,
UINTstopAddr,
int loopBackAddr,
double awgInterval= 10.0,
BOOL runMode = AWG_SINGLE,
UINTdelayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 **AWG_SINGLE**。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 FPVle 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRun。

Example

例 1：fpvie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。

```
int sam = 830;           //size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);

// pattern named as 'Sin_AWG, FV mode
fpvie0.AwgLoader("Sin_AWG",FV, FPVle_5V, FPVle_1A,awg_pattern,sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
fpvie0.Set(FV, 0, FPVle_5V, FPVle_1A, FPVle_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
fpvie0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
```

6.4.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 FPVie 通道的 AWG 循环运行。

Example

例 1: fpvie0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150;           //AWG waveform data length
double interval = 10;    //AWG data interval time, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0], sam, 1, 2, 3, 0);
// set waveform data array for Sin_AWG

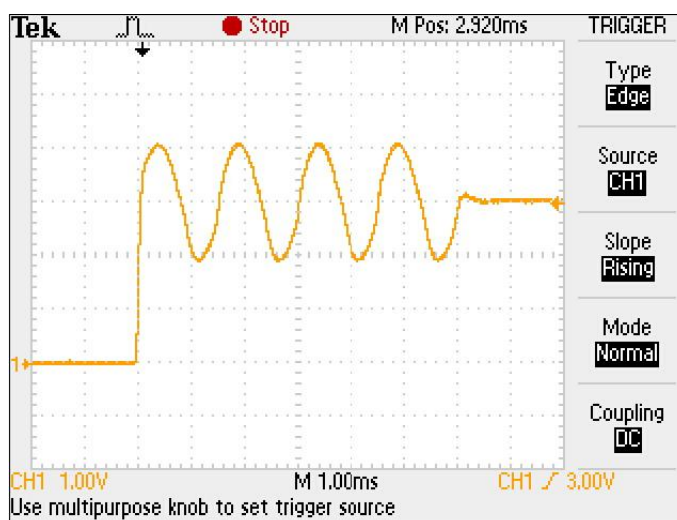
fpvie0.AwgLoader("Sin_AWG", FV, FPVie_5V, FPVie_1A, awg_pattern, sam);
// pattern named as 'Sin_AWG', FV mode

fpvie0.Set(FV, 0, FPVie_5V, FPVie_1A, FPVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'

fpvie0.AwgRun("Sin_AWG", 0, (sam-1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode

delay_ms(5);           //fpvie0 AWG pattern keeps running for 5mS
fpvie0.AwgStop();      //stop loops for fpvie0 AWG running
```

该正弦波周期为 1.5mS，延时 5mS 时，实际运行了 4 个周期的 AWG 波形。下图为示波器抓取的波形。



6.4.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 FPVle 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 fpvie0 通道的 AWG 数据。

```
fpvie0.AwgClear();           // clear AWG data of fpvie0
fpvie0.AwgLoader("P1",FV,FPVle_40V,FPVle_100MA,awg_pattern1,sam);
```

6.4.14 SetMeasVTrig()

```
int SetMeasVTrig(
double vTrig,
TRIG_MODE trigMode = TRIG_FALLING,
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

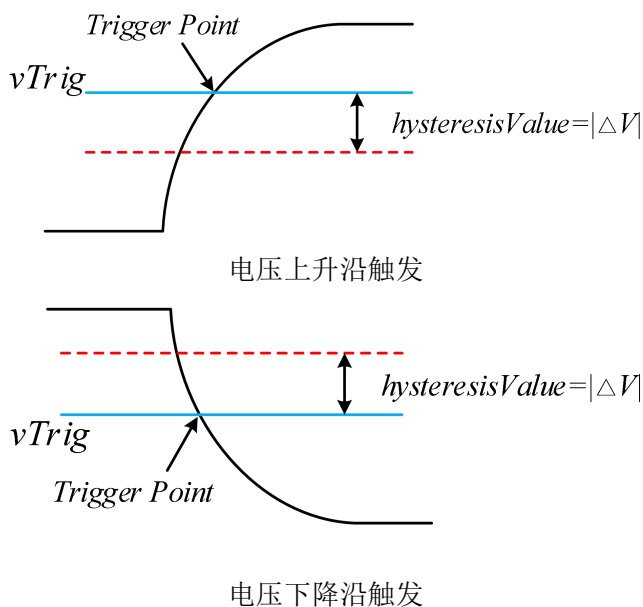
hysteresisValue

double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($vTrig + hysteresisValue$) 值之后又小于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发。



Remarks

电压测量触发设置。

注意：

SetMeasVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，fpvie0 从 10.5V 下降到 8V，步距 0.1V。设置 fpvie1 下降沿触发，触发电平 2V。

```
fpvie1.Set(FI, 0, FPVIE_10V, FPVIE_10MA, FPVIE_RELAY_ON);
//set the state of fpvie1

double Trig = 2;    //triggering voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 25;       //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.

fpvie0.AwgLoader("Uvlo",FV,FPVIE_40V,FPVIE_100MA,awg_pattern,sam);
//import the awg_pattern array into fpvie0 channel, named it as 'Uvlo'. Set to FV mode.

fpvie0.Set(FV, 10.5, FPVIE_40V, FPVIE_100MA, FPVIE_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'

fpvie0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

设置触发电平的正确做法是：

```
fpvie0.MeasureVI(sam,interval,FPVIE_MV_X1,FPVIE_MI_X1,MEAS_AWG);
fpvie1.MeasureVI(sam,interval,FPVIE_MV_X1,FPVIE_MI_X1,MEAS_AWG);
fpvie1.SetMeasVTrig(Trig, TRIG_FALLING); //trigger value is 2V, falling edge
STSEnableAWG(&fpvie0); //enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0, &fpvie1); //enable measurement for fpvie0 and fpvie1
STSAWGRun(); //Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fpvie0.MeasureVI(sam,interval,FPVIE_MV_X1,FPVIE_MI_X1,MEAS_AWG);
fpvie1.MeasureVI(sam,interval,FPVIE_MV_X1,FPVIE_MI_X1,MEAS_AWG);
fpvie1.Set(FI, 0, FPVIE_5V, FPVIE_10MA, FPVIE_RELAY_ON);
//insert a Set() call, change its voltage range from FPVIE_10V to FPVIE_5V, it will cause a utility error.

fpvie1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, falling edge
STSEnableAWG(&fpvie0); //enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0, &fpvie1); //enable measurement for fpvie0 and fpvie1
STSAWGRun(); //Enable AWG and measurement synchronously
```

6.4.15 SetMeasITrig()

```
int SetMeasITrig(  
double iTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

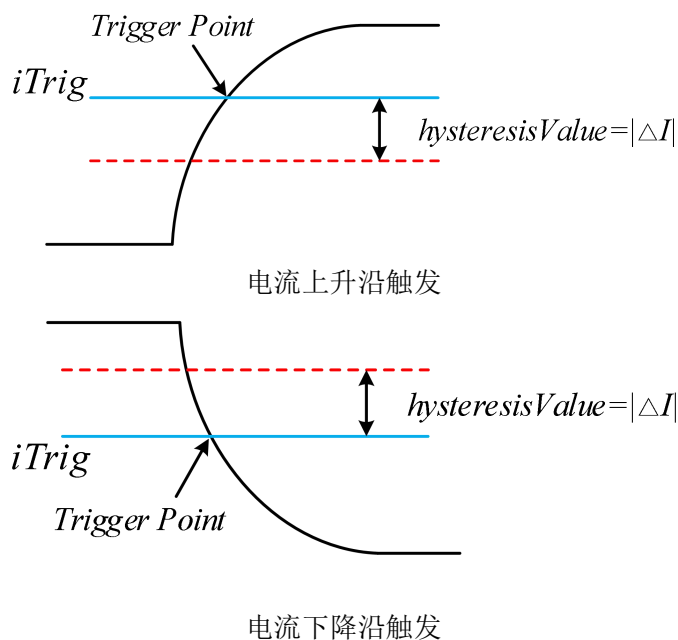
hysteresisValue

double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (*iTrig*-*hysteresisValue*) 值之后又大于 *iTrig* 值时，此时在 *iTrig* 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 (*iTrig* + *hysteresisValue*) 值之后又小于 *iTrig* 值时，此时在 *iTrig* 处判定为触发。



Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，fpvie0 从 16.5V 上升到 21.5V，步距 0.1V。设置 fpvie0 上升沿触发，触发电流值 200uA。

```
double Trig = 200e-6;           // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                   // data size of AWG pattern
int interval = 200;             // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into fpvie0 channel, named it as 'Vst'. Set to FV mode.
fpvie0.AwgLoader("Vst ", FV, FPVle_40V, FPVle_100MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
```

```
fpvie0.Set(FV, 16.5, FPVle_40V, FPVle_100MA, FPVle_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fpvie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
fpvie0.MeasureVI(sam, interval, FPVle_MV_X1, FPVle_MI_X1, MEAS_AWG);
fpvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge

STSEnableAWG(&fpvie0); // enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0); // enable measurement for fpvie0
STSAWGRun(); // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fpvie0.MeasureVI(sam, interval, FPVle_MV_X1, FPVle_MI_X1, MEAS_AWG);

//insert a Set() call, change its current range from FPVle_100MA to FPVle_10MA, it will cause a utility
error.
fpvie0.Set(FV, 16.5, FPVle_40V, FPVle_10MA, FPVle_RELAY_ON);

fpvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge
STSEnableAWG(&fpvie0); // enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0); // enable measurement for fpvie0
STSAWGRun(); // Enable AWG and measurement synchronously
```

6.4.16 ContactCheck()

```
int ContactCheck(FPVle_CONTACTMODE checkMode);
```

Parameters

checkMode

接触测试的模式，可设置为：

FPVle_HIGH_SIDE	检测高端电路的连接
FPVle_LOW_SIDE	检测低端电路的连接
FPVle_ALL_SIDE	同时检测高端和低端电路的连接

Remarks

设置 FPVle 的 Contact 检测模式并进行 Contact 检测。

注意：

1)通过该函数对高端或低端电路进行接触检测，检测结束后，VI 源将恢复至检测

之前的状态;

2) 该函数支持 Group 操作。

Example

例 1: 对 fpvie0 进行 High 端 Contact 检测。

```
fpvie0.Set(FV, 15, FPVle_20V, FPVle_10MA, FPVle_RELAY_ON);  
fpvie0.ContactCheck(FPVle_HIGH_SIDE); // After Contact detection, fpvie0 will restored to the state of  
ContactCheck before.
```

6.4.17 GetContactCheckResult()

```
int GetContactCheckResult (UINT siteCount);
```

Parameters

siteCount

UINT 型变量, 选择读取接触检测结果的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

返回值小于 0: 工位无效;

返回 0: Contact 检测 Pass;

返回值大于 0: 在 FPVle_HIGH_SIDE 和 FPVle_LOW_SIDE 两种模式下, Contact 检测 Fail; 在 FPVle_ALL_SIDE 模式下, 返回 1: Contact 检测高端 Fail 低端 Pass; 返回 2: Contact 检测高端 Pass 低端 Fail; 返回 3: Contact 检测两端 Fail。

Pass 的判据说明:

当测得接触电阻小于 100 Ω , 可认为搭接正常, 返回 PASS。大于等于 100 Ω , 可认为搭接异常。误差约 $\pm 20 \Omega$ 。

Remarks

读取该工位对应 Contact 检测模式下的检测结果。

Example

例 1：对 fpvie0 进行 High 端 Contact 检测并获取检测结果。

```
fpvie0.Set(FV, 0, FPVle_10V, FPVle_1MA, FPVle_RELAY_ON);
fpvie0.ContactCheck(FPVle_HIGH_SIDE);           // High Side check
int result[SITENUM] = { 0 };
for(iSite = 0; iSite < SITENUM; iSite++)
{
    result[iSite] = fpvie0.GetContactCheckResult(iSite);    // Get High Side check result
}
```

6.4.18 SetAlarmMask()

```
void SetAlarmMask(
UINT siteCount,
bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
fpvie0.SetAlarmMask(0);           // mask the Alarm information of SITE_1  
// The alarm information about overranging voltage scale is not output  
fpvie0.Set(FV, 5, FPVle_2V, FPVle_10MA, FPVle_RELAY_ON);
```

6.4.19 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息;

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = fpvie0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.4.20 SiteBindModify ()

```
int SiteBindModify(char* modifySiteList);
```

Parameters

modifySiteList

工位重配置的字符串，格式为：“原工位号 1:目标工位号 1,原工位号 2:目标工位号 2, ...”，原工位和目标工位之间用冒号分隔，多组修改工位间用逗号分隔。

原工位号和目标工位号最小为 0（SITE_1），最大为 250（SITE_251）。

例如，将 SITE_1 的通道修改到 SITE_3，SITE_2 的通道修改到 SITE_4，对应的字符串格式为“0:2,1:3”。

Return Values

返回 0：工位重配置成功；

返回非 0：工位重配置失败。

注意：

以下情况会导致工位重配置失败：

- (1) 多组重配置工位之间必须以逗号分隔，第一个字符和最后一个字符不能为逗号，两个逗号间必须有工位修改信息；
- (2) 原工位号和目标工位号之间必须以冒号分隔，只能有一个冒号，且冒号前后必须都是数字，不能包含其它字符；
- (3) 原工位号和目标工位号必须大于等于 0 且小于系统最大工位数 251；
- (4) 不可以对定义为 NO_SITE 的通道进行工位重配置；
- (5) 同一次修改时（调用一次函数），同一个工位不能被修改两次，如“0:1,0:2”；
- (6) 同一次修改时（调用一次函数），不同工位不能被修改到同一个工位上，如“0:2,1:2”；
- (7) 目标工位如果已经进行过通道配置，此次也必须被修改。示例如下：

```
FPVle fpvie0("S4_0,S12_0");
```

初始的工位配置如下：

```
STSSetMultiSiteBind(MD_FPVle, SITE_1, "S4_0,S4_1");  
STSSetMultiSiteBind(MD_FPVle, SITE_2, "S12_0,S12_1");
```

工位重配置的正确做法（将 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4）：

```
fpvie0.SiteBindModify("0:2,1:3")
```

工位重配置的错误做法（仅将 SITE_1 重配置为 SITE_2）：

```
fpvie0.SiteBindModify("0:1")
```

Remarks

对通道的工位配置信息进行重配置，未进行工位重配置的通道工位信息保持不变。

注意：

该函数的重配置过程不受工位有效性信息的影响，例如将 SITE_1 重配置成 SITE_2，无论当前 SITE_1 是否无效，都会将 SITE_1 对应的物理通道配置到 SITE_2。

Example

通道定义和初始工位配置如下：

```
FPVle fpvie0("S4_0,S12_0");  
FPVle fpvie1("S4_1,S12_1");  
FPVle gpfpvie0("S4_0,S12_0, S4_1,S12_1");
```

```
STSSetMultiSiteBind(MD_FPVle, SITE_1, "S4_0,S4_1");  
STSSetMultiSiteBind(MD_FPVle, SITE_2, "S12_0,S12_1");
```

例 1：设置 fpvie0 的 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4。

```
fpvie0.SiteBindModify("0:2,1:3");
```

注：由于 fpvie0 工位信息重新进行了配置，gpfpvie0 由原来的 Group Pin 变为了 Multisite Pin。

例 2：设置 fpvie0 的 SITE_1 重配置为 SITE_2，SITE_2 重配置为 SITE_3。

```
fpvie0.SiteBindModify("0:1,1:2");
```

例 3：设置 fpvie0 的 SITE_1 重配置为 SITE_3，SITE_2 保持不变。

```
fpvie0.SiteBindModify("0:2");
```

例 4：设置 gpfpvie0 的 SITE_1 重配置为 SITE_3，SITE_2 重配置 SITE_4。

```
gpfpvie0.SiteBindModify("0:2,1:3");
```

注：由于 gpfpvie0 由 fpvie0 和 fpvie1 组成，因此 fpvie0 和 fpvie1 对应的工位信息都发生了变化。

6.4.21 获取板卡固有信息函数

6.4.21.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
fpvie0.GetBoardSN(0, boardSN, 255);    // Get the board serial number
```

6.4.21.2 GetBoardHDRev()

```
int GetBoardHDRev(  
    UINT siteCount,  
    char* hardRev,  
    UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡硬件版本号。。

Example

```
char hardRev [255] = {0};  
fpvie0.GetBoardHDRev(0, hardRev, 255);           // Get the board hardware revision
```

6.4.22 获取校准信息函数

6.4.22.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = fpvie0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.4.22.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = fpvie0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.4.22.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = fpvie0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.4.22.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS;
CAL_FAIL	校准 FAIL;
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = fpvie0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

6.4.22.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量, 选择读取校准时逻辑版本信息的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

正常返回逻辑版本号; 异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = fpvie0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.4.22.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char* meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000	Keithley2000
STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0};           // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter
```

```
calMeterType = fpvie0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.4.22.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteCount,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* boardHdRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回“N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回“N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 boardHdRev 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
intlogicRev= 0;
char boardSN[255] = {0};          // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};       // the buffer for saving hardware version of calibration board
logicRev = fpvie0. GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.4.22.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev (
UINT siteCount,
char* softRev,
UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回“N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0};      // the buffer for saving softwareversion
// Get the version of calibration software
fpvie0.GetCalibrationSoftRev(0, softRev, 255);
```

6.4.22.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = fpvie0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.4.23 编程范例

6.4.23.1 设置一组波形数据组的拼接。

测试条件：

- 第一组：设置一个正弦波数据组，首地址从 0 开始，数据长度为 100；
- 第二组：设置一个三角波数据组，首地址从 100 开始，数据长度为 100；
- 第三组：设置一个方波数据组，首地址从 200 开始，数据长度为 100；
- 第四组：设置一个斜波数据组，首地址从 300 开始，数据长度为 100。

```
int sam=400; //AWG waveform data length
int interval=200; //AWG data interval time, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],100,1,-5,0,180);
// Set a sinewave data array, the start address starts from 0, data size is 100.

STSAWGCreateTriangleData(&awg_pattern[100],100,4,-5,0,0);
//Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200],100,4,-5,0,50);
//Set a square wave data array, the start address starts from 200, data size is 100.
```



```
STSAWGCreateRampData(&awg_pattern[300],100,1,1,5);
//Set an ramp data array, the start address starts from 300, data size is 100.

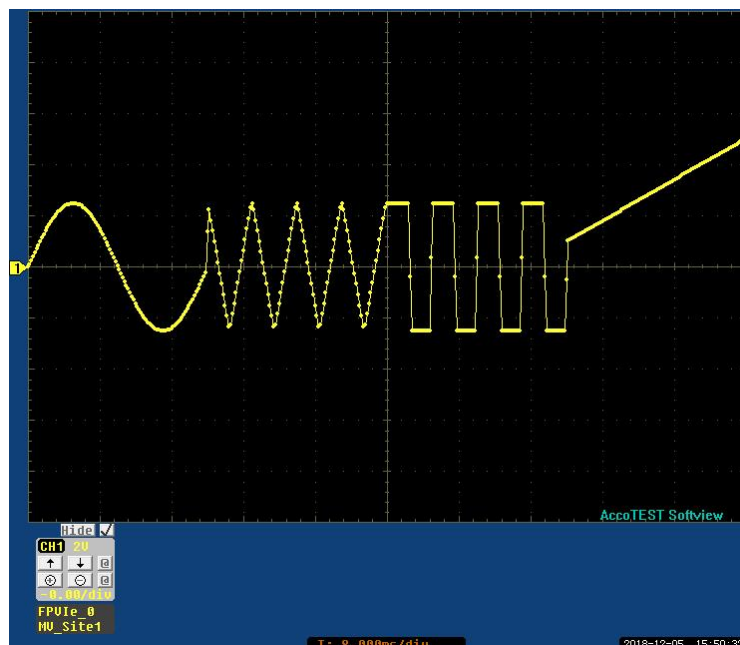
fpvie0.AwgLoader("Vcc_awg",FV,FPVle_10V,FPVle_10MA,awg_pattern,sam);
//import the awg_pattern array into fpvie0 channel, named it as 'Vcc_awg'. Set to FV mode.

fpvie0.Set(FV, 0, FPVle_10V, FPVle_10MA, FPVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

fpvie0.AwgSelect("Vcc_awg", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fpvie0.MeasureVI(sam, interval, FPVle_MV_X1, FPVle_MI_X1,MEAS_AWG);
STSEnableAWG(&fpvie0);//enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0);//enable measurement for fpvie0
STSAWGRun();//Enable AWG and measurement synchronously
```

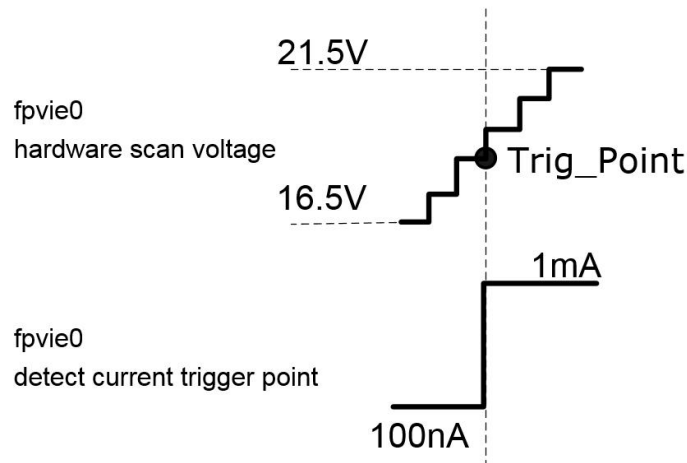
组合之后的波形如下：



6.4.23.2 同一路源生成 AWG 且作为触发源

测试条件:

建立一个波形数据组，fpvie0 从 16.5V 上升到 21.5V，步距 0.1V。测试 fpvie0 电流大于 200uA 时的跳变点的电压值。



```
double Trig = 200e-6; // trigger current level
double Trig_Point[4] = {0.0};
double result[4] = {0.0};
int sam = 50; // data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[100] = {0.0};

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
// Set an ramp data array, the start address starts from 0, data size is 50.

fpvie0.AwgLoader("Vst", FV, FPVLe_40V, FPVLe_100MA, awg_pattern, sam);
// import the awg_pattern array into fpvie0 channel, named it as 'Vst'. Set to FV mode.

fpvie0.Set(FV, 16.5, FPVLe_40V, FPVLe_100MA, FPVLe_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'

fpvie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fpvie0.MeasureVI(sam, interval, FPVLe_MV_X1, FPVLe_MI_X1, MEAS_AWG);

fpvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge

STSEnableAWG(&fpvie0); // enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0); // enable measurement for fpvie0
STSAWGRun(); // Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] = fpvie0.GetMeasResult(i, MIRET, TRIG_RESULT);
    // Get the position of trigger point

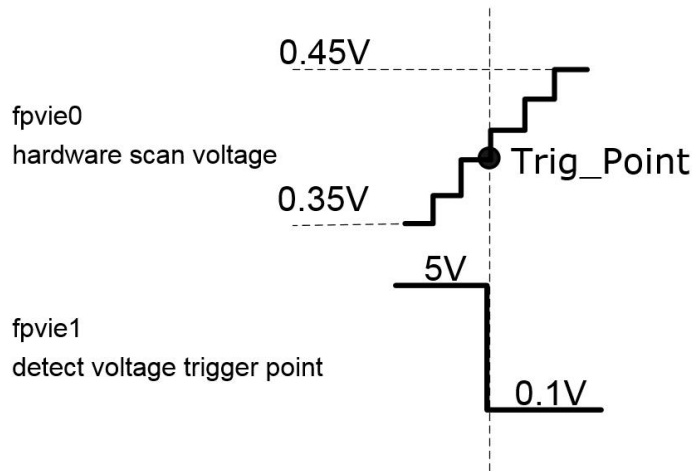
    result[i] = fpvie0.GetMeasResult(i, MVRET, Trig_Point[i]);
    // Read the voltage value on trigger position

    Vst->SetTestResult(i, 0, result[i]);
}
```

6.4.23.3 一路源生成 AWG 另一路源作为触发源

测试条件:

建立一个波形数据组，fpvie0 从 0.35V 上升 0.45V，步距 0.001V。测试 fpvie1 端的电压小于 2V 时，fpvie0 跳变点的电压值。



```
double Trig = 2; //trigger voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

fpvie1.Set(FI, 0, FPVie_10V, FPVie_1MA, FPVie_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0],sam,1,0.35,0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

fpvie0.AwgLoader("Vcspre_AWG",FV,FPVie_1V,FPVie_100MA,awg_pattern,sam);
//import the awg_pattern array into fpvie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

fpvie0.Set(FV, 0.30, FPVie_1V, FPVie_100MA, FPVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

fpvie0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS
fpvie0.MeasureVI(sam,interval, FPVie_MV_X1, FPVie_MI_X1,MEAS_AWG);
fpvie1.MeasureVI(sam,interval, FPVie_MV_X1, FPVie_MI_X1,MEAS_AWG);

fpvie1.SetMeasVTrig(Trig,TRIG_FALLING); // trigger value is 2V, Falling edge

STSEnableAWG(&fpvie0); //enable AWG pattern for fpvie0
STSEnableMeas(&fpvie0,&fpvie1); //enable measurement for fpvie0 and fpvie1
STSAWGRun(); //Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] =fpvie1.GetMeasResult(i,MVRET,TRIG_RESULT);
    //Get the position of trigger point for fpvie1
}
```

```

result[i] = fpvie0.GetMeasResult(i,MVRET,Trig_Point[i]);
//Read the voltage value of fpvie0 on trigger position

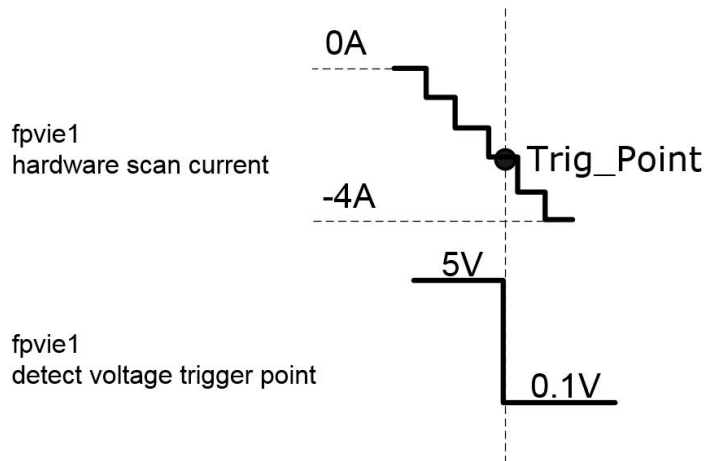
Vcspre->SetTestResult(i, 0, result[i]);
}

```

6.4.23.4 AWG 方式扫描极限电流

测试条件:

AWG 硬件扫描测试极限电流。fpvie1 从 0 向下扫描到-4A, 步距 40mA, 测试 fpvie1 电压小于 0.5V 时的跳变点的电流值。



```

double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100;//data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[200] = {0.0}; //define an AWG waveform data array

STSAWGCreateRampData(&awg_pattern[0],sam,1,0,-4);
//Set an ramp data array, the start address starts from 0, data size is 100.

fpvie1.AwgLoader("IOut", FI, FPVle_5V, FPVle_10A, awg_pattern, sam);
//import the awg_pattern array into fpvie1 channel, named it as 'IOut'. Set to FI mode.

fpvie1.Set(FI,0,FPVle_5V,FPVle_10A,FPVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'IOut' of its 'AwgLoader()'

fpvie1.AwgSelect("IOut", 0, sam-1, 0, interval );
//Select pattern of 'IOut', start step=0, stop=sam-1; loop= 0; interval time=100uS
fpvie1.MeasureVI(sam, interval, FPVle_MV_X1, FPVle_MI_X1,MEAS_AWG);

fpvie1.SetMeasVTrig(0.5,TRIG_FALLING); // trigger value is 0.5V, faling edge

STSEnableAWG(&fpvie1); //enable AWG pattern for fpvie1

```

```
STSEnableMeas(&fpvie1); // enable measurement for fpvie1
STSAWGRun();//Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] = fpvie1.GetMeasResult(i,MVRET,TRIG_RESULT);
    //Get the position of trigger point for fpvie1

    result[i] = fpvie1.GetMeasResult(i,MIRET,Trig_Point[i]);
    // Read the current value on trigger position for fpvie1

    IPeak->SetTestResult(i, 0, result[i]);
}
```

6.4.23.5 AWG 生成正弦波测试纹波抑制比

测试条件:

fpvie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。测试 fpvie1 输出纹波抑制比。

```
int Datacount=0;
double AD_SAMPLE=500;//sampling points
double sum[4]={0.0};
double temp[4]={0.0},temp2[4]={0.0};
int sam = 830; //data size of AWG pattern
int interval = 10; // interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
double data[5000]={0.0};
double result[4]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3

fpvie0.AwgLoader("Sin_AWG",FV,FPVle_5V,FPVle_1A,awg_pattern,sam);
// pattern named as 'Sin_AWG, FV mode

fpvie0.Set(FV, 0, FPVle_5V, FPVle_1A, FPVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'

fpvie0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
delay_ms(1);

fpvie1.Set(FI, -100e-3, FPVle_5V, FPVle_100MA, FPVle_RELAY_ON);
//Vout FI mode, prepares to measure
delay_ms(2);
fpvie1.MeasureVI(AD_SAMPLE, 20); //fpvie1 measurement

for (i=0;i<SITENUM;i++)
{
```

```
for(Datacount=0;Datacount<AD_SAMPLE;Datacount++)
{
    data[Datacount]= fpvie1.GetMeasResult(i, MVRET, Datacount);
    //fpvie1 acquires every voltage values from 500 sampling points
    result[i]= fpvie1.GetMeasResult(i, MVRET);
    //fpvie1 acquires average voltage of 500 sampling points
    sum[i]+=(data[Datacount]-result[i])*(data[Datacount]-result[i]);
}
temp[i]=(sqrt(sum[i]/AD_SAMPLE))*2.828;
//calculate peak to peak value of sine wave received from output
temp2[i]=-20*log10((temp[i]+1e-12)/0.66);
//calculate PSRR
sum[i]=0;
}

for(i=0; i<SITENUM; i++)
{
    PSRR->SetTestResult(i, 0, temp2[i]);
}

fpvie1.Set(FI, 0, FPVle_20V, FPVle_100MA, FPVle_RELAY_OFF);
fpvie0.AwgStop();//stop running AWG waveform of fpvie0
fpvie0.Set(FV, 0, FPVle_5V, FPVle_1A, FPVle_RELAY_OFF);
```

6.5 FOVle

6.5.1 FOVle()

```
FOVle(char*channelList, char*chName = NULL);
```

Parameters

channelList

FOVle 绑定通道字符串。形如“S5_0, S5_1”，其中“S5”表示槽位号 5（S 即 Slot），“0、1”表示板内序号。

注意：

FOVle 一块板卡最多 8 个通道，因此板内序号最大值为 7。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 FOVle_xx。

Remarks

定义一路 FOVle 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 4 块 FOVle（分别位于 Slot5、Slot11、Slot22 和 Slot28），四工位并行工作，定义可使用的 FOVle。

```
FOVle fovie0("S5_0,S11_0,S22_0,S28_0");  
FOVle fovie1("S5_1,S11_1,S22_1,S28_1");  
FOVle fovie2("S5_2,S11_2,S22_2,S28_2");  
FOVle fovie3("S5_3,S11_3,S22_3,S28_3");  
FOVle fovie4("S5_4,S11_4,S22_4,S28_4");  
FOVle fovie5("S5_5,S11_5,S22_5,S28_5");  
FOVle fovie6("S5_6,S11_6,S22_6,S28_6");  
FOVle fovie7("S5_7,S11_7,S22_7,S28_7");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_FOVle, SITE_1, "S5_0-7"); // set Slot5 ch0-7 to SITE1
STSSetMultiSiteBind(MD_FOVle, SITE_2, "S11_0-7"); // set Slot11 ch0-7 to SITE2
STSSetMultiSiteBind(MD_FOVle, SITE_3, "S22_0-7"); // set Slot22 ch0-7 to SITE3
STSSetMultiSiteBind(MD_FOVle, SITE_4, "S28_0-7"); // set Slot28 ch0-7 to SITE4
```

6.5.2Set()

```
int Set(
VIMode viMode,
double setValue,
FOVle_VRNG vRange,
FOVle_IRNG iRange,
FOVle_OUT_RELAY relayStatus = FOVle_RELAY_HOLD,
double risingTime = 0.1);
```

Parameters

viMode

FOVle 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

vRange

FOVle 电压量程。

FOVle_40V

FOVle_20V

FOVle_10V

FOVle_5V

FOVle_2V

FOVle_1V

iRange

FOVle 电流量程。

FOVle_1A

FOVle_100MA

FOVle_10MA

FOVle_1MA

FOVle_100UA

FOVle_10UA

注意：

FOVle_10UA 电流档位仅能用于测量。

relayStatus

设置输出继电器如何动作，分为四种情况：

FOVle_RELAY_ON：打开输出继电器；

FOVle_RELAY_OFF：关闭输出继电器；

FOVle_RELAY_HOLD：输出继电器保持上一个状态；

FOVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FOVle 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默认为 0.1mS；

注意：

- 1) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Slow 模式；
- 3) 该参数设置大于 0.2mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升

或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

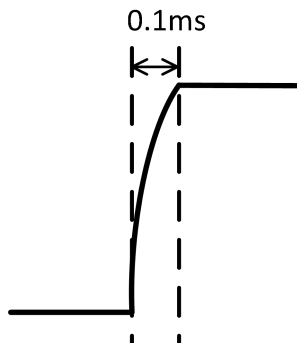
Remarks

设置 FOVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

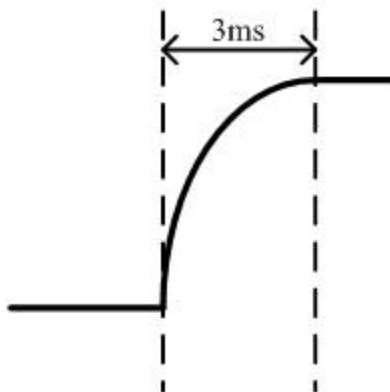
例 1：设置 fovie0 电压输出 5V，risingTime 不填则默认为 0.1mS。

```
fovie0.Set(FV,5,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
```



例 2：设置 fovie0 电压输出 5V，risingTime 设置为 3mS。

```
fovie0.Set(FV,0,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);  
delay_ms(1);  
fovie0.Set(FV,5,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON, 3);
```



6.5.3 SetSyn()

```
int SetSyn(  
VIMode viMode,  
const double* setValue,  
UINT siteSize,  
FOVie_VRNG vRange,  
FOVie_IRNG iRange,  
FOVie_OUT_RELAY relayStatus = FOVie_RELAY_HOLD,  
double risingTime = 0.1);
```

Parameters

viMode

FOVie 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-1~1。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1) setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2) 若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3) 若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

FOVie 电压量程。

FOVle_40V

FOVle_20V

FOVle_10V

FOVle_5V

FOVle_2V

FOVle_1V

iRange

FOVle 电流量程。

FOVle_1A

FOVle_100MA

FOVle_10MA

FOVle_1MA

FOVle_100UA

FOVle_10UA

注意：

FOVle_10UA 电流档位仅能用于测量。

enableRelay

设置输出继电器如何动作，分为四种情况：

FOVle_RELAY_ON: 打开输出继电器

FOVle_RELAY_OFF: 关闭输出继电器

FOVle_RELAY_HOLD: 输出继电器保持上一个状态

FOVle_RELAY_SENSE_ON: 启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FOVle 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默认为 0.1mS；

注意：

- 1) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Slow 模式；
- 3) 该参数设置大于 0.2mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1：fovie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

```
double setValue[2] = {3, 5};  
int siteSize = 2;  
  
fovie0.SetSyn(FV, setValue, siteSize, FOVIe_10V, FOVIe_100MA, FOVIe_RELAY_ON);
```

例 2：fovie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};  
int siteSize = 2;  
  
fovie0.Set(FV, 0, FOVIe_10V, FOVIe_100MA, FOVIe_RELAY_ON);  
delay_ms(1);  
fovie0.SetSyn(FV, setValue, siteSize, FOVIe_10V, FOVIe_100MA, FOVIe_RELAY_ON, 3);
```

6.5.4 SetClamp()

```
int SetClamp(  
double percent_PFS,  
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：10~102。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：10~102。

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-1：函数调用失败。

Remarks

设置 FOVle 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 102%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 fovie1 为 FI=0 模式，箝位到(50, 20)，然后切换 fovie1 为 FV=1V 模式，箝位到(25, 25)。

```
fovie1.Set(FI, 0, FOVle_10V, FOVle_10MA, FOVle_RELAY_ON);  
//positive clamp to 50% of full range(FOVle_10V), negative clamp to 20% of full range  
fovie1.SetClamp(50, 20);  
  
//switch from FI to FV mode, then positive/negative clamps recover back to 102% of full range  
fovie1.Set(FV, 0, FOVle_10V, FOVle_100MA, FOVle_RELAY_ON);  
//reset positive clamp to 25% of full range(FOVle_100MA), negative clamp to 25% of full range  
fovie1.SetClamp(25, 25);  
//Set FV=1V
```

```
fovie1.Set(FV,1,FOVie_10V,FOVie_100MA,FOVie_RELAY_ON);
```

例 2：设置 fovie1 为 FV=1V 模式，箝位到(50, 25)，然后切换 fovie1 为 FV=35V，测量结束后设置 fovie1 为 FV=0，箝位到(100, 100)。

```
fovie1.Set(FV, 0, FOVie_10V, FOVie_100MA, FOVie_RELAY_ON);  
//positive clamp to 50% of full range(FOVie_100MA), negative clamp to 25% of full range  
fovie1.SetClamp(50, 25);  
fovie1.Set(FV, 1, FOVie_10V, FOVie_100MA, FOVie_RELAY_ON);  
delay_ms(1);  
  
//still in FV mode, clamp setting will keep unchanged  
fovie1.Set(FV, 35, FOVie_40V, FOVie_100MA, FOVie_RELAY_ON);  
fovie1.MeasureVI(100, 20);  
  
fovie1.Set(FV, 0, FOVie_40V, FOVie_100MA, FOVie_RELAY_ON);  
//after FV=0, set clamps back to 100%  
fovie1.SetClamp(100, 100);
```

6.5.5 MeasureVI()

```
int MeasureVI(  
    UINT sampleTimes,  
    double samplePeriod,  
    FOVie_MI_GAIN miGain = FOVie_MI_X1,  
    FLOATMEASMODE measMode = MEAS_NORMAL,  
    UINT T1 = 0,  
    UINT T2 = 0,  
    UINT T3 = 0,  
    UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096。

注意：

当 sampleTimes < 1 时，sampleTimes = 1；

当 sampleTimes > 4096 时，sampleTimes = 4096。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：5~30000。

注意：

当 $\text{samplePeriod} < 5$ 时， $\text{samplePeriod} = 5$ ；

当 $\text{samplePeriod} > 30000$ 时， $\text{samplePeriod} = 30000$ 。

miGain

FOVle 测流增益。

FOVle_MI_X1

FOVle_MI_X10

该参数可以不写，默认值为 FOVle_MI_X1。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数可以不写，默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 FOVie 的测量次数、测量间隔、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1: 设置 fovie0 为普通模式测量, 采样 20 次, 间隔 10uS。

```
fovie0.MeasureVI(20,10);
```

6.5.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量, 选择测量数据工位。0 代表 Site1, 1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时, siteCount 可以为任意值。

retType

选择测量读回的数据类型, 可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写, 默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

- 1) FOVie 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。
- 2) 当参数 *sampleNumber* 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回值为-3，如果测量模式与回读模式不符，则函数返回值为-5。

Example

例 1：返回 fovie0 电流测量结果平均值。

```
//fovie0 sampleTimes=20, samplePeriod=10uS
fovie0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value
    of current
}
```

```
adresult[i]=fovie0.GetMeasResult(i, MIRET);  
}
```

例 2：返回 fovie0 电压测量结果第 15 个点的值。

```
//fovie0 sampleTimes=20, samplePeriod=10uS  
fovie0.MeasureVI(20, 10);  
for(i=0; i<SITENUM; i++)  
{  
    // the result is fifteenth voltage value  
    adresult[i]=fovie0.GetMeasResult(i, MVRET, 14);  
}
```

例 3：AWG 同步测量时，返回 fovie0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge  
fovie0.SetMeasVTrig(2, TRIG_FALLING);  
// fovie1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG  
fovie1.MeasureVI(50, 20, FOVle_MI_X1, MEAS_AWG);  
// the fovie0's parameters of MeasureVI function are same with fovie1  
fovie0.MeasureVI(50, 20, FOVle_MI_X1, MEAS_AWG);  
// fovie1 AWG enable  
STSEnableAWG(&fovie1);  
// fovie0 and fovie1 measure enable  
STSEnableMeas(&fovie1, &fovie0);  
STSAWGRun();  
for(i=0; i<SITENUM; i++)  
{  
    // Get the voltage position of first trigger  
    Trig_Point0[i] =fovie0.GetMeasResult(0, MVRET, TRIG_RESULT, 0);  
    // Get the voltage position of second trigger  
    Trig_Point1[i] =fovie0.GetMeasResult(0, MVRET, TRIG_RESULT, 1);  
}
```

6.5.7 BlockRead()

```
int BlockRead(  
    UINT siteCount,  
    UINT start,  
    UINT size,  
    double* buffer,  
    MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1)buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200]={0.0};
fovie0.MeasureVI(200, 10);
for(i=0; i<SITENUM; i++)
{
    //start=0; size=200; Get MVRET, saved to result.
    fovie0.BlockRead(i, 0, 200, result[i], MVRET);
}
```

6.5.8Pulse()

```
int Pulse(  
double pulseValue,  
UINT pulseTime = 300,  
int measStartTime = 0,  
UINT sampleTimes = 10,  
double sampleInterval = 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-40~40；

选择 FI 模式时，单位 A。最大范围为-1~1。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。最小采样间隔为 5uS，分辨率 1uS。
可以不写，默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

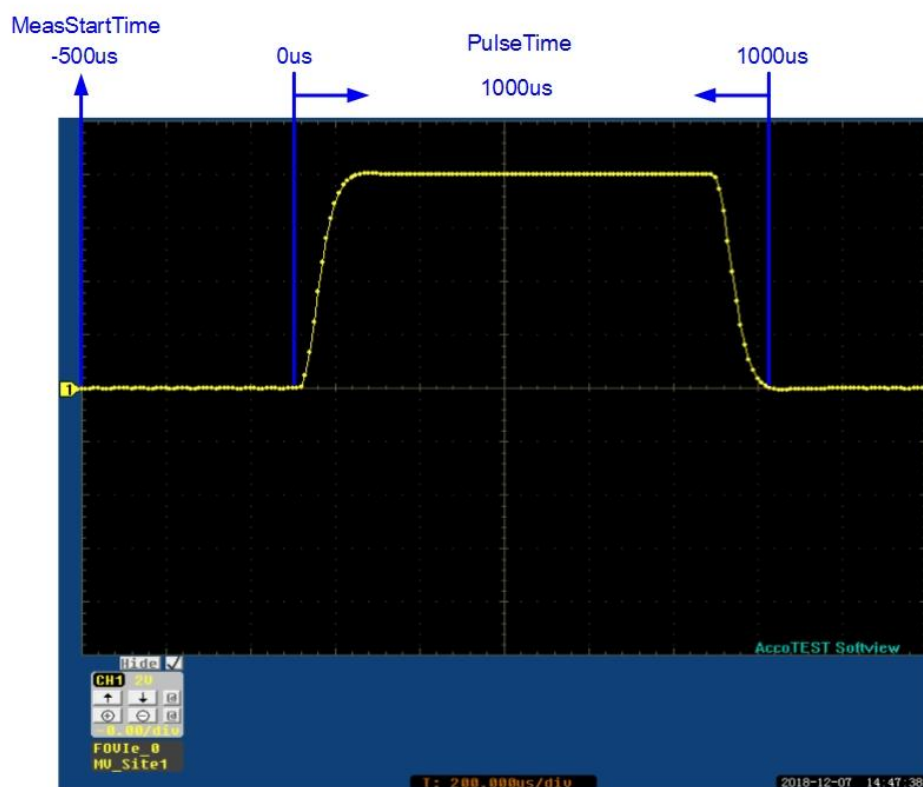
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 fovie0 起始状态 FV 为 0。设置 fovie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
fovie0 .Set(FV,0,FOVle_10V,FOVle_10MA,FOVle_RELAY_ON);  
// first set fovie0 in FV=0 mode,  
fovie0.Pulse (8,1000, -500, 200, 10);  
//pulse output value=8v, pulse duration=1000uS, start time for measuring is -500uS  
//sample=200,interval time=10uS
```

使用 SoftView 软件抓取的波形如下：



6.5.9 AwgLoader()

```
int AwgLoader(
char* awgName,
VIMode viMode,
FOVie_VRNG vRange,
FOVie_IRNG iRange,
double* awgData,
UINT awgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

FOVie 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

FOVle 电压量程。

FOVle_40V

FOVle_20V

FOVle_10V

FOVle_5V

FOVle_2V

FOVle_1V

iRange

FOVle 电流量程。

FOVle_1A

FOVle_100MA

FOVle_10MA

FOVle_1MA

FOVle_100UA

FOVle_10UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 FOVle 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 FOVle 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 FOVle 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 FOVle 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，fovie0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50;           //AWG waveform data length
int interval = 200;     //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5); //pattern array saved in 'awg_pattern',
start from 16.5 to 21.5
fovie0.AwgLoader("Vst",FV,FOVle_40V,FOVle_100MA,awg_pattern,sam);
```

例 2：建立一个波形数据组，fovie0 输出频率为 100Hz、峰峰值为 4V、直流偏置为 10V 的正弦波。

```
int sam = 1000;        //data size for AWG pattern
int interval = 10;     //interval time for AWG pattern
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0); //set a sinewave data array of 100Hz,
Vpp=4, DC Offset=10
fovie0.AwgLoader("Sine",FV,FOVle_10V,FOVle_1MA,awg_pattern, sam); // pattern named as 'SINE',
FV mode
```

6.5.10 AwgSelect()

```
int AwgSelect(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 FOVle 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;         //data size for AWG pattern
int sam2 = 500;          // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,16.5,21.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,16.5,10.5);

// pattern named as 'AWG_Vst', FV mode
fovie0.AwgLoader("AWG_Vst",FV,FOVle_40V,FOVle_100MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
fovie0.AwgLoader("AWG_Vuvlo",FV,FOVle_20V,FOVle_100MA,awg_pattern2,sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of
its 'AwgLoader()'
fovie0.Set(FV, 16.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
fovie0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
.....
STSAWGRun();    //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.5.11 AwgRun()

```
int AwgRun(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval = 10.0,  
BOOL runMode = AWG_SINGLE,  
UINT delayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 AWG_SINGLE。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 FOV1e 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRUN。

Example

例 1: fovie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。

```
int sam = 830;          //size of AWG pattern
double interval = 10;   //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);

// pattern named as 'Sin_AWG, FV mode
fovie0.AwgLoader("Sin_AWG",FV, FOVle_5V, FOVle_1A,awg_pattern,sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
fovie0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
```

6.5.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 FOVle 通道的 AWG 循环运行。

Example

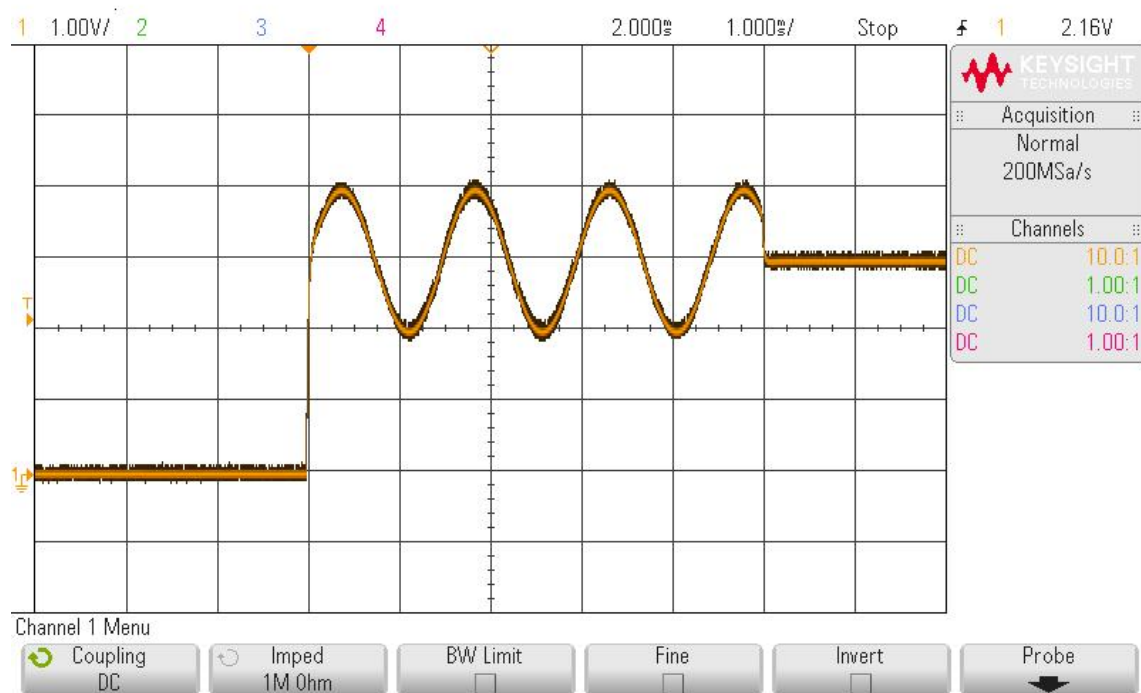
例 1: fovie0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150;          //data size of AWG pattern
double interval = 10;   //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,2,3,0);
//Set a Sine wave data array, data size is 150, period is 1, Vpp=2V, DC offset = 3V, phase = 0.
fovie0.AwgLoader("Sin_AWG", FV, FOVle_5V, FOVle_1A, awg_pattern, sam);
//Import awg_pattern data array into fovie0 channel, pattern named as 'Sin_AWG'
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
fovie0.AwgRun("Sin_AWG", 0, (sam-1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
delay_ms(6); //fovie0 AWG pattern keeps running for 6mS
fovie0.AwgStop();
//stop loops for fovie0 AWG running
```

该正弦波周期为 1.5mS，延时 5mS 后，停止 awg 波形。

下图为示波器抓取的波形。



6.5.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 FOVle 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 fovie0 通道的 AWG 数据。

```
fovie0.AwgClear(); //clear AWG data of fovie0
fovie0.AwgLoader("P1",FV,FOVle_40V,FOVle_100MA,awg_pattern1,sam);
```

6.5.14 SetMeasVTrig()

```
int SetMeasVTrig(  
double vTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

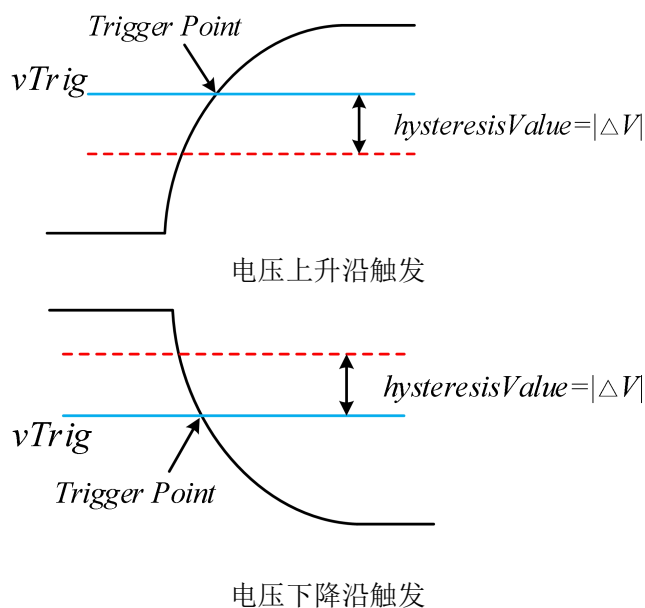
hysteresisValue

double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (*vTrig*-*hysteresisValue*) 值之后又大于 *vTrig* 值时，此时在 *vTrig* 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 (*vTrig*+*hysteresisValue*) 值之后又小于 *vTrig* 值时，此时在 *vTrig* 处判定为触发。



Remarks

电压测量触发设置。

注意：

SetMeasVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1：建立一个波形数据组，fovie0 从 10.5V 下降到 8V，步距 0.1V。设置 fovie1 下降沿触发，触发电平 2V。

```
fovie1.Set(FI, 0, FOVle_10V, FOVle_10MA, FOVle_RELAY_ON);

double Trig = 2;    //triggering voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 25;       //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.

fovie0.AwgLoader("Uvlo",FV,FOVle_40V,FOVle_100MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Uvlo'. Set to FV mode.

fovie0.Set(FV, 10.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
```

```
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
fovie0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

设置触发电平的正确做法是：

```
fovie1.Set(FI, 0, FOVIe_10V, FOVIe_10MA, FOVIe_RELAY_ON);
//set fovie1 on FI mode.
fovie1.SetMeasVTrig(Trig, TRIG_FALLING);
//trigger value is 2V, falling edge
fovie0.MeasureVI(sam,interval,FOVIe_MI_X1, MEAS_AWG);
fovie1.MeasureVI(sam,interval,FOVIe_MI_X1, MEAS_AWG);
STSEnableAWG(&fovie0);           //enable AWG pattern for fovie0
STSEnableMeas(&fovie0, &fovie1); //enable measurement for fovie0 and fovie1
STSAWGRun();                     //Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fovie1.SetMeasVTrig(Trig, TRIG_FALLING);
// trigger value is 2V, falling edge
fovie1.Set(FI, 0, FOVIe_5V, FOVIe_10MA, FOVIe_RELAY_ON);
//insert a Set() call, change its voltage range from FOVI_10V to FOVI_5V, it will cause a utility error.
fovie0.MeasureVI(sam,interval,FOVIe_MI_X1, MEAS_AWG);
fovie1.MeasureVI(sam,interval,FOVIe_MI_X1, MEAS_AWG);
STSEnableAWG(&fovie0);           //enable AWG pattern for fovie0
STSEnableMeas(&fovie0, &fovie1); //enable measurement for fovie0 and fovie1
STSAWGRun();                     //Enable AWG and measurement synchronously
```

6.5.15 SetMeasITrig()

```
int SetMeasITrig(
double iTrig,
TRIG_MODE trigMode = TRIG_FALLING,
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

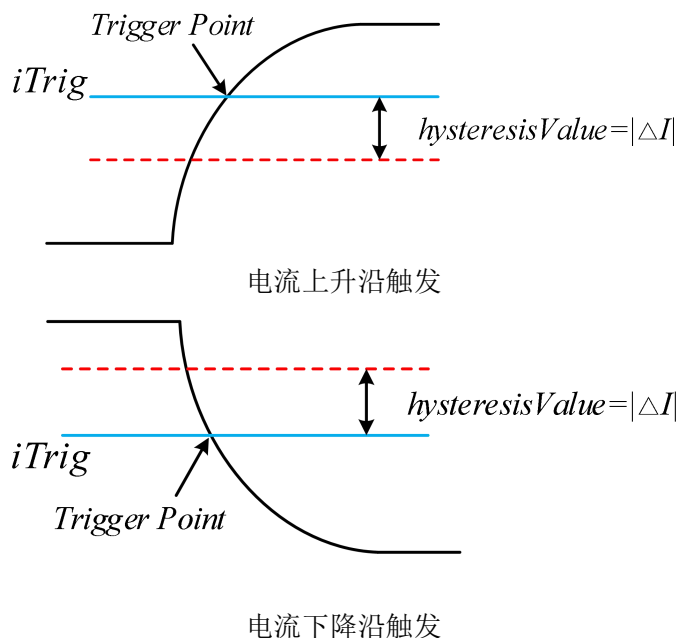
hysteresisValue

double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($iTrig - hysteresisValue$) 值之后又大于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($iTrig + hysteresisValue$) 值之后又小于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发。



Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，fovie0 从 16.5V 上升到 21.5V，步距 0.1V。设置 fovie0 上升沿触发，触发电流值 200uA。

```
double Trig = 200e-6;           // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                   // data size of AWG pattern
int interval = 200;             // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into fovie0 channel, named it as 'Vst'. Set to FV mode.
fovie0.AwgLoader("Vst ", FV, FOVLe_40V, FOVLe_100MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
fovie0.Set(FV, 16.5, FOVLe_40V, FOVLe_100MA, FOVLe_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fovie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
fovie0.MeasureVI(sam, interval, FOVLe_MI_X1, MEAS_AWG);
fovie0.SetMeasITrig(Trig, TRIG_RISING);           // trigger value is 200uA, rising edge

STSEnableAWG(&fovie0); // enable AWG pattern for fovie0
STSEnableMeas(&fovie0); // enable measurement for fovie0
STSAWGRun();           // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fovie0.MeasureVI(sam, interval, FOVLe_MI_X1, MEAS_AWG);

//insert a Set() call, change its current range from FOVLe_100MA to FOVLe_10MA, it will cause a utility
error.
fovie0.Set(FV, 16.5, FOVLe_40V, FOVLe_10MA, FOVLe_RELAY_ON);

fovie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge
STSEnableAWG(&fovie0); // enable AWG pattern for fovie0
STSEnableMeas(&fovie0); // enable measurement for fovie0
STSAWGRun();           // Enable AWG and measurement synchronously
```

6.5.16 ContactCheck()

```
int ContactCheck(FOVie_CONTACTMODE checkMode);
```

Parameters

checkMode

接触测试的模式，可设置为：

FOVie_HIGH_SIDE 检测高端电路的连接

Remarks

设置高端接触检测的模式。

注意：

- 1) 通过该函数对高端电路进行接触检测，检测结束后，VI 源将恢复至检测之前的状态；
- 2) 该函数支持 Group 操作。

Example

```
fovie0.ContactCheck(FOVie_HIGH_SIDE); // set the contact check mode to high side
```

6.5.17 GetContactCheckResult()

```
int GetContactCheckResult (UINTsiteCount);
```

Parameters

siteCount

UINT 型变量，选择读取接触检测结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回-1：该工位无效

返回 0：该工位接触检测 Pass

返回 1：该工位接触检测 Fail

Remarks

读取该工位接触检测的结果。

Example

```
int result = fovie0. GetContactCheckResult(0);    // Get contact check result of SITE_1
```

6.5.18 SetAlarmMask()

```
void SetAlarmMask(  
    UINT siteCount,  
    bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
fovie0.SetAlarmMask(0);           // mask the Alarm information of SITE_1
// The alarm information about overranging voltage scale is not output
fovie0.Set(FV, 5, FOVIe_2V, FOVIe_10MA, FOVIe_RELAY_ON);
```

6.5.19 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息；

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = fovie0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.5.20 获取板卡固有信息函数

6.5.20.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
fovie0.GetBoardSN(0, boardSN, 255);           // Get the board serial number
```

6.5.20.2 GetBoardHDRev()


```
int GetBoardHDRev(  
UINT siteCount,  
char* hardRev,  
UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡硬件版本号。。

Example

```
char hardRev [255] = {0};  
fovie0.GetBoardHDRev(0, hardRev, 255);           // Get the board hardware revision
```

6.5.21 获取校准信息函数

6.5.21.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = fovie0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.5.21.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = fovie0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.5.21.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;
```

```
calHumidity = fovie0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.5.21.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS;
CAL_FAIL	校准 FAIL;
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = fovie0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

6.5.21.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = fovie0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.5.21.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000	Keithley2000
STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0};          // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = fovie0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.5.21.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(
```

```
UINT siteCount,  
char* boardSN = NULL,  
UINT snSize = 0,  
char* boardHdRev = NULL,  
UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回 “N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 *boardHdRev* 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255]= {0};          // the buffer for saving Serial NO. of calibration board
char boardHdRev[255]= {0};       // the buffer for saving hardware version of calibration board
logicRev = fovie0. GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.5.21.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(
    UINT siteCount,
    char* softRev,
    UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回 “N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0};      // the buffer for saving softwareversion
// Get the version of calibration software
fovie0.GetCalibrationSoftRev(0, softRev, 255);
```

6.5.21.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = fovie0.GetCalibrationSlotID(0);// Get the slot ID of calibration
```

6.5.22 编程范例

6.5.22.1 设置一组波形数据组的拼接。

测试条件:

- 第一组：设置一个正弦波数据组，首地址从0开始，数据长度为100；
- 第二组：设置一个三角波数据组，首地址从100开始，数据长度为100；
- 第三组：设置一个方波数据组，首地址从200开始，数据长度为100；
- 第四组：设置一个斜波数据组，首地址从300开始，数据长度为100。

```
int sam=400;//AWG waveform data length
int interval=200;//AWGdata interval time, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],100,1,-5,0,180);
// Set a sinewave data array, the start address starts from 0, data size is 100.

STSAWGCreateTriangleData(&awg_pattern[100],100,4,-5,0,0);
//Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200],100,4,-5,0,50);
//Set a square wave data array, the start address starts from 200, data size is 100.

STSAWGCreateRampData(&awg_pattern[300],100,1,1,5);
//Set an ramp data array, the start address starts from 300, data size is 100.

fovie0.AwgLoader("Vcc_awg",FV,FOVLe_10V,FOVLe_10MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Vcc_awg'. Set to FV mode.

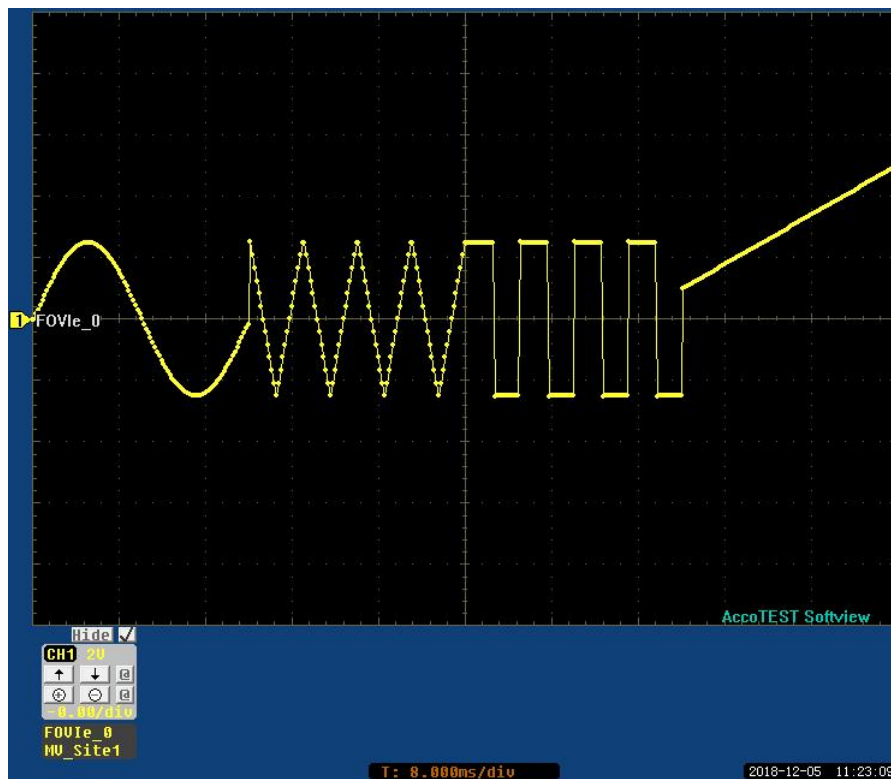
fovie0.Set(FV, 0, FOVLe_10V, FOVLe_10MA, FOVLe_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

fovie0.AwgSelect("Vcc_awg", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fovie0.MeasureVI(sam, interval, FOVLe_MI_X1, MEAS_AWG);
```

```
STSEnableAWG(&fovie0);//enable AWG pattern for fovie0
STSEnableMeas(&fovie0);//enable measurement for fovie0
STSAWGRun();//Enable AWG and measurement synchronously
```

组合之后的波形如下：



6.5.22.2 生成正弦波测试纹波抑制比

测试条件：

fovie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。测试 fovie1 输出纹波抑制比。

```
int Datacount=0;
double AD_SAMPLE=500;//sampling points
double sum[4]={0.0};
double temp[4]={0.0},temp2[4]={0.0};
int sam = 830; //data size of AWG pattern
int interval = 10; // interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
double data[5000]={0.0};
double result[4]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3

fovie0.AwgLoader("Sin_AWG",FV,FOVle_5V,FOVle_1A,awg_pattern,sam);
// pattern named as 'Sin_AWG, FV mode
```

```
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'

fovie0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
delay_ms(1);

fovie1.Set(FI, -100e-3, FOVle_5V, FOVle_100MA, FOVle_RELAY_ON);
//Vout FI mode, prepares to measure
delay_ms(2);
fovie1.MeasureVI(AD_SAMPLE, 20); //fovie1 measurement

for (i=0;i<SITENUM;i++)
{
for(Datacount=0;Datacount<AD_SAMPLE;Datacount++)
{
data[Datacount]= fovie1.GetMeasResult(i, MVRET, Datacount);
//fovie1acquires every voltage values from 500 sampling points
result[i]= fovie1.GetMeasResult(i, MVRET);
//fovie1acquiresaverage voltage of 500 sampling points
sum[i]+=(data[Datacount]-result[i])*(data[Datacount]-result[i]);
}
temp[i]=(sqrt(sum[i]/AD_SAMPLE))*2.828;
//calculate peak to peak value of sine wave received from output
temp2[i]=-20*log10((temp[i]+1e-12)/0.66);
//calculate PSRR
sum[i]=0;
}

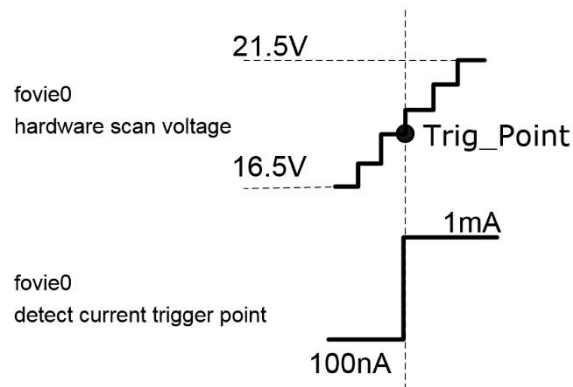
for(i=0; i<SITENUM; i++)
{
PSRR->SetTestResult(i, 0, temp2[i]);
}

fovie1.Set(FI, 0, FOVle_20V, FOVle_100MA, FOVle_RELAY_OFF);
fovie0.AwgStop();//stop running AWG waveform of fovie0
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_OFF);
```

6.5.22.3 同一路生成 AWG 且作为触发源

测试条件:

建立一个波形数据组，fovie0从16.5V上升到21.5V，步距0.1V。测试fovie0电流大于200uA时的跳变点的电压值。



```
double Trig = 200e-6; //trigger current level
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 50; //data size of AWG pattern
int interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5);
//Set an ramp data array, the start address starts from 0, data size is 50.

fovie0.AwgLoader("Vst",FV,FOVLe_40V,FOVLe_100MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Vst'. Set to FV mode.

fovie0.Set(FV, 16.5, FOVLe_40V, FOVLe_100MA, FOVLe_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'

fovie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
//Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fovie0.MeasureVI(sam, interval, FOVLe_MI_X1, MEAS_AWG);

fovie0.SetMeasITrig(Trig,TRIG_RISING); // trigger value is 200uA, rising edge

STSEnableAWG(&fovie0); //enable AWG pattern for fovie0
STSEnableMeas(&fovie0); //enable measurement for fovie0
STSAWGRun(); //Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] =fovie0.GetMeasResult(i,MIRET,TRIG_RESULT);
    //Get the position of trigger point

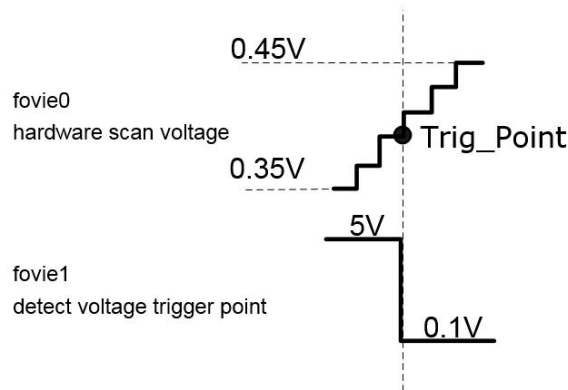
    result[i] = fovie0.GetMeasResult(i,MVRET,Trig_Point[i]);
    //Read the voltage value on trigger position

    Vst->SetTestResult(i, 0, result[i]);
}
}
```

6.5.22.4 一路源生成 AWG 另一路作为触发源

测试条件:

建立一个波形数据组，fovie0从0.35V上升0.45V，步距0.001V。测试fovie1端的电压小于2V时，fovie0跳变点的电压值。



```
double Trig = 2; //trigger voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};

fovie1.Set(FI, 0, FOVle_10V, FOVle_1MA, FOVle_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0],sam,1,0.35,0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

fovie0.AwgLoader("Vcspre_AWG",FV,FOVle_1V,FOVle_100MA,awg_pattern,sam);
//import the awg_pattern array into fovie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

fovie0.Set(FV, 0.30, FOVle_1V, FOVle_100MA, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

fovie0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

fovie1.SetMeasVTrig(Trig,TRIG_FALLING); // trigger value is 2V, Falling edge

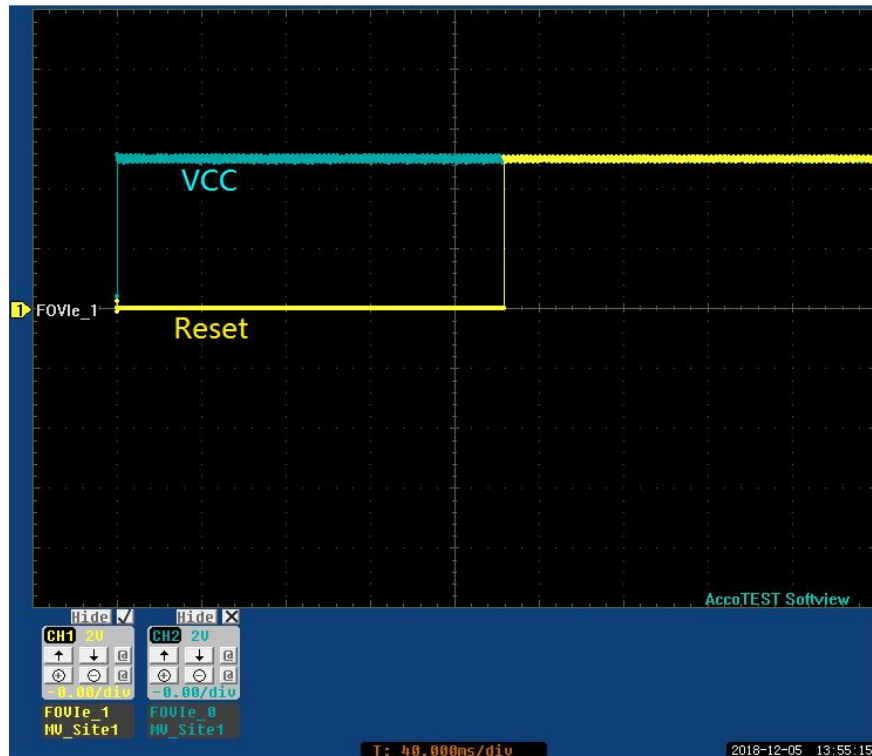
fovie0.MeasureVI(sam, interval, FOVle_MI_X1, MEAS_AWG);
fovie1.MeasureVI(sam, interval, FOVle_MI_X1, MEAS_AWG);
STSEnableAWG(&fovie0); //enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1); //enable measurement for fovie0 and fovie1
STSAWGRun(); //Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i]=fovie1.GetMeasResult(i,MVRET,TRIG_RESULT);
    //Get the position of trigger point for fovie1
    result[i] = fovie0.GetMeasResult(i,MVRET,Trig_Point[i]);
    //Read the voltage value of fovie0 on trigger position
    Vcspre->SetTestResult(i, 0, result[i]);
}
```

6.5.22.5 测试传输延时

测试条件:

基于MAX809测试传输延时。设置VCC（fovie0）从0上升到5V，同步测量VCC（fovie0）和Reset（fovie1）的波形，查找Reset端电压Trigger跳变高于2V的跳变点Trig_Point，则 $(\text{Trig_Point}-1) * \text{Interval_Meas}$ 即为传输延时时间。



```
int sam = 2; // data size of AWG pattern
int interval = 200; //interval time of AWG pattern, unit is 'uS'
int interval_Meas = 200; // interval time for measurement
double awg_pattern[100]={0.0};
double Trig = 2; //Trigger level
double Trig_Point[4]={0};

awg_pattern[0]=0;
awg_pattern[sam-1]=5;

fovie1.Set(FI, 0, FOVie_10V, FOVie_1MA, FOVie_RELAY_ON); //Reset FI mode, prepare to measure
delay_ms(1);

fovie0.AwgLoader("VCC_AWG",FV,FOVie_10V,FOVie_100MA,awg_pattern,sam);
//load the data of "awg_pattern"(sam=2), named "VCC_AWG"

fovie0.Set(FV, 0, FOVie_10V, FOVie_100MA, FOVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'VCC_AWG' of its
'AwgLoader()'
```

```
fovie0.AwgSelect("VCC_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'VCC_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fovie1.SetMeasVTrig(Trig,TRIG_RISING); //trigger value is 2V, Rising edge

fovie0.MeasureVI(2000, interval, FOVIe_MI_X1, MEAS_AWG);
//fovie0 AWG Mode, SampleTimes=2000, samplePeriod=200uS.

fovie1.MeasureVI(2000, interval, FOVIe_MI_X1, MEAS_AWG);
//fovie1 AWG Mode, SampleTimes=2000, samplePeriod=200uS.

STSEnableAWG(&fovie0); //enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1); //enable measurement for fovie0 and fovie1
STSAWGRun(); //Enable AWG and measurement synchronously

for(int i=0; i<SITENUM; i++)
{
    Trig_Point[i]=fovie1.GetMeasResult(i, MVRET, TRIG_RESULT);
    //Get the position of trigger point for fovie1

    T_delay->SetTestResult(i, 0, (Trig_Point[i]-1)*interval_Meas/1000); //mS
}

fovie0.Set(FV, 0, FOVIe_10V, FOVIe_100MA, FOVIe_RELAY_ON);
fovie1.Set(FV, 0, FOVIe_10V, FOVIe_1MA, FOVIe_RELAY_ON);
fovie0.Set(FV, 0, FOVIe_10V, FOVIe_100MA, FOVIe_RELAY_OFF);
fovie1.Set(FV, 0, FOVIe_10V, FOVIe_1MA, FOVIe_RELAY_OFF);
```


6.6 HVLe

6.6.1 HVLe()

```
HVLe(char*channelList, char*chName = NULL);
```

Parameters

channelList

HVLe 绑定通道字符串。形如“S3_0”，其中“S3”表示槽位号 3（S 即 Slot），“0”表示板内序号。

注意：

HVLe 一块板卡最多 1 个通道，因此板内序号最大值为 0。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 HVLe_xx。

Remarks

定义一路 HVLe 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 4 块 HVLe（分别位于 Slot3、Slot13、Slot20 和 Slot30），四工位并行工作，定义可使用的 HVLe。

```
HVLe hvie0("S3_0,S13_0,S20_0,S30_0");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_HVLe, SITE_1, "S3_0");           // set Slot3 ch0 to SITE1
STSSetMultiSiteBind(MD_HVLe, SITE_2, "S13_0");          // set Slot13 ch0 to SITE2
STSSetMultiSiteBind(MD_HVLe, SITE_3, "S20_0");          // set Slot20 ch0 to SITE3
STSSetMultiSiteBind(MD_HVLe, SITE_4, "S30_0");          // set Slot30 ch0 to SITE4
```

6.6.2Set()

```
int Set(  
VIMode viMode,  
double setValue,  
HVIE_VRNG vRange,  
HVIE_IRNG iRange,  
HVIE_OUT_RELAY relayStatus = HVIE_RELAY_HOLD,  
double risingTime = 0.3);
```

Parameters

viMode

HVIE 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-10~1000 或-1000~10。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.01~0.01。

vRange

HVIE 电压量程。

HVIE_N10P1000V

HVIE_P10N1000V

HVIE_N10P500V

HVIE_P10N500V

HVIE_N10P200V

HVIE_P10N200V

HVIE_N10P100V

HVIE_P10N100V

HVIE_N10P10V

HVle_P10N10V

注意：

HVle_N10P1000V，HVle_N10P500V，HVle_N10P200V，HVle_N10P100V 和 HVle_N10P10V 这几个档位对应的电源输出极性为正极性，此时负向最大输出电压为 -10V，这几个档位在 FV0 ~ -10V 时，无法满载输出；

HVle_P10N1000V，HVle_P10N500V，HVle_P10N200V，HVle_P10N100V 和 HVle_P10N10V 这几个档位对应的电源输出极性为负极性，此时正向最大输出电压为 10V，这几个档位在 FV0 ~ 10V 时，无法满载输出。

iRange

HVle 电流量程。

HVle_10MA

HVle_1MA

HVle_100UA

HVle_10UA

HVle_1UA

注意：

HVle_10UA 和 HVle_1UA 电流档位仅能用于测量。

relayStatus

设置输出继电器如何动作，分为五种情况：

HVle_RELAY_ON：打开该通道默认的输出继电器；

HVle_RELAY_FANOUT_ON：打开该通道的 FANOUT 输出继电器；

HVle_RELAY_ALL_ON：打开该通道所有的输出继电器；

HVle_RELAY_OFF：关闭所有的输出继电器；

HVle_RELAY_HOLD：输出继电器保持上一个状态；

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默

认为 0.3mS;

- 1) 该参数设置小于等于 0.1mS 时, risingTime=0.1mS, 为 Fastest 模式;
- 2) 该参数设置小于等于 0.2mS 时, risingTime=0.2mS, 为 Fast 模式;
- 3) 该参数设置小于等于 0.3mS 时, risingTime=0.3mS, 为 Normal 模式;
- 4) 该参数设置小于等于 0.4mS 时, risingTime=0.4mS, 为 Slow 模式;
- 5) 该参数设置大于 0.4mS 时, 自动启动 Capload 功能, risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿, 并且上升或者下降时间可以根据需要来设定, Capload 的最大值为 65mS。

Remarks

设置 HVLe 的状态, 包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

例 1: 设置 hvie0 为恒压模式, 输出电压值为 650V, 使用 1000V 电压量程档, 测试流量程为 1UA 档, 闭合默认的输出继电器, risingTime 设置为 1mS。

```
// Set the voltage to 650V
hvie0.Set(FV, 650, HVLe_N10P1000V, HVLe_1UA, HVLe_RELAY_ON, 1);
```

6.6.3 SetSyn()

```
int SetSyn(
VIMode viMode,
const double* setValue,
UINT siteSize,
HVLe_VRNG vRange,
HVLe_IRNG iRange,
HVLe_OUT_RELAY relayStatus = HVLe_RELAY_HOLD,
double risingTime = 0.3);
```

Parameters

viMode

HVLe 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-10~1000 或 -1000~10。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-0.01~0.01。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

HVLe 电压量程。

HVLe_N10P1000V

HVLe_P10N1000V

HVLe_N10P500V

HVLe_P10N500V

HVLe_N10P200V

HVLe_P10N200V

HVle_N10P100V

HVle_P10N100V

HVle_N10P10V

HVle_P10N10V

注意：

HVle_N10P1000V，HVle_N10P500V，HVle_N10P200V，HVle_N10P100V 和 HVle_N10P10V 这几个档位对应的电源输出极性为正极性，此时负向最大输出电压为 -10V，这几个档位 FV0 ~ -10V 时，无法满载输出；

HVle_P10N1000V，HVle_P10N500V，HVle_P10N200V，HVle_P10N100V 和 HVle_P10N10V 这几个档位对应的电源输出极性为负极性，此时正向最大输出电压为 10V，这几个档位 FV0 ~ 10V 时，无法满载输出。

iRange

HVle 电流量程。

HVle_10MA

HVle_1MA

HVle_100UA

HVle_10UA

HVle_1UA

注意：

HVle_10UA 和 HVle_1UA 电流档位仅能用于测量。

relayStatus

设置输出继电器如何动作，分为五种情况：

HVle_RELAY_ON：打开该通道默认的输出继电器；

HVle_RELAY_FANOUT_ON：打开该通道的 FANOUT 输出继电器；

HVle_RELAY_ALL_ON：打开该通道所有的输出继电器；

HVle_RELAY_OFF：关闭所有的输出继电器；

HVle_RELAY_HOLD：输出继电器保持上一个状态；

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默认为 0.3mS；

- 1) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Fastest 模式；
- 2) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Fast 模式；
- 3) 该参数设置小于等于 0.3mS 时，risingTime=0.3mS，为 Normal 模式；
- 4) 该参数设置小于等于 0.4mS 时，risingTime=0.4mS，为 Slow 模式；
- 5) 该参数设置大于 0.4mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1: hvie0 通道的 SITE1 设置其输出 30V，SITE2 设置其输出 50V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

```
double setValue[2] = {30, 50};  
int siteSize = 2;  
  
hvie0.SetSyn(FV, setValue, siteSize, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);
```

例 2: hvie0 通道的 SITE1 设置其输出 30V，SITE2 设置其输出 50V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 Normal 模式。

```
double setValue[2] = {30, 50};  
int siteSize = 2;  
  
hvie0.Set(FV, 0, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);  
delay_ms(1);  
hvie0.SetSyn(FV, setValue, siteSize, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON, 0.3);
```


6.6.4 SetClamp()

```
int SetClamp(  
double percent_PFS,  
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：10~102。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：10~102。

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-1：函数调用失败。

Remarks

设置 HV1e 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 102%。电压档位极性切换时，箝位值会自动还原成 102%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 hvie0 为 FI=-100uA 模式，箝位到(50, 20)；然后设置 hvie0 到 FV=100V 模式，箝位到(80, 50)。

```
hvie0.Set(FI, 0, HVLe_P10N1000V, HVLe_1MA, HVLe_RELAY_ON);
//Before functionSetClamp, first switch the gear to the FI or FV state of the target
hvie0.SetClamp(50, 20);
//positive clamp to 50% of full range(1000V),negative clamp to 20% of full range(-1000V)
hvie0.Set(FI, -100e-6, HVLe_P10N1000V, HVLe_1MA, HVLe_RELAY_ON);
hvie0.MeasureVI(30, 20);

hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
//switch from FI to FV mode, then positive/negative clamps recover back to 102% of full range

hvie0.SetClamp(80, 50);
// reset positive clamp to 80% of full range(HVLe_1MA), negative clamp to 50% of full range
hvie0.Set(FV, 100, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
```

例 2：设置 hvie0 为 FV=600V 模式，电流档位为 10MA，箝位到(80, 50)；然后改变电流档为 1UA，测量之后设置 hvie0 到 FV=0 模式，箝位到(100, 100)。

```
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_10MA, HVLe_RELAY_ON);
hvie0.SetClamp(80, 50);
// positive clamp to 80% of full range(HVLe_10MA), negative clamp to 50% of full range
hvie0.Set(FV, 600, HVLe_N10P1000V, HVLe_10MA, HVLe_RELAY_ON);
delay_ms(10);
hvie0.Set(FV, 600, HVLe_N10P1000V, HVLe_1UA, HVLe_RELAY_ON);
//still in FV mode, clamp setting(80,50) will keep unchanged,positive clamp to 80% of full
range(HVLe_1UA), Negative clamp to 50% of full range
delay_ms(10);

hvie0.MeasureVI(100, 20);

hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1UA, HVLe_RELAY_ON);
delay_ms(2);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_10MA, HVLe_RELAY_ON);
hvie0.SetClamp(100, 100); // after FV=0, set clamps back to 100%
delay_ms(10);
```

6.6.5 MeasureVI()

```
int MeasureVI(
    UINT sampleTimes,
    double samplePeriod,
    HVLe_MI_GAIN miGain = HVLe_MI_X1,
    FLOATMEASMODE measMode = MEAS_NORMAL,
    UINT T1 = 0,
    UINT T2 = 0,
```

```
UINT T3 = 0,  
UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~8192。

注意：

当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；

当 $sampleTimes > 8192$ 时， $sampleTimes = 8192$ 。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：5~30000。

注意：

当 $samplePeriod < 5$ 时， $samplePeriod = 5$ ；

当 $samplePeriod > 30000$ 时， $samplePeriod = 30000$ 。

miGain

HVle 测流增益。

HVle_MI_X1

HVle_MI_X10

该参数可以不写，默认值为 HVle_MI_X1。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数可以不写，默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 HV1e 的测量次数、测量间隔、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1：设置 hvie0 为普通模式测量，采样 20 次，间隔 10uS。

```
hvie0.MeasureVI(20,10);
```

6.6.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

retType

选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

- 1) HVIE 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。

2)当参数 sampleNumber 选择 TRIG_RESULT 时, 如果没有发生触发, 则函数返回值为-3, 如果测量模式与回读模式不符, 则函数返回值为-5。

Example

例 1: 设置 hvie0 输出电流-500UA, 获取其电压 100 次采样的平均值。

```
hvie0.SetClamp(90, 90);
hvie0.Set(FI, -500e-6, HVIE_P10N1000V, HVIE_1MA, HVIE_RELAY_ON);
hvie0.MeasureVI(100, 30); // Select the pressure measurement mode and sample 100 times. The interval
                             between each sample is 30uS
for(int siteID=0; siteID <4; siteID ++ )
{
    adresult [siteID]= hvie0.GetMeasResult(siteID); // Gets the average value of 100 samples
    param->SetTestResult(siteID, 0, adresult [siteID]);
}
hvie0.Set(FI, 0, HVIE_P10N1000V, HVIE_1MA, HVIE_RELAY_OFF); // Disconnect HVIE
```

例 2: 设置 hvie0 输出电压 600V, 获取其电流 100 次采样的结果。

```
double adresult[100][SITENUM]={0};
hvie0.Set(FV, 600, HVIE_N10P1000V, HVIE_1MA, HVIE_RELAY_ON);
delay_ms(10);
hvie0.MeasureVI(100, 20); // sampleNumbe=100
for(int siteID = 0; siteID < SITENUM; siteID ++ )
{
    for(int i=0; i<100; i++)
    {
        adresult[i][siteID] = hvie0.GetMeasResult(siteID, i);
    }
}
hvie0.Set(FV, 0, HVIE_N10P1000V, HVIE_1MA, HVIE_RELAY_OFF);
delay_ms(1);
```

6.6.7 BlockRead()

```
int BlockRead(
UINT siteCount,
UINT start,
UINT size,
double* buffer,
MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1)buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200]={0.0};
hvie0.MeasureVI(200, 10);
for(i=0; i<SITENUM; i++)
{
    hvie0.BlockRead(i, 0, 200, result[i]);
}
```

6.6.8Pulse()

```
int Pulse(  
double pulseValue,  
UINT pulseTime = 300,  
int measStartTime = 0,  
UINT sampleTimes = 10,  
double sampleInterval = 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-10~1000 或-1000~10。

选择 FI 模式时，单位 A。最大范围为-0.01~0.01。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~8192。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。最小采样间隔为 5uS，分辨率 1uS。
可以不写，默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

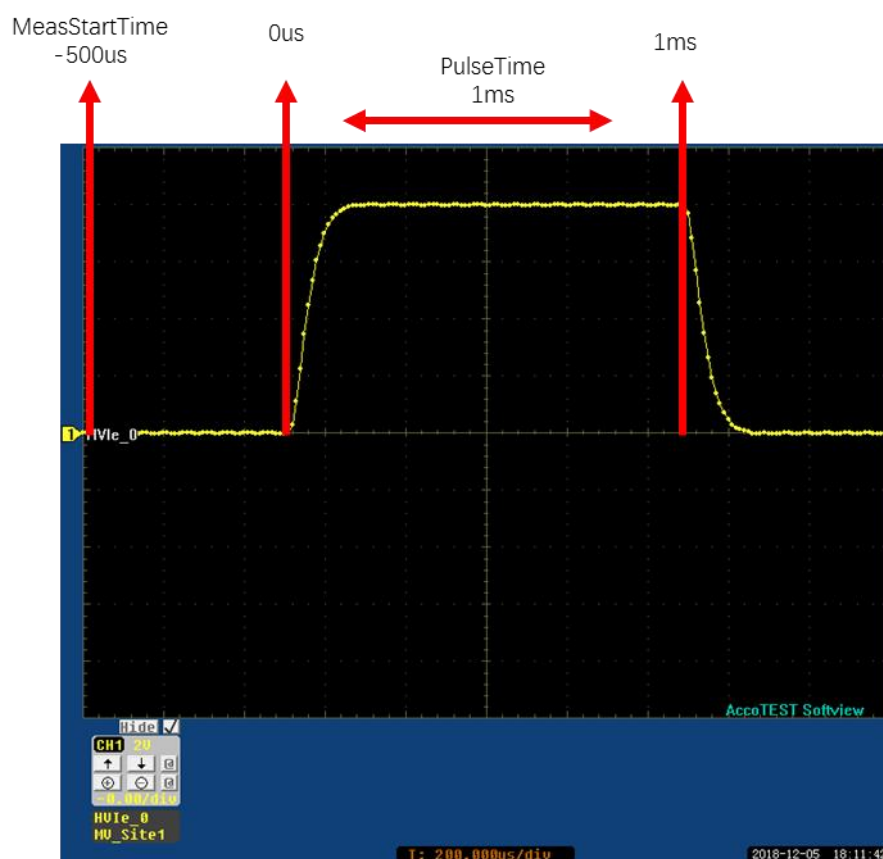
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 hvie0 起始状态 FV 为 0。设置 hvie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
// The set function is used to set the voltage of hvie0 as 0, voltage range as 10V and current gear as 1mA, and  
output relay state as RELAY_ON  
hvie0 .Set(FV,0,HVle_N10P10V,HVle_1MA,HVle_RELAY_ON);  
hvie0.Pulse (8,1000, -500, 200, 10);
```

使用 SoftView 软件抓取的波形如下:



6.6.9 AwgLoader()

```
int AwgLoader(
char* awgName,
VIMode viMode,
HVIe_VRNG vRange,
HVIe_IRNG iRange,
double* awgData,
UINT awgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

HVIe 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

HVLe 电压量程。

HVLe_N10P1000V

HVLe_P10N1000V

HVLe_N10P500V

HVLe_P10N500V

HVLe_N10P200V

HVLe_P10N200V

HVLe_N10P100V

HVLe_P10N100V

HVLe_N10P10V

HVLe_P10N10V

iRange

HVLe 电流量程。

HVLe_10MA

HVLe_1MA

HVLe_100UA

HVLe_10UA

HVLe_1UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~8192。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信

息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 HVLe 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 8192 个点的数组，执行 AwgLoader 函数需要 10mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 HVLe 通道每次导入的 AWG 波形总长度必须小于等于 8192，超出 8192 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 HVLe 通道导入的多个 AWG 波形总长度小于 8192。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 HVLe 通道导入的多个 AWG 波形总长度大于 8192。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，hvle0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50;           // The length of AWG waveform data
int interval = 200;     // AWGdata interval time, Unit uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5); // Set an array of Ramp,the first address
starts at 0,the data length is 50,,The period number is 1, the initial value is 16.5v, and the end value is 21.5v.
hvle0.AwgLoader("Vst ", FV, HVLe_N10P100V, HVLe_1MA, awg_pattern, sam);
// Import the first 50 points in the waveform array awg_pattern and name the waveform Vst.
```

例 2：建立一个波形数据组，hvle0 输出频率为 100Hz、幅值为 2V、直流偏置为 10V 的正弦波。

```
int sam = 1000;         //The length of AWG waveform data
int interval = 10;      //AWGdata interval time, Unit uS
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0);          //Set a sinusoidal wave data set with a
frequency of 100Hz, the first address starts from 0, the data length is 1000, the number of cycles is 1, the
peak-to-peak value is 4V, the dc offset is 10V, and the phase is 0 degrees
```

```
hvie0.AwgLoader("Sine",FV,HVLe_N10P100V,HVLe_1MA,awg_pattern,sam); //Import the first 1000  
points in the waveform array awg_pattern, and name the waveform name Sine
```

6.6.10 AwgSelect()

```
int AwgSelect(  
char*awgName,  
UINTstartAddr,  
UINTstopAddr,  
int loopBackAddr,  
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 HVLe 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;          //data size for AWG pattern
int sam2 = 500;           // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,16.5,21.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,16.5,10.5);

// pattern named as 'AWG_Vst', FV mode
hvie0.AwgLoader("AWG_Vst",FV,HVLe_N10P100V,HVLe_1MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
hvie0.AwgLoader("AWG_Vuvlo",FV,HVLe_N10P100V,HVLe_1MA,awg_pattern2,sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of
its 'AwgLoader()'
hvie0.Set(FV, 16.5, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
hvie0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
```

```
.....  
STSAWGRun(); //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.6.11 AwgRun()

```
int AwgRun(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0,  
BOOL runMode = AWG_SINGLE,  
UINT delayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 AWG_SINGLE。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 HV1e 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRUN。

Example

例 1: hvie0 输出 20mS 的 200Hz 正弦波，幅值 100V，直流偏置为 150V。

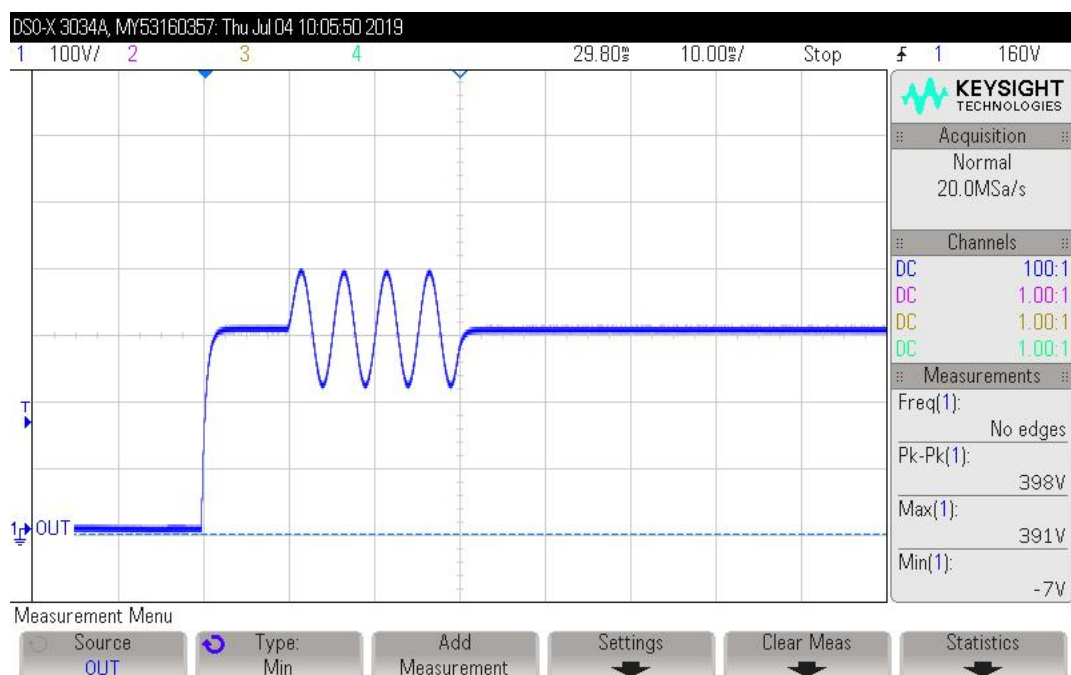
```
double sam = 500; // size of AWG pattern
double interval = 10; // interval time of AWG pattern, unit is uS
double awg_pattern[1000] = { 0.0 };

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 100, 150, 0);

// pattern named as 'Sin_AWG, FV mode
hvie0.AwgLoader("Sin_AWG", FV, HVLe_N10P1000V, HVLe_1MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.305);
delay_ms(1);
hvie0.Set(FV, 150, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.305);
// Set dc bias to 300V
delay_ms(10);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
hvie0.AwgRun("Sin_AWG", 0, (sam - 1), (sam - 1), interval, AWG_LOOP);
delay_ms(20); // The AWG waveform of hvie0 lasts for 20mS
hvie0.AwgStop(); // Stop the loop function of hvie0's AWG
delay_ms(1);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.305);
delay_ms(1);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_OFF, 0.305);
```

下图为示波器抓取的实际波形。



6.6.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 HVLe 通道的 AWG 循环运行。

Example

例 1: hvie0 生成一个周期为 5mS 的正弦波，循环运行该正弦波，15mS 之后停止运行。

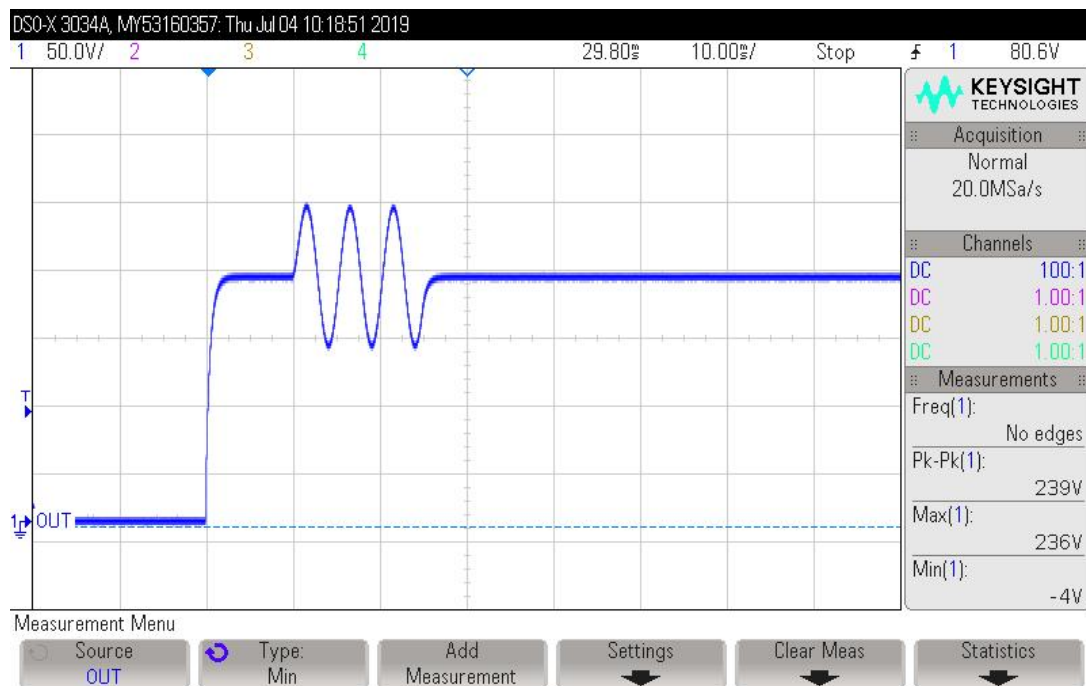
```
double sam = 500;          // size of AWG pattern
double interval = 10;      // interval time of AWG pattern, unit is uS
double awg_pattern[1000] = { 0.0 };

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 120, 180, 0);

// pattern named as 'Sin_AWG, FV mode
hvie0.AwgLoader("Sin_AWG", FV, HVLe_N10P1000V, HVLe_1MA, awg_pattern, sam);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.35);
delay_ms(20);

hvie0.Set(FV, 180, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.35);
// Set dc bias to 300V
delay_ms(10);
//AWGloop 功能
hvie0.AwgRun("Sin_AWG", 0, (sam - 1), 0, interval, AWG_LOOP);
delay_ms(15);          // The AWG waveform of hvie0 lasts for 15mS
hvie0.AwgStop();       // Stop the loop function of hvie0's AWG
delay_ms(1);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON, 0.35);
delay_ms(1);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_OFF, 0.35);
```

下图为示波器抓取的实际波形。



6.6.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 HVLe 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 hvie0 通道的 AWG 数据。

```
hvie0.AwgClear(); //Clear AWG data from hvie0
hvie0.AwgLoader("P1",FV,HVLe_N10P100V,HVLe_1MA,awg_pattern1,sam);
```

6.6.14 SetMeasVTrig()

```
int SetMeasVTrig(
double vTrig,
TRIG_MODE trigMode = TRIG_FALLING,
```

```
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

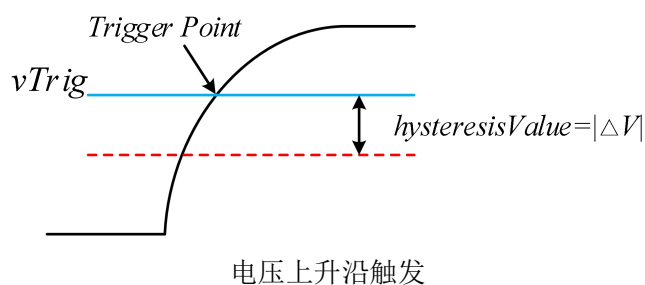
hysteresisValue

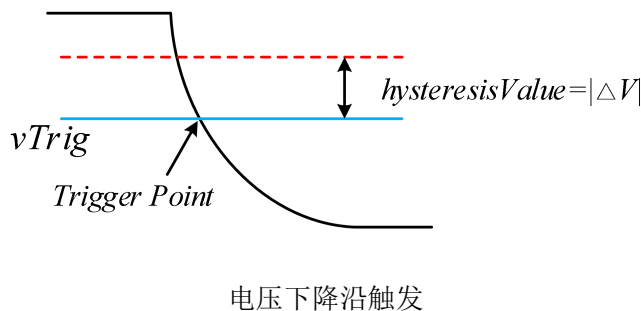
double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($vTrig - hysteresisValue$) 值之后又大于 *vTrig* 值时，此时在 *vTrig* 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($vTrig + hysteresisValue$) 值之后又小于 *vTrig* 值时，此时在 *vTrig* 处判定为触发。





Remarks

电压测量触发设置。

注意：

SetMeasureVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1：建立一个波形数据组，hvie0 从 10.5V 下降到 8V，步距 0.1V。设置 hvie1 下降沿触发，触发电平 2V。

```
//Set hvie1 as constant current voltage measurement state, voltage tap HVLe_N10P100V, and current tap
HVLe_1mA
hvie1.Set(FI, 0, HVLe_N10P100V, HVLe_1mA, HVLe_RELAY_ON);

double Trig = 2;    //Trigger level value
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 25;       //The length of AWG waveform data
double interval = 200;    // AWGdata interval time, Unit uS
double awg_pattern[100]={0.0};

//A ramp wave data set is set. The first address starts from 0, the data length is 25, the number of cycles is 1,
the starting value is 10.5v, and the end value is 8V
STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);

//Import the first 25 points in the waveform array awg_pattern and name the waveform Uvlo
hvie0.AwgLoader("Uvlo",FV,HVLe_N10P100V,HVLe_1mA,awg_pattern,sam);
//Set the Set statement's range tap to be the same as the range tap in the selected AwgLoader
hvie0.Set(FV, 10.5, HVLe_N10P100V, HVLe_1mA,HVLe_RELAY_ON);
//Set the waveform starting point to 0, ending point to sam-1, and stopping point to sam-1.AWG data interval
is 200uS
hvie0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
//Set hvie1 voltage jump point to 2V, and the descending edge is triggered
hvie1.SetMeasVTrig(Trig, TRIG_FALLING);
//Hvie0 is AWG measurement mode with 25 samples at an interval of 200uS
hvie0.MeasureVI(sam, interval, HVLe_MI_X1, MEAS_AWG);
//Set to AWG measurement mode, sampling 25 times with an interval of 200uS
hvie1.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&hvie0);           // hvie0 enable synchronization of AWG
STSEnableMeas(&hvie0, &hvie1); // hvie0 and hvie1 enable simultaneous measurements
STSAWGRun();                     //AWG starts synchronously
```

设置触发电平的错误做法是：

```
//Set hvie1 voltage jump point to 2V, and the descending edge is triggered
hvie1.SetMeasVTrig(Trig, TRIG_FALLING);
// SetMeasVTrig function and MeasureVI function are intersected with Set statement, and the statement
changes the voltage gear from the initial state of HVLe_N10P100V to HVLe_N10P10V, which will cause the
triggering value calculation error
hvie1.Set(FI, 0, HVLe_N10P10V, HVLe_1MA, HVLe_RELAY_ON);
// hvie0 is AWG measurement mode with 25 samples at an interval of 200uS
hvie0.MeasureVI(sam, interval, HVLe_MI_X1, MEAS_AWG);
// hvie1 is AWG measurement mode with 25 samples at an interval of 200uS
hvie1.MeasureVI(sam, interval, MEAS_AWG);
STSEnableAWG(&hvie0);           // hvie0 enables synchronization of AWG
STSEnableMeas(&hvie0, &hvie1); // hvie0 and hvie1 enable simultaneous measurements
STSAWGRun();                     //AWG starts synchronously
```

6.6.15 SetMeasITrig()

```
int SetMeasITrig(
double iTrig,
TRIG_MODE trigMode = TRIG_FALLING,
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

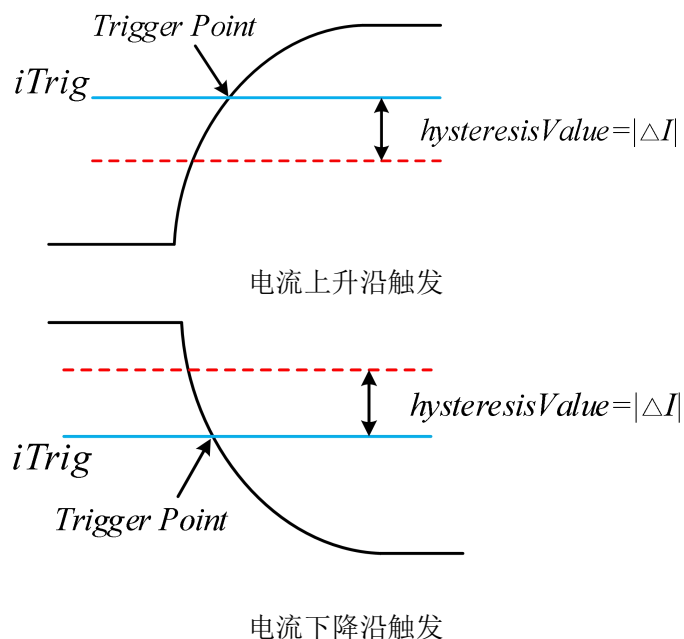
hysteresisValue

double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($iTrig - hysteresisValue$) 值之后又大于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($iTrig + hysteresisValue$) 值之后又小于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发。



Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，hvie0 从 16.5V 上升到 21.5V，步距 0.1V。设置 hvie0 上升沿触发，触发电流值 200uA。

```
double Trig = 200e-6;          // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                  // data size of AWG pattern
int interval = 200;            // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into hvie0 channel, named it as 'Vst'. Set to FV mode.
hvie0.AwgLoader("Vst", FV, HVLe_N10P100V, HVLe_1MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
hvie0.Set(FV, 16.5, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
hvie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
hvie0.MeasureVI(sam, interval, HVLe_MI_X1, MEAS_AWG);

hvie0.SetMeasITrig(Trig, TRIG_RISING);          // trigger value is 200uA, rising edge
STSEnableAWG(&hvie0);                          // enable AWG pattern for hvie0
STSEnableMeas(&hvie0);                          // enable measurement for hvie0
STSAWGRun();                                    // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
hvie0.MeasureVI(sam, interval, HVLe_MI_X1, MEAS_AWG);

//insert a Set() call, change its current range from HVLe_100MA to HVLe_10MA, it will cause a utility error.
hvie0.Set(FV, 16.5, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);

hvie0.SetMeasITrig(Trig, TRIG_RISING);          // trigger value is 200uA, rising edge
STSEnableAWG(&hvie0);                          // enable AWG pattern for hvie0
STSEnableMeas(&hvie0);                          // enable measurement for hvie0
STSAWGRun();                                    // Enable AWG and measurement synchronously
```

6.6.16 SetAlarmMask()

```
void SetAlarmMask (
```



```
UINT siteCount,  
bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
hvie0.SetAlarmMask(0);                    // mask the Alarm information of SITE_1  
// The alarm information about overranging voltage scale is not output  
hvie0.Set(FV, 105, HVLe_N10P100V, HVLe_1MA, HVLe_RELAY_ON);
```

6.6.17 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息;

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = hvic0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.6.18 获取板卡固有信息函数

6.6.18.1 GetBoardSN()

```
int GetBoardSN(  
UINT siteCount,  
char* boardSN,  
UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
hvie0.GetBoardSN(0, boardSN, 255);           // Get the board serial number
```

6.6.18.2 GetBoardHDRev()

```
int GetBoardHDRev(  
    UINT siteCount,  
    char* hardRev,  
    UINT revSize) ;
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char *hardRev*[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡硬件版本号。。

Example

```
char hardRev [255] = {0};  
hvie0.GetBoardHdRev(0, hardRev, 255);           // Get the board hardware revision
```

6.6.19 获取校准信息函数

6.6.19.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = hvie0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.6.19.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = hvie0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.6.19.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = hvie0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.6.19.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS；
CAL_FAIL	校准 FAIL；
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = hvie0.GetCalibrationResult(0);           // Get calibration result of SITE_1
```

6.6.19.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = hvie0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.6.19.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000 Keithley2000

STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0}; // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = hvie0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.6.19.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteCount,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* boardHdRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回“N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回“N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 *boardHdRev* 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = hvie0. GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.6.19.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(  
    UINT siteCount,  
    char* softRev,  
    UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回 “N/A” 。
由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0}; // the buffer for saving softwareversion
// Get the version of calibration software
hvie0.GetCalibrationSoftRev(0, softRev, 255);
```

6.6.19.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
int slotID = hvie0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.6.20 编程范例

6.6.20.1 测试 IDSS

```
double adresult[4] = { 0.0 };
int SITEID = 0;

hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
delay_ms(1);

hvie0.Set(FV, 800, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
//Using 1MA range to set the 800V.
delay_ms(10); //delay for 10mS to ensure 800V is set

hvie0.Set(FV, 800, HVLe_N10P1000V, HVLe_1UA, HVLe_RELAY_ON);
//change 1MA range to 1UA, in order to measure small current.
delay_ms(10);

hvie0.MeasureVI(100, 20); //set sample = 100, interval time = 20uS
for (SITEID = 0; SITEID < SITENUM; SITEID++)
{
    adresult[SITEID] = hvie0.GetMeasResult(SITEID, MIRET); //return the MI average value to adresult
    IDSS_POS->SetTestResult(SITEID, 0, adresult[SITEID] * 1e9); //convert unit from A to nA
}

hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1UA, HVLe_RELAY_ON); //set voltage to 0
delay_ms(2);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON); //set voltage to 0
delay_ms(10); //set delay to wait voltage reduce to 0

hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_OFF); //set output relay to RELAY_OFF
delay_ms(1);
```

6.6.20.2 测试 BVDSS

```
double adresult[4] = { 0.0 };
int SITEID = 0;

hvie0.Set(FI, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
delay_ms(1);

hvie0.Set(FI, 250e-6, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
delay_ms(30);

hvie0.MeasureVI(10, 20); //set sample = 10, interval time = 20uS
for (SITEID = 0; SITEID < SITENUM; SITEID++)
{
    adresult[SITEID] = hvie0.GetMeasResult(SITEID); //return the MV average value to adresult
    BVDSS_POS->SetTestResult(SITEID, 0, adresult[SITEID]); //V
}

hvie0.Set(FI, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON); //set current to 0
delay_ms(5);
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_ON);
delay_ms(10); //set delay to wait voltage reduce to 0
```

```
hvie0.Set(FV, 0, HVLe_N10P1000V, HVLe_1MA, HVLe_RELAY_OFF);//set output relay to RELAY_OFF  
delay_ms(1);
```

6.7 HPVie

6.7.1 HPVie()

```
HPVie(char* channelList, char* chName = NULL);
```

Parameters

channelList

HPVie 绑定通道字符串。形如“S4_0”，其中“S4”表示槽位号 4（S 即 Slot），“0”表示板内序号。

注意：

HPVie 一块板卡最多 1 个通道，因此板内序号最大值为 0。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 HPVie_xx。

Remarks

定义一路 HPVie 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 2 块 HPVie（分别位于 Slot4 和 Slot12），两工位并行工作，定义可使用的 HPVie。。

```
HPVie hpvie0("S4_0,S12_0");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_HPVie, SITE_1, "S4_0");           // set Slot4 ch0 to SITE1  
STSSetMultiSiteBind(MD_HPVie, SITE_2, "S12_0");          // set Slot12 ch0 to SITE2
```

6.7.2Set()

```
int Set(  
VIMode viMode,  
double setValue,  
HPVIE_VRNG vRange,  
HPVIE_IRNG iRange,  
HPVIE_OUT_RELAY relayStatus = HPVIE_RELAY_HOLD,  
double risingTime = 0.2);
```

Parameters

viMode

HPVIE 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-100~100。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-100~100。

vRange

HPVIE 电压量程。

HPVIE_100V

HPVIE_40V

HPVIE_20V

HPVIE_10V

HPVIE_5V

HPVIE_2V

iRange

HPVIE 电流量程。

HPVle_100A

HPVle_50A

HPVle_20A

HPVle_10A

HPVle_2A

HPVle_1A

HPVle_100MA

HPVle_10MA

HPVle_1MA

HPVle_100UA

HPVle_10UA

relayStatus

设置输出继电器如何动作，分为以下几种情况：

HPVle_RELAY_ON：打开输出继电器；

HPVle_RELAY_FANOUT_ON：打开该通道的 FANOUT 输出继电器；

HPVle_RELAY_ALL_ON：以 HPVle_RELAY_ON 的输出继电器为基础，同时输出 HPVle_RELAY_FANOUT_ON 的 Force 继电器，该选择用于 100A 测试连接，减少传输损耗；

HPVle_RELAY_OFF：关闭输出继电器；

HPVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 HPVle 处于独立电压表模式。

HPVle_RELAY_SENSE_FANOUT_ON：启动独立测量模式。只打开 SENSE 线 FANOUT 输出继电器，关闭 FORCE 线输出继电器。此时 HPVle 处于独立电压表模式。

HPVle_RELAY_HOLD：输出继电器保持上一个状态。

risingTime

double 型变量，设置积分时间。单位为 mS，分辨率为 0.1mS(设定值大于 0.5mS 时)。该参数不写，则默认为 0.2mS；

- 1) 该参数设置小于等于 0.1mS 时, risingTime=0.1mS, 为 Fast 模式;
- 2) 该参数设置小于等于 0.2mS 时, risingTime=0.2mS, 为 Normal 模式;
- 3) 该参数设置小于等于 0.5mS 时, risingTime=0.5mS, 为 Slow 模式;
- 4) 该参数设置大于 0.5mS 时, 自动启动 Capload 功能, risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿, 并且上升或者下降时间可以根据需要来设定, Capload 的最大值为 65mS。

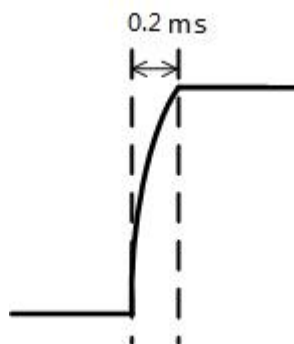
Remarks

设置 HPVle 的状态, 包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

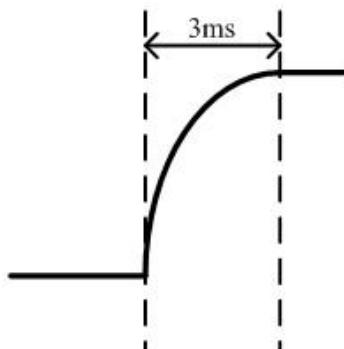
例 1: 设置 hpvie0 电压输出 5V, risingTime 不填则默认为 0.2mS。

```
hpvie0.Set(FV,5,HPVle_10V,HPVle_100MA,HPVle_RELAY_ON);
```



例 2: 设置 hpvie0 电压输出 5V, risingTime 设置为 3mS。

```
hpvie0.Set(FV,0,HPVle_10V,HPVle_100MA,HPVle_RELAY_ON);  
delay_ms(1);  
hpvie0.Set(FV,5,HPVle_10V,HPVle_100MA,HPVle_RELAY_ON, 3);
```



6.7.3 SetSyn()

```
int SetSyn(  
VIMode viMode,  
const double* setValue,  
UINT siteSize,  
HPVIE_VRNG vRange,  
HPVIE_IRNG iRange,  
HPVIE_OUT_RELAY relayStatus = HPVIE_RELAY_HOLD,  
double risingTime = 0.2);
```

Parameters

viMode

HPVIE 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-100~100。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-100~100。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

HPVle 电压量程。

HPVle_100V

HPVle_40V

HPVle_20V

HPVle_10V

HPVle_5V

HPVle_2V

iRange

HPVle 电流量程。

HPVle_100A

HPVle_50A

HPVle_20A

HPVle_10A

HPVle_2A

HPVle_1A

HPVle_100MA

HPVle_10MA

HPVle_1MA

HPVle_100UA

HPVle_10UA

relayStatus

设置输出继电器如何动作，分为以下几种情况：

HPVle_RELAY_ON：打开输出继电器；

HPVle_RELAY_FANOUT_ON：打开该通道的 FANOUT 输出继电器；

HPVle_RELAY_ALL_ON：以 HPVle_RELAY_ON 的输出继电器为基础，同时输出 HPVle_RELAY_FANOUT_ON 的 Force 继电器，该选择用于 100A 测试连接，减少传输损耗；

HPVle_RELAY_OFF：关闭输出继电器；

HPVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，

关闭 FORCE 线输出继电器。此时 HPVie 处于独立电压表模式。

HPVie_RELAY_SENSE_FANOUT_ON: 启动独立测量模式。只打开 SENSE 线 FANOUT 输出继电器，关闭 FORCE 线输出继电器。此时 HPVie 处于独立电压表模式。

HPVie_RELAY_HOLD: 输出继电器保持上一个状态。

risingTime

double 型变量，设置积分时间。单位为 mS，分辨率为 0.1mS(设定值大于 0.5mS 时)。该参数不写，则默认为 0.2mS；

- 1) 该参数设置小于等于 0.1mS 时，risingTime=0.1mS，为 Fast 模式；
- 2) 该参数设置小于等于 0.2mS 时，risingTime=0.2mS，为 Normal 模式；
- 3) 该参数设置小于等于 0.5mS 时，risingTime=0.5mS，为 Slow 模式；
- 4) 该参数设置大于 0.5mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

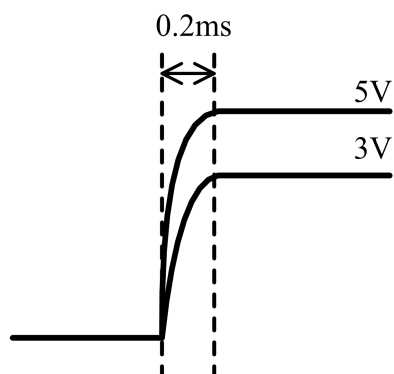
Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1: hpvie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

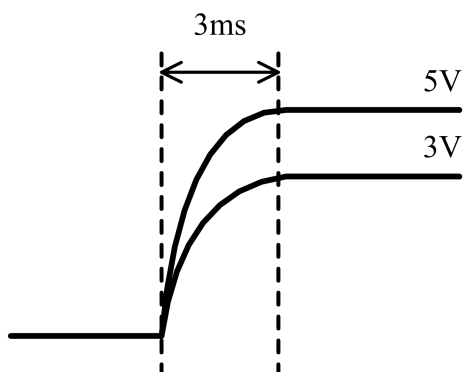
```
double setValue[2] = {3, 5};  
int siteSize = 2;  
  
hpvie0.SetSyn(FV, setValue, siteSize, HPVie_10V, HPVie_100MA, HPVie_RELAY_ON);
```



例 2: hpvie0 通道的 SITE1 设置其输出 3V, SITE2 设置其输出 5V, 且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;

hpvie0.Set(FV, 0, HPVLe_10V, HPVLe_100MA, HPVLe_RELAY_ON);
delay_ms(1);
hpvie0.SetSyn(FV, setValue, siteSize, HPVLe_10V, HPVLe_100MA, HPVLe_RELAY_ON, 3);
```



6.7.4 SetClamp()

```
int SetClamp(
double percent_PFS,
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：10~110。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：10~110。

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-1：函数调用失败。

Remarks

设置 HPVLe 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 110%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 hpvie1 为 FI=0 模式，箝位到(50, 20)，然后切换 hpvie1 为 FV=1V 模式，箝位到(25, 25)。

```
hpvie1.Set(FI, 0, HPVLe_10V, HPVLe_10MA, HPVLe_RELAY_ON);  
// positive clamp to 50% of full range(HPVLe_10V), negative clamp to 20% of full range  
hpvie1.SetClamp(50, 20);  
  
// switch from FI to FV mode, then positive/negative clamps recover back to 110% of full range  
hpvie1.Set(FV, 0, HPVLe_10V, HPVLe_100MA, HPVLe_RELAY_ON);  
// reset positive clamp to 25% of full range(HPVLe_100MA), negative clamp to 25% of full range  
hpvie1.SetClamp(25, 25);  
// Set FV=1V  
hpvie1.Set(FV, 1, HPVLe_10V, HPVLe_100MA, HPVLe_RELAY_ON);
```

例 2：设置 hpvie1 为 FV=1V 模式，箝位到(50, 25)，然后切换 hpvie1 为 FV=35V，

测量结束后设置 hpvie1 为 FV=0，箝位到(100, 100)。

```
hpvie1.Set(FV, 0, HPVIE_10V, HPVIE_100MA, HPVIE_RELAY_ON);  
// positive clamp to 50% of full range(HPVIE_100MA), negative clamp to 25% of full range  
hpvie1.SetClamp(50, 25);  
hpvie1.Set(FV, 1, HPVIE_10V, HPVIE_100MA, HPVIE_RELAY_ON);  
delay_ms(1);  
  
// still in FV mode, clamp setting will keep unchanged  
hpvie1.Set(FV, 35, HPVIE_40V, HPVIE_100MA, HPVIE_RELAY_ON);  
hpvie1.MeasureVI(100, 20);  
  
hpvie1.Set(FV, 0, HPVIE_40V, HPVIE_100MA, HPVIE_RELAY_ON);  
// after FV=0, set clamps back to 100%  
hpvie1.SetClamp(100, 100);
```

6.7.5 MeasureVI()

```
int MeasureVI(  
    UINT sampleTimes,  
    double samplePeriod,  
    HPVIE_MV_GAIN mvGain = HPVIE_MV_X1,  
    HPVIE_MI_GAIN miGain = HPVIE_MI_X1,  
    FLOATMEASMODE measMode = MEAS_NORMAL,  
    UINT T1 = 0,  
    UINT T2 = 0,  
    UINT T3 = 0,  
    UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096。

注意：

当 sampleTimes < 1 时，sampleTimes = 1；

当 sampleTimes > 4096 时，sampleTimes = 4096。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 1uS，分辨率 1uS。范

围：1~30000。

注意：

当 samplePeriod < 1 时，samplePeriod = 1；

当 samplePeriod > 30000 时，samplePeriod = 30000。

mvGain

HPVIE 测压增益。

HPVIE_MV_X1

HPVIE_MV_X2

HPVIE_MV_X5

HPVIE_MV_X10

该参数可以不写，默认值为 HPVIE_MV_X1。

miGain

HPVIE 测流增益。

HPVIE_MI_X1

HPVIE_MI_X2

HPVIE_MI_X5

HPVIE_MI_X10

该参数可以不写，默认值为 HPVIE_MI_X1。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数可以不写，默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个

参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 HPV1e 的测量次数、测量间隔、测压增益、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1：设置 hpvie0 为普通模式测量，采样 20 次，间隔 5uS。

```
hpvie0.MeasureVI(20,5);
```

6.7.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

retType

选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

1)HPVle 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。

2)当参数 *sampleNumber* 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回

值为-3，如果测量模式与回读模式不符，则函数返回值为-5。

Example

例 1：返回 hpvie0 电流测量结果平均值。

```
//hpvie0 sampleTimes=20, samplePeriod=5uS
hpvie0.MeasureVI(20, 5);
for(i=0; i<SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value
    of current
    adresult[i]=hpvie0.GetMeasResult(i, MIRET);
}
```

例 2：返回 hpvie0 电压测量结果第 15 个点的值。

```
//hpvie0 sampleTimes=20, samplePeriod=5uS
hpvie0.MeasureVI(20, 5);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=hpvie0.GetMeasResult(i, MVRET, 14);
}
```

例 3：AWG 同步测量时，返回 hpvie0 两次跳变触发的位置。

```
// hpvie1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG
hpvie1.MeasureVI(50, 20, HPV1e_MV_X1, HPV1e_MI_X1, MEAS_AWG);
// the hpvie0's parameters of MeasureVI function are same with hpvie1
hpvie0.MeasureVI(50, 20, HPV1e_MV_X1, HPV1e_MI_X1, MEAS_AWG);
// the trigger value is 2V, falling edge
hpvie0.SetMeasVTrig(2, TRIG_FALLING);
// hpvie1 AWG enable
STSEnableAWG(&hpvie1);
// hpvie0 and hpvie1 measure enable
STSEnableMeas(&hpvie1, &hpvie0);
STSAWGRun();
for (i = 0; i < SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] = hpvie1.GetMeasResult(0, MVRET, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
    Trig_Point1[i] = hpvie1.GetMeasResult(0, MVRET, TRIG_RESULT, 1);
}
```

6.7.7 BlockRead()

```
int BlockRead(
```

```
UINT siteCount,  
UINT start,  
UINT size,  
double* buffer,  
MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200]={0.0};
hpvie0.MeasureVI(200, 10);
for(i=0; i<SITENUM; i++)
{
    //start=0; size=200; Get MVRET, saved to result.
    hpvie0.BlockRead(i, 0, 200, result[i], MVRET);
}
```

6.7.8Pulse()

```
int Pulse(
double pulseValue,
UINT pulseTime = 300,
Int measStartTime = 0,
UINT sampleTimes= 10,
double sampleInterval= 30,
HPVie_MV_GAIN mvGain = HPVie_MV_X1,
HPVie_MI_GAIN miGain = HPVie_MI_X1););
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-100~100；

选择 FI 模式时，单位 A。最大范围为-100~100。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

实际脉冲建立速度受限于积分时间和电压档位，电压档位越大积分时间越长，设置脉冲时间的最小值需要根据积分时间和电压档位做适当调整。0.2mS 积分时间下，10V 档满量程建立时间为 200uS，100V 档满量程建立时间为 500uS。各电流档位脉冲最大时间受限于 HPVie 输出功率曲线，需要根据 HPVie 输出功率曲线进行设置。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。最小采样间隔为 1uS，分辨率 1uS。

可以不写，默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

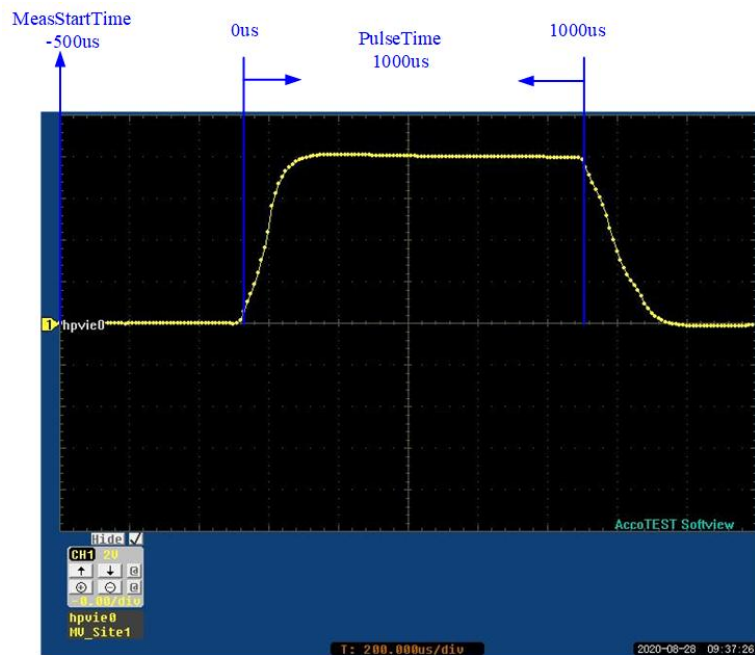
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 hpvie0 起始状态 FV 为 0。设置 hpvie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
// use set() function to set the state of hpvie0
hpvie0.Set(FV, 0, HPVLe_10V, HPVLe_10MA, HPVLe_RELAY_ON);
hpvie0.Pulse(8, 1000, -500, 200, 10);
```

使用 SoftView 软件抓取的波形如下：



6.7.9AwgLoader()

```
int AwgLoader(
char*awgName,
VIMode viMode,
HPVie_VRNG vRange,
HPVie_IRNG iRange,
double* awgData,
UINTawgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

HPVie 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

HPVle 电压量程。

HPVle_100V

HPVle_40V

HPVle_20V

HPVle_10V

HPVle_5V

HPVle_2V

iRange

HPVle 电流量程。

HPVle_100A

HPVle_50A

HPVle_20A

HPVle_10A

HPVle_2A

HPVle_1A

HPVle_100MA

HPVle_10MA

HPVle_1MA

HPVle_100UA

HPVle_10UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信

息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 HPVle 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 HPVle 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 HPVle 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 HPVle 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，hpvie0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50;          // AWG waveform data length
int interval = 200;    // AWG data interval time, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
// pattern array saved in 'awg_pattern', start from 16.5 to 21.5
hpvie0.AwgLoader("Vst", FV, HPVle_40V, HPVle_100MA, awg_pattern, sam);
//pattern named as 'Vst', FV mode
```

例 2：建立一个波形数据组，hpvie0 输出频率为 100Hz、幅值为 2V、直流偏置为 10V 的正弦波。

```
int sam = 1000;        //AWG waveform data length
int interval = 10;     //AWG data interval time, unit is uS
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0);
// set a sinewave data array of 100HZ, Vpp=4, DC Offset=10
hpvie0.AwgLoader("Sine",FV,HPVle_10V,HPVle_1MA,awg_pattern, sam);
```

```
//pattern named as 'SINE', FV mode
```

6.7.10 AwgSelect()

```
int AwgSelect(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 HPVle 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;         //data size for AWG pattern
int sam2 = 500;          // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
double awg_pattern2[500]={0.0};     //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,16.5,21.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,16.5,10.5);

// pattern named as 'AWG_Vst', FV mode
hpvie0.AwgLoader("AWG_Vst",FV,HPVle_40V,HPVle_100MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
hpvie0.AwgLoader("AWG_Vuvlo",FV,HPVle_20V,HPVle_100MA,awg_pattern2,sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of
its 'AwgLoader()'
hpvie0.Set(FV, 16.5, HPVle_40V, HPVle_100MA, HPVle_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
hpvie0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
```

```
.....  
STSAWGRun(); //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.7.11 AwgRun()

```
int AwgRun(  
char* awgName,  
UINT startAddr,  
UINT stopAddr,  
int loopBackAddr,  
double awgInterval= 10.0,  
BOOL runMode = AWG_SINGLE,  
UINT delayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 **AWG_SINGLE**。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 *delayTime_ms* 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，*delayTime_ms* 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 HPV1e 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRUN。

Example

例 1: hpvie0 连续输出 120Hz 正弦波, 幅值 0.33V, 直流偏置为 3.3V。

```
int sam = 830;          //size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);

// pattern named as 'Sin_AWG, FV mode
hpvie0.AwgLoader("Sin_AWG", FV, HPVLe_5V, HPVLe_1A, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
hpvie0.Set(FV, 0, HPVLe_5V, HPVLe_1A, HPVLe_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
hpvie0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
```

6.7.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 HPVLe 通道的 AWG 循环运行。

6.7.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前, 清除该 HPVLe 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 hpvie0 通道的 AWG 数据。

```
hpvie0.AwgClear(); // clear AWG data of hpvie0  
hpvie0.AwgLoader("P1",FV,HPVle_40V,HPVle_100MA,awg_pattern1,sam);
```

6.7.14 SetMeasVTrig()

```
int SetMeasVTrig(  
double vTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue

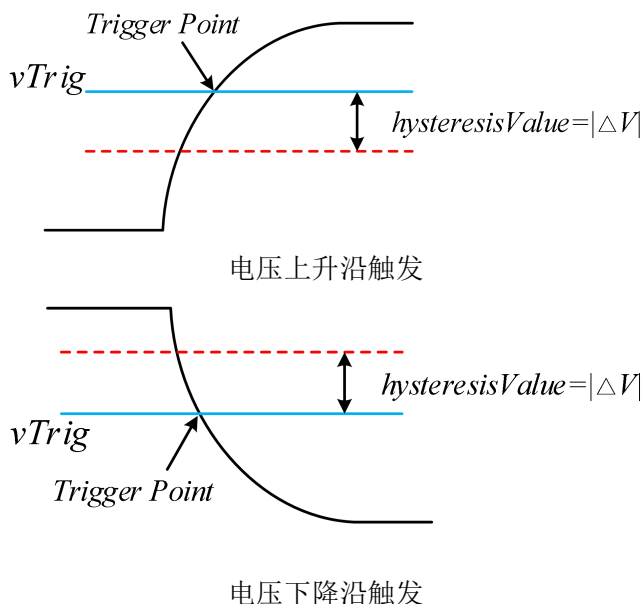
double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (*vTrig*-*hysteresisValue*) 值之后又大于 *vTrig* 值时，此时在 *vTrig* 处判定为触发；

3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($vTrig + hysteresisValue$)值之后又小于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发。



Remarks

电压测量触发设置。

注意：

SetMeasureVTrig 函数与 MeasureVI 函数运行时, 对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1: 建立一个波形数据组, hpvie0 从 10.5V 下降到 8V, 步距 0.1V。设置 hpvie1 下降沿触发, 触发电平 2V。

```
hpvie1.Set(FI, 0, HPVle_10V, HPVle_10MA, HPVle_RELAY_ON);
// set the state of hpvie1

double Trig = 2;    //triggering voltage
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 25;       //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 10.5, 8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.
```

```
hpvie0.AwgLoader("Uvlo", FV, HPVLe_40V, HPVLe_100MA, awg_pattern, sam);  
//import the awg_pattern array into hpvie0 channel, named it as 'Uvlo'. Set to FV mode.  
  
hpvie0.Set(FV, 10.5, HPVLe_40V, HPVLe_100MA, HPVLe_RELAY_ON);  
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'  
  
hpvie0.AwgSelect("Uvlo", 0, sam - 1, sam - 1, interval);  
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

设置触发电平的正确做法是：

```
hpvie0.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);  
hpvie1.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);  
hpvie1.SetMeasVTrig(Trig, TRIG_FALLING); //trigger value is 2V, falling edge  
STSEnableAWG(&hpvie0); //enable AWG pattern for hpvie0  
STSEnableMeas(&hpvie0, &hpvie1); //enable measurement for hpvie0 and hpvie1  
STSAWGRun(); //Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
hpvie0.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);  
hpvie1.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);  
hpvie1.Set(FI, 0, HPVLe_5V, HPVLe_10MA, HPVLe_RELAY_ON);  
//insert a Set() call, change its voltage range from HPVLe_10V to HPVLe_5V, it will cause a utility error.  
  
hpvie1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, falling edge  
STSEnableAWG(&hpvie0); //enable AWG pattern for hpvie0  
STSEnableMeas(&hpvie0, &hpvie1); //enable measurement for hpvie0 and hpvie1  
STSAWGRun(); //Enable AWG and measurement synchronously
```

6.7.15 SetMeasITrig()

```
int SetMeasITrig(  
double iTrig,  
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

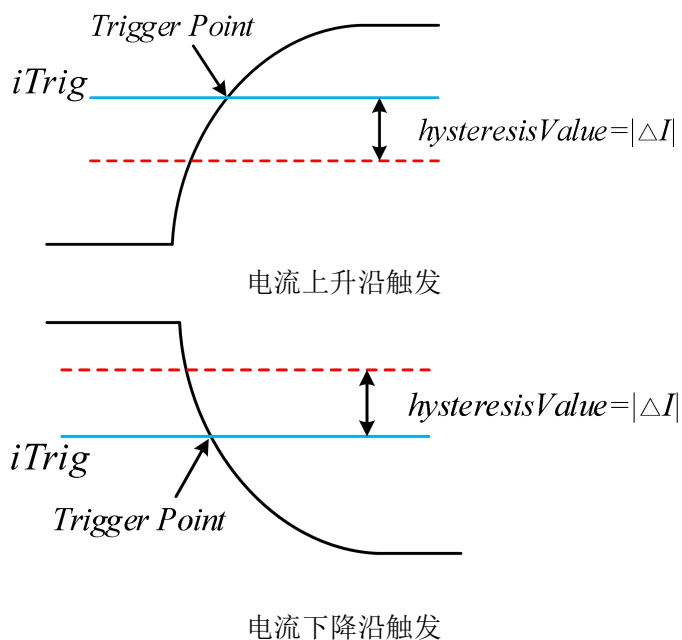
hysteresisValue

double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($iTrig - hysteresisValue$) 值之后又大于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($iTrig + hysteresisValue$) 值之后又小于 $iTrig$ 值时，此时在 $iTrig$ 处判定为触发。



Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，hpvie0 从 16.5V 上升到 21.5V，步距 0.1V。设置 hpvie0 上升沿触发，触发电流值 200uA。

```
double Trig = 200e-6;          // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                  // data size of AWG pattern
int interval = 200;            // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into hpvie0 channel, named it as 'Vst'. Set to FV mode.
hpvie0.AwgLoader("Vst ", FV, HPVLe_40V, HPVLe_100MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
hpvie0.Set(FV, 16.5, HPVLe_40V, HPVLe_100MA, HPVLe_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
hpvie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
```

设置触发电平的正确做法是：

```
hpvie0.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);
hpvie0.SetMeasITrig(Trig, TRIG_RISING);          // trigger value is 200uA, rising edge

STSEnableAWG(&hpvie0);    // enable AWG pattern for hpvie0
STSEnableMeas(&hpvie0);    // enable measurement for hpvie0
STSAWGRun();              // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
hpvie0.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);

//insert a Set() call, change its current range from HPVLe_100MA to HPVLe_10MA, it will cause a utility
error.
hpvie0.Set(FV, 16.5, HPVLe_40V, HPVLe_10MA, HPVLe_RELAY_ON);

hpvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge
STSEnableAWG(&hpvie0);    // enable AWG pattern for hpvie0
STSEnableMeas(&hpvie0);    // enable measurement for hpvie0
STSAWGRun();              // Enable AWG and measurement synchronously
```

6.7.16 KelvinCheck()

```
int KelvinCheck (
HPVie_KELVINMODE checkMode,
HPVie_KELVINCH checkCh,
UINT sampleTimes = 200);
```

Parameters

checkMode

开尔文测试的模式，可设置为：

HPVie_HIGH_SIDE 检测高端电路的连接
HPVie_LOW_SIDE 检测低端电路的连接
HPVie_HIGH_LOW_SIDE_QUICK 串行检测高端和低端电路的连接

checkCh

开尔文测试的通道，可设置为：

HPVie_CHA 仅 HPVie_RELAY_ON 连接进行 kelvin 检测
HPVie_CHB 仅 HPVie_RELAY_FANOUT_ON 连接进行 kelvin 检测
HPVie_CHAB HPVie_RELAY_ALL_ON 连接进行 kelvin 检测

sampleTimes

Kelvin 检测过程中，设置 AD 采样次数。范围 50~500。默认值为 200。

注意：

当 sampleTimes < 50 时，sampleTimes = 50；

当 sampleTimes > 500 时，sampleTimes = 500。

Remarks

设置 HPVie 的 Kelvin 检测模式并进行 Kelvin 检测。

注意：

1) 通过该函数对高端或低端电路进行 kelvin 检测，检测结束后，VI 源 DA 值为 0，其它将恢复至检测之前的状态；

2) 该函数不支持 Group 操作。

Example

例 1：对 hpvie0 的 CHA 进行 High 端 Kelvin 检测，采样次数为 200。

```
hpvie0.Set(FV, 1, HPVLe_10V, HPVLe_1MA, HPVLe_RELAY_OFF);  
hpvie0.KelvinCheck(HPVLe_HIGH_SIDE, HPVLe_CHA);
```

6.7.17 GetKelvinCheckResult()

```
int GetKelvinCheckResult (  
UINT siteCount,  
HPVLe_KELVINCH checkCh,  
double&highRes,  
double&lowRes);
```

Parameters

siteCount

UINT 型变量，选择读取开尔文检测结果的工位。0 代表 Site1，1 代表 Site2.....

checkCh

选择读取开尔文检测结果的通道，可设置为：

HPVLe_CHA	CHA 连接的 kelvin 检测结果
HPVLe_CHB	CHB 连接的 kelvin 检测结果
HPVLe_CHAB	CHAB 连接的 kelvin 检测结果

highRes

double 型变量，输出高端检测的电阻值。

lowRes

double 型变量，输出低端检测的电阻值。

Return Values

返回值小于 0：工位无效；

返回值为 32bit 数据，其中低 16bit 表示低端测量异常，高 16bit 表示高端测量异常，每 bit 位为 1 则表示异常，为 0 则表示正常，具体各 bit 位的含义见下表：

Bit 位	测试端	异常描述
0	LOW	计算电阻超[0Ohm,80hm]范围
1		SLS 连接异常
2		FLS 连接异常
3		SLS 或 FLS 连接异常
4		SL 连接异常
5		恒流 1 异常
6		FL 或 SL 连接异常
7		恒流 2 异常
8		FLB 连接异常
9		恒流 3 异常
10~15		均为 0
16	HIGH	计算电阻超[0Ohm,80hm]范围
17		SHS 连接异常
18		FHS 连接异常
19		SHS 或 FHS 连接异常
20		SL 连接异常
21		恒流 1 异常
22		FH 或 SH 连接异常
23		恒流 2 异常
24		FHB 连接异常
25		恒流 3 异常
26~31		均为 0

Remarks

读取该工位对应检测通道下的 kelvin 检测结果。

Example

例 1：对 hpvie0 的 CHA 进行 High 端 kelvin 检测并获取检测结果。

```

hpvie0.Set(FV, 0, HPVle_10V, HPVle_1MA, HPVle_RELAY_OFF);
hpvie0.KelvinCheck(HPVle_HIGH_SIDE, HPVle_CHA);           // High Side check
int result[STS_SITE_NUM] = { 0 };
double highRes = 0;
double lowRes = 0;
for (iSite = 0; iSite < STS_SITE_NUM; iSite++)
{
    result[iSite] = hpvie0.GetKelvinCheckResult(iSite, HPVle_CHA, highRes, lowRes);
    // Get Kelvin check result
}

```

6.7.18 SetAlarmMask()

```
void SetAlarmMask(  
    UINT siteCount,  
    bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
hpvie0.SetAlarmMask(0);                    // mask the Alarm information of SITE_1  
// The alarm information about overranging voltage scale is not output  
hpvie0.Set(FV, 5, HPVle_2V, HPVle_10MA, HPVle_RELAY_ON);
```

6.7.19 GetAlarmMask()

```
bool GetAlarmMask(UINT siteCount);
```


Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息;

返回 false: 该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = hpvie0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.7.20 SiteBindModify ()

```
int SiteBindModify(char* modifySiteList);
```

Parameters

modifySiteList

工位重配置的字符串，格式为：“原工位号 1:目标工位号 1,原工位号 2:目标工位号 2, ...”，原工位和目标工位之间用冒号分隔，多组修改工位间用逗号分隔。

原工位号和目标工位号最小为 0（SITE_1），最大为 250（SITE_251）。

例如，将 SITE_1 的通道修改到 SITE_3，SITE_2 的通道修改到 SITE_4，对应的字符串格式为“0:2,1:3”。

Return Values

返回 0：工位重配置成功；

返回非 0：工位重配置失败。

注意：

以下情况会导致工位重配置失败：

- (8) 多组重配置工位之间必须以逗号分隔，第一个字符和最后一个字符不能为逗号，两个逗号间必须有工位修改信息；
- (9) 原工位号和目标工位号之间必须以冒号分隔，只能有一个冒号，且冒号前后必须都是数字，不能包含其它字符；
- (10) 原工位号和目标工位号必须大于等于 0 且小于系统最大工位数 251；
- (11) 不可以对定义为 NO_SITE 的通道进行工位重配置；
- (12) 同一次修改时（调用一次函数），同一个工位不能被修改两次，如“0:1,0:2”；
- (13) 同一次修改时（调用一次函数），不同工位不能被修改到同一个工位上，如“0:2,1:2”；
- (14) 目标工位如果已经进行过通道配置，此次也必须被修改。示例如下：

```
HPVle hpvie0("S4_0,S12_0");
```

初始的工位配置如下：

```
STSSetMultiSiteBind(MD_HPVle, SITE_1, "S4_0");  
STSSetMultiSiteBind(MD_HPVle, SITE_2, "S12_0");
```

工位重配置的正确做法（将 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4）：

```
hpvie0.SiteBindModify("0:2,1:3")
```

工位重配置的错误做法（仅将 SITE_1 重配置为 SITE_2）：

```
hpvie0.SiteBindModify("0:1")
```

Remarks

对通道的工位配置信息进行重配置，未进行工位重配置的通道工位信息保持不变。

注意：

该函数的重配置过程不受工位有效性信息的影响，例如将 SITE_1 重配置成

SITE_2, 无论当前 SITE_1 是否无效, 都会将 SITE_1 对应的物理通道配置到 SITE_2。

Example

通道定义和初始工位配置如下:

```
HPVle hpvie0("S4_0,S12_0");  
HPVle hpvie1("S5_0,S13_0");  
HPVle gphpvie0("S4_0,S12_0,S5_0,S13_0");
```

```
STSSetMultiSiteBind(MD_HPVIe, SITE_1, "S4_0,S5_0");  
STSSetMultiSiteBind(MD_HPVIe, SITE_2, "S12_0,S13_0");
```

例 1: 设置 hpvie0 的 SITE_1 重配置为 SITE_3, SITE_2 重配置为 SITE_4。

```
hpvie0.SiteBindModify("0:2,1:3");
```

注: 由于 hpvie0 工位信息重新进行了配置, gphpvie0 由原来的 Group Pin 变为了 Multisite Pin。

例 2: 设置 hpvie0 的 SITE_1 重配置为 SITE_2, SITE_2 重配置为 SITE_3。

```
hpvie0.SiteBindModify("0:1,1:2");
```

例 3: 设置 hpvie0 的 SITE_1 重配置为 SITE_3, SITE_2 保持不变。

```
hpvie0.SiteBindModify("0:2");
```

例 4: 设置 gphpvie0 的 SITE_1 重配置为 SITE_3, SITE_2 重配置 SITE_4。

```
gphpvie0.SiteBindModify("0:2,1:3");
```

注: 由于 gphpvie0 由 hpvie0 和 hpvie1 组成, 因此 hpvie0 和 hpvie1 对应的工位信息都发生了变化。

6.7.21 获取板卡固有信息函数

6.7.21.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
hpvie0.GetBoardSN(0, boardSN, 255);    // Get the board serial number
```

6.7.21.2 GetBoardHdRev()

```
int GetBoardHdRev(  
    UINT siteCount,  
    char* hardRev,  
    UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 N/A。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 *hardRev* 的大小。

Return Values

返回 0：读取成功；

返回非零：读取失败。

Remarks

获取板卡硬件版本号。。

Example

```
char hardRev [255] = {0};  
hpvie0.GetBoardHDRev(0, hardRev, 255);           // Get the board hardware revision
```

6.7.22 获取校准信息函数

6.7.22.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = hpvie0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.7.22.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = hpvie0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.7.22.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = hpvie0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.7.22.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS;
CAL_FAIL	校准 FAIL;
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = hpvie0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

6.7.22.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = hpvie0.GetCalibrationLogicRev(0);    // Get calibration logic Version of SITE_1
```

6.7.22.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char* meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 meterSN 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000	Keithley2000
STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0};           // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = hpvie0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.7.22.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteCount,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* boardHdRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回 “N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 *boardHdRev* 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
intlogicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = hpvie0.GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.7.22.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev (
UINT  siteCount,
char*  softRev,
UINT  revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回 “N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0};      // the buffer for saving softwareversion
// Get the version of calibration software
hpvie0.GetCalibrationSoftRev(0, softRev, 255);
```

6.7.22.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = hpvie0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.7.23 编程范例

6.7.23.1 设置一组波形数据组的拼接。

测试条件:

- 第一组：设置一个正弦波数据组，首地址从 0 开始，数据长度为 100；
- 第二组：设置一个三角波数据组，首地址从 100 开始，数据长度为 100；
- 第三组：设置一个方波数据组，首地址从 200 开始，数据长度为 100；
- 第四组：设置一个斜波数据组，首地址从 300 开始，数据长度为 100。

```
int sam=400;//AWG waveform data length
int interval=200;//AWGdata interval time, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],100,1,-5,0,180);
// Set a sinewave data array, the start address starts from 0, data size is 100.

STSAWGCreateTriangleData(&awg_pattern[100],100,4,-5,0,0);
//Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200],100,4,-5,0,50);
//Set a square wave data array, the start address starts from 200, data size is 100.

STSAWGCreateRampData(&awg_pattern[300],100,1,1,5);
//Set an ramp data array, the start address starts from 300, data size is 100.

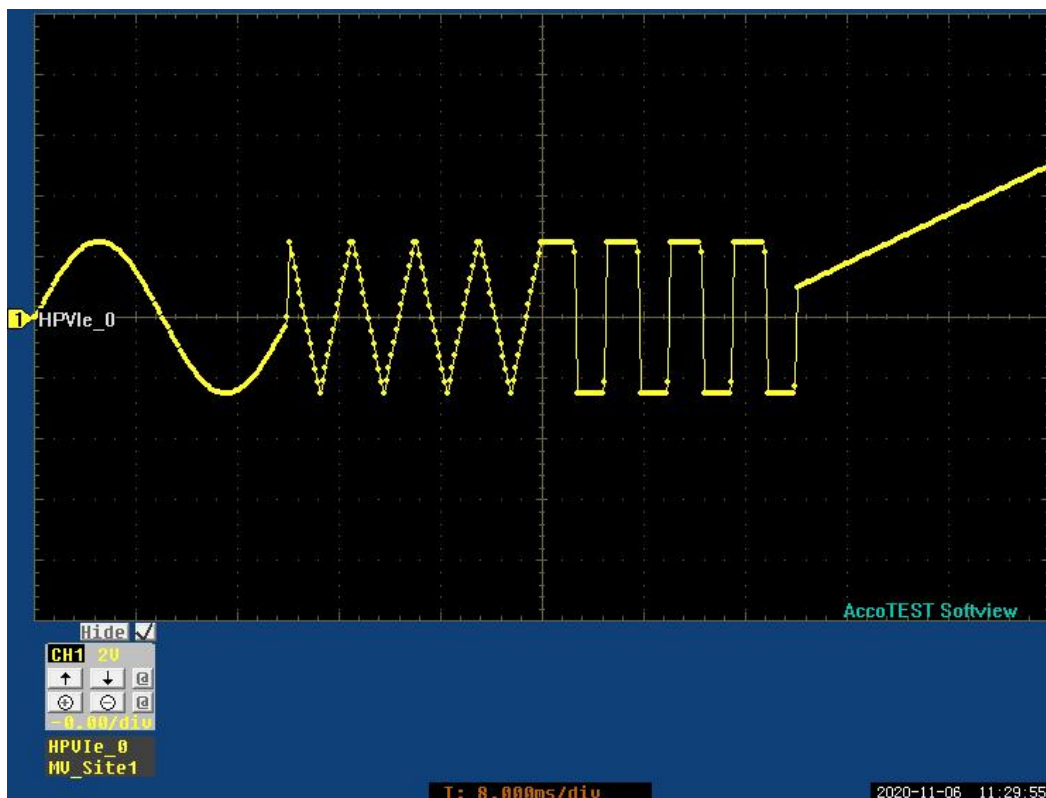
hpvie0.AwgLoader("Vcc_awg",FV,HPVle_10V,HPVle_10MA,awg_pattern,sam);
//import the awg_pattern array into hpvie0 channel, named it as 'Vcc_awg'. Set to FV mode.

hpvie0.Set(FV, 0, HPVle_10V, HPVle_10MA, HPVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

hpvie0.AwgSelect("Vcc_awg", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

hpvie0.MeasureVI(sam, interval, HPVle_MV_X1, HPVle_MI_X1,MEAS_AWG);
STSEnableAWG(&hpvie0);//enable AWG pattern for hpvie0
STSEnableMeas(&hpvie0);//enable measurement for hpvie0
STSAWGRun();//Enable AWG and measurement synchronously
```

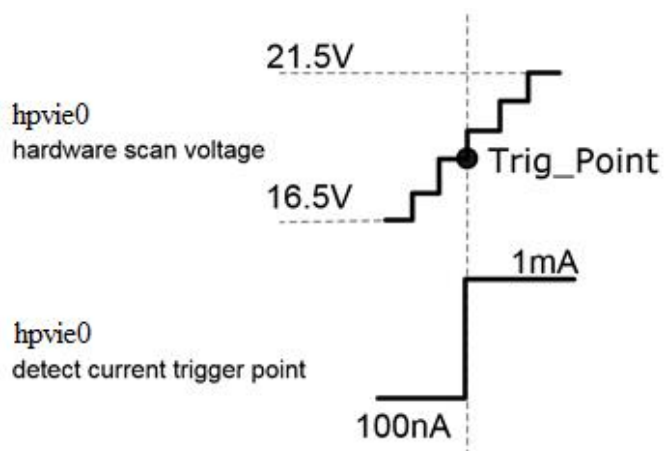
组合之后的波形如下：



6.7.23.2 同一路源生成 AWG 且作为触发源

测试条件:

建立一个波形数据组，hpvie0 从 16.5V 上升到 21.5V，步距 0.1V。测试 hpvie0 电流大于 200uA 时的跳变点的电压值。



```
double Trig = 200e-6;           // trigger level
double Trig_Point[4] = { 0 };
double result[4] = { 0 };
```

```
int sam = 50;           // awgSize
int interval = 200;     // awgInterval, unit=us
double awg_pattern[100] = { 0.0 };
int i = 0;

// Rampwave, awgData=0, awgSize=50, wavecycle=1, startValue=16.5V, stopValue=21.5V
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// load the data of "awg_pattern"(sam=50), named "Vst"
hpvie0.AwgLoader("Vst", FV, HPVLe_40V, HPVLe_100MA, awg_pattern, sam);

// the V_range(HPVLe_40V) and I_range(HPVLe_100MA) should be the same with AwgLoader
hpvie0.Set(FV, 16.5, HPVLe_40V, HPVLe_100MA, HPVLe_RELAY_ON);

// select AWGpattern of "Vst", startAddr=0, stopAddr=sam-1, loopBackAddr=sam-1, awgInterval=200us
hpvie0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);

// AWG Mode, SampleTimes=sam, samplePeriod=interval.
hpvie0.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);

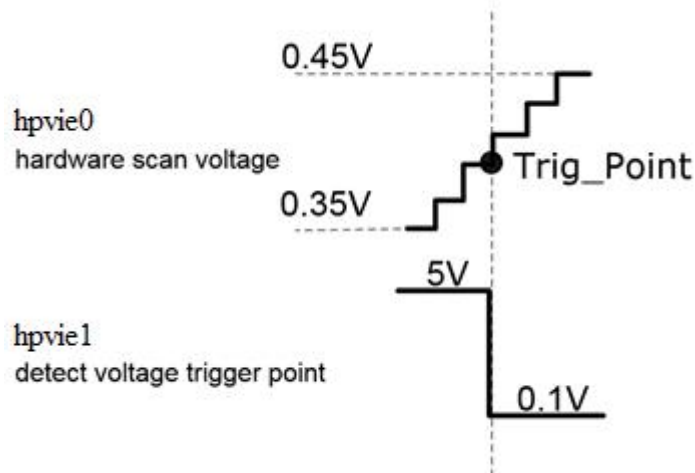
// set I_trigger level to 200uA, Rising edge
hpvie0.SetMeasITrig(Trig, TRIG_RISING);

STSEnableAWG(&hpvie0);           // hpvie0 enable AWG
STSEnableMeas(&hpvie0);         // hpvie0 enable AWG measure
STSAWGRun();                    // AWG synchronous starting
for (i = 0; i < STS_SITE_NUM; i++)
{
    Trig_Point[i] = hpvie0.GetMeasResult(i, MIRET, TRIG_RESULT);
    // read the trigger point
    result[i] = hpvie0.GetMeasResult(i, MVRET, Trig_Point[i]);
    // read the V_value of trigger point
    Vst->SetTestResult(i, 0, result[i]);
}
```

6.7.23.3 一路源生成 AWG 另一路源作为触发源

测试条件:

建立一个波形数据组，hpvie0 从 0.35V 上升 0.45V，步距 0.001V。测试 hpvie1 端的电压小于 2V 时，hpvie0 跳变点的电压值。



```
double Trig = 2; //trigger voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100; // data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[100]={0.0};
int i = 0;

hpvie1.Set(FI, 0, HPVle_10V, HPVle_1MA, HPVle_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

hpvie0.AwgLoader("Vcspre_AWG", FV, HPVle_2V, HPVle_100MA, awg_pattern, sam);
//import the awg_pattern array into hpvie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

hpvie0.Set(FV, 0.30, HPVle_2V, HPVle_100MA, HPVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

hpvie0.AwgSelect("Vcspre_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
hpvie0.MeasureVI(sam, interval, HPVle_MV_X1, HPVle_MI_X1, MEAS_AWG);
hpvie1.MeasureVI(sam, interval, HPVle_MV_X1, HPVle_MI_X1, MEAS_AWG);

hpvie1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, Falling edge
STSEnableAWG(&hpvie0); //enable AWG pattern for hpvie0
STSEnableMeas(&hpvie0, &hpvie1); //enable measurement for hpvie0 and hpvie1
STSAWGRun(); //Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
    Trig_Point[i] = hpvie1.GetMeasResult(i, MVRET, TRIG_RESULT);
    //Get the position of trigger point for hpvie1

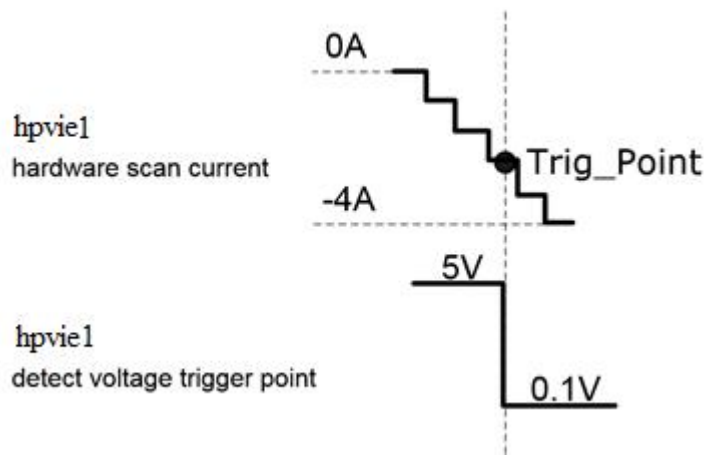
    result[i] = hpvie0.GetMeasResult(i, MVRET, Trig_Point[i]);
    //Read the voltage value of hpvie0 on trigger position

    Vcspre->SetTestResult(i, 0, result[i]);
}
```

6.7.23.4 AWG 方式扫描极限电流

测试条件:

AWG 硬件扫描测试极限电流。hpvie1 从 0 向下扫描到-4A, 步距 40mA, 测试 hpvie1 电压小于 0.5V 时的跳变点的电流值。



```
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 100;//data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[200] = {0.0}; //define an AWG waveform data array
int i = 0;

STSAWGCreateRampData(&awg_pattern[0],sam,1,0,-4);
//Set an ramp data array, the start address starts from 0, data size is 100.

hpvie1.AwgLoader("IOut", FI, HPVLe_5V, HPVLe_10A, awg_pattern, sam);
//import the awg_pattern array into hpvie1 channel, named it as 'IOut'. Set to FI mode.

hpvie1.Set(FI, 0, HPVLe_5V, HPVLe_10A, HPVLe_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'IOut' of its 'AwgLoader()'

hpvie1.AwgSelect("IOut", 0, sam - 1, 0, interval);
//Select pattern of 'IOut', start step=0, stop=sam-1; loop= 0; interval time=200uS
hpvie1.MeasureVI(sam, interval, HPVLe_MV_X1, HPVLe_MI_X1, MEAS_AWG);

hpvie1.SetMeasVTrig(0.5, TRIG_FALLING);// trigger value is 0.5V, faling edge

STSEnableAWG(&hpvie1); //enable AWG pattern for hpvie1
STSEnableMeas(&hpvie1); // enable measurement for hpvie1
STSAWGRun();//Enable AWG and measurement synchronously

for(i=0; i<SITENUM; i++)
{
```

```
Trig_Point[i] = hpvie1.GetMeasResult(i, MVRET, TRIG_RESULT);  
//Get the position of trigger point for hpvie1  
  
result[i] = hpvie1.GetMeasResult(i, MIRET, Trig_Point[i]);  
// Read the current value on trigger position for hpvie1  
  
IPeak->SetTestResult(i, 0, result[i]);  
}
```

6.7.23.5 AWG 生成正弦波测试纹波抑制比

测试条件:

hpvie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。测试 hpvie1 输出纹波抑制比。

```
int Datacount=0;  
double AD_SAMPLE=500;//sampling points  
double sum[4]={0.0};  
double temp[4]={0.0};  
double temp2[4] = { 0 };  
int sam = 830; //data size of AWG pattern  
int interval = 10; // interval time of AWG pattern, unit is uS  
double awg_pattern[1000]={0.0};  
double data[5000]={0.0};  
double result[4]={0.0};  
  
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);  
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3  
  
hpvie0.AwgLoader("Sin_AWG", FV, HPVLe_5V, HPVLe_1A, awg_pattern, sam);  
// pattern named as 'Sin_AWG, FV mode  
  
hpvie0.Set(FV, 0, HPVLe_5V, HPVLe_1A, HPVLe_RELAY_ON);  
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its  
'AwgLoader()'  
  
hpvie0.AwgRun("Sin_AWG", 0, (sam - 1), 0, interval, AWG_LOOP);  
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode  
delay_ms(1);  
  
hpvie1.Set(FI, -100e-3, HPVLe_5V, HPVLe_100MA, HPVLe_RELAY_ON);  
//Vout FI mode, prepares to measure  
delay_ms(2);  
hpvie1.MeasureVI(AD_SAMPLE, 20); //hpvie1 measurement  
  
for (i=0;i<SITENUM;i++)  
{  
    for(Datacount=0;Datacount<AD_SAMPLE;Datacount++)  
    {  
        data[Datacount]= hpvie1.GetMeasResult(i, MVRET, Datacount);  
        //hpvie1acquires every voltage values from 500 sampling points  
        result[i]= hpvie1.GetMeasResult(i, MVRET);  
    }  
}
```

```
//hpvie1acquiresaverage voltage of 500 sampling points
sum[i]+=(data[Datacount]-result[i])*(data[Datacount]-result[i]);
}
temp[i]=(sqrt(sum[i]/AD_SAMPLE))*2.828;
//calculate peak to peak value of sine wave received from output
temp2[i]=-20*log10((temp[i]+1e-12)/0.66);
//calculate PSRR
sum[i]=0;
}

for(i=0; i<SITENUM; i++)
{
    PSRR->SetTestResult(i, 0, temp2[i]);
}

hpvie1.Set(FI, 0, HPVLe_20V, HPVLe_100UA, HPVLe_RELAY_OFF);
hpvie0.AwgStop();//stop running AWG waveform of hpvie0
hpvie0.Set(FV, 0, HPVLe_5V, HPVLe_1A, HPVLe_RELAY_OFF);
```

6.8 QVMe

6.8.1 QVMe()

```
QVMe(char*channelList, char*chName = NULL);
```

Parameters

channelList

QVMe 绑定通道字符串。形如“S2_0, S2_1”，其中“S2”表示槽位号 2（S 即 Slot），“0、1”表示板内序号。

注意：

QVMe 一块板卡最多 4 个通道，因此板内序号最大值为 3。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 QVMe_xx。

Remarks

定义一路 QVMe 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 1 块 QVMe（位于 Slot2），四工位并行工作，定义可使用的 QVMe。

```
QVMe qvme0("S2_0-3");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_QVMe, SITE_1, "S2_0");    // set Slot2 ch0 to SITE1
STSSetMultiSiteBind(MD_QVMe, SITE_2, "S2_1");    // set Slot2 ch1 to SITE2
STSSetMultiSiteBind(MD_QVMe, SITE_3, "S2_2");    // set Slot2 ch2 to SITE3
STSSetMultiSiteBind(MD_QVMe, SITE_4, "S2_3");    // set Slot2 ch3 to SITE4
```

6.8.2Init()

```
int Init();
```

Remarks

初始化 QVMe 通道。执行该语句之后：

- 1) 所有输入输出继电器断开；
- 2) 低速通道和高速通道未设定有效量程并且停止数据转换；
- 3) 低速通道默认 10KHz 滤波；
- 4) 无法回读之前测量结果。

注意：

使用 QVMe 测试结束时建议调用此函数初始化。

Example

```
qvme0.Init();    //Initializellogical channel 0  
qvme1.Init();    // Initializellogical channel 1  
qvme2.Init();    // Initializellogical channel 2  
qvme3.Init();    // Initializellogical channel 3
```

6.8.3Connect()

```
void Connect(void);
```

Remarks

接通 QVMe 通道的输入输出继电器。在测量之前应先接通输入输出继电器。

Example

```
qvme0.Connect(); // Connect Logic Channel 0 Input /Output Relay
```

6.8.4 Disconnect()

```
void Disconnect(void);
```

Remarks

断开 QVMe 通道的输入输出继电器。在测量完成之后应断开输入输出继电器。

Example

```
qvme0.Disconnect(); // Disconnect Logic Channel 0 Input/Output Relay
```

6.8.5 SetMeasTrig()

```
void SetMeasTrig(  
double VTrig,  
TRIG_MODE trigMode,  
double hysteresisValue);
```

Parameters

VTrig

触发阈值，范围与选定的测量量程相关。

trigMode

触发方向。

TRIG_FALLING: 下降沿触发

TRIG_RISING: 上升沿触发

Remarks

设置测量的触发阈值和触发方向。

注意：

当 MeasureLADC()或 MeasureHADC()中的 measMode 选择 MEAS_NORMAL 模式时，此函数设置的触发阈值无效。

Example

例 1：假设选用 10V 测量量程，设置向下触发阈值为 6.0V。

```
qvme0.MeasureLADC(200, 5, QVMe_LADC_10V, QVMe_LADC_10KHz, MEAS_AWG, 0, 0, 0, 0);  
qvme0.SetMeasTrig(6.0, TRIG_FALLING);
```

6.8.6 MeasureLADC()

```
void MeasureLADC(  
    UINT sampleTimes = 100,  
    double sampleInterval = 10,  
    QVMe_LADC_VRANG VRange = QVMe_LADC_10V,  
    QVMe_LADC_FILTER Filter = QVMe_LADC_10KHz,  
    FLOATMEASMODE measMode = MEAS_NORMAL,  
    UINT T1 = 0,  
    UINT T2 = 0,  
    UINT T3 = 0,  
    UINT T4 = 0);
```

Parameters

sampleTimes

采样次数，取值范围 1~65534，默认为 100。

如果需进行 FFT，采样点数范围为 64~32768。

sampleInterval

采样间隔，取值范围 1~1300，单位为 uS。分辨率为 0.1uS，默认为 10uS。

VRange

测量量程，取值如下：

QVMe_LADC_100V

QVMe_LADC_50V

QVMe_LADC_20V

QVMe_LADC_10V

QVMe_LADC_5V

QVMe_LADC_2V

QVMe_LADC_1V

QVMe_LADC_500MV

QVMe_LADC_200MV

QVMe_LADC_100MV

默认为 QVMe_LADC_10V。

Filter

滤波器设置，取值如下：

QVMe_LADC_400KHz

QVMe_LADC_100KHz

QVMe_LADC_40KHz

QVMe_LADC_10KHz

默认为 QVMe_LADC_10KHz。

measMode

测量模式，取值如下：

MEAS_NORMAL : 普通测量模式

MEAS_AWG : AWG 模式

默认为 MEAS_NORMAL。

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

- 1) T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。
- 2) T1、T2、T3、T4 这四个参数仅在 MEAS_AWG 模式下起作用。

Remarks

设置低速通道工作参数并进行采样，将采样结果存储在板上存储器中。在调用此函数前需调用 Connect()函数接通输入输出继电器。

Example

例 1：低速通道采样 1000 点，采样间隔 1uS，10V 量程，40KHz 滤波，普通测量模式

```
qvme0.MeasureLADC(1000, 1, QVMe_LADC_10V, QVMe_LADC_40KHz, MEAS_NORMAL, 0,0,0,0);
```

6.8.7 MeasureHADC()

```
void MeasureHADC (
UINT sampleTimes = 100,
double sampleInterval = 1,
QVMe_HADC_VRange VRANGVRange = QVMe_HADC_4V,
FLOATMEASMODE measMode = MEAS_NORMAL,
UINT T1 = 0,
UINT T2 = 0,
UINT T3 = 0,
UINT T4 = 0);
```

Parameters

sampleTimes

采样次数，取值范围 1~65534，默认为 100。

如果需进行 FFT，采样点数范围为 64~32768。

sampleInterval

采样间隔，取值范围 0.1~1，单位为 uS。分辨率为 0.1uS，默认为 1uS。

VRange

测量量程，取值如下：

QVMe_HADC_4V

QVMe_HADC_2V

QVMe_HADC_1V

默认为 QVMe_HADC_4V。

measMode

测量模式，取值如下：

MEAS_NORMAL: 普通测量模式

MEAS_AWG: AWG 模式

默认为 MEAS_NORMAL。

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

1) T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

2) T1、T2、T3、T4 这四个参数仅在 MEAS_AWG 模式下起作用。

Remarks

设置高速通道工作参数并进行采样，将采样结果存储在板上存储器中。在调用此函数前需调用 Connect()函数接通输入输出继电器。

Example

例 1：高速通道采样 1000 点，采样间隔 0.1uS，4V 量程，普通测量模式

```
qvme0.MeasureHADC(1000, 0.1, QVMe_HADC_4V, MEAS_NORMAL, 0, 0, 0, 0);
```

6.8.8 GetMeasResult()

```
double GetMeasResult(  
UINT siteNo,  
int sampleNumber = AVERAGE_RESULT);
```

Parameters

siteNo

选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

sampleNumber

返回结果类型标识：

AVERAGE_RESULT: 返回所有采样点的平均值

MAX_RESULT: 返回所有采样点的最大值

MIN_RESULT: 返回所有采样点的最小值

TRIG_RESULT: 返回触发时对应采样点的位置，需配合 SetMeasTrig()使用

RMS_RESULT: 返回所有采样点扣除直流分量后的交流有效值

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 MeasureLADC

或 MeasureHADC 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

默认为 AVERAGE_RESULT。

注意：

当 MeasureLADC()或 MeasureHADC()中的 *measMode* 选择 MEAS_AWG 模式时，GetMeasResult()中的 *sampleNumber* 设置为 RMS_RESULT 无效。

Return Values

当 *sampleNumber* 为负值时，返回特定分析结果；当 *sampleNumber* 为非负值时，返回单一采样点结果。

Remarks

获取单一工位测量结果，若要获取某一工位多个采样点结果，则需做个循环。在调用此函数前需调用 MeasureLADC()或 MeasureHADC()函数进行采样。

Example

获取 qvme0 的采样平均值。

```
int SiteID=0;
double result[SITENUM] = {0.0};
qvme0.MeasureLADC(1000,1,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL,0,0,0,0);
for(SiteID=0; SiteID<SITENUM;SiteID++)
{
    result[SiteID] = qvme0.GetMeasResult(SiteID, AVERAGE_RESULT);
}
```

6.8.9 StartFFT()

```
void StartFFT(QVMe_FFT_TYPE fftType);
```

Parameters

fftType

FFT 类型选择，有两种：

QVMe_DIRECT_FFT: 直接 FFT

QVMe_BLACKMAN_HARRIS_FFT: 加 BLACKMAN_HARRIS 窗 FFT

Remarks

启动 FFT，此函数对采样数据进行 FFT，FFT 点数范围：64~32768，且采样点数应为 2 的整数次幂。在调用此函数之前需先调用 MeasureLADC() 函数或者 MeasureHADC() 函数进行采样。

注意：

当 MeasureLADC() 或 MeasureHADC() 中的 measMode 选择 MEAS_AWG 模式时，StartFFT 函数无效。

Example

qvme0 启动 LADC 测量，并转换成 FFT。

```
double fft_result[2048]={0.0};
qvme0.MeasureLADC(2048, 5, QVMe_LADC_10V, QVMe_LADC_40KHz, MEAS_NORMAL, 0,0,0,0);
//low speed mode, sample=2048, interval=5uS, 10V vrage, 40KHz filter, normal mode.
qvme0.StartFFT(QVMe_DIRECT_FFT);//start FFT directly
```

6.8.10 GetFFTResult()

```
double GetFFTResult(
UINT siteNo,
int retResult = THD_RESULT,
BYTE harmonicType = TOTAL_HARM,
BYTE harmonicNum = 5,
double FreqBand = -1);
```

Parameters

siteNo

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retResult

返回结果类型:

THD_RESULT: 返回 THD

SNR_RESULT: 返回 SNR

SINAD_RESULT: 返回 SINAD

非负整数 N: 返回 $N * ((1/\text{sampleInterval}) / \text{sampleTimes})$ 的频点的电压幅值, $0 \leq N \leq \text{sampleTimes} - 1$ 。

默认为 THD_RESULT

harmonicType

谐波类型, 用于 THD 计算。取值如下:

TOTAL_HARM: 总谐波

ODD_HARM: 奇次谐波

EVEN_HARM: 偶次谐波

该参数可以不写, 默认为 TOTAL_HARM。

注意:

当返回结果类型为 SNR_RESULT 或 SINAD_RESULT 时, 此参数无效。

harmonicNum

谐波次数, 用于 THD 计算, 取值范围 2~50。

默认为 5。

注意:

当返回结果类型标识为 SNR_RESULT 或 SINAD_RESULT 时, 此参数无效。

FreqBand

频带范围, 用于 SNR 和 SINAD 计算指定频带范围内的谐波和噪声, 单位为 KHz。

取值范围: $0 \sim ((1/\text{sampleInterval})/2) * 1000$ 。

默认为 -1, 取全部谐波和噪声。

注意：

- 1) 当取值不在给定范围时，取全部谐波和噪声。
- 2) 当返回结果类型标识为 THD_RESULT 时，此参数无效。

Remarks

调用此函数获取与 FFT 相关的测试结果。

注意：

当 MeasureLADC()或 MeasureHADC()中的 measMode 选择 MEAS_AWG 模式时，GetFFTResult 函数无效。

Example

使用 GetFFTResult 直接获取 THD、SNR、SINAD。

```
double THD_result[SITENUM]={0.0};
double SNR_result[SITENUM]={0.0};
double SINAD_result[SITENUM]={0.0};

qvme0.Connect();
delay_ms(1);

qvme0.MeasureLADC(4096,1,QVMe_LADC_10V,QVMe_LADC_400KHz,MEAS_NORMAL,0,0,0,0);
// low speed mode, sample=4096, interval=1uS, 10V vrage, 400KHz filter, normal mode
qvme0.StartFFT(QVMe_DIRECT_FFT);// start FFT directly

int SiteID=0;
for(SiteID=0; SiteID<SITENUM;SiteID++)
{
    THD_result[SiteID]=qvme0.GetFFTResult(SiteID,THD_RESULT,TOTAL_HARM,20,-1);
    // total harmonic, harmonics number is 20
    SNR_result[SiteID]=qvme0.GetFFTResult(SiteID,SNR_RESULT,TOTAL_HARM,5,-1);
    SINAD_result[SiteID]=qvme0.GetFFTResult(SiteID,SINAD_RESULT,TOTAL_HARM,5,-1);
}

for(SiteID=0; SiteID<SITENUM;SiteID++)
{
    // THD->SetTestResult(SiteID,0, THD_result[SiteID]);
    // SNR->SetTestResult(SiteID,0, SNR_result[SiteID]);
    // SINAD->SetTestResult(SiteID,0, SINAD_result[SiteID]);
}
```


6.8.11 BlockRead()

```
void BlockRead(  
    UINT siteNo,  
    long startAddr,  
    long size,  
    double*buffer,  
    QVMe_MEAS_RET retType = QVMe_SAMPLE_DATA);
```

Parameters

siteNo

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

startAddr

数据块的起始地址，非负整数。

size

数据块的大小，正整数。

buffer

存放数据结果的数组，由用户定义，如：double fft_result[2048]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误。
- 2) 建议把该数组定义成全局变量。

retType

返回的数据类型：

QVMe_SAMPLE_DATA: 时域采样数据

QVMe_FFT_DATA: 频域 FFT 数据，需先调用 StartFFT()函数进行 FFT 变换
默认为 QVMe_SAMPLE_DATA。

注意：

当 MeasureLADC()或 MeasureHADC()中的 *measMode* 选择 MEAS_AWG 模式时，BlockRead 中的 *retType* 设置为 QVMe_FFT_DATA 无效。

Remarks

此函数读取一个数据块，数据块的起始地址和大小由用户定义。

Example

例 1：读取 FFT 结果

```
int SiteID=0
double fft_result[SITENUM][2048]={0.0};
qvme0.MeasureLADC(2048,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
// low speed mode, sample=2048, interval=5uS, 10V vrang, 40KHz filter, normal mode.
qvme0.StartFFT(QVMe_DIRECT_FFT);// satrt FFT directly

for(SiteID=0; SiteID<SITENUM;SiteID++)
{
    qvme0.BlockRead(SiteID, 0, 2048, fft_result[SiteID], QVMe_FFT_DATA);
}
```

例 2：直接读取数据

```
int SiteID=0
double data_result[SITENUM][2048]={0.0};
qvme0.MeasureLADC(2048,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
// low speed mode, sample=2048, interval=5uS, 10V vrang, 40KHz filter, normal mode.

for(SiteID=0; SiteID<SITENUM;SiteID++)
{
    qvme0.BlockRead(SiteID, 0, 2048, data_result[SiteID], QVMe_SAMPLE_DATA);
}
```

6.8.12 SetAlarmMask()

```
void SetAlarmMask(
UINT siteNo,
bool maskStatus = true);
```

Parameters

siteNo

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

maskStatus

bool 型变量，可设置为：

true: 屏蔽该工位的 Alarm 信息。

false: 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽指定工位的 Alarm 信息。

Example

```
qvme0.SetAlarmMask(0, true);           // mask the Alarm information of SITE_1
// The alarm information about sampleInterval is not output
qvme0.MeasureHADC(1000, 10, QVMe_HADC_4V, MEAS_NORMAL, 0, 0, 0, 0);
```

6.8.13 GetAlarmMask()

```
bool GetAlarmMask (UINT siteNo);
```

Parameters

siteNo

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true: 该工位状态为屏蔽 Alarm 信息；

返回 false：该工位状态为输出 Alarm 信息。

Remarks

读取指定工位的 Alarm 状态。

Example

```
bool status = qvme0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.8.14 SiteBindModify ()

```
int SiteBindModify(char* modifySiteList);
```

Parameters

modifySiteList

工位重配置的字符串，格式为：“原工位号 1:目标工位号 1,原工位号 2:目标工位号 2, ...”，原工位和目标工位之间用冒号分隔，多组修改工位间用逗号分隔。

原工位号和目标工位号最小为 0（SITE_1），最大为 250（SITE_251）。

例如，将 SITE_1 的通道修改到 SITE_3，SITE_2 的通道修改到 SITE_4，对应的字符串格式为“0:2,1:3”。

Return Values

返回 0：工位重配置成功；

返回非 0：工位重配置失败。

注意：

以下情况会导致工位重配置失败：

(15) 多组重配置工位之间必须以逗号分隔，第一个字符和最后一个字符不能为逗号，两个逗号间必须有工位修改信息；

(16) 原工位号和目标工位号之间必须以冒号分隔，只能有一个冒号，且冒号前后

必须都是数字，不能包含其它字符；

- (17) 原工位号和目标工位号必须大于等于 0 且小于系统最大工位数 251；
- (18) 不可以对定义为 NO_SITE 的通道进行工位重配置；
- (19) 同一次修改时（调用一次函数），同一个工位不能被修改两次，如“0:1,0:2”；
- (20) 同一次修改时（调用一次函数），不同工位不能被修改到同一个工位上，如“0:2,1:2”；
- (21) 目标工位如果已经进行过通道配置，此次也必须被修改。示例如下：

```
QVMe qvme0("S4_0,S4_1");
```

初始的工位配置如下：

```
STSSetMultiSiteBind(MD_QVMe, SITE_1, "S4_0,S4_2");  
STSSetMultiSiteBind(MD_QVMe, SITE_2, "S4_1,S4_3");
```

工位重配置的正确做法（将 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4）：

```
qvme0.SiteBindModify("0:2,1:3")
```

工位重配置的错误做法（仅将 SITE_1 重配置为 SITE_2）：

```
qvme0.SiteBindModify("0:1")
```

Remarks

对通道的工位配置信息进行重配置，未进行工位重配置的通道工位信息保持不变。

注意：

该函数的重配置过程不受工位有效性信息的影响，例如将 SITE_1 重配置成 SITE_2，无论当前 SITE_1 是否无效，都会将 SITE_1 对应的物理通道配置到 SITE_2。

Example

通道定义和初始工位配置如下：

```
QVMe qvme0("S4_0,S4_1");  
QVMe qvme1("S4_2,S4_3");  
QVMe gpqvme0("S4_0,S4_1,S4_2,S4_3");
```

```
STSSetMultiSiteBind(MD_QVMe, SITE_1, "S4_0,S4_2");  
STSSetMultiSiteBind(MD_QVMe, SITE_2, "S4_1,S4_3");
```

例 1：设置 qvme0 的 SITE_1 重配置为 SITE_3，SITE_2 重配置为 SITE_4。

```
qvme0.SiteBindModify("0:2,1:3");
```

注：由于 qvme0 工位信息重新进行了配置，gpqvme0 由原来的 **Group Pin** 变为了 **Multisite Pin**。

例 2：设置 qvme0 的 SITE_1 重配置为 SITE_2，SITE_2 重配置为 SITE_3。

```
qvme0.SiteBindModify("0:1,1:2");
```

例 3：设置 qvme0 的 SITE_1 重配置为 SITE_3，SITE_2 保持不变。

```
qvme0.SiteBindModify("0:2");
```

例 4：设置 gpqvme0 的 SITE_1 重配置为 SITE_3，SITE_2 重配置 SITE_4。

```
gpqvme0.SiteBindModify("0:2,1:3");
```

注：由于 gpqvme0 由 qvme0 和 qvme1 组成，因此 qvme0 和 qvme1 对应的工位信息都发生了变化。

6.8.15 获取板卡固有信息函数

6.8.15.1 GetBoardSN()

```
int GetBoardSN(  
    UINT siteNo,  
    char*boardSN,  
    UINT snSize);
```

Parameters

siteNo

UINT 型变量，选择读取板卡 SN 号的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的 SN 号。未存储返回“N/A”。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

获取指定工位和指定通道对应板卡不同模块的 SN 号。

Example

例 1：获取主模块的 SN 信息。

```
char boardSn[255] = { 0 };  
// read the serial number  
qvme0.GetBoardSN(0, boardSn, 255);
```

6.8.15.2 GetBoardHDRev()

```
int GetBoardHDRev(  
    UINT siteNo,  
    char* hardRev,  
    UINT revSize);
```

Parameters

siteNo

UINT 型变量，选择读取板卡硬件版本的工位。0 代表 Site1，1 代表 Site2.....

hardRev

char 型变量，存储板卡的硬件版本号。未存储返回 “N/A”。

由用户定义，如：char hardRev[255]={0}。

revSize

UINT 型变量，数组 hardRev 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

获取指定工位和指定通道对应板卡不同模块的的硬件版本号。

Example

例 1：获取主模块的硬件版本号。

```
char hardRev[255] = { 0 };  
// read the hard version  
qvme0.GetBoardHDRev(0, hardRev, 255);
```

6.8.16 获取校准信息函数

6.8.16.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择要获取校准日期的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回指定通道校准发生的日期。未存储或工位无效时返回 1970-1-1 8:0:0。

Remarks

获取指定通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = qvme0.GetCalibrationTime(0); // Get calibration date of SITE_1
```

6.8.16.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = qvme0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.8.16.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;  
calHumidity = qvme0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.8.16.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择要获取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果，为枚举值。

CAL_PASS:	校准 PASS
CAL_FAIL:	校准 FAIL
STS_NO_RESULT_RECORD:	未存储或该工位无效。

Remarks

获取指定通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = qvme0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

6.8.16.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量, 选择读取校准时逻辑版本信息的工位。0 代表 Site1, 1 代表 Site2.....

Return Values

正常返回逻辑版本号; 异常返回 255。

注意:

异常情况发生的原因可能有: (1) 该通道未进行校准; (2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;
```

```
calLogicRev = qvme0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.8.16.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteNo,  
char *meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteNo

UINT 型变量，选择要获取校准表信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表的 SN 号。未存储返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认值为 NULL，不获取校准表的 SN 号。

snSize

UINT 型变量，数组 *meterSN* 的大小，正整数。

默认值为 0，不获取校准表的 SN 号。

Return Values

返回该通道校准表的类型，为枚举值。

STS_KEITHLEY2000:	Keithley2000
STS_AGILENT34401:	Agilent34401
STS_AGILENT3458A:	Agilent3458A
STS_NO_METER_RECORD:	未存储或该工位无效。

Remarks

读取指定通道校准表的类型和校准表的 SN 号。

Example

例 1：获取校准表的类型和 SN 号。

```
char meterSN[255] = {0};  
CAL_METER calMeter;  
calMeter = qvme0.GetCalibrationMeter(0, meterSN, 255);
```

6.8.16.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(  
    UINT siteNo,  
    char* boardSN = NULL,  
    UINT snSize = 0,  
    char* HardRev = NULL,  
    UINT revSize = 0);
```

Parameters

siteNo

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回“N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

HardRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回“N/A”。

由用户定义，如：char HardRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 HardRev 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
BYTElogicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = qvme0.GetCalibrationCalBoardInfo(0, boardSN, 255, boardHdRev, 255);
```

6.8.16.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev(
UINT siteNo,
char*softRev,
UINT revSize);
```

Parameters

siteNo

UINT 型变量，选择要获取校准使用的软件版本的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。未存储返回 “N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小，正整数。

Return Values

返回 0：读取成功；

返回-1：工位无效，读取失败。

Remarks

读取该通道校准时使用的软件版本。

Example

例 1：获取 qvme0 校准时使用的软件版本。

```
char softRev[255] = { 0 };  
qvme0.GetCalibrationSoftRev(0, softRev, 255); // Get calibration software version of SITE_1
```

6.8.16.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteNo) const;
```

Parameters

siteNo

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = qvme0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.8.17 编程范例

6.8.17.1 直流精密电压的测试

```
double avg_result[SITENUM] = {0.0};
qvme0.Connect();
delay_ms(1);

qvme0.MeasureLADC(200,5,QVMe_LADC_5V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
//sample is 200;interval time is 5uS
for(int SiteID=0; SiteID<SITENUM; SiteID++)
{
    avg_result[SiteID] = qvme0.GetMeasResult(SiteID, AVERAGE_RESULT);
    // Average->SetTestResult(SiteID, 0, avg_result[SiteID]);
}
```

6.8.17.2 RMS 的测试

用 QVMe 测量正弦波有效值 RMS。

```
int SiteID=0;
double rms_result[SITENUM] = {0.0};
qvme0.Connect();
delay_ms(1);

//Set 1KHz 4Vpp sinewave input

qvme0.MeasureLADC(200,5,QVMe_LADC_5V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
//sample is 200;interval time is 5uS, make sure sample number could contain a whole period
for(SiteID=0; SiteID<SITENUM; SiteID++)
{
    rms_result[SiteID] = qvme0.GetMeasResult(SiteID, RMS_RESULT);
    // RMS->SetTestResult(SiteID, 0, rms_result[SiteID]);
}
```

6.8.17.3 GetFFTResult 直接测试 THD、SNR、SINAD

用 QVMe 的 GetFFTResult 函数直接测量 0.9765625KHz 4Vpp 正弦波的 THD、SNR、SINAD。

```
int SiteID=0;
double THD_result[SITENUM]={0.0};
double SNR_result[SITENUM]={0.0};
double SINAD_result[SITENUM]={0.0};

qvme0.Connect();
delay_ms(1);

//Set 0.9765625KHz 4Vpp sinewave input

qvme0.MeasureLADC(4096,1,QVMe_LADC_10V,QVMe_LADC_400KHz, MEAS_NORMAL, 0, 0, 0, 0);
//sample is 4096, interval time is 1uS (sample rate=1M) ,400KHz filter
qvme0.StartFFT(QVMe_DIRECT_FFT);//start FFT

for(SiteID=0; SiteID<SITENUM; SiteID++)
{
    THD_result[SiteID]=qvme0.GetFFTResult(SiteID,THD_RESULT,TOTAL_HARM,20, -1);
    //get THD_RESULT
    SNR_result[SiteID]=qvme0.GetFFTResult(SiteID,SNR_RESULT, TOTAL_HARM, 5, -1);
    //get SNR_RESULT
    SINAD_result[SiteID]=qvme0.GetFFTResult(SiteID,SINAD_RESULT, TOTAL_HARM, 5, -1);
    //get SINAD_RESULT

    // THD->SetTestResult(SiteID, 0, THD_result[SiteID]);
    // SNR->SetTestResult(SiteID, 0, SNR_result[SiteID]);
    // SINAD->SetTestResult(SiteID, 0, SINAD_result[SiteID]);
}
```

6.8.17.4 BlockRead 测试 THD、SNR、SINAD

用 QVMe 的 BlockRead 函数测量 1KHz 4Vpp 正弦波的 THD、SNR、SINAD。

```
int i=0;
int j=0;
int sample=2048;

double P1[SITENUM]={0};
double P2[SITENUM]={0};
double P3[SITENUM]={0};
double P4[SITENUM]={0};
double thd[SITENUM]={0};
double snr[SITENUM]={0};
double sinad[SITENUM]={0};

qvme0.Connect();
delay_ms(1);

//Set 1KHz 4Vpp sinewave input

double fft_result[2048]={0.0};
qvme0.MeasureLADC(sample,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
//sample is 2048,interval time is 5uS(sample rate=200KHz), 40KHz filter
qvme0.StartFFT(QVMe_BLACKMAN_HARRIS_FFT);//start FFT
for(i=0; i<SITENUM; i++)
{
    qvme0.BlockRead(i, 0, sample, fft_result, QVMe_FFT_DATA);
    //get the 2048 FFT result,saved into fft_result

    //effective signal
    for(j=-3;j<=3;j++)
    {
        P1[i]= P1[i]+fft_result[10+j]*fft_result[10+j];
    }

    //noise signal + harmonic signal
    for(j=4;j<7;j++)
    {
        P2[i]=P2[i]+fft_result[j]*fft_result[j];
    }
    for(j=14;j<191;j++)
    {
        P2[i]=P2[i]+fft_result[j]*fft_result[j];
    }

    //harmonic signal
    for(j=20; j<201; j=j+10)
    {
        for(k=-3;k<=3;k++)
        {
            P3[i]=P3[i]+fft_result[j+k]*fft_result[j+k];
        }
    }

    //noise signal
    P4[i]=P2[i]-P3[i];
```

```
//figure
thd[i]=sqrt(P3[i]/(P1[i]+1e-100));//THD
snr[i]=sqrt(P1[i]/(P4[i]+1e-100));//SNR
sinad[i]=sqrt(P1[i]/(P2[i]+1e-100));//SINAD

thd[i]=20*log10(thd[i]+1e-100);
snr[i]=20*log10(snr[i]+1e-100);
sinad[i]=20*log10(sinad[i]+1e-100);

// THD->SetTestResult(i, 0, thd[i]);//dB
// SNR->SetTestResult(i,0,snr[i]);//dB
// SINAD->SetTestResult(i,0,sinad[i]);//dB
}
```

6.8.17.5 快速取回所有数据

取回所有时域数据

```
double data_result[2048]={0.0};
qvme0.MeasureLADC(2048,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
// sample is 2048,interval time is 5uS(sample rate=200KHz), 40KHz filter
for(int SiteID=0; SiteID<SITENUM; SiteID++)
{
    qvme0.BlockRead(SiteID, 0, 2048, data_result, QVMe_SAMPLE_DATA);
    // get the 2048 result,saved into data_result
}
```

取回所有 FFT 数据

```
double fft_result[2048]={0.0};
qvme0.MeasureLADC(2048,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
//sample is 2048,interval time is 5uS(sample rate=200KHz), 40KHz filter
qvme0.StartFFT(QVMe_DIRECT_FFT);// start FFT directly
for(int SiteID=0; SiteID<SITENUM; SiteID++)
{
    qvme0.BlockRead(SiteID, 0, 2048, fft_result, QVMe_FFT_DATA);
    // get the 2048 FFT result,saved into fft_result
}
```

取回所有加窗 FFT 数据

```
double blackman_harris_fft_result[2048]={0.0};
qvme0.MeasureLADC(2048,5,QVMe_LADC_10V,QVMe_LADC_40KHz,MEAS_NORMAL, 0, 0, 0, 0);
// sample is 2048,interval time is 5Us(sample rate=200KHz), 40KHz filter
qvme0.StartFFT(QVMe_BLACKMAN_HARRIS_FFT);// start BLACKMAN_HARRIS_FFT
for(int SiteID=0; SiteID<SITENUM; SiteID++)
{
    qvme0.BlockRead(SiteID, 0, 2048, blackman_harris_fft_result, QVMe_FFT_DATA);
    // get the 2048 FFT result,saved into blackman_harris_fft_result
}
```

6.8.17.6 QVMe 与 FPVie/FOVie 同步

以 FPVie 为例，介绍 QVMe 与浮动源的同步使用。建立一个波形数据组，fpvie0 从 8V 下降到 3V，步距 0.1V。同时 QVMe 同步测量该波形，测试 fpvie0 和 qvme0 的电压小于 5V 时，fpvie0 和 qvme0 分别在跳变点的电压值。

```
int sample = 100;
double Trig = 5;//trigger level
double Trig_Point_qvme[SITENUM] = { 0.0 };
double Trig_Point_fpvie[SITENUM] = { 0.0 };
double result_qvme[SITENUM] = { 0.0 };
double result_fpvie[SITENUM] = { 0.0 };
int sam = 50; //data size is 50
int interval = 20; //interval time is 20uS, unit=uS
double awg_pattern[1000] = { 0.0 };

qvme0.Connect();
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 8, 3);
//Ramp data,start address =0; data size=50, wavecycle=1, startValue=8V, stopValue=3V

fpvie0.AwgLoader("Vst", FV, FPVie_10V, FPVie_10MA, awg_pattern, sam);
//load the data of "awg_pattern"(sam=50), named "Vst"

fpvie0.Set(FV, 8, FPVie_10V, FPVie_10MA, FPVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'

fpvie0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=20uS

fpvie0.SetMeasVTrig(Trig, TRIG_FALLING);//set V_trigger level as 5V, falling edge
fpvie0.MeasureVI(sam, interval, FPVie_MV_X1, FPVie_MI_X1, MEAS_AWG);

qvme0.MeasureLADC(sample, 10, QVme_LADC_10V, QVme_LADC_10KHz, MEAS_AWG, 0, 0, 0, 0);
qvme0.SetMeasTrig(Trig, TRIG_FALLING);//set V_trigger level as 5V, falling edge
//Low speed mode, SampleTimes=100, samplePeriod=10US, AWG Mode

STSEnableAWG(&fpvie0); // fpvie0 enable AWG
STSEnableMeas(&fpvie0,&qvme0); // fpvie0 and qvme0 enable measuring

STSAWGRun();//Enable AWG and measurement synchronously

int SiteID = 0;
for (SiteID = 0; SiteID < SITENUM; SiteID++)
{
    Trig_Point_qvme[SiteID] = qvme0.GetMeasResult(SiteID, TRIG_RESULT);
    //qvme0 Get TRIG_RESULT
    result_qvme[SiteID] = qvme0.GetMeasResult(SiteID, Trig_Point_qvme[SiteID]);
    //qvme0 Get Value of TRIG_RESULT
    Trig_Point_fpvie[SiteID] = fpvie0.GetMeasResult(SiteID, MVRET, TRIG_RESULT);
    //fpvie0 Get TRIG_RESULT
    result_fpvie[SiteID] = fpvie0.GetMeasResult(SiteID, MVRET, Trig_Point_fpvie[SiteID]);
    //fpvie0 Get Value of TRIG_RESULT
}
```

```
// TirgQVMe->SetTestResult(SiteID, 0, result_qvme[SiteID]);  
// TrigFPVie->SetTestResult(SiteID, 0, result_fpvie[SiteID]);  
}
```

6.9 FXVie

6.9.1 FXVie()

```
FXVie(char* channelList, char* chName = NULL);
```

Parameters

channelList

FXVie 绑定通道字符串。形如“S5_0, S5_1”，其中“S5”表示槽位号 5（S 即 Slot），“0、1”表示板内序号。

注意：

FXVie 一块板卡最多 12 个通道，共两个 bank，每个 bank 最多有 6 个通道。因此板内序号最大值为 11。

其中，通道 4、5、10、11 支持 TMU 和扇出功能。

chName

通道的自定义名称。

该参数可不写，默认为 NULL，系统将该通道命名为 FXVie_xx。

Remarks

定义一路 FXVie 通道，并指定其包含的实际通道信息（“槽位号+板内序号”）。

Example

例 1：假设系统配置为 4 块 FXVie（分别位于 Slot5、Slot11、Slot22 和 Slot28），四工位并行工作，定义可使用的 FXVie。

```
FXVie fxvie0("S5_0,S11_0,S22_0,S28_0");  
FXVie fxvie1("S5_1,S11_1,S22_1,S28_1");  
FXVie fxvie2("S5_2,S11_2,S22_2,S28_2");  
FXVie fxvie3("S5_3,S11_3,S22_3,S28_3");  
FXVie fxvie4("S5_4,S11_4,S22_4,S28_4");  
FXVie fxvie5("S5_5,S11_5,S22_5,S28_5");
```

```
FXVle fxvie6("S5_6,S11_6,S22_6,S28_6");
FXVle fxvie7("S5_7,S11_7,S22_7,S28_7");
FXVle fxvie8("S5_8,S11_8,S22_8,S28_8");
FXVle fxvie9("S5_9,S11_9,S22_9,S28_9");
FXVle fxvie10("S5_10,S11_10,S22_10,S28_10");
FXVle fxvie11("S5_11,S11_11,S22_11,S28_11");
```

对应的工位绑定如下：

```
STSSetMultiSiteBind(MD_FXVle, SITE_1, "S5_0-11"); // set Slot5 ch0-11 to SITE1
STSSetMultiSiteBind(MD_FXVle, SITE_2, "S11_0-11"); // set Slot11 ch0-11 to SITE2
STSSetMultiSiteBind(MD_FXVle, SITE_3, "S22_0-11"); // set Slot22 ch0-11 to SITE3
STSSetMultiSiteBind(MD_FXVle, SITE_4, "S28_0-11"); // set Slot28 ch0-11 to SITE4
```

6.9.2Set()

```
int Set(
VIMode viMode,
double setValue,
FXVle_VRNG vRange,
FXVle_IRNG iRange,
FXVle_OUT_RELAY relayStatus = FXVle_RELAY_HOLD,
double risingTime = 0.02);
```

Parameters

viMode

FXVle 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-30~30。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

vRange

FXVle 电压量程。

FXVle_30V

FXVle_10V

FXVle_3p6V

iRange

FXVle 电流量程。

FXVle_1A

FXVle_100MA

FXVle_10MA

FXVle_1MA

FXVle_100UA

FXVle_10UA

relayStatus

设置输出继电器如何动作，分为七种情况：

FXVle_RELAY_HOLD：输出继电器保持上一个状态；

FXVle_RELAY_ON：打开输出继电器；

FXVle_RELAY_FANOUT_ON：打开扇出继电器；

FXVle_RELAY_ALL_ON：打开输出和扇出继电器；

FXVle_RELAY_OFF：关闭输出和扇出继电器；

FXVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FXVle 处于独立电压表模式。

FXVle_RELAY_SENSE_FANOUT_ON：启动独立测量模式。只打开 SENSE 线扇出继电器，关闭 FORCE 线扇出继电器。此时 FXVle 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；

- 1) 该参数设置小于等于 0.02mS 时，risingTime=0.02mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.04mS 时，risingTime=0.04mS，为 Slow 模式；
- 3) 该参数设置大于 0.04mS 时，自动启动 Capload 功能，risingTime 为 Capload 上

升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

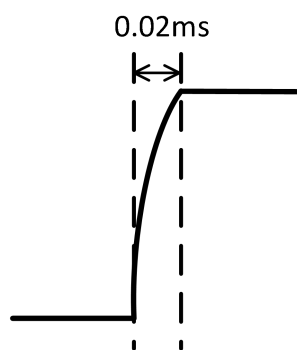
Remarks

设置 FXVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Example

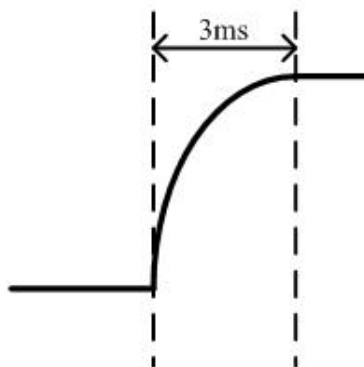
例 1：设置 fxvie0 电压输出 5V，risingTime 不填则默认为 0.02mS。

```
fxvie0.Set(FV, 5, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
```



例 2：设置 fxvie0 电压输出 5V，risingTime 设置为 3mS。

```
fxvie0.Set(FV, 0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);  
delay_ms(1);  
fxvie0.Set(FV, 5, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON, 3);
```



6.9.3SetSyn()

```
int SetSyn(  
VIMode viMode,  
const double* setValue,  
UINT siteSize,  
FXVie_VRNGvRange,  
FXVie_IRNGiRange,  
FXVie_OUT_RELAYrelayStatus = FXVie_RELAY_HOLD,  
double risingTime = 0.02);
```

Parameters

viMode

FXVie 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

etValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-30~30。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange

FXVie 电压量程。

FXVle_30V

FXVle_10V

FXVle_3p6V

iRange

FXVle 电流量程。

FXVle_1A

FXVle_100MA

FXVle_10MA

FXVle_1MA

FXVle_100UA

FXVle_10UA

relayStatus

设置输出继电器如何动作，分为七种情况：

FXVle_RELAY_HOLD：输出继电器保持上一个状态；

FXVle_RELAY_ON：打开输出继电器；

FXVle_RELAY_FANOUT_ON：打开扇出继电器；

FXVle_RELAY_ALL_ON：打开输出和扇出继电器；

FXVle_RELAY_OFF：关闭输出和扇出继电器；

FXVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FXVle 处于独立电压表模式。

FXVle_RELAY_SENSE_FANOUT_ON：启动独立测量模式。只打开 SENSE 线扇出继电器，关闭 FORCE 线扇出继电器。此时 FXVle 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；

1) 该参数设置小于等于 0.02mS 时，risingTime=0.02mS，为 Normal 模式；

2)该参数设置小于等于 0.04mS 时，risingTime=0.04mS，为 Slow 模式；

3) 该参数设置大于 0.04mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Example

例 1：fxvie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为默认值。

```
double setValue[2] = {3, 5};
int siteSize = 2;
fxvie0.SetSyn(FV, setValue, siteSize, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
```

例 2：fxvie0 通道的 SITE1 设置其输出 3V，SITE2 设置其输出 5V，且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;

fxvie0.Set(FV, 0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
delay_ms(1);
fxvie0.SetSyn(FV, setValue, siteSize, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON, 3);
```

6.9.4 SetClamp()

```
int SetClamp(
double percent_PFS,
double percent_NFS);
```

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围：2~105。

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围：2~105。

Return Values

返回 3：正负箝位皆发生变化；

返回 2：负箝位变化；

返回 1：正箝位变化；

返回 0：正负箝位皆未发生变化；

返回-1：函数调用失败。

Remarks

设置 FXVle 的正负箝位值。

注意：

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 105%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Example

例 1：设置 fxvie1 为 FI=0 模式，箝位到(50, 20)，然后切换 fxvie1 为 FV=1V 模式，箝位到(25, 25)。

```
fxvie1.Set(FI, 0, FXVle_10V, FXVle_10MA, FXVle_RELAY_ON);  
// positive clamp to 50% of full range(FXVle_10V), negative clamp to 20% of full range  
fxvie1.SetClamp(50, 20);  
  
// switch from FI to FV mode, then positive/negative clamps recover back to 110% of full range  
fxvie1.Set(FV, 0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);  
// reset positive clamp to 25% of full range(FXVle_100MA), negative clamp to 25% of full range  
fxvie1.SetClamp(25, 25);  
// Set FV=1V  
fxvie1.Set(FV, 1, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
```

例 2：设置 fxvie1 为 FV=1V 模式，箝位到(50, 25)，然后切换 fxvie1 为 FV=25V，测量结束后设置 fxvie1 为 FV=0，箝位到(100, 100)。

```
fxvie1.Set(FV, 0, FXVie_10V, FXVie_100MA, FXVie_RELAY_ON);  
// positive clamp to 50% of full range(FXVie_100MA), negative clamp to 25% of full range  
fxvie1.SetClamp(50, 25);  
fxvie1.Set(FV, 1, FXVie_10V, FXVie_100MA, FXVie_RELAY_ON);  
delay_ms(1);  
  
// still in FV mode, clamp setting will keep unchanged  
fxvie1.Set(FV, 25, FXVie_30V, FXVie_100MA, FXVie_RELAY_ON);  
fxvie1.MeasureVI(100, 20);  
  
fxvie1.Set(FV, 0, FXVie_30V, FXVie_100MA, FXVie_RELAY_ON);  
// after FV=0, set clamps back to 100%  
fxvie1.SetClamp(100, 100);
```

6.9.5 MeasureVI()

```
int MeasureVI(  
    UINT sampleTimes,  
    double samplePeriod,  
    FXVie_MI_GAIN miGain = FXVie_MI_X1,  
    FLOATMEASMODE measMode = MEAS_NORMAL,  
    UINT T1 = 0,  
    UINT T2 = 0,  
    UINT T3 = 0,  
    UINT T4 = 0);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096

注意：

当 sampleTimes < 1 时，sampleTimes = 1；

当 sampleTimes > 4096 时，sampleTimes = 4096。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：5~30000。

注意：

当 samplePeriod < 5 时，samplePeriod = 5；

当 samplePeriod > 30000 时，samplePeriod = 30000。

miGain

FXVle 测流增益。

FXVle_MI_X1

FXVle_MI_X10

该参数可以不写，默认值为 FXVle_MI_X1。

measMode

测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG AWG 测量模式。只做设置，运行 AWG 时启动。

该参数可以不写，默认值为 MEAS_NORMAL 模式

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

Remarks

设置 FXVle 的测量次数、测量间隔、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Example

例 1：设置 fxvie0 为普通模式测量，采样 20 次，间隔 10uS。

```
fxvie0.MeasureVI(20,10);
```

6.9.6 GetMeasResult()

```
double GetMeasResult(  
UINT siteCount,  
MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT,  
BYTE triggerNum = 0);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

注意：

当获取结果的通道定义为 NO_SITE 时，siteCount 可以为任意值。

retType

选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam, 则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写, 默认值为 AVERAGE_RESULT。

triggerNum

触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时, 该参数有效, 最多支持两次触发。

该参数可以不写, 默认值为 0, 表示回读第一次发生触发的位置。该参数设置为 1, 表示回读第二次发生触发的位置。

Remarks

获取 MeasureVI()函数的测量结果。

- 1) 如果选择 MVRET 测压模式, 则回读的结果为电压值, 单位为 V;
- 2) 如果选择 MIRET 测流模式, 则回读的结果为电流值, 单位为 A。

注意:

- 1) FXVie 每次测量时, 电压和电流是同时测量的, 用户只需要选择读回 MVRET 或者 MIRET 即可。
- 2) 当参数 *sampleNumber* 选择 TRIG_RESULT 时, 如果没有发生触发, 则函数返回值为-3, 如果测量模式与回读模式不符, 则函数返回值为-5。

Example

例 1: 返回 fxvie0 电流测量结果平均值。

```
// fxvie0 sampleTimes=20, samplePeriod=10uS
fxvie0.MeasureVI(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is
    // average value of current
    adresult[i] = fxvie0.GetMeasResult(i, MIRET);
}
```

例 2: 返回 fxvie0 电压测量结果第 15 个点的值。

```
//fxvie0 sampleTimes=20, samplePeriod=10uS
```

```
fxvie0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=fxvie0.GetMeasResult(i, MVRET, 14);
}
```

例 3：AWG 同步测量时，返回 fxvie0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge
fxvie0.SetMeasVTrig(2, TRIG_FALLING);
// fxvie1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG
fxvie1.MeasureVI(50, 20, FXVle_MI_X1, MEAS_AWG);
// the fxvie0's parameters of MeasureVI function are same with fxvie1
fxvie0.MeasureVI(50, 20, FXVle_MI_X1, MEAS_AWG);
// fxvie1 AWG enable
STSEnableAWG(&fxvie1);
// fxvie0 and fxvie1 measure enable
STSEnableMeas(&fxvie1, &fxvie0);
STSAWGRun();
for (i = 0; i < SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] = fxvie0.GetMeasResult(0, MVRET, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
    Trig_Point1[i] = fxvie0.GetMeasResult(0, MVRET, TRIG_RESULT, 1);
}
```

6.9.7 BlockRead()

```
int BlockRead(
UINT siteCount,
UINT start,
UINT size,
double* buffer,
MeasRet retType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中

设置的采样次数-1)

size

UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Example

```
double result[SITENUM][200] = { 0.0 };
fxvie0.MeasureVI(200, 10);
for (i = 0; i < SITENUM; i++)
{
    // start=0; size=200; Get MVRET, saved to result.
    fxvie0.BlockRead(i, 0, 200, result[i], MVRET);
}
```

6.9.8Pulse()

```
int Pulse(  
double pulseValue,  
UINT pulseTime = 300,  
int measStartTime = 0,  
UINT sampleTimes = 10,  
double sampleInterval = 30);
```

Parameters

pulseValue

double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-30~30；

选择 FI 模式时，单位 A。最大范围为-1~1。

pulseTime

UINT 型变量，设置脉冲时间。单位 uS。

分辨率：10uS。范围：300~40000。

可以不写，默认为 300uS。

measStartTime

int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

可以不写，默认为 0。

sampleTimes

UINT 型变量，设置采样次数。范围 1~4096。

可以不写，默认为 10。

sampleInterval

UINT 型变量，设置采样间隔时间，单位 uS。范围：5~30000。最小采样间隔为 5uS，分辨率 1uS。

可以不写，默认为 30uS。

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意：

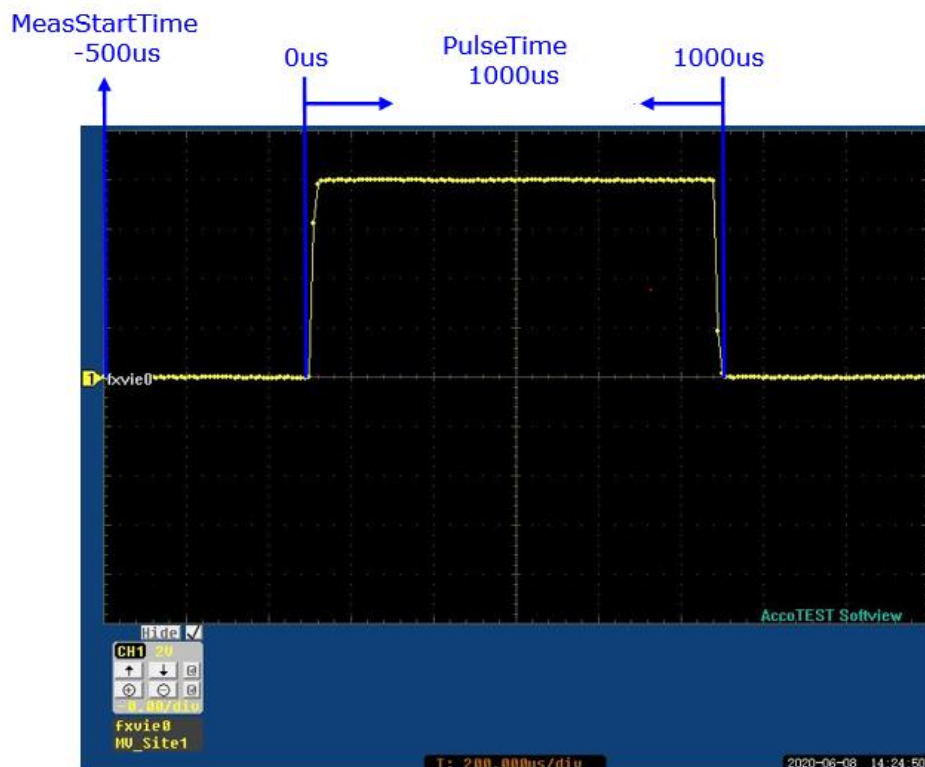
Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Example

例 1：设置 fxvie0 起始状态 FV 为 0。设置 fxvie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
fxvie0.Set(FV, 0, FXVie_10V, FXVie_10MA, FXVie_RELAY_ON);  
// first set fxvie0 in FV=0 mode,  
fxvie0.Pulse (8,1000, -500, 200, 10);  
//pulse output value=8v, pulse duration=1000uS, start time for measuring is -500uS  
//sample=200,interval time=10uS
```

使用 SoftView 软件抓取的波形如下：



6.9.9 AwgLoader()

```
int AwgLoader(  
char* awgName,  
VIMode viMode,  
FXVie_VRNG vRange,  
FXVie_IRNG iRange,  
double* awgData,  
UINT awgSize);
```

Parameters

awgName

char 型变量，AWG 波形名称。命名不区分大小写。

viMode

FXVie 工作模式，有两种模式：

FV 选择恒压模式

FI 选择恒流模式

vRange

FXVie 电压量程。

FXVie_30V

FXVie_10V

FXVie_3p6V

iRange

FXVie 电流量程。

FXVie_1A

FXVie_100MA

FXVie_10MA

FXVie_1MA

FXVie_100UA

FXVle_10UA

awgData

double 型指针，存储 AWG 波形数据。

awgSize

UINT 型变量，设置 AWG 波形数据的大小。该参数范围为 1~4096。

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意：

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 FXVle 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 FXVle 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 FXVle 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 FXVle 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

Example

例 1：建立一个波形数据组，fxvie0 从 16.5V 上升到 21.5V，步距 0.1V。

```
int sam = 50; //AWG waveform data length
```

```
int interval = 200; //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};

STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5); //pattern array saved in 'awg_pattern',
start from 16.5 to 21.5
fxvie0.AwgLoader("Vst ", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam);
```

例 2：建立一个波形数据组，fxvie0 输出频率为 100Hz、峰峰值为 4V、直流偏置为 10V 的正弦波。

```
int sam = 1000; //data size for AWG pattern
int interval = 10; //interval time for AWG pattern
double awg_pattern[2000]={0.0};

STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0); //set a sinewave data array of 100Hz,
Vpp=4, DC Offset=10
fxvie0.AwgLoader("Sine", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam); // pattern named as
'SINE', FV mode
```

6.9.10 AwgSelect()

```
int AwgSelect(
char*awgName,
UINTstartAddr,
UINTstopAddr,
int loopBackAddr,
double awgInterval= 10.0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。默认值为 10uS。

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 FXVle 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Example

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200;    // interval time for AWG pattern, unit is uS
int sam1 = 1000;         //data size for AWG pattern
int sam2 = 500;          // data size for AWG pattern

double awg_pattern1[1000]={0.0};    //define AWG_Vst data array
```

```
double awg_pattern2[500]={0.0};          //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0], sam1, 1, 16.5, 21.5);
// set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0], sam2, 1, 16.5, 10.5);

// pattern named as 'AWG_Vst', FV mode
fxvie0.AwgLoader("AWG_Vst", FV, FXVle_30V, FXVle_100MA, awg_pattern1, sam1);
// pattern named as 'AWG_Vuvlo', FV mode
fxvie0.AwgLoader("AWG_Vuvlo", FV, FXVle_10V, FXVle_100MA, awg_pattern2, sam2);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of its
'AwgLoader()'
fxvie0.Set(FV, 16.5, FXVle_30V, FXVle_100MA, FXVle_RELAY_ON);
// Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
fxvie0.AwgSelect("AWG_Vst", 0, sam1 - 1, 0, interval);

STSAWGRun();    //AWG synchronous running started for pattern of 'AWG_Vst'
```

6.9.11 AwgRun()

```
int AwgRun (
char*awgName,
UINTstartAddr,
UINTstopAddr,
int loopBackAddr,
double awgInterval= 10.0,
BOOL runMode = AWG_SINGLE,
UINTdelayTime_ms = 0);
```

Parameters

awgName

char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr

UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr

UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr

UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时，AWG 波形保持在运行结束的位置；当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval

double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为 10~60000。

该参数可以不写，默认 10uS。

runMode

运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 AWG_SINGLE。

delayTime_ms

UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。

注意：

建议不填该参数！

Remarks

某一路 FXVle 通道 AWG 单独启动函数。

注意：

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合 AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRUN。

Example

例 1：fxvie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。

```
int sam = 830;          //size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 0.66, 3.3, 0);

// pattern named as 'Sin_AWG', FV mode
fxvie0.AwgLoader("Sin_AWG", FV, FXVle_10V, FXVle_1A, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_ON);

//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
fxvie0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
```

6.9.12 AwgStop()

```
int AwgStop(void);
```

Remarks

停止某一路 FXVle 通道的 AWG 循环运行。

Example

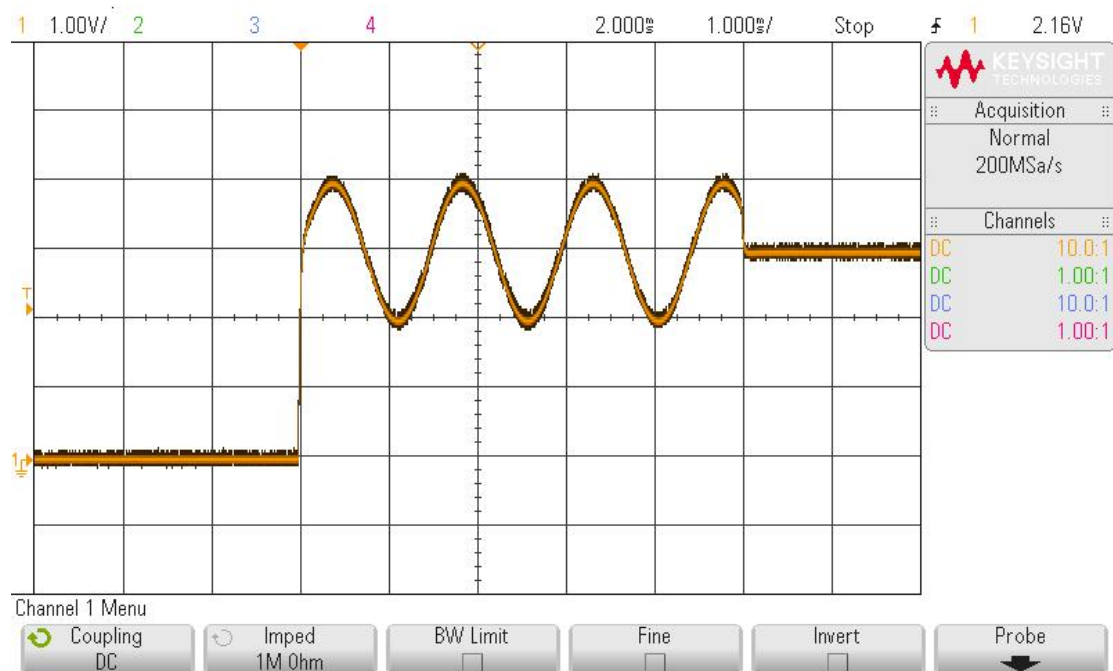
例 1: fxvie0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150;           //data size of AWG pattern
double interval = 10;    //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

STSAWGCreateSineData(&awg_pattern[0], sam, 1, 2, 3, 0);
//Set a Sine wave data array, data size is 150, period is 1, Vpp=2V, DC offset = 3V, phase = 0.
fxvie0.AwgLoader("Sin_AWG", FV, FXVle_10V, FXVle_1A, awg_pattern, sam);
//Import awg_pattern data array into fxvie0 channel, pattern named as 'Sin_AWG'
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
'AwgLoader()'
fxvie0.AwgRun("Sin_AWG", 0, (sam - 1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(6); //fxvie0 AWG pattern keeps running for 6mS
fxvie0.AwgStop();//stop loops for fxvie0 AWG running
```

该正弦波周期为 1.5mS，延时 5mS 后，停止 awg 波形。

下图为示波器抓取的 actual 波形。



6.9.13 AwgClear()

```
int AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 FXVle 通道的 AWG 数据。

Example

例 1：Load 波形之前清除 fxvie0 通道的 AWG 数据。

```
fxvie0.AwgClear();           //clear AWG data of fxvie0
fxvie0.AwgLoader("P1", FV, FXVle_30V, FXVle_100MA, awg_pattern1, sam);
```

6.9.14 SetMeasVTrig()

```
int SetMeasVTrig(
double vTrig,
TRIG_MODE trigMode = TRIG_FALLING,
double hysteresisValue = 0.0);
```

Parameters

vTrig

double 型变量，设置触发电压。单位 V。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

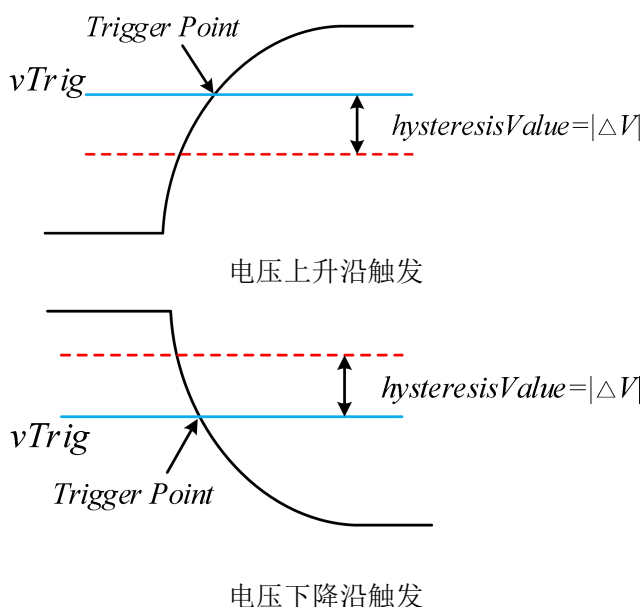
hysteresisValue

double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 ($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 ($vTrig + hysteresisValue$) 值之后又小于 $vTrig$ 值时，此时在 $vTrig$ 处判定为触发。



Remarks

电压测量触发设置。

注意：

SetMeasVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Example

例 1：建立一个波形数据组，fxvie0 从 10.5V 下降到 8V，步距 0.1V。设置 fxvie1

下降沿触发，触发电平 2V。

```
fxvie1.Set(FI, 0, FXVie_10V, FXVie_10MA, FXVie_RELAY_ON);

double Trig = 2;    //triggering voltage
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 25;        //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 10.5, 8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.

fxvie0.AwgLoader("Uvlo", FV, FXVie_30V, FXVie_100MA, awg_pattern, sam);
//import the awg_pattern array into fxvie0 channel, named it as 'Uvlo'. Set to FV mode.

fxvie0.Set(FV, 10.5, FXVie_30V, FXVie_100MA, FXVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
fxvie0.AwgSelect("Uvlo", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

设置触发电平的正确做法是：

```
fxvie1.Set(FI, 0, FXVie_10V, FXVie_10MA, FXVie_RELAY_ON);
//set fxvie1 on FI mode.
fxvie1.SetMeasVTrig(Trig, TRIG_FALLING);
//trigger value is 2V, falling edge
fxvie0.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
fxvie1.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0, &fxvie1); //enable measurement for fxvie0 and fxvie1
STSAWGRun(); //Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fxvie1.SetMeasVTrig(Trig, TRIG_FALLING);
// trigger value is 2V, falling edge
fxvie1.Set(FI, 0, FXVie_30V, FXVie_10MA, FXVie_RELAY_ON);
//insert a Set() call, change its voltage range from FXVie_10V to FXVie_30V, it will cause a utility error.
fxvie0.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
fxvie1.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0, &fxvie1); //enable measurement for fxvie0 and fxvie1
STSAWGRun(); //Enable AWG and measurement synchronously
```

6.9.15 SetMeasITrig()

```
int SetMeasITrig(
double iTrig,
```



```
TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Parameters

iTrig

double 型变量，设置触发电流。单位 A。

注意：

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode

触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

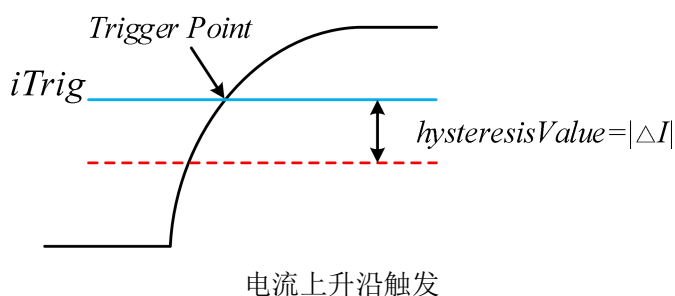
hysteresisValue

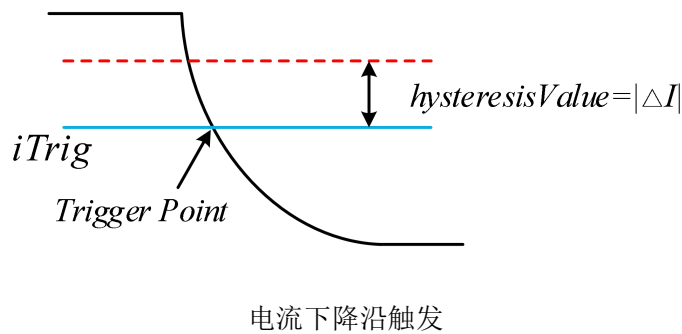
double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

注意：

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值；
- 2) 在上升沿 (TRIG_RISING) 触发的情况下，当判定到测量结果先小于 (*iTrig*-*hysteresisValue*) 值之后又大于 *iTrig* 值时，此时在 *iTrig* 处判定为触发；
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下，当判定到测量结果先大于 (*iTrig* + *hysteresisValue*) 值之后又小于 *iTrig* 值时，此时在 *iTrig* 处判定为触发。





Remarks

电流测量触发设置。

注意：

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Example

例 1：建立一个波形数据组，fxvie0 从 16.5V 上升到 21.5V，步距 0.1V。设置 fxvie0 上升沿触发，触发电流值 200uA。

```
double Trig = 200e-6;           // trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50;                   // data size of AWG pattern
int interval = 200;             // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

// Set a ramp wave data array, data size is 50, period is 1, from 16.5 to 21.5.
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);

// import the awg_pattern array into fxvie0 channel, named it as 'Vst'. Set to FV mode.
fxvie0.AwgLoader("Vst", FV, FXVie_30V, FXVie_100MA, awg_pattern, sam);

// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its
'AwgLoader()'
fxvie0.Set(FV, 16.5, FXVie_30V, FXVie_100MA, FXVie_RELAY_ON);

// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fxvie0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);
```

设置触发电平的正确做法是：

```
fxvie0.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
fxvie0.SetMeasITrig(Trig, TRIG_RISING);           // trigger value is 200uA, rising edge

STSEnableAWG(&fxvie0);           // enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0);          // enable measurement for fxvie0
```

```
STSAWGRun(); // Enable AWG and measurement synchronously
```

设置触发电平的错误做法是：

```
fxvie0.MeasureVI(sam, interval, FXVie_MI_X1, MEAS_AWG);
```

//insert a Set() call, change its current range from FXVie_100MA to FXVie_10MA, it will cause a utility error.

```
fxvie0.Set(FV, 16.5, FXVie_30V, FXVie_10MA, FXVie_RELAY_ON);
```

```
fxvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge
```

```
STSEnableAWG(&fxvie0); // enable AWG pattern for fxvie0
```

```
STSEnableMeas(&fxvie0); // enable measurement for fxvie0
```

```
STSAWGRun(); // Enable AWG and measurement synchronously
```

6.9.16 ContactCheck()

```
int ContactCheck(FXVie_CONTACTMODE checkMode);
```

Parameters

checkMode

接触测试的模式，可设置为：

FXVie_HIGH_SIDE 检测高端电路的连接

FXVie_LOW_SIDE 检测低端电路的连接

Remarks

设置高低端接触检测的模式。

注意：

- 1) 通过该函数对高低端电路进行接触检测，检测结束后，VI 源将恢复至检测之前的状态；
- 2) 该函数支持 Group 操作；
- 3) 同一个 bank 的通道不能同时进行 LOW 端接触检查。

Example

```
fxvie0.ContactCheck(FXVie_HIGH_SIDE); // set the contact check mode to high side
```

6.9.17 GetContactCheckResult()

```
int GetContactCheckResult(  
UINT siteCount,  
double&highRes,  
double&lowRes);
```

Parameters

siteCount

UINT 型变量，选择读取接触检测结果的工位。0 代表 Site1，1 代表 Site2.....

highRes

double 型变量，输出高端检测的电阻值。

lowRes

double 型变量，输出低端检测的电阻值。

Return Values

返回-1：该工位无效

返回 0：该工位接触检测 Pass

返回 1：该工位接触检测高端 Fail

返回 2：该工位接触检测低端 Fail

Remarks

读取该工位接触检测的结果。

Example

```
double highRes = 0;  
double lowRes = 0;  
int result = fxvie0.GetContactCheckResult(0, highRes, lowRes);           // Get contact check result of  
SITE_1
```

6.9.18 SetFLOff()

```
int SetFLOff();
```

Remarks

断开当前通道所在 bank 的 ForceLow 继电器。

Example

```
fxvie0.SetFLOff();
```

6.9.19 TMUConnect()

```
int TMUConnect (UINT usDelay = 500);
```

Parameters

usDelay

UINT 型变量，设置接通继电器后的延时时间，单位 uS，取值范围为 0~10000，默认值为 500。

Remarks

接通当前 TMU 通道的输入继电器。

Example

```
fxvie0.TMUConnect();
```

6.9.20 TMUDisconnect()

```
int TMUDisconnect(UINT usDelay = 100);
```

Parameters

usDelay

UINT 型变量，设置断开继电器后的延时时间，单位 uS，取值范围为 0~10000，默认值为 100。

Remarks

断开当前逻辑通道的输入继电器。

Example

```
fxvie0.TMUDisconnect();
```

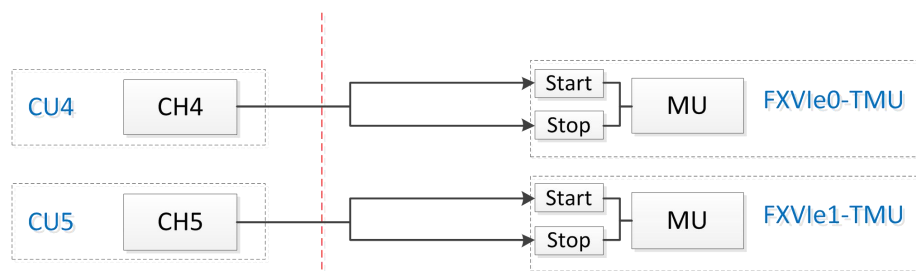
6.9.21 TMUSetInSource()

```
int TMUSetInSource();
```

Remarks

设置当前测量信号源为单信号源模式。用户端通道单元和板卡内部测量单元一一对应。

典型单信号测量矩阵关系如下：



注意：

- 1) 该函数必须在 TMUStart 和 TMUStop 函数之前设置，保证实际设置到硬件的通道和 TMUStart（TMUStop）函数中参数相关的通道保持一致；
- 2) 由 1) 延伸，如果 TMUSetInSource 在 Start（Stop）函数之后设置，可能会导致两者使用的信号通道不匹配，在 Measure 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 3) 系统每次测试结束后（InitAfterTestFlow 函数）后调用 TMUSetInSource，恢复默认单信号状态。
- 4) 如果需要设置复杂的矩阵关系，请参考函数 TMUSetMatrix。

Example

例 1：fxvie0 的测量信号源为单信号。

```
fxvie0.TMUSetInSource(); // Singlesource
```

6.9.22 TMUSetMatrix()

```
int TMUSetMatrix(  
FXVie*startChSel,  
FXVie*stopChSel);
```

Parameters*StartChSel*

FXVie 类型变量，设置 Start 信号源通道。

StopChSel

FXVie 类型变量，设置 Stop 信号源通道。

Remarks

设置用户端通道单元和板卡内部测量单元的矩阵关系，矩阵关系请参考下面 Example。

注意：

- 1) 该函数和 TMUSetInSource 都是设置矩阵，实际硬件以最后一次设置为准；
- 2) 该函数设置矩阵后，一直有效，除非设置 TMUSetInSource；
- 3) 该函数必须在 TMUStart 和 TMUStop 函数之前设置，保证实际设置到硬件的通道和 TMUStart（TMUStop）函数中参数相关的通道保持一致；
- 4) 由 3) 延伸，如果 TMUSetMatrix 在 TMUStart（TMUStop）函数之后设置，可能会导致两者的使用的信号通道不匹配，在 Measure 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 5) 该函数主要用于设置 TMU 为双信号测量模式，当设置单信号测量时，和 TMUSetInSource 函数功能一致；
- 6) 对于双信号测试，同一 Bank 的两个 TMU 通道必须在同一个 Site；
- 7) 如果不满足以下情况，则不符合矩阵设置条件，不进行矩阵设置：
 - a) *startChSel* 和 *stopChSel* 包含的通道个数必须相同；
 - b) *startChSel* 和 *stopChSel* 一一对应的通道定义为一组信号通道，同一组信号通道必须在同一个 bank 内，且为每个 bank 的 TMU 通道；
 - c) 当前定义的逻辑通道包含的物理通道，必须和 *startChSel* 或 *stopChSel* 包含的物理通道相同；
 - d) 该函数设置矩阵，要么是双信号设置，要么是单信号设置，不能单信号和双信号混合设置。

Example

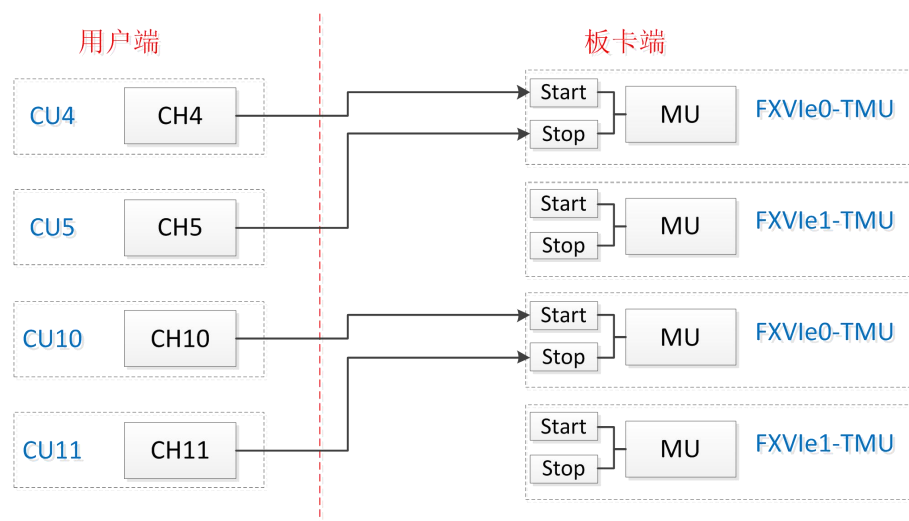
双工位配置，SITE_1 包含通道 4 和 5，SITE_2 包含通道 10 和 11，fxvie0 包含通道 4 和 10，fxvie1 包含通道 5 和 11。

```
FXVle fxvie0("S5_4, S5_10");  
FXVle fxvie1("S5_5, S5_11");
```

例 1：双信号测试，选择 fxvie0 的 MU 作为测量单元，fxvie0 包含的通道作为 start 信号，fxvie1 包含的通道作为 stop 信号。

```
fxvie0.TMUSetMatrix(&fxvie0, &fxvie1);
```


对应矩阵关系如下：



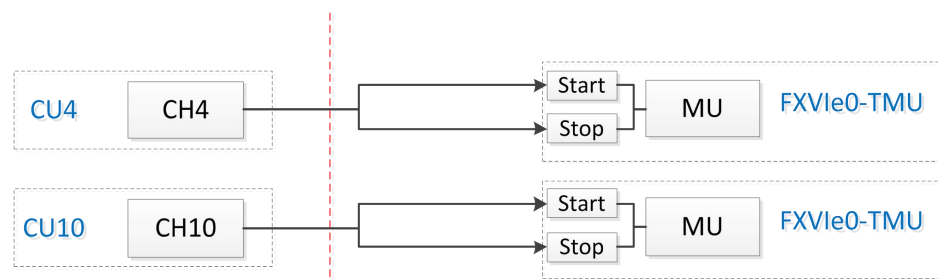
例 2：单信号测试，选择 fxvie0 的 MU 作为测量单元，fxvie0 包含的通道分别作为 start 和 stop 信号。

```
fxvie0.TMUSetMatrix(&fxvie0, &fxvie0);
```

上述 fxvie0 设置的矩阵和下面 TMUSetInSource 设置的矩阵相同

```
fxvie0.TMUSetInSource();
```

对应矩阵关系如下：



6.9.23 TMUStart()

```
int TMUStart(
FXVie_VRNGvRange= FXVie_30V,
FXVie_TMU_SLOPEslope=FXVie_TMU_POS,
double triggerLevel= 0.0,
FXVie_FILTERfilter= FXVie_FILTER_PASS,
UINT triggerWidth= 200,
UINT holdoffTime= 0,
UINT holdoffNum= 0);
```

Parameters

vRange

设置信号电压量程。

FXVle_30V: $\pm 30V$

FXVle_10V: $\pm 10V$

FXVle_3p6V: $\pm 3.6V$

默认值为 FXVle_30V。

slope

设置比较器的触发极性。

FXVle_TMU_POS: 上升沿触发

FXVle_TMU_NEG: 下降沿触发

默认值为 FXVle_TMU_POS

triggerLevel

设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

filter

设置滤波器。

FXVle_FILTER_PASS: 直通通路，不使用滤波器

FXVle_FILTER_1MHz: 1MHz 低通滤波器

FXVle_FILTER_100KHz: 100KHz 低通滤波器

默认值为 FXVle_FILTER_PASS。

注意：

- 1)如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；
- 2)被测信号为方波信号时不建议使用输入滤波器。

triggerWidth

设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。默认值为 200。

例如：输入值为 18，内部处理为 20。

注意：

被测信号到达触发电平 TriggerLevel 之后，若在 TriggerWidth 时间之内一直高于（上升沿）或低于（下降沿）TriggerLevel，则满足触发条件，否则，继续等待下次触发电平。

holdoffTime

设置延迟触发时间，取值范围 0、20~1e9，步进为 5，单位 nS。如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍，如果输入值小于 20，将会取为 0，默认值为 0。例如：输入值为 18，内部处理为 0；输入值为 28，内部处理为 30。

holdoffNum

忽略的 START 事件个数，取值范围 0~65535，默认值为 0。

Remarks

设置 START 信号。

注意：

当进行单信号测量时，TMUStart 和 TMUStop 函数的参数 VRange、Filter 也必须设置一致。

Example

例 1：设置 fxvie0 的 START 信号电压量程为 10V，上升沿 3.5V 触发，使用 1MHz 滤波器。

```
fxvie0.TMUStart(FXVle_10V, FXVle_TMU_POS, 3.5, FXVle_FILTER_1MHz);
```

例 2：设置 fxvie0 的 START 信号电压量程为 3.6V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10，延迟触发时间为 0，忽略 1 个 START 事件个数。

```
fxvie0.TMUStart(FXVie_3p6V, FXVie_TMU_POS, 1.5, FXVie_FILTER_PASS, 10, 0, 1);
```

6.9.24 TMUStop()

```
int TMUStop(  
FXVie_VRNG vRange= FXVie_30V,  
FXVie_TMU_SLOPE slope=FXVie_TMU_POS,  
double triggerLevel= 0.0,  
FXVie_FILTER filter= FXVie_FILTER_PASS,  
UINT triggerWidth= 200,  
UINT holdoffTime= 0,  
UINT holdoffNum= 0);
```

Parameters

vRange

设置信号电压量程。

FXVie_30V: ±30V

FXVie_10V: ±10V

FXVie_3p6V: ±3.6V

默认值为 FXVie_30V。

slope

设置比较器的触发极性。

FXVie_TMU_POS: 上升沿触发

FXVie_TMU_NEG: 下降沿触发

默认值为 FXVie_TMU_POS

triggerLevel

设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

filter

设置滤波器。

FXVle_FILTER_PASS: 直通通路，不使用滤波器

FXVle_FILTER_1MHz: 1MHz 低通滤波器

FXVle_FILTER_100KHz: 100KHz 低通滤波器

默认值为 FXVle_FILTER_PASS。

注意：

- 1)如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；
- 2)被测信号为方波信号时不建议使用输入滤波器。

triggerWidth

设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。默认值为 200。

例如：输入值为 18，内部处理为 20。

注意：

被测信号到达触发电平 TriggerLevel 之后，若在 TriggerWidth 时间之内一直高于（上升沿）或低于（下降沿）TriggerLevel，则满足触发条件，否则，继续等待下次触发电平。

holdoffTime

设置延迟触发时间，取值范围 0、20~1e9，步进为 5，单位 nS。如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍，如果输入值小于 20，将会取为 0，默认值为 0。例如：输入值为 18，内部处理为 0；输入值为 28，内部处理为 30。

holdoffNum

忽略的 START 事件个数，取值范围 0~65535，默认值为 0。

Remarks

设置 STOP 信号。

注意：

- 1) 当进行单信号测量时, Start 和 Stop 函数的参数 VRange、Filter 也必须设置一致。
- 2) 测量信号频率时, 此函数可以不用调用。

Example

例 1: 设置 fxvie0 的 STOP 信号电压量程为 10V, 下降沿 3.5V 触发, 使用 1MHz 滤波器。

```
fxvie0.TMUStop(FXVie_10V, FXVie_TMU_NEG, 3.5, FXVie_FILTER_1MHz);
```

例 2: 设置 fxvie0 的 STOP 信号电压量程为 3.6V, 上升沿 1.5V 触发, 不使用滤波器, 忽略 2 个 STOP 事件个数, 延迟触发时间为 0, 触发脉宽为 10。

```
fxvie0.TMUStop(FXVie_3p6V, FXVie_TMU_POS, 1.5, FXVie_FILTER_PASS, 10, 0, 2);
```

6.9.25 TMUMeasure()

```
int TMUMeasure(  
FXVie_TMU_MEASmeasType= FXVie_MEAS_TIME,  
UINTsampleNum= 1,  
doublemsTimeout= 1.0,  
FXVie_TRNGtRange= FXVie_TRANGE_MS);
```

Parameters

measType

设置测量类型。

FXVie_MEAS_TIME: 测量时间

FXVie_MEAS_FREQ: 测量频率

FXVle_MEAS_HIGH_DUTY: 测量高电平在一个周期之内所占的时间比率

FXVle_MEAS_LOW_DUTY: 测量低电平在一个周期之内所占的时间比率

FXVle_MEAS_EVENT: 测量事件

默认值为 **FXVle_MEAS_TIME**。

sampleNum

设置采样个数，取值范围为 1~2048，默认值为 1。

当 MeasType 为 **FXVle_MEAS_TIME** 类型时，SampleNum 为采样次数；

当 MeasType 为 **FXVle_MEAS_FREQ** 类型时，SampleNum 为周期数；

当 MeasType 为 **FXVle_MEAS_HIGH_DUTY** 或 **FXVle_MEAS_LOW_DUTY** 类型时，SampleNum 为跳变沿个数；

当 MeasType 为 **FXVle_MEAS_EVENT** 类型时，SampleNum 为事件个数（包含 Start 和 Stop）。

注意：

当 MeasType 为 **FXVle_MEAS_HIGH_DUTY** 或 **FXVle_MEAS_LOW_DUTY** 类型时，为保证正确测试，要求 SampleNum 不小于 3 以采样至少一个周期。并且，Start() 和 Stop() 的 slope 设置必须一个为上升沿，另一个为下降沿，函数 Start() 和 Stop() 中的参数 holdoffNum 必须设置为 0。

msTimeout

设置测量等待时间，等待 FXVle 完成设定次数采样，取值范围 0.001~2.1e4，单位 mS，默认值为 1mS。

注意：

如果超范围，默认设置为 1。

tRange

设置测量时间量程。

FXVle_TRANGE_MS: ms 级量程档(30nS~5.368S)

FXVle_TRANGE_US: us 级量程档(800nS~4mS)

FXVle_TRANGE_NS: ns 级量程档(30nS~950nS)

默认值为 FXVle_TRANGE_MS。

注意：

MeasType 为 FXVle_MEAS_HIGH_DUTY、FXVle_MEAS_LOW_DUTY 或类型 FXVle_MEAS_EVENT 时，Trange 必须为 FXVle_TRANGE_MS。

Remarks

启动测量。

注意：

如果 TMUStart 和 TMUStop 函数中使用的信号通道和 TMUSetMatrix（或 TMUSetInSource）函数中设置的信号通道不一致，则不进行测量，返回异常值。

Example

例 1：设置 fxvie0 测量时间，采样点数为 1，超时时间为 2mS。

```
fxvie0.TMUMeasure(FXVle_MEAS_TIME, 1, 2);
```

6.9.26 TMUGetMeasResult()

```
double TMUGetMeasResult (  
    UINT siteCount,  
    int retType = AVERAGE_RESULT);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retType

返回结果类型。

AVERAGE_RESULT

非负值 N, $0 \leq N \leq \text{SampleNum}-1$

默认值为 AVERAGE_RESULT。

当 Measure() 函数中的 MeasType 为 FXVle_MEAS_TIME 类型时, retType 为 AVERAGE_RESULT 时, 返回时间平均值; retType 为非负值 N 时, 返回第 N+1 次采样的测量值。单位为 uS。

当 Measure() 函数中的 MeasType 为 FXVle_MEAS_FREQ 类型时, retType 参数无效, 返回频率值, 单位 KHz。

当 Measure() 函数中的 MeasType 为 FXVle_MEAS_HIGH_DUTY 或 FXVle_MEAS_LOW_DUTY 类型时, retType 参数无效, 返回高低占空比, 单位%。

当 Measure() 函数中的 MeasType 为 FXVle_MEAS_EVENT 类型时, retType 为 AVERAGE_RESULT 时, 返回满足触发条件的事件个数; retType 为非负值 N 时, 返回第 N+1 次事件的时刻, 单位为 uS。

Remarks

根据 TMUMeasure() 函数中的 MeasType 参数类型, 获取相应的测量结果。

Example

例 1: fxvie0 获取工位 1 的时间测量结果。

```
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVle_30V, FXVle_TMU_POS, 1.5, FXVle_FILTER_PASS);
fxvie0.TMUStop(FXVle_30V, FXVle_TMU_POS, 13.5, FXVle_FILTER_PASS);
fxvie0.TMUConnect();
fxvie0.TMUMeasure(FXVle_MEAS_TIME, 10, 20, FXVle_TRANGE_MS); //sampleNum=10,
Timeout=20mS

fxvie0.TMUGetMeasResult(0, AVERAGE_RESULT); // //Get average time of site1

fxvie0.TMUGetMeasResult(0, 5); // Get the of sixth point time of site1
```

例 2: fxvie0 获取工位 0 的高占空比测量结果。

```
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVle_10V, FXVle_TMU_POS, 2.0, FXVle_FILTER_PASS);
fxvie0.TMUStop(FXVle_10V, FXVle_TMU_NEG, 2.0, FXVle_FILTER_PASS);
fxvie0.TMUConnect();
fxvie0.TMUMeasure(FXVle_MEAS_HIGH_DUTY, 20, 10, FXVle_TRANGE_MS); //sampleNum=20,
```

```
Timeout=10mS
```

```
fxvie0.TMUGetMeasResult(0, AVERAGE_RESULT); //Get average time of site1
```

6.9.27 FXVieDiffMeasure()

```
int FXVieDiffMeasure(  
FXVie*baseFXVie,  
FXVie*measFXVie,  
UINTsampleTimes,  
doublesamplePeriod,  
FXVie_DIFF_FILTERfilter = FXVie_DIFF_FILTER_OFF);
```

Parameters

baseFXVie

FXVie 类型变量，基准通道。

measFXVie

FXVie 类型变量，被测通道。

sampleTimes

UINT 类型变量，差分测量采样点数，范围 1~2048。

samplePeriod

double 类型变量，差分测量采样间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。

范围：5~30000。

filter

设置差分测量滤波器状态。

FXVie_DIFF_FILTER_OFF: 关闭滤波器

FXVie_DIFF_FILTER_ON: 接通滤波器

默认值为 FXVie_DIFF_FILTER_OFF。

Remarks

启动差分测量。

注意：

- 1) 该函数为全局函数；
- 2) 进行差分的两个通道必须在同一个 bank 内；
- 3) 不支持两个相同的通道进行差分测量；
- 4) 该函数不支持 GROUP 操作；
- 5) baseFXVle 和 measFXVle 包含的物理通道数必须相同。

Example

双工位配置，SITE_1 包含通道 0 和 1，SITE_2 包含通道 6 和 7，fxvie0 包含通道 0 和 6，fxvie1 包含通道 1 和 7。

```
FXVle fxvie0("S5_0, S5_6");  
FXVle fxvie1("S5_1, S5_7");
```

例 1：测试通道 0 和 1、通道 6 和 7 之间的差分结果，采样测试为 100，采样间隔 10uS。

```
FXVleDiffMeasure(&fxvie0, &fxvie1, 100, 10);
```

6.9.28 FXVleDiffGetMeasResult()

```
double FXVleDiffGetMeasResult(  
    UINT siteCount,  
    int sampleNumber = AVERAGE_RESULT);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 FXVieDiffMeasure 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

Remarks

获取 FXVieDiffMeasure()函数的测量结果，单位为 V。

注意：

该函数为全局函数；

测量完成后需及时回读测量结果，否则下次差分测量时，会覆盖上次测量结果。

Example

例 1：返回 fxvie0 和 fxvie1 的差分测量结果。

```
FXVieDiffMeasure(&fxvie0, &fxvie1, 100, 10);
for (i = 0; i < SITENUM; i++)
{
    adresult[i] = FXVieDiffGetMeasResult(i);
}
```

6.9.29 FXVieDiffBlockRead()

```
int FXVieDiffBlockRead(
UINT siteCount,
UINT start,
UINT size,
double* buffer);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。

size

UINT 型变量，数据块的大小，正整数。

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

该函数为全局函数；

Example

```
double result[SITENUM][200] = { 0.0 };
FXVleDiffMeasure(&fxvie0, &fxvie1, 100, 10);
for (i = 0; i < SITENUM; i++)
{
    // start=0; size=200;
    FXVleDiffBlockRead(i, 0, 200, result[i]);
}
```

6.9.30 GangRelay()

```
int GangRelay(  
FXVieParallel_CH0);
```

```
int GangRelay(  
FXVieParallel_CH0,  
FXVieParallel_CH1);
```

```
int GangRelay(  
FXVieParallel_CH0,  
FXVieParallel_CH1,  
FXVieParallel_CH2);
```

```
int GangRelay(  
FXVieParallel_CH0,  
FXVieParallel_CH1,  
FXVieParallel_CH2,  
FXVieParallel_CH3);
```

```
int GangRelay(  
FXVieParallel_CH0,  
FXVieParallel_CH1,  
FXVieParallel_CH2,  
FXVieParallel_CH3,  
FXVieParallel_CH4);
```

Parameters

Parallel_CHx

FXVie 类型变量，并联通道 x，将该通道并联到主通道上

Remarks

设置通道并联，将选定的通道并联到一起。

注意：

- 7) 该函数不支持 group;
- 8) 并联通道属性必须都是多工位或都是 NOSITE;
- 9) 相同工位下，所有并联通道必须在同一 bank;

- 10) 不同工位下，所有通道必须在不同 bank;
- 11) 对于相同 bank 的任一通道，使用该函数后，若未进行复位操作，则该 bank 所有通道的 TMUChannel 函数设置无效;
- 12) 对于并联通道，使用该函数后，若未进行复位操作，则 Set 函数设置无效;
- 13) 该函数设置后，所有并联通道状态为 FI 为 0，电压档位 30V，电流档位 100UA。

Example

例 1: 设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上。复位后，方可进行 TMUChannel 操作或 Set 操作。

正确示例如下:

```
FXVle fxvie0("S18_0,S18_6", "fxvie0");
FXVle fxvie1("S18_1,S18_7", "fxvie1");
FXVle fxvie2("S18_2,S18_8", "fxvie2");
FXVle fxvie3("S18_3,S18_9", "fxvie3");
FXVle fxvie4("S18_4,S18_10", "fxvie4");
FXVle fxvie5("S18_5,S18_11", "fxvie5");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");

fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangReset();

fxvie4.TMUChannel(fxvie0);
fxvie4.TMUChannelReset();

fxvie0.Set(FV, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_OFF);
```

错误示例 1:

对应注意事项 2)，fxvie0 和 fxvie1 的通道属性为多工位，fxvie2 属性为 NOSITE。

```
FXVle fxvie0("S18_0", "fxvie0");
FXVle fxvie1("S18_1", "fxvie1");
FXVle fxvie2("S18_5", "fxvie2");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1");
STSSetMultiSiteBind(MD_FXVle, NO_SITE, "S18_5");
fxvie0.GangRelay(fxvie1, fxvie2);
```

错误示例 2:

对应注意事项 3)，S18_8 和 S18_1 不属于相同的 bank

```
FXVle fxvie0("S18_2,S18_8", "fxvie0");
FXVle fxvie1("S18_0,S18_1", "fxvie1");
FXVle fxvie2("S18_3,S18_9", "fxvie2");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_2,S18_3");
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_1,S18_8,S18_9");

fxvie0.GangRelay(fxvie1, fxvie2);
```

错误示例 3:

对应注意事项 4), S18_0 和 S18_3、S18_1 和 S18_4、S18_2 和 S18_5 属于相同的 bank

```
FXVle fxvie0("S18_0,S18_3", "fxvie0");
FXVle fxvie1("S18_1,S18_4", "fxvie1");
FXVle fxvie2("S18_2,S18_5", "fxvie2");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1,S18_2");
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_3,S18_4,S18_5");

fxvie0.GangRelay(fxvie1, fxvie2);
```

6.9.31 GangSet()

```
int GangSet(
double setValue,
double risingTime = 0.02);
```

Parameters

setValue

double 型变量，设置恒流值。

单位为 A。最大范围为-1.2~1.2。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；最大范围为 0.02~65。

1) “该参数/并联通道个数”后，其值小于等于 0.02mS 时，risingTime=0.02mS，为 Normal 模式；

2) “该参数/并联通道个数”后，其值小于等于 0.04mS 时，risingTime=0.04mS，为

Slow 模式;

3) “该参数/并联通道个数”后，其值大于 0.04mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

设置通道电流并联功能，包括电流值及 Capload 时间。

注意：

- 4) 该函数不支持 group;
- 5) 该函数必须在 GangRelay 和 GangReset 函数之间使用。
- 6) 该函数设置后，所有并联通道电压档位 30V，电流档位 1A。

Example

例 1：设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上，输出电流为 1A，risingTime 默认为 0.02mS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
fxvie0.GangReset();
```

6.9.32 GangSetSyn()

```
int GangSetSyn(  
const double*setValue,  
UINT siteSize,  
double risingTime = 0.02);
```

Parameters

setValue

double 型变量，设置恒流值。

单位为 A。最大范围为-1.2~1.2。

siteSize

UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的

工位为 SITE1、SITE2、SITE3、SITE4。

注意：

- 1)setValue 数组长度必须大于 siteSize 的值，否则可能会发生不可预期的错误；
- 2)若 siteSize 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3)若 siteSize 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；最大范围为 0.02~65。

- 1) “该参数/并联通道个数”后，其值小于等于 0.02mS 时，risingTime=0.02mS，为 Normal 模式；
- 2) “该参数/并联通道个数”后，其值小于等于 0.04mS 时，risingTime=0.04mS，为 Slow 模式；
- 3) “该参数/并联通道个数”后，其值大于 0.04mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

设置通道电流并联功能，对不同的工位设定不同的电流值，且该操作对不同工位完全同步。

注意：

- 4) 该函数不支持 group；
- 5) 该函数必须在 GangRelay 和 GangReset 函数之间使用。
- 6) 该函数设置后，所有并联通道电压档位 30V，电流档位 1A。

Example

例 1：设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上，SITE1 设置其输出电流为 1A，SITE2 设置其输出 1.2A,risingTime 默认为 0.02mS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
double setValue[2] = { 1, 1.2 };  
int siteSize = 2;  
fxvie0.GangSetSyn(setValue, siteSize)  
fxvie0.GangReset();
```

6.9.33 GangMeasure()

```
int GangMeasure(  
    UINT sampleTimes,  
    double samplePeriod);
```

Parameters

sampleTimes

设置 AD 采样次数。范围 1~4096

注意：

当 sampleTimes < 1 时，sampleTimes = 1；

当 sampleTimes > 4096 时，sampleTimes = 4096。

samplePeriod

设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：5~30000。

注意：

当 samplePeriod < 5 时，samplePeriod = 5；

当 samplePeriod > 30000 时，samplePeriod = 30000。

Remarks

设置 FXVle 并联模式下的测量次数、测量间隔。

注意：

- 3) 该函数不支持 group；
- 4) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

例 1：设置 fxvie0 采样 20 次，间隔 10uS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
fxvie0.GangMeasure(20, 10);  
fxvie0.GangReset();
```

6.9.34 GangGetMeasResult()

```
double GangGetMeasResult(  
    UINT siteCount,  
    MeasRet retType = MVRET,  
    int sampleNumber = AVERAGE_RESULT);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retType

选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 GangMeasure 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

Remarks

获取 GangMeasure()函数的测量结果。

- 1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；
- 2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

- 3) FXVie 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET 或者 MIRET 即可。
- 4) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

例 1：返回 fxvie0 电流测量结果平均值。

```
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
delay_ms(1);
// fxvie0 sampleTimes=20, samplePeriod=10uS
fxvie0.GangMeasure(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average
    value of current
    adresult[i] = fxvie0.GangGetMeasResult(i, MIRET);
}
fxvie0.GangReset();
```

例 2：返回 fxvie0 电压测量结果第 15 个点的值。

```
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
delay_ms(1);
//fxvie0sampleTimes=20, samplePeriod=10uS
fxvie0.GangMeasure(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=fxvie0.GangGetMeasResult (i, MVRET, 14);
}
fxvie0.GangReset();
```

6.9.35 GangBlockRead()

```
int GangBlockRead(  
UINT siteCount,  
UINT start,  
UINT size,  
double* buffer,  
MeasRetretType = MVRET);
```

Parameters

siteCount

UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start

UINT 型变量，数据块的起始地址，非负整数。

size

UINT 型变量，数据块的大小，正整数。

buffer

存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) buffer 数组长度需不小于 size 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retType

选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

Remarks

此函数读取并联测量时一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

该函数必须在 GangRelay 和 GangReset 函数之间使用。

Example

```
double result[SITENUM][200] = { 0.0 };
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
delay_ms(1);
fxvie0.GangMeasure(200, 10);
for (i = 0; i < SITENUM; i++)
{
    // start=0; size=200; Get MVRET, saved to result.
    fxvie0.GangBlockRead(i, 0, 200, result[i], MVRET);
}
fxvie0.GangReset();
```

6.9.36 GangReset ()

```
int GangReset();
```

Remarks

并联通道复位功能，将并联通道复位，并联模式解除。

恢复至恒压为 0，电压档位为 30V，电流档位为 1mA，继电器 RelayOff。

注意：

在进行其它非并联模式操作之前，必须先使用该函数解除并联模式。

Example

例 1：将 fxvie0、fxvie1 和 fxvie2 的并联模式解除。

```
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
fxvie0.GangMeasure(20, 10);
fxvie0.GangReset();
```

6.9.37 TMUChannel()

```
int TMUChannel (
    FXVie FXVieChannel );
```

Parameters

FXVleChannel

FXVle 类型变量，设置需要 TMU 功能的通道。

Remarks

将 bank 内的通道通过内部总线连接到 TMU 通道，让该通道具有时间测量功能。

注意：

- 1) 该函数不支持 group;
- 2) TMU 通道及其搭接通道属性必须都是多工位或都是 NOSITE;
- 3) 相同工位下，TMU 通道及其搭接通道必须在同一 bank;
- 4) 不同工位下，TMU 通道及其搭接通道必须在不同 bank;
- 5) 对于相同 bank 的任一通道，使用该函数后，若未进行复位操作，则该 bank 所有通道的 GangRelay 函数设置无效;
- 6) 对于 TMU 通道及其搭接通道，使用该函数后，若未进行复位操作，则 Set 函数设置无效;
- 7) 每个 bank 仅有一个内部总线，同一时刻仅支持 bank 内一个通道连接到 TMU 通道。

Example

例 1：将 fxvie0 搭接到 TMU 通道 fxvie4 上

```
FXVle fxvie0("S18_0,S18_6", "fxvie0");
FXVle fxvie4("S18_4,S18_10", "fxvie4");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");

fxvie4.TMUChannel(fxvie0);
fxvie4.TMUConnect();
fxvie4.TMUStart(FXVle_10V, FXVle_TMU_POS, 3, FXVle_FILTER_PASS, 100);
fxvie4.TMUStop(FXVle_10V, FXVle_TMU_NEG, 3, FXVle_FILTER_PASS, 100);
fxvie4.TMUMeasure(FXVle_MEAS_TIME, 1, 1.0, FXVle_TRANGE_MS);
for (int i = 0; i < STS_SITE_NUM; i++)
{
    Time_Result[i] = fxvie4.TMUGetMeasResult(i);
    Gang_TMU->SetTestResult(i, 0, Time_Result[i]);
}
```



```
fxvie4.TMUDisconnect();  
fxvie4.TMUChannelReset();
```

6.9.38 TMUChannelReset ()

```
int TMUChannelReset ();
```

Remarks

TMU 通道搭接功能复位。

继电器恢复 RelayOff。

注意：

在进行其它非 TMU 操作之前，必须先使用该函数解除 TMU 通道搭接。

Example

例 1：将 fxvie0 和 fxvie4 的 TMU 通道断开搭接。

```
fxvie4.TMUChannel(fxvie0);  
fxvie4.TMUChannelReset();
```

6.9.39 TSet()

```
int TSet(  
VIMode viMode,  
double setValue,  
FXVIE_VRNG vRange,  
FXVIE_IRNG iRange,  
FXVIE_OUT_RELAY relayStatus = FXVIE_RELAY_HOLD,  
double risingTime = 0.02);
```

Parameters

viMode

FXVIE 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue

double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-30~30。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

vRange

FXVle 电压量程。

FXVle_30V

FXVle_10V

FXVle_3p6V

iRange

FXVle 电流量程。

FXVle_1A

FXVle_100MA

FXVle_10MA

FXVle_1MA

FXVle_100UA

FXVle_10UA

relayStatus

设置输出继电器如何动作，分为七种情况：

FXVle_RELAY_HOLD：输出继电器保持上一个状态；

FXVle_RELAY_ON：打开输出继电器；

FXVle_RELAY_FANOUT_ON：打开扇出继电器；

FXVle_RELAY_ALL_ON：打开输出和扇出继电器；

FXVle_RELAY_OFF：关闭输出和扇出继电器；

FXVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FXVle 处于独立电压表模式。

FXVie_RELAY_SENSE_FANOUT_ON: 启动独立测量模式。只打开 SENSE 线扇出继电器，关闭 FORCE 线扇出继电器。此时 FXVie 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；

- 1) 该参数设置小于等于 0.02mS 时，risingTime=0.02mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.04mS 时，risingTime=0.04mS，为 Slow 模式；
- 3) 该参数设置大于 0.04mS 时，自动启动 Capload 功能，risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

Remarks

以 Turbo 模式设置 FXVie 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间，配合 STSTArming 函数保证状态设置完成。

注意：

- 1) 该函数与 Set 函数的区别是：
 - a. Set 函数执行结束时，对应通道已达到目标状态，而 TSet 函数只是设置了对应通道的目标状态，需要配合 STSTArming 函数等待状态设置完成；
 - b. Set 函数在进行 Group 操作时，要求所有通道的 VI 源模式，电压/电流量程，继电器状态保持一致，而 TSet 函数进行 Group 操作时没有这样的要求；
- 2) 不同通道间设置状态时可依次使用 TSet 函数，增加一条 STSTArming 函数等待所有状态设置完成，可达到节省测试时间的目的；
- 3) 使用该函数要求软件为 AccoTestSystemVersion2100build200814_r p2.0 及以上版本，要求 FXVie 板卡 B 模块的逻辑为 0x12 及以上版本。

Example

例 1: Turbo 模式下，设置 fxvie0 在恒压模式下，使用 5V/100MA 档位，输出 4V

电压，risingTime 使用默认值；设置 fxvie1 在恒压模式下，使用 2V/10MA 档位，输出 1.5V 电压，risingTime 使用默认值。

```
fxvie0.TSet(FV, 4, FXVle_5V, FXVle_100MA, FXVle_RELAY_ON);  
fxvie1.TSet(FV, 1.5, FXVle_2V, FXVle_10MA, FXVle_RELAY_ON);  
STSTArming();
```

例 2：fxvie0 在恒压模式下，使用 5V/100MA 档位，输出 4V 电压，risingTime 使用默认值；fxvie1 在恒压模式下，使用 2V/10MA 档位，输出 1.5V 电压，risingTime 使用默认值；对这两个通道进行 Group 操作（gpfxvie），目标状态设置为恒压为 0，10V/1MA 档位，RelayOFF。

```
fxvie0.Set(FV, 4, FXVle_5V, FXVle_100MA, FXVle_RELAY_ON);  
fxvie1.Set(FV, 1.5, FXVle_2V, FXVle_10MA, FXVle_RELAY_ON);  
  
// Turbomodel  
gpfxvie.TSet(FV, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_OFF);  
STSTArming();
```

6.9.40 SetAlarmMask()

```
void SetAlarmMask(  
    UINT siteCount,  
    bool status = true);
```

Parameters

siteCount

UINT 型变量，选择设置屏蔽 Alarm 信息的工位。0 代表 Site1，1 代表 Site2.....

status

bool 型变量，可设置为：

true 屏蔽该工位的 Alarm 信息。

false 输出该工位的 Alarm 信息。

默认值为 true。

Remarks

设置屏蔽某一工位的 Alarm 信息。

注意：

通过该函数设置的 Alarm 屏蔽信息会在 InitBeforeTestFlow()函数中被重新初始化。

Example

```
fxvie0.SetAlarmMask(0);           // mask the Alarm information of SITE_1
// The alarm information about overranging voltage scale is not output
fxvie0.Set(FV, 5, FXVie_10V, FXVie_10MA, FXVie_RELAY_ON);
```

6.9.41 GetAlarmMask()

```
bool GetAlarmMask (UINT siteCount);
```

Parameters

siteCount

UINT 型变量，选择读取 Alarm 状态的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回 true：该工位状态为屏蔽 Alarm 信息；

返回 false：该工位状态为输出 Alarm 信息。

Remarks

读取该工位的 Alarm 状态。

Example

```
bool status = fxvie0.GetAlarmMask(0);           // Get Alarm status of SITE_1
```

6.9.42 GetBoardSN()

```
int GetBoardSN(  
    UINT siteCount,  
    char* boardSN,  
    UINT snSize);
```

Parameters

siteCount

UINT 型变量，选择读取板卡序列号信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储板卡的序列号。未存储或工位无效返回空值。

由用户定义，如：char boardSN[255]={0}。

snSize

UINT 型变量，数组 *boardSN* 的大小。

Remarks

获取板卡的 SN 号。

Example

```
char boardSN[255] = {0};  
fxvie0.GetBoardSN(0, boardSN, 255);    // Get the board serial number
```

6.9.43 获取校准信息函数

6.9.43.1 GetCalibrationTime()

```
SYSTEMTIME GetCalibrationTime (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准日期信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准发生的日期，异常返回的时间为 1970-1-1 8:0:0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准发生的日期。

Example

```
SYSTEMTIME calDate;  
calDate = fxvie0.GetCalibrationTime(0);    // Get calibration date of SITE_1
```

6.9.43.2 GetCalibrationTemperature()

```
double GetCalibrationTemperature (UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时温度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的温度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的温度。

Example

```
double calTemperature = 0.0;  
calTemperature = fxvie0.GetCalibrationTemperature(0); // Get calibration temperature of SITE_1
```

6.9.43.3 GetCalibrationHumidity()

```
double GetCalibrationHumidity(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时湿度信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常则返回该通道校准时的湿度，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时的湿度。

Example

```
double calHumidity = 0.0;
```



```
calHumidity = fxvie0.GetCalibrationHumidity(0); // Get calibration humidity of SITE_1
```

6.9.43.4 GetCalibrationResult()

```
CAL_RESULT GetCalibrationResult(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准结果的工位。0 代表 Site1，1 代表 Site2.....

Return Values

返回该通道的校准结果为枚举值：

CAL_PASS	校准 PASS;
CAL_FAIL	校准 FAIL;
STS_NO_RESULT_RECORD	读取校准结果异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准结果。

Example

```
CAL_RESULT calResult;  
calResult = fxvie0.GetCalibrationResult(0); // Get calibration result of SITE_1
```

6.9.43.5 GetCalibrationLogicRev()

```
int GetCalibrationLogicRev(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时逻辑版本信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回逻辑版本号；异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道的校准时的逻辑版本号。

Example

```
int calLogicRev = 0;  
calLogicRev = fxvie0.GetCalibrationLogicRev(0); // Get calibration logic Version of SITE_1
```

6.9.43.6 GetCalibrationMeter()

```
CAL_METER GetCalibrationMeter(  
UINT siteCount,  
char*meterSN = NULL,  
UINT snSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准表相关信息的工位。0 代表 Site1，1 代表 Site2.....

meterSN

char 型变量，存储校准表序列号。异常返回“N/A”。

由用户定义，如：char meterSN[255]={0}。

默认为 NULL，不读取校准表的序列号。

snSize

UINT 型变量，数组 *meterSN* 的大小。

默认为 0。

Return Values

返回校准时使用的校准表类型，枚举值：

STS_KEITHLEY2000	Keithley2000
STS_AGILENT34401	Agilent34401
STS_AGILENT3458A	Agilent3458A
STS_NO_METER_RECORD	读取校准表信息异常。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准表类型及表的序列号。

Example

```
CAL_METER calMeterType;  
char calMeterSN[255] = {0};           // the buffer for saving Serial NO. of calibration meter  
// Get the information of calibration meter  
calMeterType = fxvie0.GetCalibrationMeter(0, calMeterSN, 255);
```

6.9.43.7 GetCalibrationCalBoardInfo()

```
int GetCalibrationCalBoardInfo(
```

```
UINT siteCount,  
char*boardSN = NULL,  
UINT snSize = 0,  
char*boardHdRev = NULL,  
UINT revSize = 0);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的校准板的相关信息的工位。0 代表 Site1，1 代表 Site2.....

boardSN

char 型变量，存储校准板的序列号。异常返回 “N/A”。

由用户定义，如：char boardSN[255]={0}。

默认为 NULL，不读取校准时使用的校准板的序列号。

snSize

UINT 型变量，数组 *boardSN* 的大小。

默认为 0。

boardHdRev

char 型变量，存储校准时使用的校准板的硬件版本信息。异常返回 “N/A”。

由用户定义，如：char boardHdRev[255]={0}。

默认为 NULL，不读取校准时使用的校准板的硬件版本信息。

revSize

UINT 型变量，数组 *boardHdRev* 的大小。

默认为 0。

Return Values

正常返回校准时使用的校准板的逻辑版本号，异常返回 255。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的校准板相关信息。

Example

```
// Get the logicversion,serialnumber and hardware version of calibration board
int logicRev= 0;
char boardSN[255] = {0};           // the buffer for saving Serial NO. of calibration board
char boardHdRev[255] = {0};        // the buffer for saving hardware version of calibration board
logicRev = fxvie0. GetCalibrationCalBoardInfo (0, boardSN, 255, boardHdRev, 255);
```

6.9.43.8 GetCalibrationSoftRev()

```
int GetCalibrationSoftRev (
UINT siteCount,
char* softRev,
UINT revSize);
```

Parameters

siteCount

UINT 型变量，选择读取校准时使用的软件版本信息的工位。0 代表 Site1，1 代表 Site2.....

softRev

char 型变量，存储校准使用的软件版本。异常返回 “N/A”。

由用户定义，如：char softRev[255]={0}。

revSize

UINT 型变量，数组 *softRev* 的大小。

Return Values

正常返回 0，异常返回非零。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时使用的软件版本信息。

Example

```
char softRev[255] = {0};      // the buffer for saving softwareversion
// Get the version of calibration software
fxvie0.GetCalibrationSoftRev(0, softRev, 255);
```

6.9.43.9 GetCalibrationSlotID()

```
int GetCalibrationSlotID(UINT siteCount) const;
```

Parameters

siteCount

UINT 型变量，选择读取校准时板卡所处的槽位信息的工位。0 代表 Site1，1 代表 Site2.....

Return Values

正常返回板卡校准时所处的槽位信息，异常返回 0。

注意：

异常情况发生的原因可能有：(1) 该通道未进行校准；(2) 该工位无效。

Remarks

获取该通道校准时所处的槽位信息。

Example

```
intslotID = fxvie0.GetCalibrationSlotID(0); // Get the slot ID of calibration
```

6.9.44 VI 源编程范例

6.9.44.1 设置一组波形数据组的拼接

测试条件:

第一组：设置一个正弦波数据组，首地址从0开始，数据长度为100；

第二组：设置一个三角波数据组，首地址从100开始，数据长度为100；

第三组：设置一个方波数据组，首地址从200开始，数据长度为100；

第四组：设置一个斜波数据组，首地址从300开始，数据长度为100。

```
int sam = 400; // AWG waveform data length
int interval = 200; // AWG data interval time, unit is uS
double awg_pattern[1000] = { 0.0 };

STSAWGCreateSineData(&awg_pattern[0], 100, 1, -5, 0, 180);
// Set a sinewave data array, the start address starts from 0, data size is 100.

STSAWGCreateTriangleData(&awg_pattern[100], 100, 4, -5, 0, 0);
// Set a triangle wave data array, the start address starts from 100, data size is 100.

STSAWGCreateSquareData(&awg_pattern[200], 100, 4, -5, 0, 50);
// Set a square wave data array, the start address starts from 200, data size is 100.

STSAWGCreateRampData(&awg_pattern[300], 100, 1, 1, 5);
// Set an ramp data array, the start address starts from 300, data size is 100.

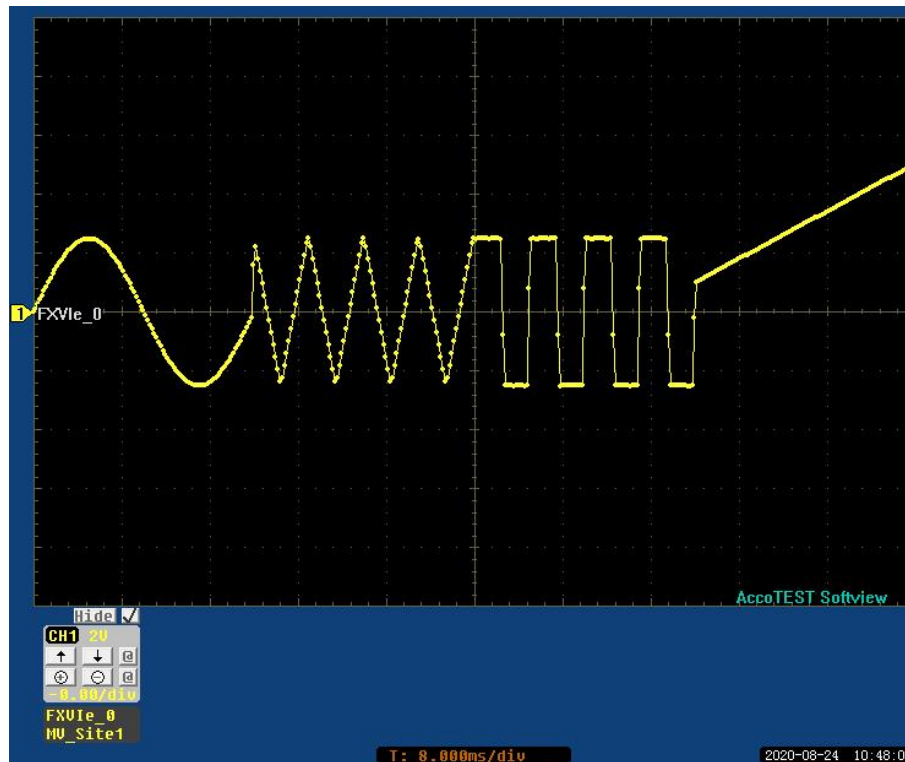
fxvie0.AwgLoader("Vcc_awg", FV, FXVIE_10V, FXVIE_10MA, awg_pattern, sam);
// import the awg_pattern array into fxvie0 channel, named it as 'Vcc_awg'. Set to FV mode.

fxvie0.Set(FV, 0, FXVIE_10V, FXVIE_10MA, FXVIE_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcc_awg' of its
'AwgLoader()'

fxvie0.AwgSelect("Vcc_awg", 0, sam - 1, sam - 1, interval);
// Select pattern of 'Vcc_awg', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

```
fxvie0.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);
STSEnableAWG(&fxvie0);//enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0);//enable measurement for fxvie0
STSAWGRun();//Enable AWG and measurement synchronously
```

组合之后的波形如下：



6.9.44.2 生成正弦波测试纹波抑制比

测试条件：

fxvie0 连续输出 120Hz 正弦波，幅值 0.33V，直流偏置为 3.3V。测试 fxvie1 输出纹波抑制比。

```
int Datacount = 0;
double AD_SAMPLE = 500;//sampling points
double sum[4] = { 0.0 };
double temp[4] = { 0.0 }, temp2[4] = { 0.0 };
int sam = 830; //data size of AWG pattern
int interval = 10; // interval time of AWG pattern, unit is uS
double awg_pattern[1000] = { 0.0 };
double data[5000] = { 0.0 };
double result[4] = { 0.0 };

STSAWGCreateSineData(&awg_pattern[0], sam, 1, 0.66, 3.3, 0);
// set waveform data array for Sin_AWG, Vpp=0.66, DC=3.3

fxvie0.AwgLoader("Sin_AWG", FV, FXVle_10V, FXVle_1A, awg_pattern, sam);
// pattern named as 'Sin_AWG, FV mode

fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_ON);
```



```
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its
// 'AwgLoader()'

fxvie0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(1);

fxvie1.Set(FI, -100e-3, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
//Vout FI mode, prepares to measure
delay_ms(2);
fxvie1.MeasureVI(AD_SAMPLE, 20); //fxvie1 measurement

for (i = 0; i < SITENUM; i++)
{
    for (Datacount = 0; Datacount < AD_SAMPLE; Datacount++)
    {
        data[Datacount] = fxvie1.GetMeasResult(i, MVRET, Datacount);
        //fxvie1 acquires every voltage values from 500 sampling points
        result[i] = fxvie1.GetMeasResult(i, MVRET);
        //fxvie1 acquires average voltage of 500 sampling points
        sum[i] += (data[Datacount] - result[i]) * (data[Datacount] - result[i]);
    }
    temp[i] = (sqrt(sum[i] / AD_SAMPLE)) * 2.828;
    //calculate peak to peak value of sine wave received from output
    temp2[i] = -20 * log10((temp[i] + 1e-12) / 0.66);
    //calculate PSRR
    sum[i] = 0;
}

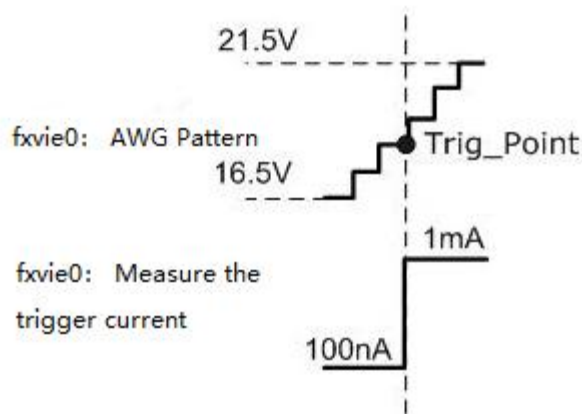
for (i = 0; i < SITENUM; i++)
{
    PSRR->SetTestResult(i, 0, temp2[i]);
}

fxvie1.Set(FI, 0, FXVle_30V, FXVle_100MA, FXVle_RELAY_OFF);
fxvie0.AwgStop(); //stop running AWG waveform of fxvie0
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_OFF);
```

6.9.44.3 同一路生成 AWG 且作为触发源

测试条件:

建立一个波形数据组，fxvie0从16.5V上升到21.5V，步距0.1V。测试fxvie0电流大于200uA时的跳变点的电压值。



```
double Trig = 200e-6; //trigger current level
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 50; //data size of AWG pattern
int interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
//Set an ramp data array, the start address starts from 0, data size is 50.

fxvie0.AwgLoader("Vst", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam);
//import the awg_pattern array into fxvie0 channel, named it as 'Vst'. Set to FV mode.

fxvie0.Set(FV, 16.5, FXVle_30V, FXVle_100MA, FXVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its
'AwgLoader()'

fxvie0.AwgSelect("Vst", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fxvie0.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);

fxvie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge

STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0); //enable measurement for fxvie0
STSAWGRun(); //Enable AWG and measurement synchronously

for (i = 0; i < SITENUM; i++)
{
    Trig_Point[i] = fxvie0.GetMeasResult(i, MIRET, TRIG_RESULT);
    //Get the position of trigger point

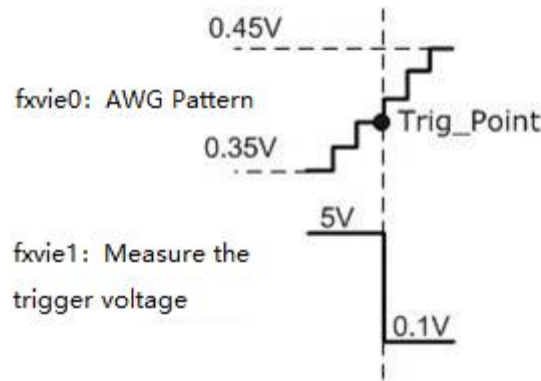
    result[i] = fxvie0.GetMeasResult(i, MVRET, Trig_Point[i]);
    //Read the voltage value on trigger position

    Vst->SetTestResult(i, 0, result[i]);
}
```

6.9.44.4 一路源生成 AWG 另一路作为触发源

测试条件:

建立一个波形数据组, fxvie0从0.35V上升0.45V, 步距0.001V。测试fxvie1端的电压小于2V时, fxvie0跳变点的电压值。



```
double Trig = 2; //trigger voltage
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 100; // data size of AWG pattern
int interval = 100; // interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };

fxvie1.Set(FI, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_ON); //OUT Measure
delay_ms(1);

STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
//Set an ramp data array, the start address starts from 0, data size is 50.

fxvie0.AwgLoader("Vcspre_AWG", FV, FXVle_3p6V, FXVle_100MA, awg_pattern, sam);
//import the awg_pattern array into fxvie0 channel, named it as 'Vcspre_AWG'. Set to FV mode.

fxvie0.Set(FV, 0.30, FXVle_3p6V, FXVle_100MA, FXVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its
'AwgLoader()'

fxvie0.AwgSelect("Vcspre_AWG", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100uS

fxvie1.SetMeasVTrig(Trig, TRIG_FALLING); // trigger value is 2V, Falling edge

fxvie0.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);
fxvie1.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);
STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0, &fxvie1); //enable measurement for fxvie0 and fxvie1
STSAWGRun(); //Enable AWG and measurement synchronously

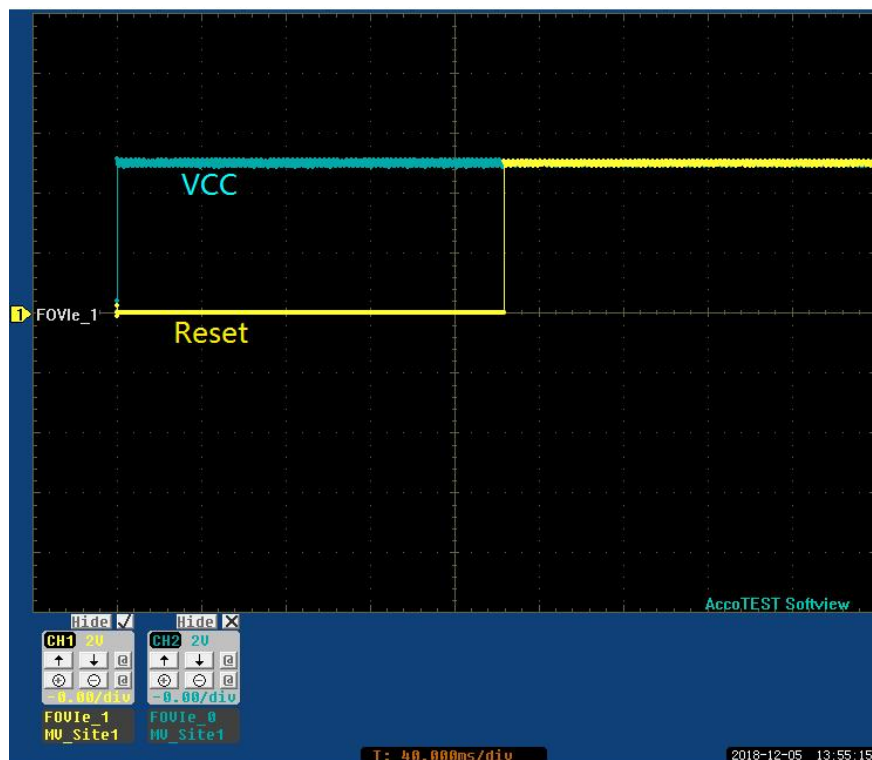
for (i = 0; i < SITENUM; i++)
{
    Trig_Point[i] = fxvie1.GetMeasResult(i, MVRET, TRIG_RESULT);
    //Get the position of trigger point for fxvie1
    result[i] = fxvie0.GetMeasResult(i, MVRET, Trig_Point[i]);
    //Read the voltage value of fxvie0 on trigger position
}
```

```
Vcspre->SetTestResult(i, 0, result[i]);
}
```

6.9.44.5 测试传输延时

测试条件:

基于MAX809测试传输延时。设置VCC（fxvie0）从0上升到5V，同步测量VCC（fxvie0）和Reset（fxvie1）的波形，查找Reset端电压Trigger跳变高于2V的跳变点Trig_Point，则（Trig_Point-1）*Interval_Meas即为传输延时时间。



```
int sam = 2; // data size of AWG pattern
int interval = 200; //interval time of AWG pattern, unit is uS
int interval_Meas = 200; // interval time for measurement
double awg_pattern[100] = { 0.0 };
double Trig = 2; //Trigger level
double Trig_Point[4] = { 0 };

awg_pattern[0] = 0;
awg_pattern[sam - 1] = 5;

fxvie1.Set(FI, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON); //Reset FI mode, prepare to measure
delay_ms(1);

fxvie0.AwgLoader("VCC_AWG", FV, FXVie_10V, FXVie_100MA, awg_pattern, sam);
//load the data of "awg_pattern"(sam=2), named "VCC_AWG"
```

```
fxvie0.Set(FV, 0, FXVie_10V, FXVie_100MA, FXVie_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'VCC_AWG' of its
'AwgLoader()'

fxvie0.AwgSelect("VCC_AWG", 0, sam - 1, sam - 1, interval);
//Select pattern of 'VCC_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=200uS

fxvie1.SetMeasVTrig(Trig, TRIG_RISING); //trigger value is 2V, Rising edge

fxvie0.MeasureVI(2000, interval, FXVie_MI_X1, MEAS_AWG);
//fxvie0 AWG Mode, SampleTimes=2000, samplePeriod=200uS.

fxvie1.MeasureVI(2000, interval, FXVie_MI_X1, MEAS_AWG);
//fxvie1 AWG Mode, SampleTimes=2000, samplePeriod=200uS.

STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0, &fxvie1); //enable measurement for fxvie0 and fxvie1
STSAWGRun(); //Enable AWG and measurement synchronously

for (int i = 0; i < SITENUM; i++)
{
    Trig_Point[i] = fxvie1.GetMeasResult(i, MVRET, TRIG_RESULT);
    //Get the position of trigger point for fxvie1

    T_delay->SetTestResult(i, 0, (Trig_Point[i] - 1)*interval_Meas / 1000); //mS
}

fxvie0.Set(FV, 0, FXVie_10V, FXVie_100MA, FXVie_RELAY_ON);
fxvie1.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
fxvie0.Set(FV, 0, FXVie_10V, FXVie_100MA, FXVie_RELAY_OFF);
fxvie1.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_OFF);
```

6.9.44.6 并联大电流测试

测试条件:

每个 bank 的六个通道并联测试，输出电流为 1.2A。由普通参数测试切换到大电流测试，测试完成后再切换到普通参数测试。

```
double Gang_V_Result[STS_SITE_NUM] = { 0.0 };
double Gang_I_Result[STS_SITE_NUM] = { 0.0 };
fxvie0.Set(FV, 5, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
fxvie1.Set(FV, 2, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
delay_ms(1);
fxvie0.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
fxvie1.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
delay_ms(1);
fxvie0.GangRelay(fxvie1, fxvie2, fxvie3, fxvie4, fxvie5);
delay_ms(1);
fxvie0.GangSet(1.2, 0.02);
delay_ms(1);
fxvie0.GangMeasure(100, 10);
for (int i = 0; i < STS_SITE_NUM; i++)
{
```

```

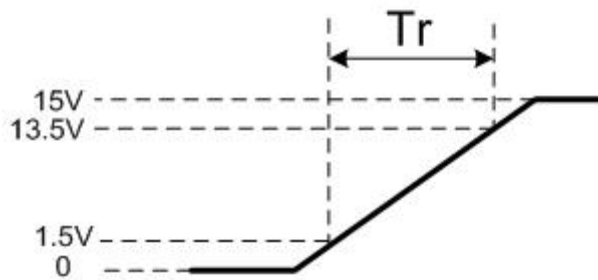
Gang_V_Result[i] = fxvie0.GangGetMeasResult(i, MVRET);
Gang_I_Result[i] = fxvie0.GangGetMeasResult(i, MIRET);
Gang_V->SetTestResult(i, 0, Gang_V_Result[i]);
Gang_I->SetTestResult(i, 0, Gang_I_Result[i]);
}
fxvie0.GangSet(0, 0.02);
delay_ms(1);
fxvie0.GangReset();
fxvie0.Set(FV, 4, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
fxvie1.Set(FV, 1, FXVie_10V, FXVie_1MA, FXVie_RELAY_ON);
delay_ms(1);
fxvie0.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_OFF);
fxvie1.Set(FV, 0, FXVie_10V, FXVie_1MA, FXVie_RELAY_OFF);

```

6.9.45 TMU 编程范例

6.9.45.1 测试时间

例 1：测试下图所示信号的上升时间 T_r 。



测试程序：

```

double val[4] = { 0.0 };
int i = 0;

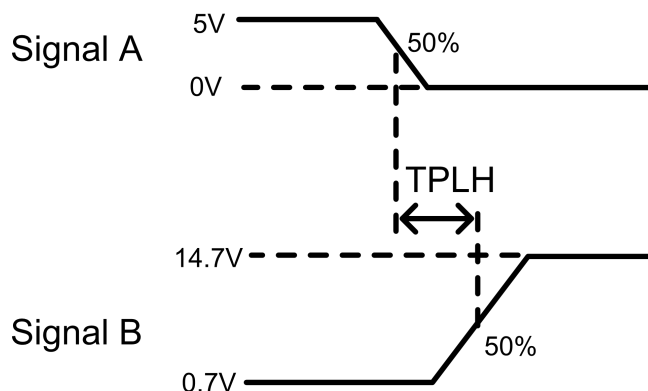
fxvie0.TMUConnect();
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVie_30V, FXVie_TMU_POS, 1.5, FXVie_FILTER_PASS);
fxvie0.TMUStop(FXVie_30V, FXVie_TMU_POS, 13.5, FXVie_FILTER_PASS);
fxvie0.TMUMeasure(FXVie_MEAS_TIME, 1, 3, FXVie_TRANGE_MS);

for (i = 0; i < SITENUM; i++)
{
    val[i] = fxvie0.TMUGetMeasResult(i, AVERAGE_RESULT);
    Tr->SetTestResult(i, 0, val[i]); //uS
}

fxvie0.TMUDisconnect();

```

例 2：测试下图所示信号的时间间隔 TPLH。



测试程序:

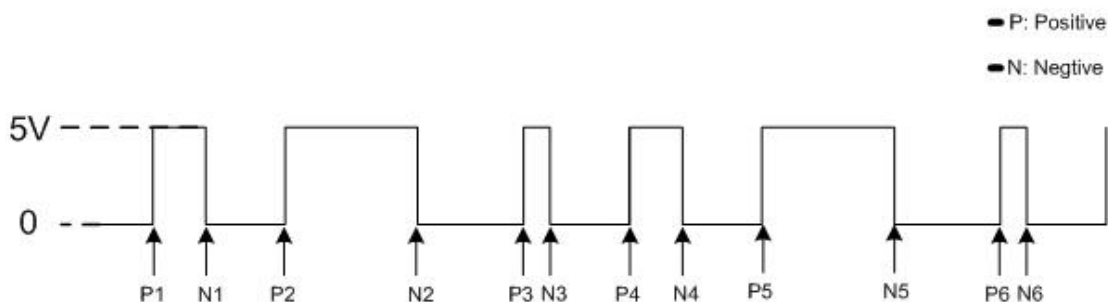
```
double val[4] = { 0.0 };
int i = 0;

fxvie0.TMUConnect();
fxvie1.TMUConnect();
fxvie0.TMUSetMatrix(&fxvie0, &fxvie1); //fxvie0 connect signal A, fxvie1 connect signal B
fxvie0.TMUStart(FXVle_30V, FXVle_TMU_NEG, 2.5, FXVle_FILTER_PASS);
fxvie0.TMUStop(FXVle_30V, FXVle_TMU_POS, 7.7, FXVle_FILTER_PASS);
fxvie0.TMUMeasure(FXVle_MEAS_TIME, 1, 10, FXVle_TRANGE_US);

for (i = 0; i < SITENUM; i++)
{
    val[i] = fxvie0.TMUGetMeasResult(i, AVERAGE_RESULT);
    TPLH->SetTestResult(i, 0, val[i]); //uS
}

fxvie0.TMUDisconnect();
fxvie1.TMUDisconnect();
```

例 3：测试下图所示信号 P2 到 N3 的时间间隔。



测试程序:

```
double val[4] = { 0.0 };
int i = 0;

fxvie0.TMUConnect();
```

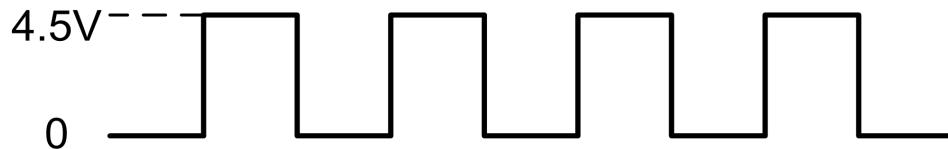
```
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVle_10V, FXVle_TMU_POS, 2.5, FXVle_FILTER_PASS, 10, 0, 1);
fxvie0.TMUStop(FXVle_10V, FXVle_TMU_NEG, 2.4, FXVle_FILTER_PASS, 10, 0, 2);
fxvie0.TMUMeasure(FXVle_MEAS_TIME, 1, 10, FXVle_TRANGE_US);

for (i = 0; i < SITENUM; i++)
{
    val[i] = fxvie0.TMUGetMeasResult(i, AVERAGE_RESULT);
    TP2N3->SetTestResult(i, 0, val[i]); //uS
}

fxvie0.TMUDisconnect();
```

6.9.45.2 测试频率

例 1：测试下图所示信号的频率。



测试程序：

```
double val[4] = { 0.0 };
int i = 0;

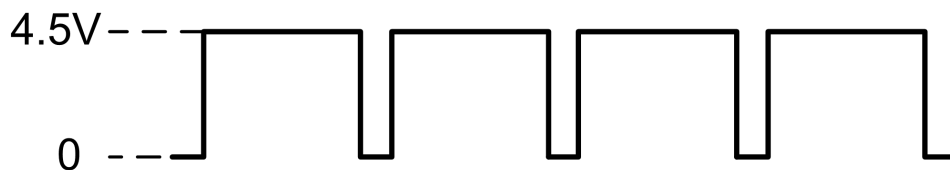
fxvie0.TMUConnect();
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVle_10V, FXVle_TMU_POS, 2.0, FXVle_FILTER_PASS, 10);
fxvie0.TMUMeasure(FXVle_MEAS_FREQ, 1, 10, FXVle_TRANGE_US);

for (i = 0; i < SITENUM; i++)
{
    val[i] = fxvie0.TMUGetMeasResult(i, AVERAGE_RESULT);
    Freq->SetTestResult(i, 0, val[i]); //KHz
}

fxvie0.TMUDisconnect();
```

6.9.45.3 占空比

例 1：测试下图所示信号的高占空比。



测试程序:

```
double val[4] = { 0.0 };
int i = 0;

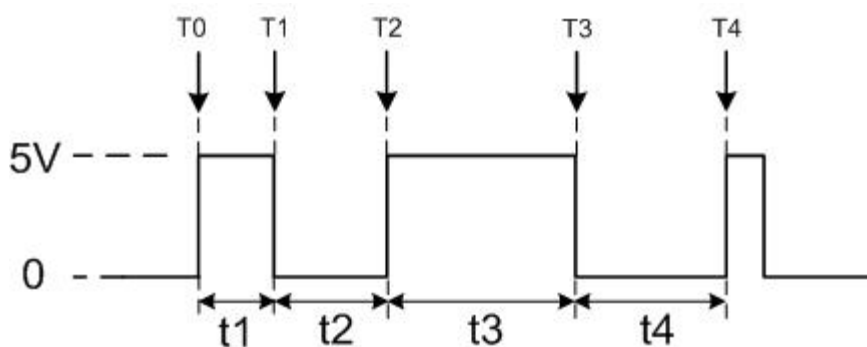
fxvie0.TMUConnect();
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVie_10V, FXVie_TMU_POS, 2.1, FXVie_FILTER_PASS, 10);
fxvie0.TMUStop(FXVie_10V, FXVie_TMU_NEG, 2.0, FXVie_FILTER_PASS, 10);
fxvie0.TMUMeasure(FXVie_MEAS_HIGH_DUTY, 20, 10, FXVie_TRANGE_MS);

for (i = 0; i < SITENUM; i++)
{
    val[i] = fxvie0.TMUGetMeasResult(i, AVERAGE_RESULT);
    Duty->SetTestResult(i, 0, val[i]); //uS
}

fxvie0.TMUDisconnect();
```

6.9.45.4 事件计数

例 1：测试下图所示信号，从第一个有效的上升沿开始捕捉，记录并获取五个跳变时刻（T0、T1、T2、T3、T4）。并计算 t1、t2、t3、t4。



测试程序:

```
double Tevent[5][SITENUM] = { 0.0 };

fxvie0.TMUConnect();
fxvie0.TMUSetInSource();
fxvie0.TMUStart(FXVie_10V, FXVie_TMU_POS, 2.5, FXVie_FILTER_PASS, 10);
fxvie0.TMUStop(FXVie_10V, FXVie_TMU_NEG, 2.4, FXVie_FILTER_PASS, 10);
fxvie0.TMUMeasure(FXVie_MEAS_EVENT, 5, 10, FXVie_TRANGE_MS);
```

```
//Apply excitation to generate signal

for (int SITEID = 0; SITEID < SITENUM; SITEID++)
{
    for (int i = 0; i < 5; i++)
    {
        Tevent[i][SITEID] = fxvie0.TMUGetMeasResult(SITEID, i);
    }
    t1->SetTestResult(SITEID, 0, (Tevent[1][SITEID] - Tevent[0][SITEID]));
    t2->SetTestResult(SITEID, 0, (Tevent[2][SITEID] - Tevent[1][SITEID]));
    t3->SetTestResult(SITEID, 0, (Tevent[3][SITEID] - Tevent[2][SITEID]));
    t4->SetTestResult(SITEID, 0, (Tevent[4][SITEID] - Tevent[3][SITEID]));
}
fxvie0.TMUDisconnect();
```

6.9.45.5 非 TMU 通道的 TMU 测试

例 1：使用 fxvie0（非 TMU 通道）进行 TMU 测试。测量信号由 fxvie0 输入，将 fxvie0 与 fxvie4（TMU 通道）搭接，测量该信号脉宽。

```
FXVle fxvie0("S18_0,S18_6", "fxvie0");
FXVle fxvie4("S18_4,S18_10", "fxvie4");

STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");

double Time_Result[STS_SITE_NUM] = { 0.0 };
fxvie4.TMUChannel(fxvie0);
fxvie4.TMUConnect();
fxvie4.TMUStart(FXVle_10V, FXVle_TMU_POS, 3, FXVle_FILTER_PASS, 100);
fxvie4.TMUStop(FXVle_10V, FXVle_TMU_NEG, 3, FXVle_FILTER_PASS, 100);
fxvie4.TMUMeasure(FXVle_MEAS_TIME, 1, 1.0, FXVle_TRANGE_MS);
for (int i = 0; i < STS_SITE_NUM; i++)
{
    Time_Result[i] = fxvie4.TMUGetMeasResult(i);
    Gang_TMU->SetTestResult(i, 0, Time_Result[i]);
}
fxvie4.TMUDisconnect();
fxvie4.TMUChannelReset();
```

7 数据分析算法库

7.1 STSInitArray()

```
int STSInitArray(  
double*array,  
long arraySize,  
double defaultValue= 0.0);
```

Parameters

array

double 类型数组。

arraySize

数组大小。

defaultValue

初始化默认值，默认值为 0.0。

Remarks

数组的初始化函数，可以将数组的数据统一赋一个指定的值。

Example

```
double data[10] = {0};  
STSInitArray(data, 10, 1.0); // Initialize the array to 1.0
```

7.2 查找触发值

7.2.1 STSFindTrigger()

```
STSFindTrigger();
```

Remarks

通过给定上升沿和下降沿触发阈值，查找上升沿和下降沿的触发值数据及触发值数据在数组中的索引。

7.2.2 Find()

```
long Find(  
const double* dataArray,  
long begin,  
long end,  
double threshold1,  
double threshold2);
```

Parameters

dataArray

数据数组。

begin

数组中起始索引，*begin* 应该大于等于 0，小于 *dataSize*。

end

数组中结束索引，*end* 应该大于 *begin* + 1，小于 *dataSize*。

threshold1, threshold2

上升沿和下降沿触发阈值（触发条件包括该值）。

注意：

threshold1 和 *threshold2* 较大的值作为上限阈值，较小的值作为下限阈值。

Return Values

成功返回 0。否则返回-1。

Remarks

查找触发值。

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

STSTFindTrigger trigger;
trigger.Find(data, 0, 9, 1.75, 0.78); // Find trigger data in data array
```

7.2.3 UpperTriggerCount()

```
long UpperTriggerCount() const;
```

Return Values

返回上升沿触发数据个数。

Remarks

获取上升沿触发数据个数

Example

```
long nTriggerCount = trigger. UpperTriggerCount(); // Get the count of rising edge trigger data
```

7.2.4 UpperTriggerData()

```
long UpperTriggerData(  
int index,  
double& triggerValue,  
long& triggerIndex);
```

Parameters

index

查找上升沿触发数据的索引。其中 $index \geq 0$ 且 $index < \text{UpperTriggerCount}()$ 。

triggerValue

返回上升沿触发数据的值。

triggerIndex

返回上升沿触发数据的在 `dataArray` 中的索引。

Return Values

成功返回 0。否则返回-1。

Remarks

获取上升沿触发数据值及在数组中的索引。

Example

```
double data[10] = {0};  
data[0] = 1.0;  
data[1] = 3.7;
```

```
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

STSTFindTrigger trigger;
trigger.Find(data, 0, 9, 1.75, 0.78);
long nTriggerCount = trigger.UpperTriggerCount();

for (int i = 0; i < nTriggerCount; ++i)
{
    double dTriggerValue = 0.0;
    long nTriggerIndex = -1;
    if (trigger.UpperTriggerData(i, dTriggerValue, nTriggerIndex))
    {
        // Acquiring Rising Edge Triggering Data Successfully
        // Use this trigger to do other things
    }
}
```

7.2.5 LowerTriggerCount()

```
long LowerTriggerCount() const;
```

Return Values

返回下降沿触发数据个数。

Remarks

获取下降沿触发数据个数

Example

```
long nTriggerCount = trigger.LowerTriggerCount(); // Get the count of trigger data along the descent edge
```

7.2.6 LowerTriggerData()

```
long LowerTriggerData(  
int index,  
double&triggerValue,  
long&triggerIndex);
```

Parameters

index

查找下降沿触发数据的索引。值范围：其中 $index \geq 0$ 且 $index < \text{LowerTriggerCount}()$ 。

triggerValue

返回下降沿触发数据的值。

triggerIndex

返回下降沿触发数据的在 `dataArray` 中的索引。

Return Values

成功返回 0。否则返回-1。

Remarks

获取下降沿触发数据值及在数组中的索引。

7.3 数组四则运算

7.3.1 STSArrayAdd()

```
long STSArrayAdd(  
const double*array1,  
const double*array2,  
double*resultArray,  
longarraySize,
```



```
long begin = 0,  
long end = -1);
```

Parameters

array1,array2

计算的数组

resultArray

计算结果输出数组

arraySize

数组大小

注意：

array1，*array2* 和 *resultArray* 的大小不小于 *arraySize*。

begin

起始位置，*begin* 应该大于等于 0，小于 *arraySize*。默认是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *arraySize*。默认是-1。

注意：

当 *end* = -1 时计算从 *begin* 到 *arraySize*-1 之间的数据对应的加法。

Return Values

计算成功返回 0

Remarks

数组的加法操作，计算 *begin* 到 *end* 间，数组 *array1* 与数组 *array2* 加法，*array1* 中不属于 *begin* 到 *end* 间的数据直接复制到输出结果数组中。

Example

```
const int DATASIZE = 20;
double data1[DATASIZE]={ 0.0 };
data1[0]=0.7;
data1[1]=1.7;
data1[2]=4.9;
data1[3]=2.5;
data1[4]=8.9;
data1[5]=3.7;
data1[6]=1.2;
data1[7]=5.2;
data1[8]=1.3;
data1[9]=1.1;
data1[10]=0.2;
data1[11]=7.2;
data1[12]=3.2;
data1[13]=5.2;
data1[14]=2.18;
data1[15]=1.02;
data1[16]=7.25;
data1[17]=4.82;
data1[18]=31.2;
data1[19]=1.333;

double data2[DATASIZE]={ 0.0 };
data2[0]=0.7;
data2[1]=1.7;
data2[2]=4.9;
data2[3]=2.5;
data2[4]=8.9;
data2[5]=3.7;
data2[6]=1.2;
data2[7]=5.2;
data2[8]=1.3;
data2[9]=1.1;
data2[10]=0.2;
data2[11]=7.2;
data2[12]=3.2;
data2[13]=5.2;
data2[14]=2.18;
data2[15]=1.02;
data2[16]=7.25;
data2[17]=4.82;
data2[18]=31.2;
data2[19]=1.333;

double dataAdd[DATASIZE]={0};
double dataSub[DATASIZE]={0};
double dataMULT[DATASIZE]={0};
double dataDiv[DATASIZE]={0};
STSTArrayAdd(data1, data2, dataAdd, DATASIZE); // Array addition operation
STSTArraySub(data1, data2, dataSub, DATASIZE); //Array subtraction operation
STSTArrayMULT(data1, data2, dataMULT, DATASIZE); // Array multiplication operation
STSTArrayDiv(data1, data2, dataDiv, DATASIZE); // Array division operation
```

7.3.2 STSArraySub()

```
long STSArraySub(  
const double*array1,  
const double*array2,  
double*resultArray,  
longarraySize,  
longbegin = 0,  
longend = -1);
```

Parameters

array1, array2

计算的数组

resultArray

计算结果输出数组

arraySize

数组大小

注意：

array1, array2 和 *resultArray* 的大小不小于 *arraySize*。

begin

起始位置，*begin* 应该大于等于 0，小于 *arraySize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *arraySize*。默认值是-1。

注意：

当 *end* = -1 时计算从 *begin* 到 *arraySize*-1 之间的数据对应的减法。

Return Values

计算成功返回 0

Remarks

数组的减法操作，计算 *begin* 到 *end* 间，数组 *array1* 与数组 *array2* 减法，*array1* 中不属于 *begin* 到 *end* 间的数据直接复制到输出结果数组中。

Example

参考 STSArrayAdd()函数的 Example。

7.3.3 STSArrayMULT()

```
long STSArrayMULT(  
const double*array1,  
const double*array2,  
double*resultArray,  
longarraySize,  
longbegin = 0,  
longend = -1);
```

Parameters

array1, *array2*

计算的数组

resultArray

计算结果输出数组

arraySize

数组大小

注意：

array1, *array2* 和 *resultArray* 的大小不小于 *arraySize*。

begin

起始位置，*begin* 应该大于等于 0，小于 *arraySize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *arraySize*。默认值是-1。

注意：

当 *end* = -1 时计算从 *begin* 到 *arraySize*-1 之间的数据对应的乘法。

Return Values

计算成功返回 0

Remarks

数组的乘法操作，计算 *begin* 到 *end* 间，数组 *array1* 与数组 *array2* 的乘法，*array1* 中不属于 *begin* 到 *end* 间的数据直接复制到输出结果数组中。

Example

参考 STSArrayAdd() 的 Example。

7.3.4 STSArrayDiv()

```
long STSArrayDiv(  
const double*array1,  
const double*array2,  
double*resultArray,
```

```
long arraySize,  
long begin = 0,  
long end = -1);
```

Parameters

array1, *array2*

计算的数组

resultArray

计算结果输出数组

arraySize

数组大小

注意：

array1, *array2* 和 *resultArray* 的大小不小于 *arraySize*。

begin

起始位置，*begin* 应该大于等于 0，小于 *arraySize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *arraySize*。默认值是-1。

注意：

当 *end* = -1 时计算从 *begin* 到 *arraySize*-1 之间的数据对应的除法。

Return Values

计算成功返回 0。

Remarks

数组的除法操作，计算 *begin* 到 *end* 之间，数组 *array1* 与数组 *array2* 的除法，*array1*

中不属于 *begin* 到 *end* 间的数据直接复制到输出结果数组中。

注意：

如果除数数组中含有 0，则除法操作可能回出现异常，将退出计算并返回-1。在 Alarm 中会有错误提示信息。

Example

参考 STSArrayAdd()的 Example。

7.4 STSAverage()

```
double STSAverage(  
const double* dataArray,  
long begin,  
long end);
```

Parameters

dataArray

求平均值的数组数据。

begin

起始位置，*begin* 应该大于等于 0。

end

结束位置，*end* 应该大于等于 *begin*，小于数组的大小。

Return Values

返回数组数据的平均值

Remarks

求数组数组的平均值

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

double dAverage = STSAverage(data, 1, 7); // Find the average of data from 1 to 7 (including 7)
```

7.5 STSRootMeanSquare ()

```
double STSRootMeanSquare (
const double* dataArray,
long begin,
long end);
```

Parameters

dataArray

数据数组。

begin

起始位置，*begin* 应该大于等于 0。

end

结束位置，*end* 应该大于等于 *begin*，小于数组的大小。

Return Values

返回数组的均方根

Remarks

求数组的均方根。

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

// Find the root mean square of data from 1 to 7 (including 7)
double dRootMeanSquare = STSRootMeanSquare(data, 1, 7);
```

7.6 STSMovAvg()

```
int STSMovAvg(
const double* dataArray,
double* resultDataArray,
long dataSize,
long breadth,
long begin = 0,
long end = -1);
```

Parameters

dataArray

数据数组。

resultDataArray

输出数据移动平均处理后的缓存地址要求 `resultDataArray` 数组长度不小于 `dataArray` 数组长度。

dataSize

数据数组大小。

breadth

计算平滑的处理宽度。

begin

起始位置，*begin* 应该大于等于 0，小于 *dataSize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *dataSize*。默认值是-1。

Return Values

计算成功返回 0

Remarks

求数组的移动平均数据，计算 *begin* 到 *end* 间的移动平均数据，不属于 *begin* 到 *end* 间的数据直接复制到输出结果数组中。

if (*begin* ≤ *i* && *i* ≤ *end*)

$$resultDataArray[i] = \sum_{j=i-breadth}^{i+breadth} dataArray[j] / 2$$

else

resultDataArray[*i*] = *dataArray*[*i*]

Example

```
double data[10] = {0};  
data[0] = 1.0;  
data[1] = 3.7;  
data[2] = 1.5;  
data[3] = 3.1;
```

```
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

double dataMovavg[10]={ 0 };

// // Get the smooth data from 1 to 7 in the data data array and assign the results to the data Smooth
STSMovAvg (data, dataMovavg, 10, 5, 1, 7);
for (int i=1; i <= 7; ++i)
{
    dataMovavg[i]; //Smoothing data
}
```

7.7 STSVariance()

```
double STSVariance(
const double*dataArray,
long begin,
long end);
```

Parameters

dataArray

数据数组。

begin

起始位置，*begin* 应该大于等于 0。

end

结束位置，*end* 应该大于等于 *begin*，小于数组的大小。

Return Values

返回数组的方差。

Remarks

求数组的方差。

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

double dVariance = STSVariance(data, 1, 7); // Calculate the variance of data from 1 to 7 (including 7)
```

7.8 STSStandardDeviation()

```
double STSStandardDeviation(
const double* dataArray,
long begin,
long end);
```

Parameters

dataArray

数据数组。

begin

起始位置，*begin* 应该大于等于 0。

end

结束位置，*end* 应该大于等于 *begin*，小于数组的大小。

Return Values

返回数组的标准差。

Remarks

求数组的标准差。

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

// Calculate the standard deviation of data 1 to 7 (including 7)
double dStandardD = STSStandardDeviation(data, 1, 7);
```

7.9 STSFindMaxValueAndPos()

```
void STSFindMaxValueAndPos (
const double*dataArray,
long dataSize,
long&pos,
double&maxValue,
long begin = 0,
long end = -1);
```

Parameters

dataArray

数据数组。

dataSize

数据数组大小。

pos

返回最大数的在数组中的索引。

maxValue

返回最大数的值。

begin

起始位置，*begin* 应该大于等于 0，小于 *dataSize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *dataSize*。默认值是-1。

Remarks

查找数据数组的最大值及位置。

Example

```
double data[10] = {0};
data[0] = 1.0;
data[1] = 3.7;
data[2] = 1.5;
data[3] = 3.1;
data[4] = 7.9;
data[5] = 0.71;
data[6] = 7.1;
data[7] = 8.1;
data[8] = 1.8;
data[9] = 5.71;

long nPos=-1;
double dMaxValue=0.0;
STSTFindMaxValueAndPos(data, 10, nPos, dMaxValue, 1, 7); // Find the most data and location from 1 to 7
if (-1 != nPos)
{
    // TODO:
}
```

7.10 STSFindMinValueAndPos()

```
void STSFindMinValueAndPos(  
    const double* dataArray,  
    long dataSize,  
    long& pos,  
    double& minValue,  
    long begin = 0,  
    long end = -1);
```

Parameters

dataArray

数据数组。

dataSize

数据数组大小。

pos

返回最小数的在数组中的索引。

minValue

返回最小数的值。

begin

起始位置，*begin* 应该大于等于 0，小于 *dataSize*。默认值是 0。

end

结束位置，*end* 应该大于等于 *begin*，小于 *dataSize*。默认值是-1。

Remarks

查找数据数组的最小值及位置。

7.11 STSGetFFTData()

```
int STSGetFFTData(  
const double* sampleData,  
double* fftResult,  
long sampleCount);
```

Parameters

sampleData

采样数据数组。

fftResult

傅立叶变换结果输出数组，要求 *fftResult* 数组长度不小于 *sampleData* 数组长度。

sampleCount

采样数据个数 *sampleCount* 必须是 2 的 n 次方。

Return Values

计算成功返回 0

Remarks

快速傅立叶变换。

Example

```
const int DATASIZE=20;
```



```
double data1[DATASIZE] = { 0.0 };
data1[0] = 0.7;
data1[1] = 1.7;
data1[2] = 4.9;
data1[3] = 2.5;
data1[4] = 8.9;
data1[5] = 3.7;
data1[6] = 1.2;
data1[7] = 5.2;
data1[8] = 1.3;
data1[9] = 1.1;
data1[10] = 0.2;
data1[11] = 7.2;
data1[12] = 3.2;
data1[13] = 5.2;
data1[14] = 2.18;
data1[15] = 1.02;
data1[16] = 7.25;
data1[17] = 4.82;
data1[18] = 31.2;
data1[19] = 1.333;

double data2[DATASIZE] = { 0.0 };
STSGetFFTDData(data1, data2, 16); // Note that the number of samples must be the n th power of 2
for (int i = 0; i < 16; ++i)
{
    data2[i]; // Fourier Transform Result Value
}
```

7.12 STSLeastSquareLinearFitting()

```
int STSLeastSquareLinearFitting(
const double*xArray,
const double*yArray,
long arraySize,
double&rakeRatio,
double&intercept);
```

Parameters

xArray

x 轴数据数组。

yArray

y 轴数据数组。

arraySize

x 轴数据数组和 y 轴数据数组大小。

rakeRatio

返回计算出的斜率。

intercept

返回计算出的截距。

Return Values

计算成功返回 0。

Remarks

线性拟合，求出斜率和截距。

Example

```
const int DATASIZE=20;
double data0[DATASIZE] = { 0.0 };
double data1[DATASIZE] = { 0.0 };

// k = 5, b = 7
int i = 0;
for (i = 0; i < DATASIZE; ++i)
{
    data0[i] = i;
    data1[i] = i * 5 + 7;
}

double k=0;
double b=0;
STLeastSquareLinearFitting(data0, data1, 20, k, b);

assert(k == 5.0);
assert(b == 7.0);
```

7.13 STSSort()

```
void STSSort(  
const double* dataArray,  
long begin,  
long end,  
bool ascending);
```

Parameters

dataArray

数据数组。

begin

起始位置，*begin* 应该大于等于 0。

end

结束位置，*end* 应该大于等于 *begin*，小于数组的大小。

ascending

可选择值如下：

true: 是升序排序

false: 是降序排序

Remarks

对数组左闭右开区间[*begin*, *end*)的数进行排序。

Example

```
const int DATASIZE=20;  
double data1[DATASIZE] = { 0.0 };  
data1[0] = 0.7;  
data1[1] = 1.7;  
data1[2]=4.9;  
data1[3]=2.5;  
data1[4]=8.9;  
data1[5]=3.7;  
data1[6]= 1.2;  
data1[7]=5.2;  
data1[8]=1.3;
```

```
data1[9]=1.1;  
data1[10]= 0.2;  
data1[11]=7.2;  
data1[12]= 3.2;  
data1[13]= 5.2;  
data1[14]= 2.18;  
data1[15]= 1.02;  
data1[16]=7.25;  
data1[17]=4.82;  
data1[18]=31.2;  
data1[19]=1.333;  
STSSort(data1, 0, DATASIZE, true);
```